

Владимир
Завертайлов

основатель и руководитель
scrum-студии «Сибирикс»

настольная книга *project-* менеджера



что нужно знать,
чтобы управлять **IT**,
digital и другими
проектами с учетом
российских реалий



pro

Владимир
Завертайлов

основатель и руководитель
scrum-студии «Сибирикс»

настольная книга *project-* менеджера



что нужно знать,
чтобы управлять **IT**,
digital и другими
проектами с учетом
российских реалий

◀ *proj*

 **БОМБОРА**
ИЗДАТЕЛЬСТВО

Москва

Друзья! Всем, кто приобрел эту книгу, мы дарим полгода бесплатной работы в лучшем персональном планировщике задач SingularityApp. Вы экономите 1000 рублей. Чтобы узнать, как получить подарок — переходите по ссылке <https://singularity-app.com/pmbook/> или QR-коду



СОДЕРЖАНИЕ

Введение	9
----------------	---

ЧАСТЬ 1. КАРТИНА МИРА DIGITAL-ПРОЕКТА. ТОНКОСТИ ДЕЛЕГИРОВАНИЯ В IT

1.1 Картина мира digital-проекта	15
1.2 Делегирование	27
1.3 Уровни делегирования	37
1.4 SMART-делегирование	39
1.5 Постановка задач на рисерч	41
1.6 Фальшивые автобусные остановки. Тонкое искусство контрольных точек	43
1.7 Ошибки делегирования	45
1.8 Правила письменной контрацепции. Как ставить задачи, не убив друг друга	48
1.9. Семь простых истин из авиации о делегировании и контроле ..	57
1.10 Для тренировки	61
Литература	61
Пояснения	62

ЧАСТЬ 2. ТРЕБОВАТЕЛЬНОСТЬ: КАК РАЗВИВАТЬ ЕЕ У СЕБЯ И СВОИХ СОТРУДНИКОВ

2.1 Сколько программистов нужно, чтобы выкопать яму?	64
2.2 Требовательность	66
2.3 Механизм власти	68
2.4 Нормальная эксплуатация и мозгоклюйство	81
2.5 Когнитивные искажения у дизайнеров и программистов	86
2.6 Для тренировки	94
Литература	95
Пояснения	95

ЧАСТЬ 3. ПЯТИМИНУТНЫЙ SCRUM

3.1 Схема Scrum.Arteфакты. Роли. Процедуры	98
3.2 Внедряем!	104
3.3 Подготовка и планирование спринта	105

Содержание	3
------------------	---

3.4	Planning Poker	108
3.5	Стендапы. Burn Down Chart	109
3.6	Стратегии тестирования	111
3.7	Демонстрация продукта. Завершение спринта	113
3.8	Ретроспективы. Бородачи тоже плачут	115
3.9	Канбан. Когда лучше, чем Scrum	123
3.10	Куда дели менеджера	123
3.11	Аналитика, дизайн, разработка — параллельно	125
3.12	Документация	126
3.13	Метод красной рамки	128
3.14	Потому что Карго-культ!	129
3.15	Для тренировки	131
	Литература	132
	Пояснения	132
ЧАСТЬ 4. АНАЛИТИКА И ПРОДУКТОВЫЕ ТЕХНИКИ		134
4.1	Цель аналитики digital-проектов	135
4.2	Агрегация требований в заказной разработке	138
4.3	Продуктовые техники	184
4.4	Юнит-экономика без перелома головы	199
4.5	Техники скоринга и приоритизации бэклогов	208
4.6	UML. Самые нужные диаграммы	236
4.7	Собираем все вместе	246
4.8	Для тренировки	248
	Литература	248
	Пояснения	250
ЧАСТЬ 5. ДЕКОМПОЗИЦИЯ, ОЦЕНКА, СМЕТЫ, СЕТЕВЫЕ ГРАФИКИ, ИЛИ КАК ДАВАТЬ ПРОГНОЗЫ		252
5.1	Как дать адекватную оценку времени, когда неопределенность бьет по башке	252
5.2	Три уровня декомпозиции	268
5.3	Декомпозиция декомпозиции, или откуда эта хрень на странице?	271
5.4	Подготовка вилочных и точных смет	277
5.5	Экспресс-версия диаграммы Ганта	291
5.6	Сетевые графики PERT	295

5.7 Обзор программного обеспечения для сетевых графиков и диаграмм Ганта	298
5.8 Что делать, если проект нетиповой	303
5.9 Метод освоенного объема	306
5.10 Два типа людей. Итоги	311
5.11 Для тренировки	313
Литература	313
Пояснения	313

ЧАСТЬ 6. ДИЗАЙН. УПРАВЛЯЕМЫЙ КРЕАТИВ 315

6.1 Артефакты дизайна. Краткий справочник по применению	316
6.2 О креативности	320
6.3 Будничная сакральность	320
6.4 Качество дизайна	322
6.5 Управляемость и креативность. Самодвижение. Устойчивость по предсказуемости	328
6.6 Бегунок креативности	333
6.7 Этапы линейного дизайн-процесса	339
6.8 Где сидит дизайнер, или Модели организации дизайн-команд	385
6.9 Стратегии параллельной работы	390
6.10 Agile + UX. Дизайн-спринты	403
6.11 Строим дизайн-команду	419
6.12 Data Driven Design: Как использовать данные в дизайне и можно ли переложить ответственность за красоту в макетах на цифры	423
6.13 Лайфхаки, вопросы и ответы	434
6.14 Для тренировки	448
Литература	448
Пояснения	449

ЧАСТЬ 7. СТРОИМ И РАЗВИВАЕМ КОМАНДУ 452

7.1 Анатомия командообразования	452
7.2 Три способа постановки целей	477
7.3 Добавить ясностей, по Сунь-цзы	479
7.4 А было бы нефигово!	480
7.5 Ритуалы, предметы, удаленка	481
7.6 KPI, или пособие по командному самоубийству	482

7.7 Object Key Result (OKR) для синхронизации целей всех уровней	490
7.8 Agile и целеполагание. Фокусируем команды на цели	496
7.9 Карты компетенций. Аттестации и чеки. Ясный карьерный трек сотрудника	498
7.10 360	500
7.11 Один на один	501
7.12 Разбор полетов	502
7.13 Обратная связь	502
7.14 Стратегические сессии	515
7.15 Профилактика эмоциональных выгораний	517
7.16 Геймификация. Тюнинг. Корпоративная культура	541
7.17 Домашнее задание	554
Литература	555
Пояснения	556

ЧАСТЬ 8. ВРЕМЯ И ЭНЕРГИЯ РУКОВОДИТЕЛЯ. ЭТЮДЫ 557

Этюд #1. Микропереключения контекстов	560
Этюд #2. Пушинки	560
Этюд #3. Период полураспада информации	561
Этюд #4. Грыжа	562
Этюд #5. Причины или Следствия?	562
Этюд #6. Лучший менеджер свободного острова	564
Этюд #7. Про ПМ, У Которого Много Проектов	565
Этюд #8. Уведомления, чаты, почты... ..	566
Этюд #9. Перфекционизм, сроки, планы	566
Этюд #10. Тендерное	567
Этюд #11. Супер-базовый навык	567
Этюд #12. Сковорода	567
Этюд #13 Золотые слова руководителя	567
Этюд #14 Динамично развивающееся digital-агентство	568
Этюд #15 Не продолбать момент!	568
Этюд #16 Социалочки	569
Этюд #17 Хаос-Порядок-Хаос. Синусоида	570
Этюд #18 Скорость притирки. Переводи непереводи	570
Этюд #19 Типовые задачи	571
Этюд #20 Невозможные задачи	572

Этюд #21 Заваленные сигналы или делегирование задач самим себе	573
Заключение. RIGOR	573
Литература	574

ЧАСТЬ 9. СТРАТЕГИИ ЗАКАЗНОЙ И ПРОДУКТОВОЙ РАЗРАБОТКИ 575

9.1 Заказная или продуктовая команды?	576
9.2 Digital как услуга	582
9.3 Организация работы технической поддержки в заказной разработке	598
9.4 Тактика организации in-house-команды	617
Литература	619
Пояснения	620

ЧАСТЬ 10. ИНТЕГРАЦИЯ. РИСКИ И НЕМНОГО РЕАЛЬНОСТИ 621

10.1 Почему не хватает штатных интеграций?	623
10.2 Человеческий фактор	623
10.3 Рабочая группа	631
10.4 Организуйте итерации обсуждений	632
10.5 Формат данных	632
10.6 Интеграционный протокол	639
Пояснения	648

ЧАСТЬ 11. РАБОТА С ФАКАПАМИ 649

11.1 Ретроспективы	650
11.2 Восемь сортов муды в digital-компании	652
11.3 Хвостовые риски в digital-проектах	660
11.4 Алгоритм разбора проблем	666
11.5 Риск-менеджмент. Лженаука?	668
11.6 Заключение	674
11.7 Домашнее задание	674
Литература	674
Пояснения	675

ЧАСТЬ 12. ОСНОВЫ ТЕХНИЧЕСКОЙ ГРАМОТНОСТИ 676

12.1 Язык отрасли	676
12.2 Как работает интернет	678

12.3	Языки программирования и фреймворки	683
12.4	Хостинг. Серверное хозяйство	688
12.5	Скорость загрузки	705
12.6	Системы контроля версий	711
12.7	Адаптивные и мобильные сайты	714
12.8	Мобильная разработка	719
12.9	Интеграция со всякой всячиной	722
12.10	Заключение	724
12.11	Домашние задания	725
	Пояснения	725
	Заключение	728
	Благодарности	729
	Об авторе	730
	 ПРИЛОЖЕНИЕ. КАРТА КОМПЕТЕНЦИЙ РУКОВОДИТЕЛЯ DIGITAL-ПРОЕКТОВ	 731

ВВЕДЕНИЕ

Что нужно, чтобы стать руководителем digital-проектов или руководителем продукта в IT? Уж точно не классическое вузовское образование (хотя без него будет трудно). На пути к должности руководителя вы потратите огромное количество времени на практику, не раз набьете шишки и столкнетесь с факапами, прольете немало пота, крови и слез, вложите в это много труда. Но самое главное — приготовьтесь каждый день ставить свою шкуру на кон. Потому что руководитель проектов и продуктов отвечает за свое детище, как никто в команде. И делает все то, что нельзя делегировать.

Вас к такому не готовили

Почти 20 лет я руковожу студией разработки и не один год провожу стажировки для студентов. Статистика печальна: востребованных специалистов для управления проектами и продуктами в вузах, увы, не готовят. Поэтому мне приходится отсматривать десятки претендентов на стажировку и отсеивать тех, кто хочет халявы в роли ничего не делающего надзирателя или легких денег (проджекты же много зарабатывают, правда?). Но те, кого я беру на стажировку — счастливики. Потому что они моментально погружаются в рабочую атмосферу. Сразу вникают в процессы, самостоятельно делают аналитику, продумывают структуру проектов (пусть и не существующих в реальности), ведут их от начала и до конца, наступают на грабли, совершают ошибки. Сразу — действительно учатся. И даже если в дальнейшем наши пути расходятся, ребята уходят счастливыми и окрыленными.

Но что делать, если у вас за плечами нет такой стажировки, а вы хотите быть руководителем продукта или проекта? Или, возможно, уже им стали, но что-то постоянно не клеится: сроки срываются, команда косячит, и вам опять надо это разгрести в стопятьсотый раз? Знакомо? Знакомо. Нет, можно, конечно, уволиться и начать все сначала. Но собственные системные ошибки догонят где угодно — потому что от себя не убежишь.

На должность руководителей проектов я собеседовал множество раз, и это были самые разные люди. Одни работали в каких-нибудь госзакупках, другие — в прошлом были инженерами, но доросли до руководящих позиций, третьи — менеджерили команду в банке или вообще были маркетологами. Меня мало интересует опыт из резюме — он вряд ли расскажет о том, способен ли человек решать проблемы здесь и сейчас или нет. И уж точно не покажет, насколько крутой из него переговорщик и справится ли он с интеграцией на проекте. Меня больше интересуют навыки — те самые Soft Skills, о которых мы будем говорить в этой книге. И которых часто не хватает многим, и не только в IT-среде.

Кто виноват и что делать

Хорошая новость: вы не виноваты в том, что на старте в профессию не обладали огромным багажом знаний и что даже многолетний опыт не спас вас от ошибок. Так уж повелось, что в классических учебных заведениях необходимую информацию почти не дают, а на предыдущих местах работы редко удается прокачать все нужные скиллы.

Но есть и плохая новость: вы в ответе за то, что все так, как есть. Сейчас вы должны решить, что делать дальше: учиться или продолжать слепо ходить по старым дорожкам в надежде получить другой результат.

Если выбираете второй путь — ставьте книгу обратно на полку, диалога у нас не получится. Если все-таки первый — то придется потрудиться. Но оно того стоит.

В digital все крутится вокруг всего двух компетенций: технологии и коммуникации. Hard Skills & Soft Skills. Если проседают коммуникации, руководитель сможет завалить проект, сделанный сильной технической командой. И наоборот: позитивный и энергичный менеджер, который умеет договариваться — вытянет слабый проект и сделает так, что заказчик попросит еще. Технологии же нужны, чтобы общаться с командой и заказчиками на уровне эксперта.

Что вам точно понадобится?

- ▶ Честность. На уровне «пацан — сказал, пацан — сделал». Без этого с человеком, по-моему, вообще нельзя иметь дел.
- ▶ Интеллект. В первую очередь — системное мышление.
- ▶ Зрелость. Готовность брать на себя ответственность.
- ▶ Позитив и энтузиазм. Если менеджер по натуре депрессивен, и у него на лице написано: «Господи, когда же это все дерьмо закончится» — не стоит ожидать, что команда радостно подхватит знамя и с криками «Эге-гей! Дерьмо! Кончайся!» побежит делать проект.
- ▶ Внимание к мелочам. Большие проблемы вырастают из мелочей, которые проглядели на старте работ. Или заметили, но решили, что они никак не повлияют на будущее. Мелочи ничего не прощают большому.
- ▶ Готовность незамедлительно действовать.
- ▶ Требовательность. Умение пойти на конфликт, если что-то сделано плохо, отстаивать свои интересы и интересы дела.

Вообще, путь менеджера — это трудный путь. В нем много сложностей и много работы над собой, характером, привычками. А это больно. Приходится нести ответственность не только за себя, но и за других, и за весь проект. Отвечать за чужие косяки. Двойные неприятности.

Зато это очень интересный путь, интересная судьба. Практически бесконечные возможности как для горизонтального роста, так и для вертикального, если он вдруг вам понадобится. Хорошо платят. Вы приобретаете чертовски полезные навыки для повседневной жизни. И это определенная степень свободы, умение владеть собой. И еще — вы будете востребованы, и скучно точно не будет. Но это — если оно вам действительно надо. А раз уж вы держите в руках эту книгу, хочется надеяться, что это так.

Что вас ждет в этой книге

В 2017-м году мы в студии создали обучающий курс на Skillbox «Руководитель digital-проектов», который был одним из первых на рынке по этой теме. За 4 года существования его прошли сотни студентов.

И каждый из них хотел бы иметь под рукой инструмент, на который всегда можно положиться в сложной и неоднозначной ситуации.

Цель — дать этот инструмент тем, кто только собирается в профессию руководителя проекта или руководителя продукта, и тем, кто уже в ней работает и развивается, но испытывает сложности. Можно сказать, это настольное практическое руководство для проджектов и продактов, которым приходится либо заказывать digital-услуги на стороне, либо управлять своим собственным digital-производством.

Почему книга сразу и для проджектов, и для продактов? Потому что в небольших проектах эти роли часто выполняет один человек. Project Manager (диджитал-продюсер, РМ, менеджер, руководитель проектов — зовите, как хотите) — специалист, который своей шкурой отвечает за проект и несет ответственность за то, чтобы в процессе работы ни одно животное не пострадало. Графики, сроки, команда, бюджеты, релизы, качество — вот что в фокусе. Product Manager — это, скорее, про маркетинг, стратегию, приоритеты, трафик, сегменты, конкурентов.

Но работы полно и там, и там. Более того, хороший руководитель проекта должен владеть продуктовыми техниками, чтобы лучше понимать потребности бизнеса. А хороший руководитель продукта должен вырасти из руководителя проектов, чтобы понимать возможности и ограничения своей команды и иметь адекватную картину мира. Именно поэтому в этой книге есть техники для обеих специальностей.

В ней также найдутся ответы на те непростые вопросы, с которыми сталкиваются начинающие и уже опытные менеджеры проектов и продуктов ежедневно:

- ▶ как общаться с подчиненными, чтобы не быть в их глазах мямлей или деспотом;
- ▶ как грамотно делегировать и что придется в себе искоренять, чтобы это уметь;
- ▶ почему Scrum — все еще передовой фреймворк для разработки проектов и продуктов в digital и как его внедрить в свою команду;
- ▶ как и с помощью каких инструментов проводить аналитику перед стартом проекта, чтобы в процессе разработки не случилось неприятных сюрпризов;
- ▶ что такое декомпозиция, зачем она нужна на проекте и как грамотно составлять сметы, чтобы потом не было мучительно больно и стыдно перед заказчиком;

- ▶ как управлять творческими сотрудниками и креативным процессом без магии, шантажа и подкупа;
- ▶ как строить команду, чтобы людям хотелось приходить на работу и расти профессионально;
- ▶ как распределять собственное время менеджера эффективно и при этом не выгореть и не уехать в бессрочный отпуск;
- ▶ что лучше: собственная команда или аутсорс и как быть с тех-поддержкой;
- ▶ как не плодить бюрократию, но при этом четко оформить процесс разработки документально;
- ▶ как интегрировать сайты и продукты с внешними сервисами и системами без моря слез;
- ▶ что делать, если все пошло не по плану и случился факап на проекте;
- ▶ и как, черт побери, понять все то, что ежедневно говорит тебе команда на своем айтишном языке.

Полистайте содержание. Посмотрите главы. Загляните в задания после каждого блока. Если есть сомнения — поставьте, где взяли. Не поможет. Особенно, если не планируете постоянно практиковаться. В книге, конечно, есть забавные истории, примеры, кейсы, чек-листы и шаблоны, которые можно брать и сразу применять. Но это — инструмент, а им нужно пользоваться!

ЧАСТЬ 1



КАРТИНА МИРА DIGITAL-ПРОЕКТА. ТОНКОСТИ ДЕЛЕГИРОВАНИЯ В IT

Возьмем такую задачку: **довести сайт до ума.**

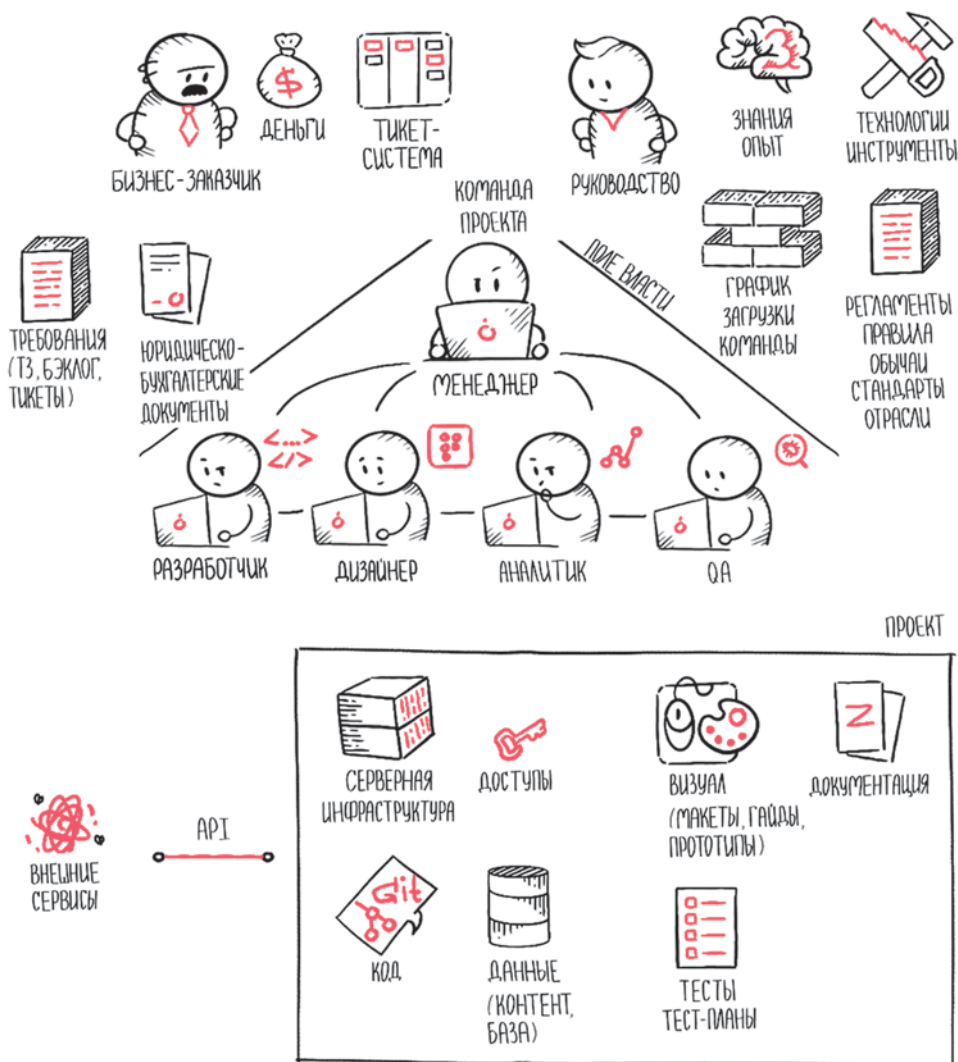
Нет, я не пошутил с формулировкой. Таких задач — доделать за предыдущими разработчиками — на фриланс-биржах пруд пруди. И несчастные сотрудники в них периодически вляпываются. Допустим, вы — менеджер, у вас микрокоманда: дизайнер, программист, аналитик, тестировщик. С которыми вы раньше не работали. И есть еще бизнес-заказчик, не шибко разбирающийся в digital-премудростях, но уже почему-то раздраженный. Он эту карусель оплачивает, если его устроит цена и качество. Понятно, что его в первую очередь интересуют деньги и сроки. Ваши действия?

Правильно: бежать!

А что нас смущает? Запредельная степень неопределенности? Тухлая перспектива? Неуверенность в команде? Непонятки с оплатой? Неадекватный заказчик? Поехали, потом заведешься? Гарантированный геморрой при негарантированном результате?

Давайте мысленно представим, как выглядит картина мира digital-проекта и сколько возни тут вырисовывается.

1.1 КАРТИНА МИРА DIGITAL-ПРОЕКТА



Итак, в нашей минимальной картине мира сразу получается куча объектов. И с каждым из них много возни и неопределенности. Объекты связаны, влияют друг на друга, за всеми приходится следить и что-то корректировать. Давайте разбираться, пока поверхностно, потом пойдем в дебри.



Заказчик. Бывают два типа заказчиков. Внутренний, когда разработка идет in-house. И внешний, когда проект разрабатывается на заказ.

Заказчик может быть чертовски сложно устроен: не один человек, а группа (с противоречивыми интересами, требованиями к проекту и даже конфликтами). А еще бывают скрытые агенты влияния, например, у заказчика — жена, с которой он советуется, и ее мнение в итоге будет определять эстетическую сторону проекта. Но вы об этом не узнаете.

У заказчика определяющая роль. Он формирует концепцию проекта, приоритеты, финансирует весь карнавал. От него зависит все: что бы вы как менеджер ни делали, всегда остается шанс упереться в стенку на той стороне. Именно заказчик определяет степень успешности проекта. Подробнее о работе с заказчиком поговорим в главе 9.



Деньги. Энергия проекта, без них ничего не получится. Даже если это фановые, волонтерские проекты в свободное время или проекты за долю в будущих прибылях. Тогда деньги не присутствуют в явном виде — источником может стать сама команда. Классика провала: два друга пилили проект вместе по вечерам, а потом один женился и взял кредит. И все, приоритеты поменялись, разошлись дорожки...

Различают два ключевых формата работы. Time & Material — оплачивается время, затраченное командой, в таком случае задачи можно менять как угодно. И Fixed Price — цена и задачи оговариваются на берегу. У обоих вариантов есть как плюсы, так и минусы. Time & Material переносит риски на заказчика, но работу можно организовать быстрее и гибче. Fixed Price — заставляет разработчика зашивать риски и подстраховку в смету, лишает гибкости и снижает скорость выпуска функций из-за процедуры согласования бюджета, но яснее воспринимается клиентом. Существуют также гибридные модели.

Условия заказчика регламентируются тремя способами:



Требования. Постановки задач, технические задания (ТЗ), **тикеты**¹, рисунки на салфетках, **бэклоги**... Управление требованиями, выявление, уточнение, актуализация, расстановка приоритетов занимают массу времени. Рассмотрим подробно работу с требованиями и способы приоритизации в главе 4.



Юридические документы. Иными словами, бюрократия, более актуальная для заказной разработки. **NDA**, договоры, приложения, акты и т. д.



Тикет-система. Хранит задачи для команды и статусы по ним. Может быть частью системы управления требованиями или существовать отдельно. Может быть только для внутренних нужд команды, а может пускать заказчика внутрь.

Команда проекта. В digital команда может сжиматься до одного человека, а-ля веб-мастера, и включать в себя заказчика. Либо наоборот: разрастаться, выделяя отдельные функции в роли. Как строить и выращивать команды, мы разберем в главе 7. Командная работа подразумевает делегирование. Например, менеджер справится с задачами аналитика и тестировщика, но с ростом проекта целесообразно выделить это в отдельные роли. Или программист может администрировать серверы, если квалификация соответствует. Но как только серверного хозяйства становится много — приходится выносить это в отдельную роль — администратора. Для примера будет такая команда:



Менеджер. Ну тут все понятно, это — вы. Головой отвечаете за проект, работаете в области высоких энергий и мгновенной ответственности за базар. Про вас вся эта книга.

¹ Расшифровку терминов, выделенных серым, можно найти в «пояснениях» (конец каждой главы).



Разработчик (программист). Бородатый чувак, который пилит код. Пока без дебрей на чем, бэкэнд-фронтенд. Универсальный боец.



Дизайнер. Отвечает за весь визуал и интерфейсы. Хотя пошла мода на специализации UI/UX-дизайнеров и т. д. Подробнее об организации дизайна поговорим в главе 6.



Аналитик. Конкретизирует требования. Подробнее об этом поговорим в главе 4, посвященной аналитике.



QA (quality assurance, тестировщик). Тестирует продукт.

Поле власти. Что-то типа электрического поля, к которому подключаются менеджерские инструменты типа «делегирования». Подробнее про власть разберем в главе 2.



Руководство. Ваш непосредственный руководитель и бизнес-заказчик могут быть разными людьми. С одной стороны, у руководства свои требования и интересы, с другой — у него есть ресурс, который недоступен вам, но полезен проекту. Власть, опыт, переговорные навыки, право закупать оборудование и софт, менять процессы работы и т. д.

Кто для вас главнее: заказчик или руководитель, и что делать, если у них противоречивые требования? Например, заказчик попросил изменить процесс: нарисовать дизайн до аналитики. Можно ли? Да, если согласуете с руководителем. Приоритет отдавайте руководителю. При этом никто не против, чтобы клиент был счастлив.

Еще несколько компонентов digital-команды:



Знания и опыт. Экспертиза команды. То, благодаря чему заказчик обратился к вам, а не к конкурентам. Знания и опыт индивидуальны у каждого члена команды.



Регламенты, правила, обычаи, стандарты отрасли.

Документы или принципы, по которым организован рабочий процесс в команде. Формализуют знания и опыт команды. Такие регламенты собираются в корпоративные базы знаний и доступны всем участникам, но это не гарантирует 100% осведомленность или применяемость.



Инструменты и технологии. Компьютеры и программное обеспечение, необходимые для разработки нового софта. Навыки владения такими инструментами принято называть Hard Skills.



График загрузки команды. Чтобы спрогнозировать сроки готовности каждой функции, приходится планировать календарную загрузку каждого члена команды и учитывать зависимости задач. Если же вы работаете в мультипроектной среде — приходится учитывать загрузку команды на других проектах. Тут помогут сетевые графики, теория ограничений и метод освоенного объема из главы 5.

Проект. Это набор артефактов, в основном — файлов:

- ▶ **Код**, как правило, в репозитории.
- ▶ **Визуал** — макетов, прототипов, гайдлайна и т. д.
- ▶ **Серверная инфраструктура**, на которой ведется разработка, тестируется или работает сам проект.
- ▶ Разноуровневый **Доступ** и хранение паролей.
- ▶ Кроме кода и визуала в проекте обязательно есть **Данные**. Это либо база, либо контент, ради обработки которых проект и затевался.
- ▶ В развитых проектах код покрывают **Тестами** и пишут формальные **Тест-планы** выпуска продукта.

- ▶ Большинство проектов взаимодействуют с **Внешними сервисами**. Например, авторизуют пользователей через социальные сети или рассчитывают стоимость доставки товара. Для этого разработчики выполняют интеграцию через **API**. Подробности в главе 10.
- ▶ Наконец, в зрелом проекте обязана быть **Документация**. Ее главная задача — отчуждение знаний о проекте из голов конкретных исполнителей. Документация снижает риски.

1.1.1 ПРОГУЛКА ПО КАРТИНЕ МИРА

Итак, мы героически вляпались в задачу «довести сайт до ума». Как бы я действовал (кое-где — чужими руками)?

Жуть, этот план даже читать больно! А ведь вам завтра крутить окровавленную и облитую слезами ручку digital-разработки :)

- ▶ Поинтересовался, почему расстались с предыдущим разработчиком, на какой ноте, кто это был. Навел бы справки. По возможности переговорил с разработчиком, составил свое мнение.
- ▶ Получил от клиента предварительный список требований и задач. Рассказал всю процедуру и описал риски. Запланировал время на то, чтобы вникнуть в чужой код. Возможно, его придется полностью удалить, и все написать по новой. Первое время наша скорость будет ниже, чем у предыдущей команды, и у нас будет больше ошибок, потому что мы не знаем всех взаимосвязей. Также обсудил условия, при которых можно попробовать взяться за задачу.
- ▶ Грубо оценил стоимость проекта (вилочно, от-до), получил подтверждение бюджета у клиента.
- ▶ Согласовал работу по Time & Material. Работать с чужим проектом по Fixed Price — самоубийство. Выяснил, сколько примерно часов готов выкупать клиент в будущем и какие дальнейшие планы на сотрудничество. Разовый, короткий контракт, отношения на одну ночь меня бы не заинтересовали — в этом случае посоветовал бы закончить проект с предыдущей командой.

- ▶ Запросил доступы к коду.
- ▶ Провел код-ревью — процедуру анализа и оценки качества кода.
- ▶ Изучил текущую документацию. Если документации нет — это тоже показатель.
- ▶ Принял решение, можно ли работать с текущим кодом или нужно выбросить и все делать с нуля.
- ▶ Подписал контракт.
- ▶ Получил предоплату.
- ▶ Еще раз уточнил требования. Собрал их в бэклог. Отсортировал по приоритетам. Организовал работу спринтами.
 - Нарисовал прототипы. Сдал клиенту.
 - Нарисовал дизайн, если задача это предполагает. Также сдал клиенту.
 - Еще раз проговорил голосом результат с заказчиком, убедился, что мы все одинаково понимаем.
 - Доформировал требования на уровне задач в тикет-системе с учетом изменений, которые появились в дизайне и прототипе. Обычно это мелочи, но иногда все разворачивается на 180°.
 - Дал вычитать требования разработчикам.
 - Проговорил задачи голосом с командой, ответил на вопросы. Получил оценки от команды, например, методом Planning Poker (подробнее в главе 3).
 - При необходимости провел рисерч. Это нужно для задач, с которыми команда никогда не сталкивалась.
 - Запланировал разработку в календарном плане.
 - Написал тест-кейсы или критерии приемки по каждой из задач.
 - Запрограммировал. Следил за ходом работ, решал «затыки».
 - По итогам разработки провел код-ревью нового кода.
 - Протестировал, исправил баги.
 - Проверил производительность.
 - Сдал заказчику проект на тестовом сервере, получил и обработал обратную связь. Мелкие недочеты исправил сразу, остальное перенес в будущие спринты.
 - Провел ретроспективу с командой, посмотрел, что можно улучшить в проекте и процессах работы.
 - Актуализировал документацию.
 - Провел деплой (выкладку на боевые серверы).

- Обновил контент.
- Проверил работу на боевом сервере.
- Подписал акты, получил постоплату.
- ▶ Выяснил, что еще хотел бы клиент, повторил бы цикл. Сам предложил улучшения проекта: по коду, функциям, дизайну, удобству и простоте использования.
- ▶ Организовал работу по мелко-срочным отвлекающим тикетам (по более дорогой ставке), выполнения которых клиент не готов ждать.

В общем, у менеджера тут много дел. Причем, ценность большинства из них воспринимается клиентом весьма гадательно. И хорошо бы что-то делегировать.

Час нормального проджекта должен стоить не менее часа нормального психолога.



1.1.2 КРИВАЯ РОСТА МАСТЕРСТВА

Как выглядит кривая роста компетенций и профессионализма руководителя? А это действительно кривая.



Кривая роста мастерства

1. Бурный рост

Для человека все ново, он заинтересован и очень быстро осваивает то, что называется «язык отрасли». Это как игра на гитаре, мальчишки поймут: вам показали пять аккордов, вы освоили их за три дня и, в принципе, большинство песен во дворе уже можете петь под гитару. Все девушки ваши.

Хотя уже и на этом этапе часть людей отвалится, потому что поймут — это не для них. Такой начальный профессиональный сплит-тест — «да-нет».

2. Страшно и ни черта не понятно

Дальше расти сложнее. Надо барре ставить, табулатуры разбирать, или (боже упаси) ноты учить. И получается, что усилий нужно прилагать все больше и больше, а видимых достижений — все меньше и меньше.

Уменьшается и количество людей, которые могут оценить ваши успехи. Страшно и ни черта не понятно.

Здесь-то многие и сдаются. Особенно юные быстро обучаемые дарования. Интересней нахвататься по верхам, стать лучшим математиком среди гуманитариев или лучшим менеджером среди программистов. Лучшим летчиком среди парашютистов и так далее.

В психологии известен эффект Даннинга-Крюгера. Смысл его в том, что дилетант ведет себя, как правило, очень самоуверенно. Он чего-то нахватался, каких-то догм, и свою уверенность строит на этих догмах.

В свою очередь более профессиональный человек начинает терзаться сомнениями, поскольку уже отходит от догм, видит их ограничения, исключительные ситуации и нюансы. Начинает думать не так категорично. Поэтому со стороны порой кажется, что знания новичка, его повадки — более убедительны, чем знания профессионального специалиста, который говорит с большим количеством оговорок. Но со временем, с опытом, если человек прогрессирует — неуверенность уходит. Когда человек становится экспертом — он не так категоричен, как новичок, но уже точно знает, что сработает, а что — нет.

В росте компетенций менеджера такие перепады настроения — от ощущения всемогущества до ощущения собственного бессилия — абсолютно нормальная история. Более того, если вы таких перепадов не испытываете — значит, перестали расти.

Что делать? Просто продолжать, несмотря на страх и неясность. В какой-то момент картинка встанет на место. У кого-то этот путь занимает полгода, у кого-то — год. При этом нужно уделять больше времени практической работе на боевых проектах, а не теоретизированию.

3. Ясность мысли

Итак, после того, как с «детскими болезнями» разобрались, менеджер постепенно начинает набирать авторитет, опыт, компетенции. Вырабатывает, что называется, стиль.

Он хорошо знает правила компании, методологии, и мало-помалу начинает успешно вести проекты.

И вот тут, после первых нескольких успешных кейсов, его ждет ловушка. Дело в том, что многие вещи для менеджера уже привычны. Если раньше, например, он кропотливо работал по правилам, стара-

тельно фиксировал договоренности, мало импровизировал — то с какого-то момента он начинает чувствовать себя уверенным и крутым. И начинает эти правила нарушать.

4. Яма

Менеджер не справился с управлением, но вовремя ушел на больничный.

Стратегия полу-управленца

Менеджер почувствовал силу и начинает экспериментировать с правилами и подходами. И в какой-то момент у него начинают «сыпаться» проекты.

Например, сначала, работая с клиентами, вы все им подробно объясняли. Потому что вам самим нужно было проговорить, что и как будет. А потом вам это стало ясно-понятно. И вы перестали объяснять. И врезались в стену непонимания. Вам все очевидно, а клиенты стали как-то тупить (так это будет выглядеть для вас) и стали несговорчивыми. Причем, резко, все разом. Обычно критическая масса косяков копится, и все они вываливаются на разных проектах, но примерно в одно и то же время.

Как-то один из наших руководителей проектов пожаловался, что клиенты не понимают очевидные вещи: что такое верстка, чем она отличается от дизайна и программирования. Начались недопонимания — за что выставляется каждый конкретный счет. Приходится тратить время на урегулирование этих непониманий, а времени лишнего нет. Как выяснилось, руководитель проекта где-то поймал установку «раз уж мне очевидно — значит и клиенты должны знать, как все устроено». Однако это так не работает. Мы разобрали ситуацию. Поняли, что без резкого снижения нагрузки на менеджера нам не вырулить. Передали два проекта другим руководителям и составили «график некидалова» — недельные планы, в которых менеджер имел права заниматься не более чем двумя проектами в день. Но полноценно и сосредоточено, с созвонами, отчетами и временем на обучение клиента. Примерно через 6 недель ситуация стабилизировалась: менеджер стал меньше уставать, клиенты начали ставить хорошие оценки за этапы. Но если бы ситуацию пустили на самотек — менеджер вряд ли справился бы самостоятельно.

Это — яма. Из нее выкарабкиваются не все. Где-то семь-восемь из десяти. Тут некоторые, скажем так, кривые на душу люди, проявляют себя не с лучшей стороны, показывают свое «искусство вовремя уйти в сторонку». Причем, желательно «пока еще все хорошо». Кровь и ошметки приходится расчищать другим.

Яма неизбежна. Лучше всего, если в этот момент рядом будет опытный наставник, который вовремя протянет руку и из этой ямы вытащит.

Карабкаться сложно. Моя рекомендация: если вам невмоготу, если вы понимаете, что запутались — поговорите об этом с вашим начальником открыто, поищите вместе пути решения. В большинстве случаев вы сможете составить план и за пару месяцев выйти из кризисной ситуации, многому для себя научившись.

И даже если вы решите, что дальше с этой компанией вам не по пути — не бросайте за собой горящие города. Да, по закону вы можете махнуть рукой или шашкой: «А любись оно все конем», и через две недели ощутить вкус свободы. Но таких блуждающих менеджеров на рынке много. И кармически правильно будет привести дела в порядок, пускай это займет и два, и три месяца вашей плотной работы с вашим руководителем. Так правильно, так экологичнее. Вы большему научитесь и сможете сохранить лицо.

К счастью, большая часть людей яму преодолевает.

5. Мастерство

После кризиса уровень менеджера радикально меняется. У него появляется внутренняя уверенность. Он находит решения даже в сложных ситуациях. Такие менеджеры могут не только делать проекты, но и научить других. Это — опора компании. Факапы на этом уровне, безусловно, останутся, но справляться с ними будет уже легче.

На этом этапе руководитель проектов ищет задачи сложнее, интереснее. Он выступает на конференциях и тестирует новые технологии. Дорога в мир настоящих, жирных факапов!

Тут появляется риск перегореть — это когда после рабочего дня ничего не хочется, а сил хватает, только чтобы смотреть в стену и молча ненавидеть людей. Возникает желание уйти в монастырь, заняться духовными практиками или стать буддой. В общем, уйти в себя. Что делать — жизнь-боль, а в конце мы умираем.

На этой стадии очень важно научиться быстро расслабляться и быстро концентрироваться на задачах. Попробуйте спорт, хобби, медитации, путешествия. Кому что. Важно найти баланс между отдыхом и работой и перейти в режим гроссмейстера — когда задач решаете много, но в каждый конкретный момент сконцентрированы.

Самое интересное, что на этом развитие не заканчивается. Управленческое мастерство ничем не ограничено, нет предела совершенству, но к нему нужно стремиться.

1.2 ДЕЛЕГИРОВАНИЕ

Делегирование — это передача части полномочий или ответственности другим людям. Как правило — своим подчиненным.

Делегирование невозможно без грамотного планирования. Банально, у вас не останется времени ни на постановку задач в нужной форме, ни на контроль. О планировании времени мы поговорим в главе 8.

Делегирование — ахиллесова пята российского менеджмента. Руководители делегируют плохо, боятся или ленятся это делать. Многие просто не умеют, поскольку делегированию, как и вождению автомобиля, нужно учиться. Менеджер, который не умеет делегировать, — бутылочное горлышко организации: он набирает на себя слишком много задач и стремится самостоятельно сделать их большую часть. Грамотное делегирование — это точка роста организации.



Вы как менеджер можете вообще ничего не делать своими руками и ничего не производить. Но при этом вы отвечаете за выработку управленческих решений, распределение задач, нагрузку своих подчиненных. Делегирование — это передача подчиненному задачи вместе с необходимыми для ее выполнения ресурсами. Например, полномочиями и ответственностью: за качество, сроки и другие явно или неявно согласованные параметры.

1.2.1 ЧТО ДЕЛЕГИРОВАТЬ. ДЕЛЕГИРУЮЩАЯ СИЛА

Я завел себе регулярную привычку — раз в полгода просматриваю список дел, которыми занимаюсь. Смотрю, сколько времени они у меня отнимают. Оцениваю, насколько они мне нравятся и насколько они стали рутинными. Смотрю, должен ли я их делать сам. Можно ли делегировать что-то из рутины, неприятных мне дел или тех задач, где лучше справится специалист. Планирую: когда, кому и что я могу делегировать и в каком виде.

ЗАДАЧИ	ХОЧУ ДОЛЖЕН ДЕЛЕГИРОВАТЬ		
ЕЖЕДНЕВНЫЕ ПЛАН-КИ МЕНЕДЖЕРОВ			
ЕЖЕДНЕВНЫЕ ПЛАН-КИ АККАУНТОВ			
ЧТЕНИЕ ОТЧЕТОВ			
ПРОВЕДЕНИЕ РЕТРОСПЕКТИВ			
ОЦЕНКА ПРОЕКТОВ			
ПРОВЕРКА СЧЕТ			
ПРОВЕРКА ДОГОВОРОВ			
REVIEW ПРОЕКТОВ			

 УЖЕ
 ПОЧТИ
 КОНЬ НЕ ВАЛЯЛСЯ

Помогает табличка — что можно, а что нельзя делегировать:

Что можно делегировать	Что нельзя
Рутинная (тестирование, перенос задач в тикет-систему)	Планирование, целеполагание, выработка стратегии, контроль результатов
Специализированная работа (программирование, дизайн)	Руководство сотрудниками, мотивация, поощрения и наказания
Подготовительная работа (подготовка отчетов, договоров)	Задачи с высокой степенью риска, неопределенности, новизны, необычности или задачи личного (строго доверительного) характера
	Горящие дела, без запасов времени на делегирование и проверку

Советую завести себе подобный ритуал или регулярную задачу в вашем таск-трекере (планировщике). И постепенно увеличивать вашу делегирующую силу.

1.2.2 УСТНО ИЛИ ПИСЬМЕННО

И тот, и другой вариант уместен, но у обоих есть ограничения.

Основной плюс устного делегирования — это быстрее. Годится только для небольших, ясных и привычных заданий. Для команд, которые все понимают одинаково. Для сложных, непривычных заданий есть минусы:

- ▶ Сложно учесть все детали при постановке. Кто мало думал, тот много плакал.
- ▶ Подчиненному сложно запомнить множество деталей.
- ▶ Может аукнуться временем на переделку и повторным делегированием.
- ▶ Не оставляет артефактов. Появляется искушение при разборе полетов переиграть постановку («а я тебе сказал так-то», «а я тебе этого не говорил...»). Очень плохая практика, которая разлагает рабочую атмосферу. Да и в эту игру могут играть оба.

Письменное делегирование дает время обдумать детали. В итоге получается точнее. А размышления о предстоящих деталях минимизируют количество ошибок. Минусы:

- ▶ Требуется время на написание ТЗ.
- ▶ У подчиненного нет возможности уточнить задание в момент его получения.
- ▶ У вас нет возможности предварительно убедиться, что вас поняли правильно. Пропущен предварительный контроль.

Лучше работают гибридные варианты.

Текст + обсудить. Для сложных заданий — с чек-листами, условиями, пунктами и подпунктами — сначала пишем текст, потом даем прочитать и обсуждаем устно вопросы. Идеально для постановки задач программистам.

Если деталей в задаче слишком много, лучше разбить такие задания на несколько. Проще пропустить один пункт из 30, чем один пункт из пяти.

Выработайте ясный, четкий стиль. Помогает книга Ильяхова «Пиши, сокращай» (в конце главы есть список литературы). Можете даже первое время для тренировки прогонять постановки через сервис glvrd.ru — программа дает оценку качества стиля. Пользуйтесь сервисом, пока не почувствуете, что справляетесь без него. Постановки длиннее трех-четырех предложений навевают тоску. Не стоит писать длинные инструкции из-за паранойи и личных страхов.

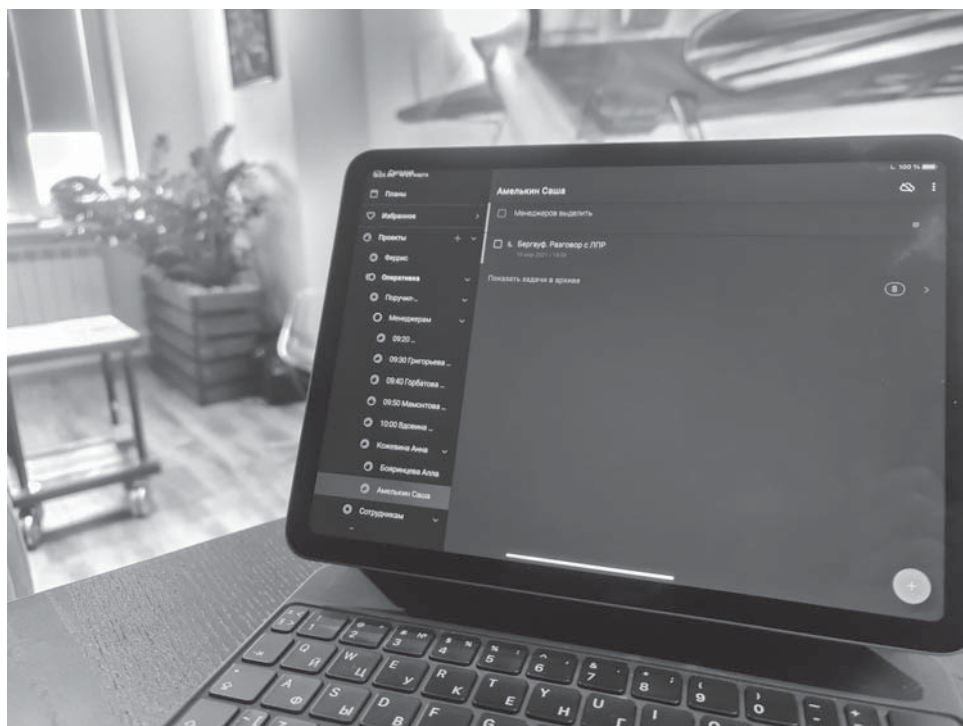


Сервис
«Главред»

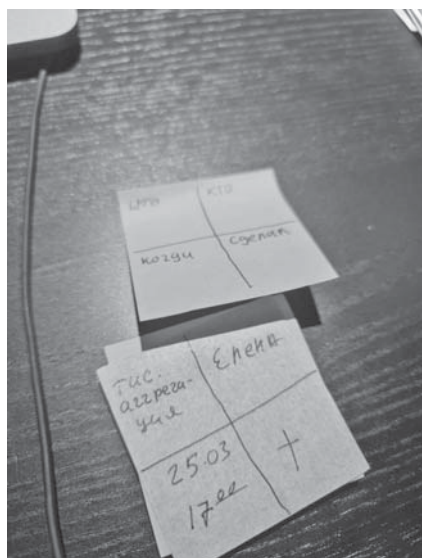
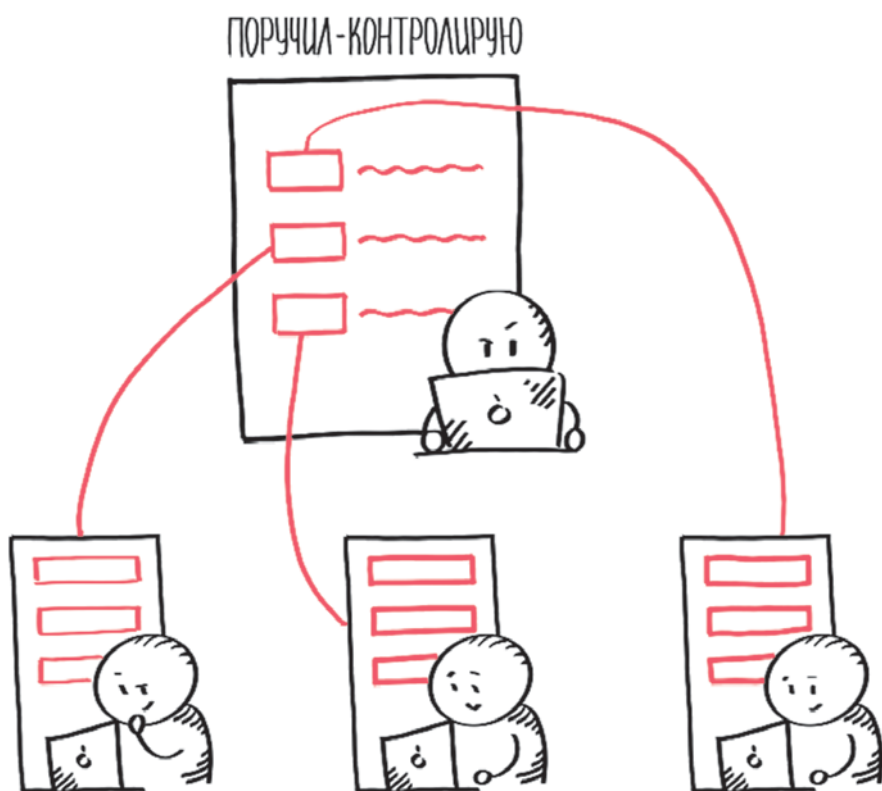
Устно + проверка фиксации. Дать устное распоряжение или даже провести брейншторм, обсудить вопросы, спросить план действий и проверить, как задание было зафиксировано. Тут работает «эффект генерации» — мы лучше запоминаем то, что изложили сами. Можно зафиксировать итоги и ключевые точки самому, но это не так эффективно. Большинству клиентов удобно давать постановки в таком виде. Хорошо работает с дизайнерами, да и в принципе удобный способ.

1.2.3 СПИСОК «ПОРУЧИЛ-КОНТРОЛИРУЮ» И СРОКИ О СРОКАХ

При устном делегировании вам нужно будет каким-то образом вспомнить о своем задании и своевременно его проверить. Я держу в личном таск-трекере список «поручил-контролирую», где кратко, пофамильно записано: кто что обещал и когда (или когда будет контрольная точка).



Список «поручил-контролирую» удобно вести в любом тудушнике или блокноте.



Подобную же технику можно организовать в бумажном планировании — стопка стикеров, каждый из которых разбит на четыре сектора: что, кто, когда и отметка «сделал». В том смысле, что сделал вовремя, как и обещал. Раз в неделю отработанные стикеры выбрасываются. Или можно накопить статистику по людям: кто подводит вас чаще всего, а кто — четкий.

При этом я понимаю, что в режиме «здесь и сейчас» не всегда получится спрогнозировать дату выполнения задания. Но всегда есть возможность спрогнозировать срок получения срока. С этим списком проходят планерки. Ребята знают, что я вряд ли что-то забуду. Повышает качество фиксации заданий с их стороны.

Кроме того, мне нужно время на проверку задания. Поэтому, если договорились сделать задачу «сегодня», то по умолчанию я ожидаю ее до 17:00 (а не в 23:59).

Рекомендую завести себе такой список, задавать вопрос про сроки о сроках, а также определиться, что значит «будет готово сегодня».

1.2.4 ГДЕ ДЕЛЕГИРОВАТЬ, А ГДЕ — НЕТ. ТУАЛЕТНОЕ ДЕЛЕГИРОВАНИЕ

Туалетное делегирование — это когда сотрудника в неформальной обстановке (в курилке, туалете, на кухне или прогулке с собакой после работы) коллеги нагружают задачами. Задачами правильными, нужными.



Но когда человек к этому морально не готов и сконцентрирован на том, чтобы НЕ работать. Из-за этого он может забыть даже факт того, что его о чем-то просили. И еще виноватым останется. Коллеги же, которые задачу выдали, чувствуют, что свой долг выполнили.

Таким образом, у туалетного делегирования три признака:

1. **Неформально.** Задача выдается в неформальной обстановке. Курилка. Кухня. Коридор. Любое нерабочее пространство.
2. **Корректно.** Задача выдается по адресу и корректно. Если не исполнили — понятно, кто не исполнил. И понятно, что именно.
3. **Безвозвратно.** Поставивший задачу сотрудник считает, что выполнил свою миссию и повторять информацию в рабочем пространстве уже не будет.

Туалетное делегирование встречается в любых коллективах, от «Газпрома» до семьи (мама попросила помыть посуду, когда сын был в разгаре онлайн-боя — как же это запомнить?). И всегда заканчивается обидами и расшатыванием авторитета озадаченного.

Так что практиковать такое делегирование — дело тухлое. Часть задач постоянно будет теряться. Уважение к человеку, которого нагружают задачами — тоже. Поэтому, если у себя в компании вы заметили, что подчиненные делятся задачами в нерабочее время или в нерабочем месте, боритесь с этим.

Туалетное делегирование — очень тонкая вещь. Подчиненный может подойти с ВОПРОСОМ (дескать, как мне быть с компанией А) — руководитель отмахнется фразой «надо подумать». И тут подчиненный получит моральное право считать, что ситуацию с компанией А теперь разруливает его руководитель. Итог: через пару дней подчиненный с чистой совестью подойдет и спросит: «Глебываныч, как там с моим ВОПРОСОМ дела?» Спасибо, что не: «Как там с моим ПОРУЧЕНИЕМ дела». Это самозахват полномочий. За такое надо бить, и бить жестко!

В отличие от пересаживания обезьян, туалетное делегирование работает в две стороны. Делегировать можно не только руководителю, но и сам начальник может бросить мимоходом: «У меня идея! Круто

бы было, чтобы у нас на проекте была такая-то фишка!» Подчиненный кивнет, скажет: «Да, реально прикольно». Итог: начальник считает, что задача поставлена. Подчиненный — что просто поговорили. Через некоторое время обоих ждет большой облом.

Решение

Избегайте делегирования в нерабочих местах. Иначе подчиненный не сможет зафиксировать постановку и взять на себя обязанность. Лучше всего подходят запланированные встречи, один на один (например, регулярные планерки раз в неделю-две или **стендапы**). Да, вы имеете право дернуть подчиненного в любой момент, но это непрофессионально и говорит о вашем слабом планировании. А подчиненного выбивает из потока — о важности работы в потоке подробнее поговорим в главе 2. Важно оставлять время на разбор вопросов и задавать конкретные контрольные вопросы о деталях. Так вы поймете, что подчиненный все правильно понял и составил адекватный план действий. Следите, не пытаются ли делегировать на бегу вам. Введите в коллективе термин «туалетное делегирование» и пресекайте это явление.

1.2.5 САБОТАЖ ПРИНЯТИЯ ЗАДАЧИ. ВОПРОСЫ РАДИ ВОПРОСОВ. ВЯЛЕНЬКИЙ АЛГОРИТМ ХРАНИТЕЛЕЙ КОСМОСА

Есть, говорят, такая секта — «Хранители Космоса». Предсказав однажды конец света, они побросали семьи, квартиры, работы и что там у них еще было. И стали ждать часа X, неистово молясь.

Настал час X. Земля не взорвалась. Пришельцы никого не забрали. Небеса не разверзлись.

Вы думаете, они сильно расстроились и разошлись по домам? Хренушки! Хранители космоса объявили, что ТОЛЬКО БЛАГОДАРЯ ИХ МОЛИТВАМ мир получил отсрочку! Хитро, не поспоришь.

Алгоритм такой:

1. Начиная новое дело, объявить о возможных негативных последствиях. Что уже само по себе КАК БЫ дает право не верить в результат и действовать деструктивно.

2. Вяленько что-то делать. Право действовать «вяленько», не напрягаясь, человек как бы отвоевал себе своим скепсисом и неверием в результат.
3. В случае наступления негативных последствий — высунуться и сказать: «А Я ЖЕ ГОВОРИЛ!!!» (Если, конечно, будет кому «а-я-же-говорить»: в случае с Хранителями Космоса и спрашивать, и отвечать было бы некому.)
4. В случае, если негативных последствий не наступило, говорить, что это ТОЛЬКО БЛАГОДАРЯ МОЕЙ САМООТВЕРЖЕННОЙ РАБОТЕ.

Алгоритм часто применяют неосознанно. Он вшит где-то на уровне BIOS-а в мозг (см. А. Курпатов, «Красная Таблетка»). В digital этим особенно грешат программисты. Иногда пропуская 4-й пункт, ибо он как бы подразумевается.

Противоядие:

1. Знать, что Хранители Космоса не уgomонились, и что их алгоритм, зараза, действует.
2. Поймать человека на применении этого алгоритма. И рассказать, что именно он делает. При необходимости — повторить. При острой необходимости и частых «А-Я-ЖЕ-ГОВОРИЛ!» поинтересоваться, не сектант ли он.

Подобная же схема работает, когда подчиненный задает много вопросов ради вопросов, уводя разговор в очень частные случаи, запутывая и как бы намекая, что начальство не продумало все до конца. Таких хитрецов надо заставлять формулировать вопросы письменно, заранее. И отказываться обсуждать с ними вопросы, которых не было в списке.

Тех, кто регулярно включает дурачка («А почему вы не сказали, что нельзя сушить кота в микроволновке?»), неплохо бы спросить, что он этим хочет вам доказать? Вы же ведь можете и поверить...

1.3 УРОВНИ ДЕЛЕГИРОВАНИЯ

1.3.1 ДЕЛЕГИРОВАНИЕ НА УРОВНЕ ИДЕИ

Подойдет для делегирования подчиненному с примерно таким же, как у вас, уровнем компетентности и ответственности, со схожей мотивацией. Вы ему доверяете, он ведет дела в том же стиле, что и вы. Таким людям можно не разжевывать задачу на атомы — хватит и постановки на уровне идеи, они сами ее «докрутят» и сделают все в лучшем виде.

Руководитель: Давай организуем семинар для клиентов?

Сотрудник: Давай, а на какую тему?

Р: Не знаю, что-нибудь про интернет-магазины.

С: Ок, в июне есть время. Давай я придумаю тему, составлю план подготовки, можем обсудить.

Р: Да сам как-нибудь самоорганизуйся :)

С: Ок.

1.3.2 ДЕЛЕГИРОВАНИЕ НА УРОВНЕ ТЕЗИСОВ

Такой способ делегирования тоже подойдет для человека из предыдущего пункта. Единственное отличие — уверенности, что тот поймет все правильно, у вас чуть меньше. Таким людям, помимо идеи, нужно задать направление.

Р: Я хочу провести семинар, как в прошлом году, на 300–400 человек. Придумай тему, напиши план подготовки и какие ресурсы тебе понадобятся для этого.

С: Ок.

1.3.3 ДЕЛЕГИРОВАНИЕ НА УРОВНЕ ЗАДАЧ

Так делегируют сотрудникам с высоким уровнем квалификации, если они уже решали задачи подобного уровня. И у которых представление о выполнении задачи сходится с вашим.

Р: Нужно организовать семинар, как в прошлом году, на 300–400 человек. Тема — интернет-магазины.

С: Что требуется от меня?

Р: Подбери зал на 300 человек в пределах 1-го кольца и не дороже 100 000 рублей.

С: Хорошо, я составлю список подходящих, через два дня покажу — обсудим.

1.3.4 ДЕЛЕГИРОВАНИЕ НА УРОВНЕ ИНСТРУКЦИЙ

В остальных случаях придется разжевывать. Особенно, если дело незнакомое, рисковое и ответственное, где в результате можно либо многое приобрести, либо многое проиграть.

Совет

Там, где уровень риска высокий, степень **декомпозиции** и детализации задач тоже должны быть высокими.

Р: Мы организовываем семинар. Вот список клиентов, вот **скрипт**. Тебе нужно будет обзвонить клиентов по списку, используя этот скрипт. Сейчас ты заучиваешь его, подходишь через 15 минут ко мне и воспроизводишь его слово в слово. Если все ок — начинаешь обзвон, вот телефон. Тех, до кого удалось дозвониться, помечаешь зеленым. Как понял?

С: Взять скрипт, взять список, скрипт выучить, подойти и через 15 минут сдать его тебе, обзвонить. Потом пометить тех, кому удалось дозвониться, зеленым.

Р: Тебе всего хватает для этого?

С: Да, все есть.

Р: А зеленый маркер? :)

Со стороны может показаться, что делегирование на таком уровне — это маразм. Дается много вводных, которые сложно запомнить и удержать в голове, поэтому тикет-системы и письменные постановки здесь спасают.

Но при этом вы не застрахованы от неверного выбора уровня делегирования, и это может произойти как устно, так и письменно.

Если вы делегируете на слишком низком уровне и слишком детально описываете задачу более компетентному сотруднику, это может его обидеть, разозлить или привести к саботажу. Чем больше в задаче будет нюансов, пунктов и детальных описаний, тем выше риск потерять какой-то из этих пунктов в процессе работы.

1.4 SMART-ДЕЛЕГИРОВАНИЕ

Популярный **фреймворк** постановки задач. SMART — аббревиатура, где каждая буква обозначает:

- ▶ Specific — конкретный;
- ▶ Measurable — измеримый;
- ▶ Achievable — достижимый;
- ▶ Relevant — значимый;
- ▶ Time-bound — ограниченный во времени.

Конкретность — хотим, чтобы сотрудник выкопал яму. Измеримость — яма должна быть метр на метр, в сквере. Достижимость — за счет каких ресурсов можно достичь результата: лопата лежит в сарае, для копания можно привлечь разнорабочих, бюджет 2000 рублей. Значимость — подчиненным важно объяснить, зачем мы достигаем этого результата, и как этот результат связан с целями компании: яма нужна, чтобы посадить дерево, это нужно для поддержания имиджа экологичной компании. Ограниченность во времени — указать конкретные сроки: через 3 дня, потому что идет сезон посадки деревьев.

Цели, которые вы ставите, должны быть трудными,
но выполнимыми.

SMART — это популярный метод постановки задач, но это не панацея. Кому-то нужно более детально разжевывать, кому-то — менее. Метод хорошо вписывается в классический цикл Деминга «планируй-делай-проверяй-воздействуй». Только «делать» будет кто-то другой:



Планирование

Предполагает, что вы оставляете в графике время на постановку задач и доведение их до подчиненных (обсуждения, брейнштормы), а также время на контроль. Если вы не запланируете на эти процедуры достаточного количества времени, не удивляйтесь, что делегирование идет как-то криво, а задачи выполняются халтурно.

К планированию относится и выбор исполнителя: важно учитывать загрузку сотрудника, рабочий график, уровень знаний и умений. На этом этапе вы также определяете параметры задания в зависимости от уровня исполнителя: постановка на уровне целей, тезисов, задач или жесткое «упал-отжался». И вот здесь, если постановка должна быть более детальной, пригодится SMART-модель. Возможно, параметры задачи придется разобрать по приоритетам: на обязательные и желательные.

Количество времени на делегирование напрямую зависит от исполнителя: опытный сотрудник, который поймет на уровне идей, или новичок, которому придется давать детальные указания. Поэтому выгодно повышать уровень компетентности сотрудников — это снижает управленческие издержки. Но если люди жалуются, что требования невменяемы, стоит к ним прислушаться и изменить уровень детализации.

При планировании вы определяете методы и ресурсы и проговариваете цели, реальные и достижимые. И, конечно, формируете сроки, которые тесно взаимосвязаны с методами и ресурсами. Например, яму можно выкопать экскаватором — это быстрее, но стоит будет доро-

же. И не забывайте о разнице между трудоемкостью и финальными сроками выполнения задачи. Сотрудник может сказать, что на задачу уйдет пять часов, но при этом у него не получится заниматься ей весь день из-за вороха других задач — из-за этого результат вы увидите через пару-тройку дней.

Контроль

Контрольных точек может быть несколько — единого правила, определяющего их количество, нет. Зависит от параметров задачи, уровня сложности, уровня компетенции подчиненных. Бывает, что может потребоваться итеративная работа по контролю (свят-свят). Бывает, что задача новая, неизвестная — и тогда приходится уточнять по ходу работы и сроки, и другие параметры. После прохождения этой стадии сроки уже фиксируются железобетонно.

Санкционирование

По итогам контроля нужно проводить управляющие воздействия: поощрять или наказывать, давать обратную связь либо менять параметры задания. Например, сменить исполнителя, поскольку тот делает задачу критически медленнее плановой скорости.

1.5 ПОСТАНОВКА ЗАДАЧ НА РИСЕРЧ

К счастью, мы работаем не в конвейерной сфере, и у наших разработчиков и дизайнеров есть поле для творчества. К сожалению, это означает, что не все задачи мы можем декомпозировать и оценить, даже примерно. Появляются новые, незнакомые технологии или темы, с которыми мы не сталкивались. В этом случае разумно выделить время на исследование. Результатом такого исследования должен стать план действий и оценка необходимых ресурсов.

Исследование требует ресурсов, иногда — серьезных, и если задача новая и неизвестная, то процесс оценки стоит разбить на стадии:

1) Постановка задачи, анализ и предварительная оценка

Например, задача — анимированная 3D-модель, а вы такого раньше не делали. Предварительно вы закладываете 8–16 часов на всю эту задачу.

2) Фаза рисерча

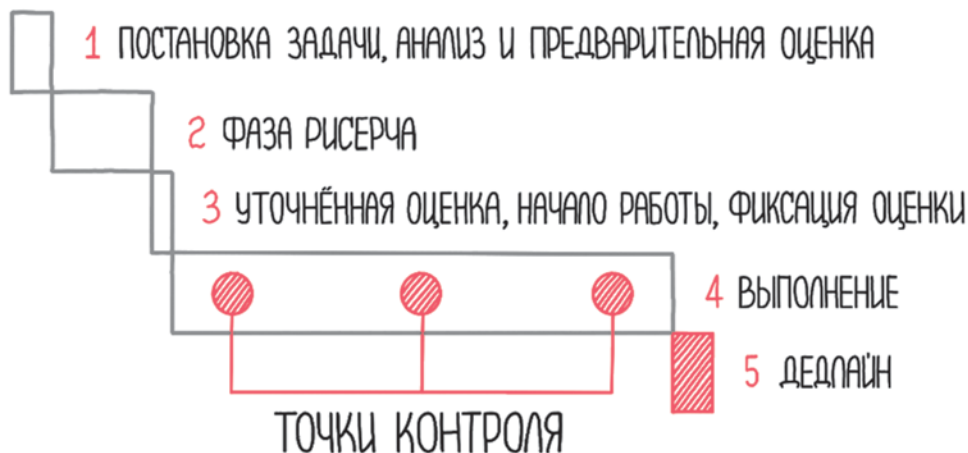
Мы берем себе час-два на «поиграться» с какой-то технологией или новым **стэком**, провести нужные эксперименты. В примере про 3D-модель нам нужно будет посмотреть, как «запекаются» текстуры, посмотреть фреймворки для 3D-анимации, изучить софт, посмотреть примеры. В итоге оценка может увеличиться в 4 раза, или вы поймете, что только на стадию рисерча вам нужны исходные 8–16 часов. Бывает. Возможно, в этот момент вы упростите задачу, обойдетесь без анимации. Или поменяете исполнителя, найдете опытного. Или добавите ресурсов. Я называю это «контролируемый факап». Провал возможен, но глубина провала известна заранее.

3) Уточненная оценка, начало работы, фиксация оценки

Когда вы поняли, с чем имеете дело, оценки фиксируются железобетонно.

4) Выполнение задания. Согласованные контрольные точки

Дальнейшие изменения оценки возможны только как форс-мажор или как косяк. И как только появился какой-то затык, нужно немедленно о нем сообщить. В длительных и сложных задачах также согласовываются контрольные точки.



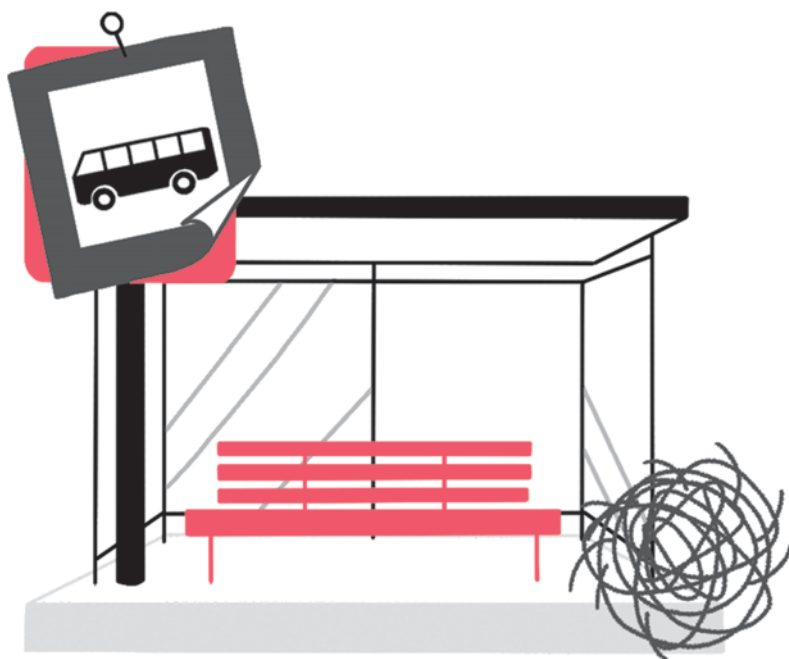
На первом и втором этапе оценки и прочие параметры можно менять.
Начиная с третьего — уже нет.

Если у вас нет навыков в сфере, в которой нужно делать декомпозицию, ее нужно поручить сделать подчиненному с соответствующими навыками. Результат получить в письменном виде. И если что — проверить сторонним специалистом, мнению которого вы доверяете.

1.6 ФАЛЬШИВЫЕ АВТОБУСНЫЕ ОСТАНОВКИ. ТОНКОЕ ИСКУССТВО КОНТРОЛЬНЫХ ТОЧЕК

Есть такое явление — фальшивые автобусные остановки. На них висит расписание, но автобус к ним никогда не приезжает. Остановки устанавливаются на территории или рядом с домами престарелых или другими клиниками, где содержатся люди, страдающие старческим слабоумием. Пациент, решив сбежать из стационара, часто остается на фиктивной остановке ждать автобус в уверенности, что он сможет оттуда уехать. Здесь его может обнаружить или догнать медицинский работник.

Это яркий пример, как стереотипы и привычки можно применять для управления и выстраивания контрольных точек. Старческое слабоумие — момент далеко не ключевой.



Пример из практики: один мой знакомый очень любит внедрять KPI. Из пяти метрик одну делает фальшивой. В реальной работе получить ее нельзя. Хитрецы же могут накрутить любую херабору.

Другой пример: как-то я подготовил чек-лист, первым пунктом которого было: «Внимательно прочитать документ до конца». Дальше было примерно 30 пунктов. Последним был «Выполнить пункты 1 и с 5 по 12, остальные проигнорировать». Жестко, да?

Нет четкого алгоритма расстановки контрольных точек. Их не должно быть очень много. Это лишняя нагрузка на вас и раздражитель для исполнителей. Попробуйте мысленно представить, как выглядит провал по задаче. Теперь мысленно представьте, что было до этого провала. Попробуйте найти такую точку, где провал уже возможен, но ситуацию можно исправить. Постарайтесь использовать «предсказывающие» контрольные точки.

Например, можно отслеживать, что исполнитель **закончил** задачу (и тогда о провале вы узнаете без запасов времени), а можно — что **приступил**. В этом случае хотя бы синдром студента — прокрастинация до последнего — вашему исполнителю не грозит. Договориться о контроле лучше заранее, во время делегирования. Избыточный или внезапный контроль бесит умных, талантливых людей.

Используйте предсказывающие контрольные точки.
Оговорите их заранее.

1.6.1 КОНТРОЛЬНЫЕ ТОЧКИ МЕТОДОМ ПОВОДКА

Посмотрите, как собаководы выгуливают своих питомцев. Ведет себя хорошо — дают полную свободу. Отпускают с поводка. Ведет хуже — поводок натягивается. И чем хуже — тем короче поводок. А могут и за ошейник взять.

Вот так и с контрольными точками. Справился ваш подопечный — молодец! Даем больше свободы. Большие сложные задачи, меньше контроля. Не справился — значит, пока рановато такие задачи решать. Уменьшаем поводок. Задачи мельче и проще, больше контрольных точек.

1.7 ОШИБКИ ДЕЛЕГИРОВАНИЯ

Но если все ясно, то почему руководители не делегируют или делегируют через ж... плохо?

1.7.1 НЕУМЕНИЕ ДЕЛЕГИРОВАТЬ

Бывает, когда руководитель — адреналиновый наркоман, которому нравится работа в аврале. Когда все срочно, все важно и все горит. Проблема в том, что, как и с любой зависимостью, уровень адреналина нужно постоянно повышать, иначе становится пресно и неинтересно.

И тогда человек накидывает на себя все больше задач, а сотрудников погружает во все больший форс-мажор — чтобы только опять все «горело». Как следствие, в какой-то момент менеджер превращается в супермена, который прилетает и решает мирские проблемы. Собственно, ради этого ощущения «на острие атаки» весь аврал и затевается. Другой бонус — алиби очень занятого человека и повод для оправданий в духе «я впахивал и сделал все, что мог». Иными словами, «я в домике».

Вторая причина так-себе-делегирования — ЧСВ. Чувство собственной важности. Это когда руководитель намеренно ставит задачу так, чтобы подчиненный сделал ее плохо или не сделал вообще. В итоге руководитель приходит к сотруднику, отодвигает его в сторонку и начинает делать сам с гордым выражением лица «смотри, как надо». К сожалению, в российских реалиях это встречается чаще, чем хотелось бы.

Другой вариант — руководителю страшно хочется что-то сделать самому. Ну вот прямо руки чешутся. Стоит вспомнить, что многие руководители вырастают из хороших специалистов. А это люди, которые годами нарабатывали свою квалификацию и которым сложно взять и вот так просто расстаться с этими навыками. Время от времени это нормально, если руководитель добивается результата своими руками, и при этом большая часть работы делается руками других людей через делегирование. Но важно отдавать себе отчет: для чего вы делаете эту работу. Просто чтобы поддержать свою квалификацию или потому что вы уклоняетесь от руководства?

Понятно, что не бывает без форс-мажоров, когда нужно все бросать и лично подключаться к команде, чтобы задача была решена вовремя,

и по ней все было хорошо. Единственная проблема — большую часть форс-мажоров создаем мы сами, своими руками: например, неправильным планированием своего времени и времени подчиненного. А потом внем себе, что виноваты звезды.

1.7.2 БОЯЗНЬ ИДТИ НА КОНФЛИКТ

Подробнее об этом поговорим в главе 2, про власть и мозгоклюйство. Одна из частых причин избегания делегирования — неуверенность в моральном праве давать распоряжения. Вчера вы сидели за одним столом, ходили вместе на обед, а сегодня — вы уже руководитель, а он — подчиненный. И с этим теперь надо как-то жить.

Другая причина — нежелание столкнуться с сопротивлением. Действительно бывает, что подчиненный либо морально сильнее вас, либо превосходит по уровню эмоционального интеллекта. Поэтому, если появляются какие-то проблемы по задаче, возникает желание сделать ее самостоятельно — лишь бы не провоцировать конфликт, навязывая кому-нибудь свою волю.

Часто молодые руководители проектов опасаются испортить отношения с подчиненными — эдакое желание быть хорошим и добрым для всех. Со временем проходит, но не у всех. И, конечно, менеджеры часто стремятся избежать публичных ошибок. Когда вы отдаете в распоряжение сотруднику какую-либо задачу, вы можете ошибаться в ее постановке или в технических нюансах. Либо ваше распоряжение может привести к негативным последствиям. Когда происходит ошибка или такие последствия наступают, и вам на них указывают, это всегда удар и по власти, и по чувству собственной значимости. Самое противное — потом кто-то может долго припоминать эту ситуацию.

1.7.3 НЕУВЕРЕННОСТЬ В СВОИХ ПОДЧИНЕННЫХ

Для неуверенности могут быть разные причины. Подчиненные могут быть слишком загружены — и тогда из-за неуверенности в том, что задача втиснется в чей-то график, руководитель ее просто не ставит.

Также вы можете быть не уверены в квалификации подчиненных. Но по-хорошему грамотный руководитель должен заниматься посто-

янным мониторингом квалификации сотрудников и наращивать ее до необходимого уровня. Другая крайность — неуверенность в мотивации подчиненных: а хватит ли им мотивации выполнить поставленную задачу?

Еще один вариант — сомнения, что сотрудники верно понимают поставленную задачу и сделают ее именно так, как вы того хотите. Бывает, что вы даете распоряжение в общих словах. Например, «поедь к Иван Иванычу, возьми такой-то документ и привези его мне». И вот, приходит подчиненный и говорит: «Не получилось». Он не съездил, а позвонил. Иван Иваныч был на совещании и не перезвонил. А вы-то знаете, что Иван Иванычу звонить бесполезно — он такой человек, к которому нужно именно приехать. И здесь не остается никаких рычагов, кроме режима бармалея: «Делай так, а то уволю!» Просто потому что на рассказ о всех тонкостях взаимоотношений с Иван Иванычем у вас уйдет полдня.

1.7.4 НЕТ ДОСТАТОЧНЫХ УПРАВЛЕНЧЕСКИХ КАЧЕСТВ

Если руководитель не умеет планировать свое рабочее время и время своих подчиненных, не оставляет места для постановки задач и для контроля, то ничего удивительного, что делегирование начнет сыпаться. Для контроля за выполнением работ действительно нужно закладывать время, а после контроля — давать обратную связь и следить, чтобы эти изменения были внесены. Это тоже не все умеют.

Часто бывает, что руководитель и подчиненные очень по-разному понимают круг своих задач и ответственности. Это также может стать причиной конфликтов и неудачного делегирования.

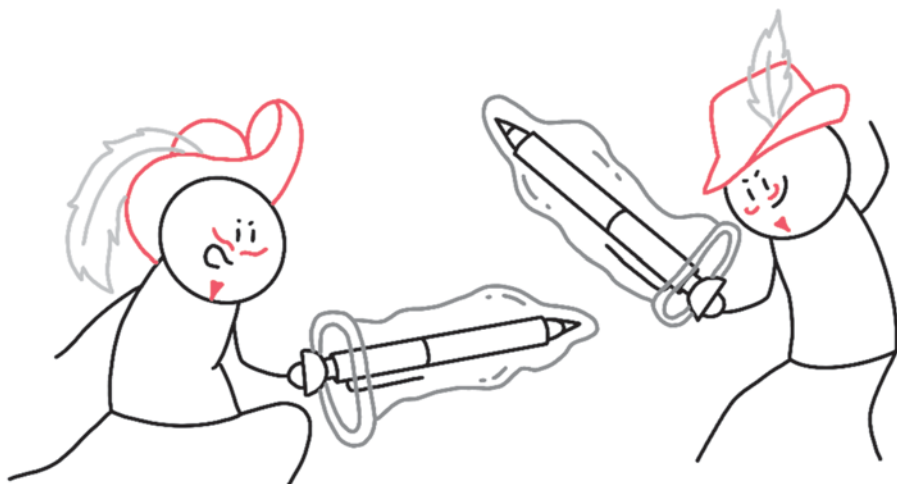
Иногда руководителям не хватает умения замечать командные достижения. Например, вы попросили свою команду самоорганизоваться и проверить проект. Сами убежали заниматься великими делами — скажем, спасать другой проект. А потом вы предъявляете претензии к команде: почему на проекте есть баги, почему не все проверено. И вот вы уже снова в роли спасателя проекта: героически до часу ночи тестируете, а потом с гордой миной заявляете, что только благодаря вам все работает, как надо. А команда — так, мимо проходила.

Сама процедура делегирования зависит не только от того, как поставлена задача, но и от уровня подчиненного. Отправляя человека по пути, думайте, кто по этому пути пойдет.

1.8 ПРАВИЛА ПИСЬМЕННОЙ КОНТРАЦЕПЦИИ. КАК СТАВИТЬ ЗАДАЧИ, НЕ УБИВ ДРУГ ДРУГА

Программисты — как джинны. Осторожнее в своих желаниях, они могут исполниться.

Как письменно формулировать свои требования к программистам? С одной стороны, навык несложный, но именно из-за деталей чаще всего можно получить на проекте долгоиграющий геморрой, а потом ходить и удивляться, почему все складывается наперекосяк.



Итак, солнечное утро. Вы сидите за чашкой кофе. Лениво просматриваете свой проект. И находите в нем какой-нибудь баг. Рука тянется открыть отдельное окно браузера, открыть таск-менеджер, например Jira. Авторизоваться там, найти нужный проект, поставить задачу с описанием бага в грамотных формулировках, приложить скриншоты, описать, как этот баг воспроизвести, назначить ответственных, запланировать себе контроль и так далее. Вот если так — то вы святой человек. Ведь интуитивно в этот момент хочется сказать: «Опять эти упыри накосячили, я что им — **бета-тестер** что ли?!» Ну, и устроить этим уродам веселую жизнь. И не дай бог они начнут сопротивляться.

Почему так? Почему даже мелкий баг на проекте может приводить в бешенство и заказчиков, и менеджеров проектов? Причина тому — транзакционные издержки.

Мы интуитивно чувствуем, что на решение мелкой проблемы придется затратить довольно много энергии. На мелкие огрехи часто проще махнуть рукой, чем доделывать те последние 10% работы, которые занимают оставшиеся 50% времени. Возможность быстро поставить задачу, прилагая минимум усилий мозга и телодвижений, напрямую влияет на качество проекта и добрые отношения внутри рабочей группы. Хотя и двояким образом.

Минимум транзакционных издержек — это когда программист сидит рядом, под боком, и вы силой голоса призываете его, тычете пальцем в монитор и говорите: «Опять ничерта не работает, пойдй разберись!» Нормально ли это? Отчасти.

Совет

Большая часть крупных релизов перед дедлайном и перед выпуском проходят через фазу стресса. И здесь возможно все — вплоть до трехэтажных матов и жестких конфликтов. В целом, это даже полезно. Главное, чтобы такое не вошло в традицию. День-два в таком ритме можно пережить, дальше — нет. Все должно войти в русло, задачи должны поступать из одного источника, а не сыпаться на бедного-несчастливого разработчика с пометкой «СРОЧНО», «КОГДА УЖЕ БУДЕТ» и «НЕМЕДЛЕННО».

Рекомендую посмотреть старое интервью со Стивом Джобсом про драки внутри команды и как они влияют на качество продукта — я его периодически показываю своим ребятам, когда начинаются конфликты.

Вернемся к разработчику, который услышал резко высказанное замечание «баг на проекте» или «ничего не работает». У него всего два варианта действий: либо уйти в оборону, либо — в нападение. Но что адекватного можно ответить на формулировку «Ничего не работает»?! У тебя компьютер что ли не включается?!

Часто из-за нехватки времени на постановку, из-за транзакционных издержек и из-за нежелания думать менеджеры и клиенты склонны ставить неконкретные задачи. На что программисты абсолютно ре-

зонно попросят больше конкретики — мол, пиши техзадание. А могут и психануть.

Когда атмосфера накаляется, диалог может складываться как-то так:

— У вас баг на проекте!

— А где именно?!

— ВОТ ТУТ, ЕПТ!!!

Плюс скриншот.

Когда пишешь разработчику матом, либо просто «баг» или даже «косяк» — это воспринимается как личное оскорбление (чуть позже мы посмотрим на когнитивные искажения). Это серьезный плевок в душу разработчика. Пишите простыми, понятными формулировками. Не машите кулаками там, где не должно быть драки. А с надписями КАПСОМ будьте совсем осторожны — это воспринимается так, будто вы на человека ОРЕТЕ. Конечно, только если это не ваш управленческий ход, чтобы загнать кого-то под тумбочку. Только ведь он потом оттуда вылезет! В общем, давайте не капсить. В мире и так слишком много этого дерьмеца.

1.8.1 МЕНЬШЕ НАСИЛИЯ, ДЕТКА!

Вообще, речь на письме выглядит значительно жестче, чем то же самое, произнесенное устно. В подтверждение — известный анекдот про фразы «Папа! Вышли денег!» и «Папа, вышли денег». Если вы имели опыт переписки с клиентами из Европы или США, то наверняка заметили, что те очень часто используют вежливые слова — «пожалуйста», «спасибо» — и часто благодарят. Слова не влияют на суть, но могут сгладить стиль общения. В русских реалиях такое встречается редко. Причина — якобы, вежливая речь может быть воспринята как слабость. На мой взгляд, эта идея глупая, и письменную речь нужно обязательно смягчать.

На тон сообщения влияет буквально все — особенно смайлики и знаки препинания в конце предложений. У Пелевина есть интересная фраза: «Смайлик — это визуальный дезодорант». Поэтому стоит приучить себя к крайне вежливому стилю, не лениться писать «пожалуйста» и «спасибо», ставить смайлики на письме, чтобы сгладить жесткость и сформировать позитивное отношение к себе в коллективе. В **баг-**

листах или задачах это может быть не всегда уместно, но в переписке, в постановке задач, в комментариях — точно не навредит. Посмотрите, как по-разному будут читаться, вроде бы, хвалебные слова, в зависимости от знака в конце предложения:

Молодец...

Молодец.

Молодец!!!

Молодец :(

Молодец :)

ХОРОШ!

Оформляя баг-листы или письма, всегда задумывайтесь хоть на полсекунды, как это прочитают и с какой интонацией. У нас в студии терпимо относятся, если разработчик использует ненормативную лексику в баг-листах и в комментариях. Для менеджеров и тестировщиков это — табу, для разработчика — иногда можно, если он этим не злоупотребляет, и если эти комментарии не увидят клиенты. А вот за неуважительные высказывания, устные или письменные, по отношению к клиенту уже надо наказывать или, как минимум, — разбираться, разговаривать про добро и зло. Для меня это индикатор «все ли нормально на проекте?!». Потому что, если появляется ругань в сторону клиента или проекта, дальше по инерции ситуация становится все менее и менее управляемой.

Совет

Не допускайте КАПСА, мата и ругани в письме и старайтесь отучать от этого коллег. И не стесняйтесь использовать смайлы, чтобы смягчить жесткость, присущую письменной речи.

Окей, вы такой молодец, написали разработчику вежливое обращение со смайликом и скинули ссылку. Насколько это конкретно? Приложили скриншот? Уже лучше. Можно стрелок к нему еще нарисовать. Уже почти хорошо. Только все равно не спасает. Многие грешат надписями на скриншотах. Но часто конкретики с ними все равно нет.

Я не преувеличиваю ни на грамм. Ставится задача с формулировкой «Косяк на сайте». КАПСОМ. Ставится ссылка на скриншот, где тоже что-то неминуемо написано капсом. Не удивляйтесь, если в комментариях к такой задаче вам также начнут отвечать капсом, нервно и грубо. Тикет-система тут будет не виновата — так накуролесить можно и в Jira, и в Trello, и в чем угодно еще.

Надписи на скриншотах не возбраняются, но они должны быть конкретными.

1.8.2 НАЙДИ МЕНЯ ПОТОМ, ПОПРОБУЙ!

Вместо «Вот тут проблема» можно написать: «В тексте ссылки на отзыв — абракадабра». Гордый собой менеджер или заказчик считает, что на этом его миссия по формулированию проблемы — заметьте, четкому формулированию — закончена. Мол, вот баг, вот скрин, описание — смотри на скрине. Но так не пойдет: если таких багов двадцать (или двести), баг-лист превращается в мешанину, и как что искать в таком случае, неясно.

Сейчас за кадром оставим вопрос, почему у вас заказчик нашел баги, и их так много, но акцент не на этом. Это могут быть и формулировки заданий — разницы нет.

Если в таком виде с вами работает ваш клиент, и у вас не получается договориться о нормальных формулировках (ему тоже не до формулирования и четкости и проще вам заплатить побольше денег, чтобы вы сами все фиксировали с его слов), тогда проблем нет. Фиксируйте все сами, сами переводите с клиентского на программистский. Берите за это деньги. Это воспринимается абсолютно нормально в 80% случаев и ценится (если вы не сильно косячите). Вы экономите клиенту время — и это тоже часть менеджерской работы. **К исполнителям формулировки должны попадать такие, чтобы задачу можно было найти поиском.** Очевидная вещь, но приходится и про такое рассказывать.

Формулировки стоит писать так, чтобы по ним было легко найти, где случился и как проявился баг. Подробно описывайте, что заставило баг вылезти. И поменьше оценочных формулировок: что это за «абракадабра в тексте ссылки»? В том же Битриксе, например, по умолчанию не генерируются ссылки с человеко-понятными названиями, любой

программист вам скажет про это. И даже не попытается понять вашу проблему. Мол, в коробочной версии это работает именно так, чего вы от меня хотите?! Это нормальное поведение программистов — и нельзя их за него осуждать.

1.8.3 ПРОБЛЕМА ИЛИ ТРЕБОВАНИЕ?

Бывают формулировки, из которых непонятно: это проблема или требование? Пример — «ссылка фиолетовая». И что? Она должна такой быть или ошибка в том, что она фиолетовая, а должна быть серой? Дело в том, что непонятно, что именно вы хотите, чтобы с этим фактом сделали и как именно его поправили.

1.8.4 МНЕ КАЖЕТСЯ...

Другая частая ошибка менеджеров — формулировки задач в виде пожеланий. С оговорками «мне кажется», «я думаю», «а как на ваш вкус» и так далее. Опять же, если такие формулировки приходят от клиента — нормально, если менеджер их уточняет и конкретизирует. Слово «кажется» — это из словаря терминов-паразитов. В баг-листах, в формулировках задач прилагательные и местоимения — это слова, которые можно трактовать двояко. «Каждый», «любой», «все», «больше», «меньше» и так далее. Это признак нечеткой постановки. Как минимум — перепроверьте, можно ли такие слова перевести в четкие цифры или конкретизировать (не всегда, но можно).

1.8.5 ПОПЛАЧЬ О НЕМ... В ДРУГОМ МЕСТЕ!

Еще одна дичь — эмоциональность (или даже скрытая пассивная агрессия) в постановках, чатах или рабочих артефактах. Ниже — пара примеров из практики: правильно-неправильно и пара примеров на «подумать», как могло бы быть правильно, а главное — чтобы поставить себя на место того человека, который получил задачу или обратную связь о своей работе в таком виде:

Неправильно	Правильно
Имею подозрения, что плавающая шапка не по макету, если таковой имеется. Если не имеется, то я утверждаю, что надо переделывать.	Плавающая шапка. Сверить с макетом.
А шо с заголовками?	Сделать заголовок по макету.
Карточки товаров не по макету. Почему?	Сделать карточки товаров по макету.
Идей нет. Предлагайте. ФуллХД выглядит стремно. Нарушена высота по сравнению с макетом.	<ваш вариант и чувства в этот момент?>
Мне плохо (((((Можно я не буду тестировать дальше адаптив, а просто скажу, что ВЕЗДЕ НАДО ПЕРЕДЕЛЫВАТЬ? У меня на нервной почве живот заболел уже. Я же ненавижу баги. Переживаю всем сердцем ((((((	<ваш вариант и чувства в этот момент?>

Представьте, что программист переживает расставание с подругой, пришел с плохим настроением. Открывает текстовый редактор. Создает файл VsePloho.php. В нем — **класс** PolniyPipes. А в нем метод KakViVseDostali (vi, vse). На другой день настроение наладилось, помирились, новый файл уже SuperPuperMegaKorzina.php и все в таком роде. И это все идет на ревью службе безопасности. Думаю, там разговор будет короткий.

1.8.6 ПРИНЦИП ПИРАМИДЫ

Организуйте постановки по принципу «пирамиды». Заголовок задачи должен быть такой, чтобы была понятна суть и не приходилось много читать. Детали помещайте в описание.

1.8.7 ПРИОРИТЕТЫ

В баг-листах, помимо описания проблемы, ее места на сайте и способов воспроизвести баг, также указывается **приоритет**. В разных компаниях системы приоритетов обычно складываются исторически.

У нас она — вот такая:

- ▶ 0 — критические баги (падает сервер или не работает оплата), нулевой приоритет ставим в случае, если тестировщик из-за этого бага не может дальше продолжать проверку проекта.
- ▶ 1 — либо что-то забыли реализовать, либо что-то критичное по юзабилити.
- ▶ 2, 3 — менее критичные вещи.
- ▶ 4 — ошибки в текстах или текстовые правки.
- ▶ 8 — это «хотелки», некритичные баги.

Замечу, что это именно наши приоритеты по разработке. Например, у рейтингового агентства «Тэглайн» приоритеты другие: для них ошибка в продающем тексте может быть более приоритетна, чем криво работающая форма подачи какой-нибудь анкеты. Ну, ради бога.

Мы намеренно пропускаем 5, 6, 7 — если такие приоритеты понадобятся на конкретном проекте, мы их просто придумаем, не ломая основную систему.

Соответственно, если поставить нашей задаче приоритет, четко показать программисту, что это — «хотелка», он, скорее всего, это сделает, не задавая лишних вопросов. Кстати, приоритеты — очень полезная вещь, поскольку мы можем управлять качеством продукта. Например, закрываем все под 4-й приоритет включительно и запускаемся. Почему бы и нет, если вдруг скорость запуска нам приоритетнее полировки проекта.

1.8.8 ПЕРФЕКЦИОНИСТЫ ДОЛЖНЫ ГОРЕТЬ В АДУ. НО ЭТО НЕ ТОЧНО

Есть клиенты-перфекционисты, которые считают, что нельзя выпускать проект с незакрытыми багами. Увы, в наше время это утопия. Любой софт выпускается с какой-то долей некритичных ошибок. С какой именно — это вопрос дискуссионный. Бывает, что пользователи используются как

бета-тестеры. Скорость на запуске (или Time To Market, TTM — время от начала разработки идеи до ее конечной реализации) важнее. Понять это можно, посмотрев на обновления приложений, которые прилетают к вам в телефон. Большая часть обновлений в описании содержит две строчки: «Увеличена стабильность, улучшена производительность». На русский язык переводится как: «Мы поправили пару багов». Даже у Apple ошибки, которые известны годами, — это норма.

Исключения — софт, отвечающий за критические элементы инфраструктуры: ядерные реакторы, системы управления двигателями и так далее. Но и там не все гладко, можете почитать про компанию «Тойота», у которой 11 тысяч глобальных переменных и 82 тысячи нарушений в коде. В общем, ошибки в коде будут всегда, как только проект перерастает уровень песочницы.

1.8.9 ГОРШОЧЕК, НЕ ВАРИ

Было ли у вас такое, что вы моете посуду, а вам все время подкидывают новую? Опа, а тут еще и сковородка! Это откровенно бесит. Я считаю, что программисты должны видеть конечный набор багов и задач. Поэтому, если тикетов много, стоит организовать работу мелкими итерациями (спринтами). И даже если параллельно с исправлением багов идет процесс тестирования, то новая пачка багов должна попасть к программистам, только когда они отработают и закроют уже выданный им объем. Это не всегда возможно и не везде применимо, но стоит к этому стремиться, потому что так в итоге спокойнее.

1.8.10 ПОДВЕДЕМ ИТОГИ. ПРАВИЛА ГРАМОТНОЙ ПОСТАНОВКИ ЗАДАЧ РАЗРАБОТЧИКАМ

- 1. Маты и КАПС.** Вам — нельзя. Разработчикам — иногда можно.
- 2. Место.** Указывайте конкретное место, где возникла ошибка.
- 3. Поископригодность.** Пишите так, чтобы запись легко искалась по ключевых словам.
- 4. Пирамида.** Используйте принцип пирамиды. Важное — в начало. Детали — в кончало.

- 5. Скриншоты — важное дополнение.** Подкрепите текстовое описание скриншотом.
- 6. Решение, а не мнение.** Формулируйте не только проблему, но и ожидаемое решение — четко и однозначно, а не в виде абстрактных пожеланий и мнений.
- 7. Не впадайте в истерику.** Пишите по делу, без эмоций.
- 8. Приоритизируйте.** Расставляйте приоритеты и организуйте по большим баг-листам работу микро-спринтами.
- 9. Закончите уже!** Не подкидывайте посуду!

Вот тогда вы как менеджер будете сильно нравиться программистам, и в мире будет больше добра, радости и взаимопонимания. Но это не точно :)

1.9. СЕМЬ ПРОСТЫХ ИСТИН ИЗ АВИАЦИИ О ДЕЛЕГИРОВАНИИ И КОНТРОЛЕ

В авиацию я попал не через авиацию, а почти случайно. Просто повезло столкнуться с правильными людьми. С тех пор прошло несколько лет. Кроме самолета освоил вертолет, посчастливилось полетать по Алтаю, освоить строгий спортивный самолет Су-29, 3-ю лигу по пилотажу, поучаствовать в паре авиашоу. Летаю меньше, чем хотелось бы, и до сих пор чувствую себя дилетантом. Постоянно узнаю новое. Этот опыт меняет картину мира в управленческой работе над digital-проектами.

1. Все, что можно — проверяй сам

Как правило, самолеты должны обслуживать специально обученные техники. Их у нас мало.

Да, есть специальные чек-листы проверки самолета перед вылетом. Обойди самолет, все проверь, все пошатай, бла-бла-бла... Но есть и фактор разгильдяйства. Ребята рассказывали про забытые техником в самолете пассатижи. Рули переклинило. В тот раз обошлось, летчик чудом посадил машину. В авиации, если ты хочешь жить и летать долго — ты просто обязан проверять все сам.

Делаешь проект, управляешь проектом — да, у тебя есть команда, которая вроде как все смотрит, видит, знает, тестирует. Но глуп и беспечен тот руководитель, который только и делает, что верит всему на слово. Не проверит проект лично, на себе. Не сложит своего мнения о происходящем.

2. Зрелищно — не всегда круто

Из опыта участия и наблюдения за авиашоу — публику больше всего заводят проходы на предельно-малых высотах с грохотом и дымами (восторг!). Технически сложные фигуры и обратный (красный) пилотаж, когда от перегрузок лезут глаза на лоб — любят, но не ценят пропорционально тем усилиям, с которыми даются эти фигуры. Например, штопорные бочки — крайне противные фигуры, съедающие все силы. Но они не настолько вау-эффектные. Профессионал оценит. Публика — нет.

Я видел в проектах, когда море сил тратится на иллюстрации (в том числе — 3D), а по итогу — ну да, картинка и картинка. И наоборот, когда стоковое видео или фото в банальном слайдере вызывает реакцию: «Вау! Какой дизайн!»

3. Мелочь ничего не прощает большому

В авиации мелочей не бывает. Никаких. Нигде. Любая мелочь может тебя угробить.

Незакрытые лючки в полете или незакрытый фонарь, недотянутая гайка, и дальше уже ситуация развивается быстро, и, как правило, хреново.

Как-то после зимы я выполнял тренировочный полет. Ремни на пассажирском месте были благоразумно пристегнуты. Все пять раз проверено. Выполняю фигуру «вертикаль» (самолет летит вертикально вверх, теряет скорость, вверху переворачивается и летит вертикально вниз, быстро набирая огромную скорость). На выходе из вертикали ручка управления вдруг становится невероятно тугой, самолет еле слушается.

Ориентируюсь: поджопник с пассажирского кресла заблокировал дублирующую ручку управления. Как позже оказалось — за зиму высох клей. Благо, в RV-7 пассажир и пилот сидят бок о бок, ситуацию удалось мгновенно исправить.

Ты можешь сделать круто на своем проекте глобальные вещи. Но

мелочи — это то, что может погубить твой проект. Обращай внимание на каждую деталь, какой бы мелкой и незначительной она ни казалась. Мелочи — это то, что может стать большой проблемой, если вовремя на них не отреагировать.

4. Тщательно выбирай, кого брать на борт

Я очень избирательно отношусь к пассажирам-покатушникам на борту и очень тщательно их инструктирую. Вряд ли возьму незнакомого или показавшегося хоть в чем-то неадекватным человека.

Был случай, когда на взлете испугавшийся (или от эйфории) пассажир попытался перехватить у меня управление. Благо в RV-7 он сидел рядом, и можно было успокоить пассажира. В самолетах типа Extra, Як-52 или Су-29, где кабины разделены и посадка друг-за-другом — такой возможности нет.

Ребята рассказывали про ситуации, когда испуганная девушка-пассажир крепко зажимала сильными ногами ручку управления самолетом и на призывы: «Расслабься. Расслабь ноги. Ноги расслабь! РАЗДВИНЬ НОГИ \$%^...!!!» — реагировала слабо.

Кто у тебя будет на проекте, и как он себя поведет — от этого во многом зависит будущее твоего проекта. В идеале, у тебя должна быть возможность менять членов команды, особенно если ты головой отвечаешь за результат.

5. Вся ответственность, один черт, на тебе

Да, есть куча всяких контролирующих органов, транспортной прокуратуры, проверок, центров сертификации, подач планов, диспетчеров, техников, метеорологов, и всякого такого. Но как только ты — КВС (командир воздушного судна) и принимаешь решение о взлете — ВСЯ (ВСЯ!) ответственность на тебе. Отмазки не канают. Никакие.

Ты управляешь проектом (или компанией) и позволяешь себе оправдания, типа «ой, я тут всего лишь не посмотрел/не подумал», или любишь переваливать ответственность на внешние обстоятельства, сложного заказчика или рукожопых исполнителей? Сорян, тогда рано тебе еще этим заниматься.

Брать на себя всю ответственность — это, кстати, прикольное чувство, попробуй.

6. Летать больше — безопаснее, чем летать мало

Чем больше ты летаешь — тем больше будет разных ситуаций.

Вот недавно знакомые летчики в районе Телецкого озера попали в облако саранчи. Одно зеленое насекомое попало в ПВД — приемник воздушного давления, который обеспечивает определение скорости относительно воздуха. Итог — отказал указатель скорости.

Скорость — наша жизнь. В горах — особенно интересно. Но ничего, сориентировались. Дошли до ближайшего аэродрома, выдерживая скорость по GPS, а изменение маршрута объяснили «санитарной необходимостью» (хорошая формулировка, емкая!).

Лучше летать много, чем 1–2 раза в год. Пожалуй, пару раз в год — это самое опасное. Либо не летай, либо летай постоянно. Да, вероятность попасть в веселую ситуацию — выше, но и опыт здесь — бесценный помощник.

То же и про управление. Занимаясь чем-то профессионально — нельзя делать это время от времени.

7. Ищи наставников и оставляй следы

Так уж получилось, что Су-29, который пригнали из-за границы, два года пылился в ангаре. Самолет сложный, суровый. Летать на нем стало некому, однако с учителями везло. Познакомился с Михаилом Переверзевым, мастером спорта России международного класса, многократным призером чемпионатов России, Европы и мира... Короче, крутым парнем.

Сколько было скепсиса у него в глазах, когда я обратился к нему с просьбой о тренировках: «На таких самолетах только мастера спорта раньше летать могли, и то не все... Это не учебная машина. Эх, времена!» Тем не менее, желание вновь полетать на пилотажнике пересилило, и Михаил согласился. Каково же было наше с ним удивление, когда на третьем часу тренировок мы поняли, что я готов к самостоятельному вылету. На четвертом часу полетел сам, в сложных метеоусловиях. Самолет очень живой, и я даже на 10% не освоил его потенциал.

Помогли увидеть ошибки недавно установленные дымы. Когда за тобой дымный след — сразу видно косяки. В кабине газовая камера, жарко, пот разъедает глаза, а солнышко спит, но ничего: назвался пилотом — терпи и тренируйся.

Учить и тренировать вас никому неинтересно. И уж точно — никто не обязан вами заниматься. Это время, нервы, и в конце концов — риск. Если вдруг вас учат чему-то на работе и в рабочее время (управлению проектами, дизайну, программированию, soft skills), и вам это идет на пользу — относитесь к этому с благодарностью. А не как к само собой разумеющейся херне. В жизни очень сложно найти хороших учителей и уж тем более заинтересовать/убедить их заниматься вами. Большая удача.

1.10 ДЛЯ ТРЕНИРОВКИ

Итак, мы посмотрели картину мира digital-проекта, убедились, что для нас там полно работы и посмотрели, как эту работу делегировать (не поубивав друг друга и с возрастающими шансами на успех).

1. Какие еще объекты вы бы добавили в картину мира digital-проекта? Что вы с ними делаете? Как они связаны друг с другом?
2. Понаблюдайте в течение недели на какие операции и сколько вы тратите времени и энергии. Нравится ли вам это делать, или это рутина? Вы обязаны ее делать, или можно ее кому-то делегировать? Составьте план делегирования рутины. Приводите его в действие.
3. В течение недели попробуйте делать постановки четко по SMART, а письменные постановки с использованием принципа «пирамиды». Проверяйте их через сервис glvrd.ru.

ЛИТЕРАТУРА



Андрей Курпатов, «Красная Таблетка».
Максим Ильяхов, «Пиши, сокращай».
Максим Ильяхов, «Новые правила деловой переписки».
Старое интервью Стива Джобса о командной работе
(ссылка спрятана в QR-коде слева).

ПОЯСНЕНИЯ

Тикет-система — система онлайн-постановки и контроля задач.

Тикеты — собственно, задачи.

Бэклог — инструмент Скрама, по-простому — это список всех хотелок проекта, отсортированный в порядке приоритетов.

NDA — Non-disclosure agreement, соглашение о неразглашении: юридический договор, между сторонами с ограничением доступа третьим лицам к каким-либо данным и/или результату работ.

Софт — программное обеспечение, или ПО.

Репозиторий — место, где хранится код и вся информация о его изменениях. Репозитории могут находиться на компьютере разработчика, на компьютерах его коллег и на удаленном сервере.

Гайдлайн — руководство по правильному использованию визуальных элементов на сайте.

Тест-план — документ, где описывается весь объем работ по тестированию проекта: как, что и когда проверять. В идеале, тест-план пишет опытный тестировщик, а выполнить тест-план способна обезьянка с мышкой.

API — описание способов, которыми один сервис или программа может взаимодействовать с другим сервисом, программой или сайтом.

Спринт — временной интервал, обычно в 2–4 недели, в течение которого scrum-команда выполняет заданный объем работы.

Рисерч — (**en: research**) исследование: поиск и сбор необходимой для выполнения задачи информации, проба новой технологии и так далее.

Ретроспектива — собрание команды проекта, чтобы проанализировать прошедший спринт и создать план улучшений для следующего спринта.

Деплой — развертывание кода сайта на сервере.

Боевой сервер — обычно весь программный код пишется на тестовом сервере в студии, а после переносится на боевой, рабочий сервер заказчика.

Артефакт — результат какого-то процесса: в разработке — файлы исходного кода или файлы данных, в управлении — поставленные в тикет-системе задачи.

Пересаживание обезьян — когда подчиненные свешивают своих обезьян, свои проблемы на своего босса, а тот покорно их принимает, потому что «без вас никак не разобраться» или «вам нужно подписать», или «давайте подумаем, что можно сделать» (см. «Одноминутный менеджер и обезьяны», Бланшар К., Онкен-мл. У., Берроуз Х.).

Стендап — короткая планерка менеджера с командой проекта на 10–15 минут, где обсуждаются три главных вопроса: что было сделано вчера, что будет делаться сегодня и какие есть проблемы.

Декомпозиция — разбивка большого и неповоротливого проекта по аккуратным и понятным блокам. Об этом подробнее поговорим в главе 5, посвященной декомпозированию.

Скрипт — это последовательность действий, описанных с помощью скриптового языка программирования.

Фреймворк или, по-русски, каркас — программное обеспечение, которое облегчает разработку. С ним код пишется быстрее, а в дальнейшем его проще поддерживать.

Стэк — в разработке: набор инструментов для создания проектов, который включает языки программирования, фреймворки, системы управления базами данных, системы управления контентом (CMS) и прочие используемые технологии.

Бета-тестер — почти настоящий пользователь, который интенсивно использует почти готовую версию продукта/сайта/приложения, чтобы выявить максимальное число ошибок для их последующего устранения перед окончательным выходом (релизом).

Баг — ошибка на сайте, в сервисе или в приложении.

Баг-лист — список всех найденных тестировщиком багов, который используется разработчиками для того, чтобы эти баги пофиксить (отловить и починить).

Класс — шаблон кода, по которому создается какой-то объект.

Код-ревью — процесс ревизии кода одного программиста другим, более опытным. Цель: улучшить качество кода и обеспечить соблюдение принятых в команде стандартов.

ЧАСТЬ 2



ТРЕБОВАТЕЛЬНОСТЬ: КАК РАЗВИВАТЬ ЕЕ У СЕБЯ И СВОИХ СОТРУДНИКОВ

2.1 СКОЛЬКО ПРОГРАММИСТОВ НУЖНО, ЧТОБЫ ВЫКОПАТЬ ЯМУ?

Давайте решим такую задачу (гротескная, но так будет веселее).

Допустим, у вас есть сотрудник, и вам по каким-то причинам нужно, чтобы он выкопал яму. Все необходимые параметры, в рамках разумного, придумайте самостоятельно.

Пока не читайте дальше, потратьте минутку для того, чтобы сформулировать (лучше вслух), как вы будете такую задачу ставить. Я подожду.

Сформулировали? Если вы использовали, например, смарт-подход и умеете развешивать контрольные точки, у вас могло получиться что-то типа: «Вася, нужно выкопать яму 30х30х30 см для захоронения офисного котика. А то он сдох, и нам тут скоро всем будет очень душно. Вон в том углу забора. Сегодня до обеда. Лопату возьми в сарае, вот тебе ключ. Справишься? А есть шанс, что не справишься?»

Теперь усугубим. Допустим, вы руководитель диджитал-проектов, а Вася — программист. Вы произносите похожую мантру (мол, выкопай яму, очень надо). Он смотрит на вас честными глазами. И говорит: «Не-хо-чу!» Ваши действия?

Первая естественная реакция: «Ну что за бред! Почему я должен уговаривать программиста заниматься непрограммистскими делами?» Потом идут попытки:

- надавить на Васю трудовыми обязанностями (облом, Вася знает, что его дело — код писать, а не ямы копать);

- ▶ уболтать на личном уровне: либо по-дружески, либо девушки-ными ходами, строя глазки (облом, Вася — интроверт, идеалист и женатик);
- ▶ подкупить премией или даже напугать штрафами (не прошло, у программистов все нормально с зарплатами, а с угрозами штрафов идите вы знаете куда? — правильно, в трудовую инспекцию);
- ▶ или сослаться, что это дебильное задание дало вышестоящее руководство (совсем гнилой ход, особенно с точки зрения морали — фу так делать! — и Вася вас раскусит).

Самые чуткие, внимательные и креативные руководители проектов (а таких попадается 5–7%) отреагируют на Васиного «Не-хо-чу!». Попробуют выяснить, чего хочет Вася, что ему интересно, и связать свое задание с этим интересом. Такая гибкость ума и чуткость воистину достойна уважения.

Так о чем эта задача? И какое отношение она имеет к реальной жизни? Ведь бред же сивой кобылы — просить программистов ямы копать. Тем более, не имея на это внутреннего морального права.

Теперь представьте, что вы приходите к программисту, вам срочно надо кнопку на сайте перекрасить. Из красной в зеленую. Потому что клиент рвет и мечет (мантры, вроде «URGENT! IMPORTANT! ВЫ НЕ КЛИЕНТООРИЕНТИРОВАННЫЕ» и т. д.). Никак свою задачу не мотивируя. Клиент имеет, наверное, право ничего не объяснять за свои деньги, но это не точно.

А программист Вася вместо того, чтобы за две минуты поправить CSS-файлик, да еще и варианты вам показать с экрана в **Developer Tools**, начинает артачиться: «Не-хо-чу! Дизайн — это дело не программистское! Вот принесете мне отрисованный макет, утвержденный клиентом, я и поправлю. А то уже пять раз за день туда-сюда-обратно. Достали, менеджеры, блин, работать спокойно не дают».

И в чем-то есть его правда. Но для вас он ведет себя как говнюк. А отфутболить неопытного руководителя проектов фразами «задача не моя» или «предоставьте мне то, это, и еще вон-то, поменяйте кресло, поставьте новый комп и монитор побольше, сделайте нормальные постановки, и тогда я, может быть, соизволю что-то там запрограммировать» — Вася может практически по любому поводу, по любой задаче. С этим все сталкиваются. Мне особенно везло на специалистов 1С на стороне клиентов, на которых смотрят как на священную корову. А если он, ко

всему, по совместительству родственник директора, заставить сделать его что-то полезное для проекта, да еще и в срок — тот еще квест.

Ключевых вопроса тут два:

1. Где та грань, когда вы требуете от программиста какого-то бреда (вроде выкапывания ямы), а где он просто говнит, чтобы вы отстали?
2. На что опираться руководителю проекта, чтобы добиться нужного ему результата?

Начнем со второго вопроса. Требовательности.

2.2 ТРЕБОВАТЕЛЬНОСТЬ

Требовательность — базовое качество руководителя, в том числе — и руководителя digital-проектов. Это умение настоять на своем, отстоять интересы компании, команды, проекта и свои, если на эти интересы покушается вселенная.

В идеале управление digital-проектом сводится к расстановке приоритетов: это делаем в первую очередь, это — во вторую. Команда весело разбирает тикеты и дружно бежит их выполнять. Желательно автономно: сами подумали, сами быстро и качественно сделали. Вас же на всякую фигню не отвлекают. А только радуют результатами.

Размечтался. Гораздо чаще можно встретить сопротивление требованиям, затупы, а иногда — скрытый или даже явный саботаж.

Требовательность плотно связана с понятиями «власть» и «эксплуатация». Про источники власти много любопытного у Макиавелли, Ницше и других великих умов мира сего. Власть божественная или государственная, бла-бла-бла... — нам так круто не надо. Для управления digital-проектами интересна власть, которая устанавливается внутри команды проекта и разделяет людей на тех, кто командует, и тех, кто подчиняется.

Требовательность — токсична, и в больших количествах — яд (вплоть до психосоматики). Давят сроки, обстоятельства, клиенты, объем параллельных проектов. И малейшее сопротивление требованиям или уточняющий вопрос команды исполнителей вызывает у перегруженного руководителя раздражение. Остаться при этом еще и в позитивном настроении — сложно.

Я знаю целый класс руководителей уровня «директор по маркетингу», бегающих с конференции на конференцию, с выставки на выставку (в промежутках — в отпуск), забегающих в офис поработать на пару дней и тут же сливающихся на следующую выставку — лишь бы не заниматься управлением.

Требовать мягко и тактично они не умеют. На внятное распоряжение способны только в состоянии аффекта, если их сильно разозлить. Надо ли говорить, что такое поведение деструктивно не только для самого руководителя, но и для его рабочей группы?

Приходится учиться требовать. А в книгах по менеджменту тема табуирована. Принято писать о мотивации, командном духе, самоорганизации и бирюзовых компаниях. Вести дискуссии о том, нужен ли вообще менеджер на проекте. Искать всевозможные поводы как можно дальше дистанцироваться от этого геморроя. При этом забывается, что в западных компаниях субординация, бизнес-процессы, требовательность и исполнительность нормально работают по умолчанию. Никто в голос не ругается, не орет, но если пригласили на разговор за закрытыми дверями — все прекрасно понимают, что дело дрянь. Мотивационные плюшки и самоорганизация — это уже тюнинг.

Забавно выглядит, когда руководитель, начитавшись книг по мотивации и самоорганизации или вернувшись с семинара, вдохновенно решает убрать вот эту самую токсичность управления от себя по-дальше. Прибегает в компанию и с криками «Ага! Мы с завтрашнего дня — бирюзовые!» пытается перевалить «головняк» на команду. А потом расстраивается, что не работает. И почему-то я не удивлен. Делегировать сначала научиться!

Уверен, если бы в таск-трекерах была кнопка «Долбануть током» — она бы пользовалась популярностью. Пока такой кнопки нет, ею должны уметь становиться вы сами.

Самоорганизующимся командам нужен самоорганизатор. Уметь планировать, требовать, настаивать, при этом соизмеряя необходимую и достаточную силу. Хотя говорить про требовательность не принято: сочтут упырем, эксплуататором и фашистом.

В долгосрочной перспективе для блага проекта, организации и ее обитателей сильная власть и требовательность предпочтительнее хаоса и анархии. Известный афоризм Гарри Трумэна, 34-го президента

США: «Любое действительно эффективное управление на поверку оказывается диктатурой». При этом никто не против, чтобы все были довольны, счастливы и мотивированы. Как такового противоречия здесь нет.

Самоорганизующимся командам нужен самоорганизатор.

2.3 МЕХАНИЗМ ВЛАСТИ

Винтовка рождает власть.

Мао Цзэдун

Механизм власти до ужаса прост: выполняй мою волю, а иначе наступят некоторые (по умолчанию — хреновые) последствия. В основе лежат **воля** и **угроза** (явная или неявная). С одной стороны, чертовски обидно (от отвращения до бунта, когда понимаешь, как и когда этот механизм применяют лично к тебе). С другой — факт, что все хорошее в этом мире сделано с использованием механизма власти. С третьей — никуда не денешься.

Наша задача — научиться применять этот механизм профессионально.

2.3.1 ПОЛЕ ВЛАСТИ

У Александра Фридмана вычитал интересную аналогию с полем власти: это вроде электричества, к которому подключаются управленческие инструменты типа делегирования, контроля, мотивации и так далее. Если поле власти сконфигурировано правильно — инструменты работают адекватно. Аналогично с электричеством: параметры тока в розетке верные — устройства работают нормально. Если нет — устройства глючат или даже сгорают.

Всю власть в компании возьмем за 100%, поделенных между сотрудниками. Причем, зачастую это распределение образовывается стихийно. По историческим причинам. И в результате взаимодействия сотрудников власть перераспределяется (у одного уменьшается, у другого увеличивается). Это архиважный тезис — **вы что-то с кем-**

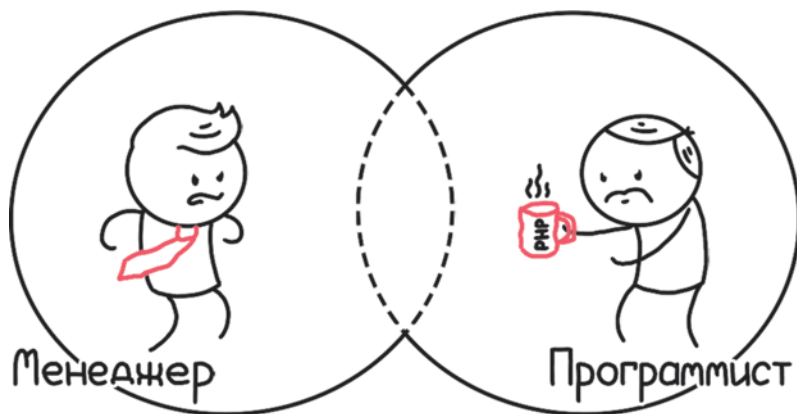
то делаете (или не делаете), а в результате у вас и у него меняется реальная власть.

Рекурсия: источник власти — в ее грамотном применении. Поэтому молодым руководителям проектов важно сразу начать применять ту небольшую власть, которая у них есть, для тех дел, для которых она годится.

2.3.2 САМОЗАХВАТ

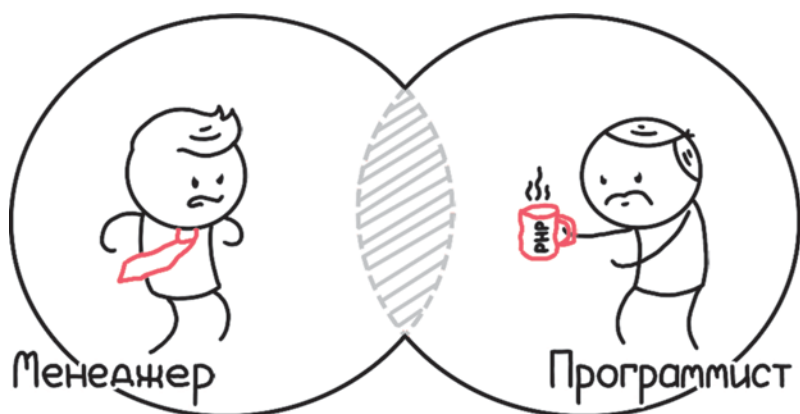
Давайте посмотрим на поле власти на простой модели.

Вот компания состоит всего из 2-х человек — менеджера и программиста — и между ними как-то распределилась власть. У кого власти должно быть больше?



Менеджеры тут же скажут: «У менеджера!», а программисты на это ничего не скажут, но про себя хмыкнут: «Ну-ну, давай, покажи, что ты там из себя за менеджер». И можете не удивляться, если программисты считают менеджеров обслуживающим персоналом. Причем, назойливым, глупым, мешающим работать и думать об **архитектуре**, ставящим неадекватные задачи и вечно лезущим со всякой несущественной фигней.

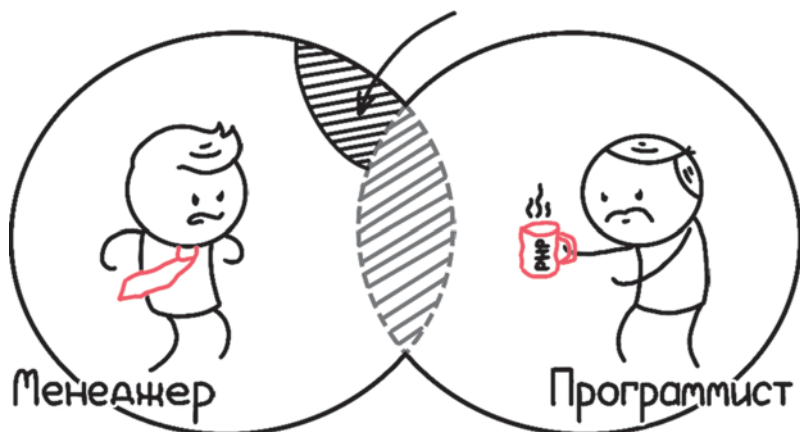
Таким образом, в поле власти может возникнуть красная зона, где люди по-разному понимают круг своих обязанностей, свою зону ответственности и свои права.



Плохо, если сотрудники в компании не знают, как именно власть распределена; плохо, если знают по-разному. Такая мышиная возня отнимает у компании много энергии. В таких случаях вопрос решается повышением компетенций руководителей (нужны энергичные живчики), введением хороших корпоративных правил и наказаний (вплоть до замены нелояльных сотрудников). То есть, долго и дорого.

Вариантов потерять власть — много. Например, самозахват.

Допустим, у вас есть правило: «Получил задачу — оцени». Программист может сначала, как бы случайно, не оценивать задачи (типа, забыл). Если на это не отреагируют (ну сделал, и ладно) — не оценивать специально. Если попросят оценки — сказать: «Не знаю, не могу». А потом и вовсе поскандальить и отказаться оценивать задачи: «Вы, менеджеры, ни черта не понимаете в природе программирования, никаких оценок там дать нельзя...» Если менеджер не отреагирует — сформируется право обычая «Васе можно». И все: власть менеджера уменьшилась, а программист отвоевал себе свободу не думать перед началом работы и не отвечать за сроки.



Другой пример: одна моя сотрудница отвечала за организацию пятничного дизайн-баттла. Каждую пятницу ей нужно было придумать какую-то тему, собрать вечером дизайнеров на 1 час, в рабочее время. Дизайнеры ровно час рисовали по макету. Дальше нужно было организовать публичную презентацию (каждый дизайнер защищал свою работу), и можно было проголосовать за какой-то из макетов, выбрав победителя.

С одной стороны, это прокачивало навыки презентации, ускоряло дизайнеров и просто было весело. С другой, за долгое время проведения таких баттлов пошла профанация: часто работы не имели художественной ценности, а выигрывала самая укуренная. И вот в одну из пятниц я узнаю, что баттла не будет. «Как так?» — спрашиваю я. Ответ был таким: «А они отказались сегодня. Говорят, каждую неделю — слишком часто, давайте через неделю хотя бы». Тут много чего было нарушено, но острее всего — самозахват: дизайнеры захватили право решать, будет баттл или нет, а также, чем они будут заниматься в рабочее время по пятницам.

За самозахват бьют, и больно.

Чаще всего менеджеров продавливают, лишая прав:

- ▶ получать оценку оставшейся работы — от этого и программисты, и дизайнеры будут увиливать под любым соусом: мол, я не гадалка и не знаю, сколько времени это займет;
- ▶ выбирать инструменты и методы работы — проще говоря, технологии;
- ▶ выбирать, использовать ли готовые модули или писать с нуля;
- ▶ оценивать выполненную работу.

2.3.3 КАК БЫ ШУТКА

Допустим, я программист и сделал какую-то противозаконную гадость. Например, отказался оценивать задачи. Наотрез, категорически, навсегда. И менеджер-слабак это покорно принял.

Но, скорее всего, это не за один день произошло. До этого я готовил почву: всерьез обсуждал абсурдность тратить время на оценку задач. И никто меня в этом не остановил.

А до этого я прощупал почву — пошутил про глупость оценки задач, все посмеялись, и никто меня не остановил. Слова, сказанные как бы

в шутку нужны, чтобы прощупать границу допустимого. Чуть-чуть наступаем на чужую территорию, кончиком носочка, но готовы в любой момент ногу одернуть.

А до шутки я серьезно думал, что хорошо бы было избавиться от оценок, и никто меня в этих мыслях не остановил.

А еще что-то было до того, как я про это подумал.

И чем дальше заходило дело, тем больше усилий нужно было, чтобы меня остановить.

Самозахват (да и любое нарушение) спускать с рук нельзя, но отвечать нужно соразмерно. Шутливый самозахват лучше остановить шуткой. Твердой по содержанию, но мягкой по форме. Например, программист как бы в шутку произносит: «А сдается мне, что оценивать задачи — это лишняя трата времени». И смотрит на вас добрыми глазами. Можно так же в тон, шутливо ответить: «А сдается мне, что не оценивать задачи — это попытка увильнуть от ответственности». Верните ему это же без агрессии, но с улыбкой.

2.3.4 ВОЛЯ К ВЛАСТИ

Кстати, вряд ли программисты сидят и серьезно думают «как бы так у менеджера власть оттяпать» — им это неинтересно (и даже безразлично). Но вот как сделать, чтобы:

- ▶ можно было играть с прикольными и новыми технологиями (о которых другие программисты отзываются позитивно — так действует механизм причастности к успешной группе);
- ▶ работать по тем правилам, которые удобны;
- ▶ не отвечать за сроки и ошибки (намеренно использовал слово «ошибки», хотя оно не очень корректно — за ошибки наказывать нельзя).

И несильно себя напрягая? Как-то само собой происходит, что менеджеру навязывается воля программиста, а скрытой или явной угрозой будет его ворчание, токсичное поведение или программистозависимость. Борьба за власть выглядит как рациональная борьба за свободу.

Вторая частая причина потери власти — отказ от ее применения. В силу лени или страха. Применение власти требует поставить свою

шкуру на кон (см. Н. Талеб, «Рискуя собственной шкурой. Скрытая асимметрия повседневной жизни»). Выбирая комфорт, мы выбираем отказ от власти. Или нам не нравится применять власть, потому что мы это делаем плохо. Не умеем, а прокачивать себя — ленимся. В итоге полно менеджеров, которые ни черта не решают. Робкие, пугливые, рукожопые, что-то вечно «передающие» и «выясняющие».

Становиться руководителем пора только тогда, когда у вас появились важные личные цели, а добиться своими руками вы либо не можете, либо это слишком долго. Если со всем этим «управлением» разбираться не хочется, а хочется сидеть в своей норке и ни за что не отвечать — в менеджеры вам рановато. Выбор — либо командовать, либо подчиняться. Третьего не дано.

Кроме самозахвата, лени и страха, менеджер может потерять власть и из-за своих неаккуратных действий. Есть власть реальная, а есть — декларируемая или номинальная. И если менеджер очень сильно увлекается декларируемой властью, пишет какие-то грозные указивки, письма, задачи и требования, которые никто в итоге не исполняет, — его власть уменьшается.

Например, менеджер может потребовать у программиста каких-то новых отчетов, которых раньше не было. С точки зрения программиста — менеджер залез на его территорию. Что произойдет, если программист эти отчеты писать не будет, а менеджер при этом не будет настаивать? Очевидно, что реальная власть менеджера в этот момент уменьшится, а авторитет программиста в коллективе может увеличиться: какой он крутой, отстоял свои права.

Совет

Если вы только начинаете отстаивать свою власть и авторитет в коллективе — для начала давайте сотрудникам небольшие и простые задания с большим запасом времени. Но добивайтесь 100%-го выполнения. Не 98%, а именно 100%. И если все идет хорошо — постепенно увеличивайте нагрузку и сложность задач.

2.3.5 ФУНКЦИИ ВЛАСТИ

Цикл Деминга «планируй-делай-проверяй-воздействуй» (PDCA, Plan-Do-Check-Act) — классическая диаграмма с точки зрения организации грамотной работы, о которой мы уже упоминали в главе 1. Но если дополнить цикл с точки зрения функций власти, получится следующее:

- ▶ планируй — формируй принципы работы и картины мира подчиненных;
- ▶ делай — доводи правила до сотрудников;
- ▶ проверяй — обеспечивай соблюдение правил;
- ▶ воздействуй — награждай и наказывай.

Правда, есть этому циклу и классическая альтернатива, которая встречается в организациях довольно часто.



Функции власти

Планируй

Планирование не терпит суеты, тут нужно действовать постепенно, настойчиво и последовательно. Мы должны продумать в своей голове порядок, по которому другие люди будут делать наш проект или вообще работать. То есть создать принципы, парадигмы, картину мира, правила и задачи, приоритеты.

Делай

Доводите правила до подчиненных: где-то это уместно письменно, декларативно, где-то — устно, в виде обсуждений.

Декларативный способ — это как публикация закона: документ вышел, остается только его прочитать и исполнять. Метод резко снижает затраты на доведение правил. Минусы — не согласные с этими правилами подчиненные могут подвергать письменные документы неправильной трактовке, оглуплять и дискредитировать. Исказить не суть, а дух. Как-то так его прочитать вслух с хитрящей интонацией, что всем кругом станет смешно. Начните с описания проблемы, из-за которой возникло конкретное правило, объясните причины и приведите примеры. Еще один минус деклараций — для честных подчиненных, в случае возникновения вопросов, их будет некому задать.

Альтернатива — беседовать и доводить правила до каждого сотрудника лично. Затратно по ресурсам, хотя, в случае наставничества дает обучаемому больше суперсилы. Как вариант, используйте видео-формат. У меня большая часть регламентов текстовые, но снабжены видео. Так сложнее исказить дух или трактовать по-своему. Но, если возникают вопросы и нет куратора, то и этого формата может быть недостаточно.

Еще один вариант — доводить регламенты на общих собраниях. Техника хороша тем, что можно и ответить на вопросы, и посмотреть на реакцию сотрудников. В **Scrum** это уместно делать на ретроспективах — специальных собраниях, где обсуждаются проблемы проекта, ищутся решения и вырабатываются новые правила.

Минусы: вы работаете с умными людьми, и, когда будете доводить до них регламенты, — встретитесь с сопротивлением, подколками, хитрыми вопросами. Особенно если новые регламенты меняют распределение власти. Готовьтесь к работе с возражениями заранее.

Доведение регламента до пользователей мало чем отличается от процесса продажи. Поэтому умение грамотно продать свое видение будущего, правил работы, регламентов, требований или выполняемых задач — очень важный навык для менеджера и для руководителя проектов.

Проверяй

Наблюдать за ходом работ. Здесь мне нравится понятие «гемба» из японского менеджмента — это то место, где производится ценность. С ним также связан принцип «Тойоты» «Иди и смотри» или, в зависи-

мости от перевода, «Иди сначала в гемба и осмотри!» Вылезь из своего долбанного онлайна, дойди ножками, и посмотри, как на самом деле люди работают.

Если что-то несложно проверить — проверяйте сами, смотрите сами, вникайте и разбирайтесь сами. Покликайте мышкой проект, а не слепо верьте тестировщикам, что там багов нет. Проверьте со своего телефона. Поиграйте с проектом. Ну и смотрите, как люди делают работу на своих рабочих местах. Наблюдайте сами, как люди работают, а не просто слушайте то, что они об этом говорят.

Воздействуй

По итогам контроля производим корректирующие воздействия. Раздаем поощрения и наказания.

К сожалению, без инвестиции вашей личной энергии, без регулярного воздействия и выполнения этих операций, дело протухает. Кто-то должен крутить это колесо. Вялый, угрюмый, депрессивный руководитель убивает команду. И наоборот, позитивный и энергичный способен творить чудеса.

Сама система управления должна периодически пересматриваться: нет ли перегибов, тухляка, можно ли что-то улучшить. Так рождаются самообучающиеся, самоорганизующиеся команды, где роль менеджера — задавать ритм и производить энергию.

2.3.6 ЧТО БУДЕТ, ЕСЛИ НЕТ?

Среди менеджеров-интровертов редко, но встречается паттерн поведения, когда они все общение по проекту переводят в почту или в тикет-систему, а лично с исполнителями никогда не общаются. Причина? Поскольку требовать трудно и токсично, можно столкнуться с сопротивлением — придется отстаивать свою позицию здесь и сейчас, но не у каждого на это хватает «пороху».

Поэтому менеджер открывает свою любимую тикет-систему, пишет там задачи и ждет, когда они будут сделаны. Вообще, тикет-системы — это отличный инструмент, но вот чисто письменный подход работает на очень небольших проектах с низкой долей неопределенности. Чуть проект сложнее, чуть выше неопределенность — без личных обсуждений, без брейнштормов, без споров уже не обойтись.

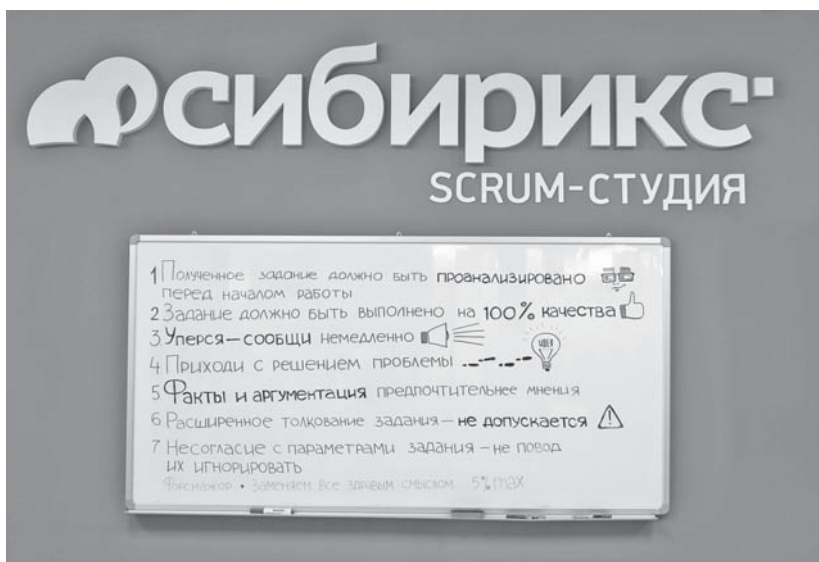
Совет

Отдавая распоряжение на проекте, особенно если оно касается каких-то непопулярных мер, — я рекомендую вспомнить замечательную песенку Семена Слепакова «Что будет, если нет?»

Что будет, если ваше распоряжение или обещание, или угроза не будут выполнены? Как вы будете действовать в этой ситуации? Важно заранее продумать меры, на которые вы лично готовы пойти, чтобы добиться нужного вам результата. Если не уверены, что распоряжение будет выполнено — не надо такие распоряжения отдавать. Вы просто дискредитируете себя, свою власть и свои полномочия.

2.3.7 ЦЕНТРИРУЮЩИЕ ПАРАДИГМЫ ФРИДМАНА

Центрирующие парадигмы — набор принципов, который был предложен Александром Фридманом для снижения управленческих издержек, формирования адекватного поля власти и выработки одинаковых правил игры. У нас они написаны на доске, на самом видном месте. Настоятельно рекомендую прочитать оригинал.



2.3.8 ИСТОЧНИКИ ВЛАСТИ: НА ЧТО ОПИРАТЬСЯ В САМОМ НАЧАЛЕ

Итак. Развитие требовательности — это не самая простая штука на земле. И начинать нужно с личного уровня. С требовательности к самому себе. Учиться не врать себе. Смотреть, где ты ленишься, не дорабатываешь. И заставлять себя доделывать или делать необходимые вещи. Это база.

Дальше нужно постепенно освоить источники власти:

- ▶ Полномочия или роль. Это то, что вам досталось, когда вас назначили менеджером.
- ▶ Право наказывать или награждать. Если вам оно дано; если нет — нужно его получить у своего руководства. Алгоритм можно найти в книге Сивожелезова «Сложные переговоры с подчиненными» — держите ее под рукой.
- ▶ Информация. Если вы знаете больше о проекте, волей-неволей люди к вам прислушиваются, у них формируются ожидания, что вы знаете, как надо действовать, и это дает вам дополнительную власть.
- ▶ Квалификация. Опыт. И результативность. Если вы крутой в каком-то деле — это тоже наделяет вас определенной властью. Авторитетом. Люди будут вас уважать, так как знают, что с вами можно добиться успеха и нужного им результата.
- ▶ Регламенты, стандарты отрасли, центрирующие парадигмы и другие нормирующие документы.
- ▶ Обычаи компании, неписанные правила.
- ▶ Ну и иррациональные вещи, такие как харизма, возраст, тембр голоса. То, что подсознательно воспринимается как «О, это наверное начальник. Крупная шишка». То есть, многие люди охотнее начнут подчиняться здоровому, брутальному, бородатому мужику, который говорит басом и со всех чего-то требует, чем молодому человеку с писклявым голосом, который прячет глаза за дипломом MBA.

Дальше можно найти того, кто явно слабее вас, и попрактиковаться на нем. Чем-то озадачить, посмотреть, как реагирует. Сможете ли вы справиться с его бунтами и закидонами. Если есть под рукой дети — попробуйте заставить их что-нибудь сделать. Например, съесть кашу

вместо того чтобы играть в айпад. Уверяю, это бесценный опыт, после которого вам будет практически все понятно про управление людьми.

Дальше. Если вы вступили в роль менеджера, и ваш руководитель обладает авторитетом, властью — можете смело поначалу опираться на его авторитет. То есть, поначалу отдавать распоряжения, действуя как бы от его имени: «Иван Иванович попросил, чтобы ты сделал кнопку красной. К какому часу это будет готово, скажи, пожалуйста, мне, чтобы я его сориентировал?»

Как только все ваши подчиненные к этому привыкнут — все, отдавайте распоряжения без ссылки на вашего руководителя. С опорой на вашу должность. Вы руководитель проектов, поэтому и требуете.

Далее — опора на привычку подчиняться.

И дальше уже выход на иррациональный уровень. Харизма, требовательный вид, голос, взгляд (жесткий или мягкий), интонация. Вы чего-то требуете, потому что так надо. Никому ничего не объясняя.

Вот примерно такие ступеньки. Вообще, развитие требовательности можно вынести для себя в такой личный отдельный проект. Где-то на полгода-год.

2.3.9 СТУПЕНЬКИ ТРЕБОВАТЕЛЬНОСТИ И МАСТЕРСТВА

Придется ли вам всегда проявлять агрессию? А вот это не факт. Давайте посмотрим, как это развивается.

Уровень «Слабак»

Допустим, вы не готовы идти на конфликт. Окей. Конфликтов будет мало. Но результат будет за ваш счет. Ваших выходных или овертаймов, например.

Уровень «Злоблин»

Допустим, вы заводитесь с полоборота. Чуть что — сразу готовы идти на конфликт. Тоже не очень хорошо, окружающие будут от вас шарахаться.

Уровень «Боец»

Готов конфликтовать ради дела. Если надо подраться — подерусь. В общем, с этого уровня уже можно становиться менеджером. Хотя не все окружающие будут вам сильно рады. При этом руководство тут

осуществляется лично. Лично отдаете распоряжения и лично, фактом своего существования, обеспечиваете, чтобы эти распоряжения выполнялись. Уже неплохо, но трудозатратно.

Уровень «Дипломат»

Умеет не только конфликтовать, но и договариваться. Конфликты редкие, но и готовность идти на них — низкая. Тоже неплохо, но дипломат при прямом столкновении с бойцом, скорее всего, проиграет. Поэтому предпочитает управлять через бюрократию, регламенты. А это не всегда продуктивно.

Уровень «Нормальный руководитель»

Он, в случае чего, — готов идти на конфликт. Поставить негодяев на место. Обеспечить плохим людям проблемы и веселую жизнь. Все про это прекрасно знают. Поэтому на конфликт не нарываюся. И конфликты становятся все реже и реже.

Уровень «Хороший руководитель»

Вот это очень интересно. Хочу — иду на конфликт, хочу — не иду. Могу и так, и так и сам выбираю, когда и что применить. При этом все подчиненные довольны и счастливы — ну, потому что адекватное руководство, хорошая управляемость и высокая результативность.

Ну а дальше выход на **уровень иррационального**. Когда все происходит вроде как само собой.

Заметьте, с опытом необходимость идти на конфликт сначала возрастает, а затем спадает.

Через это, наверное, все руководители проходят. И пока они учатся конфликтовать, обязательно будут жертвы. Будут обиженные. Просто по неопытности. Кого-то слишком сильно отменеджерили — и все, теперь он ваш заклятый враг. Экологичнее тренироваться на тех, кого не жалко. Ну и ни в коем случае не на домашних, родных и близких. Ваша требовательность — это для работы. Домашние тут как бы ни при чем.

2.4 НОРМАЛЬНАЯ ЭКСПЛУАТАЦИЯ И МОЗГОКЛЮЙСТВО

Итак. Слабые руководители не готовы проявлять требовательность. Испытывают дискомфорт или моральные муки из-за того, что им приходится заставлять коллег дело делать. Выражаться это может по-разному: от состояния «верните меня обратно с управленческой должности на линейную» до высказывания: «А я не хочу быть объектом эксплуатации и не хочу эксплуатировать!»

Гадость в том, что вас никто не спрашивает — хотите вы или нет. Вы либо командуете, либо подчиняетесь. Других вариантов не существует.

Мы как руководители заинтересованы, чтобы сотрудник оставался с нами как можно дольше и работал как можно продуктивнее.

Нужно еще и понимать, кем вы управляете. В диджитале это — ай-тишники: дизайнеры и программисты. А они, как правило, люди сытые. Сытыми управлять сложнее, чем голодными — приходится дополнительно напрягаться, ведь рублем их вряд ли замотивируешь (чем мотивировать — так сразу и не скажешь). Это люди умные, часто — с системным мышлением, и их задурить не получится. А вот с их стороны вы постоянно будете испытывать подколки и попытки проверить вашу власть на прочность. Кроме того, каждый из них — личность, каждый требует индивидуального подхода и по-разному реагирует на одни и те же слова и задачи.

Итак, вам, как руководителю digital-проекта, выпала доля иметь дело с наиболее трудно управляемой категорией людей, у которых свое на уме и к которым порой нет физического доступа (удаленка рулит?). Но даже если вы рядом, посмотреть глаза в глаза и поговорить один на один зачастую трудно. Просто потому, что интроверты прячут взгляд и не любят подобного рода разговоров. Короче — сложно.

Задача менеджера — использовать потенциал, интеллект, время и ресурсы этих людей во благо своих проектов, и делать это грамотно. Грамотный руководитель должен понимать интересы сотрудника, его характер, мотивацию, знать его тактико-технические характеристики (хотя бы на уровне **карт компетенций**), уважать его свободу воли. В общем, знать и учитывать особенности при постановке задач и ведении проектов.

ПРОСТО

СЛОЖНО

Голодные

Сытые

Глупые

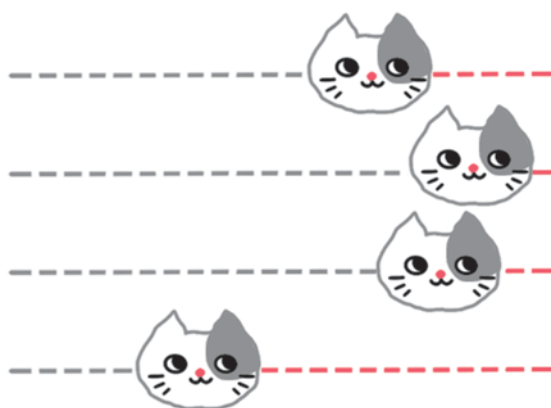
Умные

Одинаковые

Разные

Близкие

Далёкие



Нормальная эксплуатация

- ▶ своевременное планирование;
- ▶ грамотная настройка: постановка задач и целей;
- ▶ мониторинг и контроль;
- ▶ обратная связь;
- ▶ четкие границы допустимого (зеленая, желтая и красная зоны);
- ▶ системное сервисное обслуживание.

Диапазон допустимых режимов эксплуатации часто зависит от целей организации. Есть разница, когда компания работает ради денег здесь и сейчас, или когда она планирует зарабатывать и в будущем. Есть разница в ценностях компании: важно ли, чтобы сотрудники были счастливы, или этот параметр желателен (иногда и вовсе необязателен). Есть разница и в выпускаемом продукте — он рядовой (как туалетная бумага «Обычная») или им хочется гордиться?

Это те параметры, которые в западной литературе задаются в документах с громкими названиями «миссия» или «ценности компании». Сама идея таких документов — неплохая, но глупые шарлатаны ее окончательно испортили.

2.4.1 МОЗГОКЛЮЙСТВО

Наверное, каждый хоть раз ощущал на себе разницу, когда к нему отнеслись требовательно, но справедливо, а когда занимались мозгоклюйством? К сожалению, эту разницу не так-то просто формализовать. Если клиент говорит: «Сделайте мне сайт, как у Apple», как понять, это нормальное требование или мозгоклюйство? Вполне возможно, что нормальное, ведь допустимо, если мы в самом начале плохо представляем конечный результат. Ненормально, если мы не готовы тратить и выделять **достаточное количество ресурсов** исполнителю на поиск решения, зато готовы выделять бесконечное число яда для критики результата и личности исполнителя.

Причем, ресурсы — это не только деньги. Это и время, знания, навыки. Это конкретизация задачи, то есть готовность того человека, который ставил задачу, погрузиться в проект, потратить свое время на брейнштурмы, обсуждения требований и обратную связь. Когда мы требуем от исполнителя «сделай то, не знаю что» или хотим, смутно понимая результат и не обсуждая конкретику, без экспериментов и права на ошибки, чтобы тот сам догадался, каким должен быть результат и как его достичь, — это уже не требовательность, а мозгоклюйство.

Мозгоклюйство — это требовательность без предоставления необходимых конкретному исполнителю ресурсов.

2.4.2 РАБОТА В ПОТОКЕ

Естественное состояние человека — расслабленное. Так задумано природой. Экономим энергию или, как говорит Макс Дорофеев, — мыслотепливо. И, если нас никто не напрягает, ничего не требует — мы на расслабоне: сделать что-нибудь не против, но так, чтобы это было интересненько и не слишком напряжно. Мороженку скушать, киношку посмотреть, новую программку потыкать.

Мозг легко заинтересовывается любой фигней, даже работой. Вы когда-нибудь читали за завтраком этикетки, например, от сока? Зачем? Непонятно, но очень интересно. А если еще и на казахском...

Поэтому задача менеджера — создать такую ситуацию или атмосферу, при которой человеку будет проще начать выполнять дело, чем разгребать последствия — «почему не начал?».

Но опять же. Мы управляем умными, творческими людьми. И минут через 15–30 работы они входят в состояние потока. Это когда ты сконцентрирован, мир вокруг как бы перестает существовать, тебя прет и хочется работать, работать и работать. Это супер-продуктивное состояние, результативность выше в десятки раз.

Помните, когда вы до ночи засиживались за делом или за игрой? Время течет незаметно, а все, что вы делаете, — доставляет удовольствие. Усталость не чувствуется, а если чувствуется — это «добрая» усталость, как после пробежки. Вот в этом блаженном состоянии программисты и дизайнеры кайфуют от работы.

Чтобы попасть в поток, нужно минут 15 или больше сосредоточенно заниматься делом (позже рассмотрим технику Помодоро). Из потока легко вылететь — достаточно, чтобы дернули по мелочи. Сосед задал дурацкий вопрос, например. Вы чертыхаетесь, полчаса продуктивной работы летит в корзину, а с ними и хорошее настроение.

Посчитайте, сколько раз вас нужно дернуть за день, чтобы вы задолбались, стали злым и раздражительным, но ничего толком не сделали? Вот поэтому я запрещаю дергать ребят без экстренных причин (по экстренным все же можно, только для начала мысленно сравните выгоду от «дернуть» и полчаса времени в утиль, а также готовьтесь обосновать экстренность) и прошу вырубить всплывающие уведомления чатиков.

Менеджеры в таком состоянии пребывают редко — нас слишком часто пытаются поюзать (коллеги, клиенты, чатики, почта). Учитесь закрываться. А для этого нужны такие правила игры, когда менеджер тоже может работать в потоке, и дергать его запрещено. Клиентам это, кстати, очень не нравится: нас приучили к мгновенным ответам. Нам кажется, что то, на чем мы сфокусированы в данный момент, и есть «самое важное», и если на наше «самое важное» не реагируют — мы либо теряем его из фокуса, переключаясь на другое «самое важное», либо раздражаемся. Стоит ли говорить, что через пару часов «самое важное» зачастую превращается в пустяк? Предварительный вывод: чатики — зло для продуктивности (про голосовые сообщения вообще молчу).

Чатики — зло для продуктивности.

Ну да ладно. Допустим, вы попали в состояние потока. Знаете свою цель, вам — хорошо, и хочется работать еще и еще.



И здесь появляется риск ухода в перфекционизм. Человек может начать полировать код. Вылизывать пиксели. Долго размышлять об архитектуре. Или по три дня кряду подбирать наилучший оттенок фиолетового. Тут экономика не сходится.

Менеджерская задача — держать человека в этой зеленой зоне графика, где ему интереснее сделать, чем не делать, и при этом вовремя забирать результат. То есть, обеспечить и интерес сотрудника к делу, и рентабельность его работы.

Для этого мы должны грамотно планировать его задачи и следить — интересно ли сотруднику, или он заскучал. Давать обратную связь, отдых, обучение. Словом, сервисное обслуживание. Можно ли тут сэкономить? Угу. Кратковременно добьетесь всплеска результативности, но в итоге получите текучку талантливых кадров, жуткую нагрузку на кадровый отдел и хреновую репутацию. А вместе с этим в долгосрочной перспективе — кратное увеличение себестоимости проектов.

2.4.3 ФОРСАЖ

Можно ли использовать людей в режиме форсажа? Ответ — да, можно. К форсажу можно прибегать, если этого требует дело, и если результат главнее последствий и выгорания людей. При этом вы должны понимать, что усталость, ошибки, демотивация возрастают. Но для понятной и великой цели, которую ЧЕТКО объяснили, и указали сроки, когда форсаж закончится, форсаж допустим и иногда даже полезен. Этакая встряска для организма.

В digital-компаниях рекомендую пару раз в год проводить **хакатоны** или делать внутренние фановые проекты. Но допустимость форсажа — это не повод устраивать в компании вечную «сессию». Никто не призывает эксплуатировать людей на износ. Но, опять же, всякое бывает.

2.5 КОГНИТИВНЫЕ ИСКАЖЕНИЯ У ДИЗАЙНЕРОВ И ПРОГРАММИСТОВ

В человеческой психике есть баги, которые мешают нам объективно воспринимать реальность. Они — причина прокрастинации, желания отморозить уши назло маме и того, что временами люди не понимают друг друга, хотя, вроде бы, не дураки и общаются на одном языке. В психологии причины таких проблем называются когнитивными искажениями.

Как и баги в коде, когнитивные искажения можно исправить. Но для этого нужно понимать, где их искать и какими они бывают.

На странице Википедии в списке когнитивных искажений целых 200 пунктов, и 199 вы, скорее всего, найдете у себя. Рассмотрим те, которые часто встречаются у работников digital-сферы и у людей интеллектуальных и творческих профессий. Зная, как работают когнитивные искажения, вы научитесь понимать, почему подчиненные капризничают, и сможете регулировать их поведение.

2.5.1 СЛИШКОМ ОПТИМИСТИЧНЫЕ ОЦЕНКИ РАБОТЫ

Или, как вариация — пессимистичные оценки, только чтобы от вас отстали. Искажение встречается часто и лечится повторяющимися вопросами. Если человек отвечает не на тот вопрос, что вы задавали,

нормальная практика — повторять вопрос несколько раз, пока вы не добьетесь нужного вам ответа.

— Когда будет готово?

— Ну, мне осталось вот это, это и это доделать, и будет готово.

— Отлично, но когда именно будет готово?

— Ну, тут недолго, обычно...

— Так когда будет готово?

— Через 20 минут.

Еще один вариант — техника «А на какой вопрос ты сейчас отвечаешь?!» Если человек не дает конкретики и пускается в объяснения или изложение последовательности своих действий, просто скажите ему напрямую, что он отвечает не на тот вопрос, который вы задали.

— Когда будет готово?

— Ну, мне осталось вот это, это и это доделать...

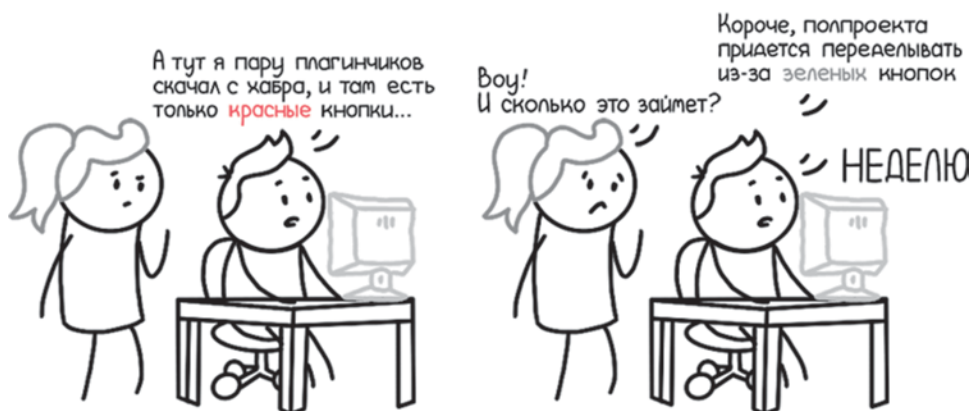
— Ты сейчас на какой вопрос отвечаешь?

— Будет готово через 20 минут.

Кстати, если задаете несколько вопросов письменно (в одном письме или чате) — скорее всего, вам ответят только на самый удобный. Заведите себе привычку нумеровать вопросы, приучите своих орлов нумеровать ответы. Это несложно.

2.5.2 ГЕНЕРАЛИЗАЦИЯ ЧАСТНЫХ СЛУЧАЕВ





Генерализация частных случаев — когнитивное искажение, из-за которого человек расширяет поставленную задачу. При этом он даже не осознает, что ее можно выполнить проще и быстрее.

Генерализация фиксируется варан-менеджментом, который пришел к нам из дикой природы. Существует байка, что варан кусает свою жертву, отравляя ее. Яд действует не сразу, поэтому после укуса варан ходит за жертвой и ждет, когда та умрет. И уже тогда обедает.

Как и варан, менеджер кусает человека задачей. А после садится рядом и внимательно смотрит в его мониторчик. Он может даже не понимать, что делает подчиненный — это необязательно. Главное — быть рядом и следить. И — о чудо! — через непродолжительное время исполнитель поймет, что таки существует более простое и быстрое решение, чем то, которое он придумал сначала.

Варан-менеджмент иногда обоснован, в отличие от двух других животных стилей управления: чайки и дятла.

Чайка-менеджмент — это когда менеджер прилетел, наорал, нагадил и улетел. Такой стиль поведения никогда не решает проблем, зато всегда понятно, кто виноват, а у подчиненных много активной бурной, но чаще всего нерезультативной деятельности.

Дятел-долбоклюй — это ну очень эффективный менеджер: вместо того, чтобы поставить задачку в трекер и назначить конкретный срок, он ходит и каждые пять минут спрашивает сотрудника: «Уже готово?! А когда будет? А сейчас готово? А теперь?!»

Грамотное делегирование — не самая простая задача на земле, но это не повод вести себя как животное :)

2.5.3 ЭТО НЕВОЗМОЖНО!

Среди когнитивных искажений можно выделить целую группу, которые мешают взяться за задачу. И все они сводятся к тому, что человек, чуть поковырявшись в постановках, упирается и заявляет: «Это невозможно!»

Миша, заказчик попросил, чтобы цвет шапки менялся в зависимости от времени суток. Сделашь?



ЧТО ЗА БРЕД?



А с текстами что делать?
А если они с шапкой
сопьются? И где он вообще
такое увидел? Бред какой-то!
Так же не делается!



ЭТО
Невозможно!



У такой реакции несколько причин:

- ▶ исполнителю просто лень;
- ▶ сработал эффект сопротивления, и человек хочет доказать, что он сам волен решать, что делать, а что нет;
- ▶ задача противоречит его чувству прекрасного.

В любом случае менеджеру нельзя верить в то, что задача невыполнима. Наоборот — он должен выяснить, какие ресурсы нужны исполнителю, чтобы ее сделать. Снова и снова долбить подчиненного вопросом: «Расскажи, пожалуйста, что тебе потребуется, чтобы сделать эту задачу?»

Опыт показывает, что со временем подчиненный понимает, что ему не верят, да и действительно задача не так уж и невыполнима... И в конце концов он ее выполняет.

В будущем этот случай можно будет использовать как тыкательную палку в аналогичных ситуациях. На категоричное «Это невозможно!» у вас будет кейс, когда сотрудник тоже считал задачу невыполнимой, а потом сделал ее за 20 минут.

2.5.4 КРИТИКА КАК ЛИЧНОЕ ОСКОРБЛЕНИЕ





Это искажение часто встречается у личностей творческих, особенно невыспавшихся и в плохом настроении. Чтобы вырулить ситуацию в конструктив и никого не обидеть, нужно действовать по следующему алгоритму:

- 1) Отделить хорошее и плохое.
- 2) В работе, пусть ее и нужно переделать, все равно были какие-то положительные моменты — вспомните их и похвалите исполнителя.
- 3) Объясните, в чем ошибка. Возможно, человек не знает, как эту ошибку исправить, и именно поэтому сопротивляется и бунтует.
- 4) Иницилируйте повторное обсуждение или брейншторм по уже пройденным вопросам.
- 5) Попросите помощи коллег или дайте дополнительный ресурс — например, время на рисерч.
- 6) Настаивайте на переделке плохого.
- 7) Закрепите пройденный урок: выясните причину, почему искажение активировалось, и запомните, как вы его пофиксили.

К сожалению, многие начинающие менеджеры в этот момент либо боятся доводить дело до конца, либо ленятся. Но если следовать этому алгоритму — можно выйти на конструктив даже быстрее, чем вы ожидали.

2.5.5 ПРОКЛЯТИЕ ЗНАНИЯ

Вот пример когнитивного искажения, которое называется «проклятие знания». Это когда человек более информированный не может рассмотреть проблему с точки зрения человека, который знает меньше. Отсюда столько непонятых гениев.

Проклятие знания устраняется самодрессировкой. Нужно отлавливать свое нелогичное поведение и наступать себе на хвост. Пытаться выстроить конструктивный диалог, даже если очень не хочется. А то всю жизнь можно прожить, думая, что все вокруг глупые, а ты один в пальто стоишь красивый. А на деле окажется, что все совсем наоборот.





2.5.6 ЭФФЕКТ ГЕНЕРАЦИИ



Не все когнитивные искажения — причина проблем и непонимания. Бывают и полезные. Например, «эффект генерации». Благодаря этому искажению человек лучше запоминает информацию, когда воспроизводит ее сам, а не воспринимает извне.

Это искажение используется в авиации. Например, когда диспетчер общается с пилотом. Если диспетчер на земле передал какую-то

информацию, пилот в самолете должен повторить все параметры, которые были заданы. Менеджерам тоже полезно использовать этот прием, чтобы проверять, правильно ли подчиненные поняли задачу. Называется — «выписать квитанцию».

2.5.7 СЛЕПОЕ ПЯТНО

Еще одно когнитивное искажение: слепое пятно в отношении когнитивных искажений. Человек, который в курсе, что искажения существуют, отрицает их влияние на свое поведение. А последствия списывает на обстоятельства и на глупость окружающих. И, соответственно, не делает ничего, чтобы пофиксить свои когнитивные искажения.

Так что если подчиненный говорит вам, что вы начитались книг по психологии, а задачу «Ну правда невозможно никак выполнить» — может быть, пора отдать задачу другому исполнителю.

2.6 ДЛЯ ТРЕНИРОВКИ

Выработка требовательности и уверенности, если вы не обладаете этими качествами, должна стать для вас приоритетным проектом.

Начинайте с выдачи небольших заданий, но добивайтесь по ним 100% выполнения. Сначала ставьте задачи тем исполнителям, которых вы психологически сильнее. Опирайтесь на власть своего руководителя, правила компании, парадигмы, стандарты отрасли, регламенты. Затем — на здравый смысл и эстетику. Постепенно переходите к иррациональному уровню.

В течение недели следите за собой и за своими коллегами: подмечайте, какие неконкретные слова и выражения и в каких ситуациях они применяют. Когда вам вроде бы что-то ответили, но легче от этого не стало. Иными словами, отлавливайте отмазки: «не знаю», «я подумаю», «скоро» и так далее. Сюда же — три импотентских глагола: попробую, постараюсь, попытаюсь. «Попробую» с русского на менеджерский язык переводится — «отстань, я этого не смогу».

Составьте свой словарь таких слов и натренируйте слух, чтобы, как только эти слова прозвучат в вашем присутствии, у вас рефлекторно появлялось желание ~~дать~~ ~~леща~~ попросить человека, как минимум, конкретизировать эти слова до сроков, цифр и конкретных артефактов.

И постарайтесь не перегнуть палку — уж больно хлопотно исправлять управленческие ошибки.

ЛИТЕРАТУРА

- ▶ Нассим Николас Талеб, «Рискуя собственной шкурой. Скрытая асимметрия повседневной жизни».
- ▶ Александр Фридман, «Вы или хаос. Профессиональное планирование для регулярного менеджмента».
- ▶ Александр Фридман, «Управление мышлением подчиненных: центрирующие парадигмы», аудиокнига.
- ▶ Павел Сивожелезов, «Сложные переговоры с подчиненными».
- ▶ Максим Дорофеев, «Джедайские техники» и «Путь джедая».

ПОЯСНЕНИЯ

Рекурсия — ситуация, когда объект является частью самого себя. Если у вас украли кредитную карту, позвоните по номеру на обороте кредитной карты.

Архитектура — это общее устройство кода сайта или приложения: используемые библиотеки, модули, классы, функции и их отношения. Иными словами, общее описание, внутри которого разработчик пишет код. Вопрос об идеальной архитектуре философский, и многие разработчики стремятся к совершенству архитектуры как йоги к Нирване (зачастую зря тратя время).

Большинство современных промышленных систем реализованы с использованием паттерна проектирования приложения **MVC (Model View Controller)** или его производных. Не все, это не догма, есть и альтернативные варианты, но этот подход чаще всего применяется. Model-View-Controller или MVC, или «Модель-Представление-Контроллер» — предполагает разделение данных приложения, пользовательского интерфейса и управляющей логики на три отдельных компонента:

Модель (Model) предоставляет данные, как правило, лежащие на сервере, например, в базе. И эти данные со временем могут как-то модифицироваться. Например, на сервере хранится информация о товаре.

Представление (View) отвечает за отображение данных модели пользователю, реагируя на изменения модели. То есть, как этот товар показывается на странице.

Контроллер (Controller) интерпретирует действия пользователя, оповещая модель о необходимости изменений.

Чтобы было понятнее, пример. В базе данных у вас лежит товар. Его можно вывести и на страницу с карточкой товара, и на страницу списка товара и в корзине пользователя. Физически в базе это будет один и тот же товар, но отображаться он будет по-разному. За хранение данных о товаре отвечает модель. За то, как этот товар можно показать пользователю — представление, или View, их может быть несколько. Ну и есть некие операции, которые можно выполнять с товаром: положить в корзину, удалить из корзины, удалить из базы, добавить остатки на складе и так далее. За них отвечает контроллер.

Понимание паттерна проектирования может быть важно, когда вы оцениваете проект: расписываете его по экранам и операциям с каждой сущностью данных.

Scrum — передовой фреймворк (платформа), созданный в 90-е специально для разработки, передачи и поддержки сложных продуктов. Сейчас используется и в других сферах. Суть: весь объем работы делится на короткие этапы в 2–4 недели (спринты), в рамках которых выполняется конкретный перечень задач из бэклога (списка всех задач, упорядоченных по приоритетности). Подробнее о Скраме мы поговорим в главе 3.

Карта компетенций — таблица со списком и уровнем необходимых навыков по каждому сотруднику. Благодаря ей можно оптимально распределять людей по командам, следить за прогрессом каждого из них и давать задачи на прокачку недостающих навыков. Подробнее о картах компетенций говорим в главе 7.11. Пример карты компетенций ищите в Приложении 1.

Хакатон — форум для разработчиков, во время которого специалисты из разных областей разработки программного обеспечения сообща решают какую-либо проблему на время.

CSS-файл — каскадная таблица стилей, которая применяется для оформления веб-страниц. С помощью CSS-файла задается цвет, шрифт, положения отдельных блоков на странице.

Developer Tools — панель инструментов веб-разработчика. Обычно встроены по умолчанию в современные браузеры, чтобы можно было легко просматривать исходный код сайта.

ЧАСТЬ 3

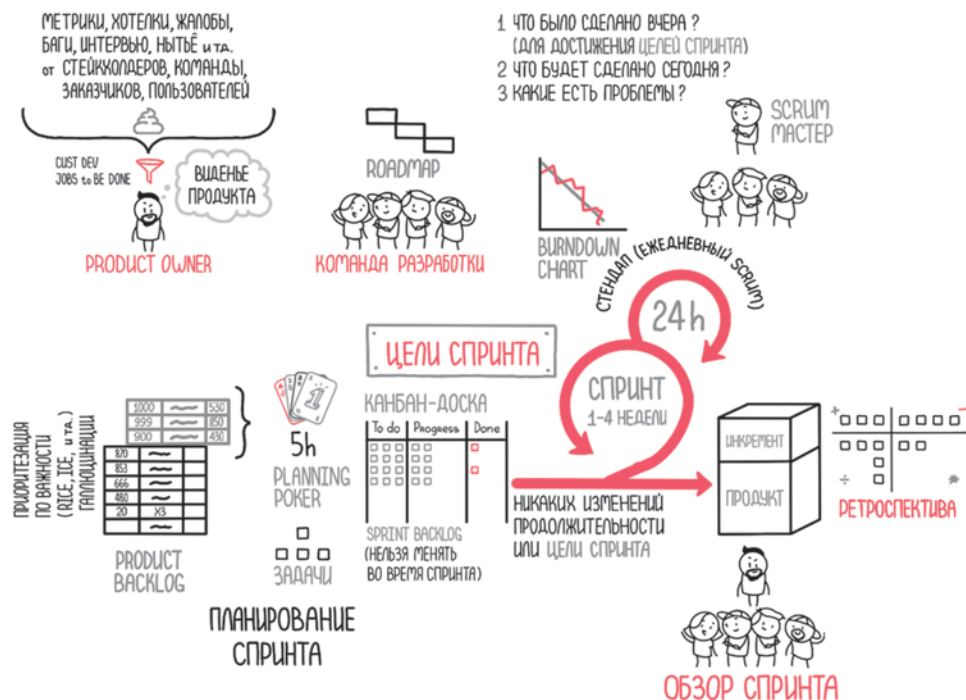


ПЯТИМИНУТНЫЙ SCRUM

Настало время кратко посмотреть на Scrum — фреймворк разработки проектов. «Фреймворк» означает, что в чистом виде, по книге, у вас это вряд ли заработает. Ладно-ладно, заработает, но не с первого раза. Его нужно будет настраивать и сращивать с вашими текущими процессами.

Фреймворк уже зрелый: по нему много литературы, курсов и сертификаций. Поэтому здесь мы поговорим о нем коротко — только в той мере, в которой он нам нужен для повседневной, практической работы. Фреймворк не без косяков, с известной зоной действия и ограничениями. Но пока принципиально лучше ничего не придумали. И Scrum хорошо работает с удаленными командами.

3.1 СХЕМА SCRUM. АРТЕФАКТЫ. РОЛИ. ПРОЦЕДУРЫ



Итак, продукт выпускается поэтапно. Сначала минимальная версия. Затем постепенно наращиваем функциональность.



Каждый цикл разработки называется **Спринтом**. Рекомендованная продолжительность спринтов — от 1 до 4 недель, подбирается экспериментально. Нужен ритм «рывок-отдых». В итоге каждого спринта должна получиться новая версия продукта с приростом функций — **инкрементом**.

Product Owner (Владелец Продукта) отвечает башкой за стратегию и приоритеты разработки. К нему поступают «хотелки» пользователей, метрики, жалобы, баги, идеи стейкхолдеров. Коллективная ответственность в расстановке приоритетов не работает. Но рутину вроде сбора обратной связи можно делегировать.

1000	~	530
999	~	850
900	~	430
870	~	
853	~	
666	~	
480	~	
20	x3	
	~	

В Scrum-е нет нудного толстого технического задания, которое выполняется от корки до корки. Нет попытки предусмотреть все и сразу. Вместо ТЗ хотелки складываются в специальную табличку — **Product Backlog** — и сортируются по приоритетам с точки зрения бизнеса. Можно по ходу работы менять, выкидывать и добавлять функции.

Примерно так это выглядит:

	A	B	C	D
1	Priority	Feature	Description	Est, Days
2	900	Главная страница	Слайдер. Меню разделов "Партнеры", "О компании", "Каталог товаров".	10
3	800	Страница "О компании"	Текстовая страница с инфографикой	2
4	700	Каталог товаров	Список товаров с картинками и основными характеристиками	5
5	600	Карточка товара	Название, текстовое описание, цена, фотогалерея	2
6	500	Добавление товара в корзину	Список товаров в корзине. Возможность менять количество	1
7	450	Страница корзины	С карточки товара	2
8	400	Оплата через банковские карты	Платежная система Yandex.Money	2
9	300	Форма контактов	Карта + форма обратной связи	2
10	220	Раздел "Партнёры"	Список партнёров с логотипами	1
11	200	Поиск по каталогу		1
12	100	Расчет стоимости доставки	Почтой России или слоупоками	2
13	10	Оплата через расчетный счет	Выставление счета на заказанный товар	2
14		Регистрация, авторизация, восстановление пароля		3
15		Личный кабинет		

Простой бэклог в Google Docs

Минимальный состав бэклога: хотелка и приоритет. Можно добавлять свои поля. Например, описание и предварительная трудоемкость. Чаще всего, в условных единицах — Story Point. Берем за 1 балл самую простую хотелку, все остальные оцениваем относительно нее. Бэклоги удобно хранить в Google-таблицах. Можно дать доступ команде и синхронизировать с таск-трекером типа Jira.

Приоритет — чем больше число, тем выше. При ручной расстановке приоритетов удобно делать между ними зазоры (100, 200, 500). Так проще будет вставлять хотелку между двумя другими. Одинаковых приоритетов, по классике, быть не должно.

Чем ниже спускаемся по бэклогу — тем меньше необходима степень проработки. Пустая трата времени формализовывать то, до чего годами не дойдут руки. И наоборот, хотелки вверху списка должны быть ясные, с высокой степенью готовности к работе.

Все заинтересованные лица могут добавлять что-то в бэклог. Но только Product Owner определяет приоритеты. Для этого нужно периодически просматривать бэклог, выкидывать оттуда мусор, обновлять приоритеты и улучшать формулировки.

Есть специальные методики приоритизации, снижающие субъективное мнение (галлюцинации) Product Owner о важности той или иной хотелки. Подробнее о техниках поговорим в главе 4.

RoadMap — предварительный календарный график выпуска релизов. Этого нет в Scrum, но без него картинка проекта теряется.

Версия	Июль 2030	Июль 2031	Июль 2032	Июль 2033	Июль 2034	Июль 2035	Июль 2036	Июль 2037	Июль 2038	Июль 2039	Июль 2040
САЙТ (v1)											
Главная страница, каталог, контакты (v1)											
Личный кабинет (v2)											
Личный кабинет для физических лиц											
САЙТ (v2)											
Корзина, заказ, интеграция с ERP											
Мобильное приложение (v1)											
Личный кабинет (v2)											
Дилеры, оптовики, геймификация											

Как выглядит Roadmap

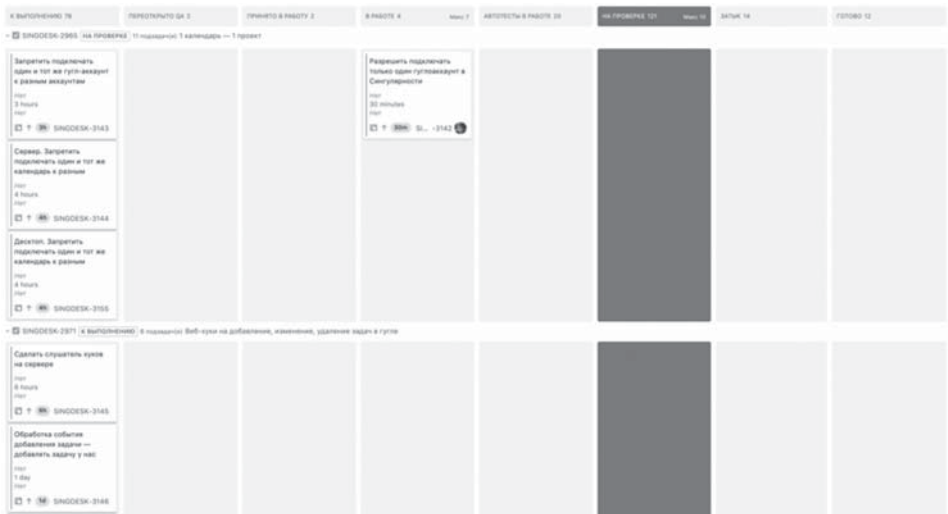


Команда разработки. По умолчанию имеются в виду программисты. Команда забирает верхнюю часть бэклога в работу, дербанит каждую хотелку на технические тикеты и оценивает. Мы используем Planning Poker, о нем чуточку позже. Так формируется бэклог спринта (**Sprint Backlog**) — то, что команда считает реальным сделать за следующий Спринт.

Задачи, попавшие в Sprint Backlog, менять нельзя. В отличие от задач из Product Backlog, в котором можно менять все что угодно до тех пор, пока команда разработки не заберет и не запланирует верхние хотелки.

Обычно **Sprint Backlog** у нас попадает на **канбан-доску** в тикет-системе. Это привычный многим инструмент, где есть карточки с задачами и колонки, соответствующие статусам работы. Как минимум это Plan, In Progress, Check, Done. Чертовски напоминает цикл Деминга.

Можно придумывать свои колонки, добавлять критерии готовности, чек-листы, ограничения одновременно выполняемой работы (WIP, work in progress) — тут все гибко.



Канбан-доска спринта в Jira

Команда обычно работает с этой доской: каждый разработчик забирает интересный ему тикет. Выбирает сам, а не назначает начальство. Пишет код, перемещает карточки по доске, трекает время и так далее. Доска может быть как физической, так и электронной.

Команда — такой мини-спецназ **в тылу врага**. Компактная: рекомендуется не более семи человек. Кросс-функциональная: есть все компетенции, чтобы сделать проект. Самоорганизующаяся (упс!), самообучаемая и ответственная. Большие проекты дробятся на компоненты и распределяются по своим командам.

Ежедневно, в одно и то же время и в одном и том же месте, команда собирается на **Стендап (Daily Scrum)**, где каждый по очереди отвечает на три вопроса:

1. Что было сделано вчера (для достижения целей спринта)?
2. Что будет сделано сегодня?
3. Какие есть проблемы?

Если вдруг случится амнезия, и вы забудете все из этой книги — хотя бы эти три вопроса оставьте при себе. Помогают вернуть управляемость в самом гиблом деле вроде затянувшейся стройки.



Вопросы задает специально обученный человек, **Scrum Master**. Он следит за соблюдением процедур Scrum. Например, может отбиваться от Product Owner, возжелавшего поменять состав спринта прямо по ходу работы. Scrum Master также помогает команде оперативно решить возникающие проблемы.



Как проходят стендапы в «Сибирикс»

Важно модерировать такие встречи, не давать уходить в технические дебри и не говниться. Если есть вопросы, требующие детального разбора, их нужно выносить на отдельные встречи.

Тоскливо, когда на вопрос: «Что было сделано вчера», вам отвечают что-то вроде: «Я размышлял об архитектуре», перечисляют номера тикетов или закидывают техническими деталями. Пальцем, дружок, на экране покажи, что там нового напрограммировалось!



На стендапах удобно повесить перед глазами сформулированную кратко **цель спринта** — помогает сфокусировать команду не на микро-тикетах, а на цели.



И **Burndown Chart** (диаграмму сгорания), по которой видно, успеваем ли мы, или все пошло «через плохо». Эти и другие метрики полезны, но нос держим по ветру и чутко реагируем на то, что говорит команда. Управлять только через метрики не получится.

В конце спринта команда проводит **демонстрацию** — показывает Product Owner и всем заинтересованным прирост функциональности. Тут же получает обратную связь. Иногда жесткую — это, кстати, бодрит. И идет на **ретроспективу**. На ретроспективе все вспоминают, что было хорошего, что плохого, и думают, как улучшить рабочий процесс. По итогам ретроспективы команда может насоздавать себе тикетов на самоулучшение.

Намылить, смыть, повторить.

3.2 ВНЕДРЯЕМ!

Мы внедряли у себя Scrum в начале 2000-х, когда это еще не было мейнстримом, а про Agile не говорил Греф из телевизора. Наломали, конечно, дров, но быстро все исправили. Первое, что внедрили — итерации и стендапы с тремя каверзными вопросами. Скорость и управляемость сначала выросли, а потом упали. Команды начали либо грустить, либо бунтовать. В воздухе витало «ща долбанет». И долбануло бы!

А что не так? Не было ретроспектив. У команд копились вопросы «а зачем все это» и ощущение, что с них выдаивают результат. Этакая вечная сессия.

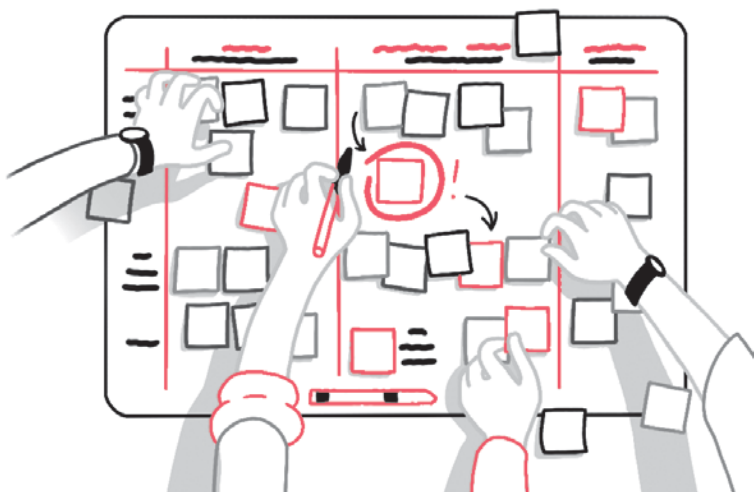
Начали проводить ретроспективы, объяснять, как устроен Scrum. Реально собирать обратную связь от команд. Решать их проблемы. Дали почитать литературу. Обучили Scrum-мастеров внутри команд и сказали, что теперь менеджеру можно говорить слово из трех букв («Цыц!»), если он нахальничает. И начало получаться.

Примерный план действий при внедрении:

1. Дать команде почитать про Scrum, какие есть роли и артефакты, и какие плюсы он дает.
2. Нулевая ретроспектива. Собрать команду. Обсудить ее проблемы. Посмотреть, сможет ли Scrum улучшить ситуацию, обсудить, как именно. Рассказать еще раз весь процесс. Ответить на вопросы. Распределить роли.
3. Организовать итеративную работу (спринты) и стендапы. Запретить менять постановки внутри спринтов. Для начала выбрать недельные циклы. Потом — откорректировать.
4. Приучиться расставлять приоритет, создать бэклог и регулярно его пересматривать. За основу можно взять ТЗ (если оно было) или смету, или просто собрать все хотелки проекта из головы в один файл.
5. Внедрить регулярные ретроспективы и корректировать процесс. Чем более зрелая команда, чем меньше проблем в проекте, тем меньше времени нужно на ретроспективы (если не накопилось фундаментальных косяков, которые непонятно даже с какой стороны решать).

Я бы вообще начал внедрение Scrum-а с ретроспектив. Раз в неделю-две собирал бы команду, обсуждал проблемы и решал их, улучшая рабочий процесс. Глядишь, через месяц весь Scrum нормально заработает.

3.3 ПОДГОТОВКА И ПЛАНИРОВАНИЕ СПРИНТА



Вы примерно понимаете, что именно хотелось бы успеть за спринт. Не факт, что по итогам планирования вы заберете в спринт все эти хотелки. Но предварительное понимание все-таки должно быть.

Очень важно, чтобы к моменту планирования были проработаны постановки задач, проведена промежуточная аналитика, готов весь необходимый дизайн, были подключены все необходимые доступы, ключи к API или описания протоколов и всякое такое.

Бывают задачи или темы, к которым вообще непонятно, с какой стороны подходить. Нужен рисерч. Тут две стратегии: кому-то из команды потратить пару часов на исследование перед спринтом, либо взять задачу на рисерч в спринт, а реализацию делать уже в следующем спринте.

Если проект только-только начинается, там есть инфраструктурные задачи, без которых команда не сможет стартовать: подготовка репозитория, развертка фреймворка, создание базы данных, проектирование архитектуры. Все это стоит наладить немного заранее, за день-два до планирования первого спринта.

Подготовку лучше поручить самому опытному бойцу. Волей-неволей ему придется погрузиться в проект и продумать детали. У него могут возникнуть правильные вопросы, на которые лучше сразу найти ответы. Например, он найдет нестыковку интерфейсов и API. Если это обнаружится в ходе спринта — скорее всего, места для маневров будет мало.

В итоге один из ваших орлов проведет разведку боем, будет знать карту местности. И сможет отвечать на вопросы команды во время планирования. Это сильно помогает и ускоряет планирование.

Опытным путем мы пришли к тому, что нужно дать команде за час-два до планирования прочитать постановки и сформулировать вопросы. Ребята делать этого не любят и читают уклончиво. Логика такая: зачем напрягаться-читать, если на планинге все равно голосом проговорим. Мозг экономит энергию. Но если этого не проделать — планирование спринта может превратиться в многочасовой склочный базар. Приходится заставлять читать и думать. Как тренер в спортзале заставляет сделать пару дополнительных приседаний.

В итоге команда приходит на планирование с предварительной картиной спринта в голове. Осталось сверить картинки, декомпозировать хотелки на тикеты, оценить и понять, сколько задач мы реально успеем за спринт.

Нужно стремиться так декомпозировать задачи, чтобы их легко можно было посмотреть визуально. Это не всегда возможно, особенно в проектах с обилием математики или интеграций. Но для веба и мобильных приложений, в основном, получается.

Размер задач зависит от опыта команды. Мне нравится, когда каждый день можно глазами посмотреть, что поменялось в проекте. Постарайтесь делать декомпозицию, чтобы задачи было просто проверить, а трудоемкость была от 1 до 8 часов (если вы оцениваете в часах, а не в Story Points). Опять же, не всегда получается, да и опытная команда будет сопротивляться такой мелкой разбивке. Опытным нужны крупные куски. Но для молодых и дерзких управляемость сильно возрастает.

Сугубо технические задачи, типа «Удали поле id из таблицы Users» — дрянь. Формулировки лучше делать на уровне фич: «Форма обратной связи» или «Отправка письма о восстановлении пароля».

Долгий планинг, больше полутора часов, говорит о плохой предварительной подготовке, плохих формулировках или вовремя не проведенном рисерче. Или что вы откусили слишком жирный кусок. Пифии предсказали провальный спринт и геморрой.

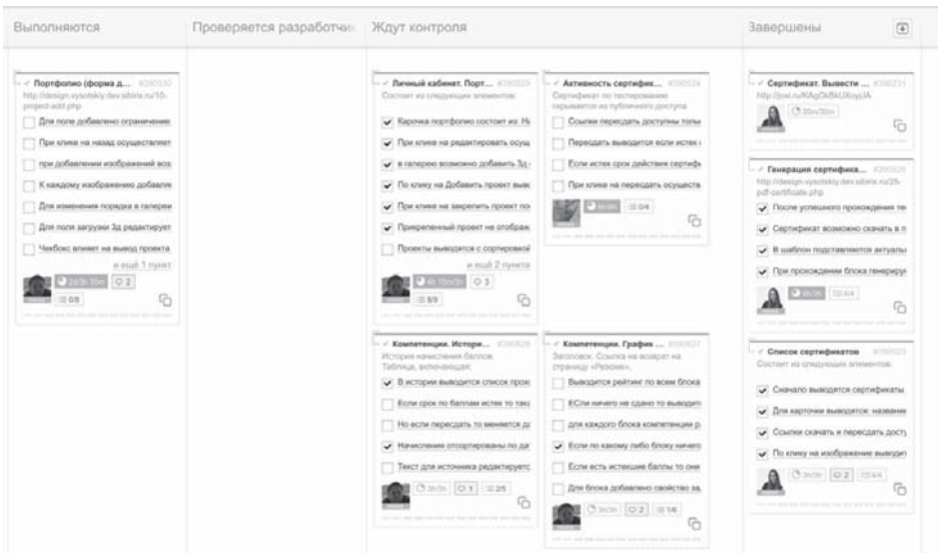
Не старайтесь забить время спринта под завязку. Например, в двухнедельный спринт команды из трех человек засунуть задач на 240 часов. Лучше иметь небольшую подстраховку на код-ревью, рефакторинг, отладку и закон Мерфи. Про него известно, что он точно случится. Сколько взять подстраховки — зависит от опыта команды и задач. Нужно подбирать эмпирически. Начните с 20%: не 240 часов, а 190. Через пару спринтов нащупаете свою реальную производительность.

Такая подстраховка не нужна, если вы работаете в Story Point. Она зашита внутри оценки.

Кроме разработчиков, на планирование я приглашаю тестировщика. Так он будет в контексте и меньше рисков, что под видом «багов» он насыплет команде отсебятины.

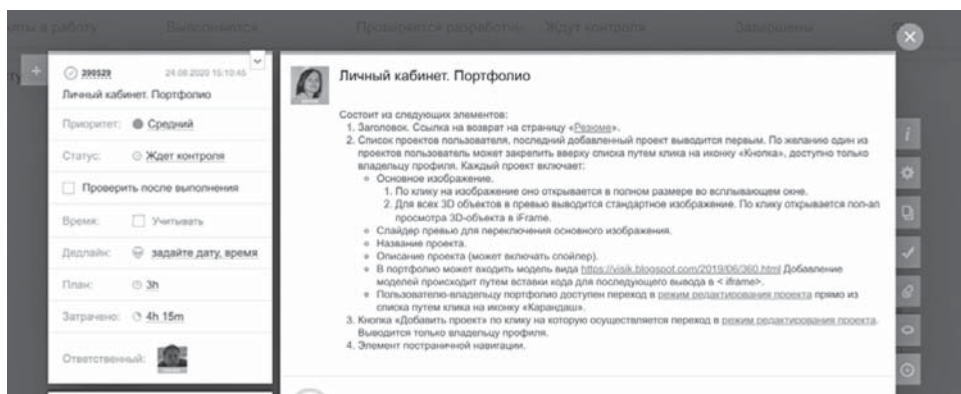
Фиксируем цели спринта. Две-три, не больше. Кратко описываем будущий прирост функциональности. «Реализовать личный кабинет дилера», например. Хорошо бы их вывесить на стене, чтобы спотыкаться о них глазами.

Сразу после планирования я прошу одного из разработчиков и тестировщика еще раз пройти по задачам и составить краткие чек-листы с критериями приемки. Контекст еще очень свеж. По этим чек-листам разработчики смогут сделать самопроверку, да и тестировщику потом будет проще работать. В итоге карточки задач после планирования выглядят примерно так:



Карточки задач в канбан-доске спринта

Внутри каждой карточки уже развернутое описание.



Карточка задачи с детальным описанием

Очевидно, что процедура планирования занимает кучу времени. Поэтому я не люблю короткие спринты. Две недели, на мой взгляд, — в самый раз.

3.4 PLANNING POKER

Planning Poker — инструмент для оценки задач. Карты удобны тем, что участники команд не могут ориентироваться на мнение своих коллег — так оценки получаются максимально очищенными от субъективизма и влияния «авторитетов».

В колоде 4 разноцветных набора для 4 участников. Если участников больше — добавляем колоду.

Достоинства карт: 0 (значит задача готова или слишком мелкая), $\frac{1}{2}$, 1, 2, 3, 5, 8, 13, 20, 40, 100. Бывают карты с числами Фибоначчи или другой развесовкой, но мне нравятся эти.

Цифры — это либо часы, либо Story Point, в зависимости от того, как вы привыкли. Нам не нужно пытаться сделать сверхточную оценку. Это невозможно, скоро будут доказательства. Нам нужна адекватная оценка.

Итак, ведущий планинга зачитывает задачу. Участники выкидывают карты с оценкой. **Рубашкой вверх!** — это важно, иначе самый авторитетный товарищ продавит свои оценки, а робкие тихони отсилятся.

Карты Planning Poker
(planningpoker.ru)



Далее карты вскрываются. Если оценки плюс-минус совпали — фиксируем их в задаче спринта. Если нет — надо обсудить дополнительно.

Например, кто-то выкидывает оценку гораздо большую, чем остальные. Он либо про эту задачу что-то знает-замышляет. Например, был хреновый опыт в прошлый раз или там в коде ТАКООООЕ! И надо его расспросить. Либо спит (разбудим). После обсуждений — уточняем нюансы и переигрываем кон.

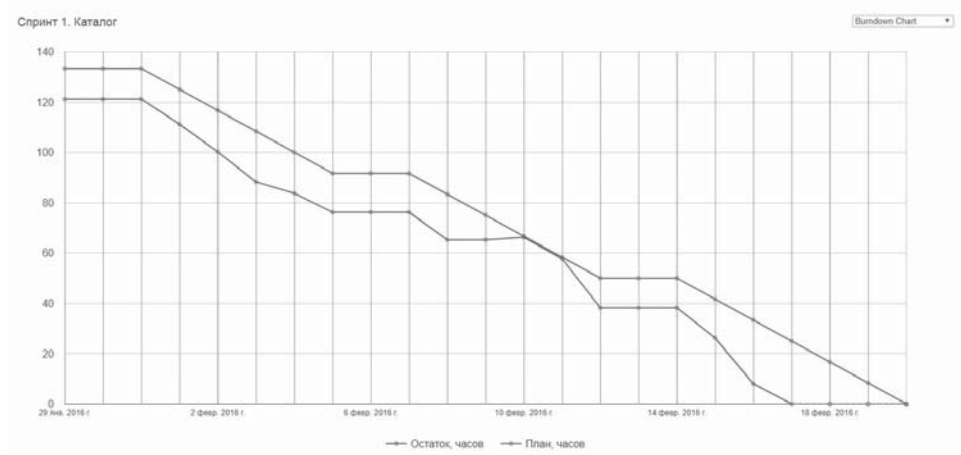
Еще две специальные карты: Coffee/beer — если все затянулось, и надо сделать перерыв, и WTF — для джуниоров, которые вообще ни в чем не уверены.

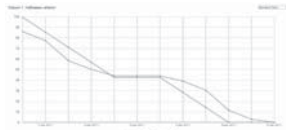
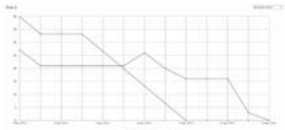
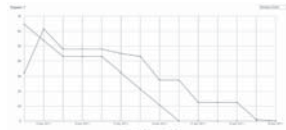
При планировании спринта я предпочитаю физические карты. С удаленными — online-сервис или простой видеочат, куда на «раз-два-три» все скидывают свои оценки задач.

3.5 СТЕНДАПЫ. BURN DOWN CHART

Это простая диаграмма «сгорания» спринта. Ее удобно держать под рукой на стендах.

Нижняя линия — фактически оставшееся время оцененных задач. Верхняя — плановое время. Мы видим, как по ходу спринта «сгорала» работа. Команда доделала все намного раньше плана, однако во второй трети спринта линии совпали. Тут нужно внимание.



 Сначала все шло хорошо. Потом стало плохо.	 Очевиден факап. Внезапно работы стало больше, чем казалось в начале. Так бывает, если по каким-то из задач спринта увеличили оценки.	 Почти сразу все пошло «через плохо».
---	---	--

Управлять только через диаграммы не получится, но такие простые, наглядные метрики держите под рукой. И задавайте каждый день три волшебных вопроса.

3.6 СТРАТЕГИИ ТЕСТИРОВАНИЯ

ТЕСТ В КОНЦЕ СПРИНТА



ТЕСТ В КОНЦЕ КАЖДОГО ДНЯ



Две крайности: отложить тестирование на конец спринта или тестировать каждую закрытую свежевыполненную задачу. Как говорил товарищ Сталин — обе хуже.

Откладывать на потом — плохая идея, однозначно! Вы можете забуксовать в самом конце, и у вас получится сырой спринт. Сдавать стыдно, выбросить жалко. Product Owner будет недоволен. А баги все равно придется фиксить.

Тестировать каждую мелочь сразу — тоже не круто. Во-первых, проверенную задачу могут поломать, когда будут делать какую-то другую. Это называется регрессией (regression bugs). Брукс в «Мифическом человеко-месяце» писал, что это фундаментальная (значит, толком нерешаемая) проблема — исправление одной ошибки с вероятностью 20–50% влечет появление новой. Полное покрытие автотестами стоит неоправданно дорого, да и никто не даст на него времени. Надо дело делать, а не абстракциями заниматься.

Во-вторых, часто отдельный тикет бессмысленно проверять, пока не готовы еще парочка связанных. В-третьих, держать тестировщика «на стреме», чтобы сидел и ждал, когда же наконец на него свалится пара задач на проверку — малопродуктивная затея. В-четвертых, пока проект в руках разработчиков, его постоянно ломают и чинят.

Рекомендую проводить ручное тестирование раз в несколько дней и сделать еще один-два финальных, приемочных теста ближе к концу спринта. Как часто тестировать — решите экспериментально. Начните

с раза в два-четыре дня. Ошибки исправляйте сразу после тестов, пока воспоминания о коде еще свежи.

Вы помните, мы добавили к задачам критерии приемки. Этаким чек-лист. Пусть его один раз прочекает разработчик при переносе задачи на проверку. Для самоконтроля. А затем еще раз проверит тестировщик.

Если команда зрелая, а бюджеты позволяют (для сайтов это почти всегда не так — всем дорого), покрывайте рутинные проверки авто-тестами. Или, как минимум, формализуйте тест-план для критических маршрутов проведением **смоук-тестов** на **продуктиве**. Например, в интернет-магазинах покрываем тестами маршрут Каталог → Карточка товара → Добавление в корзину → Корзина → Чекаут. Критических тестов не должно быть много, и они не должны занимать много времени. Но должны вселять уверенность, что ключевые функции в порядке.

В заказной разработке редко, но встречается клиент, который не видит ценности в тестировании. Речь идет не о манипулятивном приеме, попытках отжать скидки или загнать разработчика под плинтус («Я что, должен за ваши косяки платить?!»). А об искреннем непонимании, как возможно, что после тестирования, тест-кейсов и всех этих довольно дорогих процедур — я, как заказчик, нахожу ошибки. Причем такие, которые кажутся мне очевидными. Они меня бесят! Я не понимаю, на что тратятся время и деньги.

Совет

Немного помогает, если у заказчика есть доступ на чтение баг-листов, и он видит, что команда нашла и исправила несколько сотен ошибок. Но утешение слабое, осадочек остается. Отправлять баг-листы нужно до того, как претензия созреет, а не после. Действовать от силы, а не от слабости. При этом должна быть определенная культура ведения баг-листов: смотрите главу «Правила письменной контрацепции». Однако лучше всего попадать в ожидаемое качество и давать гарантию на свою работу.

3.7 ДЕМОНСТРАЦИЯ ПРОДУКТА. ЗАВЕРШЕНИЕ СПРИНТА

Говорят, в Царской России при испытании нового железнодорожного моста под него ставили всех строителей. А сверху пускали паровоз. Мосты век стояли. А разработчик — либо герой, либо — труп.



Нужно демонстрировать результат работы лично, ставить шкуру на кон! Это бодрит. Подстегивает ответственность и рост качества продукта. Боли, правда, будет больше. Но это полезная боль.

Согласуйте демонстрацию заранее. Если проект долгоиграющий — запланируйте стандартное время, в которое будете показывать спринты. Например, каждый вторник, 16:00. Времени резервируйте около часа. Нужна будет либо личная встреча, либо видеосвязь с демонстрацией экрана.

Со стороны бизнеса присутствует как минимум Product Owner. Пользователи и другие сочувствующие — допускаются. Со стороны разработки — вся команда. Понятно, что бывают исключения, но стремимся к этому.

Для начала я вкратце рассказываю, что успела команда за спринт. Вспоминаем цели спринта. Дальше, с экрана, показываю инкремент. Команда помогает, рассказывая, что, как и почему было сделано. В идеале каждый рассказывает про свой вклад. Разработчики помогают с ответами на технические вопросы и обоснованием решений.

Ради чего все это затевается: обратная связь в режиме реального времени. Пусть жесткая, но честная. По сути, обсуждается только первое впечатление, но оно не врет. WYSIWYG — What You See Is What You Get, или «если тебе кажется, что что-то не так, — скорее всего, тебе не кажется».

По опыту большая часть обратной связи будет конструктивной и позитивной. Особенно с англоговорящими заказчиками. Бояться демонстраций не надо. Надо готовиться. Продумайте чуть заранее:

- ▶ По каким путям проведете зрителей в продукте.
- ▶ Какие кейсы покажете.
- ▶ Какие могут понадобиться данные для демонстрации: контент, тестовые пользователи и так далее.
- ▶ Какие вопросы скорее всего возникнут. И как на них отвечать.

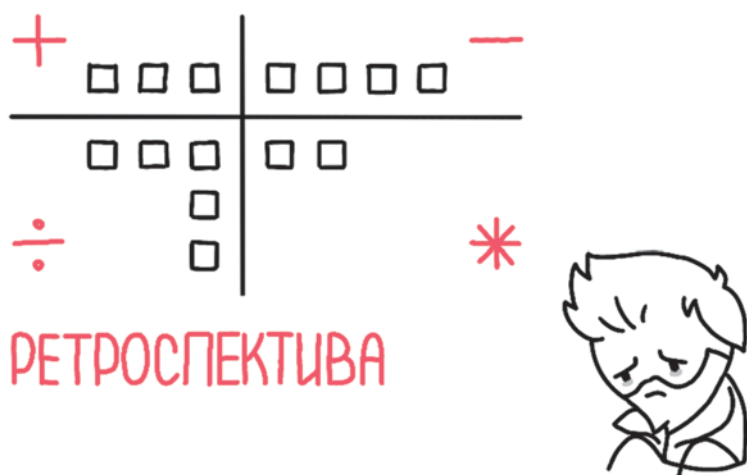
Позднее Product Owner может потребоваться время помедитировать, поразглядывать продукт, поиграться с ним. Но это он будет делать в одиночестве, сублимируя в бэклог. Возникшие вопросы ему будет не с кем обсудить, и, наверное, он будет предполагать только худшее. Но с этой проблемой он должен справиться сам.

Мы же зафиксируем обратную связь и пойдем с ней на ретроспективу.

Аварийное завершение спринта

Редко, но гадко. Бывает, приходится останавливать спринт. Дело идет медленно и не туда. Команда упарывается, нужных доступов нет, кризис у клиента или еще какая-нибудь гадость. Дергаем стоп-кран, останавливаем спринт. Проводим ретроспективу, решаем проблемы. Делаем рескоупинг (перепланируем спринт). Едем дальше. Таких форс-мажоров допустимо не больше 5%. Ибо нефиг.

3.8 РЕТРОСПЕКТИВЫ. БОРОДАЧИ ТОЖЕ ПЛАЧУТ



Допустим, на демонстрации Product Owner разнес вашу работу в пух и прах. Справедливо, методично. Или просто очень эмоционально: он оказался злобным, неуязвимым говнюком. Команда сидит подавленная. Кто-то курит прямо в опенспейсе. Провал. Полный капут. «Что я тут делаю?» — читается в глазах бородатых программистов. Пятница, вечереет. Ваши действия?

Рохля разведет руками. Распустит команду по домам. Ага, щас! Мы пойдем в ближайший бар. И будем всякую фигню думать. Про процессы. Технологии. И ме-е-неджера. Рано его руководить поставили.

Люди будут смаковать неудачу, искать причины провала. На минутку, не в себе, а в руководителе продукта, менеджере, проекте или процессах. Сами выберут такую позицию, когда они — Д’Артаньяны, а остальные — нет. И будут ныть. Пару-тройку раз повторятся такие ситуации, и дело разладится.

Сильный, во-первых, такого не допустит. Во-вторых, если уж подобное случилось, примет огонь на себя. Прикроет команду. И сразу начнет действовать: обсудит с сотрудниками ситуацию, вместе они разберут ошибки, выработают меры на будущее. У команды по итогу сложится чувство, что **меры помогут**.

В скраме есть специальная активность, где мы с командой подводим и обсуждаем итоги спринта. Ретроспектива. Цель ретроспективы — не

понять (хотя иногда хочется), не выпить (хотя кому-то иногда необходимо), не байки потравить. А улучшить рабочие процессы.

Давайте исходить из того, что **намеренно никто не гадит**. Ну ладно, ладно. За редким исключением отбитых чудаков на букву М, которых легко вычислить и отчислить. Но вы серьезно думаете, что программист специально пишет хреновый код? Дизайнер специально рисует дрянной интерфейс? Сложно управлять людьми, которых вы считаете упырями.

Все не так. Им может не хватать ресурсов: времени, мотивации, энергии. В том числе квалификации, чтобы сделать на должном уровне. Вот с этим уже можно работать.

Итак, намеренно никто не гадит. На основе этой идеи вы должны гарантировать безопасность команде на ретроспективах. У людей должна быть уверенность, что:

- ▶ по итогам ретроспективы никого не накажут;
- ▶ на ретроспективе ни на кого не наорут, не затроллят и морально не опустят;
- ▶ за обсуждение проблем не посчитают слабаком (но только на ретроспективе — ежедневных нытиков отправим к мамочке);
- ▶ и так далее.

Придется стать сильным и тактичным, чтобы вскрывать проблемы команды, не вскрывая при этом людей.

3.8.1 ФОРМАТ РЕТРОСПЕКТИВЫ

1. Подготовка

Мы заранее планируем встречу и собираем команду в одной комнате, без посторонних ушей. Сложно, знаете ли, исповедаться на базарной площади. Напомните ребятам, где и когда планируете провести ретроспективу — возможно, они захотят посмотреть свои записи, **коммиты** или еще как-то подготовиться.

На 2-х недельный спринт и команду в 3–4 человека планируем минут 40–60. На месячные спринты или большие команды может уйти часа полтора. Я бы не советовал делать ретроспективы еще длиннее — это контрпродуктивно.

Иногда уместна пицца: душевные разговоры за едой сплавляют команду и снижают уровень агрессии.

Один человек будет ведущим. Как правило — скрам-мастер или менеджер, если он совмещает эти роли.

2. Настройка

Первые пару минут включаем команду в групповой тране работу.

Здороваемся, налаживаем контакт. Например, задайте простой вопрос, типа «Как дела?». И получите хоть какую-то реакцию. Кивок головы или угрюмое: «Расскажи, дружок мой, Вова, отчего мне так хреново», это окей.

Альтернативные техники, которые мне нравятся:

- 1. 10-пальцевый опрос. Попросите всех выбросить на пальцах, насколько довольны текущим спринтом.
- 2. Happiness radar. Чертим матрицу 3x3. По вертикали — смайлики настроения. По горизонтали — Технологии, Люди, Процесс. Каждый ставит палочку, насколько удовлетворен каждым из аспектов. Стикеры нужны для фиксации предложений по ходу.

	People in room	People in sf	Process	Technology
😊	☒			
😐		П		Г
☹		П	☒	С

- 3. Проверка безопасности. Просим также на пальцах выкинуть, насколько каждый себя чувствует сейчас в безопасности.

Напоминаем цель ретроспективы. Если есть новые участники, не привыкшие к ретроспективам — рассказываем про формат и гарантируем безопасность. Рассказываем про «намеренно никто не гадит».

3. Накидывание на вентилятор

Далее просим по кругу рассказать о плюсах (хорошем) и минусах (плохом) на спринте. Начинайте не с новичков. Идеи приветствуются и фиксируются, но не критикуются.

Знаю пару ребят, которые ведут блокнотики и записывают туда всю бяку по ходу спринта. А потом зачитывают по пунктам. Боюсь, что если прочитать вслух блокнотик от корки до корки, — явится ноющий дьявол. Но раз им так проще — пусть пишут.

Модератор должен чувствовать настроение команды. Уметь разговаривать. Уметь слушать. В споры не вступать. Не говнить. Не обещать предлагаемые идеи. Модерировать, если идет неконструктивный треп. Подталкивать к поиску решений. Параллельные потоки (когда параллельно обсуждается парочка-другая тем) и болтовню — закрывать. Вытаскивать тыкающихся в телефон наружу. Стараться выявить те проблемы, в которых команда старается не сознаваться даже сама себе.

Модерируем глупые споры, является ли озвученная проблема проблемой. Или про важность проблем. Вместо споров — просто фиксируем. Может, человеку это важно.

4. Подрывай!

Это не каноническая техника ретроспектив. Но иногда я так делаю.

Я люблю порой посидеть на чужих ретроспективах. Одна из интересных проблем, с которой сталкиваешься в сработавшихся коллективах, — ребята не любят взрывать медленно тлеющие конфликты. Всех бередит, но по-тихому.

Например, между тестировщиком и программистом возникла вялая напряженность из-за того, что тестировщик очень тщательно и придирчиво все проверяет. А программист считает, что это излишние придирки и пиксель-дрючево. Но вслух никто ничего не высказывает. Так, срутся в комментариях к баг-листам, вяло препираются «баг-не-баг», на что улетает вагон времени и нервов. Копится недовольство: друг другом, проектом, менеджером, клиентом или погодой за окном.

Одна из интересных задач модератора — по косвенным признакам найти такой конфликт и вскрыть (взорвать) его. Еще дядька Макаренко завещал.

Впрочем, на ровном месте накалять не надо. И так, знаете ли, хватает!

5. Идеи. Генератор добра

Нам нужны идеи. Что поменять, чтобы стало чуть легче и светлее. На первой стадии годятся любые, самые безумные мысли. Критиковать идеи нельзя. Отсев будет дальше.

Тут важно выделить две фазы, как в брейншторме:

1. обсуждение проблем и генерация идей по устранению проблем;
2. выбор среди тех идей, которые будем реализовывать.

В идеале, на каждый наш минус придумываем две-три идеи по его устранению. Собираем, аккуратно записываем. Могут помочь техники из главы 11 по работе с факапами.

6. Хороший план! Плохой план

Далее из идей выбираем несколько (5 ± 2) конкретных улучшений, которые команда готова сделать. Устройте голосование, если идей целая куча. Желательно успеть за следующий спринт.

Некоторые типы идей я отсеиваю. Или прошу переформулировать.

1. *«Мы работали плохо, теперь давайте работать лучше».*

Спасибо за лозунги. Но я не знаю, как это реализовать.

2. *«Не жрать после 18:00» или «Проводить стендап за 10 минут ровно».*

Богатая идея! Больше похожа на правило.

Если бездумно добавлять правила — место кончится. Или будет вагон правил, которые не блюдут. А это дискредитирует власть. Или это будут правила, которые помнит только их автор. И ненавидит коллег за несоблюдение.

Переформулируйте на локальное и исполнимое: «Не жрать после 18:00 в течение следующего спринта», например. Вот это реалистичнее.

3. *«Давайте будем закладывать больше времени на подготовку и архитектуру».*

Во-первых, это тоже правило. Во-вторых, не люблю, когда добавляются буферы времени.

На это есть Закон Паркинсона: работа занимает все отведенное на нее время. И даже чуть больше. Сколько буферов ни закладывай — будет мало.

Давайте лучше адекватно оценивать задачи и делать дело так быстро, как это возможно, не?

4. *«Давайте проверять после тестировщика — другим тестировщиком».*

Ну уж нет! Чем больше проверяющих, тем хуже качество. Зачем мне стараться, если за мной перепроверят и под носом подотрут? Рассеивается ответственность.

Я хочу «встроенное» качество как можно ближе к центру создания ценности. Я хочу встроенное в программистов качество. И они это могут!

5. *«Пусть менеджер нам кофе приносит. С козинаками. И веером нас обдувает».*

На ретроспективах менеджеру легко наловить себе обезьян на голову. Я видел ретроспективы, где команда навешивала на менеджера-слабака кучу правил, которые должен выполнять только менеджер.

С одной стороны, решение проблем команды и правда потребует менеджерского ресурса. С другой стороны, если после ретроспективы вы выходите с ворохом задач чисто для себя — тут явно какой-то косяк.

Надо помогать команде самостоятельно справляться с проблемами, а не быть еврейской мамочкой. Следите за тенденциями.

6. *«Давайте все к черту перепишем».*

Вот это происходит у меня прямо сейчас, в работающем проекте аптечной сети, где 5000 аптек, первая в стране доставка лекарств online, тысячи пользователей. Просто проекту 4 года, и там нет реактивного фреймворка. А это не так весело и круто. Программисту скучно.

То есть, идея предложена как бы во благо проекта, но чувствуется сильный личный мотив. Почесать ЧСВ, поиграть с технологиями, стать незаменимым и так далее. Словом, почувствовать себя крутым! Я не вижу ничего дурного в таких личных мотивах — они полезны. И без их удовлетворения не будет кайфа на работе.

Но формулировку мы поменяем. «Составить план рефакторинга» — уже теплее. Этот план я внимательно изучу на целесообразность, адекватность прогнозов. Согласую с клиентом и внедрю. Постепенно.

7. *«Пусть дизайнеры всегда теперь делают по-другому».*

Это когда проблемы пытаются решить за счет отсутствующих на ретроспективе людей. Иногда — оправданно. Но! Естественно, отсутствующие будут не согласны.

Или придумываем правила, которые нужно распространить на всю компанию, а не только на одну команду.

Во-первых, такие правила сложно внедрять: нужно сформулировать, донести до пользователей, придумать контроль и наказания и потом постоянно накачивать в них энергию. А где возьмете дополнительную?

Во-вторых, у менеджера нет полномочий решать за всю компанию. Можно **предложить** какое-то изменение руководству. С должным обоснованием и планом внедрения. В такой формулировке задача уже более вкусная. Но это же думать надо!

А можно собрать дизайнеров и разработчиков вместе, и пусть они глаза в глаза расскажут, что думают друг о друге. Взорвать ситуацию. Как-то сразу тональность критики и категоричность уменьшаются. Я не против глобальных изменений, но тут очень аккуратно работать надо. Нежненько.

*8. «Не хочу я быть римскою папой,
А хочу быть владычицей морскою».*

Ну, сорян :)

В план должны попадать конкретные, выполнимые идеи. Можно даже по SMART. С конкретными исполнителями. Следующую ретроспективу мы начнем с проверки, что из этого плана получилось. И стало ли от этого лучше (бывает, сделали, как просили, и стало хуже, ага).

7. Заключительная часть Марлезонского кордебалета

Подводим итоги. Кратко зачитываем план. Спасибо, все свободны.

Можно повторно замерить настроение команды, убедиться, что ребята считают, что меры помогут.

Можно спросить обратную связь на ретроспективу.

Если все недовольны решениями, ретроспективой — вы в беде. Очень жаль. Надо чинить ретроспективу.

3.8.2 ФОРМАТЫ ФИКСАЦИИ

Мы используем три формата ретроспектив.

1) Неформальные. Короткая встреча на 15–30 минут, где по очереди даем высказаться всей команде. Плюсы, минусы, идеи, предложения. Конкретные решения фиксируем, если все с ними согласны — забираем в план работ. Подходит, если на проекте все хорошо. Просто градусник. Артефактов не остается.

2) Формат с доской 2x2: плюсы-минусы, идеи, план. И стикеры.

Самое муторное потом — перенабрать писанину со стикеров и завести реальные задачи в тикет-системе. Но вы взросленькие, как-нибудь справитесь.

3) Электронные шаблоны.

Грамотный шаблон фиксации доступен в [Confluence](#). Мы используем чуть попроще. Главное — вывести его на большой экран, чтобы все видели, как идет фиксация, и что ничего не забыто и не перевернуто.

2020-02-03 Ретроспектива / архитектура для новых деревьев

Создан: Vladimir Zaitsev
Последнее обновление: фев 03, 2020

Одна из задач ретры — рефакторинг дерева. Нужно понять зависимости от этого рефакторинга и план работ

Date	3 февр. 2020 г.
Team	Singularity
Репартиция	@Алексей Динкин @Денис Владимирович
Участники	Господство: @Иван Игоревич Ковкина

Background на поговорить:

- ☒ Импорты. Как их начать делать, не дожидаясь 100 лет рефактинга дерева?
- ☒ Аналитический контор: растут?
- ☒ Можно ли архитектурно сделать единый механизм REST-импортов, чтобы не париться на досчете чуть ли не в прямые запросы к Базу?
 - ☐ Состояние по расположению дат. Кто этим занимается? Какой статус?
 - ☐ Запросы у Лео — Почему у меня так консоль растет долго? Проверять, что именно и как можно оптимизировать поиск.
 - ☒ Запросы у Лео — База весит более 100 метров. Почему?
 - ☐ Добавить батчсет: индикатор создаст много больших запросов
 - ☐ Поисковик в рабочем режиме жрет батарею (зависает). Мы так всем ставили не переопределяется?

Рефакторинг: План первых шагов рефактинга, что бы по минимуму ловить сдвигание (а лучше — вообще не ловить)

- ☒ 1) "Точки" строить по дате
 - ☐ Не трогать все базу сразу, если она не нужна (Как минимизируй контакт с диском и корзины, но не лезть внутрь)
- ☒ Не лезть синхронизации на старых клиентах

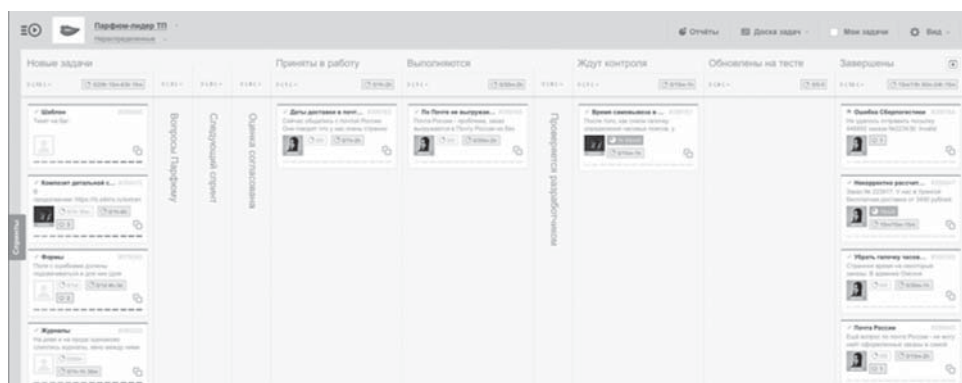
Retrospective

●

Добавить в Jira в виде задач:

3.9 КАНБАН. КОГДА ЛУЧШЕ, ЧЕМ SCRUM

Канбан — легковесный и прозрачный процесс. Задачи по мере поступления вывешиваются на доску, с которой разработчик может их забирать и программировать.



Канбан-доска технической поддержки. Фрагмент.

В отличие от Scrum-а, тут нет спринтов. Канбан больше подходит для контроля текущей техподдержки, маркетинга или обработки лидов. В разработке новых функций и версий лучше работает Scrum.

3.10 КУДА ДЕЛИ МЕНЕДЖЕРА

В теории нет разницы между теорией и практикой. А на практике есть.

Йоги Берра

Допустим, мы создаем продукт in-house. Собрали разработчиков в одной комнате, сказали работать по Скраму. Самоорганизуйтесь там как-нибудь. Получится? Маловероятно. На хакатон дня в три, может, и хватит, а на полноценный продукт уже нет. И канонический скрам-мастер тут вряд ли поможет.

Самоорганизующейся команде нужен самоорганизатор.

Другой пример. Вот идет разработка. Ни шатко ни валко, по ТЗ. Скорость медленная, баги, Product Owner ругается, команда грустит. Решают попробовать Scrum, а вдруг поможет. Опять нужен кто-то, кто сможет все самоорганизовать, сшить старые процессы с новыми, перевести работу на новые рельсы. Это искусство, тут много дел.

Или, допустим, заказная разработка. Product Owner на стороне клиента есть. Как-то на одной из встреч представитель клиента сказал на полном серьезе: «Да, у нас есть Product Owner, их целых 5» (ой!).

Что ожидается со стороны подрядчика? Что будет кто-то ответственный за проект, с кем можно вопросы порешать. А то и вовсе Proxy-Product-Owner нужен, который за заказчика все порешает, всю обратную связь со всех этих пяти Product Owner (корректнее называть их стейкхолдерами) соберет, решит противоречия и в случае чего — по башке получит. П — перспективы...

Менеджер делает все то, что нужно делать, но что не делает никто другой.

Или, как минимум, организует и делегирует, чтобы делалось все то, что почему-то никто не делает.

Но вот команда взрослеет, Scrum уже отлажен, все привыкли друг к другу и процессу, есть внутрикомандный scrum-мастер. В этом случае роль менеджера действительно уменьшается. И во что он трансформируется — вопрос на вырост. Может, в скрам-мастера. А может, в продуктового менеджера, или аналитика, или помощника Product Owner.

Кажется, редкие-редкие команды достигают такого дзена, где менеджер не нужен. Да и не в нашей ментальности работать без начальства. Короче, менеджер необходим и востребован, аллилуйя.

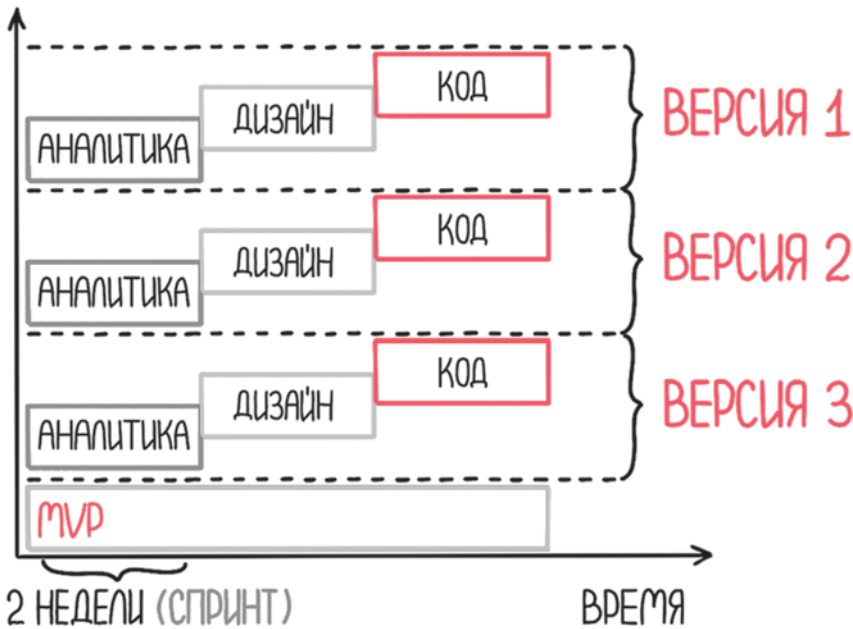
Можно ли совмещать в себе несколько ролей сразу? Быть и скрам-мастером и Product Owner? Технически — да, я такое видел в молодых командах. Но это антагонистические роли. В них специально зашит конфликт. Product Owner хочет больше и быстрее от команды, и менять все в любой момент. Scrum Master хочет, чтобы команда спокойно работала, и процесс не ломался. Сможете вы такое в своей голове

удержать (быть и добрым, и злым), или начнет шизофрения развиваться? Хорошего-то мало. Я бы начал выращивать scrum-мастера внутри команды. Благо — это несложно, дня за три можно справиться.

3.11 АНАЛИТИКА, ДИЗАЙН, РАЗРАБОТКА — ПАРАЛЛЕЛЬНО

Обычно нужно несколько спринтов, чтобы получился MVP (minimum viable product) — минимальный жизнеспособный продукт. MVP должен быть клевенький.

Цепкий, энергичный менеджер может организовать параллельную работу над аналитикой, дизайном и кодом. В Scrum-е спринт по аналитике запускаем чуть заранее. За ним — дизайн. И далее уже код. Примерно по такой схеме (правда не факт, что у вас будет так много дизайна и аналитики, но вдруг?).



При параллельной разработке нагрузка на менеджмент и коммуникации растет чуть ли не экспоненциально. Поэтому подход не для новичков, а для матерых организаторов.

3.12 ДОКУМЕНТАЦИЯ

Agile-манифест (которому сто лет в обед) — набор правильных лозунгов, которые как-то надо адаптировать к практике:

Люди и взаимодействие важнее процессов и инструментов.
Работающий продукт важнее исчерпывающей документации.
Сотрудничество с заказчиком важнее согласования условий контракта.
Готовность к изменениям важнее следования первоначальному плану.

То есть, не отрицая важности того, что справа, мы все-таки больше ценим то, что слева.

Agile-манифест разработки программного обеспечения. 2001 год.

В зрелом продукте документация нужна, Agile это не отрицает. В небольших web-проектах и приложениях — можно и без нее. Степень замороченности прямо пропорциональна объему проекта, педантизму заказчика и ресурсам. Правда, документация всегда немного отстает от реальности, и это окей.

Работая над **SingularityApp**, например, я беру какую-то крупную хотелку из бэклога и расписываю, как бы я хотел, чтобы она работала с точки зрения пользователя. Это просто схемки интерфейсов на iPad, скриншоты похожих реализаций и немного текста. Если нужно докрутить формальности — отдаю аналитикам (с проверкой, чтобы не исказили идею). Там уже могут появляться прототипы, описание граничных случаев, интеграционные протоколы и так далее, если мне нужна такая степень проработки и формализма. Такие штуки удобно хранить в **Confluence** или **википедии проекта**. Подойдет и Google Docs.

Уже эти постановки дербанятся на технические тикеты (sprint backlog), и по ним рисуются интерфейсы. Confluence позволяет отправлять задачи

в бэклог прямо из текста документов (правда, довольно коряво, но все же). Мелкие хотелки из бэклога можно так детально не разжевывать.

Кодерская и техническая документация, в основном, лежит либо в самом коде (the code speaks), либо в вики проекта.

Общее правило — положи документацию как можно ближе к месту ее использования.

Иначе про нее забудут, забудут и потеряют. Мне лениво лезть лишний раз за кусочком текста. Я хочу все понимать здесь и сейчас.

Документацию важно увязать с тест-планами, чтобы QA не плодил свои постановки и хотелки.

Пользовательская документация (инструкции, мануалы, вопросы и ответы) — обычно отдельный процесс. Полностью делегируем. В долгих и больших проектах, где важно поддерживать **консистентность документации** (и на это есть ресурс), во время написания пользовательских инструкций актуализируются и изначальные постановки: чертежи с iPad заменяются реальными интерфейсами, дополняются моменты, которые уточняются по ходу реализации.

На заказных web-проектах и мобильных приложениях такой контур слишком затратен, тут документацией должен стать сам код, бэклог, ТЗ или описания в **Swagger**. Хотя время от времени сложные, интеграционные моменты приходится документировать. Особенно, если реализуется API, с которым будут работать сторонние разработчики.

Документация требует времени, ресурсов, отдельного контура управления и внимания. Ну а хорошего технического писателя — днем с огнем.

Нужна ли документация на вашем проекте? Будете ли вы ее писать по ГОСТ 19 или ГОСТ 34, или стандартам ISO? Будете ли использовать **UML** или BDD-подход (Behavior-driven development, дословно «разработка через поведение»), **язык описания сценариев Gherkin**? Вопрос, на который не надо торопиться отвечать. Делайте минимально необходимое и дополняйте по мере необходимости. Не старайтесь заранее все предусмотреть.

ЧЕЛОВЕК, КОТОРЫЙ ВСЕ ПРЕДУСМОТРЕЛ

Однажды человек решил предусмотреть на сайте ВСЕ. ВСЕ тренды, вау-эффекты, поисковые рекомендации... да мало ли. Короче, все. Ну чтоб потом не переделывать. Это был очень предусмотрительный человек.

И на встречу со студией он решил прийти лично. Надел костюм. Пальто. Шляпу. Взял зонтик. Подумал и на всякий случай захватил солнцезащитные очки. Перчатки. Сотовый телефон. Две запасных батарейки. Это был очень предусмотрительный человек.

Взял пару бутербродов. Бутылку коньяка. Пачку презервативов. Хлоргексидин. Перочинный ножик. Вареных яиц. Ножовку по металлу. Бумажную карту Свердловской области. Аккуратно разложил это по карманам своей жилетки. И не надо думать, что это был какой-то уникум, типа Онотолия. Просто обычный человек, который всегда все предусматривал.

Аптечку. Лыжи. Акваланг. Ласты. Бинокль.

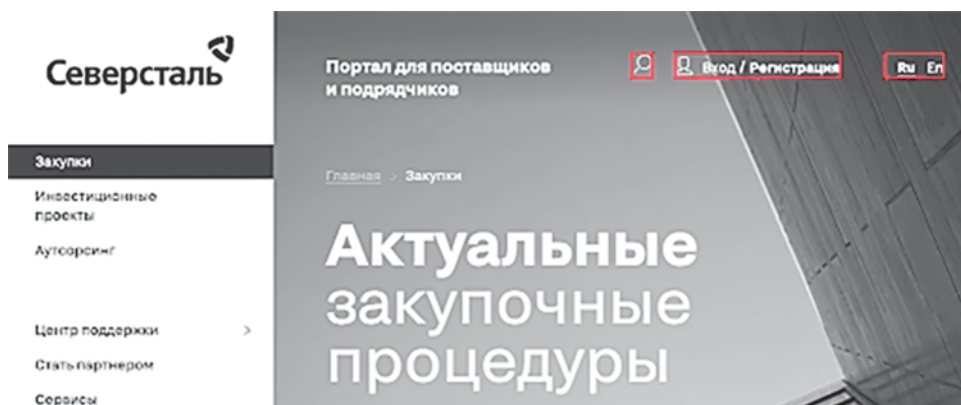
Вышел из дому. Стоит посреди лужи. И думает: «Блин, а все ли я предусмотрел на сайте?»

3.13 МЕТОД КРАСНОЙ РАМКИ

Для программистов, если они не шарлатаны, уже привычно работать итерациями. Какая-то функция на экране готова. Какая-то — нет. В новых версиях в каждый спринт, добавляется что-то новенькое. Но в целом можно уже посмотреть, как ведет себя продукт.

Чтобы не возникало вопросов, что уже готово, а что — нет, рекомендую использовать **метод красной рамки**. Суть в том, что прописывается специальный стиль (например, `todo`), выводящий тонкую красную рамку вокруг нужного нам элемента. Этот стиль присваивается тем компонентам интерфейса — кнопкам, полям, формам и т. д. — которые отражаются на экране, но еще не реализованы в коде. Таким образом наглядно видно, что в данный момент готово, а что — нет.

Этот метод работает и в заказной итеративной разработке. Заказчику не нужно будет постоянно объяснять, куда можно тыкать, а куда — нет. Если что-то не работает — мы четко разграничиваем ситуации «сломалось, баг» или «все под контролем, еще не реализовано».



Одна из первых итераций портала Северсталь. Поиск, вход, регистрация и переключение языков еще не реализованы.

Итак, плюсы:

1. Заказчик сразу видит, что готово, а что нет — не тратим время на объяснения, почему не работают некоторые поля.
2. Тестировщик понимает, какие блоки еще сырые, и не тестирует их.
3. У разработчиков не остается шанса промотать задачу.

3.14 ПОТОМУ ЧТО КАРГО-КУЛЬТ!



Во время 2-й мировой войны США строили свои военные базы по всяким разным диким островам в Тихом океане. Продовольствие и шмотки возились самолетами, причем часть груза просто сбрасывалась вниз. Ну, и кое-чего перепало диким человекам, живущим на этих островах. Причем, иногда перепало столько, что аборигены полностью забивали на свою хозяйственную деятельность, выращивание бананов и скотоводство. Туземцы быстро уловили, что американцы сами ништяки не производят, а все им достается с неба, за верность духам предков.

Война кончилась, американцы свернули базы. И вот представьте их удивление, когда лет через 20, вернувшись на эти острова, они обнаружили марширующих негров с муляжами ружей, наушниками из дерева, копии самолетов из бамбука в натуральную величину, муляжи аэродромов из соломы... В общем, туземцы довольно четко эмулировали действия американцев, с той лишь разницей, что духи предков почему-то не спешили больше сбрасывать с небес ништяки.



Туземцы не понимали, что стоит за происходящим, не понимали, как устроен мир, и почему с неба падает продовольствие. Поэтому выполняли бессмысленные ритуалы, которые не давали результата.

Не тем ли мы занимаемся, когда выполняем ритуалы, вроде стояния у канбан-доски по утрам или планирования с помощью карт Planning Poker, абсолютно не понимая смысла происходящего? Понимаете ли вы, какие выгоды дает каждый компонент?

Возможно, единственная проблема туземцев в том, что они остановились в своих поисках и экспериментах и не получали какую-либо обратную связь. Истина, видимо, где-то посередине: разбирайтесь в тех методологиях, которые используете, и постоянно экспериментируйте, оставляя только реально работающие подходы. И ништяки повалятся к вам.

Упорство отличается от **Упертости**. Упорный, если не получилось, пробует другой подход. Упертый продолжает делать то же самое, рассчитывая на другой результат.

3.15 ДЛЯ ТРЕНИРОВКИ

Итак, мы разобрали Scrum — гибкий простой фреймворк для организации работы команд над современным софтом! Он задыхается в угрюмой, бюрократичной среде и хорош там, где нужно регулярно улучшать продукт, реагировать на изменения в мире и обратную связь. Мобильные и веб-приложения — самое то!

Задание — иди и внедряй!

Попробуйте это внедрить у себя. Посмотрите, что заработает сразу, а что — ни в какую. Проведите несколько спринтов и ретроспектив.

И постарайтесь не скатываться в СКРАМНО («у нас Скрам, НО без итераций») и Карго-культ.

Успехов!

ЛИТЕРАТУРА



- ▶ Джефф Сазерленд, «Scrum: Революционный метод управления проектами».
- ▶ Фредерик Брукс, «Мифический человеко-месяц».
- ▶ Антон Макаренко, «Педагогическая поэма».
- ▶ Скрамгайд в доступном переводе от Сибирикс (QR-код слева).

ПОЯСНЕНИЯ

Канбан-доска — инструмент метода разработки «Канбан», представляет собой доску, разделенную на этапы работ, с задачами в виде карточек, которые перемещают по доске по мере их продвижения по этапам.

Рефакторинг — перепроектирование или переработка кода, изменение его структуры, которые не производят функциональных изменений, но существенно облегчают его работу.

Закон Мерфи — шуточный философский принцип, который звучит как: «Все, что может пойти не так, пойдет не так».

Story Point — метод оценки сложности задач, когда за 1 балл принимается самая простая задача, а все остальные оцениваются относительно нее.

Автотест (автоматизированный тест) — это скрипт, имитирующий взаимодействия пользователя с приложением, для локализации ошибок в работе сайта, приложения или ПО.

Смоук-тест — минимальный набор тестов на явные ошибки, который обычно проводит программист. Без прохождения такого теста нет смысла затевать более глубокое тестирование.

Продуктив — уже запущенный сайт, живущий на сервере заказчика.

Коммит — в системах управления версиями коммит добавляет последние изменения в часть исходного кода в хранилище, делая эти изменения частью основной версии хранилища.

Confluence — софт для командной работы, удобный для распределенных команд за счет опций совместной работы.

SingularityApp — мощный хаос-менеджмент планировщик, разработанный в студии «Сибирикс». Существует в виде онлайн-версии для ПК и приложения для смартфонов на Android и iOS.

Википедия проекта — база знаний, где хранятся подробные руководства функционала продукта.

Консистентность документации — цельность документации и согласованность документов друг с другом.

Swagger — язык описания интерфейсов для описания RESTful API, который используется для проектирования, создания, документирования и использования веб-сервисов RESTful.

UML — это специальный язык для описания разных бизнес-процессов, структур и прочих вещей, для которых необходимо последовательное описание. В запутанных ситуациях с большим количеством сущностей может сильно помочь. UML включает несколько видов диаграмм: диаграммы последовательности, компонентов, объектов, структуры, синхронизации и так далее.

Язык описания сценариев Gherkin — человеко-читаемый язык для описания поведения системы, который использует отступы для задания структуры документа (пробелы или символы табуляции). Каждая строка начинается с одного из ключевых слов и описывает один из шагов. Обработчик разбивает файл с тестами на функции, сценарии и входящие в них шаги.

ЧАСТЬ 4



АНАЛИТИКА И ПРОДУКТОВЫЕ ТЕХНИКИ

Большинство удачных идей оказывается неудачными.

Британские ученые подсчитали, что 92% стартапов проваливаются в течение 3 лет. Главная причина — продукт нафиг никому не сдался (42%).

Я чувствую так же. Мы плотно работали со стартапами с 2010 года. Некоторые из них живы, и даже неплохо живут. Но оглядываясь назад, я вижу кладбище.

Наверное, всем, кто работал в заказной разработке достаточно долго, знакомо это чувство. Много времени, сил, энергии, интеллекта и отваги вкладывается в какой-то проект, в какую-то идею... И где это все годика через 4? Пшик, очень жаль...

Другое дело, когда проект создавался под конкретную задачу, было заранее понятно, что есть сегмент пользователей, у которых в нем есть потребность, и посчитана экономика. Зачастую это обычные, работающие бизнесы, решившие поглубже залезть в интернет или автоматизировать рутину. Тут, несмотря на кризисы, выживаемость и успешность близки к 95%. Пойман **Product Market FIT (PMF)** — состояние, когда клиенты довольны продуктом и рекомендуют друзьям. А сам бизнес растет.

Я уже говорил об этом во введении, об экологичном пути руководителя проектов, но скажу еще раз. Моя религия и пятнадцать лет опыта выращивания руководителей проектов говорят, что если вы только осваиваете управление digital-проектами и не имеете глубокого технического бэкграунда, то **самый экологичный путь стать**

первоклассным специалистом — это поработать некоторое время в тестировании и аналитике.

В этой части мы посмотрим на техники управления продуктом, которые помогут минимизировать галлюцинации основателей, нащупать сегменты пользователей, протестировать гипотезы и т. д. Этими инструментами должны хорошо владеть руководители продуктов и аналитики. В маркетянские штуки уходить не будем.

Для руководителей проектов инструменты полезны. Во-первых, для понимания, как Product Owner принимает решение. Во-вторых, на вырост. В-третьих, скорее всего, Product Owner что-то из этих штук сам делать поленился, или его нужно будет возвращать в реальность. А кто будет делать все то, что делать надо, но не делает никто другой — вы уже в курсе.

Итак. Путь к крутому руководителю проектов и продуктов лежит через аналитику. Разберитесь в этом. Погнали!

4.1 ЦЕЛЬ АНАЛИТИКИ DIGITAL-ПРОЕКТОВ

Если не знаешь, какой херней ты занимаешься на работе, — назови это «Аналитика».

Народная мудрость

К сожалению, вокруг аналитики digital-проектов в последнее время накрутили много хераборы. Методов и подходов стало слишком много. Непонятно, за что хвататься и что применять.

Этап аналитики нужен, чтобы верно понять ожидания заказчика, спроектировать оптимальное решение поставленных задач и грамотно спланировать бюджет.

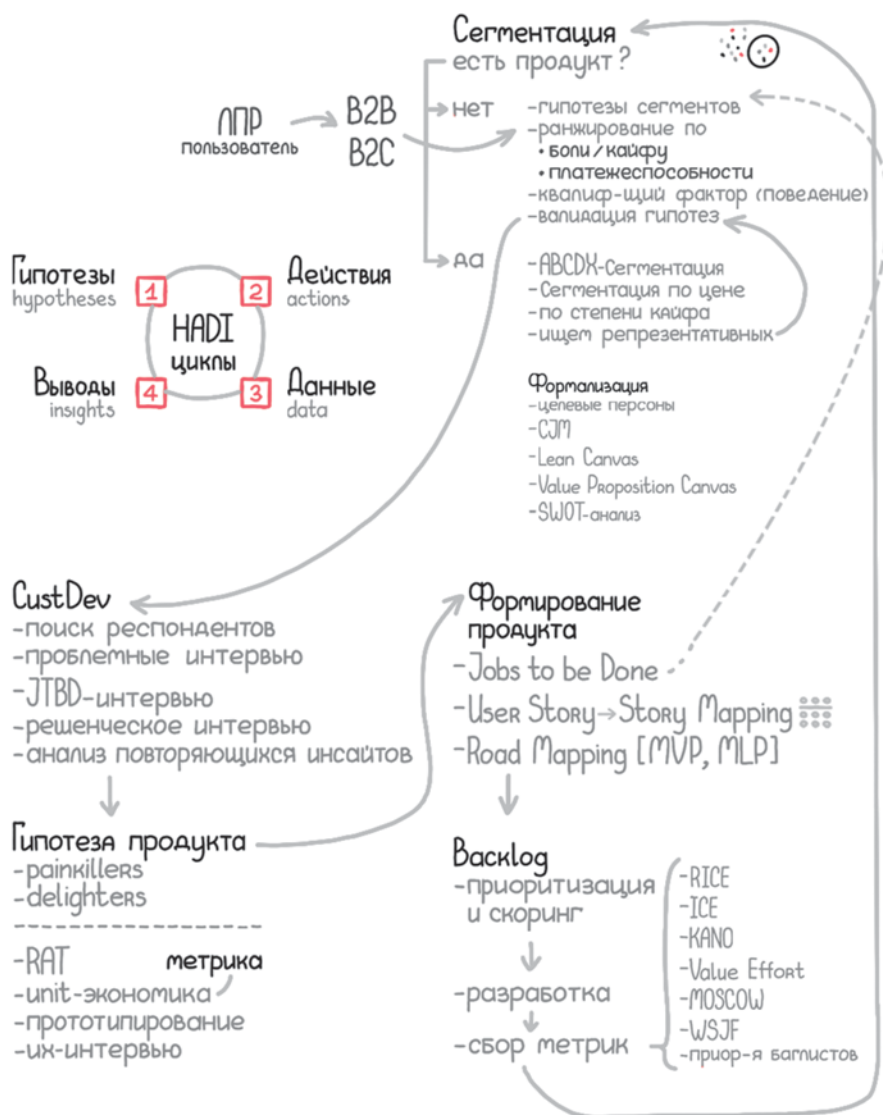
Если раньше аналитика была вспомогательной функцией, сейчас — стала чуть ли не самоцелью. Я видел команды, которые по полгода играют в «Дизайн-мышление» или еще какой-то новомодный метод, производя только трюндеж. Бюрократия — такая бюрократия. Правда, кризис оставил бездельников за бортом.

Давайте исходить из того, что:

Цель аналитики в digital-проекте — получить **структуру** будущего проекта и **четкий план действий**: сначала мы делаем это, потом то, а затем — вон то.

Понятно, что и план и структура со временем могут меняться. Понятно, что в сложном проекте лучше семь раз подумать. Поэкспериментировать. Но в какой-то момент нужно перестать анализировать все подряд. И начать писать код.

4.1.1 ЧТО НАС ЖДЕТ В ЭТОЙ ГЛАВЕ



Инструментарий руководителя digital-продукта

Для заказной разработки нам понадобится Агрегация требований, прототипирование и подготовка технической документации. Когда есть заказчик (или владелец продукта) — этого достаточно.

Для самого владельца продукта существует ряд дополнительных подходов: HADI-циклы, CustDev, методы сегментации пользователей и различные способы скоринга бэклогов.

Методы дополняют друг друга. Однако стоит к ним относиться как к арсеналу: снаряжаясь на войну, вы не сможете забрать на себе весь арсенал. Вы выберете один или два вида оружия и что-то из защиты. Кто-то возьмет лук, поскольку меткий. Кто-то меч по руке. Но весь арсенал вы на себе не потащите.

Точно так же нет смысла тащить в проект все известные способы приоритизации бэклогов. Выберите один, который вам нравится. И заточите его под вашу ситуацию.

Как этим пользоваться: прочитайте главу один раз. Ознакомьтесь со всеми методами. Выберите те, которые вам понравились. Изучите их подробнее. Начните применять. Время от времени пересматривайте подходы. Возможно, в качестве эксперимента, захочется попробовать какой-либо другой метод.

Итак. Что нас ждет.

Во-первых, мы рассмотрим метод **Агрегации требований**. Формально он состоит из 5 шагов:

1. Видение проекта. Это основные характеристики будущего продукта. Его цели и задачи — так, как их понимают создатели продукта.
2. Целевые персоны. Здесь мы идем от пользователей. Сегментируем рынок. Выделяем боли и потребности пользователей. Проводим интервью. Строим сценарии поведения и CJM — карту путешествия клиента.
3. Конкурентный анализ.
4. Структура проекта. Какие экраны нам нужны. Что на них будет.
5. Идеи на будущее. Все то, что было бы неплохо сделать.

Во-вторых, посмотрим, как делать регулярную аналитику в продуктовых командах на основе HADI-циклов. Разберем 4 силы, действующие на продукт. Поговорим о фреймворках RAT и JTBD.

В-третьих, поговорим про юнит-экономику продукта. Эта информация будет нужна руководителю проекта, чтобы понимать, на основе чего руководители продуктов принимают решения. На самом деле, метрик бесконечное множество и тут легко запутаться.

В-четвертых, поговорим про техники скоринга и приоритизации бэклогов, так, чтобы галлюцинации основателей действовали минимально.

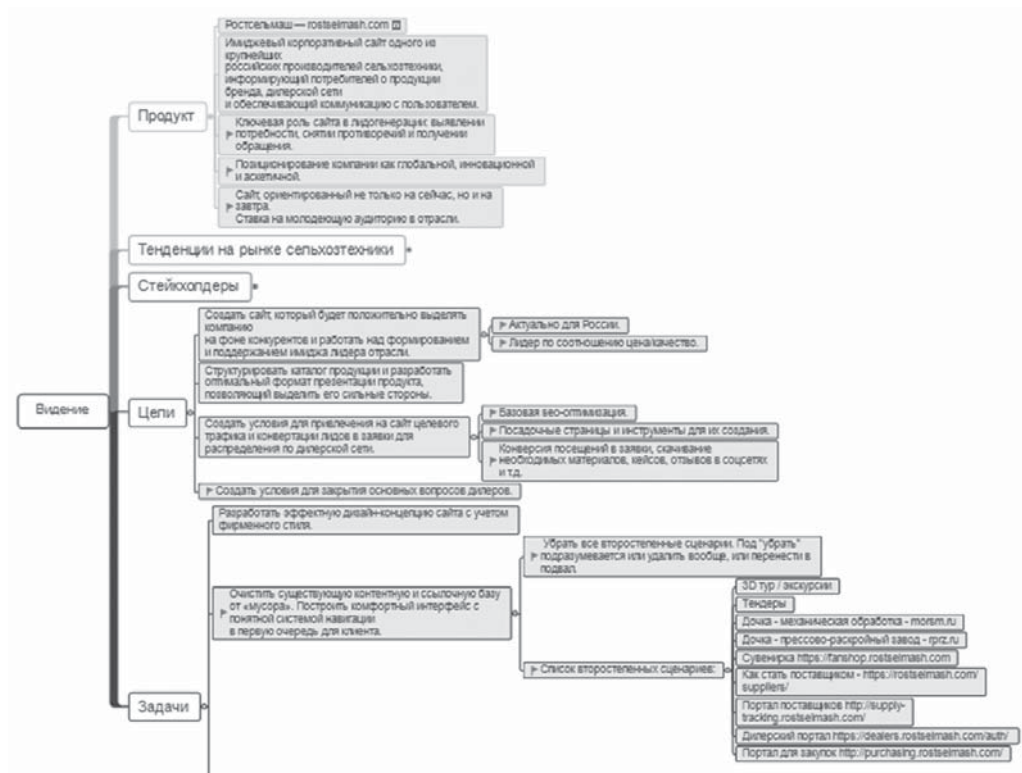
В-пятых, посмотрим, как UML-диаграммы помогают при проектировании программных продуктов, продумывании взаимодействия с пользователем.

4.2 АГРЕГАЦИЯ ТРЕБОВАНИЙ В ЗАКАЗНОЙ РАЗРАБОТКЕ

Эта техника лучше всего работает в заказной разработке. Либо когда уже в общих чертах понятна задача, осталось только вытащить детали из голов стейкхолдеров, примерить их на целевую аудиторию, убедиться, что не допущены ошибки конкурентов и учтены тренды отрасли. Подходит для большинства сайтов и мобильных приложений. Позволяет синхронизировать видение у заказчика и разработчика, а также довольно точно подсчитать бюджет разработки. Это, в основном, кабинетный анализ, который можно подкрепить интервью с потенциальными пользователями (см. главу о Customer Development) и А/В-тестами. В итоге у нас должна получиться структура будущего проекта. Для фиксации предлагаю использовать формат интеллектуальных карт.

Агрегация требований — основа проекта в заказной разработке. Именно на этом уровне мы понимаем, что на самом деле нужно заказчику и его клиентам. Где границы возможного с точки зрения человеческих и финансовых ресурсов. Как лучше всего сделать сайт или мобильное приложение, какие системы надо сшить друг с другом, чтобы решить поставленные задачи. Как выделиться в конкурентной среде и притом уложиться в реальные для заказчика бюджет и сроки.

Чаще всего, в заказной разработке мы создаем Агрегацию требований в формате интеллектуальных карт. Например, XMind или XMind Zen. Альтернативные программы можно найти в конце главы, в списке литературы.



Агрегация требований РостСельМаш (фрагмент)

По итогам агрегации мы узнаем:

- какие цели и задачи у будущего проекта;
- что ждет от него целевая аудитория;
- что хорошего и плохого в проектах конкурентов;
- как все это учесть и совместить, чтобы одно другому не мешало;
- как лучше запустить проект: весь сразу или по частям, и с чего начать;
- какой будет структура сайта, на основе которой аналитик готовит прототипы и пишет техническое задание;
- рассчитываем довольно точно сроки и бюджет проекта.

4.2.1 СТЕЙКХОЛДЕРЫ



Стейкхолдеры (stakeholder) — заинтересованные в проекте лица. В заказной разработке от них исходит большая часть требований.

Однако бывает, что стейкхолдеры явно не определены. Прячутся за корпоративной иерархией. Влияют, но не отвечают.

В процессе реализации проекта власть в организации, если взять ее за 100%, как-то перераспределяется. Как только появятся первые результаты проекта — все, кто хоть как-то был затронут этим решением, выскажут свое мнение. Иногда этих мнений слишком много. Или мнения слишком громкие. Или противоречивые. Придется разбираться. Чем раньше, тем лучше.

Проекты, в которых заинтересовано первое лицо компании-заказчика, проходят гораздо ровнее. Нужно четко понимать, кто у заказчика Альфа, кто Бета. Проекты, в которых внутри заказчика постоянные споры и конфликты, и нет никого умного, кто мог бы во всем разобраться и принять твердое, окончательное решение, с треском проваливаются. Либо идут медленно и печально. Либо требуют молотья языком больше, чем писать код.

Итак. В заказной разработке важно как можно раньше выявить стейкхолдеров. Выявить их интересы. Скрытые и явные конфликты. Понять иерархию стаи. Заручиться поддержкой Альфа- и Бета-стейкхолдеров.

4.2.2 ВИДЕНИЕ ПРОЕКТА

Первый шаг Агрегации требований — это видение проекта или его основных будущих характеристик. Здесь мы определяем, какой продукт мы делаем, и что он из себя представляет. Видение должно быть конкретным: если при смене названия бренда видение клиента останется прежним, оно никуда не годится.

Плохо

Продукт: интернет-портал для обучения специалистов в сфере строительства.

Хорошо

Продукт: учебно-рекрутинговая платформа для проектно-строительного рынка. Служит для бесплатного обучения и тестирования специалистов по BIM-технологиям (личного или корпоративного). А также подбора сотрудников в сфере информационного моделирования зданий.

Обозначаем перечень и ожидания заинтересованных лиц (стейкхолдеров). На этом этапе целесообразно рассматривать стейкхолдеров только со стороны заказчика, поскольку пользователям сайта посвящена отдельная вкладка. Чаще всего заинтересованные лица делятся на несколько уровней:

- ▶ собственники и управленцы (могут совпадать, но не всегда);
- ▶ представители отделов продвижения и продаж;
- ▶ обслуживающий персонал (операторы, контент-менеджеры, администраторы).

Разумеется, у всех у них очень разные ожидания от будущего проекта. Важно обозначить главные направления, добраться до сути: зачем на самом деле создается проект, как он должен изменить жизнь причастных к его эксплуатации людей.

Необходимо развернуто зафиксировать суть ожидания и кратко обозначить, какие пути решения мы видим для него на этом этапе. В ходе дальнейшего анализа, в том числе после получения отклика со стороны заказчика, список решений может корректироваться. Часть идей может быть отложена до будущих этапов развития сайта.

«Освежить» имидж парка в сети, сделать его более современным и привлечь к нему дополнительное внимание	Сайт с современным ярким дизайном, учитывающий как требования брендбука, так и современные тренды дизайна в рамках отрасли. Качественный контент на сайте: емкие тексты, красивые фото.
	Проработка индивидуальных страниц уникальных аттракционов и услуг парка. Связь сайта с социальными сетями парка.
Повысить узнаваемость парка в регионах России и зарубежом — статус достопримечательности Сочи, Краснодарского края, России.	Адаптивная версия сайта для корректного отображения на мобильных устройствах. Информация об истории парка и его выдающихся объектах. Английская языковая версия для удобства иностранных гостей и партнеров. Акцент на значимость продвижения в теме парка культуры русских сказок, творчества и традиционных национальных героев.

На этапе видения определяются **цели** и **задачи** проекта. Цель — чего именно хотим достичь. Увеличение конверсии, увеличение числа интернет-заказов, оптимизация работы менеджеров. Задачи — что конкретно будем делать. Семантическая разметка, фишки в юзабилити, интеграция с CRM.

Важно! Не нужно писать сюда все бизнес- и жизненные цели, о которых упоминал заказчик — только то, что реально касается разрабатываемого проекта, может быть объективно оценено и измерено. К примеру, если пишешь о росте конверсии, сразу подумай, какие наши действия к этому гарантировано приведут, и как вы с заказчиком измерите «было-стало».

Вкладка формируется на основании данных, полученных при изучении первичных материалов от заказчика, его брифования, общения со стейкхолдерами. С поправкой на опыт и здравый смысл.

Кстати, про здравый смысл и галлюцинации!

4.2.2.1 Галлюцинации основателей

Мы с тобой свернули не туда вообще
И все закончится для нас теперь так себе.
Znaki

Вы когда-нибудь пробовали отговорить основателя от его идеи? Большинство из них настолько сильно убеждают себя, что на их продукт есть спрос, и настолько харизматичны (или властны), что проще дать сделать то, что они хотят, чем объяснить, почему нет.

Вот несколько галлюцинаций основателей, которые мешают:

- «Мы знаем нашу целевую аудиторию, просто сделайте, как мы сказали». По факту аудиторию они не знают, а мантра «мы все знаем», повторенная несколько раз, дает ложную уверенность

контроля. Типичный итог — отсутствие потребности в продукте. Нет сегмента покупателей.

- ▶ «Мою идею украдут». Сразу тревожный звоночек. Идеи не стоят ничего. Артемий Лебедев писал про «Идею на минус миллион» и вообще считает, что идеи имеют отрицательную ценность.
- ▶ «Мой продукт должен быть идеальным». Перфекционизм. Во-первых, это дорого. Во-вторых, за этим прячется боязнь показать продукт рынку. А вдруг обидят?



«Идея на минус миллион» от Артемия Лебедева

В ту же копилку:

- ▶ Тантрический стартап: три года без релиза, ни дня без **чендж-реквеста!**
- ▶ Обратная связь от пользователей откладывается на месяцы.
- ▶ «Сделайте нам прибыльный проект за процент от будущей прибыли». Нет денег на фоне повышенной хитрожопости. Либо нет денег на разработку. Либо на маркетинг. Либо на то и другое.
- ▶ Усложнители. Затея либо сразу настолько сложная, что в деталях путается даже сам основатель. Либо пытаются использовать какую-то технологию там, где она нафиг не нужна. Блокчейн для учета шкурок крупного рогатого скота, например. Фаза анализа проблемы и потребности пропускается, идем сразу в архисложное решение.
- ▶ Немасштабируемые продукты.
- ▶ Подражатели. Уже давно замечал, что заявки на разные типы стартапов приходят волнами. Был когда-то всплеск на социальные сети. Аналоги купонаторов. Скандинавских аукционов. Как-то осенью нас засыпало заявками «Продайте долю в игре в соцсетях». Потом были маркетплейсы. Где-то людей «Бизнес-молодость» зомбирует, честное слово!

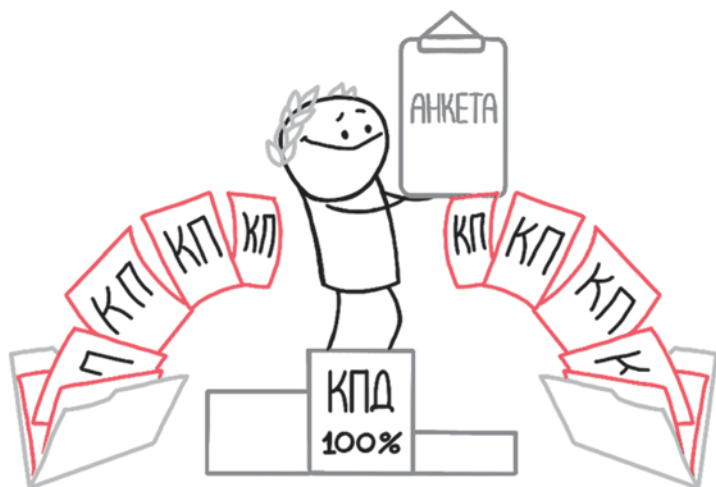
Давайте исходить из того, что у основателя есть определенная стратегия, видение, а **наше дело — помочь все это реализовать по красоте, исходя из нашего опыта и знаний.**

4.2.2.2 Почему брифы — зло

Я не люблю письменные брифы. В своей работе мы их почти не используем. Брифы не дают ясной картины. Теряется много важных деталей. И главное!

Брифы мешают строить долгосрочные отношения с клиентом!

С другой стороны, я понимаю заказчиков. Вернее, менеджеров на стороне клиента, которых отправили «найти подрядчика». Они (агентства), блин, все одинаковые! Гораздо проще заполнить один раз анкету. Разослать в сотню агентств. Получить КП-шки. Распечатать, отсортировать по «Итого:». И положить их боссу на стол. Минимальная ответственность. Минимальная трудоемкость. Божественный КПД — результата можно целую кучу навалить.



С одной стороны, так было и будет всегда. С другой стороны — понятна степень заинтересованности проектом. Агентствам нужно уметь под это подстраиваться.

Любые проекты, стоимостью в миллионы и длительностью в месяцы, обязательно обсуждаются устно. И не один раз. Есть примерный и понятный круг вопросов, с которых можно начать. Однако нужно иметь достаточно опыта, чтобы погружаться в глубину по любому из вопросов.

Компания

Какие есть компетенции?

В чем суть бизнеса? Что приносит доход?

Узнаваем ли бренд? Какова лояльность аудитории?

Какие есть сложности?

Продукт

Сильные стороны. Почему это должны купить?

В чем отличие от аналогов?

Есть ли товары-заменители (не прямые)?

Жизненный цикл. Как часто пользуются? Как долго?

Покупатель (ЦА)

Кто он?

По сегментам аудитории: объем группы, темпы роста и т. д.

Чего он хочет? Потребности?

Какая цена приемлема?

Как привык потреблять? Получать информацию?

Конкуренты

Кто конкуренты? Какие у них сильные стороны?

Насколько наполнен рынок?

Чем вы лучше других? (Осторожнее с этим вопросом — тут горят пуканы!)

Каковы преимущества? Есть ли уникальные продукты, фишки?

Какие есть трудности и барьеры?

Планы развития?

Это общие направления вопросов, которые нужно адаптировать под каждый конкретный проект после изучения вводной информации. Может быть много нюансов.

Например, на e-commerce-проектах возникает несколько десятков вопросов по экономике, налогам, логистике, возвратам, интеграциям с внешними системами и т. д. Нужно включить вопросы по всему, что кажется непонятным и странным в проекте. А также затронуть все вопросы, ответы на которые лягут в основу вложений Видение и Анализ целевых персон.

Итак. Ни один бриф не заменит голову. Письменные брифы — скорее, зло. Какова альтернатива?

4.2.2.3 Lean Canvas

Lean Canvas — это популярный шаблон для фиксации целей продукта. В его основе — шаблон бизнес-плана Александра Остервальдера, оптимизированный под стартапы. Главный плюс — всю концепцию проекта можно разложить на один лист А4. И конечно, эта бизнес-модель вдохновлена методологией Lean Startup (читайте книгу Эрика Риса) и пропитана философией бережливого мышления.



QR-код, по которому можно скачать шаблон Lean Canvas

LEAN CANVAS				
2. ПРОБЛЕМА 1-3 ключевых проблемы	4. РЕШЕНИЕ Возможные решения для каждой проблемы	3. УНИКАЛЬНАЯ ЦЕННОСТЬ Простое четкое сообщение, которое отличает вас от конкурентов и привлекает внимание	9. СКРЫТОЕ ПРЕИМУЩЕСТВО Ваша особенность, которую сложно купить или подделать	1. СЕГМЕНТЫ ПОТРЕБИТЕЛЕЙ Список целевых покупателей и пользователей
СУЩЕСТВУЮЩИЕ АЛЬТЕРНАТИВЫ Список того, как эти проблемы уже решаются сегодня	8. КЛЮЧЕВЫЕ МЕТРИКИ Список ключевых цифр, которые говорят о том, как обстоит бизнес	КОНКУРЕНТЫ Список товаров/услуг, которыми можно заменить ваш продукт (в том числе согласно концепции JTBD)	5. КАНАЛЫ Списки входящих и исходящих каналов на пути к вашим клиентам	РАННИЕ ПОСЛЕДОВАТЕЛИ Список характеристик идеального потребителя
7. СТРУКТУРА ИЗДЕРЖЕК Список постоянных и переменных издержек			6. ПОТОКИ ПРИБЫЛИ Список источников прибыли	

сибирикс
коллаборация

Сегменты потребителей

Впишите сюда не только тех, кто покупает продукт, но и тех, кто им будет пользоваться (родители покупают игрушку, а ребенок ей играет). Еще лучше — собрать реальную группу людей, которые будут первыми тестировать продукт и помогут проверить гипотезы.

Проблема

Какую потребность ваш продукт или услуга закрывают? И кто еще это делает на рынке уже сейчас? Не забудьте, что помимо прямых кон-

курентов есть вторичные и косвенные (подробнее об этом поговорим в параграфе о Jobs To Be Done).

Уникальная ценность

По-другому — УТП, или почему клиенты захотят купить именно ваш продукт? В чем его ценность?

Решение проблемы

Пригодятся интервью с потенциальными пользователями и исследования, чтобы подтвердить ваши гипотезы.

Каналы

«Детям — мороженое, бабе — цветы» — для каждой аудитории ищите свои каналы.

Потоки прибыли

Кто платит. Кому. Когда. И за что. Если продукт или услуга бесплатны, решите, как будете монетизировать проект.

Структура издержек

От зарплаты работникам и арендной платы до затрат на рекламу и создание сайта.

Ключевые метрики

Подумайте, как оценивать результат: как будете измерять интерес к продукту, на какой стадии жизненного цикла клиента вы готовы назвать его постоянным и т. д.

Скрытое преимущество

Найти его не так-то просто, поскольку это может быть не самая очевидная вещь: прямые руки работников или крутые поставщики.

4.2.2.4 Видение. Итоги

Итак.

1. Для фиксации самой верхнеуровневой информации о проекте удобно использовать Lean Canvas. Это может быть что-то типа паспорта проекта. Его отправной точки. Можно охватить взглядом и сразу понять суть.
2. Длинные письменные брифы лучше не использовать. Обсуждайте проект устно. Общайтесь. Погружайтесь в детали. Фиксируйте. Обычно для формирования видения нам нужно 1–3 сессии с клиентом. Важно быть на одной волне и строить долгосрочные отношения.

3. Для фиксации видения проекта удобно применять **Mindmap**. Важно учесть мнения всех стейкхолдеров. Выявить противоречия. Разрешить их. Сформировать цели и задачи проекта.

4.2.3 ПОТРЕБИТЕЛИ, СЕГМЕНТЫ, АВАТАРЫ И ЦЕЛЕВЫЕ ПЕРСОНЫ



Допустим, вы продаете стулья. Значит ли это, что ваша целевая аудитория — все люди, у которых есть ягодицы? Не значит. Да и достучаться до всех вы, скорее всего, не сможете.



Реальное объявление
в детском парке.
На какой сегмент
рассчитывает автор?

Сегментация — разделение всего рынка на сегменты из потенциальных клиентов со схожими потребностями и поведением. Для digital-проектов сегменты будут нужны, чтобы определиться, какие функции нужны в продукте и как расставить приоритеты разработки.

Еще раз: сегментация нам нужна, чтобы понять, как поведение пользователя влияет на продукт. Поэтому **дифференцирующим критерием** (чем же вы, елки-палки, на самом деле различаетесь) **предпочтительнее брать поведенческий фактор**. Как разные пользователи по-разному пользуются вашим продуктом и какие разные задачи они решают.

Жестких правил и критериев для сегментации нет. Если у вас ведется подробная база пользователей и обращений, можно попробовать загнать ее в систему анализа бигдаты (вроде RapidMiner), поиграть с ней в кластерный анализ и попробовать выделить сегменты.

Довольно часто для сегментации используют критерии: география, демографические признаки (пол, возраст, семейное положение, доход, образование, профессия, религия, национальность и т. д.), психографию (хобби, образ жизни и т. д.), платежеспособность (LTV — Lifetime Value, сколько денег приносит клиент за все время сотрудничества).

Это все, конечно, замечательно. Особенно для повышения среднего чека. Но для разработки минимально жизнеспособного digital-проекта — почти бессмысленно. Ищите дифференцирующий критерий в поведении.

Периодически нужно пересматривать сегментацию, искать новые сегменты и уточнять имеющиеся.

4.2.3.1 Анализ текущего поведения пользователей

Допустим, у вас задача — редизайн существующего проекта. Перед стартом работ нужно собрать срез статистики. Нам понадобятся доступы к счетчикам типа Яндекс Метрики и Google Analytics. Статистика нужна примерно за полгода-год.

Если на старом сайте есть хоть какой-то трафик — изучите его. Попытайтесь понять, кто реально ходит на текущий сайт, является

фактической онлайн-аудиторией бизнеса. И в чем возможные причины проблем, на которые жалуется клиент.

Могут всплыть противоречия. Например, заказчик говорит: «Наш журнал читают представители всех возрастных и социальных групп». Типичная, кстати, галлюцинация. А по метрикам видно, что добрая часть визитов — от женщин сильно за 50.

На это нужно обратить внимание. Обозначить задачу: сделать будущий сайт привлекательным, в том числе для ожидаемой аудитории — то есть, сделать так, чтобы тот журнал захотели прочитать, например, 16-летние хипстеры. И тут уже не редизайном пахнет. А сменой концепции журнала. «Спасибо» вам за такие выводы не скажут. Но цели могут скорректировать.

Тут же стоит обратить внимание на:

1. Точки входа.
2. Какие страницы пользуются популярностью. Какие — никому не нужны.
3. Показатели отказов.
4. Читаемость. Время сессии.
5. Поведенческие факторы. Кликабельность тех или иных элементов.
6. Реальную демографию.
7. Интересы пользователей.
8. Поисковые запросы (их можно взять за основу семантического ядра).
9. Какие события настроены в аналитике.

Безусловно, это нужно учесть при редизайне. Например, сохранить адреса популярных страниц (или, как минимум, поставить с них **301 редирект**).

Как правило, мы делаем такой срез по статистике с краткими выводами и вставляем его в отдельную вкладку Агрегации требований. Скриншотить все экраны Google Analytics смысла нет. Акцент делаем на те моменты, которые **влияют на структуру** будущего проекта.

4.2.3.2 Аватары. Анализ целевых персон

Анализ целевых персон (он же — метод Аватаров) — ключевая вкладка Агрегации требований. Метод перекочевал в digital из классического маркетинга.

Суть: нужно выделить несколько (три-семь) целевых групп пользователей продукта и одушевить их. Сделать реальных людей: с фотографией, именем, мотивами и проблемами. Кто будет (должен быть) аудиторией нашего сайта или приложения, чем эти люди живут, какие имеют потребности и боли и что мы можем сделать, чтобы эти боли разрешались в нашем проекте.

Аватар — это вымышленный персонаж, в котором отражены основные характеристики целевой аудитории.

4.2.3.3 Как выделить группы

Бывшая говорит, что я никогда
не найду такую девушку, как она...
Вы все, черт подери,
ОДИНАКОВЫЕ !!!



svetik_kissa4931
Бред...

Все люди «как бы» разные. И у каждого проекта своя, «как бы» неповторимая аудитория. Ключевое слово — **«как бы»**. Анализируя частные случаи, можно выделить закономерности и общие черты, которые позволят объединить людей в крупные группы.

Группы нужно выделять по значимым для проекта особенностям, и демографические характеристики почти никогда этими особенностями не являются, отталкиваться нужно от поведенческих и социальных моделей.

Нет смысла выделять группу «женщины средних лет», поскольку она одинаково подходит и для онлайн-магазина Ozon, и для сайта стоматологической клиники. То есть, не ответит на вопросы о потребностях в отношении вашего проекта, если он, к примеру, посвящен продаже швейных машин. Зато в аудитории проекта помогут разобраться такие персоны как «Новички» и «Вышивальщицы», так как у них очень разные запросы в отношении продукта, а значит, и схема действий при его выборе на сайте.

4.2.3.4 Гипотезы сегментов. Эксперты

Итак. Сегмент классифицируется в первую очередь поведением. И в любом случае сегментация — это гипотеза. Однако как составить такие гипотезы, если у вас нет никаких идей на этот счет?

Поговорите с экспертами. В заказной разработке это сам заказчик. Его продавники, саппорт, доставка, стейкхолдеры. Для продуктовой разработки мы можем использовать JTBD-подход (об этом ниже, в параграфе 4.3.7), чтобы сформировать и протестировать гипотезы.

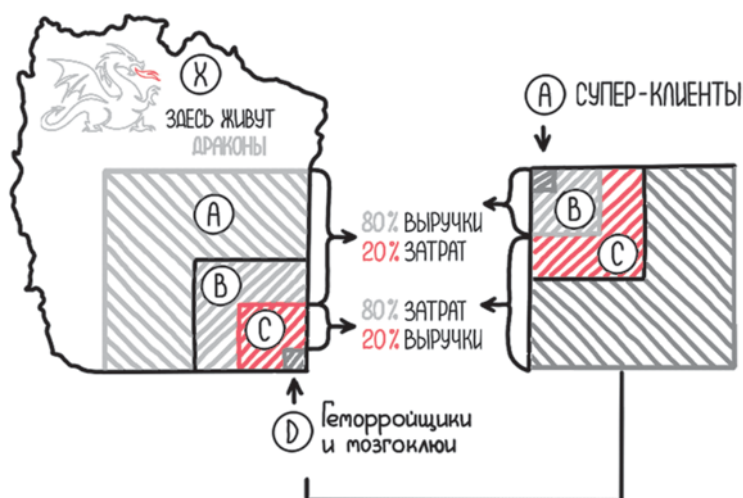
4.2.3.5 ABCDX-сегментация

Мы всегда готовы прийти к вам на выручку.
Была бы выручка!

Как в заказной, так и в продуктовой разработке (да и, наверное, не только в разработке) мы можем сегментировать клиентов по выручке и затратам. Ребята из ФРИИ назвали это ABCDX-сегментация.

\$ ВЫРУЧКА 😊

↓ ЗАТРАТЫ ☹️



- ▶ А-сегмент. Супер-клиенты. Замечательно платят, быстро покупают и принимают решение. С ними просто по кайфу работать и решать вопросы ❤️
- ▶ В-сегмент. У них есть возражения. Чего-то не хватает. Но в целом все ОК. Средний чек — высокий. Цикл сделки — относительно короткий. Платят — регулярно.
- ▶ С-сегмент. Продавать им можно сто лет. Продажники в мыле. Саппорт в ужасе. Бухтеть они будут по любому поводу. Что бы вы ни делали. Все не то. Все не так. Но иногда покупают. В заказной разработке на поддержке — один такой проект весит как пять нормальных. Разработка завалена невменяемыми тикетами. Заработаете на них вы три копейки. А отвалятся они запросто. И останутся недовольными при любом раскладе. В продуктовой разработке, если идти у них на поводу — еще и продукт испортите, и А/В-сегменты потеряете.
- ▶ D-сегмент. Просто выносят мозг. И ничего не покупают. И, кстати, слава богу! Если они что-то купят — тут вы проклянете все. Проджект-менеджеры сгорают как свечи на таком проекте. Программисты тихо матерятся и разбегаются как тараканы. Дизайнеры спиваются.

- X-сегмент. Потенциально ваш сегмент. Такой же, как и супер-клиенты из сегмента А. Но по каким-то причинам вы до них не можете добраться. Продукт не очень подходит, продажник рожей не вышел и т. д. Это наша неизведанная территория. Там живут драконы. Направляем туда спецназ и потихоньку отвоевываем новые земли.

А+В-сегменты приносят 80% выручки при 20% затрат. Именно на них нужно фокусировать сильные команды, ставить хороших менеджеров и сэйлзов. Сэйлзам надо доплачивать за таких клиентов и давать медали.

С+D-сегменты приносят 20% выручки, но сжигают 80% сил.

D-сегмент должен по умолчанию гореть в аду. Ваш звонок очень важен для нас, пи-пи-пи.

С-сегмент сложно вычислить в моменте. По крайней мере, я тут часто ошибаюсь. Нужна статистика за 3–6–12 месяцев. К сожалению, в заказной разработке этого времени достаточно, чтобы спалить команду. Именно это делает работу с С-сегментом крайне нерентабельной в долгосрочной перспективе. В продуктовой разработке С-сегмент еще можно попробовать автоматизировать. Главное, не берите от них ничего в бэклог — испортите продукт. Но в заказной разработке автоматизировать ничего не получится. Либо страдать с пользой, либо сматывать удочки. Ничего личного, просто бизнес.

Итак. Один из способов сегментации — ABCDX, где мы учитываем выручку и затраты. И концентрируемся на тех сегментах, с кем легко и по кайфу делать классные вещи.

4.2.3.6 Сегментация по цене. Как менять цену продукта

Это самая очевидная сегментация. Оптовики-розница. Премиум-дешман. Чем выше сегмент, тем, как правило, он меньше.

В экономичном сегменте все решает цена.

В сегменте чуть выше экономичного (Low middle) сидят самые геморройные клиенты (помните сегменты С и D?). Задают много вопросов, вечно всем недовольны, меняют поставщиков как перчатки и ничего не покупают. А если купят — начнется ад! Но если вы научитесь с такими работать и зарабатывать — это прорыв.

High middle — выше среднего. Прекрасный сегмент. Цена не так чувствительна, а выбор идет между средним и премиальным аналогом. Им важен бренд.



Premium. Эти ребята будут выбирать скорее долговечные и надежные продукты и готовы за это доплачивать.

Люкс или Luxury. Роскошь. Большинство людей не может себе это позволить. Вертолет «Еврокоптер» или автомобиль Maserati, например. Бренд, эксклюзивность, высокая цена, маленькая тиражность. Это то, что важно для этого сегмента.

Однако иногда случается чудо, и продукт используют все сегменты. От экономичного до премиального. В этом случае использовать ценовую сегментацию сложно (только опциями). Пример: популярные операционные системы, социальные сети, iPhone. Пожалуй, нет какого-то способа специально «попасть» в кросс-сегментный продукт, но это потенциально самый большой рынок.

Вообще решение о ценовой политике продукта — одно из самых ключевых. Будете вы продавать мало, но за дорого? Использовать **фримиум-модель** или настаивать на продаже сразу и за дорого? Подписка или единоразовый платеж? Все это очень сильно влияет на продукт и попадание в сегмент.

Менять ценовые политики после запуска — довольно сложно. Старые пользователи очень чувствительны к таким изменениям. Как минимум, вы должны сохранить для них цены на достаточное время. Посмотрите на эту прелесть:

Персональные, мать его, предложения. Внутри — декларативное увеличение тарифов. Милота какая, видимо, нашли раздел, который никто не читает.

Персональные предложения

Индексация тарифов с 1 июля 2021 г.

Новое

Уважаемый клиент, с 1 июля 2021 г. изменяются тарифы по РКО, ВЭД и универсальным бизнес-картам. До 30 сентября 2021г. по пакетам услуг «Быстрый рост» и «Свой бизнес» внесение денежных средств по бизнес-карте через АТМ без комиссии

Предложение действительно до 07 июля 2021 года

Ознакомиться

Не делайте так в своих продуктах. Да, бывает не угадали с юнит-экономикой (о ней поговорим позже, в параграфе 4.4). Или пришло время. Но не надо прятать такие вещи. Говорите открыто с пользователями, как есть. Кто-то отвалится, кто-то будет нечувствителен к повышению. Кто-то все поймет.

Старайтесь сохранить на какое-то разумное время (месяц/квартал/полгода/год) условия и тарифы для старых пользователей. Ведите себя этично в своих продуктах. Аминь.

Итак. Ценовая политика продукта — один из самых важных параметров. Помните об этом!

4.2.3.7 B2C, B2B, B2G-персоны

Если с B2C (business-to-client — бизнес для клиентов) персонами все более-менее понятно, для B2B (business-to-business — бизнес для бизнеса) и B2G (business-to-government — бизнес для государства) есть одна типовая ошибка: обобщение.

Конечным программным продуктом будут пользоваться не компании. А конкретные люди в этих компаниях. Бухгалтер. Менеджер по закупкам. Директор по логистике. И так далее.

Строить персоны здесь нужно исходя из **роли** пользователя, бизнес-процессов или структуры **информационных потоков** предприятия-кли-

ента. Как принимаются решения, кто на них влияет, какая информация нужна на каждом из этапов.

В моделировании такой персоны следует учитывать следующие характеристики:

- ▶ личные качества;
- ▶ должностные обязанности;
- ▶ стремления;
- ▶ недостатки.

4.2.3.8 Гиперсегментация

Болезнь редкая, но встречается. Заказчик просит выделить десять и более целевых персон. И всех их рассмотреть детально. О чем нам это говорит?

1. Менеджер на стороне клиента мог начитать книги, проникся идеей сегментирования и хочет сегментировать все и вся, или просто хочет получить как можно более толстую аналитику. Чтобы аналитик наработался. Плохо, потому что требует кратного увеличения ресурсов.
2. Нет вменяемого квалифицирующего фактора, который бы действительно выделял сегменты по поведенческим паттернам. Разные персоны могут отличаться полом, демографией и т. д., но это толком ни на что не влияет. В продукте они будут вести себя одинаково. И нет смысла их расписывать. Это только все запутает и создаст белый шум. Аналитикой будет невозможно пользоваться.
3. Если продукт такой, что там действительно 20 принципиально разных квалифицирующих факторов — скорее всего, основатель очень сильно расфокусирован. Делает все для всех. В итоге получится гипер-сложная, никому не нужная ерунда.

Начните с пяти-семи сегментов. Но проработайте их хорошо. В дальнейшем, после запуска продукта и при работе с рекламными компаниями, вы можете досегментировать рынок, например, по каналам. И усовершенствовать продукт или рекламные кампании. Однако на старте это только все запутает и сожжет ресурсы.

Итак. Гиперсегментация на старте разработки — это вредная затея. Рассматривайте пять-семь персон — этого достаточно.

4.2.3.9 Бредосегментация

Как-то в офис к нам пришли три представителя заказчика. Отец — собственник завода сельхозтехники. Сын — исполнительный директор этого же завода. И маркетолог. Отец, сын и маркетолог — обычное, кстати, дело. Аминь.

Из аналитики



Мария, 36

Менеджер по закупкам
-хочет быстро изучить
ассортимент и скидки



Иван, 28

IT-специалист
-хочет понять, не будет
ли проблем с внедрением,
хорош ли саппорт, изучить
технические характеристики



Настенька, 25

Оператор
-хочет платьишко, шардоне,
затуж и в Амстердам,
ненавидит зануду-Ивана
и мырму из отдела закупок

Реальные



Отец

Директор
компании-
заказчика



Сын

Замдиректор
компании-
заказчика
-хочет Настеньку
и красный Феррари



Маркетолог

Маркетолог
-хочет з/п побольше,
ненавидит аналитика
и все целевые персоны

Реальная аналитика целевых персон их мало интересовала. У них было свое видение проекта. Причем, у всех — разное, и этот конфликт мы вскрыли и всех помирили. Но факт остается фактом — в заказной разработке обычное дело, когда влияние целевых персон из аналитики на продукт будет ничтожным по сравнению с влиянием стейкхолдеров. Мы поговорим еще про это подробно в главе про галлюцинации основателей на аналитике (§ 4.2.6). Однако нельзя отбрасывать тот факт, что в заказной разработке приходится вскрывать и решать конфликты стейкхолдеров и увязывать их мнение с реальностью. Это сложно. Но нужно.

Итак. В заказной разработке нужно принимать во внимание не только целевых пользователей продукта, но и целевые персоны заказчика. Мы их рассматривали во вкладке «Видение». Однако если между интересами стейкхолдеров и пользователями есть потенциальный конфликт — нужно его решать.

4.2.3.10 Как описать персону

Так, чтобы в нее поверили.

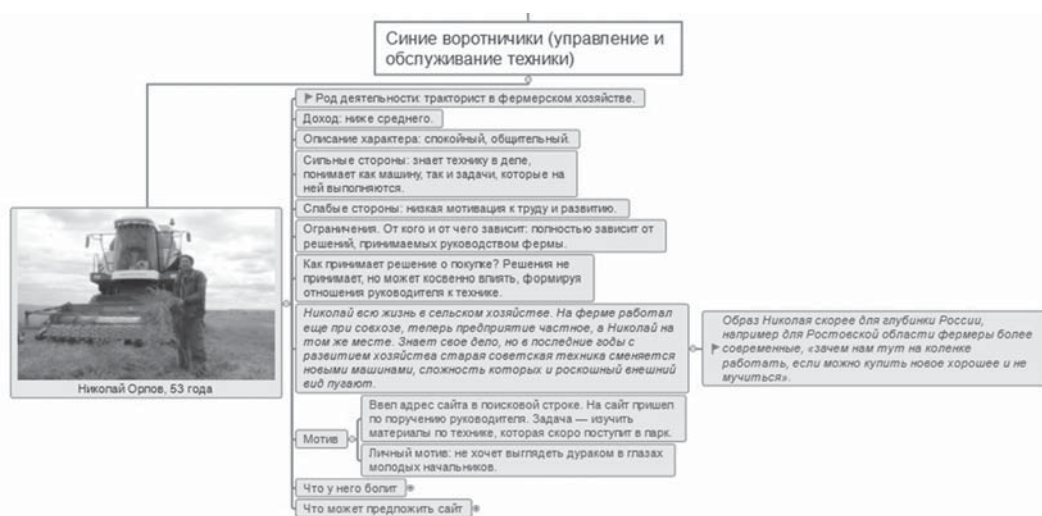
Если назвать персонажа Вася Пупкин и добавить дурацкую фотку со стока, в Васю никто не поверит. Как и в выводы, сделанные такой персоной. То есть, дурацкая фотография или нелепое имя напрочь убивают доверие к аналитике. Какая бы качественная работа ни была дальше.

Подбирайте характерные для реальной группы фотографии. Придумывайте нормальные человеческие имена. И отмечайте в описании детали, которые придают персонажу характер. Что-то, что позволит слушателю узнать в нем реального человека, с которым приходилось работать.

В идеале стоит проиллюстрировать сказанное реальными скриншотами с форумов, социальных сетей и отзывов. Выдернуть настоящих потребителей. Например, из CRM-системы клиента. Это не всегда возможно. Но попытаться стоит.

Описывая характер, ограничения и подход к принятию решений, говорите именно с точки зрения взаимодействия с будущим программным продуктом. То есть, если наш сайт посвящен продаже товаров на B2B-рынке, для нас неважно, что Иван — крепкий семьянин. Но важен его опыт в закупках и глубокое знание рынка.

Персоны удобно хранить в Google-таблицах или интеллектуальных картах.



Целевая персона РостСельМаш в XMind

Пример.

Для компании в музыкальной индустрии (скажем, звукозаписывающей студии) можно смоделировать таких персонажей:

1. Михаил, 45 лет, преподает игру на скрипке в музыкальном колледже, пользуется интернетом менее года. Выходит в сеть исключительно с домашнего компьютера через широкополосное соединение. Никогда не совершал покупки посредством интернета, предпочитает делать заказы по телефону.

2. Анна, 29 лет, исполнительный директор. Активный пользователь интернета в течение последних пяти лет, для выхода в сеть использует все, что есть под рукой: MacBook, iPad или свой iPhone.

Таким образом, потенциальными клиентами компании являются совершенно разные типы людей с абсолютно разными потребностями.

В некоторых случаях смоделированные персоны можно применять и в таком виде. Однако полностью проработанные персонажи имеют более подробное описание с указанием тех целей, ради достижения которых они зайдут на сайт компании.

Оживить персонажей помогают небольшие истории:

«Николай всю жизнь в сельском хозяйстве. На ферме работал еще при совхозе, теперь предприятие частное, а Николай на том же месте. Знает свое дело, но в последние годы с развитием хозяйства старая советская техника сменяется новыми машинами, сложность которых и роскошный внешний вид пугают».

«У Андрея советский тип воспитания, его отец, а теперь и он сам считает зазорным нанимать кого-то для ремонтных работ, это не только затратно, но и попросту недостойно. Семья расширяется, получены ключи от ипотечной трешки без ремонта. Андрей закупает материалы небольшими партиями, по шагам под каждую задачу. Так легче для семейного бюджета, да и движется процесс небыстро из-за работы и времени в пути до квартиры».

4.2.3.11 Верхнеуровневые мотивы

Зачем пользователь зашел на ваш сайт или открыл мобильное приложение (или что вы там разрабатываете?). Какую задачу он на самом деле решает? Вот это и есть — мотив. Чуть позже мы посмотрим на концепции CJM и Jobs To Be Done, чтобы разбивать мотивы на шаги.

То есть, для нас важен мотив некого Андрея найти магазин для выгодных и удобных онлайн закупок стройматериалов, а не то, что он хочет сделать ремонт в своей квартире. Это, конечно, тоже важно и легло в основу истории. Но мы моделируем персону именно с точки зрения взаимодействия с сайтом. А значит, формулируем мотив посещения сайта.

На этом этапе обдумайте, как пользователь мог узнать о сайте. Какое действие на сайте будет ключевым для данного пользователя. И как в целом будет строиться его путь.

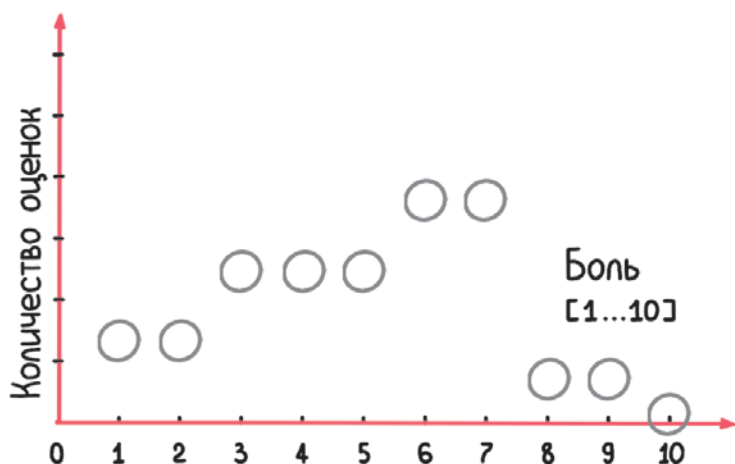
4.2.3.12 Боль. Сила боли

Нет проблемы — нет продажи.

По каждой целевой персоне нам нужно понять, что у нее **«болит»!** Именно из этого вытекает набор функций, который мы должны предусмотреть на нашем проекте.

Например. На B2B-сайте мне как менеджеру по продажам важно параллельно работать с несколькими корзинами, поскольку я формирую и уточняю одновременно несколько заказов в течение дня. Для B2C такие опции, наоборот, навредят — получим слишком сложный интерфейс и брошенные корзины. Если в проекте есть обе персоны и они равнозначны — значит, нужно предусмотреть выбор профиля. Ух!

Под болями мы подразумеваем препятствия, переживания, проблемы, барьеры, с которыми сталкивается человек на пути к своей цели. Нам необходимо устранить эти барьеры. Формулируя боль, сразу отвечайте себе на вопрос: каким образом она будет закрываться в нашем программном продукте.



Для определения приоритетов разработки нас будет интересовать два параметра: как часто «болит» и насколько сильно. И то и другое помогает выявить Проблемное интервью CustDev (см. § 4.3.8).

Как ни странно, даже в медицине, боль — субъективный фактор. Объективного «болиметра» не придумали. Кто-то от прививки в обморок падает. А кому-то лом пролетел через голову, в результате чего у него слегка испортился характер. Многие люди склонны драматизировать силу боли. Особенно свежих событий или событий, на которых они сфокусированы прямо сейчас. Отсекайте такие «выбросы» относительно средних величин.

К примеру, в офис заказчика вашего программного продукта пришел дедушка и устроил скандал, что «на сайте нет версии для слабовидящих». Он чего-то там не разглядел и не туда нажал. Откомпостировал мозг всем по дороге. Досталось даже генеральному директору. Его все очень хорошо запомнили. Однако, если это первая такая жалоба за 10 лет — бежать и экстренно реагировать не стоит. Может, дедушка со справкой, и он так свою значимость миру показывает?

Итак. В Агрегации требований для каждой персоны мы фиксируем боли. Если есть возможность собрать статистику по частоте и силе — это хорошо.

Не формулируйте боли в виде вопроса. Почувствуйте разницу:

«Не испачкаю ли я руки, когда буду есть чебурек?»

«Чебуреки жирные! С них капает сок. Их невозможно есть так, чтобы ничего не текло во все стороны. Каждый раз, когда я покупаю чебурек, я боюсь, что испачкаю одежду. Жир очень плохо отстирывается. А уж ходить целый день в грязной одежде — это мерзко. Что обо мне подумают люди? Особенно вон та мырма из соседнего отдела! Мерзко, на десять из десяти!».

То есть, боль не в том, что чебурек капает. А в том, что человеку стремно запачкать одежду — что подумают люди! И это может быть главным мотивом отказа от покупки.

4.2.4 CJM. КАРТА ПУТЕШЕСТВИЯ ПОЛЬЗОВАТЕЛЯ (CUSTOMER JOURNEY MAP)

В заказной разработке клиенты чаще и чаще стали спрашивать Customer Journey Map — карту путешествия клиента. Это наглядный инструмент для анализа проблемных точек, которые мы создаем пользователям своим сервисом. Инструмент старый. Пришел в digital из классического маркетинга. И очень вероятно, что руководитель проектов с ним рано или поздно столкнется. В чем идея?

В идеале для каждой целевой персоны (аватара) нам нужно создать и проанализировать его путь от того момента, как у него возникло смутное подозрение, что есть какая-то проблема, которую нужно решать,



до того момента, как он становится приверженцем бренда и готов его рекомендовать. Впрочем, многие так далеко не думают — ограничиваются первой покупкой. Но мы будем настойчивы!

Итак. Наш покупатель проходит через 6 крупных, условных фаз:

1. Awareness. Осознание. Осведомленность. Что-то его смущает, но что именно — он не очень понимает. Например, человеку хреновенько на работе. Он не так жжет, как раньше, и ничего не успевает. На этой фазе он начинает гуглить, думать, что с ним не так, читать статьи про эффективность и т. д.
2. Consideration. Изучение, осведомление. Например, человек понял, что ему нужен тайм-менеджмент, и именно в самоорганизации его проблема. Теперь он ищет инструмент, в котором бы мог структурировать все свои дела. Наивный!
3. Decision. Выбор и принятие решения. Решение для человека в целом понятно. Хочется выбрать что-то конкретное. Для тайм-менеджмента это может быть выбор между блокнотом, календарем или программами ведения списка дел и проектов. Он читает рекомендации, смотрит, экспериментирует, проверяет и пробует сам.
4. Purchasing. Покупка. Наконец, все понятно. Выбор среди множества вариантов сделан. Победит тот продукт, который будет проще купить. Будут ли вежливы продавцы? Нужно ли будет куда-то идти или с кем-то разговаривать? Все это влияет на легкость покупки и пользовательский опыт.
5. Loyalty. Лояльность. Появляется первый пользовательский опыт использования продукта. С офлайн-продуктами это может быть как опыт доставки и монтажа, так и опыт использования самого продукта. В случае продуктов, которые пользователь покупает регулярно — он может остаться с этим поставщиком, стать лояльным. А может в ужасе сбежать в поисках другого решения.
6. Advocacy. Адвокат бренда. Наконец, если все очень хорошо — человек может рекомендовать бренд.

CJM обычно представляет собой табличку. По горизонтали мы выписываем шаги или фазы, через которые проходит пользователь. По вертикали — те параметры, которые хотим отслеживать. Как правило, это:

1. Цели и ожидания пользователя на каждом этапе. Его активности.
2. Вопросы, которые задает себе покупатель.
3. Мысли и чувства пользователя в этот момент.
4. Точки контакта. С чем приходится взаимодействовать пользователю. С сайтом, Инстаграмом¹ компании, службой поддержки, электронным письмом и т. д. Тут есть одна тонкость, про нее напишу отдельно.
5. Уровень счастья / удовлетворенности пользователя на каждом из этапов.
6. Ожидания. Только не в духе Аршавина: «Ваши ожидания — ваши проблемы».
7. Барьеры. Что мешает, какие есть сложности. Провести анализ глазами клиента, проанализировать отзывы и обратную связь, пожелания и жалобы клиентов.
8. Решения. Способы преодоления барьеров. Штормим, думаем, как облегчить жизнь.
9. Дополнительные параметры. Например, ответственные за этап, перечень ключевых запросов и т. д.

[illegible]

CJM по одному из типов пользователей приложения SingularityApp. Фрагмент из Google Docs.

Проще всего построить такую таблицу в Google Docs. Получается практичный инструмент, в который нужно регулярно заглядывать и обновлять.

¹ Социальные сети Facebook и Instagram запрещены на территории Российской Федерации на основании осуществления экстремистской деятельности.

Стадии			
	1. Awareness: Осознание / Осведомленность "Не знаю, чего хочу / пытаюсь осознать проблему"	2. Consideration: Изучения, или Осведомления "Понимаю свою проблему, ишу решение"	3. Decision: Выбор и принятие решения "Решение принято, иду у кого купить"
Цель покупки, Активности	найти корпусную мебель для гостиной по приемлемой цене, со стильным дизайном, удобную в использовании	получать мнения от реальных покупателей и пользователей, которым клиент доверяет	определившись с компанией, у которой будет производиться заказ мебели. Понять какие конкретно виды мебели нужны для гостиной
	Хочет подарить другу подарок на новый год	Ищет подарки по ключевым словам	изучение сайта — выбор конкретных моделей, получение первичной информации о мебели и ценах
Вопросы (которые задает себе покупатель)	В чем проблема?	Что может решить мою проблему?	У кого лучшее предложение и больше экспертиз?
Точки контакта	Поисковая система Яндекс или Google	Google	Форумы — Зайти на форум — Открыть список статей — Использовать поиск на форуме
	Друзья и знакомые		
	Контекстная и таргетированная реклама	Facebook	None
	Instagram		
	Реклама на билбордах	Yandex	Прайс на сайте
	Лидеры мнений		
Уровень удовлетворенности, Чувства, Опыт (текущее состояние)	● Тревога. Растерянность. Неосознанная проблема.	● Нашел решение	● Несчастлив
	Приятно сделать подарок другу	Нравится большой выбор, но не знает, как выбрать	Не нравится
Ожидания	Получить легкий доступ к информации о скидках	Релевантные результаты поисковой выдачи	Понятный, современный дизайн на сайте
			Телефон
			Наличие цен

CJM. Xmind. Фрагмент

Однако CJM часто красиво визуализируют, рисуют чуть ли не в Фотошопе, добавляют графики. И из рабочего инструмента CJM вырождается в маркетинговую пыль в глаза для красивых презентаций. Это первый минус CJM: из инструмента он превращается в имитацию инструмента. Хотя если важны красивые презентации, CJM производит впечатление.

Второй минус — точки контакта. Их может быть очень много на каждом этапе. Сценарий перемещения по этим точкам может быть нелинейным и крайне запутанным. И содержать агентов влияния (вроде жены, мамы, подруги), общение с оператором, пересылку ссылок на избранное, корзину или оплату.

По-хорошему, нам нужно развернуть весь путь пользователя в цепочку шагов. Но это будет безумно широкая таблица, которой будет невозможно пользоваться. А как учесть разные маршруты и возвраты на предыдущие шаги — вообще не очень понятно. Поэтому ограничиваются наиболее типовыми маршрутами.



Третий минус — трудоемкость. По-хорошему, нам нужно строить такие карты для каждой целевой персоны и для каждого маршрута. Но это недели, если не месяцы, работы аналитика. Дорого и непонятно зачем. Если же рассмотреть какой-то общий сценарий — карта получается слишком неконкретная.

В заказной разработке обычно мы концентрируемся на той части CJM, которая касается взаимодействия пользователя с сайтом. И рассматриваем лишь самые важные маршруты для самых частых персон. В такой форме это оправданно и позволяет улучшать пользовательский опыт в программном продукте.

Итак. CJM — инструмент для анализа пользовательского опыта на каждом шаге и точке контакта при взаимодействии пользователя с брендом. Инструмент неплох для заказной разработки или для анализа готового программного продукта. В продуктовой разработке лучше использовать фреймворк RAT+JTBD (об этих аббревиатурах — в параграфах 4.3.3 и 4.3.7). Важно найти баланс между трудоемкостью и практичностью. А еще CJM любят вставлять в красивые презентации. Если это произошло — значит, CJM уже мертв.

4.2.5 РЕШЕНИЯ

Аналитика никому не нужна.
Нужны выводы из аналитики.

Итак. Мы определились с сегментами. Описали целевые персоны. Выявили их боли и мотивы.

Все это было проделано, чтобы понять, какие вещи (функции, разделы, экраны) мы должны предусмотреть в будущем проекте для того, чтобы эти боли закрыть.

Какими должны быть предложенные решения?

В первую очередь, не случайными, а закрывающими выявленные выше потребности персоны. Например: мы делаем отзывы со ссылками на соцсети не просто так, а потому что выяснили, что наш персонаж Леночка не решается покупать косметику без отзывов, но не так наивна, чтобы верить анонимным отзывам на сайте интернет-магазина.

Важно, чтобы решения отвечали уровню проекта. Если заказчик хочет простой корпоративный сайт, можно предложить интеграцию с чем-то диковинным, калькулятор с 3D-моделями. Но аккуратно, с оговоркой, что это идея на развитие. Иначе у заказчика останется неприятный осадок, что его не слышат либо пытаются развести на дополнительные опции.

Как правило, мы описываем предлагаемые решения сначала рядом с каждым аватаром, чтобы четко прослеживалась связь решений, болей и мотивов. Для каждой персоны решения размечаем цветными рамками (цветовое кодирование: одна персона — один цвет). Используем иконки для разметки этапов, акцентов или вопросов, которые нужно разобрать.

И ниже, в этой же вкладке Агрегации требований делаем сводную таблицу всех функций, также размеченную цветами. Не только чтобы было легче воспринять будущий проект, но и чтобы можно было отследить источник боли — то, ради чего эта функция была придумана. Пока не увлекаемся структурированием. На этом этапе нам важен сам список. Компоновать по разделам и продумывать перелинковки будем чуть позднее.

Описывайте решения достаточно подробно, чтобы было ясно, в чем фишка и как она будет работать на решение боли.



Итак. Из болей и мотивов следуют решения (но ими не ограничиваются). Предложите решение на каждую боль. И далее сведите все решения в общий список.

4.2.6 АНАЛИЗ САЙТОВ КОНКУРЕНТОВ

Кто знает врага и знает себя, не окажется в опасности и в ста сражениях. Тот, кто не знает врага, но знает себя, будет то побеждать, то проигрывать. Тот, кто не знает ни врага, ни себя, неизбежно будет разбит в каждом сражении.

Сунь-Цзы. Искусство войны

Нам нужно посмотреть и изучить проекты конкурентов, чтобы понять:

1. что у них есть прикольного, что имело бы смысл сделать у себя;
2. в чем у них есть лажа, и каких ошибок следует избегать.

Тут важно отметить и для себя, и для заказчика, что это не анализ рынка с попыткой разобраться, кто и почему зарабатывает на этом рынке больше, и не анализ маркетинговых кампаний — мы смотрим на конкуренцию только с позиции сайтов/мобильных приложений. Выявляем позитивные и негативные тренды, отмечаем особо интересные фишки.

Первое, что нужно рассмотреть — проекты, на которые сослался сам заказчик. В большинстве случаев заказчик знает конкурентов. Однако с тем же успехом имеет ряд галлюцинаций по этому вопросу. Вот ТОП-3:

1. **«У нас нет конкурентов».** Не верьте. Либо что-то не договаривает, либо что-то не понимает. Логика там примерно такая: «Мы делаем соцсеть для котиков со встроенным маркетплейсом. Ни у кого больше нет соцсети для котиков со встроенным маркетплейсом. Поэтому у нас нет конкурентов!» Логично? Хоть глупо, но логично! Посмотрите, как люди решают ту же самую проблему, которую решает компания-заказчик.
2. **«Наши главные конкуренты-враги — компания “Копыта и Рога” с соседней улицы! Они даже логотип сделали почти как у нас. С рогами и копытами».** Это обычно высказывается на эмоциях. Заказчика очень бесит какой-то местный, локальный конкурент. И он считает его главным врагом. На самом деле, в интернете он будет конкурировать с какой-нибудь Икеей, Wildberries или Озоном. Но думать про это — бздляво. А вот «Копыта и Рога» с соседней улицы — в самый раз.

Анализируйте не только конкурентов по отрасли, но и по поисковой выдаче, и по рекламным кампаниям, и по позициям в популярных маркетплейсах. Смотрите и прямую, и косвенную конкуренцию.

3. **«Все сайты наших конкурентов — ужас и кошмар. Там нечего смотреть».** Редко, но бывает. Особенно в B2B. Тогда стоит посмотреть смежные отрасли:
 - a. Как аналогичные вещи решаются на зарубежных сайтах?
 - b. Какая отрасль похожа на эту? Есть ли там сильные игроки? Как выглядят их решения?

По реальным конкурентам мы смотрим как плюсы, так и минусы. Но минусами не нужно увлекаться, описывая каждую опечатку. Только значимые для процесса продажи ошибки.

Описывая плюсы и минусы, не забывайте маркировать их иконками, иллюстрировать картинками и главное — понятно рассказывайте,

в чем суть фишки или ошибки, как это влияет на продажи, стоит ли повторять решения и в каком виде.

Если же среди реальных конкурентов что ни сайт, то ужас и кошмар, разработанный на заре интернета (а в некоторых отраслях до сих пор такое случается), то не тратьте время на подробный разбор — дайте общее заключение.

Дополнительно можно рассмотреть сайты в смежных тематиках. Например, если ваш проект про доставку пиццы, сайты по доставке роллов также могут быть источником хороших идей.



В зарубежном сегменте, в идеале, нужно смотреть ту же сферу, что и ваш проект, но если там ничего примечательного нет, то смотрите просто хоть сколько-нибудь релевантные теме крутые сайты, предлагайте фишки.

4.2.7 СЕМАНТИЧЕСКОЕ ЯДРО (ОПЦИОНАЛЬНО)

Дополнительно вы можете проанализировать, по каким запросам пользователи ищут решения своих проблем. Я считаю, что на этом этапе такой анализ несколько избыточен, так как касается, скорее, не структуры проекта, а его контентной части. Нас интересуют функции с точки зрения программного кода, а не конкретный перечень товаров в каталоге. Более того, пока проект в разработке, ситуация десять раз поменяется. Однако, если ресурсы проекта, бюджеты и компетенции позволяют (а там добавляется 20–40 часов работы) — можно собрать предварительное, базовое семантическое ядро и посмотреть, нужно ли предусматривать в структуре проекта какие-то функции.

Существует множество сервисов автоматического сбора семантических ядер вроде Key Collector. Проще всего собрать запросы вручную с помощью онлайн-сервиса Яндекса — Wordstat. Фиксируйте не только текст запроса, но и число обращений к нему.

Наша задача на этом этапе — понять и показать значимость ключевых страниц и перечень основных запросов, по которым на эти страницы может идти трафик.

Для некоторых интересных, но недостаточно раскрытых в структуре запросов по тематике проекта можно рекомендовать создание отдельных посадочных страниц. Примерно так:



4.2.8 СТРУКТУРА БУДУЩЕГО ПРОДУКТА

Кульминация процесса, то, ради чего делались все предыдущие вкладки — составление оптимальной структуры проекта. Оптимальной — значит в ней будут все необходимые страницы и функции для решения поставленных задач и закрытия болей аудитории, но не будет лишних.

Не стесняйтесь отговаривать от реализации очевидно избыточного функционала, который мог попасть в смету по историческим причинам или как хотелка заказчика. Если на сайте три страницы, и все они

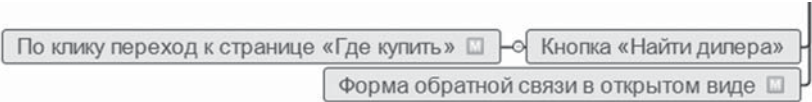
сплошь картинки, то поиск ни к чему. Заказчик оценит, что вы не только предлагаете функционал, что приводит к росту бюджета и сроков, но и советуете пути оптимизации, что, наоборот, экономит его деньги.

Мы описываем структуру достаточно подробно. Не только перечень страниц или блоков на страницах. Вплоть до состава этих блоков: заголовков, подзаголовков. Часто это помогает составить карту контента. Кроме того, по такой структуре потом очень просто сделать смету, бэклоги или контролировать состав элементов на прототипах и дизайне.

Описывая структуру страницы, не забывайте подбирать примеры для сложных и важных элементов. Это могут быть текстовые примеры — например, возможный слоган или перечень преимуществ — и примеры визуальные. Картинка особенно важна, когда показываешь какую-то визуальную фишку, например, в промо-блоке. Оформляйте так:



Если для анимации будет живой пример (ссылка) — вообще отлично. Не повторяйтесь, используйте перелинковку.

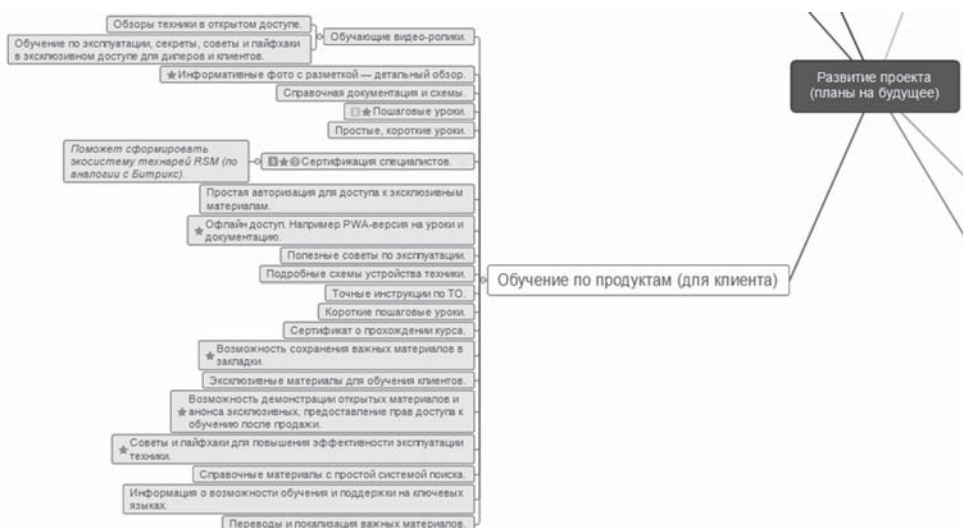


Как выглядит перелинковка: иконка «М», по клику на которую переносишься в нужный раздел агрегации

Мы еще раз вернемся к декомпозиции проектов в главе 6, поскольку нам важно понимать не только набор функций, но и техническую возможность их реализации. Например, **ERP-система** заказчика может быть просто не готова к выгрузке данных в нужном нам виде, и потребуется очень много работы, чтобы это реализовать. Но пока не забиваем этим голову.

4.2.9 ПЛАНЫ НА БУДУЩЕЕ. РАЗВИТИЕ ПРОЕКТА (ОПЦИОНАЛЬНО)

Это последняя вкладка Агрегации требований. В ней мы отражаем функции, идеи и предложения, которые нет смысла реализовывать прямо сейчас. Может, через полгода к этому вернемся. Может — через год. Но идеи хорошие, их жалко выбрасывать.



4.2.10 СТРОИМ ПРОЦЕСС АГРЕГАЦИИ ТРЕБОВАНИЙ В ЗАКАЗНОЙ РАЗРАБОТКЕ

Итак. Мы начали с того, что описали видение проекта. Далее проанализировали текущее состояние проекта (если такой был). Посмотрели демографию, точки входа. Описали целевых персон (аватары). Выявили их боли и мотивы. Предложили решения по каждой из болей. Проанализировали сайты конкурентов (что у них можно украсть, а что — ни в коем случае не повторять). Посмотрели семантику. Сделали подробную структуру проекта, возможно, сразу разметив этапность запуска. Что-то вынесли в долгосрочные планы развития. Тут много кропотливой работы, в том числе — с заказчиком. Как нам ее лучше организовать?

Нам понадобится аналитик, который сможет это все структурировать, зафиксировать и выдать предложения. Заказчик ожидает, что

аналитик будет проактивен, и большая часть идей будет исходить от него. Именно так и нужно строить процесс.

1. В начале изучите всю доступную информацию: сметы, предварительные брифы, вводные от клиента. Сформулируйте вопросы к клиенту. Дайте изучить эту же информацию вашему аналитику.
2. Проведите установочный звонок, совместно с клиентом и аналитиком.
 - a. Если вам передают проект от отдела продаж — в разговоре также должен участвовать тот человек, который совершил продажу.
 - b. Важно на установочный звонок привлечь лицо, принимающее решения. У него может быть гораздо более полная картина мира, чем у руководителя проекта, назначенного со стороны клиента.
 - c. Предупредите клиента, что часть информации, которую обсуждали на пресейле, могла потеряться во время передачи. Это, к сожалению, неизбежно. Поэтому сейчас нужно еще раз полностью обсудить вводные по проекту.
 - d. Пройдитесь по всем артефактам.
 - e. Пусть аналитик задаст все нужные вопросы.
 - f. Очень внимательно слушайте, что говорит клиент.
3. Далее работайте вкладка за вкладкой. Сделали видение. Показали клиенту со своего экрана. Обсудили устно. Внесли корректировки. Переходим к следующей вкладке. Персон лучше разбирать не более трех за раз (и не городить их суммарно больше семи). Обычно нужно два звонка в неделю, чтобы ровно выстроить такой процесс.

На агрегацию требований планируйте не менее двух недель.

Однако если клиент редко выходит на звонки — процесс может сильно растянуться.

4. Владеть документом должен аналитик. У него должна быть целостная и актуальная картина по проекту. Все правки нужно вносить в документ его руками. Иначе будут нестыковки в структуре, которые потом выйдут боком.
5. Как правило, у клиента уже согласован предварительный бюджет на проект. Пусть с какой-то вилкой. Но есть верхняя граница. Однако

в процессе работы над структурой может много чего добавиться. Аппетит приходит во время еды. Сразу помечайте функции, требующие дополнительного бюджета. Это должно быть наглядно. Например, ставьте значок доллара рядом с такой функцией.

6. Редко, но бывает: не все клиенты способны воспринимать формат **Mind Map**. Презентация в Power Point — это их предел. Мне лично тоже нравится вместо вкладки «Видение» использовать формат Lean Canvas. Однако когда дело доходит до структуры — Mind Map просто незаменим. А размазывать агрегацию по нескольким файлам («Видение» у нас в Lean Canvas, целевые персоны — в Powerpoint, структура — просто документ в Word) — откровенно плохая затея. Теряется логика, нужные связи. Тут придется потерпеть.

Вот и все. У нас получится логичный, цельный, подробный документ, по которому понятно, что за продукт, для кого мы его делаем, и почему состав функций именно такой.

4.2.11 КАК ПРОДАВАТЬ АГРЕГАЦИЮ ТРЕБОВАНИЙ

Когда мы говорим об аналитике, то подразумеваем три основных артефакта, которые появляются в ее результате: агрегация требований, прототип и техническое задание на разработку проекта.

Когда заказчик только приходит с каким-то планом будущего сайта или мобильного приложения, он ожидает услышать сумму сделки, хотя бы примерную: «от» и «до». От этого зависит, состоится контракт или нет.

В разработке софта стоимость любого проекта зависит от необходимого количества реализованных функций, их сложности и заложенного качества. Качество — это сложный комплексный параметр, который трудно объяснить двумя словами. И, конечно, стоимость проекта зависит от маржи компании-разработчика — в среднем по рынку это рентабельность в 20–25%. Рейты (вместе с зарплатами программистов) только растут, и, похоже, конца этому не будет. В ближайшее время — точно.

На этапе продажи аккаунт-менеджер строит общий каркас проекта, накидывает основные возможности сайта и рамочно их оценивает (подробно об этом мы говорим в главе о декомпозиции). У заказчика на этом этапе появляется примерное понимание по бюджету и точная

стоимость первого этапа разработки — аналитики. Собственно, на ее основе уже можно заключать сделку.

Почему так? Почему нельзя сразу сказать точную стоимость разработки? Допустим, вы решили построить уютный коттедж. Выбрали несколько подходящих компаний, которые могут это сделать. И в первом разговоре с прорабом задаете ему вопрос: «Сколько стоит дом?» Тут велик шанс, что вы получите обтекаемый ответ: «От 300 тысяч до 50 миллионов». Слишком мало данных для оценки, потому и такой ответ. Бюджет на основании этого вы вряд ли спланируете.

Возвращаясь к разработке и аналитике, на этапе продажи мы определяем, какой это будет дом и сколько у него будет этажей: один или пять. На этапе агрегации решаем, какие в этом доме будут лестничные пролеты, как будут расположены комнаты, и разбираемся, что еще нужно предусмотреть, чтобы в доме было удобно и комфортно всем членам семьи. На этапе прототипирования — делаем детальный чертеж. На нем показываем, что и где будет расположено, какого размера будут комнаты, где находятся лестницы и какой высоты и ширины будут дверные проемы.

В физическом мире проектирование зданий, сооружений — это целая отрасль, в которой в России работает порядка полумиллиона человек. Мало кто делает детальный чертеж, пока не подписан договор на проектирование. Это отдельная большая работа, которая требует времени и профессиональных качеств. Планирование и проектирование дома — не подарок, за это берутся деньги. Вы обсудите задачу, заплатите за отдельный этап, и затем уже перейдете к работе.

То же самое происходит и в сфере проектирования сложных коммуникаций, интернет-сетей и во многих других отраслях. Вы в общих чертах понимаете задачу. Можете дать прогноз по бюджету, за всю работу. Но будет определенный разбег. Однако вы можете сказать точную стоимость аналитического этапа — проектирования. После которого уже можно будет просчитать стоимость и сроки всего проекта. Либо определить, что сможет сделать клиент за свой бюджет, а что придется выкинуть на следующие этапы.

Более того, аналитику время от времени заказывают вообще отдельно. Например, чтобы сформировать тендерное задание. Понятно, что у компании, которая делала аналитику, будет самое глубокое погружение в проект и максимальная ответственность. Но, в принципе, это

отчуждаемый этап, который можно сделать по отдельному контракту и дальше уже решать, на одной волне или нет.

«Назовите точную стоимость без проектирования и аналитики и впишите ее в контракт, бу-бу-бу. А еще нарисуйте нам пять вариантов дизайна. Прямо к завтраму». Я рекомендую отказаться от таких клиентов. Шалаш без плана построить можно. Высотный дом — нет.

Итак. Стоимость проекта может быть понятна только вилочно, приблизительно. Однако стоимость аналитики можно просчитать точно. И точно показать, за что именно клиент заплатит. Вынести эти работы в отдельный этап. После чего уточнить стоимость. И попасть в ожидаемый диапазон цен.

4.2.12 ЧТО ДЕЛАЕМ ПОСЛЕ АГРЕГАЦИИ

В заказной разработке после агрегации нам нужно:

1. Актуализировать смету. Теперь у вас гораздо более четкое видение проекта, и вы можете либо вообще убрать вилки, либо сократить разбег. В смете вы также можете выделить релизы и спринты. Согласуйте смету с клиентом. Кстати, важно либо четко попасть в вилку, либо четко показать, за счет каких добавлений и новых идей ценник получился выше планируемого. Дальше уже принимать решения — оставляем ли все придуманные функции, или что-то выносим на будущие этапы.
2. Сделать прототипы ключевых страниц. Подробнее о том, как построить по ним работу, как выполнить тестирование прототипов и что нужно предусмотреть — мы поговорим в главе 7, посвященной дизайну.
3. Написать бэклог (или, в зависимости от контракта, — ТЗ, которое затем будет разделено на бэклог).
4. Рисовать. Кодить. Запускать версию за версией. При этом без проблем отклониться от изначального плана: что-то сделать раньше, что-то убрать на следующие этапы.

Этот алгоритм проверен сотнями проектов в заказной разработке. И дает стабильно хороший, предсказуемый результат и высокую управляемость. Однако в продуктовой разработке, когда нет внешнего заказчика, нет никакого видения, все зыбко и непонятно (но дорого) — помогают еще несколько техник.

4.2.12.1 ПРОТОТИПИРОВАНИЕ

Итак, агрегация требований готова и утверждена с заказчиком. Самое время приступить к прототипу — черно-белой интерактивной схеме сайта, по которой можно поводить мышкой, покликать по кнопкам и посмотреть, как будет работать проект без затрат на дизайн и программирование. На этом этапе уже видно, в каком порядке будут располагаться основные блоки на странице и где будут находиться кнопки, предполагающие целевые действия.

Цели прототипа:

- ▶ согласовать структуру сложных страниц с заказчиком;
- ▶ проверить, будет ли проект удобен для конечных пользователей с точки зрения персон;
- ▶ убедиться, что все участники проекта одинаково представляют зафиксированные в структуре идеи, а сами идеи жизнеспособны — понятны и удобны для пользователя;
- ▶ предложить альтернативные решения для нежизнеспособных идей (самое время).

Прототипа достойны не все страницы сайта, а только важные и сложные. Список страниц к прототипированию определяется сметой. Главная страница — всенепременно. Если делаем интернет-магазин — то также проектируются список товаров, карточка товара, корзина, форма заказа и личный кабинет.

Если в разделе «О нас» задумана презентация сотрудников с портретами и мини-биографией, чтобы клиент мог выбрать к кому обратиться и тут же оставить ему запрос на обратный звонок (заполнив для этого форму с обязательными и невероятно важными полями) — прототируем. Если планируется фото фасада и три строчки адресов-телефонов — просто обсуждаем с заказчиком и затем фиксируем в ТЗ.

Программ для прототипирования примерно миллион, причем, есть и онлайн-ресурсы, и десктопные программы. В последнее время мы все чаще делаем и прототипы, и проекты в Figma — уж больно она хороша и удобна. Вы же можете выбрать какую угодно программу, главное — следовать принципам:

Думайте, как этим будут пользоваться реальные люди

Будет ли это удобно? Решит ли это их проблему? Поставьте себя на их место.

Выравнивайте за собой везде и всегда

Когда вы готовите прототип, обязательно выставляйте в программе линейки, по которым проверяйте расположение элементов. Это стандартный функционал большинства специализированных программ. Прототип должен быть не только функциональным, но и красивым. Критерий красоты выделить трудно. По крайней мере, при просмотре не должно быть ощущения, что работу выполнили наспех, потому что «все равно же дизайн впереди».

Используйте реальный контент

Проверяйте орфографию. Да, ошибки в словах не исказят суть прототипа, но вот впечатление они подпортят. Особенно, если просматривать его будет человек грамотный и внимательный к мелочам. А мелочь ничего не прощает большому.

Пишите комментарии по неочевидным элементам

Если непонятно, как будет работать какая-то кнопка, — добавьте к ней комментарий прямо в прототипе, чтобы не потерять этот нюанс и заранее подготовиться к вопросам, которые вам могут задать. Оставлять комментарии позволяют очень многие программы для прототипирования.

Не шутите в аналитических документах

Смешные картинки или веселые фразы воспринимаются занятыми дядями на стороне заказчика как попытка их разозлить. А вдруг получится? Корректнее всего использовать деловой стиль общения.

Используйте метод прогрессивного джипега

Сначала накидываем общую картину, без детализации, основными блоками. Затем — постепенно детализируем каждый блок до нужной степени.

Бывают ситуации, когда на главной странице нужен супер-вау-эффект. Что делать в таких ситуациях? Рисовать в промо-блоке большой зачеркнутый прямоугольник с подписью: «Здесь будет креатив» — как-то не очень удобно, плюс порождает сотни вопросов у заказчика. Двигать этот вопрос на этап дизайна (мол, потом разберемся) — не всегда экологично: а, собственно, зачем тогда в этой схеме прототип? Совсем без прототипа — велик риск не попасть в ожидания на дизайне, и в дальнейшем поиметь множество проблем с перестановкой блоков на готовом макете.

Мы поступаем следующим образом:

1. Перед началом прототипирования проводим брейншторм. На нем сразу придумываем какую-то фишку проекта. На брейншторм обязательно зовем дизайнера и аналитика.
2. Дизайнер накидывает карандашный скетч идеи, которую мы придумали.
3. Привлекаем студийного копирайтера: он готовит тексты для страницы.
4. Аналитик делает прототип: вставляет реальные тексты, схематично показывает идею.
5. Презентуем прототип и скетч заказчику вместе с дизайнером.

У нас этот процесс работает. Но если ваш проект с действительно креативной главной страницей, которую никак не запрототипировать, — следуйте здравому смыслу. В таких случаях достаточно только скетча.

4.2.12.2 ПИШЕМ ТЗ. ГОДНЫЕ ШАБЛОНЫ

Агрегация есть, прототип готов и утвержден — самое время перейти к техническому заданию на разработку проекта. ТЗ — артефакт известный. Это толстый документ, в котором очень детально описано, что за проект в итоге получится. Беда с этим документом в том, что им очень неудобно пользоваться. Огромный объем сурового технического текста со специальной терминологией крайне сложно читать. А уж представить себе итоговую картинку в голове, понять, как это будет работать физически, — практически нереально. Особенно если опыта в чтении таких ТЗ немного.

Поэтому в digital-разработке случаются ситуации, что заказчик вдумчиво изучает ТЗ и утверждает его, а потом искренне удивляется, что в итоге получилось именно то, что и было описано.

Редко какие готовые проекты хотя бы на 70% соответствуют техническому заданию. Юзабилити, бюджет, санкции — что угодно может стать причиной изменений в проекте.

В чистом Scrum-е как такового технического задания нет. Там есть бэклоги. Бэклог — это список функционала (по-другому, фич), отсортированный по приоритетам, по которым идет разработка. Можно менять фичи в любой момент, и это не вызывает проблем на стороне команды. Приоритеты при этом регулярно пересматриваются.

У бэклогов есть свои нюансы и проблемы, и мы их уже разобрали в главе про Scrum.

Но с классическим ТЗ, в котором жестко «морозятся» все требования, проблем, на самом деле, гораздо больше. Дело в том, что невозможно описать четко и однозначно требования так, чтобы люди их поняли точно так, как вы того хотели. В любом ТЗ на спор на коньяк находят-ся какие-то прорехи, которые можно трактовать двояко. Причем, чем толще и подробнее ТЗ, тем больше таких вещей встречается.

Казалось бы, ответ простой: еще больше конкретики! Идея откровенно так себе: если вы будете подстраховываться и описывать каждую деталь максимально подробно, то скатитесь до регламента завязывания шнурков и инструкций, что коту нельзя сушить в микроволновке.

Особенно худо, если подробное техническое задание пишут несколько человек (а такое бывает) — как правило, получается шизофрения листов на 300–400. В определенный момент (часов этак через восемь чтений) человек, который внимательно изучает эти 400 листов, хватается за голову и смотрит в стену с единственным желанием — выбросить это все и начать заново. Потому что полностью разобраться в документации — займет месяцы. А этих месяцев у проекта просто нет.

Разработка прототипа и ТЗ к нему обычно съедает около 12% бюджета проекта. На первый взгляд — много (и это реально много!). Но это не более трудозатратно, чем создание традиционного «железобетонного» технического задания на разработку сайта, которое пытается учесть каждый чих. К тому же, в первом случае процесс прогнозируем по срокам и позволяет избежать гораздо большего вылезания из сроков и бюджета.

При этом потребность рынка очень четко определена: люди привыкли к ТЗ, им нужен документ, на который можно опираться и в случае чего тыкать носом: «Вот, смотрите, вы сделали не по ТЗ». Это отличное

прикрытие тылов, которое очень сильно помогает сотрудникам с той стороны проекта сохранить свои рабочие места и получить минимум негативной обратной связи от руководителей. Конечно, в теории. На практике может быть и по-другому.

А еще внутри многих компаний есть иллюзия, что, написав ТЗ и сформулировав подробно все-все-все требования, можно получить итоговую смету, итоговые сроки, функции будут железобетонными, и ничего никогда не поменяется. Стабильность, точка равновесия, в которой все познали дзен. Такое отношение к техническому заданию особенно свойственно государственным организациям.

Исходя из этих особенностей, психологических и организационных, техническое задание так или иначе вполне может присутствовать на проекте. Для нас более привычно и удобно работать с бэклогами, да и наша методология это предполагает, но, если заказчик попросит сделать ТЗ — конечно, мы его сделаем. За отдельные деньги.

Сама процедура написания технического задания может меняться в зависимости от проекта. В разработке мобильных приложений и интернет-проектов я считаю, что, если у нас есть готовый прототип, ТЗ должно содержать описание того, что мы видим в прототипе, с отклонениями, которые нужно учесть. Например, с алгоритмами, формулами, источниками, откуда берутся данные. В общем — с тем, что на прототипе показать физически невозможно.

Типовая структура нашего технического задания содержит следующие разделы:

1. Общие сведения. Здесь самое важное — приоритеты документов проекта. Наше ТЗ — приложение к прототипу сайта. В случае расхождения технического задания и прототипа — приоритет имеет техническое задание. В случае расхождения утвержденного дизайна и технического задания — приоритет имеет дизайн. Итого: что позднее сделали, то и принимается за истину. У сметы — максимальный приоритет.
2. Назначение и цели создания сайта. Пара слов: что делаем и для чего. Например, интернет-магазин на 1С-Битрикс с функциями доставки и онлайн-оплаты. Плюс сюда же мы добавляем цели, которые берутся из Агрегации.

3. Требования к верстке.
4. Настройка прав доступа. Например, два пользователя, администратор и контент-менеджер.
5. Блок по SEO-требованиям: базовые вещи, которые нужно учесть. Поле в административной панели для установки счетчиков, удобная работа с мета-тегами, микроразметка, человекопонятные ссылки.
6. Описание структуры страниц. Название страницы, что на ней будет по пунктам, описание механики работы каждого пункта.
7. Структура базы данных. Больше техническая информация, которая предназначена для программистов.
8. Описание уведомлений. Это те письма, которые получают на почту пользователи и администратор сайта. Или SMS. Например, уведомление об успешном оформлении заказа. Или о том, что поступила новая заявка.

Ради общего развития рекомендую изучить ГОСТ 2.105–95 — общие требования к выполнению конструкторских и технологических документов.

Когда дело касается разработки сложных проектов вроде интернет-магазинов, не обойтись без интеграций со сторонними системами. Например, нужно, чтобы товары передавались из 1С на сайт, а сайт отдавал в складскую систему заказы. А еще нужно, чтобы заказы передавались в CRM. Операторы колл-центра будут забирать их оттуда, обзванивать клиентов и подтверждать заявки. В подобных проектах должны быть описаны интеграционные протоколы — о них поговорим отдельно в главе 12.

4.3 ПРОДУКТОВЫЕ ТЕХНИКИ

Тут собраны несколько методик, с которыми работают опытные руководители продуктов. Руководителям проектов это будет полезно, во-первых, на вырост, во-вторых, чтобы понимать, как руководители продуктов принимают решения. Если, конечно, они не делают это наугад.

4.3.1 ЧЕТЫРЕ СИЛЫ: ВЗЛЕТИТ ИЛИ НЕ ВЗЛЕТИТ?

Есть такая классическая загадка, о которую в интернете поломано тысячи копий и которую я иногда даю на собеседовании, чтобы проверить рассудительность.

Итак. Есть самолетик — неважно, реактивный или винтовой. Самолет пытается взлететь по взлетно-посадочной полосе. Но полоса хитрая. В виде конвейера. Самолетик — вперед. Полоса — назад. Этот — вперед, та — назад. Скорости совпадают. Взлетит или не взлетит?



Прелесть загадки (несмотря на то, что она не совсем корректно сформулирована — самолетик все же движется с ускорением, его скорость не постоянна) в том, что ответ на нее зависит от множества факторов. Однако есть четыре силы, действующие на самолет. Подъемная сила Y , вес самолета G , сила тяги P и сила сопротивления Q . Две из них, грубо говоря, «помогают» лететь. А две — мешают.

А теперь интересная аналогия. Взлетит ли ваш проект?

На ваших пользователей так же действуют четыре главных силы. Две — помогают перетащить пользователей в ваш продукт. Две — мешают. Мешает **сила привычки** — невероятно сильная штука, как гравитация. И **страх неудачи**. А помогают **проблемы с текущим продуктом** и решением. И **надежда на лучшее**. Про конвейер, надеюсь, аналогия тоже понятна: приходится бежать, просто чтобы остаться на месте.

И да. Иногда достаточно просто очень мощного двигателя.

4.3.2 PAINKILLERS, DELIGHTERS И ЦИФРОВЫЕ НАРКОТИКИ

Painkillers — продукты, которые из «отвратительно» делают «нормально». Например, позволяют подать электронную декларацию вместо того, чтобы терять полдня в потной очереди. Была какая-то боль. Кто-то пришел и ее решил. Тут важно найти сегмент с сильной болью, например, проводя проблемные интервью. 7–8–9–10 из 10 по шкале, где 10 — самое отвратительное, что могло бы с вами случиться — это как раз нужные нам люди.

Delighters — продукты-витаминки. Которые из «нормально» делают «отлично!». Например, продукты линейки Apple. Проблемы-то особой не ощущалось. Тут кто-то раз — и придумал iPod, AirPods и вот это вот все. Главное, чтобы пользователь кайфанул. И перестал использовать старый продукт. Однако мы помним про 4 силы. Старые привычки будут нам мешать, и продукт нам нужно сделать в три раза лучше, чем у конкурентов. Причем, «лучше» — с точки зрения субъективного восприятия пользователя. Та еще задача.

Есть, впрочем, большой рынок продуктов, использующих когнитивные искажения человека. Почти все современные игры. Социальные сети. И тому подобная ерунда на дофаминовом цикле. Изучены и ловко применяются когнитивные искажения, механики геймификации, заставляющие пользователя как можно чаще и как можно больше времени проводить в игре. Но четкого алгоритма, что тут может сработать, нет.



Статья «47 игровых механик» — в тему.

Кстати, прикрутить к хреновому, скучному B2B-продукту геймификацию, и считать, что «вот теперь взлетит» — гиблая затея.

4.3.3 RISKIEST ASSUMPTION TEST (RAT) — ПРОВЕРКА НАИБОЛЕЕ РИСКОВАННОГО ПРЕДПОЛОЖЕНИЯ

RAT — простой как валенок фреймворк, который предлагает столкнуться с проблемами до того, как мы вложим много денег в разработку. Суть фреймворка:

1. Составьте список самых рискованных гипотез на вашем проекте.
2. Проранжируйте от наиболее опасного к наименее опасному.
3. Проверьте свои предположения сверху вниз: от наиболее рискованных к наименее.

Все элементарно! Однако стоит только несколько раз проделать такой цикл самому — вы будете применять его почти везде. Например, собеседуя кандидата, вы можете мысленно составить список, какие риски вы видите в нем. И проверять именно эти риски, а не проводить рядовое интервью с вопросами обо всем подряд.

Каким именно образом проверять гипотезы — RAT не говорит. Это может быть и интервью с потенциальными пользователями (они врут), и изучение конкурентов, и общение с экспертами. А может быть запуск какого-то простого «пилота» (пилотного проекта).

В любом программном продукте есть, как минимум, пять рискованных предположений:

1. **Есть рынок. А на рынке есть сегмент с сильной потребностью (болью).**
2. Мы **можем сделать продукт**, закрывающий эту потребность (хватит экспертизы, денег, серверных мощностей).
3. **Они это купят.** У нас.
4. **Мы на этом заработаем.** Иначе можно просто раздавать людям деньги — будет быстрее и приятнее. Прямо с вертолета скидывать. Тут помогает расчет Unit-экономики.
5. Продукт можно **масштабировать**. Например, бизнес web-студии масштабировать нельзя: если мы хотим больше зарабатывать — нам нужно больше высококвалифицированных людей. А вот бизнес по аренде самокатов или прокату музыки — можно.

Для работы по RAT на основе уже известного вам Lean Canvas был разработан альтернативный шаблон (общий вид — на иллюстрации ниже). Ссылка для скачивания шаблона спрятана в QR-коде справа.



Наиболее рискованное предположение

Покупатели Кто ваши покупатели? Чем они зарабатывают на жизнь? Где их можно найти? Сколько их? Предположение 1: Целевой пользователь — обычный работник, а не руководитель Вероятность ошибочности: (1-5) Влияние, если вы ошиблись: (1-10) Общий риск: вероятность воздействия Предположение 2: Предположение 3: Предположение 4: :	Проблема Какова потребность (проблема) клиентов? Что сейчас они делают, чтобы решить эту проблему? Предположение 1: Пользователи часто регулярно делятся друг с другом информацией Вероятность ошибочности: (1-5) Влияние, если вы ошиблись: (1-10) Общий риск: вероятность воздействия Предположение 2: Предположение 3: Предположение 4: :	Решение Решает ли ваш продукт проблему клиентов? Выполняет ли он работу для них? Предположение 1: Функционал чата и клиента поможет им общаться и делиться информацией Вероятность ошибочности: (1-5) Влияние, если вы ошиблись: (1-10) Общий риск: вероятность воздействия Предположение 2: Предположение 3: Предположение 4: :	MVP Какой должен быть минимально жизнеспособный продукт, за который люди готовы заплатить? Предположение 1: Обмен информацией по электронной почте будет вполне достаточно Вероятность ошибочности: (1-5) Влияние, если вы ошиблись: (1-10) Общий риск: вероятность воздействия Предположение 2: Предположение 3: Предположение 4: :
Конкуренты Кто еще решает эту проблему? Каково ваше конкурентное преимущество? Предположение 1: Наши бизнес-технологии более продвинуты по сравнению с конкурентами Вероятность ошибочности: (1-5) Влияние, если вы ошиблись: (1-10) Общий риск: вероятность воздействия		Каналы продаж Как вы будете продавать свой продукт или услугу? Предположение 1: Нужно, чтобы каждый продавец делал нам 20 новых учетных записей в месяц Вероятность ошибочности: (1-5) Влияние, если вы ошиблись: (1-10) Общий риск: вероятность воздействия	



Итак, RAT — это просто три шага к проверке самых рискованных гипотез на проекте. И в любом проекте есть пять основных рисков.

4.3.4 MINIMUM VIABLE PRODUCT (MVP)

Концепция MVP — minimum viable product, минимально жизнеспособного продукта, предполагает разработку с минимальным набором функций. Но при этом решающего реальную задачу пользователей. Вывод этого продукта на рынок. Получение обратной связи. И доработку продукта прямо на ходу.

У MVP есть альтернативы:

MLP (Minimum Lovable Product) — продукт с каким-то классным преимуществом, за которое потребитель готов выбрать именно его, даже если он не закрывает абсолютно все его потребности. Например, в сервисе рассылок MailChimp есть крутые эмоциональные анимации, которые сопровождают пользователя по мере

отправки рассылки — за них сервис очень любят, несмотря на то, что он может уступать другим по функциональности. О MLP поговорим чуть позже.

MVaP (Minimum Valuable Product) — минимальный ценный продукт. По факту тот же MVP, только вид сбоку. Другая аббревиатура нужна, скорее, самим командам, чтобы те концентрировались на создании именно ценности, а не просто минимального набора функций. Создание такого продукта дороже, но и рисков не попасть в ожидания аудитории меньше, поскольку важной частью такой разработки становятся исследования и аналитика пользовательского поведения.



С одной стороны, все очень правильно: мы не встраиваем миллион функций и можем относительно дешево протестировать нашу идею и программный продукт в живой среде. Дешево относительно того, если бы нас заразили галлюцинации основателей, и мы нагородили миллион функций.

С другой стороны, пользователи сейчас очень избалованы. Не каждый готов ехать на скейте, если привык к роскошному автомобилю. Да и не во всякий автомобиль сядет. Мало того, что должна быть решена проблема. Это должно быть сделано на ожидаемом уровне качества. Няшности и кайфа. А сложность программного обеспечения — очень высокая. Пока что в B2B пользователи еще готовы терпеть программ-

ные продукты, которые абы как решают бизнес-задачи и рутину. Но это ненадолго.

На текущий момент понятие MVP несколько размыто. Кто-то подразумевает под ним простой сайт на Tilda и запуск рекламы для сбора лидов и тестирования спроса. При этом самого продукта может даже не быть. Кто-то — полностью работающую систему, в которой отсутствуют пара редких сценариев.

Тем не менее, сам подход — сделать что-то небольшое, прикольное и выпустить это на рынок — правильный. Быстрее поймем, что идем не туда. И хватит времени переделать, как надо.

Итак. MVP позволяет протестировать рынки и программный продукт без необходимости полной разработки.

4.3.5 MINIMUM LOVABLE PRODUCT (MLP)

Есть сегменты и рынки программного обеспечения, где хоть какое-то решение задачи будет воспринято на ура. Пусть кривое-косое-некрасивое, хоть из командной строки, тормозящее по полдня и требующее пары мощных серверов. Обычно это B2B-задачи, автоматизация рутины, отчеты, перемалывание данных и т. д.

Однако, чем чаще задача встречается и чем ближе она к B2C, тем больше требований к продуманности интерфейсов, удобству и сексуальности продукта предъявляют пользователи. И тут концепция MVP дает сбой.

С одной стороны, в MVP мы тестируем гипотезы продукта. С другой стороны, пользователь скажет резкое «ФИ!», если продукт выглядит не так круто, как он привык или ожидает. Обман ожиданий — тоже обман.

Няшно. Секси. Ми-ми-ми. Это и есть Minimum Lovable Product (MLP). Сделать такое — та еще задача. Переход от MVP к MLP может потребовать раз в десять больше ресурсов на разработку, отладку и полировку.

4.3.6 RAT ИЛИ MVP

В отличие от MVP, RAT предполагает столкнуться с проблемами до того, как мы вложим много денег в разработку.

Типичный цикл RAT: идея → обратная связь от пользователей → MVP

Типичный цикл MVP: идея → MVP → обратная связь от пользователей

Какого-то противоречия между MVP и RAT — нет. Скорее, одно дополняет другое. Для проверки каких-то гипотез придется сделать какой-то продукт. Однако в большинстве случаев достаточно JTBD и проблемных интервью с пользователями.

4.3.7 JOBS TO BE DONE (JTBD)

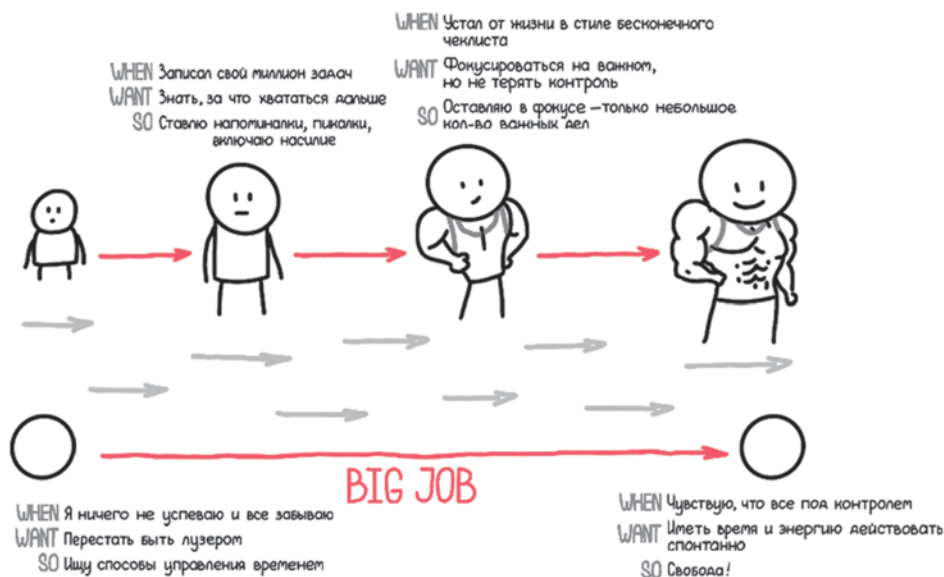
Здесь мы кратко рассмотрим суть подхода Jobs To Be Done, на том уровне, который будет достаточен руководителю проекта для понимания того, как можно принимать решения о развитии продукта. Для полного погружения рекомендую изучить книгу *The Jobs To Be Done Playbook: Align Your Markets, Organization, and Strategy Around Customer Needs 1st Edition*, автор — Джим Калбах.

Фреймворк Jobs To Be Done решает три задачи:

1. Найти сегмент и понять, какие проблемы были у пользователей при переходе из сегмента в сегмент.
2. Понять, каким должен быть программный продукт для конкретного сегмента. Конкретный набор функций.
3. Понять, каким должен быть процесс коммуникации с пользователем. От привлечения до поддержки продукта.

Итак. Любой продукт, в том числе программный, пользователь не покупает, а нанимает для выполнения **работы**. **Работа** — это изменения пользователя к лучшему. Что-то, что делает его круче. У каждого из нас есть потребность в долгосрочном преображении. Всем нам хочется как-то по-другому чувствовать себя в будущем.

Пример: какую конечную картинку пользователь держит в голове, когда ищет инструмент по управлению списком дел и проектами? Уж точно не представляет себя рабом своего ежедневника и жизнь как один бесконечный чек-лист!



Пользователь нанимает программный продукт по управлению списками дел и проектами, чтобы **стать успешным**, успевать и воплощать задуманное и иметь время и энергию действовать спонтанно. Я хочу чувствовать, что у меня все под контролем, тогда у меня останется ресурс действовать спонтанно. Сразу из лузера перепрыгнуть в целевое состояние не получится.

Переход из состояния «все плохо» в «я стал крутым» — это Big Job. По пути попадают работы поменьше. Например, собрать все свои дела в список. И совсем маленькие, типа — «скачать приложение».

Внимание пользователя очень ограничено. Для выполнения одной работы он нанимает, буквально, один продукт. И тут, по сути, все продукты конкурируют за внимание пользователя — работает косвенная конкуренция. Zoom конкурирует с перевозками бизнес-классом, например, поскольку его нанимают на работу «договориться с вон теми



Кстати, что-то подобное можно найти у Ницше в «Так говорил Заратустра», в главе «О трех превращениях»

ребятами за океаном и заключить контракт». Поэтому важна не только сама «работа», на которую нанимается продукт, но и контекст.

Здесь продукты четко подразделяются на те, которые достаточно купить один раз (или покупка — нерегулярная; например, машина). И те, которые нужно постоянно возобновлять. Например, еда, каршеринг или такси.

Состояния пользователя описываются в формате Job Story. Шаблон:

When <situation> **I want** to <motivation> **So** I can <outcome>

Формат похож на User Story, однако воспринимается и читается легче. Тут есть ситуация (контекст). Мотивация пользователя. И определенные последствия.

Примеры:

Когда у моей девушки день рождения, я **хочу** ее порадовать **и** везу ее в Амстердам.

Когда у меня миллион дел в тудушнике, я **хочу**, чтобы мне вовремя напомнили, что я должен сделать, **и я** включаю напоминания.

Когда у меня в фокусе только главные дела, я **хочу**, чтобы рутина меня не отвлекала, **и** выключаю все напоминания.

Несколько выводов из этой концепции:

1. Сегменты в данном контексте — это пользователи примерно одного уровня прогресса по Job Story. Ищем, что их объединяет с точки зрения поведения.
2. Очевидно, что в зависимости от того, как далеко пользователь продвинулся в улучшении себя, в программном продукте будут разные наборы функций. Более того, это могут быть разные продукты. Как Zoom конкурирует с авиаперевозками. Нам нужно будет рассмотреть все способы, все продукты, которые нанимает пользователь. Это наши конкуренты.
3. Пользователи периодически перетекают из сегмента в сегмент. И с этим у них может быть масса проблем. Помните про четыре силы, две из которых говорят «не дергайся», а две — толкают вперед?

4. С какого именно сегмента начать делать программный продукт — не самый простой вопрос. Но принцип такой, что ищем сегмент с сильной болью. И деньгами.
5. Если мы делаем программный продукт для какого-то сегмента — нам нужно посмотреть предыдущие сегменты, шаги по JTBD, на которых застряли пользователи. Например, потенциальные покупатели дорогих машин — те, кто уже владеет машиной среднего класса. А не те, кто только сдал на права. Это наши потенциальные клиенты. Начинать изучать их интересы и барьеры перехода между сегментами. Это сильно похоже на CJM, но несколько с другого ракурса.
6. Пользователь имеет примерное представление, насколько ему хорошо с текущим продуктом. Это можно выяснить с помощью интервью, либо придумать адекватные метрики. Вроде «количества сделанных задач за день» или «процента выполнения личного плана».

Jobs To Be Done удобно использовать как для брейнштормов внутри команды и выработки стратегии продукта, так и для поиска инсайтов и обработки обратной связи от пользователей. Нет какого-то канонического формата фиксации JTBD. Вы можете использовать формат, похожий на CJM из § 4.2.4. Либо воспользоваться подобной таблицей. Для всякого значимого шага к Big Job нужно отдельно проработать каждую такую таблицу и расположить их последовательно. В итоге нам важно понять, какие функции или мероприятия мы должны предусмотреть при разработке продукта.

Job Story	<i>When <situation> I want to <motivation> So I can <outcome></i> Когда я успеваю сделать почти все свои задачи, я хочу чувствовать себя крутым, и я смотрю в сравнительный рейтинг людей своего уровня.
Push & Push drivers	Что толкает или тянет меня к следующему шагу?
Барьеры	Что останавливает меня здесь? Страхи? Привычки? Тревоги? От чего придется отказаться?

Метрики	<i>Как я понимаю, что справляюсь с работой? Моя удовлетворенность текущими решениями?</i>
Альтернативы	<i>Конкуренты. Как они решают эти задачи?</i>
Решения	<i>Что предусмотреть в программном продукте, сервисе и коммуникации?</i>

Итак. Jobs To Be Done — простая концепция, где пользователь нанимает продукт на работу, чтобы самому стать лучше. От того, насколько далеко пользователь зашел в своем совершенствовании, зависит и набор функций продукта, и конкуренты, и нужная коммуникация с пользователем. Собственно, это и определяет сегмент.

4.3.8 ГИПОТЕЗЫ СЕГМЕНТОВ В JTBD. CUSTDEV, ПРОБЛЕМНЫЕ И РЕШЕНЧЕСКИЕ ИНТЕРВЬЮ.

Помните, как мы формировали гипотезы сегментов в заказной разработке? Один из вариантов — найти экспертов и поговорить с ними. В заказной разработке это — сам заказчик. Но в продуктовой так делать нельзя. Сработают галлюцинации основателей.

Как выйти на эксперта? Просто найдите и напишите ему в соцсети. Нежно, вежливо, аккуратно и настойчиво. Заинтересуйте эксперта — у них мало свободного времени, и им сильно не до вас, но... Это работает. По крайней мере, в России. Я проверял.

В JTBD очевидно, что сегмент классифицируется поведением, и от этого никуда не деться. Вы можете найти действительно экспертов из вашей или смежной отрасли. Поговорить с ними. Предложить им какие-то плюшки. Поговорить с конкурентами. И составить гипотезы сегментов. Подумайте вместе с экспертом, где можно найти таких людей. На каких сайтах, форумах. Возможно, эксперт даст вам несколько контактов людей из ваших целевых групп для первых интервью.

Подход Customer Development предполагает проведение **проблемных** и **решенческих интервью** с потенциальными пользователями для выявления реальной проблемы и размера сегмента.

Поговорите с вашими пользователями. Это можно делать лично, либо по Skype/Zoom. Для установления контакта обязательно используйте

камеру — интервью чисто голосом работают гораздо хуже — люди отвлекаются и теряют контакт.

Ищите инсайты. Во время интервью записывайте в реальном режиме времени все, что говорят пользователи («Чувак, мне очень важно записать все, о чем мы сегодня поговорим. Ты не против, если во время разговора я буду за тобой записывать?»). Не используйте видео-аудиозаписи — это бесполезная трата времени, двойная работа на расшифровку и перепросмотр. Вам понадобится 10–12 или более интервью с представителем каждого потенциального сегмента. И это просто куча времени.

Инсайты должны повторяться. Должна быть статистическая значимость этих повторений.

Поиск респондентов можно делегировать HR-ам. Можно поискать среди друзей и знакомых, на конференциях и митапах, фриланс-биржах, спрашивать виртуальных друзей в Фейсбуке или существующих пользователей. Многие охотно откликаются. Спрашивайте, с кем еще можно поговорить по этой теме, после каждого интервью.

Платить за интервью не надо. Но можно давать какие-то плюшки в своих продуктах. Большинство людей готовы в этом участвовать — это почетно, говорить свое мнение. Однако проведение интервью делегировать нельзя — этим должен заниматься лично руководитель продукта. Иначе инсайты будут потеряны. Магия не случится. Да и люди врут: хотят выглядеть лучше, чем они есть на самом деле. Нужно докапываться до реальных мотивов, настоящих болей и эмоций.

Анализируйте текстовые заметки. Ищите инсайты. Можете использовать Mindmap или блокнот для этого. Ищите квалифицирующие факторы: что же все-таки объединяет пользователей, что у них болит на самом деле?

Инсайты должны повторяться, это и создает сегмент. В ходе интервью намеренно проверяйте, повторяются ли те же проблемы, о которых вы слышали ранее, у других респондентов. Вполне может быть, что после десятка-другого интервью ваши гипотезы сегментов поменяются. Не делайте выводов на основе маленького количества респондентов — нужна статистика, а небольшая выборка дает слишком большие выбросы. Найдите сегмент с самой сильной болью и/или самый большой сегмент. И начинайте думать над решением.

Хорошие скрипты для интервью вы можете найти у Ивана Замесина. По ссылке в QR — статья по мотивам его доклада с перечнем годных вопросов для интервью (внутри статьи найдете и ссылку на его авторский курс по CustDev). Возьмите их за основу. И постепенно доточите под ваши задачи.



Итак. Алгоритм работы с гипотезами и проведения интервью выглядит таким образом:

1. Делаем гипотезы сегментов. Если нет идей — разговариваем с экспертами и конкурентами.
2. Ранжируем сегменты по:
 - a. размеру;
 - b. силе боли / потенциала кайфа — помните про продукты Painkillers и Delighters;
 - c. платежеспособности — например, ABCDX-сегментация или сегментация по цене.
3. Ищем респондентов. Проводим серию интервью. Записываем в блокнотик. Ищем инсайты и квалифицирующие поведенческие паттерны.
4. Вполне вероятно, что гипотезы сегментов поменяются.
5. Для самого жирного сегмента придумываем решения. И проверяем их с помощью решенческих интервью.
6. Используем RAT для тестирования рискованных гипотез.
7. Если все ок — делаем продукт небольшими итерациями. И проверяем на пользователях.

Кстати, про итерации!

4.3.9 HADI-ЦИКЛЫ

Мы уже рассматривали с вами цикл Деминга (Plan → Do → Check → Act), планируй-делай-проверяй-воздействуй. Тот же цикл в продуктовой разработке и маркетинге называется HADI.

H (Hypothesis) — гипотеза. Мы придумываем гипотезы, которые хотим протестировать. Что улучшить в продукте, каналах, маркетинге. Аналог в скраме — это формирование спринта.



A (Action) — действие. Мы реализуем наши гипотезы. Сколько сможем, за цикл. Аналог в скраме — разработка спринта.

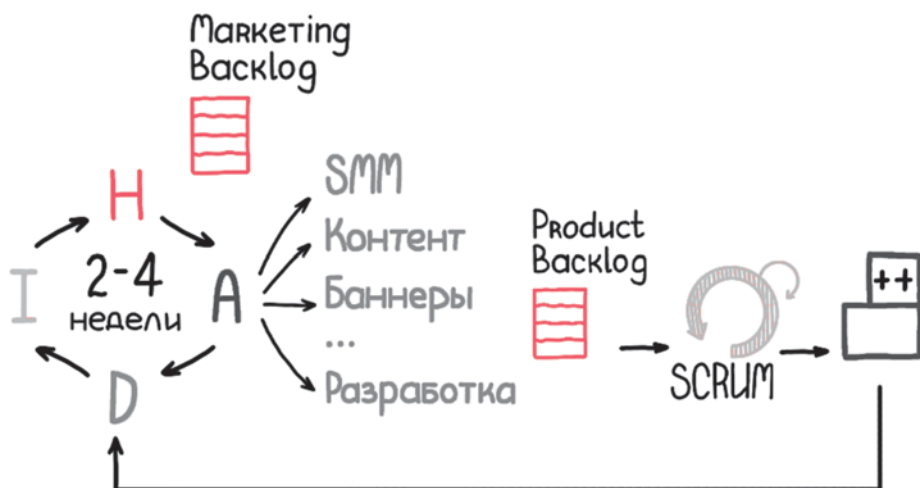
D (Data) — аналитика. Мы релизим наши гипотезы. Начинают приходить реальные пользователи, и мы собираем данные, насколько наши гипотезы верны. Аналог в скраме — демонстрация и выпуск очередной версии продукта. Сбор обратной связи. Разница в том, что мы должны заранее придумать метрики для наших гипотез и способы сбора этих метрик. Лайки, охваты, переходы или что именно вы будете наращивать — зависит от задачи.

I (Insights) — выводы. Мы собрали статистику. Теперь нам нужно сделать выводы: какие гипотезы были удачными, а что не сработало. Аналог в скраме — ретроспективы.

Все очень просто. Вы можете использовать Scrum-фреймворк для организации команд продвижения и развития продукта. Не нужно ничего выдумывать заново. Выберите длину итерации. Соберите команду роста. Руководитель продукта, маркетолог и кто там еще у вас есть, заинтересованный в росте продукта? Организуйте бэклог и работу итерациями. Можно использовать тот же инструментарий, что и команда разработки: Trello, Jira или канбан-доски — без разницы.

Однако у команды разработки и команды роста должны быть разные доски. Это связано с разной длиной цикла работ — задача, пришедшая из маркетинга, может потребовать несколько итераций разработки, и вы вряд ли сможете это синхронизировать. Кроме того, задача в разработке требует более детальных проработок и постановок.

Итак. HADI-цикл — это аналог цикла Деминга, но в маркетинге и развитии продукта. Используйте обычный Scrum-подход, но добавьте туда метрики, которые покажут удачность гипотез. Организуйте команду



роста и итеративную работу. Не смешивайте разработку и маркетинг на одной доске, но использовать тот же инструментарий, что и в разработке, не возбраняется.

Итого: мы разобрали каркас продуктовых и заказных методик аналитики. Теперь у нас есть инструментарий, по которому можно собрать концепцию продукта. Однако нам нужны метрики, которые бы говорили, достигаем ли мы успеха в своем продукте или нет. Я предлагаю использовать... деньги! В следующей главе мы рассмотрим юнит-экономику. Что это и как с этим работать.

4.4 ЮНИТ-ЭКОНОМИКА БЕЗ ПЕРЕЛОМА ГОЛОВЫ

Начиная с 1995 года, когда интернет стал развиваться все активнее, акции американских интернет-компаний начали резко расти. На рынке стало появляться все больше компаний, готовых использовать интернет для получения прибыли. Среди них были и Amazon с eBay. Олдскульные оффлайновые бизнесы также начали перепрофилироваться в интернет-компании. Новые бизнес-модели казались удачными, хотя на деле были неэффективными — большинство компаний существовали за счет больших кредитов и необоснованно много тратили на рекламу.

Как результат — финансовый пузырь («пузырь доткомов»), который лопнул после сильного падения индекса высокотехнологичных компаний

NASDAQ в марте 2001 года, спровоцировав впоследствии волну банкротств новоиспеченных интернет-компаний. А те, что смогли выстоять, серьезно потеряли в акциях — тот же Amazon тогда потерял 93% своих.

Ключевая причина краха всех этих компаний — отсутствие прибыли, неверная оценка активов и завышенная оценка стоимости самих компаний. Почему так? Дело в том, что такими бизнесами чаще всего руководили айтишники, на тот момент мало понимавшие в управлении и продажах. Собственно, для них бизнес-акселераторы и разработали упрощенную модель оценки бизнеса на основе маржинальности с единицы проданного товара. Она и легла в основу юнит-экономики. Позже методологию развили с учетом более умной сферы интернет-аналитики, которая помогает рассчитывать эффективность коммуникаций с покупателями и оценивать воронки продаж.

Юнит-экономика (unit economics) — метод экономического моделирования, который стал популярен в среде интернет-предпринимателей в последние два десятилетия (особенно среди стартапов). Его суть в том, чтобы определить прибыльность бизнес-модели через оценку прибыльности юнита: одного клиента или единицы товара. **Если один клиент или единица товара приносят прибыль, значит, и модель будет успешной.**

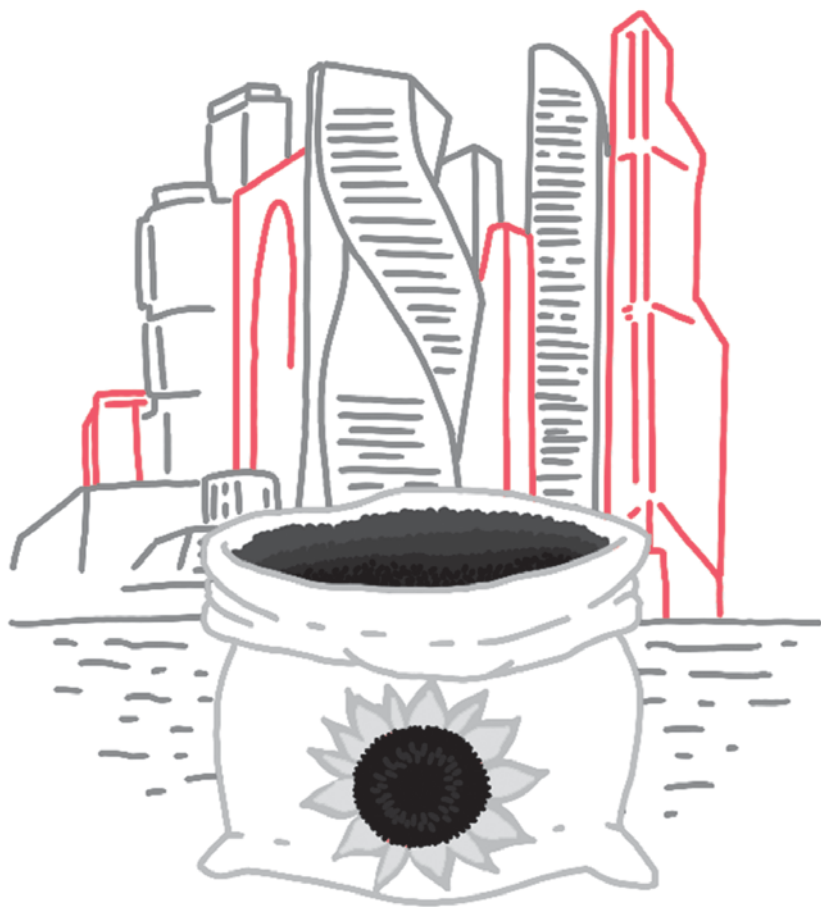
Почему юнит-экономика так нравится стартапам? Потому что она позволяет посчитать, сколько нужно продать, чтобы покрыть постоянные издержки, а также узнать точку безубыточности и доходность инвестиций.

Каждому стартапу (равно как и продукту) для роста важно постоянно генерировать и проверять гипотезы — или, по-другому, делать продуктовые решения. А что будет, если увеличить конверсию? А если снизить затраты? А если, а если, а если... Каждая гипотеза — это деньги, живые, настоящие. И их всегда мало. На проверку каждой гипотезы уходят часы работы продуктовой команды. Средняя команда за месяц сможет проверить две-четыре гипотезы, но всего одна из десяти гипотез выстреливает. Юнит-экономика тем и хороша, что позволяет смоделировать разные продуктовые решения и проверить, насколько они жизнеспособны.

Просчитанная юнит-экономика убеждает инвесторов лучше любых презентаций и обещаний — им проще оценить перспективность проекта, если стартап предоставит расчет всего трех показателей:

- ▶ стоимости единицы продукции или доход на одного клиента (тот самый юнит);
- ▶ стоимости привлечения клиента;
- ▶ стоимости обслуживания клиента.

4.4.1 UNIT-ЭКОНОМИКА НА ПРИМЕРЕ СТАКАНА СЕМЕЧЕК



Представьте, что вы нашли бездонный мешок подсолнечных семечек. Не вырастили, не купили, а именно нашли. Это чтобы пока не думать о постоянных и переменных затратах.

Нащелкались семечек до боли в зубах. Уже не лезет. Это мы тестируем, что продукт в принципе ОК и понимаем размер потребности. Отсыпали, завернули в кулечек и сунули в левый карман ватника.

Затем вы начали раздавать по стакану семечек своим друзьям. Берут! Это мы убедились, что есть интерес к продукту.

Друзья привели своих знакомых. Но они вам не друзья. Им вы отдавать не хотите. А продаете по рублю стакан. Это вы протестировали готовность рынка покупать.

Но знакомые, друзья и друзья друзей в какой-то момент наелись. Нужна реклама. И, как писал Пелевин — «С нее вся эта е..ная рыночная экономика и началась».

Вы купили рекламу у Яндекса. Люди идут. Семки по рублю за стакан покупают. Но кто-то говорит, что по рублю — дорого, кому-то до вас идти далеко, а кто-то думал, что семки жареные или тыквенные, а у вас — нет. Брать не стал. Не ваш сегмент, в общем. Конверсия 3% — обычное дело. Это сегментация, воронка, все дела.

В конце месяца посмотрели: на рекламу потратили сто рублей, получили тыщу. Получается на одного платящего пользователя тратим 10 копеек, зарабатываем 90. С учетом, что семки нам достались бесплатно — жить можно. Это сошлась юнит-экономика, где юнит = стакан.

Но тут покупатели по 10 копеек за сделку кончились. Теперь, чтобы дотянуться до новых, вы тратите 5 рублей. Это вы спалили рынок.

С одной стороны — жопа полная, трачу 5 рублей, зарабатываю рубль. Проще было бы деньги раздавать. С другой — у вас набралась постоянная клиентская база, которая ходит за семками каждый день. Ну кто-то через день, кто-то раз в неделю. Вы подсчитали, что в среднем покупатель, пока ему семки не надоели, берет 50 стаканов. Кто-то — сильно больше. Кто-то — один. Но в среднем — 50. Это LTV. Как именно вы это подсчитали — вопрос интересный, ну вот как-то подсчитали. Получается, что на рекламу тратим 5 рублей. В среднем зарабатываем 50. Это покрывает затраты на тех, кто купил всего один стакан. Жить можно. Только юнит уже не стакан, а клиент.

Тут вы решили, что нефиг продавать семки. Спрос не тот, маржа маленькая.

Масло! Лучше надавить из них подсолнечного масла высокотехнологичными нанороботами. Это **PIVOT**.

Но своих денег для наноробота у вас не хватает. Вы идете во ФРИИ. Тусуетесь со стартаперами. Там вам наливают в уши ценной информации: ARPU, ARPPU, бла-бла-бла. Спичитесь. Знакомитесь с нефтяником, долго пьете с ним водку в бане, закусывая семками. Он дает вам миллион рублей венчурных инвестиций за 49% в вашем стартапе наномаслодава, половину из которых тут же забирает на консультантов по двух-трех-четырёх-буквенным аббревиатурам. Вам же надо понимать, как на самом деле все тут устроено!

Вы покупаете нанороботов, которые жмут масло. Переезжаете из Усть-Зажопинска в Москва-Сити на 98 этаж с видом на недоделанный **стилобат**, в коворкинг со смузи и блондинкой-лаборанткой с длинными ногами. Хантите себе операционного директора из Газпрома и IT-директора из Сбера (кто-то же должен следить за роботами!). Вокруг вас уже куча консультантов, говорящих словами из ИКЕИ (вроде CM, UA, C1, 1sCOGS, APC, ROMI).

Ни с первой, ни со второй, ни с двадцать второй продажи вы реально не зарабатываете. Но какой-то кэш-флоу идет. Вы уже ни черта не понимаете, но инвестор требует кратного роста. Вы рисуете красивые графики и презентации, мол, рост прибыли клюшкой вверх, потому что (и дальше огромная туча формул, которые вы уже смутно понимаете и смутно догадываетесь, что это фуфломизин, но про которые убедительно рассказывают консультанты по юнит-экономике).

...Тут я могу долго продолжать (остановите меня). И все хорошо, пока все хорошо. Но!

В какой-то момент выясняется, что мешок оказался не бездонным, и он уже не мешок, а полмешка. Нанороботы — китайские. Аренда коворкинга в Москва-Сити на 98 этаже с блондинками и смузи стоит миллион, IT-директор из Сбера нашел работу поинтересней и забрал **рутовые пароли** от главного наноробота, а операционный директор из Газпрома слил клиентскую базу конкурентам. Нефтяник (с какими-то нервными ребятами) приезжает и забирает у вас оставшиеся семки и квартиру в Усть-Зажопинске. Это — фиаско.

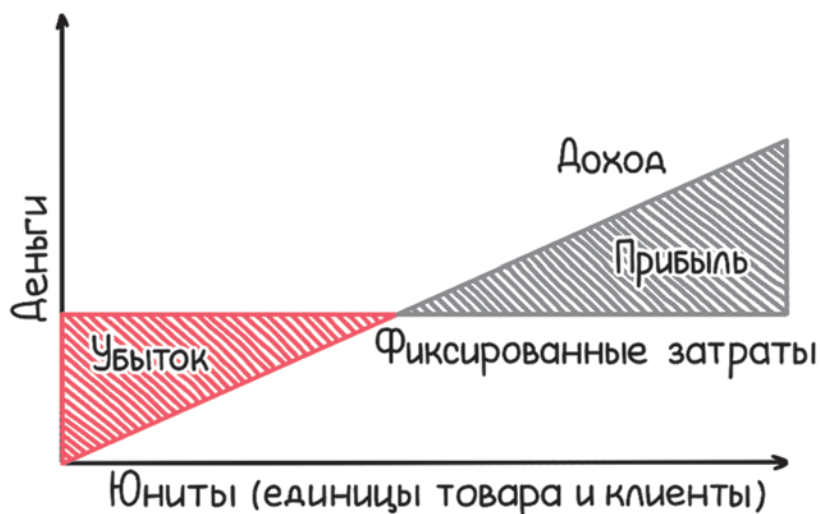
И вот вы сидите такой на завалинке. Солнышко светит, а у вас депрессия. Раздражает — да. Радует — нет. Это выгорание.

Засовываете руку в левый карман старого ватника. А там — кулек семок! «Все будет хорошо! Даже если — не будет», — думаете вы. Это — счастье. Ну или опыт, тут уже вопрос философский.

4.4.2 ПОДПИСКИ ИЛИ РАЗОВАЯ ПРОДАЖА?

Девятый, десятый, б..., пятидесятый.
Noize MC

Суть юнит-экономики — понять, какую прибыль приносит бизнес **без учета постоянных затрат**. Зная это, можно оценить, сколько бизнесу нужно продать, чтобы покрыть свои постоянные расходы.



По мере увеличения объемов доход с продажи движется вверх и вправо, пересекая линию постоянных затрат. Точка их пересечения — точка безубыточности.

Выделяют два подхода к расчету юнит-экономики:

1) Юнит — один проданный товар

Тогда юнит-экономика считается как маржинальная прибыль: доход с одной продажи за вычетом всех переменных затрат, связанных с этой продажей.

Это правда очень просто. Потратили на привлечение 5 рублей, заработали 20 — молодцы, в какой-то момент перекроем постоянные расходы, начнем получать прибыль и уедем жить в Сочи.

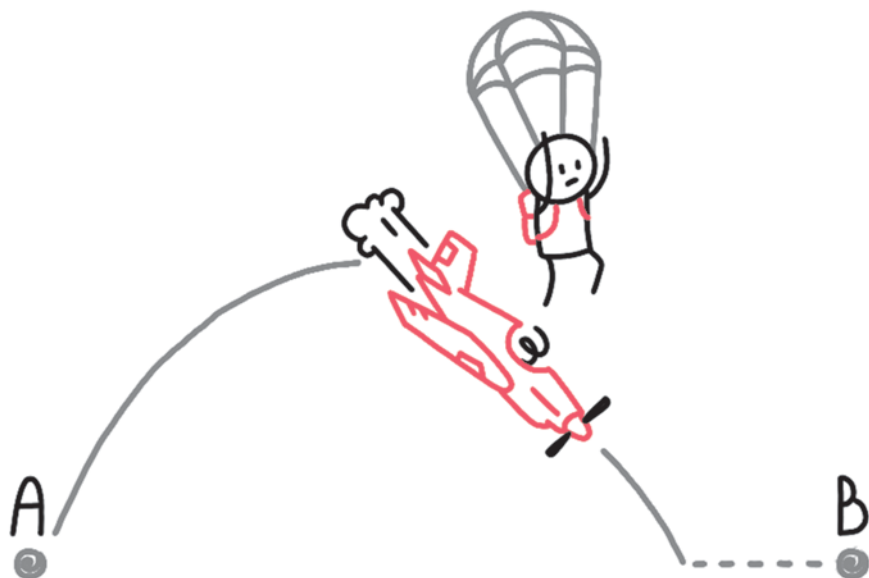
Потратили 20 рублей, заработали 5 — поедem жить в коробку у мусорного бака. Можно было просто деньги дарить, люди бы были счастливее, и быстрее бы закончили.

Такие расчеты годятся для разовых покупок типа интернет-магазина.

2) Юнит — единичный клиент

Рассчитывается через метрики пожизненной ценности клиента (CLV) и затраты на привлечение клиента (CAC).

Эта штука нужна для сервисов с подписками, когда клиент платит несколько раз. При этом первые несколько платежей могут пока не отбивать затраты на привлечение. И только начиная с какого-то (девятый, десятый, пятидесятый) мы отобьем деньги, вложенные в привлечение этого, конкретного одного пользователя.



Проблема в том, что далеко не все пользователи доживут до этого девятого-десятого-пятидесятого платежа. Ну да, редкая птица долетит до середины Днепра. А ей еще и обратно фигачить. И затраты на привлечение таких «недолетов» мы тоже должны размазать по тем, кто таки продолжит платить.

А сколько именно доживут и как хорошо они будут платить дальше — никто не знает. Серая зона догадок и статистических моделей. Единственный способ узнать это наверняка — дожидаться, пока отвалятся все пользователи и посмотреть, как оно было на самом деле. А оно вам надо?

До кучи: постоянные и переменные расходы тоже динамически меняются во времени. Например, постоянные расходы: когда стартап поднял денег и переехал из Усть-Захопинска в Москва-Сити, на самый верхний этаж башни Федерация, нанял всем по блондинке-лаборантке с длинными ногами и пьет смузи. Упс, экономика перестала сходиться. Очень жаль. Зато шиканули за счет инвесторов (на самом деле — нет, там не дураки сидят).

Или вся экономика была рассчитана на пользователей за три рубля штука. А они кончились. Остались только по 333. Бывает, обычное дело.



Короче, тут все сильно запутано. И правду найти практически невозможно. Ученые спорят. Придумывают кучу метрик и аббревиатур. Все усугубляется еще и тем, что продукт может быть убыточным: теряет деньги с каждой транзакции, при том, что акции компании растут, инвесторы вкладываются, и пузырь надувается. Пиф-паф-пфф!

Моя религия говорит, что нужно быть в плюсе с первой транзакции. Ну, максимум — с третьей. Все остальное — игры престолов, зона высоких материй и отложенной ответственности за базар.

Я намеренно не привожу тут разные модели и формулы. Во-первых, они все врут. Во-вторых, все запутывают. В-третьих, они не нужны руководителю проектов. И если ваш начальник, продукт-менеджер или маркетолог, начитавшись интернетов, начал разговаривать двух-трех-четырёхбуквенными словам — дело дрянь. Сходите в интернет и посмотрите ролик «Приклеить или прибить» от Comedy Club. Это — оно!



Приклеить
или прибить?

Простой до безобразия расчет на примере SingularityApp

Дано: для тестирования экономики продукта мы вложили в продвижение сайта через Яндекс.Директ 9 453,76 рублей. Привлекли на сайт — 5 998 пользователей. Из них платную подписку купили 35 человек (28 купили ежемесячную подписку за 199 рублей, 7 — годовую за 1 999 рублей).

Доход с подписок на сайте за месяц составил 19 565 рублей. Мы знаем, что средняя продолжительность подписки — 74 дня, это 2,5 месяца. Это не очень корректно, поскольку в итоге выяснилось, что продукт достаточно «липкий», и пользователи, не ушедшие за пару месяцев, остаются с нами на годы. Да и у нас сразу есть годовые подписки. Но при тестировании экономики мы не могли ждать годы. Итак.

Расчет:

$CRA = 9453,76 : 35 = 270,11$ рублей — столько мы потратили на то, чтобы привлечь одного платящего пользователя.

$LTV = (19565 : 35) \times 2,5 = 1397,5$ рублей — столько в среднем мы получим с каждого клиента за цикл его жизни.

Итог: $LTV > CPA$ — мы в плюсе. Можно масштабироваться. Но явно надо поработать над удержанием в продукте и конверсией. Окей, работаем дальше.

Обратите внимание, мы потратили очень мало денег, чтобы протестировать гипотезу, что продукт будут покупать, и мы на этом заработаем. В чистом виде RAT-подход.

Среди экспертов, кто может просто и вменяемо рассказать про показатели, хочу выделить Илью Красинского, который разработал собственный фреймворк для бизнес-акселератора ФРИИ (им пользуются и Яндекс, и Альфа-банк).

Итак. Юнит-экономика — это способ посчитать, будем ли мы зарабатывать с каждого пользователя или прогорим. Для единичных покупок — все просто. Для подписок — не просто, особенно если вы начинаете зарабатывать не с первой транзакции. Мораль — зарабатывайте сразу.

4.5 ТЕХНИКИ СКОРИНГА И ПРИОРИТИЗАЦИИ БЭКЛОГОВ

Давайте разберем приоритизацию бэклогов. В конечном счете туда попадают все задачи и хотелки из всех источников. Я видел бэклоги продуктов, в которых было несколько тысяч задач. С этим очень сложно работать. Сложно отделить важное от несущественного.

Итак, напомним. Когда есть бэклог, менеджеру нужно каким-то образом расставлять в нем приоритеты. Скрам говорит, что первыми должны идти самые важные с точки зрения бизнеса задачи. Но тут возникают две проблемы.

Первая — справедливый момент субъективизма: часто приоритеты выставляются так, как владельцу бизнеса взбредет в голову. Но иногда владелец может сильно «галлюцинировать» и нести откровенную чушь, но при этом быть уверенным, что все так и есть.

Вторая проблема: слишком много стейкхолдеров верхнего уровня со стороны бизнеса на больших проектах или в крупных компаниях. У каждого из них могут быть свои противоречивые требования.

Если собрать, например, пять топов большой компании и попросить их приоритизировать свои требования, скорее всего, каждый будет утверждать, что его задачи имеют наивысший приоритет, и их нужно делать прямо сейчас.

Хорошие новости в том, что есть несколько методик, которые помогают в этом случае:

- 1. Договариваться о приоритетах — выбирать, что важнее, а что можно отложить.
- 2. Устанавливать критерии приоритизации бэклога (чуть менее субъективные, чем чья-то галлюцинация — увы, совсем без субъективизма не получится).

4.5.1 КАК ВЫГЛЯДИТ БЭКЛОГ

Мы уже детально разбирали способы ведения бэклогов в главе про Scrum. Здесь кратко напомним.



Бэклог — это таблица. В одной колонке — список каких-либо хотелок, есть колонка с оценкой (estimate) и есть колонка с приоритетами. Приоритеты — это какие-то числа, как правило, большие. Большие для того, чтобы между приоритетами оставались «дырки», куда можно добавлять новые задачи (или чтобы легко менять приоритетность).

По классике приоритеты выставляются с точки зрения Business Value (ценности для бизнеса) — того, что для бизнеса нужно в первую очередь, оно и пойдет в работу на первом этапе. Но есть другие способы приоритизации, которые бывают удобнее — особенно, если у вас есть ворох разношерстных задач.

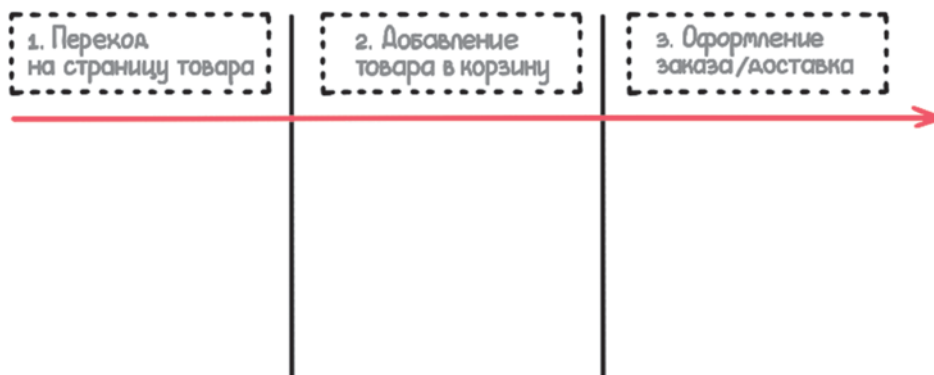
4.5.2 ТЕХНИКИ ПРИОРИТИЗАЦИИ

Здесь собраны несколько техник, которые позволяют понять, за что хвататься в первую очередь. От техник скоринга они отличаются тем, что тут не будет какой-то циферки, которую можно вписать в бэклог в колонку «Приоритет». Тем не менее, эти техники позволяют кластеризовать бэклог и подумать над стратегией развития продукта.

4.5.2.1 Story Mapping

Допустим, у вас есть очень много задач, они мелкие и вообще без приоритетов и привязки к каким-то группам. Что с ними делать? Разбейте их на Story Mapping. Как это работает:

Шаг 1. Строим последовательность того, как юзеры будут пользоваться вашим продуктом, и какие шаги они будут предпринимать. Простой пример:



Шаг 2. На стикеры выписываем, какие по каждому из этих процессов есть детали — чем ниже висит стикер у каждого этапа, тем ниже у него приоритет.

Результат: весь список задач разбит по шагам пути пользователя, плюс у каждой фичи есть приоритеты (чем ниже фича висит в листе, тем у нее ниже приоритет с точки зрения всего пути пользователя).



Где и как применять

Допустим, у вас действительно набралось очень-очень много задач. Тогда вы выписываете их все на стикеры и делаете Story Mapping. Лучше — в команде.

Другой вариант: у вас идет брейншторм, вы придумываете, каким дальше будет ваш продукт и какие фичи брать в работу. У команды много идей, какие-то хотелки вам «насыпал» маркетинг — нужно понять, каким образом это все вообще кластеризовать. В такой ситуации Story Mapping работает особенно хорошо.

Оговорка: этот метод применим именно с точки зрения продуктового менеджмента, когда есть много задач и непонятно, какие из них первыми брать в проработку. Грубо говоря, когда мы только продумы-

ваем сам продукт и то, какой функционал он будет включать. Дальше это уже режется на кусочки, из которых можно создавать спринты и забирать в работу.



Плюсы Story Mapping:

1. Метод позволяет построить связи в системе, на основе которых потом проще формировать спринты.
2. Когда много стейкхолдеров и много людей, у которых есть заинтересованность в проекте, метод позволяет договориться о реальных приоритетах: что более важно, а что — менее.

4.5.2.2 Value & Effort (или Lean Prioritization) для идей

Другой хороший метод, который позволяет построить приоритеты на шкале — Value & Effort (или Lean Prioritization).

Шаг 1. Сначала вы берете 2 шкалы:

1) Значимость фичи

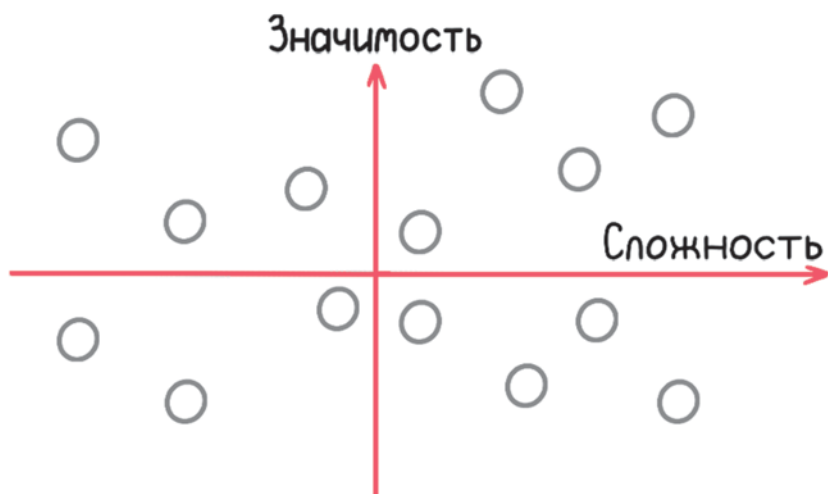
Значимость каждой фичи с точки зрения бизнеса — это примерно то же, что и «ценность для бизнеса» в обычном бэклоге. Величина оценивается в условных единицах, лучше всего — в деньгах. В целом,

любой параметр, который вы оптимизируете, можно брать за Value. Это может быть количество пользователей, которых вы привлечете в проект, или уменьшение оттока пользователей.

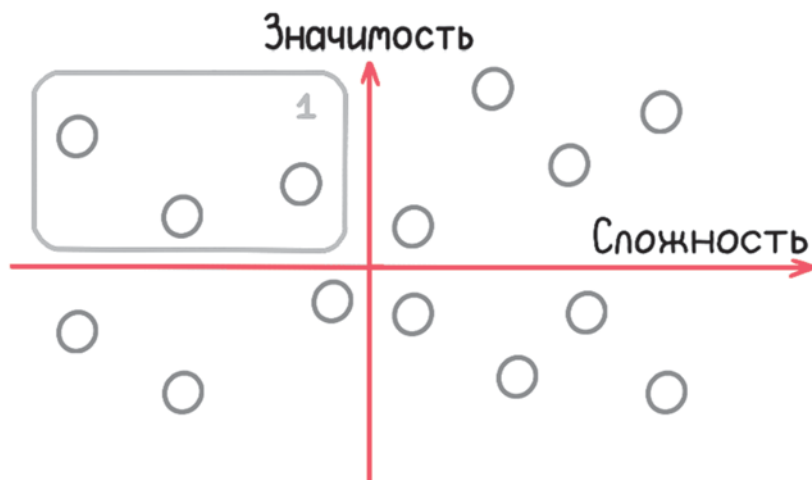
2) *Количество усилий, которое необходимо затратить на каждую из фич*

Тоже можно считать, что это оценка в часах (estimate), как в бэк-глоге. Но можно измерять и в Story Point-ax (сравнительная оценка требований относительно друг друга) или в человеко-часах.

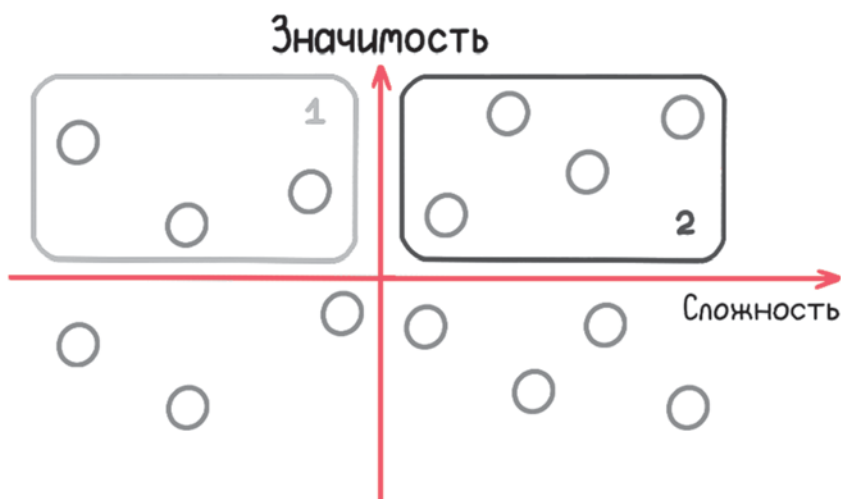
Шаг 2. Вы оцениваете все фичи по этим двум параметрам: по значимости и трудоемкости. Есть внешние системы (вроде Juggler или Airfocus Priorities & Roadmaps), которые позволяют в автоматическом режиме раскидать каким-то образом ваши фичи на такой вот доске. Оси при этом идут не от нуля — они подстраиваются под статистические данные, которые вы получили.



Какие фичи забирать в первую очередь? Самые значимые и легкие, которые ближе всего к оси значимости, но дальше от сложности — они и по важности в топе, и по стоимости адекватные:



Если в первую очередь мы забираем дешевые и хорошие, то потом — дорогие, но крутые:



Следом — все остальные.

Фичи, которые слишком дороги и не имеют никакого смысла, вы либо оставляете «на потом», либо выбрасываете.

Где и как применять

При заказной разработке такую штуку имеет смысл проделывать с клиентом, если:

- ▶ он сам запутался;
- ▶ он генерирует странные идеи и требования;
- ▶ у него есть определенное ограничение по бюджету, и он не понимает, как лучше его распределить.

Последняя ситуация — частая. Мы всегда ограничены либо бюджетом, либо ресурсами, и нужно понимать, на основе чего их распределять. Более того, управляя проектами, мы можем управлять только вниманием нашей команды и нашего заказчика — больше ничем, по сути. Этот метод позволяет концентрироваться на самом важном и самом дешевом либо на дорогом, но очень важном, и помогает выбрать, на что именно сделать ставку в данный момент.

Эта методика не заставляет вас слепо верить алгоритму и брать именно рекомендованные им задачи, но благодаря ей у вас хотя бы будет подсказка, в каком направлении двигаться. Если в систему добавить новые задачи, она может перестроиться. Ведь бывает, что вы не сильно уверены в своих оценках, и с течением времени аналитики их уточняют, либо вы сами уточняете оценки трудоемкости у программистов — в этом случае система также может изменяться в динамике. За счет этого у вас будет более адекватная картина мира.

4.5.2.3 MoSCoW

Еще один способ категоризации фич по нескольким группам — метод MoSCoW. Внутри — очень простые параметры:

M — Must Have. Функционал, без которого вообще нельзя обойтись. Без него вы не сможете выпуститься, ваш продукт не заработает и вообще не будет нужен.

S — Should Have. Функционал, который должен быть в проекте, но при прочих равных без него как-то можно обойтись.

C — Could Have. Функционал, желательный для релиза.

W — Would Have. Наименее практичный функционал — так скажем, «все остальное».

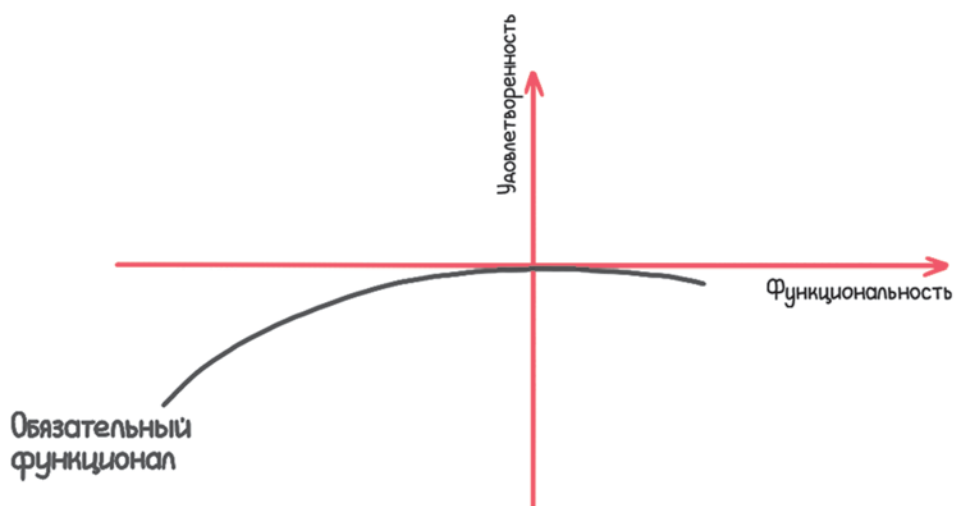
Пример. Допустим, вы разрабатываете автомобиль. Must Have будут колеса, руль, ходовая часть, двигатель. Should Have — освещение ночью, сидения вместо стульев, двери и все такое прочее. Could Have — автоматическая коробка передач и так далее. Таким же образом можно разобрать любой проект, где Must Have будет эдаким MVP.

Часто бывает, что приоритеты спускают сверху, от бизнеса, и они сконцентрированы на каких-то «хотелках» из Should Have, Could Have, Would Have, забывая на ключевые вещи (Must Have). Обычно мы это наблюдаем, например, на разработке дизайна интернет-магазина или дизайна какого-то проекта, где на систему оплаты доставки или чего-то, что на самом деле генерирует выгоду бизнесу, ставка делается в последнюю очередь. Почему? Потому что работать с Must Have больно и страшно: надо думать о том, как это будет монетизироваться, а в это никто не любит лезть, хотя это и неправильно.

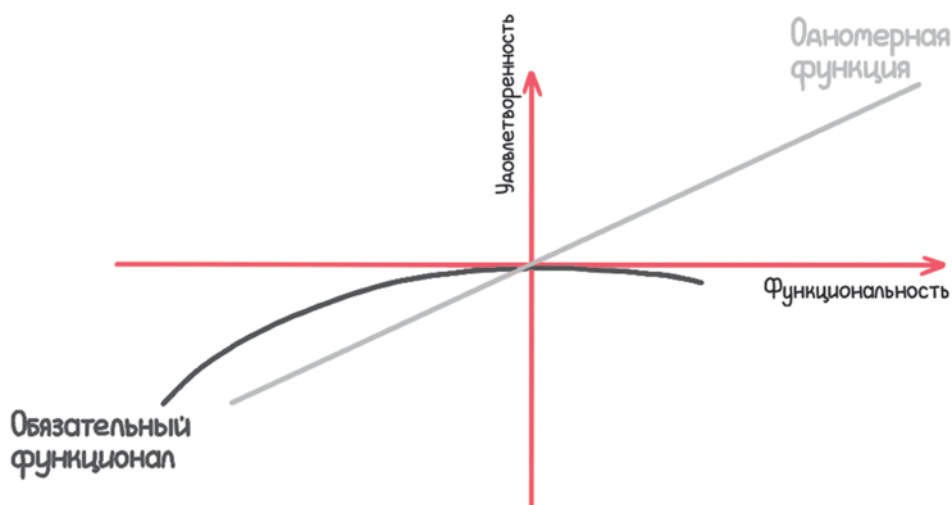
4.5.2.4 Kano

Еще один способ категоризации фичей, пришедший из маркетинга. Суть простая: есть две оси: «удовлетворенность пользователей» и «функциональность», и есть деление функционала на группы. Для каждой группы фич нужно понять, как меняется удовлетворенность пользователя от добавления этого функционала.

Первая группа — обязательный функционал: тот же Must Have из MoSCoW. Если эти фичи есть — уже хорошо. Об удовлетворенности речи не идет: без них продукт никому не нужен. Более того, с течением времени функции, которые сначала были «изюминкой» проекта, становятся все более обязательными. Пример: для CRM канбан-доска когда-то была чем-то эдаким, а сейчас это тот самый маст-хэв.



Другая группа — одномерные функции. Это значит, что есть прямая зависимость удовлетворенности пользователя от наличия этой функции. Как только функция появляется, удовлетворенность растет линейно. Если вернуться к примеру с созданием автомобиля, там это может быть климат-контроль.

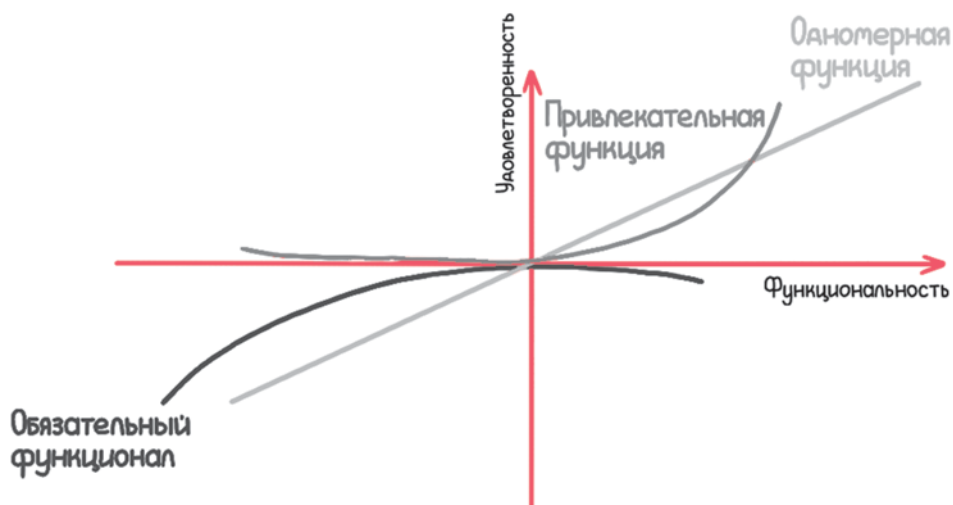


Третья группа — привлекательные функции. Это то, чего пользователь не ожидал, но когда увидел хоть какую-то реализацию этого в вашем продукте, он офигел и сказал: «О, круто!». Кто летал Аэрофлотом, наверняка видел, как детям раздают «взятки», чтобы эмоционально привязать их к бренду.

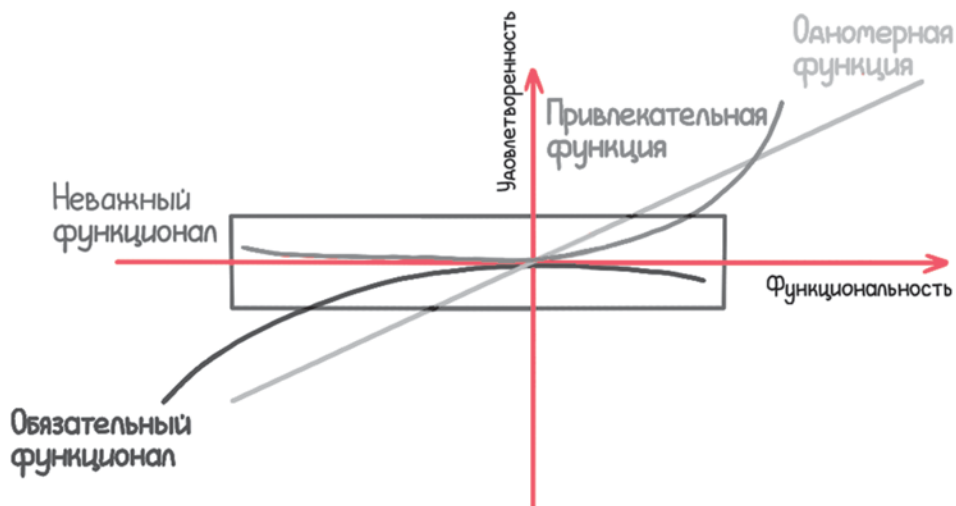
Подарки
для детей
на борту
«Аэрофлота»



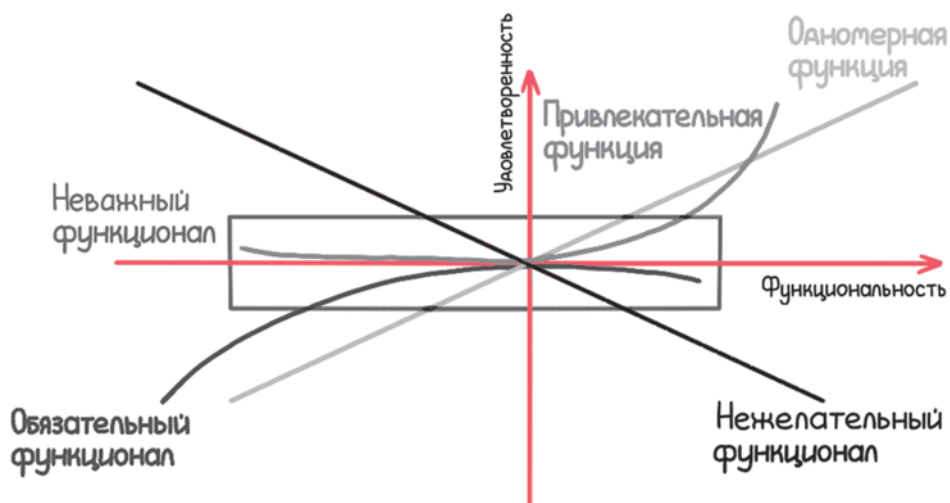
Как только такая функция появляется даже в посредственной реализации, удовлетворенность растет. А чем круче реализация, тем выше удовлетворенность.



Еще одна группа — неважные функции. Их можно делать, а можно нет — всем будет безразлично.



Последняя группа — нежелательные функции. Когда их нет — все хорошо, как только они появляются — все становится плохо. Эдакие антифичи :)



Этот метод несколько сомнителен, поскольку непонятно, каким образом выбираются приоритеты у фич. По идее нужно опрашивать пользователей: как они считают, хороша эта фишка или не очень, а потом кластеризовать на основе полученного мнения. При этом, еще и самих пользователей нужно разбивать на целевые группы и смотреть, как целевая группа к каждой функции относится.

Люди в опросах часто говорят ерунду и попросту врут. Они могут говорить, что это очень важная фишка, но по факту они никогда не заплатят за нее деньги. Более того, если опрашивать пользователей, обратная связь по продукту может быть очень токсичной.

Пример. Вы планируете делать следующие релизы и опрашиваете группу людей. Кто-то из них может сказать: «А вот вы там платную функцию сделали, из-за нее продукт стал хуже, фи!» Только потому, что она за деньги, эта функция покажется кому-то ненужной. Хотя для бизнеса это может быть ключевая вещь, которая приносит прибыль.

4.5.3 КЛАССИЧЕСКИЙ МЕТОД ПРИОРИТИЗАЦИИ БАГ-ЛИСТОВ

Поскольку баг-листы часто бывают очень объемными, в какой-то момент нужно принимать волевое решение: готовы ли мы выпускать продукт, или нужно шлифовать дальше. Помогает внятная система приоритетов. Вот уже более 15 лет мы используем список приоритетов от 0 до 8 (кратко уже говорили о нем во второй главе):

0 — Критические баги

Когда тестер уткнулся и не может дальше проверять, когда падает система либо что-то ломается — в общем, когда дальше невозможно.

1 — Критичное юзабилити и забытые фиичи

Здесь мы применяем в том числе метод покраски бэклога, технического задания, либо прототипа (в зависимости от того, что у нас есть на руках), чтобы определить, не пропустили ли мы что-то.

2 — Некритичные баги

Баги есть, но они не мешают тестировать продукт дальше, либо они позволяют пройти полностью по цепочке либо заказа, либо чего-то еще — то есть, дают полностью проверить наш продукт.

3 — Некритичное юзабилити

4 — Тексты

8 — Хотелки / не будем делать / на усмотрение менеджера

Да, промежуток между четвертым и восьмым приоритетами сделан намеренно: менеджер при необходимости может докидать туда еще какие-то задачи.

Для баг-листов способ хороший, но у него есть проблема. По большому счету мы в баг-листах оптимизируем метрику «готово — не готово». Объем работ понятен. Но часто встречается, что в некритичное юзабилити пытаются пропихнуть что-то такое, что находится на грани — вроде бы, юзабилити и, вроде бы, неплохо такое сделать, но по факту оно тянет на какую-то очень серьезную фиичу.

Другая проблема — субъективизм тестировщика, который часто приходится перепроверять. Иногда это довольно трудоемкая история, когда вы лично просматриваете все баг-листы: смотрите, чего он там такого понаписал, и принимаете решение, что выкинуть, а что оставить.

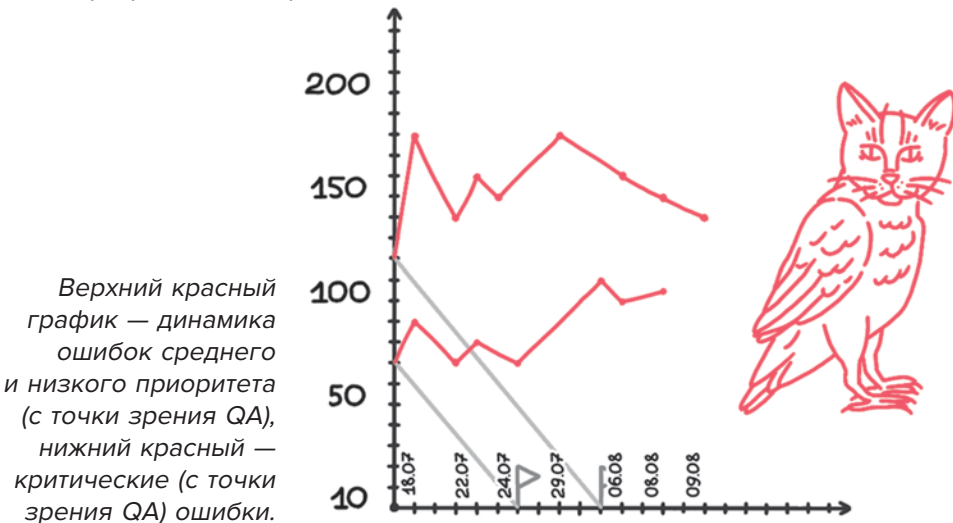
Прелесть в том, что мы можем управлять качеством продукта. Например, разрешив релизить продукт, если там остались незащищенными баги с приоритетом больше четвертого. Для приоритизации бэклогов такой метод не годится — для них нужны совсем другие критерии, и об этом будет через пару страниц. А теперь давайте обсудим, как тестировщик может вредить выпуску продукта и почему важно не только следить за бэклогом, но и за тем, какие баги правит команда.

4.5.3.1 Важное и пустяк

Как тестировщики плодят херабору

На одном крупном проекте менеджер и тестировщик «застопорили» команду (если так можно выразиться) на полгода. Шесть месяцев багфикса вперемешку с мелкими улучшениями. Баг-лист никогда не был меньше 200–300 пунктов. Даты релиза переносились. Команда вымоталась. Руководство нервничало. Мы даже вывели на доске динамику — количество оставшихся багов. Идея была в визуализации нашего прогресса. Ага, щаз!

Нет, сама по себе идея неплохая. Но после каждого тестового прогона график только рос.



Кончилось сменой менеджера. Затем приоритизацией баг-листов лично Product Owner. И запретом смешивать «баги» и «улучшения» в одну кучу. Потребовался еще месяц работы, но в итоге продукт был выпущен.

Печальная новость: с большими баг-листами перед выпуском продукта надо работать точно так же, как и с бэклогами. То есть, вникать и расставлять приоритеты ровно так же, как это делает Product Owner в бэклоге. Отдать приоритизацию на откуп тестировщику можно только в небольшом диапазоне. Какая, боги, херня!

У многих опытных тестировщиков есть боязнь пропустить ошибку. Продукт при этом лучше вообще не выпускать: а то вдруг пользователи чего-то найдут. Главное — обезопасить себя от критики. Создать алиби.

Однако прикрывается это благородными доводами. Перфекционизмом и педантичностью. «Ну я, как ответственный тестировщик, не могу такое пропустить, это ВАЖНО!»

Предъявить что-то тестировщику сложновато. Это палка о трех концах. Попросишь не заваливать команду мелочевкой — будут пропускать ошибки и отговариваться, что вы сами об этом попросили. Не попросишь — завалят мелочами.

— Как ты думаешь, сколько пользователей прочувствуют на себе, что на экране «Настройки» в полях ввода при выделении текста слегка едет шрифт? — это было начало жаркого спора с тестировщиком.

— Все! Все 100% с этим столкнутся! Рано или поздно. Это важно для меня, я как профессионал не могу такое пропустить.

Ну да. В одном из бесконечных вариантов вселенных, есть и такой, где действительно все 100% пользователей с этим столкнутся. Но на практике — всем почти плевать. Говорят, Windows 95, мегапопулярная в 90-х операционная система, выпускалась с миллионом известных багов. МИЛЛИОНОМ.

«Плодить херабору» — так мы называли поведение, когда под видом обеспечения качества команда заваливается сотнями мелочей. А выпуск продукта отходит на второй план.

При удалении задачи с детальной задачи после перехода в проект задачи из избранного открывается дровер и ранее открытая папка



Херабора

Дополнительный признак — запутанные и сложные формулировки, через которые приходится продираться мозгу. Тратить по пять-семь минут только на то, чтобы понять, что имел в виду автор. Мозг начинает кипеть на пятом-шестом пункте. А у нас их несколько сотен, и с этими списками работает десяток человек. Сотни часов улетают на расшифровку зауми.

Вообще лучше работать с QA, которые не плодят херабору, хорошо чувствуют продукт и понимают, что важно, а что пустяк. Но, допустим, вам не повезло. Очень жаль.

Как это победить:

- ▶ Посадите QA оказывать поддержку реальным пользователям. Протрезвляет. Картина мира быстро встает в правильное русло.
- ▶ Объясните QA, что он делает. Введите в обиход понятие «херабора». Сначала можно не сильно расшифровывать, что это такое. Когда вы видите херабору — говорите про это. Рано или поздно люди почувствуют, что значит эта терминология.
- ▶ Научите QA писать ясные формулировки. Как это делать — мы говорили в первой главе в § 1.8. Объясните, почему важно тратить минимум мыслетоплива для вникания в то, что написал QA.
 - Может понадобится несколько циклов объяснений.
 - Ясность должна быть и в заголовке, и в описании. Заголовки должны быть такие, чтобы было понятно, о чем идет речь, без необходимости много читать и думать.
 - Достаньте старый, заскорузлый, как дерьмо мамонта, заумно сформулированный баг и попросите QA расшифровать, что же он имел в виду. Зачастую он сам забывает и впадает в ступор. Для QA было ОЧЕНЬ ВАЖНЫМ написать этот баг, пока он был на нем сфокусирован, но теперь он сам видит, что это — чушь (когнитивное искажение — «что сейчас в фокусе, то и важ-

но»). Прекрасно, теперь у вас есть пример хераборы, можно напоминать про него. Но не слишком часто — наша задача не задрючить, а научить.

- ▶ Введите ясную систему приоритетов. Но не скатываясь в формализм: с формализмом у QA бороться формализмом — себе дороже. Забастовка по-итальянски (все работают по правилам, но дело не делается).
- ▶ WYSIWYG. То, что видишь, то и есть на самом деле. Если вам показалось, что QA формулирует заумь — значит, вам не показалось. Безжалостно отсекайте заумные вам формулировки. Если там будет что-то реально важное — QA вам про это скажет.
- ▶ Предложения и улучшения от QA собирайте в отдельный список. Время от времени просматривайте его, и рациональные вещи добавляйте в бэклог. Херабору — отсекайте.

У нас есть встроенный детектор чуши. И он работает очень хорошо. Не давайте чуши помешать выпуску продукта.

4.5.4 ОЦЕНКА ЗАДАЧ

Любая приоритизация должна отталкиваться от оценок, которые нам дали. Ведь трудоемкость действительно влияет на приоритетность какой-то функции. Но как определить трудоемкость, и в какой момент

УНИВЕРСАЛЬНАЯ ТАБЛИЦА ОЦЕНКИ ЗАДАЧ

Изян - 1ч

Изи - 2ч

Просто - 4ч

Вроде просто - 6ч

Норм - 8ч

Норм так - 12ч

Хз - 16ч

Хз как-то - 20ч

Как-то сложно - 24ч

Сложно - 30ч

Очень сложно - 40ч

Блин - 48ч

Пипец - 60ч

Пипец какой-то - 80ч

Вроде изян - 100ч

стоит обсуждать трудозатраты с программистом? Ведь до этого менеджеру все равно нужно выставить хотя бы примерные оценки.

А если серьезно, обычно есть три варианта.

4.5.4.1. Игра престолов

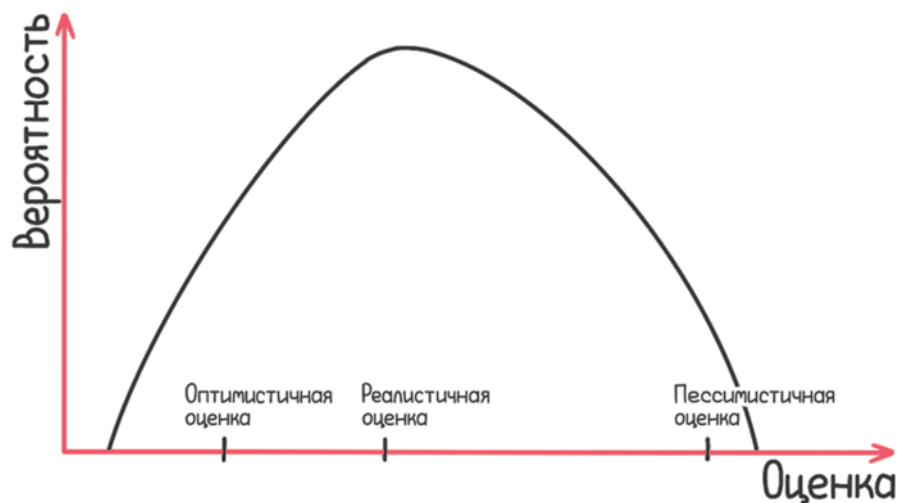
Приходит какой-то «верховный босс» и говорит, что из бэклога должно быть сделано прямо сейчас, и спускает вам оценки сверху. Это может быть технический директор, который расставляет оценки всех задач и говорит, что какая-то задача делается за столько-то, такая-то — за столько-то. Или это можете быть вы сами :)

Такое часто встречается (хотя мы так стараемся не делать). Обычно по ходу декомпозиции какие-то вещи уточняются, появляются какие-то детали и нюансы. Часто с этими оценками могут быть не согласны разработчики, или реальность может быть немного сложнее, чем вы себе запланировали. Плюс у бизнеса всегда есть подсознательное понимание, за какое количество часов он готов купить эту фичу, а за какое — уже не готов. И это всегда где-то на грани.

4.5.4.2. Оценка из теории вероятности

Мы еще поговорим об этом в главе о декомпозиции и оценках. Здесь только идея.

Берутся три оценки по времени: оптимистичная, пессимистичная и реалистичная, и строится график, называемый Гауссианой.



Для расчета наиболее вероятной оценки применяют формулу: (Оптимистичная + (Реалистичная * 3) + Пессимистичная) / 6.

$$\frac{\text{Оптимистичная оценка} + [3 \times \text{Реалистичная оценка}] + \text{Пессимистичная оценка}}{6}$$

В идеале наиболее вероятная оценка будет чуть дальше, чем реалистичная.



Нюанс в том, что не доказано, что в результате у нас получится нормальное распределение. Например, если взять группу людей и измерить их рост, то получится кривая, как на графике ниже: очень маленьких людей мало, очень высоких людей — тоже мало. И они не входят в норму. Остальные в выделенном диапазоне — и есть нормальное распределение.

Гауссиана подразумевает, что варианты, когда задача выйдет за отметку пессимистичной оценки или вообще никогда не будет сделана, стремительно уменьшаются. Но в IT-среде часто бывают задачи, которые, на первый взгляд, сделать «невозможно» — программисты для них требуют поменять постановку либо продолжают доказывать эту невозможность. Другая ситуация: человек оценил задачу в один



час, потратил все мыслимое и немыслимое время и пришел к выводу, что не может сделать эту задачу.

Поэтому опираться на этот метод можно, но по факту это та же «игра престолов», да и сложновато по каждой задаче давать три оценки плюс считать по формуле. И нет никакой гарантии, что в итоге все не выйдет за рамки этих оценок и расчетов.

4.5.4.3. Planning Poker

Мы уже разбирали этот метод в главе про Scrum. Это наиболее простой и «чистенький» способ получить нормальные оценки, обсудить какие-то фишки, нюансы, детали. Даже на верхнем уровне по бэклогу можно такое проделать.

Минус Planning Poker — это довольно ресурсоемкая операция, поскольку нужно собирать всю команду вместе и читать бэклог. Но если вы применяете методы вроде Story Mapping, вам все равно нужно собираться командой и делать предварительные оценки (хотя бы в днях, неделях — крупных величинах).

Плюсы Planning Poker

- ▶ оценки распределяются в зависимости от опыта сотрудника, и здесь можно договориться;
- ▶ оценки даются сначала закрыто, и только потом открываются — нет давления авторитета (как в случае с оценками, спущенными «сверху»).

4.5.5 ТЕХНИКИ СКОРИНГА

Итак, мы добрались до скоринга. Наша задача — отсортировать бэклог по приоритетам, причем так, чтобы эти приоритеты были числовые и хоть сколько-то объективными. Техник будет две.

4.5.5.1 ICE Scoring

Моя любимая, легковесная техника. Ее стоит попробовать в первую очередь. Каждую идею в бэклоге мы оцениваем от одного до десяти по трем параметрам:

- ▶ Impact — **Влияние на продукт.**
- ▶ Confidence — **Уверенность** в оценке сложности или в оценке влияния.
- ▶ Ease — **Простота реализации.** Чем проще функция, тем у нее выше балл.

Далее перемножаем баллы:

$$CE\ Score = Impact \times Confidence \times Ease$$

Очевидно, что формула все еще подвержена субъективным ощущениям руководителя продукта. Однако сама процедура выставления оценок заставляет задуматься. А это снижает влияние галлюцинаций основателя на итоговый балл.

Теперь нюансы.

Влияние. С ним сложнее всего разобраться. Влияние зависит от самого продукта и целей создателей. Разрабатывая SingularityApp, мы держали в голове такие вопросы:

- ▶ Как много пользователей заметят эту функцию?
- ▶ Добавляет ли эта функция ценности продукту? Добавит ли кайфа?
- ▶ Отстраивает нас от конкурентов (или мы просто догоняем)?
- ▶ Улучшает конверсии?
- ▶ Просили ли об этом пользователи (явно или в ходе интервью)? Как часто?
- ▶ Привлекает ли функция новых пользователей?
- ▶ Удерживает существующих пользователей?

Уверенность — еще один хитрый параметр. Если вы не уверены ни во влиянии, ни в сложности — зафиксируйте это. Мы можем провести дополнительные исследования как по трудоемкости, так и по степени влияния функции на продукт. Взять, например, Riskiest Assumption Test (RAT) из § 4.3.3, если идея рискованная, или провести дополнительные CustDev-интервью и A/B-тесты. Мне попалась отрезвляющая шкала уверенности, зависящая от источника наших оценок:



Нет задачи держать эту картинку перед глазами в момент выставления приоритетов (это избыточно и неудобно — использовать дробные оценки). Но ознакомиться и учитывать при выставлении оценок — дельная идея.

Итак. Уверенность может основываться на:

- ▶ личном мнении, мнении команды, мнении сторонних экспертов;
- ▶ UX-исследованиях;
- ▶ опросах;

- ▶ интервью;
- ▶ наблюдениях, в том числе за конкурентами;
- ▶ MVP;
- ▶ A/B-тестах;
- ▶ сторонних исследованиях;
- ▶ аналитике;
- ▶ топовых обращениях в техподдержку;
- ▶ топовых запросах от сейлз-менеджеров;
- ▶ топовых запросах клиентов.

Лучше использовать те источники, где есть конкретные метрики и цифры, мнение людей оставлять на «потом».

Для получения ICE нужно перемножить эти три параметра — это и будет приоритет на реализацию.

Плюсы: быстро и просто.

Минусы: у метода довольно ограниченная шкала — можно получить много задач высокого приоритета, потому что по ним станут понятны постановки, сами задачи окажутся простые и будут казаться важными.

4.5.5.2 RICE Scoring

Здесь используется 4 параметра:

- ▶ **Reach — Охват:** сколько пользователей у нас от этой функции получат хоть какое-то удовлетворение. Либо заметят эту функцию, либо будут ей пользоваться. Можно измерять количественно: например, 500 миллионов пользователей. Можно измерять в процентном отношении — например, 30% пользователей будет использовать эту функцию. Цифры прикидывайте сами.
- ▶ **Impact — Влияние:** насколько эта функция на самом деле нам нужна, насколько она нам поможет, насколько функция крутая. Примерно то же самое, что Impact в методе ICE.
- ▶ **Confidence — Уверенность:** аналогично с ICE Scoring, уверенность в наших оценках, прогнозе влияния и охвате.
- ▶ **Effort — трудоемкость.** В часах или, например, Story Point.

Далее считаем суммарный балл:

$$RICE\ Score = \frac{Reach \times Impact \times Confidence}{Effort}$$

Функции, которые дают большой охват, в которых мы уверены и которые хорошо влияют на продукт и при этом дешевые по трудозатратам, находятся в топе по приоритетности в бэклоге.

Как и ICE Scoring, RICE не исключает субъективного влияния и галлюцинаций. **Принципиального выигрыша перед ICE не дает**, однако шкала более гибкая.

4.5.6 СПЕЦИФИЧНЫЙ СОФТ ДЛЯ ПРИОРИТИЗАЦИИ

Вообще, на практике мне хватает интеграции бэклога в Google-таблицы. Там я могу применять любые формулы. Это просто и гибко. Но мне потребовалось несколько вечеров и навыки программирования, чтобы написать интеграцию Jira и Google Docs, и чтобы это заработало как надо.

Есть несколько программ для автоматической приоритизации, например, Hygger. Это система для Product Owner-ов, где есть много разных моделей, которые позволяют «поиграться» задачами.



Hygger

То же самое можно проделать в любом гуглдоке: просто добавить три параметра, собрать данные — он посчитает. Это и будут ваши приоритеты. А дальше вы уже сами выбираете, что именно брать в спринт.

Иногда менеджеры для хранения всех тикетов и задач используют Jira. К сожалению, она не очень удобна для приоритизации: заточена на то, что кто-то расставляет приоритеты вручную, просто перетаскивая тикеты вверх-вниз по бэклогу. Это прикольно, если бэклог не на 1000 строк, но в какой-то момент вам станет утомительно делать это руками.

Кроме приоритетов всегда есть проблема, как распределить по времени реализацию тех или иных задач. Когда есть только приоритеты, нет гарантии, что мы получим связанную систему (например, вы запланировали сделать корзину, а оказалось, что каталога еще нет).

Поэтому помимо приоритетов также стоит использовать диаграммы Ганта, чтобы посмотреть связи в системе, насколько она корректно работает по компонентам и насколько оптимально вы распределяете ресурсы команды по времени.



Подробнее о диаграммах Ганта

В Jira, к сожалению, из «коробки» эта опция не очень хорошо реализована, поэтому тоже могут пригодиться несколько плагинов:

- ▶ Portfolio (тоже дорогой);
- ▶ BigPicture (тяжелый);
- ▶ MS Project;
- ▶ Merlin Project для MacOS.

Итак. Софт есть разный. Но лучше Google-таблиц, кажется, пока ничего не придумали.

4.5.7 СТРАТЕГИЯ, ВЕРА И ВНИМАНИЕ

Тактика без стратегии — это суета перед поражением.

Сунь Цзы

Мало кто не считает себя стратегом. Но что такое «стратегия»? Мне нравится это определение:

Стратегия — это выделение главного в ущерб второстепенному для достижения невозможного.

Что будет «важным», а что «пустяк»? Хороший вопрос. Емкий. Нет ни одной техники, которая бы **гарантировала** правильный ответ. Но есть те, которые повышают шансы. Я рекомендую использовать ICE для скоринга бэклогов, систему приоритетов от нуля до восьми — для баг-листов и JTBD+Story Mapping для поиска стратегии. В какой-то момент все равно придется поверить интуиции и рискнуть.

Повторюсь, поверить придется несколько раз!

1. В то, что в принципе есть рынок.
2. Что на рынке есть сегмент с сильной потребностью.
3. Что мы в состоянии сделать для него продукт (хватит экспертизы, технологии, денег и т. д.).
4. Что у нас это купят.
5. А мы на этом заработаем.
6. И сможем масштабировать.

По сути, единственное, чем мы управляем — это **внимание**. Нас и нашей команды. В какой-то момент приходится отбросить методики и рискнуть.

4.5.8 ROADMAP И BACKLOG GROOMING

Иди вниз бэклога, да там и подохни!
(пожелание легкой жизни очередной хотелке)

Про Roadmap или дорожную карту проекта мы уже говорили в главе 3, когда обсуждали Scrum. Здесь еще раз акцентируюсь: бэклог — особенно большой, жирный бэклог, который потребовал применения специальных техник скоринга, — не дает понятной картинки. Более того, в какой-то момент он все запутывает. И запугивает.

Что делать? Вместе с бэклогом важно иметь Roadmap развития продукта. Мы используем стратегические сессии для его формирования раз в полгода. И актуализируем Roadmap раз в пару месяцев. А сам бэклог периодически очищается от хераборы. Прямо берем и удаляем задачи оттуда. Их очень жалко. Но Sorry, Buddy. Если ты был ценный, тебя обязательно принесут еще раз. По-умному это называется Backlog Grooming. Чик-чик-чик. Если этого не делать — сбивается фокус, и рассеивается то единственное, чем мы можем управлять — внимание.

АНИЩЕ БЭКЛОГА



Обожаю этот комикс. Оригинал поищите в интернете. Мы его перерисовали, чтобы не нарушить авторское право.

Итак. Кроме техник скоринга, на больших продуктах нам понадобится Roadmap и периодическая прополка бэклога от хераборы. Sorry, Buddy. Пиф-паф-пуф!

4.6. UML. САМЫЕ НУЖНЫЕ ДИАГРАММЫ

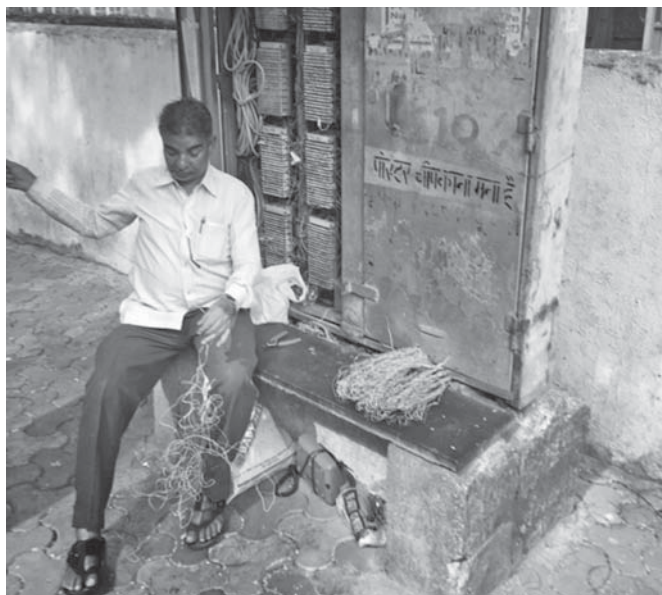
У каждого есть план действий до первого удара в челюсть.

Тот еще проджект-менеджер Майк Тайсон

UML — это стандарт из пары десятков диаграмм, позволяющих формализовать работу программного обеспечения, бизнес-процессы, структуры и прочее, прочее, прочее. По моему мнению, самые нужные диаграммы — это диаграмма последовательности и диаграмма развертывания. Факультативно почитайте Википедию или даже хорошую книгу по UML. Но не делайте из этого культа. Проектирование ради проектирования — это бред.

Я не сторонник жесткой формализации и ненавижу большие, толстые технические задания. Однако бывают ситуации, когда все так запутанно, что разработчики сами плохо понимают, что такое они наворотили. Это интеграции, сложная математика или микро-сервисные архитектуры.

Но боже вас упаси делать UML-проектирование в самом начале и пытаться построить идеальную схему взаимодействия. Это будет дырявая абстракция, которая потребует месяцев работы и разломается почти сразу, как дело дойдет до реализации. Работающий код важнее исчерпывающей документации.



Вот такие, брат, дела!

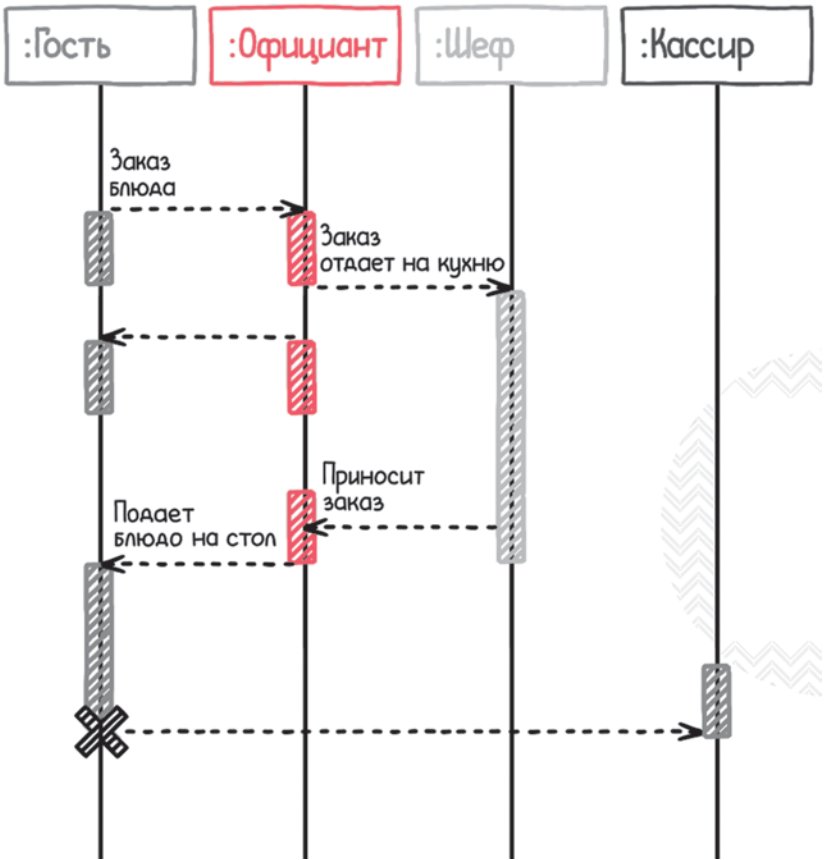
Так вот. Когда случится задница, все будет зависеть и тормозить — большинство разработчиков ниже уровня Super Senior инфантильно поднимут лапки кверху. Мол, сделать ничего нельзя. Типа, мы тут наворотили, сами не понимаем чего — уберите. Во всем виноват Чубайс. Эй, кто-нибудь, сделайте что-нибудь. И будут ждать, что кто-то придет и всех спасет. Менеджер, решай.

Проблема в том, что если вы не технарь с огромным опытом — ничего вы там не решите. Все спрятано под капотом. Запутано. И ни черта не понятно. Как на войне.

Что делать? Распутываться. И тут UML может хорошо сработать.

4.6.1 ДИАГРАММА ПОСЛЕДОВАТЕЛЬНОСТИ

Возьмем упрощенный сценарий «Заказ в ресторане».



Есть главные лица сценария: клиент, официант, повар и кассир. В UML их называют «акторы» — какие-то действующие лица или объекты, с которыми происходят операции.

Далее таймлайн — линия жизни объектов, она расположена вертикально сверху вниз.

Сообщения между акторами бывают нескольких типов. Основные, на которые нужно обратить внимание — это синхронные (сообщение инициирует обработку, и отправитель ждет ее завершения) и асинхронные (сообщение инициирует обработку, но отправитель не дожидается ее завершения). В данном случае у нас асинхронные взаимоотношения.

Читаем диаграмму:

- ▶ гость заказывает еду у официанта;
- ▶ официант его слушает и далее идет на кухню, чтобы передать заказ повару;
- ▶ повар начинает готовить;
- ▶ параллельно с этим официант приходит к клиенту и наливает ему винишко;
- ▶ пока клиент пьет винишко, повар приносит еду на стойку официанту;
- ▶ официант забирает заказ и относит его гостю. Больше клиент с официантом дел не имеет (официант выпадает из системы);
- ▶ гость оплачивает свой заказ у кассира и уходит.

В McDonald's или Michelin пять звезд диаграмма будет другая, поскольку процессы обслуживания там устроены иначе. Все зависит от конкретной ситуации, где применяется диаграмма.

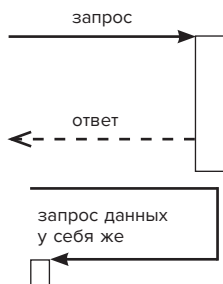
4.6.2 ДИАГРАММА ПОСЛЕДОВАТЕЛЬНОСТИ 2.0

В классической UML второй версии диаграмма стала еще более подробной, разбитой на более мелкие элементы и связи. Подходит для детальной проработки структур и систем.

Есть разные типы объектов, которые могут между собой взаимодействовать.



односторонняя связь
→



Актор — участник процесса.

Форма заказа — это какой-то интерфейс. Причем не обязательно экранный интерфейс, он может быть физическим: картридер в банкомате или форма регистрации на сайте.

Менеджер заказа. Или менеджер паролей, или любой другой программный модуль, к которому идет обращение от программной подсистемы. Например, менеджер паролей отвечает за то, чтобы делать авторизацию/регистрацию.

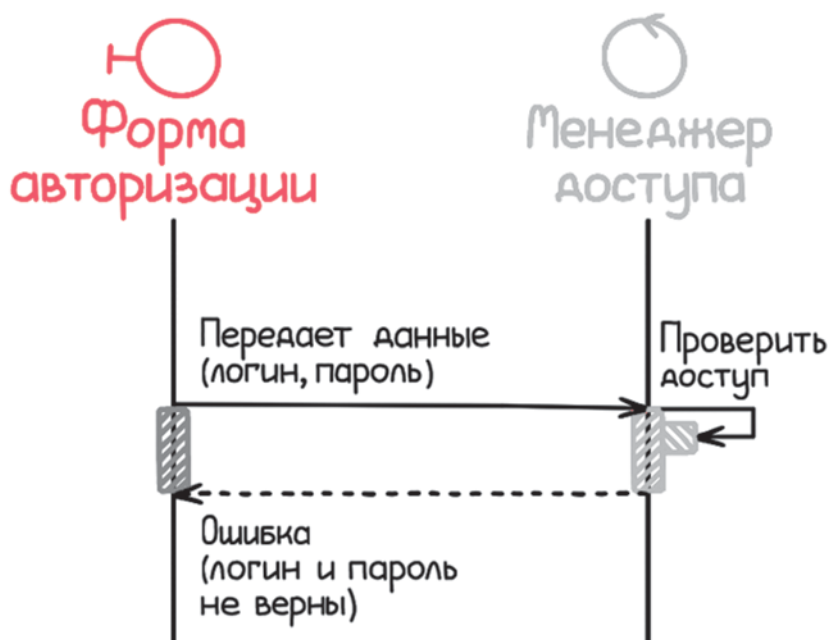
Заказ — это объект, который создается, либо с которым происходят какие-то операции. Например, счет или пользователь в базе данных.

Синхронный вызов (связь). Напомним, что пользователь отправил сообщение в систему, и пока ему она не ответит, он ничего не может делать.

Асинхронный вызов (связь). А тут пользователь отправил сообщение в систему, и может продолжать с ней взаимодействовать, не дожидаясь ответа.

Запрос данных у себя же. Объект сам у себя запрашивает данные.

Пример диаграммы



Читаем диаграмму:

Это форма авторизации. Она передает данные в менеджер паролей: логин и пароль. Менеджер паролей проверяет, есть ли такой пользователь в системе. Если нет, то выводит ошибку в форме авторизации. Еще форма авторизации ожидает ответ, пока менеджер пароля проверяет данные, и сообщает форме авторизации, какое сообщение необходимо вывести пользователю.

4.6.3 ПРАКТИКА ЕСТЬ МЕРИЛО ИСТИНЫ

Ситуация (все совпадения случайны, ситуация выдуманная и выдуманная из пальца):

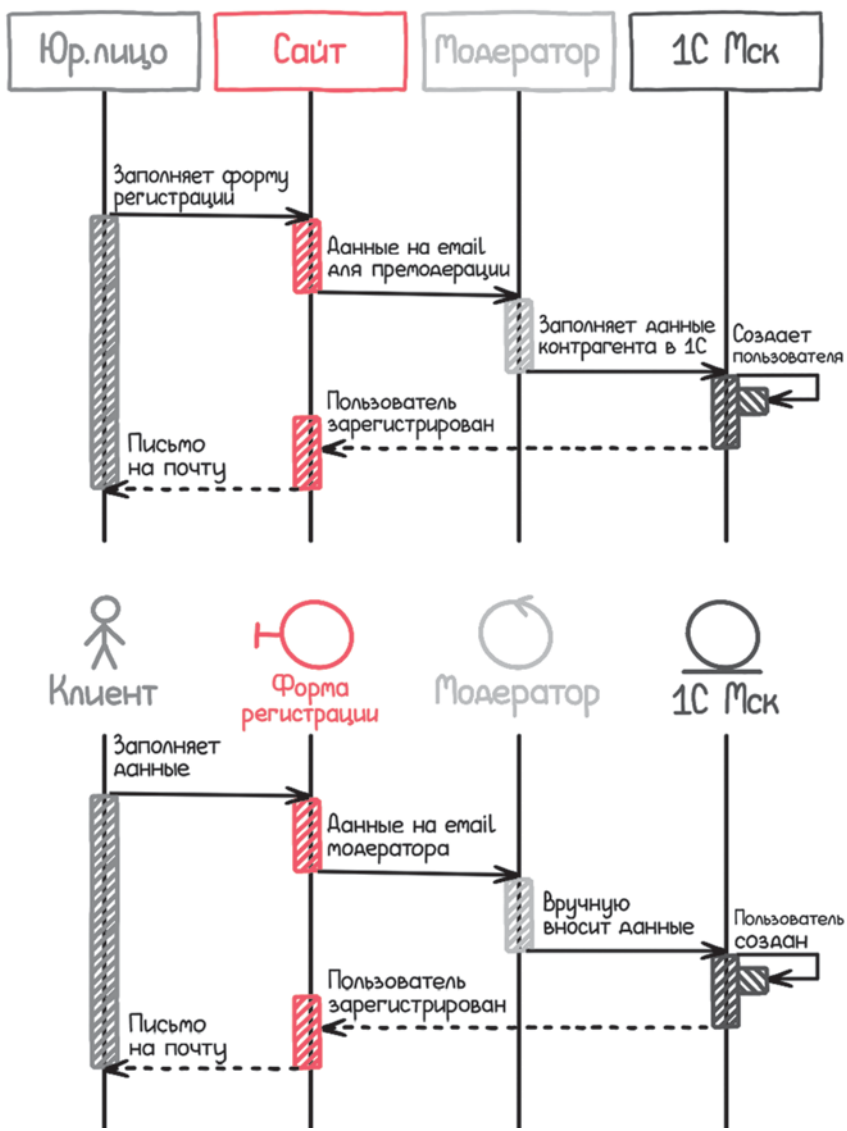
Идет интеграция сайта с 1С — разрабатывается протокол интеграции. По ходу обсуждения выясняется:

- ▶ 1С-ки будет две: Тверская и Московская;
- ▶ Основная интеграция — с Тверской;
- ▶ Вся товарная база хранится в Тверской 1С;
- ▶ Помимо товаров, нужна выгрузка контрагентов;
- ▶ Контрагент привязывается при регистрации к Москве или к Твери;
- ▶ Контрагенты хранятся частично в Московской, частично — в Тверской базе;
- ▶ Не у всех текущих контрагентов заполнены поля «емейл» и «телефон»;
- ▶ Создание новых контрагентов требует премодерации;
- ▶ Кроме того, заказы необходимо передавать в CRM.

Клиент ожидает, что ему предложат решение, как будет работать схема с CRM, сайтом и 1С-ками.

Решение: составляем диаграммы последовательности. Одна и та же структура будет описана в двух вариантах UML-диаграмм.

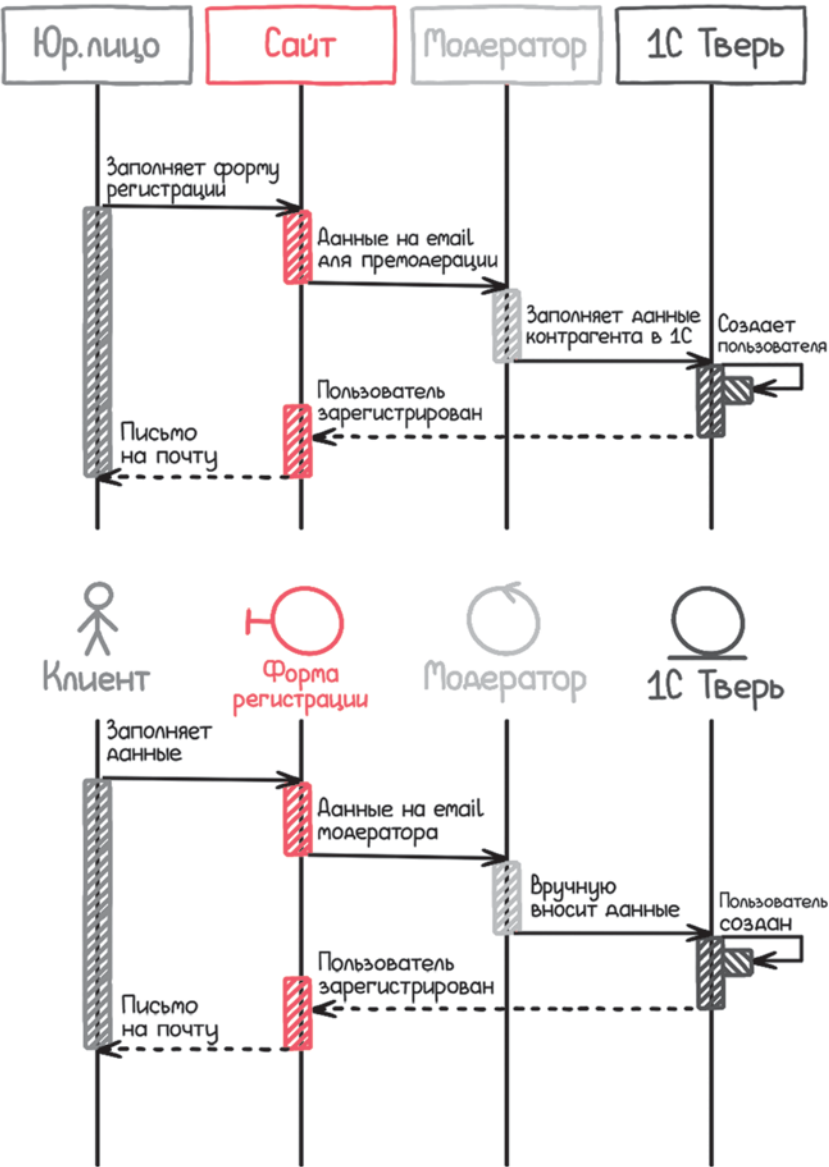
Диаграмма 1: Регистрация юридического лица (Москва)



Читаем диаграмму:

Юридическое лицо заходит на сайт и заполняет форму регистрации, указывая город Москва. Сайт передает данные на е-мэйл модератору для премодерации. Модератор вручную заполняет данные контрагента в Московской 1С, создавая нового пользователя. Сайт получает сообщение, что регистрация завершена, и отправляет письмо пользователю.

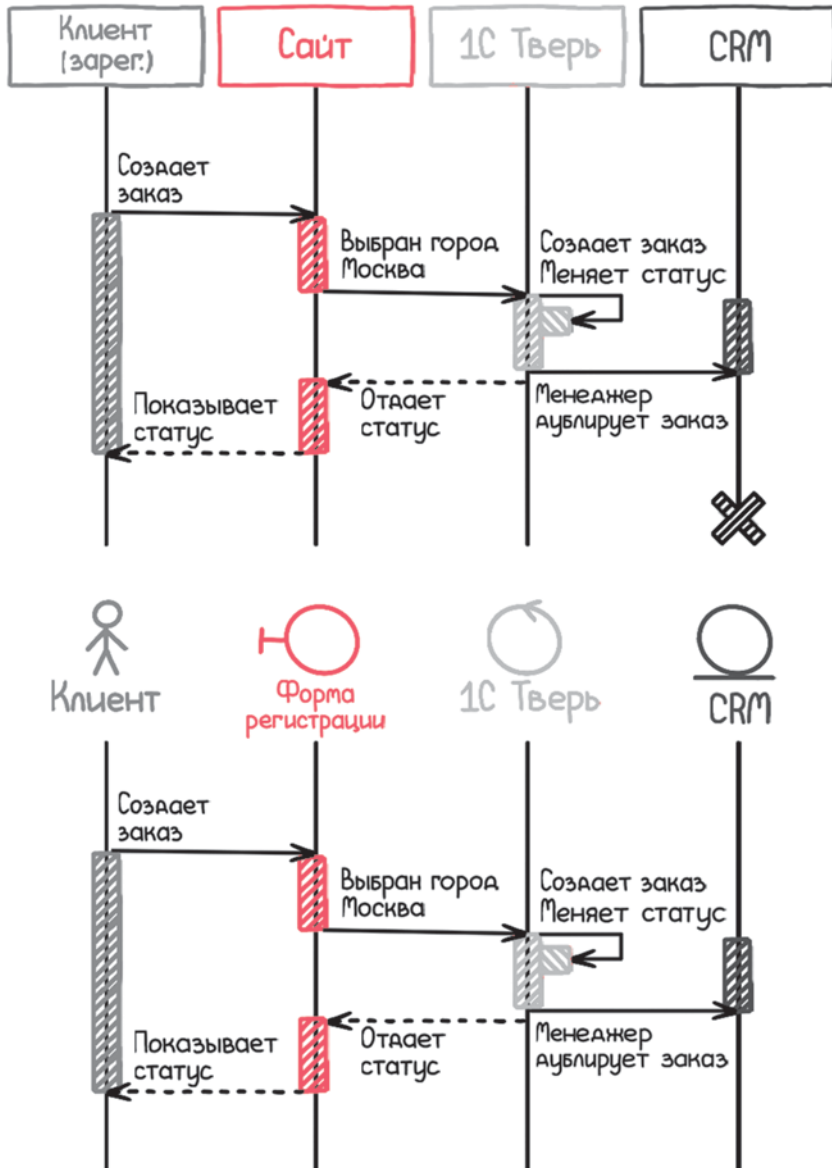
Диаграмма 2: Регистрация физического лица (Тверь)



Читаем диаграмму:

Физическое лицо заходит на сайт и заполняет форму регистрации, указывая город Тверь. Сайт передает данные в Тверскую 1С, создавая нового пользователя. Сайт получает сообщение, что регистрация завершена, и отправляет письмо пользователю.

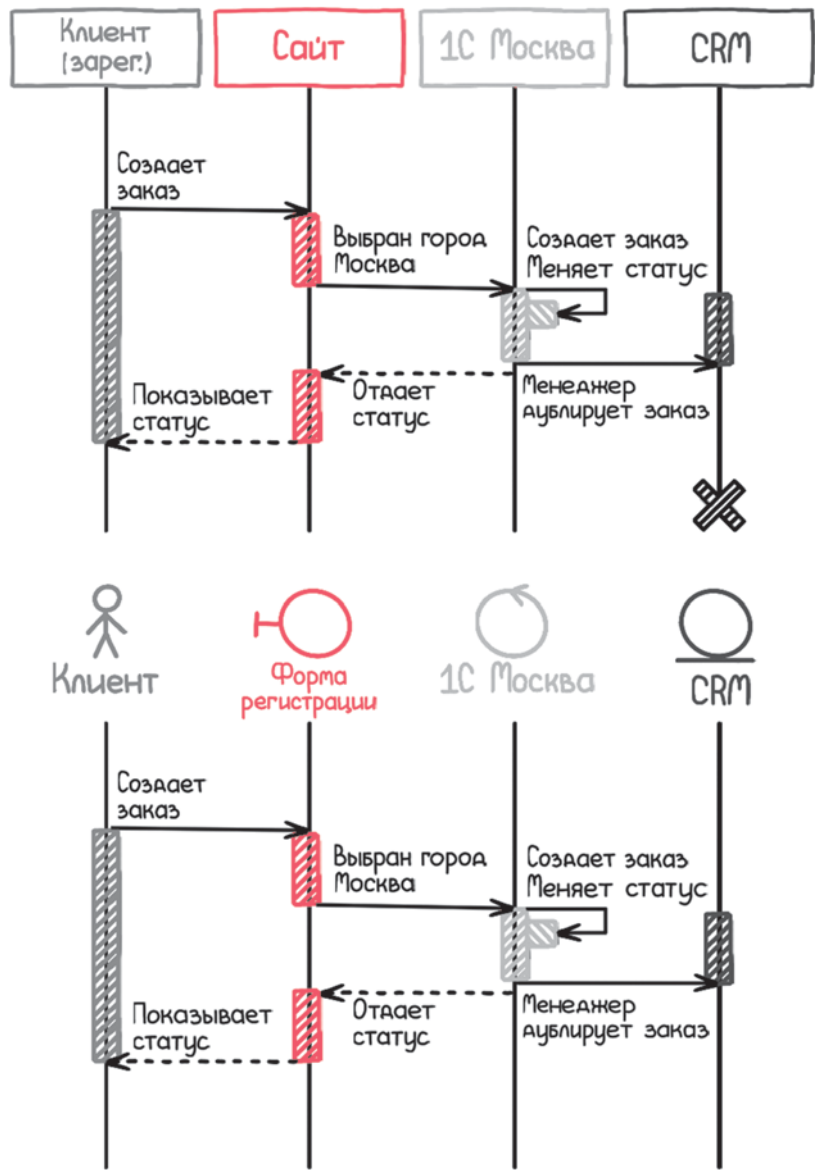
Диаграмма 3: Оформление заказа (Тверь)



Читаем диаграмму:

Зарегистрированный клиент создает заказ на сайте, выбирая город Тверь. В Тверской 1С создается заказ (и менеджер вручную дублирует заказ в CRM). После создания заказа в 1С у заказа меняется статус, который передается сайту и показывается клиенту.

Диаграмма 4: Оформление заказа (Москва)

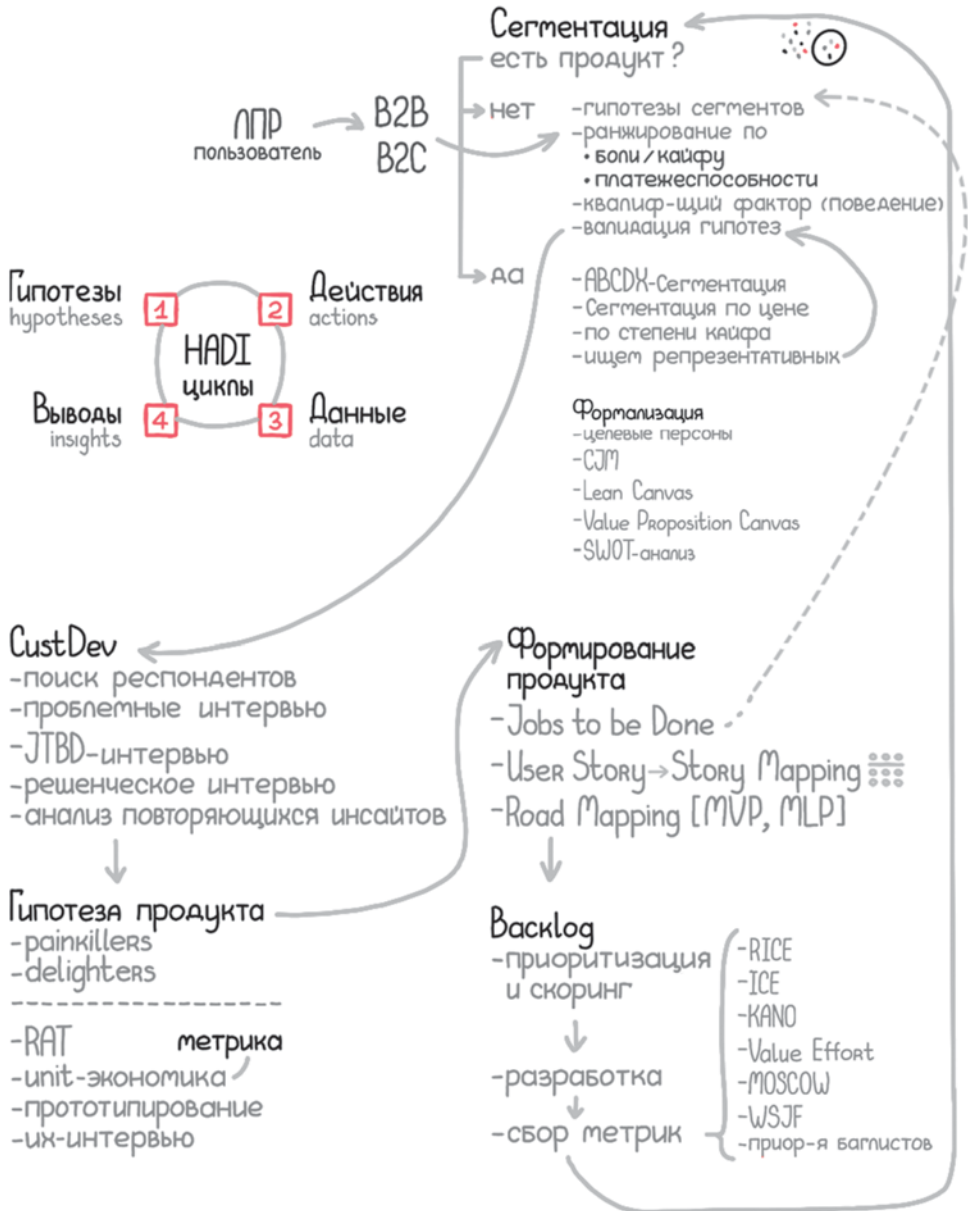


Читаем диаграмму:

Зарегистрированный клиент создает заказ на сайте, выбирая город Москва. В Московской 1С создается заказ (и менеджер вручную дублирует заказ в CRM). После создания заказа в 1С у заказа меняется статус, который передается сайту и показывается клиенту.

4.7 СОБИРАЕМ ВСЕ ВМЕСТЕ

В начале главы я уже показывал эту картинку. Действительно, тут куча всего. Но это связанные вещи, и они имеют смысл.



Итак.

1. Если у вас заказная разработка — сделайте агрегацию требований. Опишите задачи стейкхолдеров. Сделайте анализ текущего трафика. Продумайте сегменты. Используйте поведенческие паттерны в качестве квалифицирующего фактора. Опишите целевых персон (аватары). Определите их боли и потребности. Анализируйте конкурентов, подсматривайте фишки. Если есть ресурсы — постройте CJM по главным персонам. В конце концов — получите структуру будущего проекта и дальнейший план развития. Используйте MindMap, или Lean Canvas, или просто Google-таблицы для этих целей. Все должно быть в простом, легко редактируемом формате, а не в каком-нибудь Фотошопе.
2. Если вы метите в продуктовую разработку, помогаете руководителю продукта или сами играете его роль — используйте продуктовые техники из § 4.3. Особенно акцентирую на CustDev, JTBD, RAT, MVP/MLP. Тестируйте гипотезы сегментов, силу боли и объемы сегмента. Проводите интервью, делайте тесты рискованных гипотез.
3. Посчитайте Unit-экономику. Определитесь с метриками, которые вы будете собирать. Не старайтесь считать все двух-трех-четырёхбуквенные аббревиатуры, которые гуглятся по словам «Unit-экономика» — там слишком много зауми и шарлатанства. Начните с пяти простых метрик. Используйте HADI-циклы для тестирования гипотез и оптимизации экономики.
4. В заказной разработке нас постоянно просят писать технические задания (которые затем разбиваются на бэклоги, но без бумажки ты букашка). В продуктовой, или в менее формальных проектах по **Time & Material**, все прекрасно работают с бэклогом.
5. Однако! Все, чем мы управляем — это вниманием. Нашим и команды. Используйте техники формирования, скоринга и приоритизации бэклогов, Roadmap и Backlog Grooming, как мы это рассмотрели в § 4.5.8, чтобы не запутаться и не сойти с ума.
6. Если запутались и сошли с ума — распутывайтесь. UML-диаграммы как раз для этого.

Информации много. Техник и подходов — тоже. Не нужно пытаться использовать это все разом в одном проекте. Сойдете с ума. Используйте эту главу как арсенал, склад оружия. Один возьмет меч. Вто-

рой — лук. Третий — пращу. А весь арсенал с собой никто не утащит. Пробуйте, экспериментируйте и используйте то оружие, которое будет эффективно и удобно именно для вас.

Успехов!

4.8 ДЛЯ ТРЕНИРОВКИ

- ▶ Постройте сегменты пользователей по вашему проекту. Проранжируйте по силе боли и размеру сегмента. Используйте поведенческие паттерны для классификации.
- ▶ Проведите 12 решенческих или проблемных интервью с пользователями каждого сегмента. Поменялся ли у вас ТОП сегментов? Какие инсайты вы получили из интервью?
- ▶ Постройте CJM по паре ключевых аватаров вашего проекта. Что мы должны переделать в нашем продукте?
- ▶ Какие ключевые риски есть в вашем проекте? Как их проверить? Что можно с ними сделать?
- ▶ Попробуйте Story Mapping для формирования бэклога.
- ▶ Причешите свой бэклог. Какой метод вам ближе? Попробуйте ICE.
- ▶ Изучите остальные UML-диаграммы. Какие могли бы быть вам полезны? На каком уровне?

ЛИТЕРАТУРА

- ▶ ГОСТ 2.105–95 — общие требования к выполнению конструкторских и технологических документов.
- ▶ Роджер Бест, «Маркетинг от потребителя».
- ▶ Алан Купер, «Психбольница в руках пациентов».
- ▶ Элияху Голдратт, «Теория ограничений».
- ▶ Джамшид Гараедаги, «Системное мышление. Как управлять хаосом и сложными процессами. Платформа для моделирования архитектуры бизнеса».
- ▶ Дэвид Аллен, «Как привести дела в порядок: искусство продуктивности без стресса».
- ▶ Дино Эспозито, «Архитектура корпоративных приложений».
- ▶ Дональд Норман, «Дизайн привычных вещей».

- ▶ Уильям Эдвардс Деминг, «Выход из кризиса: Новая парадигма управления людьми, системами и процессами».
- ▶ Дэниел Пинк, «Драйв: Что на самом деле нас мотивирует».
- ▶ Эяль Нир, «На крючке. Как создавать продукты, формирующие привычки».
- ▶ Джозеф Кемпбелл, «Тысячеликий герой».
- ▶ Синди Альварес, «Как создать продукт, который купят. Метод Lean Customer Development».
- ▶ Даниэль Канеман, «Думай медленно. Решай быстро».
- ▶ Гэри Клайн, «Источники силы. Как люди принимают решения».
- ▶ Роберт Фитцпатрик, «Спроси маму: Как общаться с клиентами и подтвердить правоту своей бизнес-идеи, если все кругом врут?»
- ▶ Александр Остервальдер, «Разработка ценностных предложений. Как создавать товары и услуги, которые захотят купить потребители. Ваш первый шаг» и «Построение бизнес-моделей: Настольная книга стратега и новатора».
- ▶ Стив Бланк, Боб Дорф, «Стартап. Настольная книга основателя».
- ▶ Джеффри Мур, «Внутри торнадо. Стратегии развития, успеха и выживания на гиперрастущих рынках».
- ▶ Карл Вигерс, «Разработка требований к программному обеспечению».
- ▶ Эрик Рис, «Бизнес с нуля: Метод Lean Startup для быстрого тестирования идей и выбора бизнес-модели».
- ▶ Jim Kalbach, The Jobs To Be Done Playbook: Align Your Markets, Organization, and Strategy Around Customer Needs 1st Edition.
- ▶ Клейтон М. Кристенсен, «Дилемма инноватора. Как из-за новых технологий погибают сильные компании».
- ▶ Фридрих Ницше, «Так говорил Заратустра» (глава «О трех превращениях»).
- ▶ Александр Леоненков, «Самоучитель UML 2».
- ▶ Гради Буч, Джеймс Рамбо, Ивар Якобсон, «Язык UML. Руководство пользователя».
- ▶ Валентин Полежаев, Андрей Андрианов, Антон Хританков, «Проектирование на UML. Сборник задач».
- ▶ Джеймс Рамбо, М. Блаха, «UML 2.0. Объектно-ориентированное моделирование и разработка».



Digital-библиотека Tagline.



Пособие по Google Analytics для начинающих.



Как использовать Яндекс.Метрику — руководство для новичков.



Электронная книга от Intercom Inc., Jobs To be Done (на английском).

Программное обеспечение для интеллект-карт:

- ▶ XMind;
- ▶ XMind Zen;
- ▶ Freemind;
- ▶ MindJet;
- ▶ MindNote;
- ▶ Mindmeister.

ПОЯСНЕНИЯ

Mindmap — интеллектуальная карта или диаграмма связей: большой документ, где множество элементов можно связывать между собой, отражая структуру, вехи, элементы проекта наглядно.

CRM — система управления взаимоотношениями с клиентами. Особое ПО для компаний, помогающее менеджерам систематизировать данные о клиентах и автоматизировать стратегии взаимодействия с ними.

Чендж-реквест — формальное обращение, чтобы получить официальное разрешение на изменения в предметной области проекта, проектных решениях, методах, плановых сроках, затратах или других показателях.

УТП — уникальное торговое предложение: то, что выделяет бренд среди других. Шампунь от перхоти № 1, порошок в оранжевой упаковке вместо кипячения, особый соус в большом бургере от сети ресторанов быстрого питания. Ну, вы поняли :)

301 редирект — перенаправление посетителей страницы и роботов поисковых систем на новый адрес этой страницы. Обычно это нужно:

1) при переезде сайта с одного домена на другой: было www.rogaicopyta.ru, а стало www.copyta-i-roga.ru; 2) при переносе проекта с одной CMS (системы управления контентом на сайте) на другую: был Wordpress, а стал 1С-Битрикс; 3) при смене URL-адреса страницы.

Фримииум-модель — бизнес-модель, при которой пользователям предлагают приложение или продукт с бесплатным функционалом, при этом у него есть возможность перейти к расширенной версии с большим количеством фиш и плюшек уже за деньги.

PIVOT — разворот бизнес-модели, значительное изменение в бизнес-процессах и логике получения прибыли для тестирования нового направления развития. С PIVOT можно проверить фундаментальные гипотезы о продукте, бизнес-модели или механизме роста.

Стилобат — верхняя часть ступенчатого цоколя здания, в том числе подножие колоннады.

Рутовые пароли — пароли администратора системы, которые дают ему неограниченные права: изменять, удалять и делать вообще все что угодно с данными.

Time & Material — оплата за результат, основанная на трудозатратах.

ERP-система — система планирования ресурсов предприятия.

ЧАСТЬ 5



ДЕКОМПОЗИЦИЯ, ОЦЕНКА, СМЕТЫ, СЕТЕВЫЕ ГРАФИКИ, ИЛИ КАК ДАВАТЬ ПРОГНОЗЫ

Подготовка смет и оценка проектов — это сложно. И всегда было сложным. Требуется высокая компетенция. Но ошибки и неточности будут всегда. В этой главе мы разберем, почему так происходит, поговорим о природе неопределенности и как с ней жить.

5.1 КАК ДАТЬ АДЕКВАТНУЮ ОЦЕНКУ ВРЕМЕНИ, КОГДА НЕОПРЕДЕЛЕННОСТЬ БЬЕТ ПО БАШКЕ

Большинство людей не умеют адекватно оценивать сроки выполнения задач. Ой как это заставляет порой понервничать... Тут и «дедлайн подкрадывается незаметно». И перестраховка в 500% на всякий случай (все равно не хватает). И отжимание «заведомо раздутых сроков», чтобы исполнитель пообещал что-то более приемлемое. И невнятные бормотания вместо конкретных цифр.

В этой главе собраны и структурированы принципы и методы, с помощью которых можно научить себя и других давать адекватные оценки. В начале — общие принципы и чуть-чуть математики. В конце — конкретика для разработчиков.

5.1.1 САМЫЙ НЕНАВИСТНЫЙ ВОПРОС

Выступая на десятках конференций, я часто спрашивал людей в зале: «На какой вопрос вы больше всего не любите отвечать?» И всегда ответом было: «Когда будет готово?!» Вопрос волнует и вызывает эмоции. Вот прямо сейчас подойдите к вашему коллеге и спросите его, когда будет готово. Знаете, что вы, скорее всего, услышите?

Нет, необязательно вам укажут направление и пожелают доброй дороги. Все же кругом много культурных людей. Зато почти наверняка человек начнет рассказывать о своих проблемах.

Понимаете? На вопрос «Когда?» отвечают рассказом о проблемах! Так делает почти все человечество.

Еще пример — для тех, кто помнит университетский курс программирования. Какой тип данных должна вернуть программа на запрос «Когда?». `DateTime` или что-то типа того. Человек же на это стабильно выбрасывает `Exception`. Да не один, а много, желательно с `General Protection Fault`. Это реально баг в прошивке практически всех людей!

Что за гадость такая? Почему люди так сильно не любят оценивать сроки? Ведь все просто:

$$\text{Длительность работ} = \frac{\text{Объем работ}}{\text{Производительность}}$$

5.1.2 НЕОПРЕДЕЛЕННОСТЬ БЪЕТ ПО БАШКЕ

Судя по всему, даже Бог подчиняется принципу неопределенности и не может одновременно знать положение и скорость частицы.

Стивен Хокинг

Корневая причина в том, что, оценивая объем работ, мы сталкиваемся с неопределенностью.

Мы вынуждены предсказывать будущее: строить прогноз и нести за него ответственность. Начальники и клиенты требуют этого от нас, а мы — от своих подчиненных.

Но прогнозы нужны нам как опора. Как попытка отгородиться от неопределенности. Как попытка удовлетворить главное желание. И я не про секс, голод и сон, а про желание управлять реальностью. Каждый день. О, как велико в этот момент искушение спихнуть контакт с неопределенностью на другого...

Лучшее, что мы можем сделать — построить адекватную модель работы с неопределенностью, которая позволит создать алгоритм оценки или хотя бы разделить ответственность между этой моделью и оценщиком. Знать ограничения этой модели. А затем — научить пользоваться моделью тех, кто дает оценки вам.

С древности такими моделями были гадания: бросание костей, гадание по внутренностям животных, по тени и прочая мистическая радость. На гаданиях основаны мировые религии. Но это не совсем то, на чем я бы хотел основывать свои бизнес-процессы.

Вернемся к формуле. Она, в принципе, верная.

$$\text{Длительность работ} = \frac{\text{Объем работ}}{\text{Производительность}}$$

Пакость в том, что ни числитель, ни знаменатель этой дроби неизвестны. Хотя бы потому, что:

- ▶ Подпись заказчика (кровью) под техническим заданием не гарантирует, что документ описывает реальный объем работ. Она даже не гарантирует, что заказчик читал ТЗ. Реальный объем работ вы узнаете только после приемки проекта.
- ▶ Любая постановка задачи гарантированно содержит «дырки», которые можно трактовать двояко. Чем толще ТЗ, тем больше таких мест.
- ▶ Производительность людей (особенно программистов) может отличаться в РАЗЫ и зависит от столь большого числа факторов, что учесть влияние всех просто невозможно.

Все известные мне математические модели, пытающиеся ответить на вопрос «ДОКОЛЕ?», построены на серьезных упрощениях. Применять их на практике можно только для демонстрации несостоятельности этих моделей и тщетности бытия.

С определенной долей вероятности можно спрогнозировать дату завершения задачи или проекта, если:

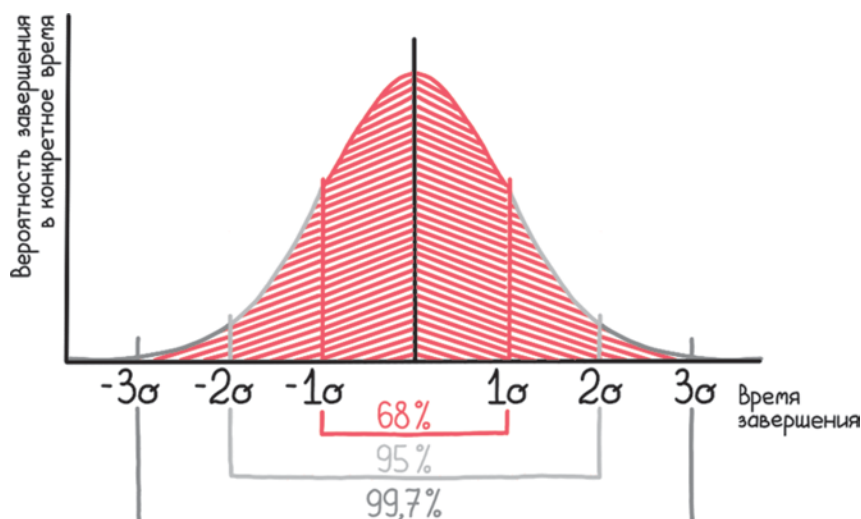
- ▶ у вас стабильная, сработавшаяся команда исполнителей;
- ▶ работы выполняются для одного и того же клиента, который дает стабильную обратную связь (читай — генерирует прогнозируемое количество «правочек») и демонстрирует стабильные требования к качеству;
- ▶ команда уже работала с подобными задачами;
- ▶ рабочий процесс, технологический стек и окружающая среда — неизменны;
- ▶ сам проект укладывается в головах команды;
- ▶ устраивает оценка в относительных, а не в абсолютных величинах (никаких тебе рублей и часов!);
- ▶ уже есть опыт работы в таких же условиях и данные, полученные в предыдущих итерациях; либо клиент готов работать итеративно и получать прогнозы, исходя из реальных замеров производительности предыдущих этапов.

Хуже всего дела обстоят у сейлзов, которые вынуждены делать вид, что все эти условия выполнены (хотя как раз наоборот). Но мы попробуем проникнуться этой галлюцинацией. В таком случае можно допустить, что вероятность завершения задачи к определенному сроку соответствует нормальному распределению Гаусса.

5.1.3 НОРМАЛЬНОЕ РАСПРЕДЕЛЕНИЕ НЕНОРМАЛЬНО!

Про колокол Гаусса что-то слышали даже гуманитарии. Это распределение часто встречается в природе. Понятие «нормальный рост», «нормальный вес» и «нормальный человек» — это как раз из области Гауссовской нормальности. Колокол Гаусса пытаются привязать везде, где можно и нельзя, если речь идет о чем-то подверженном влиянию огромного числа случайных помех.

Итак, если возникает вопрос: «Какая там вероятность уложиться в оценку?» — можно ответить: «Нормальная», и показать этот график.



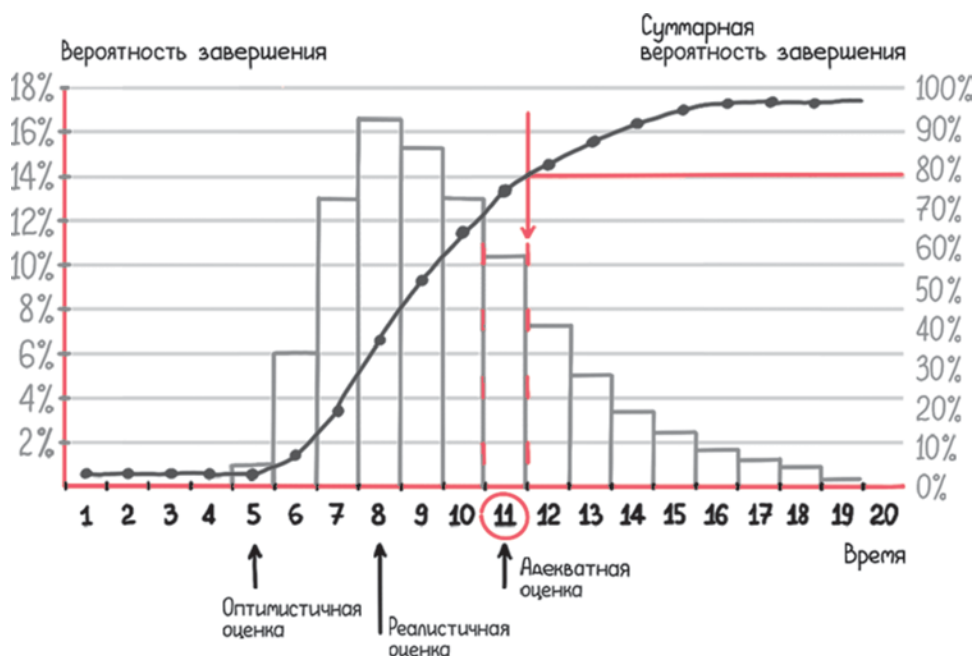
Посередине колокола — наиболее вероятная трудоемкость. Но поскольку мы работаем с величинами, подверженными случайностям, нам нужно делать допуск на величину рассеивания значений случайных величин (по-умному — среднеквадратичное отклонение или σ). Проблема в том, что, если мы берем диапазон в $\pm 1\sigma$, вероятность, что мы уложимся в наш интервал оценки, всего 68%. В оставшейся трети случаев нас обвинят в некомпетентности и поставят в угол.

На величину 2σ — вероятность будет 95%, но сам интервал получится до неприличия большой. Тут уже ваш заказчик скажет: «Фу-фу, вы не можете мне сказать точные оценки, а конкуренты сказали. Вы — упыри. До свидания». Пять процентов — это не так мало, если у вас много задач, и за срыв сроков по каждой из них мучительно бьют.

3σ — 99,7% вероятности попадания в нужный нам интервал. **Если клиент спрашивает: «К какому сроку вы гарантированно закончите проект?» — ответ: «НИКОГДА», будет математически правильным.** Даже в этих синтетических условиях математика против вас.

5.1.4 ТОРГИ И МАНИПУЛЯЦИИ

К сожалению, использовать Гауссово распределение для оценки задач некорректно. Если компания аккуратно собирает данные по прогнозам и реальным срокам, скорее всего, график распределения вероятностей будет выглядеть примерно так:



Это больше похоже на распределение Пуассона (похоже, да не оно). От нормального распределения Гаусса отличается тем, что сначала какое-то время имеет значение 0, затем начинает расти (в этой точке оценки дают оптимисты), быстро доходит до пика (это точка, в которой дают оценки реалисты) и имеет длинный хвост справа (там сидят все факапы и случившиеся риски).

Такое распределение вероятностей естественно, поскольку сделать задачу на час раньше реалистичного срока сложнее, чем затянуть на неделю. Работа занимает столько времени, сколько на нее отвели — закон Паркинсона. Поэтому раньше все равно не получится.

5.1.5 ОПТИМИСТЫ ПРАКТИЧЕСКИ ВСЕГДА НЕ УКЛАДЫВАЮТСЯ В СРОКИ

Для нас важно иметь возможность давать вилки на проекты с разбросом от реалистичной оценки до наиболее вероятной — например, 80%.

А клиенту может быть интересна именно оптимистичная оценка. Но назвать ее одну вы не можете, потому что не знаете, какие риски могут сработать с этим человеком. Поэтому сообщаете вилку.

Клиент может отреагировать на нее крайне резко (особенно если он бывший программист). Ему не понятно, почему появилась вилка — задача ведь известна! Сказать клиенту в лоб, что «вилка появилась потому, что мы еще не знаем, насколько ты адекватный» — конфликтный вариант. Проще объяснить, что вилка нужна для подстраховки.

Более веселые математические модели (PERT-анализ, логнормальное или устойчивое распределение) годятся только для изысканных понтов на конференциях. Но не особо помогают при общении с клиентом, который просит «не грузить этой фигней, а четко сказать, сколько стоит и когда будет готово».

Тем не менее, ознакомить своих сотрудников с азами этих моделей очень полезно. Ознакомить и попросить действовать адекватно.

Под адекватной моделью я понимаю такую, которая дает приемлемую для практической деятельности точность.

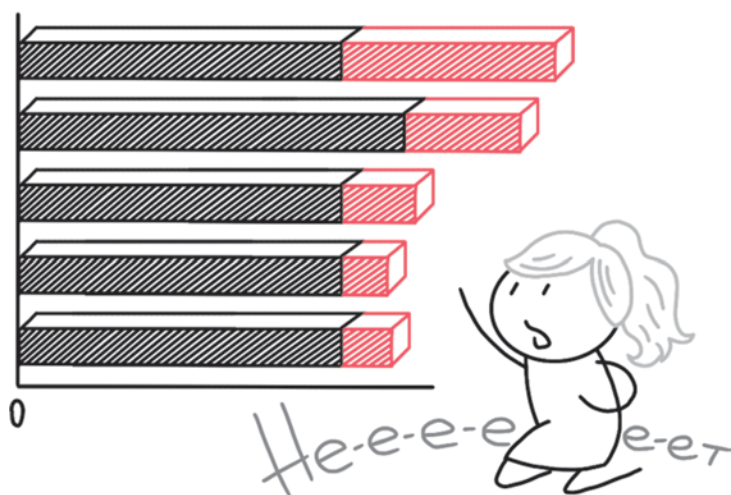
Я считаю, что нужно приучить людей жить с неопределенностью. Да, математика и бытие сильно против нас. Но нужно заставлять себя давать оценки с хорошей долей вероятности (скажем, 80% или 90% в зависимости от сферы), а оставшиеся риски брать на себя. В случае, если они возникли — извиниться (перед клиентом), поулыбаться и замять конфликт или сделать профакапленное за свой счет (не уходя в отрицательную прибыль).

5.1.6 ПРО БУФЕРЫ, ПОДСТРАХОВКУ И ЗАКОН МЕРФИ

Клиента нужно приучить к вилочным оценкам и к тому, что в них заложена подстраховка. Это не обман, а реальность. Мы можем быть уверенными только в одном: здесь работает закон Мерфи. Если он бьет часто, буферы должны быть больше. Проверить частоту Мерфи можно итеративно.

Если клиент категоричен и не хочет видеть вилки — просто уберите нижнюю границу. Но важно не только закладывать адекватные буферы, но и пристально контролировать их расход.

Если буфер съеден на треть, а работа еще не готова — это повод начать экспедирование и проталкивание. В сфере веб-разработки мы используем Burndown Chart, о котором я уже писал в главе про Scrum, чтобы визуализировать расход.



Экспедирование и проталкивание при отлаженных процессах должны занимать не более 5% всей работы. Если меньше — значит, у вас есть запас мощностей. Если больше — у вас проблема.

В сложных проектных цепочках наибольший контроль должен быть на выявлении бутылочного горлышка проекта и контроле его питающих буферов.

5.1.7 ПРИНЦИПЫ АДЕКВАТНОЙ ОЦЕНКИ

Есть всего ДВА способа делать оценки:

- ▶ опираясь на свою картину мира в прошлом (свой опыт и исторические данные);
- ▶ продлевая свою картину мира в будущее (экспертные оценки и интуиция).

На начальном этапе, когда человек еще не приучен давать точные оценки, ему надо рассказать, зачем они нужны. И не бить, если что-то пойдет не так: это вредно, поскольку будут перезакладывать, а потом сработает закон Паркинсона, затем закон Мерфи и тенденция факапить и затягивать будет прогрессировать. Но бить, если исполнитель не проэскалировал при возникновении проблемы (центрирующие парадигмы помните?).

Задача руководителя — приучить оценивать свою работу. Не просто давать задачу и смотреть, как человек с криками «Бляха-муха!» бежит ее делать. А убедиться, что нужный объем работ проанализирован.

5.1.8 ОПОРА НА ПРОШЛОЕ

В чистом виде прогноз на основе прошлого опыта возможен только в типовых, часто повторяющихся и жестко регламентированных профессиях и операциях. Такие пока что еще существуют, но будут вытеснены роботами и процессорами за двадцать долларов. Оценки конвейерной работы можно получить из статистики прошлых периодов.

Опора на прошлое имеет ряд недостатков, которые влияют на точность оценки.

Недостаток первый

Человек, делающий повторно одну и ту же (или очень похожую) операцию, начинает делать ее быстрее. С мастерством растет скорость. Поэтому нельзя сказать, что одна и та же задача у одного и того же человека займет одинаковое время.

Но есть проблема: так называемый «творческий подход», в том числе к регламентам и правилам. Он приводит к тому, что в какой-то момент делать однотипные вещи становится скучно, и мы пробуем выполнять их другим способом. Качество работ и время, на них затраченное, автоматически изменятся. Возрастет вероятность ошибок и провала по срокам.



Когда это контролируемый процесс и задача носит явно экспериментальный характер — окей. Мы ведь изначально закладываем время на эксперимент, а не получаем внезапный факт, потому что исполнитель начал экспериментировать и упоролся.

Мы уже видели похожий график роста компетенций в главе 1, но для менеджеров. У программистов, дизайнеров и кого-угодно — примерно та же история.

Недостаток второй

Мы склонны забывать детали, нюансы и мелочи тех операций, которые выполняем редко. А это значит, что наши оценки, основанные только на прошлом опыте выполнения задачи, практически всегда будут слишком оптимистичными.

Недостаток третий

В противовес забывчивости в мелочах мы хорошо помним, если ранее больно получили за превышение сроков на похожей задаче. И, если у нас не кружок садо-мазо, повторений не хотим. И начинаем перестраховываться.

Подстраховка будет основана не на здравом смысле, а лишь на страхе и нежелании испытать негативные последствия (даже если мы не уложились в свои же оценки, и нас за это не ругали).

Экспериментируем, забываем, боимся. Чтобы не было сюрпризов, грамотному руководителю важно оговаривать не только оценку сроков, но и способ достижения результата. Все, что может быть рационально автоматизировано и стандартизировано — должно быть автоматизировано и стандартизировано. А задачи, предполагающие эксперименты с новыми технологиями или способами работы, должны быть отдельно оговорены как экспериментальные (с отдельным резервом бюджета и сроков).

5.1.9 ПРОДЛЕНИЕ КАРТИНЫ МИРА В БУДУЩЕЕ

Итак, одного опыта для оценки недостаточно. Два других способа: экспертные оценки и интуиция.

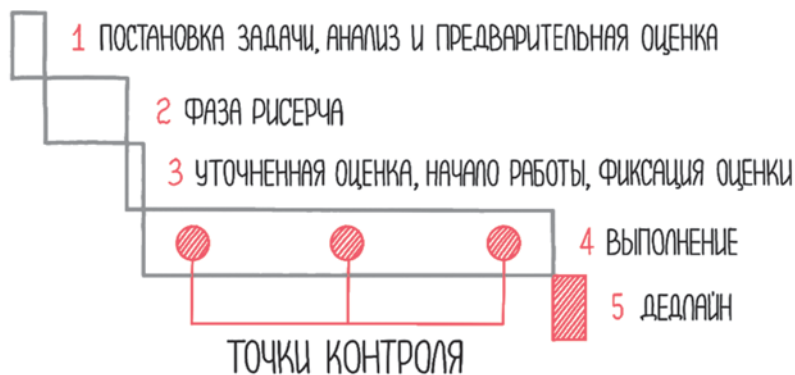
Лучшее, что мы должны проделать — представить, как именно будем реализовывать задачу. Чем детальнее представим, тем точнее будут прогнозы. Тут важно говорить правду самому себе: если в мысленном

эксперименте по ходу работы появились технологические нестыковки — они гарантированно выльются в дополнительное время.

Картина мира может быть неадекватной — тогда в процессе работы мы сталкиваемся с неожиданностями. Там, где мы не хотим разбираться в мелочах и вникать в детали, мы хотим просто закинуть много подстраховки. У программистов есть формула: **«Оцени, а затем умножь либо на π (3,14...), либо на e (2,71...)»**. Немаленькие множители, да и разброс большой. Но в итоге факапим даже с ними. Все дело в невнимательности к мелочам, лени разбираться или отсутствии адекватных ресурсов на изучение нюансов.

Да, оценка требует ресурсов, иногда — серьезных. Если задача новая, процесс оценки должен быть разбит на стадии — мы это уже разбирали в § 1.5, но давайте повторим:

1. Постановка задачи, анализ и предварительная оценка (оценивается срок получения сроков).
2. Фаза рисерча.
3. Уточненная оценка, начало работы, фиксация оценки (именно после рисерча и небольшого куска реальной работы).
4. Выполнение задания. Дальнейшее изменение оценки возможно только как форс-мажор или косяк и должно происходить по принципу «уперся — сообщи». В длительных задачах — согласованные контрольные точки.



Чем внимательнее к мелочам, тем точнее прогнозы. К сожалению, научить кого-то быть внимательным к мелочам трудно и долго. Но учить нужно, пусть даже через сопротивление. Если совсем никак — тогда расставаться.

5.1.10 СЕКРЕТНЫЕ МЕТОДИКИ МИРА IT, О КОТОРЫХ ВЫ УЖЕ ЗНАЕТЕ

В IT все сильно заморочены по поводу точности оценок, сроков и ресурсов. В других отраслях такой щепетильности не наблюдается. У IT-шников это возведено в культ. Впрочем, как и работа по фактически затраченному времени и фокусы с умножением на е и на п.

Многообразие методов говорит только о том, что IT-шники много думали, как решить проблему. Ниже — пара техник, которые помогают людям втянуться в оценки. Почти все это в чистом виде используется в Scrum, и неспроста.

5.1.10.1 Абсолютные и относительные оценки в маечку

Человеку сложно оценивать абсолютные величины. А вот с относительными проблем меньше.

Какой размер у Юпитера? Руками не показать, словами не описать: цифры слишком большие для понимания маленьким земным мозгом.

А теперь скажите, какой размер у Юпитера по сравнению с Землей, используя аналогии. Здесь уже проще: Земля размером с горошину, а Юпитер — с арбуз. Земля носит маечку размера S, а Юпитер — XXXL.

Мозг лучше воспринимает относительные величины — используйте это в оценке задач. Одну большую разбейте на десяток поменьше и дайте оценку каждой в отдельности. Сравнивайте подзадачи между собой: какая носит футболку XS, а какая L? Футболки даже не обязательно переводить в часы — вы уже поймете примерный объем работ.



В IT при правильно воспитанных заказчиках допустимо делать оценки задач не в абсолютных, а в относительных величинах. Новые задачи оцениваются относительно какой-то простой и всем известной. Кроме того, оценка в относительных величинах позволяет компенсировать естественные погрешности в самих оценках и в формулировках задач (так как на проекте количество корявых задач и кривых оценок будет примерно одинаковым от этапа к этапу).

К сожалению, автору неизвестно, применяются ли где-либо еще на практике относительные оценки. Всем подавай точные сроки и деньги :)

5.1.10.2 Planning Poker

Мы уже смотрели на этот метод в главе по Scrum и касались его в главе об аналитике. Также в литературе вы можете встретить метод Дельфи. Это про дельфийских оракулов и коллективное предсказание будущего. У нас есть эксперты, у них есть прогнозы. И нужно «помирить» их прогнозы и выдать что-то наиболее вероятное.

Если маечки и прочие сравнения вам не подходят, переходите на Planning Poker. Это способ в игровой манере вытянуть из исполнителей все мелочи процесса и возможные проблемы. И в итоге получить оценку задачи, близкую к реальности.



Соберите команду исполнителей вместе, озвучьте задачу, попросите положить на стол карту со временем, которое потребуется для выполнения. Карты выкладываются рубашкой вверх — это позволяет снизить влияние авторитетов. После все одновременно вскрываются.

Если после вскрытия карт по конкретной задаче цифры одинаковые или немного различаются — записывайте в план общее арифметическое. Если есть значительная разница — уточните у отличившихся, почему они оценили задачу так. Кто-то не учел длительную доставку деталей из другого города? Или забыл про инновационное ПО, которое установили на прошлой неделе? Всплывут новые подробности, и оценка станет точнее.

Не давите, чтобы в сроки, которые вы получили в покере, все было сделано: закон Мерфи никто не отменял. На этот случай менеджер должен закладывать подстраховку.

Если с помощью покера получить оценку невозможно (потому что для него нужен опыт) — добавьте стадию стартапа или рисерча. На ней исполнитель делает часть задачи, оценивает объем и сложность всей работы и может посчитать, сколько времени ему понадобится, чтобы закончить.

5.1.10.3 Декомпозиция

Используйте декомпозицию: разбивайте абстрактную большую задачу на маленькие и конкретные, которые легко оценить. Далее мы подробно рассмотрим техники, как это делать.

Декомпозиции нужно обучать. Если рассматривать на примере IT — примерно так:

1. Сначала берется готовый проект, который разбирается на составляющие — чтобы человек, который должен дать оценку, понял, в чем суть и что от него хотят.
2. Постепенно задача усложняется: для декомпозиции дается проект, в котором есть только прототип. Затем — стандартное техническое задание, потом — плохое и сложное ТЗ.
3. Человек должен научиться делить готовый цельный проект или ТЗ на этапы и составляющие. И каждому давать оценку. Когда научится — его оценки станут адекватными, приближенными к идеальному проценту вероятности и без необъяснимых буферов.

5.1.11 КАК НАУЧИТЬ ЛЮДЕЙ ДЕЛАТЬ ОЦЕНКИ

Чтобы человек мог ставить близкие к реальности оценки, у него должны быть прокачаны требовательность к самому себе, честность перед собой и понимание технологического процесса.

Если эти скиллы уже есть, далее воспитывайте его по схеме:

1. Покажите математические модели распределений и объясните, что слиться с оценок не получится.
2. Расскажите, что понимается под адекватной оценкой, как она будет использоваться и почему вам нужна именно она. Расскажите, что вам нужна оценка без перезаложённых в 100 500 раз подстраховок.
3. Покажите, как строится планирование потока работ всей компании на основе этой оценки или как ее будет использовать ваш клиент. Не держите людей за идиотов — если им рассказать правду и реальные потребности, большинство из них будут стараться делать наиболее точные прогнозы.
4. Приучите к принципам «уперся — сообщи» и «любая задача должна быть проанализирована и оценена».
5. Заставьте людей оценивать все свои задачи самостоятельно.
6. Поручите изучить статистику организации (сметы и реальные расходы по проектам).
7. Приучите к принципу «не знаешь, что делать — декомпозируй». Дайте несколько проектов на декомпозицию. Проверяйте и ищите дыры и нестыковки. Долго и нудно беседуйте по поводу найденных нестыковок.
8. Дайте оценивать реальные ТЗ. Проходитесь по итоговой оценке строчка за строчкой, спрашивайте, как пришел к ней. Проверяйте на полноту и непротиворечивость. Показывайте дыры в оценках, куда может улетать время.

В нашей отрасли можно научить человека делать хорошие оценки за пару месяцев (оптимистично). Для тренировки нужно использовать типовые объекты и давать постоянную обратную связь. При такой работе зона, где человек сможет делать прогнозы, будет постоянно расширяться.

5.1.12 СКАЗКА ПРО МЫШУ И ДЕДЛАЙН

Дед, генеральный директор ООО «Семья Голенкиных», запустил проект по созданию пареной репы. Его внутренности уже поуркивали от голода, поэтому Деду нужна была точная оценка: когда будет готово? Задачу принести пареную репу поставил Бабке, директору департамента по подаче продовольствия на стол. И вопросом про сроки тоже ее озадачил.

Бабка последние годы сама не готовила, только менеджерил Внучку, главного специалиста по тарелочкам. Поэтому задачу приготовить и определиться со сроками сдвинула на нее.

Внучке, чтобы начать, нужна была репка. Сама она не могла ее вытянуть, да и вообще задача не ее полета — для этого есть мелкие подрядчики, например собака Жучка. Оставалось определиться, сколько времени займет у Жучки вытягивание репки, и можно будет говорить о сроках.

Жучка тянуть репку не собиралась. Она не какая-нибудь гавкалка из породы «принеси-подай», а главная по двору — охраняет сарай, лает на калитку, подпирает стены. На нее работает кошка Мурка, которая отвечает за огородную территорию — ей Жучка и поручила тянуть репку.

Мурка сама репку тянуть не стала — для этой задачи выбрала проверенного субподрядчика Мышу. С нее и спросила, когда репка будет вытянута.

Мыша посмотрела красными глазами на плотно засевшую в сухой почве репку, на свои маленькие лапки, опять на репку. И поняла, что геморроя будет много. По смелым подсчетам, Мыша тянула бы репку



два часа, по трусливым — четыре часа. И еще неизвестно, какого размера репка под землей — если большая, сроки увеличиваются.

Мыша по опыту знала, что за провал дедлайна ей прилетит когтистой лапой Мурки по ушам. Поэтому подстраховалась и от души накинула два часа на риски — итого шесть часов на добычу репки.

Мурка, зная, что Мыша способна на факап, добавила к оценке три часа. Итого девять часов.

У Жучки репутация работника, который всегда укладывается в сроки. Ее главный секрет — ко времени на выполнение задачи прибавлять 30% от того, что заявил исполнитель. Итого получилось 12 часов на вытягивание репки.

Внучку эти данные расстроили, но спорить она не стала. Жучке виднее: она не одну репку принесла, Внучка ей доверяла.

По регламенту приготовление пареной репы занимает столько же времени, сколько и вытягивание. Внучка управилась бы за 2 часа, но испугалась обвинений в некомпетентности и поставила задачу на 12. Итого 24 часа.

На Бабке лежала главная задача: презентовать Деду пареную репу. Поэтому к 24 часам она добавила 5 на сервировку и 3 на возможные ЧП. Итого 32 часа.

От озвученных сроков Дед умер с голоду на месте. Проект пареной репы до сих пор не завершен.

5.2 ТРИ УРОВНЯ ДЕКОМПОЗИЦИИ

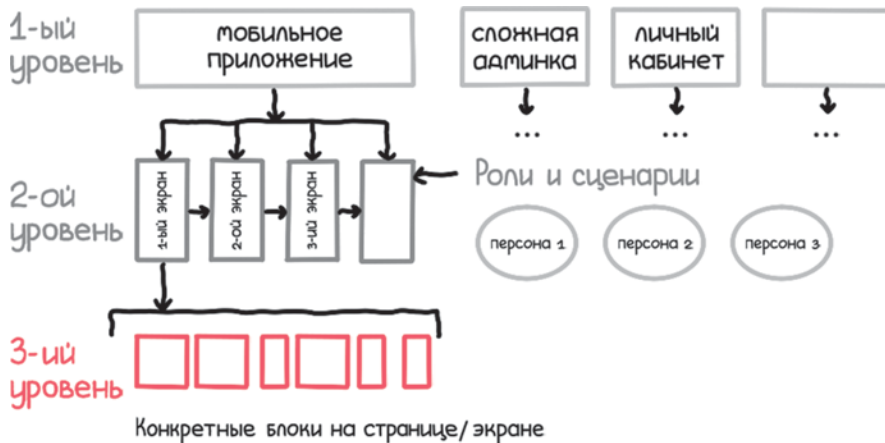
В любой непонятной ситуации — декомпозируй!

Представьте, что вам притащили с охоты мамонта: выше вас ростом, упитанного и на вид пока несъедобного. Что делать?! Декомпонзировать, конечно: лапы отдельно, шкуру отдельно. Но с чего начать? И когда хоть примерно будет готов ужин?

Если вам достался жирненький проект, вопросы примерно такие же: какой круг задач предстоит и как их предварительно оценить. Декомпозиция — крутой способ разложить все по полочкам и прикинуть объем работ, заложить резервы на трудные блоки и докопаться до неприятных задач со звездочкой.

Любую оценку на проекте стоит начинать именно с декомпозиции. Разбить проект сначала на крупные блоки, эти блоки разобрать на

страницы, а страницы — на конкретные элементы. Здесь мы упрощаем картину мира до безобразия, считая, что проект линейно зависит от количества и сложности страниц и блоков, расположенных на том или ином экране. Для простых проектов это приемлемо, для сложных кейсов с серьезными интеграциями и обилием хитрой математики такое деление не совсем корректно.



1 уровень. Крупные блоки или компоненты

Это может быть блок с е-коммерс, личный кабинет, мобильное приложение, супернавороченная админка. В общем, любые блоки работ, как-то связанные между собой, но которые можно делать изолированно друг от друга.

2 уровень. Страницы сайта или экраны мобильного приложения

Например, «мобильное приложение», как на схеме выше, разбиваем на экраны. Но как узнать, что вы учли все-все-все возможные экраны? Для проверки полноты берите в расчет сценарии использования — это даст понимание, какие задачи пользователи будут решать в приложении (или на сайте) и каким примерно образом они это будут делать.

Пример

Для е-коммерсе основной сценарий — продавать, а путь пользователя в нем выглядит так: каталог → список товаров → карточка товара и так далее.

Есть соблазн написать в смете только сценарий использования и его оценку (скажем, сценарий покупки товара или сценарий заказа такси) — ну, ведь понятно же, что там внутри. Нет, непонятно, и есть большой риск потерять множество шагов, поскольку такие сценарии большие, и их крайне сложно адекватно оценить целиком.

Когда сценарий раскладывается на экраны, шансов ошибиться становится меньше. Но помните, что каждый сценарий стоит проверять на связанность: достаточно ли вам вот этих экранов, чтобы сценарий сбылся?

Пример

У нас есть маркетплейс — магазин, куда другие производители могут загружать свои товары. Сценарии, лежащие на поверхности: загрузка своих товаров (цены и описания, разделы каталога и вот это вот все), покупка товара (шаги покупателя на пути к цели), обработка заказов (как будут распределяться деньги, как будет получать свою долю маркетплейс и так далее). Если все это не расписать подробно, можно запросто упустить кучу нюансов.

Будет еще лучше, если вы выделите ключевые роли на проекте (пользователь, администратор интернет-магазина и т. д.), у каждой из которых есть свой сценарий, а у каждого сценария — свой набор экранов. И тогда проверить полноту экранов еще проще: достаточно посмотреть, связан и выполняется ли сценарий конкретной роли.

3 уровень. Содержание экранов

В общем случае на экранах могут быть какие-то вкладки либо какие-то блоки — грубо говоря, вложенные экраны. Например, страница корзины/оформления заказа: здесь всегда есть блок товаров со своим сценарием (добавить-убавить-очистить), а еще блоки доставки, оплаты, авторизации, бонусной системы и так далее. Бывают ситуации, когда эти блоки также разбивают на экраны по шагам. Зависит от решения, принятого по итогам аналитики — бывает, что удобнее их все-таки «слить» воедино.

Задача менеджера, когда он добрался до такого экрана, — посмотреть, из чего тот состоит. Бывает, его легко разбить на блоки, быва-

ет — сложно. Яркий пример сложной разбивки — калькуляторы: по ним чаще всего неочевидно, что происходит, и как процесс расчета делить на шаги.

Когда вы добираетесь до третьего уровня, нужно быть супер-внимательными, потому что на странице могут появляться самые разные вещи. И важно понимать, откуда они там вообще берутся — от этого будут сильно зависеть ваши оценки.

5.3 ДЕКОМПОЗИЦИЯ ДЕКОМПОЗИЦИИ, ИЛИ ОТКУДА ЭТА ХРЕНЬ НА СТРАНИЦЕ?

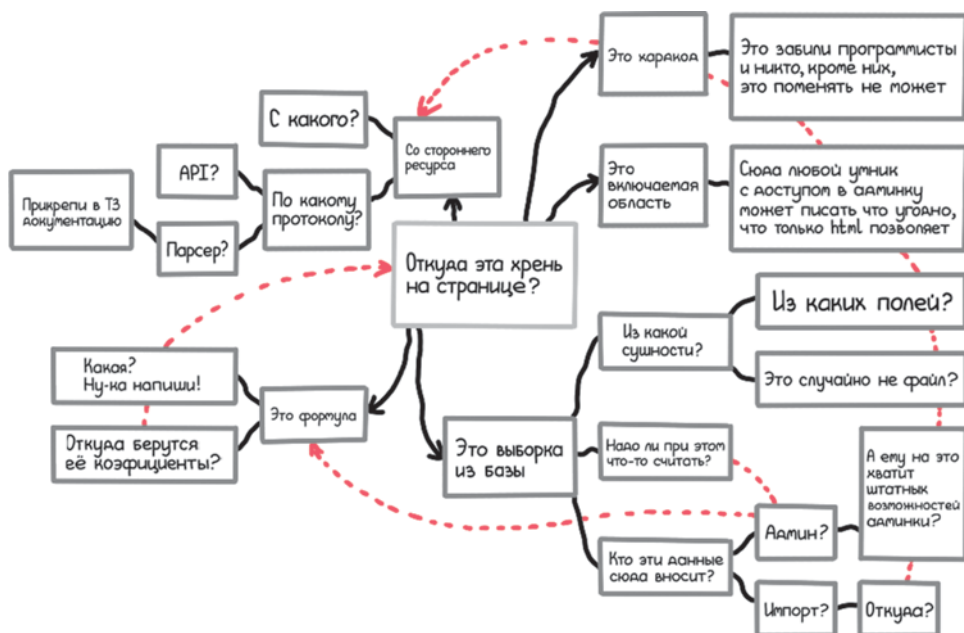
Это чертовски важная глава. Без ответа на вопрос: «Откуда это на странице?» — любая аналитика, любое ТЗ, любая декомпозиция — дырявая абстракция. Когда дело дойдет до дела, если не было внятных ответов на этот вопрос — все посыпется к чертям. Качество работы аналитика можно оценивать именно по количеству возгласов «откуда эта хрень?» в момент разработки проекта.

Итак, вы добрались до какого-то блока или страницы. Самое время задать себе вопрос: «Откуда это на странице?!» Но проджекты, аналитики и аккаунт-менеджеры (и даже заказчики) вот тут часто-часто ленятся — «подумаем об этом потом».

Например, аналитик сказал: «Это мы как-нибудь на коде решим», а потом на планинге сидят четыре умных человека, смотрят друг на друга и спрашивают: «Кто это придумал, что это за маразм?!» Такая ситуация — явный признак, что где-то раньше недоработали. Бывает, конечно, что принятие какого-то решения действительно откладывается, но это должно быть осознанно и где-то зафиксировано.

Чем меньше вы понимаете в момент «Откуда это на странице!?!», тем больше у вас должен быть зазор в смете. И когда к вам приходит клиент и говорит: «А почему тут такой большой зазор?!», у вас должен быть готовый ответ: потому что вы не понимаете, как работает то, то и это (лучше — фиксированный перечень конкретных вопросов), и что эти вопросы вы будете выяснять вместе с ним позже.

Итак, какими могут быть варианты, откуда берутся данные на странице?



5.3.1 ХАРДКОД

Самый простой в реализации вариант ответа на наш вопрос — хардкод. Это значит, что программисты сели, прямо в коде зафигачили какую-то штуку, и теперь поменять ее могут только они: ни администратор сайта, ни пользователи на нее никак повлиять не могут.

Обычно такие вещи бывают оправданы, например, скоростью, когда нужно что-то быстро сделать на проекте. Другая причина — банальная логика: нельзя ведь абсолютно все элементы сделать редактируемыми. Поэтому программисты, как правило, по умолчанию делают нередактируемым все то, о чем им не сказали прямо: «Вот этот блок должен редактироваться!» Здесь всегда стоит помнить о балансе между стоимостью и сроками разработки.

Самые частые блоки, которые делают хардкодом — логотипы компаний, иногда ссылки на соцсети, время от времени такое делают с меню (все реже), телефонами (плохо!), декоративными элементами на верстке. Все это — более-менее разумные моменты. Неразумно, это когда в код зашиваются, например, страницы целиком или SEO-тексты блоками.

5.3.2 ВКЛЮЧАЕМАЯ ОБЛАСТЬ

Включаемая область — фишка некоторых CMS-систем, когда на конкретной странице можно написать любой html-текст или код, и он будет на ней отображаться. Контент при этом неструктурированный: как администратор придумал, так он и будет выводиться. Для текстовой страницы вставка неструктурированного текста — вполне себе, но в других случаях подобные вещи могут негативно влиять на верстку и ломать дизайн.

У включаемых областей есть специфика: во-первых, их можно случайно удалить. Во-вторых, если в них указываются даты мероприятий или цены на товары, это чревато путаницей, поскольку у этих областей нет связанности: если поменять дату или цену в одном месте, в другом она останется прежней. Клиенты зачастую сразу говорят, что такого им не нужно — а значит, придется продумывать, как менять цены, даты и прочие изменяемые поля автоматически и повсеместно.

5.3.3 ИЗ АДМИНКИ (ИЗ БАЗЫ ДАННЫХ)

Итак, мы знаем, что какие-то данные выбираются из базы данных. Тогда нам нужно понимать, из какой сущности и из какого поля. Примеры сущностей в интернет-магазине: «товар», «раздел», «пользователь», «заказ» — то есть то, что состоит из каких-то полей. Поля, например, «цена».

Но достаточно ли нам будет понимать, из какой сущности и из какого поля выводятся данные? Не совсем. Когда вы выбираете какую-то информацию из базы, она может выводиться не в том виде, в котором она там хранится, а в несколько модифицированном.

5.3.4 ФОРМУЛА

Когда информация хранится в базе данных, но ее нужно как-то определенным образом модифицировать, появляется понятие «формула». **Одна из самых опасных вещей, которую менеджеры часто пропускают.**

Когда вам аналитик говорит «ну там это как-то считается» — навострите ушки, впереди точно будет затык. Математики не понимают

программистов и считают, что их формулу достаточно переписать и следом «просто» запрограммировать — делов-то. Но когда клиента начинаешь спрашивать о формулах, часто слышишь что-то вроде «ой, она у нас там в Excel», или «механика пока непонятна», или вообще «ну скопируйте вон с того сайта».

Видите формулу — копайте глубже. У нее внутри есть коэффициенты — а откуда берутся эти коэффициенты? Добро пожаловать в новый виток расследования «Откуда эта хрень на странице!?!».

5.3.5 ФАЙЛ

В зависимости от используемой технологии бывает, что часть данных хранится в файлах. Может показаться, что это какая-то сущность или поле сущности, но это все-таки **ФАЙЛ**.

Очень часто файлы в самой базе данных не хранятся, чтобы не «раздувать» ее. Из-за этого работа с ними организована иначе. В случае банального каталога товаров файлом может быть фотография, описание, спецификации в pdf и все подобное. Такие файлы находятся не совсем в базе данных, но при оценке важно четко понимать, откуда эти файлы берутся, как они попадают в базу, какие к ним нужны права доступа и как их выгружать.

С файлами бывает история, что их нужно хранить, начиная с отдельных серверов или **облаков S3 (объектных хранилищ)**, заканчивая специальным протоколом, но это уже нюансы масштабирования. На старте проекта, окупаемость которого непонятна, городить огород нет смысла. Исключение — тяжелый видеоконтент: его лучше сразу хранить на видеохостинге.

5.3.6 КАК ДАННЫЕ ПОПАДАЮТ В БАЗУ ДАННЫХ?

Обычно администратор или контент-менеджер садится и вносит данные вручную. Здесь должен возникать вопрос, а хватит ли ему стандартных компонентов админки для этого. Руководитель проекта должен быть очень хорошо знаком с возможностями административной панели. А еще с ними должны быть знакомы аналитик и тестировщик (про кодера, понятно, молчу).

Пример

У вас в дизайне есть слайдер, где расставлены точки, по клику на которые открывается всплывашка, в которой есть фотография, описание и ссылка. Вопрос: как расставлять эти точки? Как вариант — координаты X и Y, но вряд ли контентщик будет счастлив от такого функционала. А значит, придется что-то придумывать. В смету нужно будет это заложить.

Второй момент, который проджектсы часто упускают, — права доступа и хватит ли их. Это тоже нужно иметь в виду и сразу перечислять потенциальные роли. В любой момент времени вы должны понимать, кто вносит информацию в базу данных, и хватает ли штатных средств админ-панели, чтобы этими данными управлять (если не хватает — придется что-то дописывать).

5.3.7 ИНТЕГРАЦИЯ СО СТОРОННИМИ РЕСУРСАМИ

Один из источников данных в базе — пользовательский контент. И здесь важно понимать, как он попадает в базу. На этом этапе часто теряется один из крупных сценариев: например, как пользователь вносит отзывы. У отзывов часто бывает рейтинг — штука с виду простая, но внутри она может быть довольно сложно организована. У чего больше рейтинг? Там, где поставили одну оценку в десять баллов, или где тысячу оценок, но разных? Среднеарифметическое тут работает плохо. Но хитрые алгоритмы есть — привет, еще один резерв в смете.

Если данные берутся все-таки из внешнего источника, то без интеграции никак. Вариантов интеграции может быть несколько.

5.3.7.1 Парсинг

Парсинг — не самая приятная тема. Когда вам говорят «сходите за определенными данными на сторонний ресурс и аккуратненько все своруйте», заканчивается это чаще всего плохо. А когда еще этих ресурсов 10–15–20 — тогда вообще смерть. Потому что данные или их структура, как правило, могут меняться, а вы об этом узнаете в самую последнюю очередь, когда все уже отвалится.

Другая проблема — админы сайтов, с которых парсятся данные, не слишком счастливы, что эти данные кто-то «ворует», и будут всячески защищаться. А это приводит к «падению» парсинга и попаданию в черные списки. Вы попытаетесь с этим бороться добавлением каких-нибудь платных проху — короче, целый квест. Есть особые сервисы для организации парсинга, например: Mozenda, Automation Anywhere, Beautiful Soup, WebHarvy или Content Grabber.

5.3.7.2 API

Здесь имеется в виду, что есть какой-то интеграционный протокол, либо файловый протокол, либо XML, либо шина данных (сервер очередей, вроде RabbitMQ, ZeroMQ или Apache Kafka). С чем именно интегрироваться и по какому протоколу, на этапе предварительной оценки не столь важно — важнее, есть ли для этого документация. А у нее обычно бывает два состояния:

- ▶ она есть — это когда можно в любой момент зайти и посмотреть на нее глазами, а внутри реально описаны методы;
- ▶ она «почти есть» — как правило, означает то, что внутри ерунда, а вы будете подопытными обезьянками, на которых все будут эту сырую штуку тестировать.

К сожалению, заказчики на этапе продажи или обсуждения требований к проекту зачастую не готовы делиться информацией, откуда берутся те или иные данные. Иногда не хотят, иногда — просто не владеют такой информацией, а думать не хочется, потому что внутренний «процессор» от этого сильно нагревается. Если нужны какие-то коэффициенты для формул, возможно, заказчику придется походить по отделам и поспрашивать своих технических специалистов, потратить на общение с ними силы и время — а это, сами понимаете, сложно. Но если клиент требует с вас точную оценку, без этих данных не обойтись.

Хуже всего бывает, когда говорят «ну вы, программисты, между собой договоритесь и разберитесь сами как-нибудь». Если протокол не формализован и взаимной ответственности нет, критический путь проекта будет пролегать через интеграцию, и на нем он завалится. Или по крайней мере, придется потратить кучу времени на согласование с программистом протокола заказчика и его отладку.

Соответственно, если в проекте планируется интеграция с внешним сервисом, на нее нужно закладывать большие резервы. Лайфхак, если нужно интегрироваться, а протокола пока нет — используйте **МОСК-объекты**. Это специальные заглушки для интеграционного протокола, которые можно сделать быстро. А как только будет реальный протокол, заменить их (но обязательно с перепроверкой).

5.3.8 КАК ВСЕ ЭТО «ПОДРУЖИТЬ»

Начинаем с крупных компонентов: первый, второй, третий — можно расписать подробно. Следом важно примерно понять, какие есть пользователи (роли) и какие у них сценарии. Сами сценарии в смету лучше не прописывать. Дальше — идем по страницам. После — работаем с отдельными блоками, используя уже известную схему «Откуда эта хрень на странице?!».

Как только вы слышите слово «калькулятор» или «считается», направляйтесь! Когда есть интеграция со сторонним сервисом — тоже. В остальном — ничего страшного, и все довольно прозрачно.

Если в проекте есть какая-то дремучая математика, и вы живете в мире, полном злых неожиданностей, то декомпозиция по экранам будет давать сбой. Обычно она довольно хорошо показывает, что и как происходит на типовых проектах.

5.4 ПОДГОТОВКА ВИЛОЧНЫХ И ТОЧНЫХ СМЕТ

На этапе продажи клиенту хочется понимать, что его ждет на проекте: к какому бюджету готовиться и какими хоть примерно будут объемы работ и сроки. Для этого есть этап предварительной оценки проекта, с помощью которого можно максимально быстро и точно посчитать вилочную оценку проекта, обсудить ее с заказчиком и задать уточняющие вопросы. И уже потом озвучивать уточненную смету.

Этапы оценки:

- ▶ предварительная вилочная оценка по итогам брифования — озвучивается по опыту аккаунт-менеджера;
- ▶ подготовка предварительной сметы на проект;

- ▶ уточненная смета по итогу обсуждения ее с клиентом;
- ▶ уточнение сметы после подписания договора по мере выполнения аналитики и дизайна.

Дисклеймер. Цифры в часах, которые вы увидите ниже, характерны конкретно для нашего технологического процесса и уже включают в себя разного рода внутренние риски, все нужные операции и итерации. У вас же в компании все может быть по-другому. Подсматривать на наши оценки можно, но как эталон их использовать не стоит. Собирайте свою статистику, изучайте, ищите закономерности. Заведите свою таблицу с перечнем работ и их реальными оценками на основе предыдущего опыта.

5.4.1 ПРЕДВАРИТЕЛЬНАЯ ИЛИ ВИЛОЧНАЯ СМЕТА

Когда заказчик только-только попадает в цепкие лапы аккаунт-менеджера, ему приходится подробно описывать задачу. Мы очень много расспрашиваем заказчика: о самой задаче, о бизнесе и планах его развития, о целях проекта, о целевой аудитории и так далее. В 90% случаев аккаунт-менеджер может назвать приблизительный диапазон стоимости проекта: от и до. Обычно для этого хватает двенадцатиминутного разговора.

Если вилочная оценка не вызывает у заказчика паники, мы переходим к следующим этапам. Сначала обсуждаем структуру — это опционально: показываем ее в таблице Google, вносим правки, задаем вопросы.

Когда все утверждено, и вопросов стало немного меньше, переходим к составлению примерной сметы проекта, она же — основа для будущего коммерческого предложения.

5.4.2 СТРУКТУРА СМЕТЫ

На практике мы используем сметы, в которых есть вилочная трудоемкость по каждой операции конвейера разработки (аналитике, дизайну, верстке, программированию). И по каждому экрану или функции. Кроме того, к смете привязана диаграмма Ганта — будущий график выполнения работ по проекту (о ней мы поговорим отдельно).

7. Программирование, min, руб — нижняя граница вилки по разработке.
8. Программирование, max, руб — верхняя граница вилки по разработке.
9. Дизайн, min — оценка в часах.
10. Дизайн, max — оценка в часах.
11. Верстка, min — оценка в часах.
12. Верстка, max — оценка в часах.
13. Программирование, min — оценка в часах.
14. Программирование, max — оценка в часах.

Менеджер вручную заполняет разделы 9–14. Срок рассчитывается по формуле с учетом работы команды из двух-трех разработчиков, с запасом на согласования. Разделы 3–8 считаются автоматически по формуле (текущий рейтинг студии умножается на количество человеко-часов).

Мы используем именно такой набор колонок, поскольку он дает наиболее прозрачное представление процесса. Вы можете не выделять, например, этап верстки — заказчики, как правило, не понимают ценности этого этапа или воспринимают верстку как уже собранный проект и искренне удивляются, почему при верстке не работает какая-то кнопка.

Стоимость каждого конкретного этапа (кроме прототипирования) оценивается по минимуму и максимуму. Это связано с тем, что на этапе пресейла студия еще не знает, каким способом будет решаться та или иная задача — часто это можно сделать только после прототипирования.

Все оценки в смете даются в человеко-часах. В часы нужно закладывать не только время, необходимое на разработку элемента, но и прогноз по времени, которое потребуется для внесения правок от клиента и доведение элемента сметы до релизного состояния.

Клиенту стоит показывать только стоимость, БЕЗ затрат по времени. В гугл-таблицах это довольно просто сделать, скрыв часть столбцов и отдав клиенту документ с ограниченными правами доступа «Только просмотр» или «Комментирование».

5.4.3 АНАЛИТИКА

Аналитика всегда выносится в отдельную секцию документа. Содержит три блока:

- ▶ агрегация требований;
- ▶ разработка прототипа;
- ▶ разработка технического задания.

Все три блока — опциональны.

Если проект крайне запутанный, и составить его карту невозможно — включите в смету этап агрегации требований. В наших проектах мы всегда включаем этот этап. Его наличие в смете при оценке прототипа и технического задания дает вам возможность указать вилочную оценку «от» и «до» вместо фиксированной оценки. В более-менее типовых проектах стоимость прототипа и ТЗ должна быть четкой.

Часто на агрегацию требований мы закладываем неделю-другую аналитика. Этого времени обычно хватает, чтобы лучше понять не только проект, но и то, что представляет из себя сам клиент. Заказчики легко соглашаются на такую оценку: им нестрашно потратить небольшую сумму за хорошую агрегацию. Для студии риски также минимальны. Плюс результат этапа вполне отчуждаемый — это физический документ с анализом целевых персон и конкурентов, а также со структурой будущего проекта. С ним можно жить и работать дальше (даже если дальше этапа аналитики вы с заказчиком не продвинетесь). Если же UX-аналитику перед созданием агрегации придется прочитать большое техническое задание от заказчика, время на этап может увеличиться — но, по опыту, не более 60 часов.

В графе «Прототип» стоит иерархично перечислить страницы/экраны/всплывающие окна/вкладки/виды/шаги/сложные компоненты, которые необходимо показать на прототипе. При этом не забыть про вложенность иерархии. После этого оценить в часах: сколько потребуются на изготовление прототипа.

Большая часть типовых экранов (типовые формы, всплывашки и прочее) занимают два-четыре часа. Сложные экраны занимают порядка восьми часов (при этом всплывающие окна, **табы**, сложные блоки или варианты переключения вида нужно считать отдельными страницами). Первый экран может занять 16–24 часа (так как включает этап брейншторма и поиска концепта).

По опыту — прототип и техническое задание занимают 12% от стоимости и сроков всего проекта.

Разработка технического задания в более-менее типовых проектах (вроде простых интернет-магазинов) занимает около недели. В сложных проектах — от полутора недель. В о-о-очень сложных — ТЗ должно коррелировать с количеством страниц в прототипе, а также с количеством связанных подсистем. Если в проекте много интеграций, обычно стоимость технического задания на интеграционный протокол выносится отдельно, потому что это отдельная большая работа. Вполне возможно, что в проекте будет два технических задания: одно — для описания прототипов, второе — для интеграционного протокола.

5.4.4 ДИЗАЙН

Обычно оценка дизайна коррелирует с количеством страниц. Дизайн простых страниц занимает 2–4 часа, типовых 4–8, сложных 8–16.

Как и на прототипе, отдельные виды/табы и прочие фишки нужно считать отдельно. Главные страницы (или первые страницы на проекте) занимают вместе со всеми правками-брейнштормами 60 и более часов. Страницы с **параллаксами**, длинные страницы или какие-то нетиповые истории с анимацией — нужно считать отдельно. И саму анимацию тоже нужно считать отдельно.

Иконки, шрифты, иллюстрации, логотипы, фото-видео продакшн — это отдельные истории, и по умолчанию в стоимость разработки не входят. Если клиент акцентирует внимание на иконках и иллюстрациях — решите, как это включить в смету.

5.4.5 ВЕРСТКА. ФРОНТЕНД

Сейчас не будем рассматривать вариант, когда весь сайт можно собрать на конструкторе типа Tilda. Это могущественные инструменты, и очень хорошо, что они съели огромную часть рынка унылой, ручной

работы. В таких проектах достаточно хорошего дизайнера и копирайтера. Но когда нужно сделать то, чего конструктор не умеет — мы идем и верстаем все ручками.

Верстка современных систем занимает на 5–20% больше дизайна. И здесь анимацию тоже нужно считать отдельно. Она бывает довольно сложно реализуемой, особенно если там есть 3D-графика. Кроме того, нужно обязательно думать о мобильных устройствах и больших экранах.

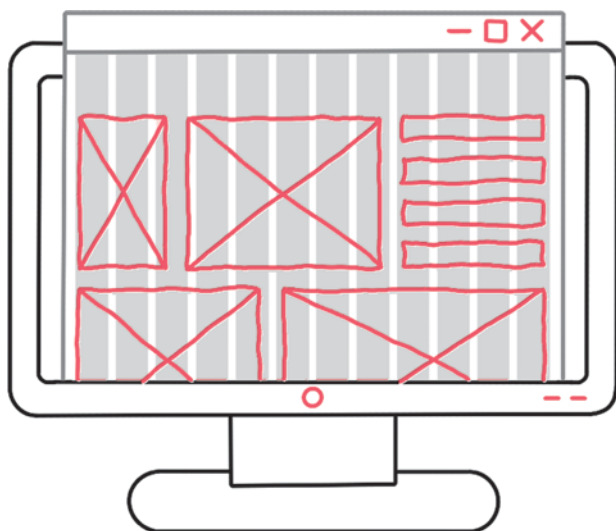
Несмотря на то, что есть много адептов Mobile First (когда первой разрабатывается версия для мобильных систем) и есть резон в таком подходе (мобильных пользователей уже более 50%), Mobile First-сайты часто выглядят как дрянь, сделанная под копирку. Опытные заказчики это знают. Определитесь на берегу, от какой версии будете плясать: мобильной или десктопной.

В зависимости от фреймворков, используемых на **фронте**, этап верстки может быть совмещен с разработкой фронтенда. Однако для сметы это не принципиальный момент.

5.4.5.1 Адаптивная версия

Здесь клиенту можно предложить два варианта:

1) Классическая адаптивная версия

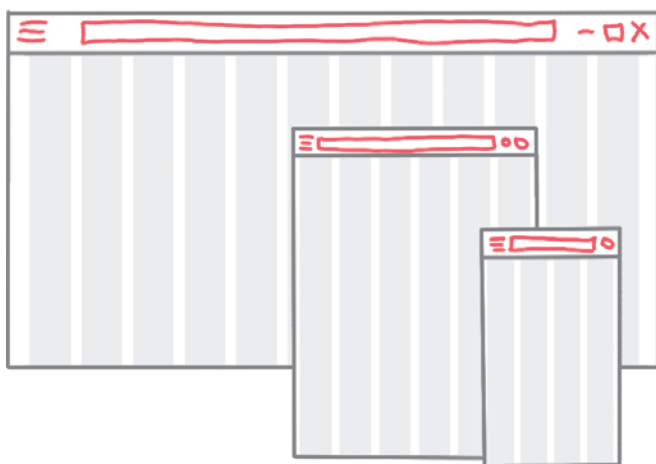


Вариант на основе 12-ти колоночной сетки с двумя точками излома. Здесь адаптивность достигается за счет:

- ▶ перестановки блоков контента на основе сетки (flexible, grid-based layout);
- ▶ изменения размеров изображений (flexible images);
- ▶ схлопывания меню в пиктограммы;
- ▶ упрощения сложных интерактивных элементов (анимации, галерей, параллаксов и других) до статичных изображений.

Такой вариант подходит для контентных проектов (корпоративных сайтов, порталов), интернет-магазинов. Адаптивная версия не предполагает согласования с заказчиком дизайн-макетов для разрешений мобильных платформ.

2) Персональная адаптивная версия (две точки излома)



Все страницы будут отрисованы и согласованы для заказчика в трех вариантах:

- ▶ компьютер;
- ▶ телефон (вертикально);
- ▶ планшет (вертикально).

Возможно применение персональных элементов управления на мобильных платформах. Точная стоимость может быть рассчитана после утверждения дизайна.

Такой вариант подходит для промо-сайтов и сайтов с большим количеством всплывающих окон, сложных интерактивных элементов управления.

Стоимость: минимальная — как в первом случае; максимальная — полная стоимость дизайна и верстки + 30% от разработки.

5.4.6 ПРОГРАММИРОВАНИЕ

Тут сложно давать абстрактные советы и нужно смотреть конкретный проект. Но я попробую. Для начала стоит определить тип проекта:

- ▶ Промо-сайт с интерактивным дизайном.
- ▶ Корпоративный сайт.
- ▶ Каталог продукции с возможностью заказа товара.
- ▶ Интернет-магазин с функциями доставки и онлайн-оплаты.
- ▶ Интернет-магазин с функциями доставки, онлайн-оплаты и интеграцией (с какой-нибудь 1С).
- ▶ Стартап, мобильное приложение и прочее.

От этого зависит объем сметы: может быть, вы будете делать проект на фреймворке, и придется прописать отдельной строкой создание, кастомизацию и настройку админ-панели. А может быть, вам будет достаточно лицензии Битрикс или бесплатного WordPress.

На этапе оценки разработки вам также пригодится уже проделанная декомпозиция. Сначала возьмите крупные блоки:

- ▶ информационная часть,
- ▶ каталог товаров;
- ▶ e-commerce;
- ▶ личные кабинеты;
- ▶ интеграция.

Следом для каждого блока пропишите иерархию ТИПОВ страниц. Для каждого ТИПА страниц — укажите иерархию табов, видов и сложных компонентов (если есть).

Каждый такой элемент должен быть в смете на отдельной строке. Базовую иерархию страниц можно взять либо из раздела «Аналитика» (если она уже готова на момент создания сметы), либо в ходе

брифования клиента (выясняете картину мира по проекту). Однако, в отличие от аналитики, где не по всем страницам мог потребоваться прототип, — в разработке нужно считать все страницы и блоки.

Общие модули (типа подписок на рассылку), использующиеся на нескольких страницах, должны содержаться в иерархии только один раз. При необходимости их можно вынести в секцию «общие блоки».

Когда иерархия готова, перепроверьте ее на полноту и связанность. Посмотрите проекты-аналоги. Поверьте, ничего ли вы не упустили, и уточните, нет ли дырок в проекте. Все неясности и вопросы пишите в комментариях. Помечайте непонятные моменты цветом. Если знаете, как можно устранить дыры и неточности в проекте, напишите в комментариях, как именно. Пометьте такие комментарии цветом, чтобы не потерять.

Следующий этап — проверьте смету на технологичность. Посмотрите, не использовал ли ваш клиент в исходных материалах слова «Каждый», «Любой», «Примерно», «На ваше усмотрение», «По аналогии, как тут». Все такие моменты нужно либо заранее выяснить, либо описать, как именно вы решили конкретно действовать по каждому пункту. Также пометьте их цветом.

Если речь идет о незнакомых вам технологиях, плагинах, методах обработки данных — отражайте такие пункты в смете, и тоже пометьте их цветом.

	A1	Функции	A	B	C	D	E	F	G	H
45	Корзина	Добавление товаров в корзину. Просмотр заказа. Корректировка количества. Блок дополнительных товаров // логика блока оценена отдельно для карточки товара			15 000	20 000	25 000	20 000	25 000	25 000
46	Оформление заказа — страница оформления заказа — страница выбора доставки и оплаты — страница успешного оформления заказа	Валидация входных данных. Отправка заказа администратору и сохранение в контрольной панели 1С-Битрикс.			15 000	20 000	25 000	20 000	25 000	25 000
47	Подключение к эквайрингу	Планируется доставка, оплата и фискализация через единого оператора. Уточнить сумму после получения API.								20 000
48	Интеграции									
49	Интеграция с CRM	Битрикс. Корпоративный портал. Передача заявок/заказов в CRM. Надо ли заявки сортировать, или распределить заявки?								10 000
50	Размываем версия									
51	Адаптивный язык	Для первого и второго этапов								95 000
52	Прочее									
53	Адаптивная версия, этап 2	Классическая 12-ти колоночная сетка. Две точки наложения. Без дополнительного сопоставления дизайнов-макетов для мобильных платформ.				3 250	3 750	16 250	26 250	
	Тестирование. — Установка на сервер и администрирование.	— Альфа-тестирование. — Бета-тестирование на сервере заказчика.						16 250	19 250	46 000

Пример сметы с пунктами для уточнения в Google Sheets

Если вы никогда раньше не сталкивались с разработкой, дать оценку этому этапу довольно сложно. Но если оценивать этот процесс грубо, приблизительно и ориентировочно, то прогнозы будут такими:

- ▶ простая страница без «космических» функций — минимум четыре часа;
- ▶ простой экран мобильного приложения без «космических» функций — четыре-восемь часов;
- ▶ чуть более сложная страница с выборкой, форматированием и выводом данных из базы данных — шесть-восемь часов;
- ▶ сложный функционал с множеством табов, переключателей — лучше оценивать каждый таб отдельно, а декомпозицию делать с запасом в четыре-восемь часов. Когда вы работаете над крупным проектом, вам, как исполнителю, выгоднее делать оценки итеративно.

5.4.7 ТЕСТИРОВАНИЕ

К разработке добавляется стоимость тестирования кода и менеджмента, к верстке — стоимость тестирования верстки. Мы закладываем по 20% суммарно. Заказчики часто не понимают, зачем нужно тестирование, и особенно — почему они должны за него платить. Мол, программисты должны сразу писать грамотный код без ошибок. Особенно когда находят какие-то баги сами. Тут важно правильно сформировать ожидания: разработка — это сложно. И оперативно устранять обнаруженные дефекты.

Некоторые команды предпочитают не показывать этот этап открыто и просто «размазывают» затраты на него по всей смете, чтобы избежать неприятных обсуждений, для чего нужно тестирование, и почему оно стоит денег. Но такая процедура есть в каждой зрелой команде. И этот процесс, как ни крути, требует определенных затрат ресурсов и времени.

Из опыта. Каждый проект проходит три уровня тестирования:

1. Программист сам проверяет свою работу по специальному чек-листу, составленному менеджером по качеству.
2. Менеджер по качеству (QA, тестировщик) проходится по этому же чек-листу.
3. Менеджер по качеству проверяет весь проект целиком.

Нет, отказаться от тестирования на проекте нельзя. Поэтому смело включайте его в смету и показывайте клиенту. Объясните, для чего нужен такой этап, и что конечная выгода для заказчика — получить в руки проект, работающий, как швейцарские часы (а не как китайская подделка).

5.4.8 АВТОТЕСТЫ. ЭТО ДОРОГО. НО ИНОГДА СТОИТ ТОГО!

В web-проектах автотесты используются редко. **Это ДОРОГО!** Как в разработке, так и в их актуализации. У команды может улететь от 20 до 50% времени на работу с тестами. То есть, вас это может замедлить и удорожить в два раза. Обычное дело.

Автотесты помогают избежать **регрессий кода**. Но панацеей не являются.

Оправданы они в нескольких случаях:

- ▶ Качество критично! Системы управления самолетами, например.
- ▶ Продуктовая разработка зрелого продукта. Прошел фазу стартапа, идут нормальные продажи и параллельно выпускаются новые версии.
- ▶ Заказная разработка зрелого проекта. Допустим, вы выпускаете апдейты на интернет-магазин с многотысячной аудиторией. На каждый **апдейт** с переносом на рабочий сервер нужно проводить смоук-тест по самым критичным сценариям (каталог → карточка → корзина → заказ, например). Критичных сценариев может быть не очень много, но их нужно проверять каждый раз. В этом случае можно автоматизировать проверку.
- ▶ API. Если вы реализовали API, которым пользуются другие проекты — важно не разломать совместимость API при его обновлении. Тут покрытие автотестами однозначно оправдано.
- ▶ Математика. То, что очень сложно проверить ручками, лучше покрыть тестами.

Однако на практике в большинстве заказных проектов все заканчивается ручным тестированием и смирением с тем, что на продакшене (реальном проекте на реальном рабочем сервере) может что-то пойти не так.

5.4.9 ПОЧЕМУ ТУТ ТРИ ЧАСА, А НЕ ДВА С ПОЛОВИНОЙ?

Бывает, что заказчик увидел оценку в часах в смете (или как-то эмпирически высчитал ее, такое бывает), и начинает интересоваться, а почему так.

Это не очень приятный разговор, поэтому лучше придерживаться подобной тактики: максимально декомпозировать задачу и как можно детальнее рассказать про ту функцию, по которой у клиента возникли вопросы. Причем, делать это так, будто вы уточняете у заказчика, правильно ли вы поняли задачу.

Например, для какого-нибудь Битрикса, заказчик может возмутиться, почему на простую форму обратной связи с тремя полями (имя, почта, текст) отводится четыре часа. Ведь все это есть в коробочной версии Битрикса. Спасибо маркетологам, которые говорят, что в коробочной версии делать ничего не надо.

На самом деле — надо. На стороне браузера сделать валидацию полей (проверку на правильность ввода). Если форма отправляется на сервер, нужно будет настроить валидацию и там. Также нужно предусмотреть вывод ошибок, если что-то пойдет не так. Собрать данные из формы и положить в нужное место базы данных. Отправить администратору письмо с содержимым этой формы.

Все это несложно: внутри систем управления или фреймворков чаще всего есть зрелые API, но на все эти простые процедуры за счет их количества потребуется время. Собственно, если декомпозировать задачу с формой до подобных шагов, сразу понятно, почему она занимает 4 часа, а не 15 минут.

5.4.10 КОГДА ПОЯВЛЯЕТСЯ ТОЧНАЯ СМЕТА

Предварительная смета проекта обсуждается с заказчиком по телефону/видеосвязи или при личной встрече. Аккаунт-менеджер показывает смету на экране и голосом рассказывает о структуре документа, кратко проходите по функциям проекта и задает по ним уточняющие вопросы. После демонстрации и обсуждения сметы, менеджер отправляет уже скорректированную смету заказчику на почту.

Перед показом финальной сметы заказчику просмотрите ее на два момента.

Во-первых, внимательно проверьте, не потерялись ли какие-то строки и везде ли посчитались формулы (бывает, что в гугл-

документах связи в ячейках ломаются). Это нужно элементарно для того, чтобы потом не оправдываться перед клиентом, что где-то что-то не посчиталось, и сумма в итоге вышла больше.

Во-вторых, проверьте, что вас не смущает итоговая сумма. Если она кажется слишком большой, есть шанс, где-то вы перезаложили риски или переоценили этапы. Если так, вернитесь и посмотрите там, где цифры кусаются — возможно, их можно уменьшить.

Почему всегда нужно обсуждать смету с заказчиком? Во-первых, обсуждение помогает согласовать точки зрения на проект и определить его функции. Во-вторых, разговаривать нужно для выяснения спорных моментов. Чем раньше мы это сделаем, тем лучше — работа после заключения договора пойдет быстрее. Обсуждение сметы сразу снимает ряд вопросов, которые пришлось бы разбирать уже после отправки коммерческого предложения. Плюс к тому, в предварительной смете можно быстро поменять данные и сразу же показать результат.

Хорошо составленная предварительная смета сокращает время, которое потребуется на разработку прототипа и технического задания.

После обсуждений в смету вносятся корректировки. А дальше эта история может быть очень длинной. Бывает всякое: иногда на принятие решения заказчиком уходят месяцы. Тогда менеджеру стоит регулярно напоминать о себе и спрашивать о статусе проекта.

Оплата за работу происходит поэтапно: сначала оплачиваются прототип и ТЗ (или только прототип), после чего составляется точная смета — без минимальной и максимальной границ, которая тоже презентуется заказчику. Эта смета всегда сравнивается с приблизительной, составленной на этапе пресейла: если оценка не попала в диапазон, обсуждаем, за счет чего сумма уменьшилась или увеличилась, смотрим, какие функции были добавлены, а какие моменты — не учтены изначально.

5.5 ЭКСПРЕСС-ВЕРСИЯ ДИАГРАММЫ ГАНТА

Диаграмма Ганта (пишут иногда с двумя «т», иногда с одной, оба написания легальны) — инструмент планирования и управления задачами, изобретенный инженером Генри Гантом в 1910 году. Коротко мы о ней упоминали в прошлой главе.

Диаграмма состоит из полос, ориентированных вдоль оси времени. Каждой полосе соответствует задача из списка слева. При этом задачи располагаются последовательно: или по приоритетности, или по логике выполнения.

	A	B	C	D	E	F	G	H	I	J	K
1		1 неделя	2 неделя	3 неделя	4 неделя	5 неделя	6 неделя	7 неделя	8 неделя		
2	задача 1										
3	задача 2										
4	задача 3										
5	задача 4										
6	задача 5										
7											
8											
9											
10											
11											
12											
13											

Пример диаграммы Ганта в Google Sheets

Основная функция диаграммы — упорядочить работу с задачами, а также наглядно сгруппировать их по датам и приоритетам. Такой подход позволяет избежать ситуаций, когда на второстепенную задачу тратится больше времени в ущерб основной, или одна из задач теряется в общем потоке.

В Scrum-е, по большому счету, диаграмма Ганта считается устаревшим инструментом (сто лет в обед). Дело в том, что попытки распланировать детально весь проект на много месяцев и потом следовать этому плану — как правило, иллюзорны. Да и принцип гибкости: «Готовность к изменениям важнее следования первоначальному плану» — никто в Agile не отменял.

Тем не менее, диаграмма Ганта полезна для формирования Roadmap-ов проекта, когда мы крупными мазками планируем наши релизы (тут у нас будет первая версия сайта, тут — мобильное приложение, тут — интеграция с ERP, а тут — личные кабинеты пользователей). Как правило, такой Roadmap используется вместе с бэклогом проекта и выглядит примерно так:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Версия	Июль 2017	Август 2017	Сентябрь 2017	Октябрь 2017	Ноябрь 2017	Декабрь 2017	Январь 2018	Февраль 2018	Март 2018	Апрель 2018	Май 2018	Июнь 2018	
2	Минимальная версия к выставке													
3	Корпоративный сайт													
4	Каталог продукции. Интернет-магазин. Оплата. Доставка.													
5	Интеграция с АХАРТА													
6	Лицевой кабинет для физических лиц													
7	Мобильное приложение													
8	Лицевой кабинет дилера													
9														
10														
11														
12														
13														
14														

Диаграмма Ганта — популярный прием при планировании и управлении проектами. Она имеет свои плюсы:

- ▶ наглядность — видны все задачи и сроки выполнения каждой из них;
- ▶ простота реализации — легко нарисовать от руки или сделать в Гугл-таблицах;
- ▶ доступность — диаграмма понятна даже восьмикласснику;
- ▶ структурированность — видна этапность и очередность работ;
- ▶ гибкость — легко адаптировать под свои потребности.

Но при всех прелестях диаграммы, у нее есть нюансы:

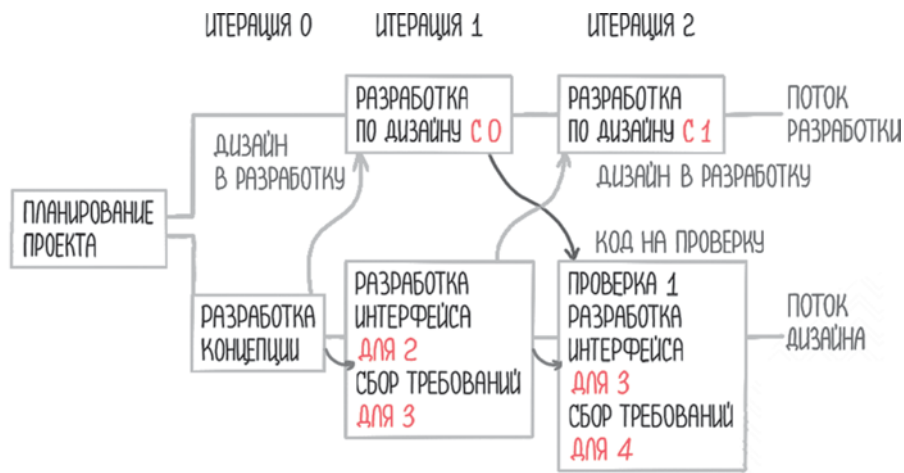
- ▶ Чаще всего не отражает всей картины проекта. Стандартная диаграмма Ганта показывает только список задач и время выполнения, не затрагивая количества ресурсов, людей, инструментов. Решить вопрос можно с использованием грамотных шаблонов и продвинутых сервисов.
- ▶ Если в проект вносятся изменения — скорее всего, диаграмму придется перестраивать.
- ▶ Плохо масштабируется для крупных проектов: если попытаться в одной диаграмме отобразить сразу все процессы или несколько разноплановых дочерних проектов, получится каша.

Диаграмма Ганта — идеальный вариант для краткосрочных и долгосрочных проектов с небольшим количеством исполнителей (до десяти человек). Она хорошо подойдет малому и среднему бизнесу для структурирования своих процессов, а также фрилансерам, у которых часто бывают проблемы с самоорганизацией и временными рамками :) Кроме того, ее удобно использовать для коммерческих предложений, смет и презентаций: заказчик наглядно видит стадии проекта и понимает действительные сроки реализации.

5.5.1 ЗАПУСК ПРОЕКТА С ПАРАЛЛЕЛЬНЫМИ ЭТАПАМИ

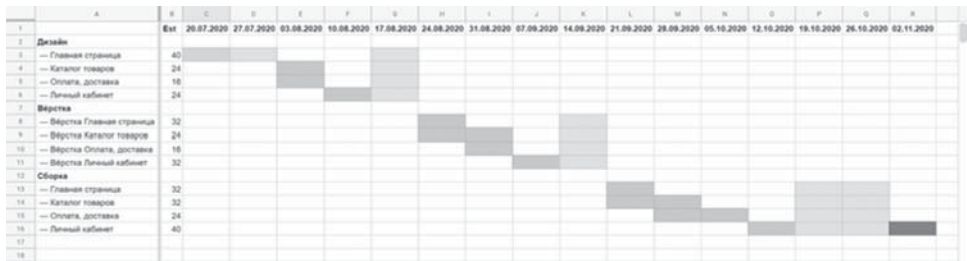
Если есть необходимость запустить проект более быстрыми темпами, готовьтесь к сложностям. Да, используя работу параллельными спринтами, можно значительно ускориться (в полтора-два раза), но при этом управлять таким проектом становится тоже значительно сложнее.

Каждый отдельный этап такой итеративной разработки будет для вас как новый микро-проект, и на нем придется делать все те же операции, что и на одном единственном проекте. Поэтому начинающим менеджерам о параллельных потоках на проектах лучше забыть: это задачка со звездочкой для тех, у кого уже есть опыт и навыки руководителя диджитал-проектов.



Мы подробно разберем этот подход в главе 6, где обсудим распараллеливание дизайна.

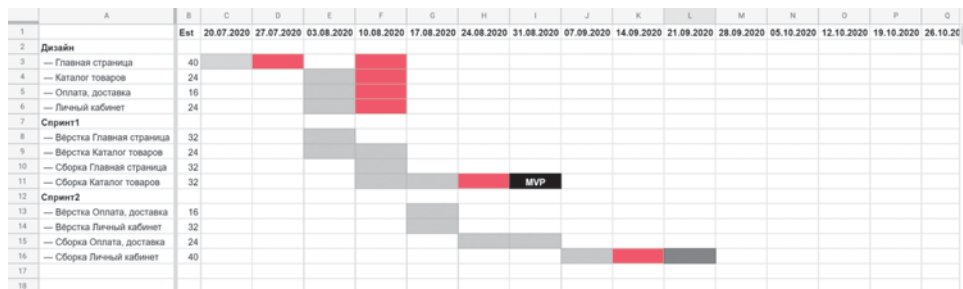
Распараллеливание процессов легко показать клиенту на диаграммах Ганта. Причем, для этого необязательно нужен какой-то специализированный софт — достаточно гугл-таблиц или Excel.



Проект без параллельных процессов развивается поэтапно: сначала дизайн, потом верстка, затем сборка. Старт в июле, релиз в ноябре. Красным — время на согласование с заказчиком.

Сроки, как на рисунке выше, мало кого устроят, но за счет распараллеливания процессов можно существенно ускориться. Например, на этапе дизайна подключить еще 1–2 дизайнеров для отрисовки внутренних страниц — это позволит сэкономить неделю. А если разбить верстку и программирование на два спринта, где в первом будет верстаться и собираться главная с каталогом, а во втором — доставка, оплата и личный кабинет, можно сэкономить еще пару недель.

Верстку можно начать пораньше, при этом согласование с заказчиком этого этапа можно и вовсе пропустить (клиент зачастую не понимает ценности верстки). А раз так, можно и сборку начать параллельно с версткой — в итоге к концу первого спринта программирования получится уже минимально-жизнеспособный продукт (MVP), который заказчик сможет тестировать. Пока он тестирует, можно приступить ко второму спринту. Во втором спринте также можно пропустить согласование верстки — и вот, проект готов на полтора месяца раньше, чем в случае с последовательными этапами. Если бы мы добавили в команду еще пару программистов, можно было бы справиться еще быстрее.



Проект закончен на полтора месяца быстрее за счет отказа от согласования верстки и распараллеливания работ, плюс — есть промежуточный релиз, который сразу готов к запуску.

Чего точно никогда не стоит делать — запускать этап программирования, пока еще нет результатов этапа дизайна. Среди программистов бывает такое: мол, давайте начнем программировать, а потом появится ТЗ или дизайн, и мы доделаем. Такой подход, к сожалению, чреват тем, что уже написанный код придется выбросить и писать все заново.

Три-четыре параллельных потока на один проект — трудно,
но можно, больше — уже ад.

5.6 СЕТЕВЫЕ ГРАФИКИ PERT

Диаграммы PERT — это та же диаграмма Ганта, но с учетом связей между этапами. В ней графически отображают сроки и ход задач на проекте с учетом зависимостей и последовательностей работ. PERT-диаграмма состоит из ключевых вех развития проекта, обозначаемых кругами или прямоугольниками (узлами), и задач, которые линиями соединяют эти узлы.



PERT-диаграммы часто используются на больших проектах, типа высадки человека на Луну. Здесь степень риска велика, а процент завершенности работ сложно оценивать.

Принципы сетевых графиков:

- ▶ номера задач регулируют последовательность событий;
- ▶ стрелки показывают направления и указывают на потоки событий: так стрелка от задачи A1 к задаче A3 на рисунке означает, что задача A1 должна быть обязательно завершена до начала задачи A3;

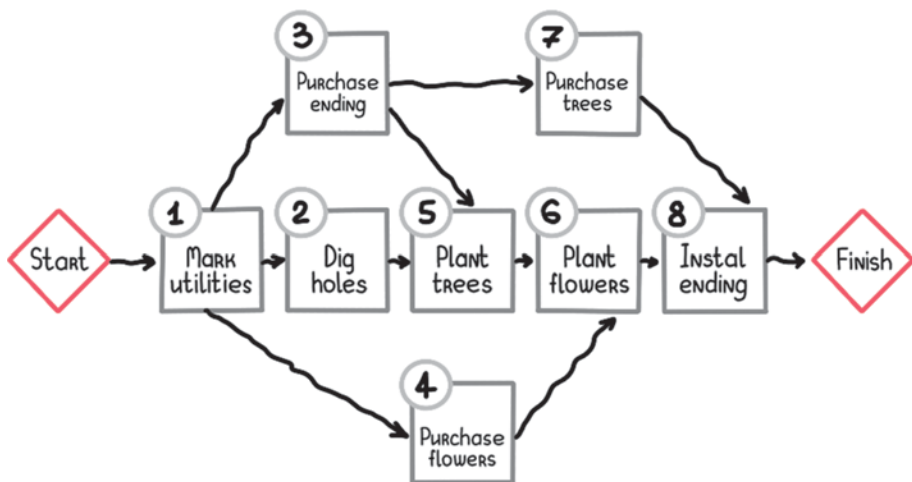
- ▶ можно создавать параллельные задачи, используя разные линии задач, которые не зависят друг от друга — как A1 и A2 на рисунке;
- ▶ график строится слева направо, и каждое событие с большим порядковым номером должно быть расположено правее предыдущего, а стрелки, изображающие работы, должны также располагаться слева направо.

С помощью PERT-диаграмм можно оценивать и анализировать время и ресурсы, необходимые для проекта, а также оптимизировать коммуникации внутри команды и распределять ответственность. За счет четкого деления на события и шаги минимизируются риски неопределенности — во время построения диаграмм менеджеру приходится делать анализ «что, если».

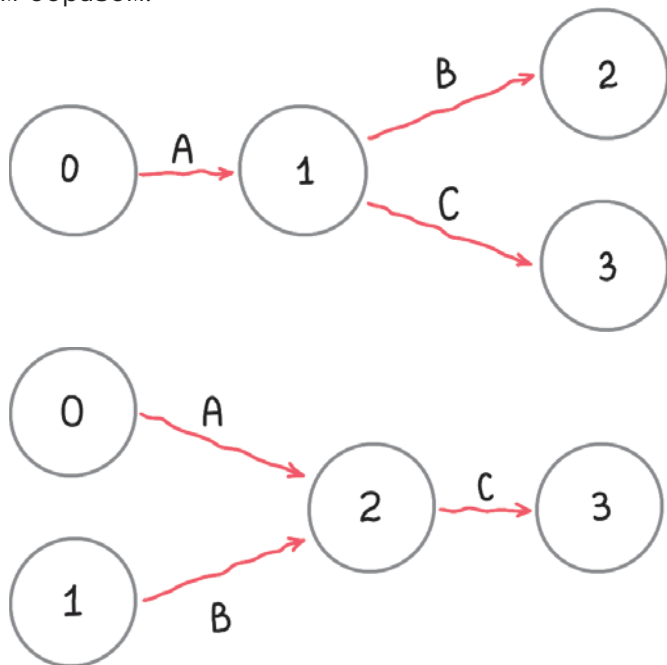
Процесс анализа с помощью PERT носит субъективный характер, поэтому не стоит с помощью этого метода пытаться точно оценить стоимость проекта и время на его выполнение. Эта техника обычно занижает предполагаемую продолжительность исполнения проектной задачи, а при увеличении числа параллельно выполняемых работ увеличивается погрешность. Другой минус подхода — трудоемкость и, как следствие, высокая стоимость.

5.6.1 КАК ПОСТРОИТЬ PERT-ДИАГРАММУ

- 1) Возьмите за основу декомпозицию проекта: составьте на ее базе список уровней и этапов проекта и определите время на завершение каждого события.
- 2) Определите исходное событие — то, которое не требует для своего начала результатов выполнения других работ.
- 3) Добавьте зависимости для создания связей между событиями.
- 4) Линиями покажите продолжительность каждого события: каждая линия должна вести к шагу, который связан с завершением события.
- 5) Следите за тем, чтобы не было тупиковых событий — тех, из которых не выходит ни одна работа, а также «хвостов» — промежуточных событий, которым не предшествует ни одной работы.



Не все события на сетевом графике могут быть последовательными — иногда для двух событий требуется результат одного предыдущего, и тогда события отображаются параллельно. Бывает, что для выполнения одного события требуется результат сразу двух предыдущих: например, для дизайна главной страницы нужны агрегация требований и брейншторм. Такие моменты на сетевых графиках отображаются следующим образом:



Чтобы рассчитать для каждой работы дату начала и окончания, нужно оценить временную длительность каждой из работ. До появления специализированного софта для построения сетевых графиков, менеджерам приходилось прибегать к самостоятельным математическим расчетам. Сначала долго и мучительно выяснять у подчиненных оптимистическую, пессимистическую и наиболее вероятностную оценки по времени. Мало того, что команда проекта может быть большой, так еще и оценки участников могут оказаться завышенными из-за перестраховки. Затем — вычислять среднее значение: техника PERT предполагает применение вероятностного подхода с использованием среднего значения β -распределения.

$$tE = (tO + 4tM + tP)/6,$$

где (tE) — средневзвешенное время операции;

(tM) — наиболее вероятное время операции;

(tO) — оптимистичное время операции;

(tP) — пессимистичное время операции.

Сейчас за сложные расчеты отвечают специализированные программы: они берут на себя создание самих графиков, расчеты ранних и поздних сроков начала или окончания задач и вычисление длины критического пути.

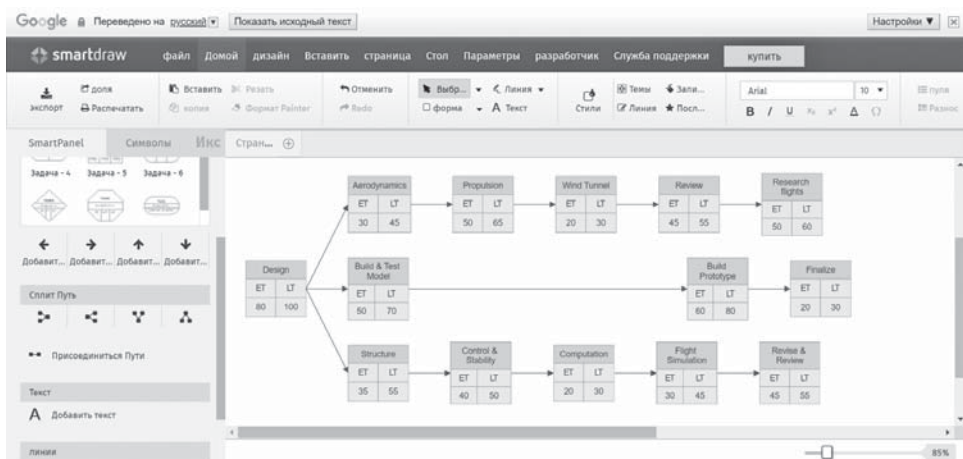
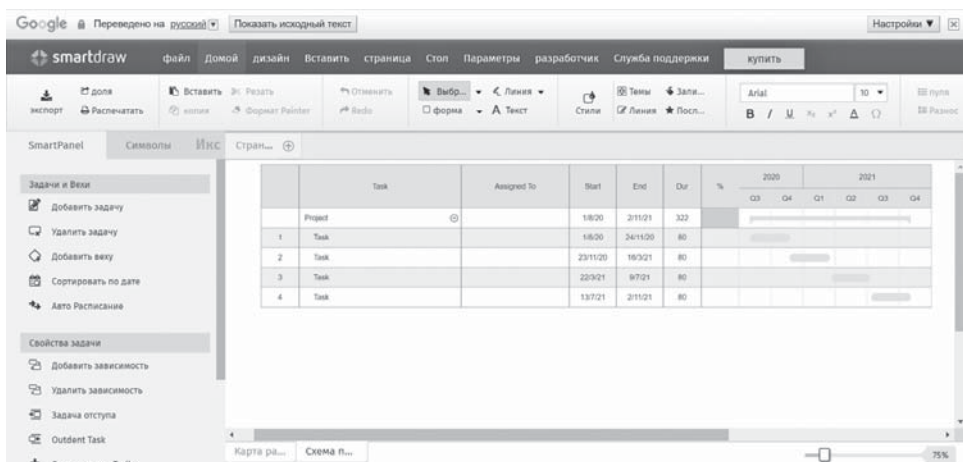
5.7 ОБЗОР ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ СЕТЕВЫХ ГРАФИКОВ И ДИАГРАММ ГАНТА

5.7.1 «УНИВЕРСАЛЬНЫЕ СОЛДАТЫ» СРЕДИ ПОДОБНОГО ПО

Хороши тем, что с построением диаграмм справится и новичок: внутри обычно много шаблонов и подсказок. Но подобные сервисы могут хромать там, где нужно выстраивать сложные взаимосвязи и зависимости (особенно это касается диаграмм Ганта).

SmartDraw

Сервис, который подойдет для рисования не только сетевых диаграмм, но и всевозможных схем и планов. В том числе — диаграмм Ганта.

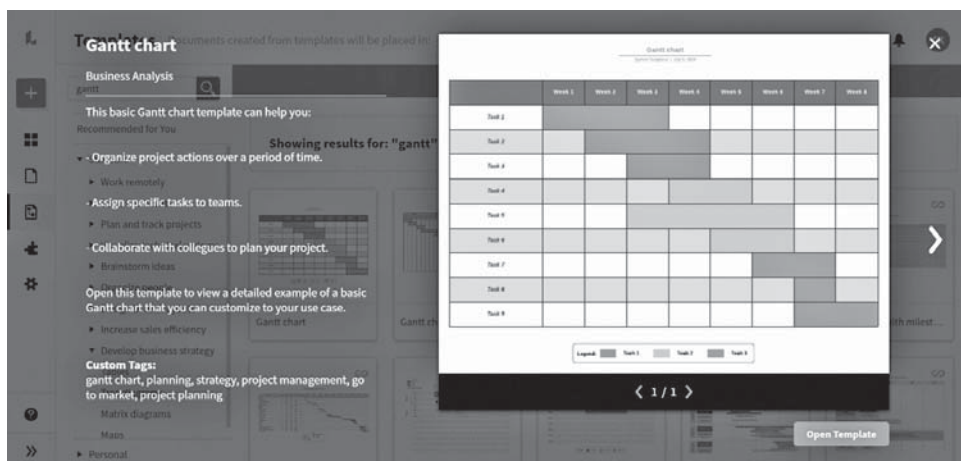


Внутри сервиса множество встроенных шаблонов и символов. В нем легко работать за счет встроенных опций выравнивания и управления. Имеются также исходные шаблоны типовых сетевых графиков, которые можно взять за основу и просто доработать под свои нужды. Удобно, что есть опция совместной работы над диаграммами.

Сервис интегрируется с G Suite, Confluence, Jira, Trello, Google Drive, Microsoft Office, а сами графики экспортируются в разных форматах для вставки в таблицы, текстовые документы и презентации. За счет интеграции с Гуглом доступно порядка ста языков.

Lucidchart

Сервис для создания диаграмм, с помощью которого можно рисовать блок-схемы, модели бизнес-процессов, системные диаграммы, организационные схемы, схемы взаимосвязей и, конечно, диаграммы Ганта. Возможна коллективная работа в режиме реального времени, есть мгновенная синхронизация, доступны шаблоны.

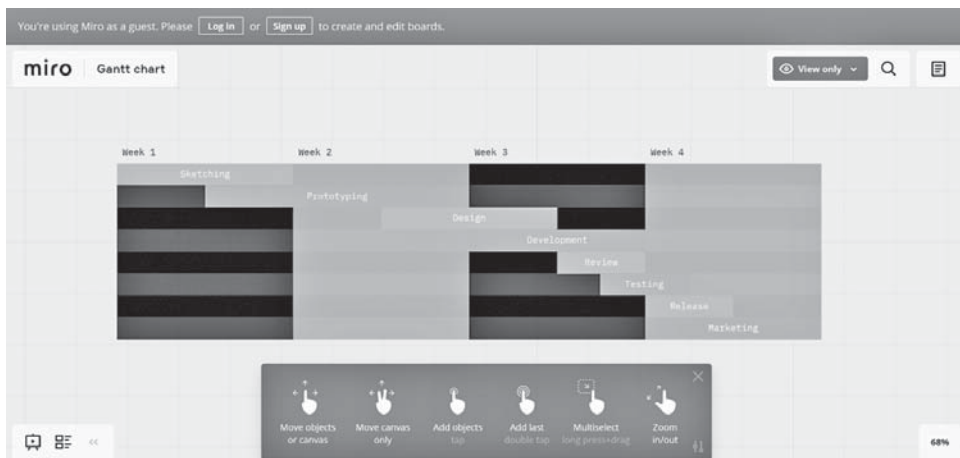


Можно импортировать данные из Excel, Zapier, Salesforce, LinkedIn Sales Navigator и других платформ, а также генерировать схемы на основе готовых данных и автоматических раскладок. Есть интеграции с G Suite, Atlassian, Slack и Microsoft Office. Среди особо приятных плюшек — контроль версий с сохранением истории изменений и групповой чат.

В пробной бесплатной версии есть какие угодно шаблоны диаграмм, но только не PERT — это минус. И да, придется уделить время на то, чтобы разобраться с интерфейсом.

Miro

Когда-то был простой и быстрый сервис RealtimeBoard, но его поглотил гигант Miro. И теперь в его экосистеме можно строить не только диаграммы Ганта, но и канбан-доски, карты путешествий пользователей (CJM), делать SWOT-анализ и еще пачку всяких специфических маркетинговых и бизнесовых диаграмм. Вообще любых, но только не сетевых графиков (уПС).



Сервис интегрируется с миллионом других: Slack, Notion, Google Drive, GitHub, Jira, Dropbox, Trello, Zendesk, Evernote.

5.7.2 СПЕЦИАЛИЗИРОВАННЫЙ СОФТ

GanttPro

Инструмент, заточенный специально под построение диаграмм Ганта. Удобен тем, что можно использовать несколько диаграмм, сделанных под разные команды или группы задач. Есть командная работа и делегирование.

Среди специфических возможностей, которые сильно пригодятся менеджерам проектов — для каждой задачи можно установить точное время выполнения, приоритет и зависимость от других задач. Отдельная фишка — опция планирования и управления ресурсами, учет времени и канбан-доска для отслеживания прогресса задач. Есть и расчет критического пути (который часто считают с помощью сетевых графиков, но как таковых PERT-диаграмм внутри GanttPro нет). А вот из интеграций только Jira, и та пока в бета-версии.

Smartsheet

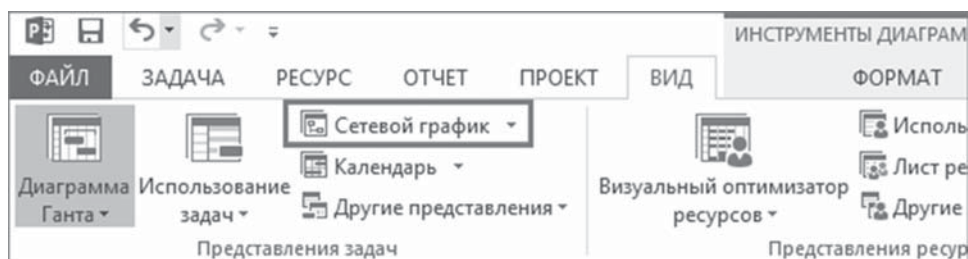
Не просто сервис для создания диаграмм, а целая платформа для управления проектами. Переключаясь между режимом календаря, карточек или диаграммой Ганта, вы видите все ваши задачи, и их прогресс легко отслеживать. Есть коллективная работа, напоминания,

отчеты и всякая разная статистика. Для построения PERT-диаграмм как таковых инструментов нет, зато в статьях по этой теме можно найти удобный калькулятор, который запросто вычислит вам средневзвешенное время для выполнения задачи на основе трех оценок времени.

Сервис интегрируется с продуктами Microsoft и Google, а также со Slack, Salesforce, Skype, Dropbox и прочими приложениями.

Microsoft MS Project

MS Project — еще более специфичный сервис, управление проектами в котором базируется на диаграмме Ганта и базовых планах. Инструмент довольно мощный, но требует сноровки. Здесь можно настраивать сценарии «что, если», устанавливать дедлайны, настраивать временные шкалы и видеть весь процесс. И да, здесь, помимо диаграмм Ганта можно построить PERT-диаграммы.



Merlin Project

Крутой сервис, доступный только для пользователей MacOS. Обещает самые красивые диаграммы Ганта в мире (правда!), четкую иерархию задач, удобное планирование, настройку зависимостей, а еще — опции создания майндмэпов и канбан-досок. PERT-диаграммы при желании тоже можно построить, и они будут такими же красивыми, как диаграммы Ганта.

Отдельный респект за вкладку с загруженностью сотрудников: удобно выбирать исполнителей на новые задачи, видя их реальную загрузку. Из интеграций доступны продукты Microsoft и сервисы из экосистемы Apple.

Битрикс24

Одна из самых известных программ по организации бизнес-процессов, которая подходит для компаний от 50 человек. Здесь можно планировать задачи, проекты и время на их выполнение, видеть про-

цесс в календаре, делиться документами. Правда, есть мнение, что для организаций до десяти человек Битрикс скорее мешает работать, чем помогает.

В Битрикс24 с PERT-диаграммами туго, зато для диаграмм Ганта есть продвинутый инструмент: меняются условия — автоматически меняется диаграмма, прогресс задач окрашивается цветом, по каждой задаче при клике выдается подробная информация об исполнителях, сроках и прочих важных аспектах.

5.8 ЧТО ДЕЛАТЬ, ЕСЛИ ПРОЕКТ НЕТИПОВОЙ

Если по проекту нет никакой статистики и с подобным вы никогда не сталкивались, вариантов у вас не так много. Например, можно привлечь более опытного специалиста, который в состоянии разобраться: провести ресерч, покопаться и сделать более адекватную оценку. Если такого человека нет — декомпозируйте. Так вы снижаете свои риски.

К сожалению, процедура декомпозиции (как и привлечение более сильного специалиста) — это дорого. Если в течение дня требуется оценить несколько нетиповых проектов, то времени на какие-то другие задачи банально не остается, и это сильно демотивирует.

Вам может прийти в голову шальная мысль: можно же взять специально обученного человека и посадить его делать эту монотонную работу за вас. Увы, это маразм: такие задачи выматывают и морально, и физически, поэтому он тоже будет плакать идохнуть. А вам ведь еще проверять за ним работу придется.

Нет ничего зазорного в том, чтобы признаться, что технология вам не знакома, и попросить у заказчика дополнительно какое-то количество денег на то, чтобы оценить проект адекватно в этих условиях. Но если клиент сложный, остается только один вариант: делать оценки с большими вилками, где разброс «от» и «до» будет не 5%, а 25%, 50% или даже 150%. На нетиповых проектах на этапе предварительной оценки, когда вы еще не занимались аналитикой и не погружались в проект, большие вилки — это нормально.

Кроме того, не за всякий проект нужно хвататься. Если это овер-сложно для вашей команды — и себя угробите, и клиента подставите. Проект должен находиться в области ближайшего развития команды. Лучше делайте хорошо то, что умеете, чем плохо то, что хочет клиент.

После предварительной грубой оценки берите деньги за аналитику, делайте более детальную декомпозицию, прописывайте бэклоги и необходимые технические документы — за счет этого вилочная оценка будет постепенно сужаться. При строительстве дома применяют такую же схему: пока нет четких чертежей, вам могут озвучить цену «среднюю по больнице». А вот финальную стоимость вы узнаете лишь на этапе отделки: потому что от материалов отделки сильно-сильно зависит итоговая сумма строительства.

Бывает, что клиент уже «подстелил соломку»: как-то где-то сделал техническое задание (оно же — ТЗ) и говорит, что этап аналитики ему не нужен. И здесь возможны два варианта.

В первом случае это будет тоненькое, хлипкое и не очень четко сформулированное ТЗ. Здесь ничего страшного: вы просто находите дыры в этом ТЗ. Они там гарантированно есть (можно поспорить на коньяк). Обсуждаете это с заказчиком, приходите к выводу, что ни он, ни вы не понимаете до конца задачу и конечный результат работ. А дальше просто делаете аналитику и спокойно работаете по обычному алгоритму.

Второй случай — толстые ТЗ — давайте разберем подробно.

5.8.1 ЧТО ДЕЛАТЬ, ЕСЛИ КЛИЕНТ ПРИСЛАЛ БОЛЬШОЕ, ТОЛСТОЕ, НУДНОЕ ТЗ

Гораздо сложнее бывает в ситуации, когда ТЗ заказчика толстое, большое, выглядит как солидный документ и кажется хорошо проработанным. В этой ситуации могут спасти прототипы: если они есть и четко соответствуют этому техническому заданию. Тогда оценку можно сделать по прототипам, а затем внимательно прочесть ТЗ и отметить в нем все непонятные места цветом, чтобы после обсудить с заказчиком и уточнить эти моменты.

Метод светофора

Метод универсален: применим и к прототипам, и к ТЗ, и к текстам, и даже к договорам. Там, где вы все понимаете, помечаете документ зеленым цветом. Там, где есть сомнения — желтым. А там, где написанное вам кажется полнейшим бредом, помечайте красным.

К сожалению, такой подход адекватен, если техническое задание объемом на 30–60, максимум 80 страниц. Если страниц куда больше (скажем, 300), то воспринять его целиком ни одна человеческая голова уже не способна. В этом случае, сколько бы раз вы его ни читали, все равно будут какие-то «дырки», разночтения и недопонимания, из-за которых риск сделать неверную оценку возрастает. И тут вариантов тоже немного.

Вы либо проводите аналитику, закладывая на нее довольно много ресурсов: внимательно читаете, проверяете, стараетесь выявить узкие места. Но потом все равно будете бояться, что какие-то тонкости по ходу проекта вы пропустите :)

Другой вариант — вы просто живете с этими рисками, закладываете их в смету и пытаетесь как-то «пропихнуть» этот проект дальше. Но такой путь, как правило, невыгоден ни клиенту, ни вам: придется закладывать большое количество ресурсов на риски или делать проект за ваш счет (клиенту понравится такой вариант, а вот вам вряд ли).

Третий вариант — правильный. **Декомпозировать проект и его огромное ТЗ на спринты, договориться с заказчиком о поэтапной работе и выполнять проект итеративно. Вам нужен Time & Material-подход.** В этом случае никакой предварительной оценки по цене у клиента не будет, или будет очень примерная, с разбросом в 40–60%. Вы, скорее всего, сможете как-то вилочно оценить количество спринтов на весь проект, но финальная его стоимость появится у заказчика лишь в самом конце.

Совет

На нетиповых проектах договаривайтесь на итерации и работу по Scrum, честно показывайте оценки. Либо просто не ввязывайтесь в такие проекты, поскольку доля риска в них очень высока. А если уж ввязались в эту авантюру, включайте в смету все возможные риски, какие только видите, и живите с ними на проекте. Других вариантов по большому счету нет.

Если вы чувствуете, что по задаче оценка слишком раздута, — декомпозируйте. И вообще, при любых сомнениях — декомпозируйте, погружайтесь глубже.

5.8.2 ТЗ, ПРОТОТИП, ДИЗАЙН, СМЕТА: ЧТО ВАЖНЕЕ?

Scrum-подход говорит о том, что ТЗ вообще не требуется. Нужны бэклоги, спринты, фокусировка команды на важном, поэтапный запуск и сбор обратной связи. И итеративное уточнение задачи. Это хорошо работает в продуктовой разработке и примерно в половине сложных проектов — заказных.

Остальные 50% проектов заказной разработки настаивают, чтобы был **водопад** и все «шло по ТЗ».

Я предлагаю такую схему. Если ТЗ для клиента критически необходимо — окей, пусть оно будет. Идем по нему. Это наш план. Но! Оставляйте заказчику возможность менять все по ходу работы. А ТЗ использовать как источник задач для бэклога.

Это достигается за счет поэтапной работы и оплаты/отгрузки каждого этапа отдельно. Да, есть ТЗ, но мы идем спринтами. И можем отклоняться от ТЗ по ходу пьесы.

В этом случае приоритет имеет тот артефакт, который появился самым последним. А это будет смета на конкретный этап. Итак, ТЗ имеет приоритет перед прототипами. Дизайн имеет приоритет перед ТЗ. А главное всего — смета конкретного этапа.

По сути, мы используем сметы как бэклог, а ТЗ — как набор постановок задач на конкретные функции.

Итак. Приоритет имеет тот артефакт, который появился позднее. В случае итеративной работы — это смета на очередной этап или итерацию.

5.9 МЕТОД ОСВОЕННОГО ОБЪЕМА

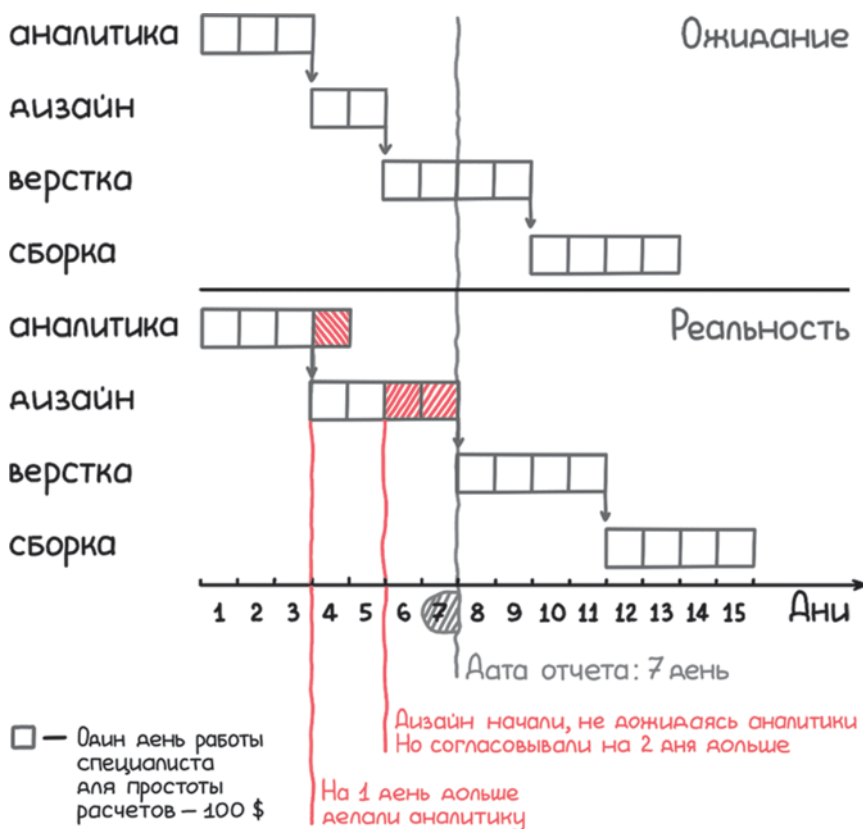
Есть способ быстро составить реальные прогнозы о расходе ресурсов проекта, если срываются сроки — применить метод освоенного объема. Он очень похож на диаграммы Burndown Chart, но имеет свои нюансы.

Этот метод пришел в менеджмент из строительной сферы. Там затягивание со сроками сдачи проекта грозит колоссальным увеличением бюджета. Не уложились в срок — все, оформляйте простой до весны.

Как применить метод

Допустим, вы довольный менеджер, который раскидал задачи в календарики исполнителей и предвкушает, как проект сдастся точно в срок. Но анализ затянулся на день дольше (клиентский рынок сложный, конкуренты «кусаются», а преимущества надо искать с лупой). И вот уже дизайнеры начали работу без аналитики.

Вы думаете: «Ничего, как-нибудь сейчас наверстаем темпы». Но они только замедляются. Когда аналитика готова, дизайн перерисовывается и переутверждается, а это еще три дня работы. Уже подошел день сдачи промежуточных этапов, но ничего нет. Все сотрудники злые и сваливают друг на друга вину. Одним словом — ад.



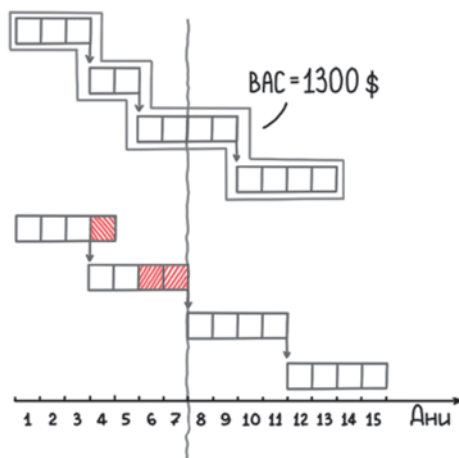
В такой ситуации все вокруг требуют от вас реальных прогнозов: когда проект завершится, сколько денег еще нужно, каков процент выполнения. Но вы можете только предполагать. А лучше бы посчитали на калькуляторе.

Метод освоенного объема и его ключевые показатели помогают понять, что и где пошло не так.

Budget at Completion

Сумма всех составляющих бюджета, установленных для работ, выполняемых в рамках проекта

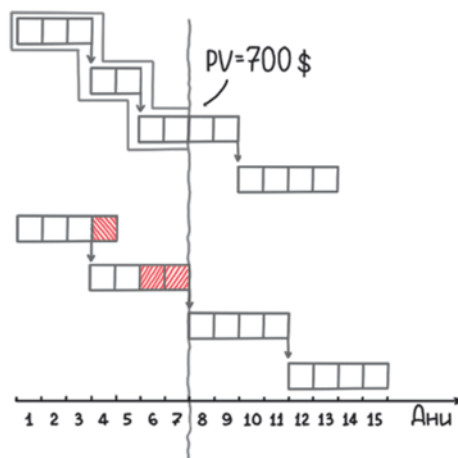
Сокращения: БПЗ, ВАС



Planned Value

Запланированная стоимость работ за рассматриваемый промежуток времени (запланированный объем)

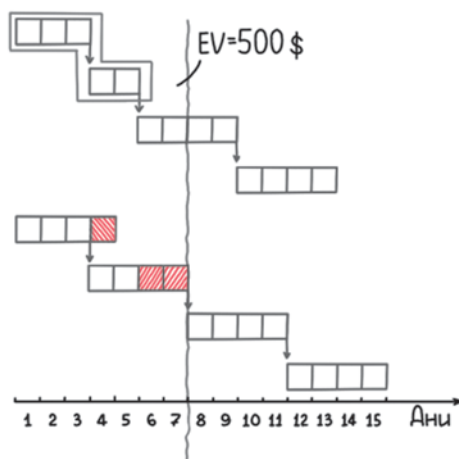
Сокращения: ЗО, БСЗР, PV



Earned Value

Запланированная стоимость фактически выполненных работ за промежуток времени

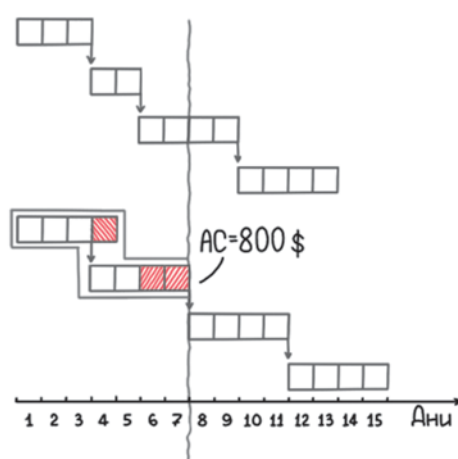
Сокращения: ОО, БСВР, EV



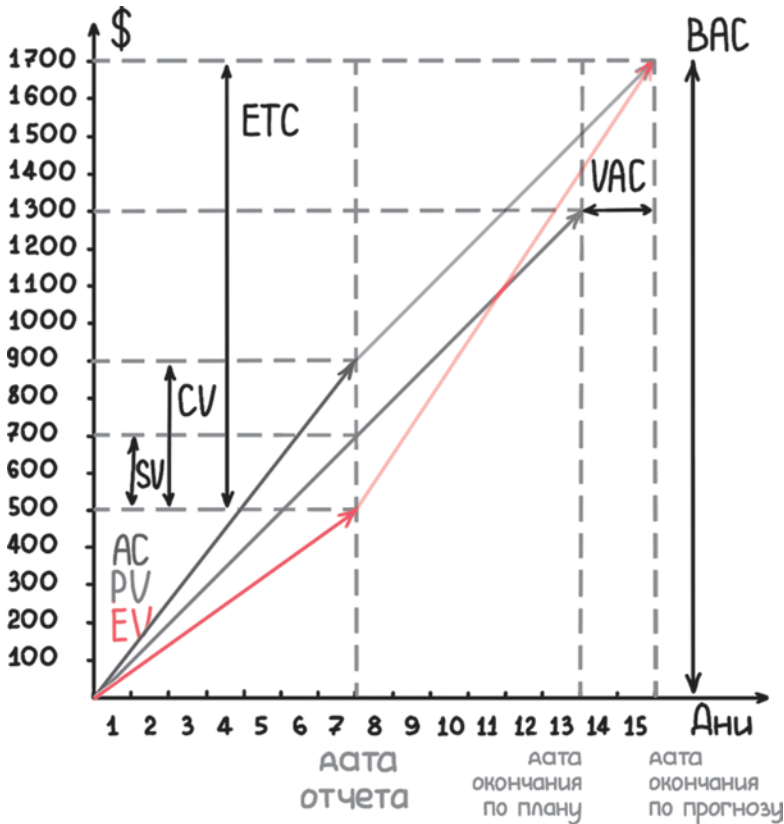
Actual Cost

Фактическая стоимость выполненных работ. Также может называться "Фактические затраты"

Сокращения: ФО, ФСВР, AC



На схемах видно, как расходится план с фактическим положением дел. А простые графики помогают оценить состояние дел и спрогнозировать, как проект будет развиваться дальше.



Прямая планируемой стоимости выше прямой ценности выполненных работ и ниже прямой фактических затрат на проект.

Чтобы не рисовать графики, можно сразу посчитать, есть ли перерасходы бюджета и времени, сколько проект будет стоить, когда завершится, какая будет отдача от вложений. Также расчеты покажут нужную производительность и процент выполнения работ.

Чтобы быстро понять состояние проекта, достаточно рассчитать четыре показателя: Cost Variance (отклонения бюджета), Cost Performance Index (отдача вложений), Schedule Variance (отклонения в расписании) и Schedule Performance Index (укладываемся ли в сроки). Если сравнить результаты, то сразу видно, где идет экономия ресурсов, а где перерасход.



0 бюджета

Есть ли отклонения по бюджету?

Если $CV < 0$, значит, перерасход есть
На графике $CV = 500 - 800 = -300\$$

$$CV = EV - AC$$

Сколько денег вы получаете за каждый вложенный доллар?

Если $CV < 1$, значит, вы вкладываете больше, чем получаете
На графике $CPI = 500/800 = 0,625$

$$CPI = EV/AC$$

Сколько проект будет стоить по завершении, если дальше вы будете укладываться в срок?

На графике $EAC = 1300/0,625 = 2080\$$

$$EAC = BAC/CPI$$

Сколько денег вы получаете за каждый вложенный доллар?

На графике $ETC = 2080 - 800 = 1280\$$

$$ETC = EAC - AC$$

Какой будет разница между запланированным и реальным бюджетом?

Если $CV < 0$, значит, у вас перерасход
На графике $VAC = 1300 - 2080 = -780\$$

$$VAC = BAC - EAC$$



0 сроков

Есть ли отклонения от расписания?

Если $SV < 0$, значит, отклонения есть
На графике $SV = 500 - 700 = -200\$$

$$SV = EV - PV$$

Укладываетесь ли вы в сроки?

Если $SV < 1$, значит, вы не укладываетесь в сроки
На графике $SPI = 500/700 = 0,7$

$$SPI = EV/PV$$

С какой производительностью нужно работать, чтобы уложиться в сроки?

На графике $TCPI = \frac{1300 - 500}{1300 - 800} = 1,6$

$$TCPI = \frac{BAC - EV}{BAC - AC}$$

На сколько процентов выполнен проект?

На графике $PC = \frac{500}{1300} = 0,38 = 38\%$

$$PC = \frac{EV}{BAC}$$

	$SV < 0$ $SPI < 1$	$SV > 0$ $SPI > 1$
$CV > 0$	😊 Экономия бюджета	😊 Экономия бюджета
$CPI > 1$	😞 Отставание по срокам	😊 опережение сроков
$CV < 0$	😞 Перерасход бюджета	😞 Перерасход бюджета
$CPI < 1$	😞 Отставание по срокам	😞 Отставание по срокам

Метод освоенного объема помогает оценить реальные расходы ресурсов, если сроки затянулись. И не надо быть Перельманом, чтобы рассчитать показатели и вывести проект из кризиса.

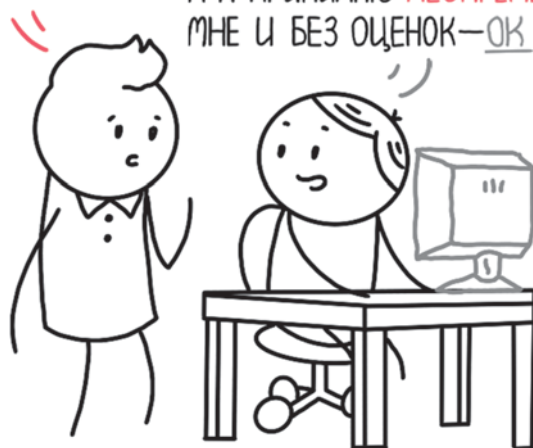
5.10 ДВА ТИПА ЛЮДЕЙ. ИТОГИ

Существуют два типа людей. Те, кто ненавидит неопределенность. И те, кто ее принимает. Нужна смелость и опыт, чтобы давать оценки. И тем не менее, оценки и прогнозы нам нужны.

Я ненавижу неопределенность!

Оцени мне эти задачи

А я принимаю неопределенность!
мне и без оценок — ОК



К сожалению, чем опытнее человек, тем больше времени на подстраховку он закладывает в свои задачи. И опять же, к сожалению, эта подстраховка в большинстве случаев будет разбазарена. Почему так происходит — настоятельно рекомендую посмотреть лекцию № 3 Элияху Голдратта, создателя теории ограничений, из его сателлитной программы. Там работает и Мерфи, и психология, и особенности создания продуктов. Вкратце:



Лекция № 3
Элияху
Голдратта

1. Синдром студента. Начинаем проект в последний момент.
2. Если мы получили выигрыш на каком-то этапе работ — он, скорее всего, будет потерян на следующих этапах, поскольку производительность всей цепочки работ равна производительности бутылочного горлышка.
3. Однако задержки будут накапливаться.
4. В точках интеграции нескольких компонент вероятность начать сборку этой компоненты вовремя зависит от вероятностей закончить вовремя составные компоненты. Не суммой, а умножением. Если собираем пять компонент, каждая из которых может быть готова к сроку с вероятностью 80%, у нас есть всего 32% вероятности начать интеграцию вовремя. Но в современном программном обеспечении — сотни компонент, и вероятности там не в вашу пользу.
5. Люди, которые получили по башке за свои оценки, будут добавлять все больше и больше подстраховки. Которая будет разбазарена.

Итак. Современная разработка софта — это сложно и дорого. Всегда было сложно и дорого, и ставки только растут. Если можно не разрабатывать — лучше не разрабатывать.

Оценка сроков и стоимости — самая сложная задача, потому что требует высокой квалификации и работы с неопределенностью. Неопределенность бьет по башке. Но оценки и сроки — нужны. Этому можно научиться.

5.11 ДЛЯ ТРЕНИРОВКИ

- ▶ Посмотреть уже упомянутую лекцию № 3 из спутниковой программы Голдратта. Как разбазаривают подстраховку? Что с этим делать? Чем может помочь Scrum, Канбан и Planning Poker для этого?
- ▶ Возьмите сайт своей компании (или сайт, который вам хорошо знаком). Декомпозируйте на экраны и функции. Составьте смету. Постройте диаграмму Ганта, как бы шла разработка такого сайта. Что бы вы распараллелили?
- ▶ По этому же сайту разберите пару страниц поэлементно и ответьте на вопрос «Откуда эта хрень на странице?!» для каждого элемента. Используйте схему из § 5.3.

Успехов!

ЛИТЕРАТУРА

- ▶ Фредерик Брукс, «Мифический человек-месяц».
- ▶ Слава Панкратов, «Черная книга менеджера».
- ▶ Нассим Николас Талеб, «Антихрупкость. Как извлечь выгоду из хаоса».
- ▶ Нассим Николас Талеб, «Одураченные случайностью. Скрытая роль шанса на рынках и в жизни».
- ▶ Нассим Николас Талеб, «Рискуя собственной шкурой. Скрытая асимметрия повседневной жизни».
- ▶ Нассим Николас Талеб, «Черный лебедь. Под знаком непредсказуемости».

ПОЯСНЕНИЯ

Data Time — тип данных, который используется для величин, содержащих информацию как о дате, так и о времени.

Exception — класс исключений.

General Protection Fault — общая ошибка защиты в ответ на нарушение.

CMS-система — система управления контентом на сайте: например, Wordpress или 1С-Битрикс.

Сущность — то, о чем нужно хранить информацию в базе данных. Сущности могут моделировать конкретные понятия (клиенты, товары, звонки, заказы) и абстрактные (менеджер отвечает за клиента, служба доставки принимает заказ).

Облака S3 (объектные хранилища) — сервисы хранения объектов, предлагаемые поставщиками облачных услуг.

МОСК-объект — заглушки или, по-другому, фиктивная реализация кода, предназначенная для тестирования.

Табы — вкладки, где можно разместить большой объем информации, используя минимальный объем места. Отлично чувствуют себя в интернет-магазинах — обычно там размещают характеристики, описание товара, отзывы и все такое.

Параллакс — эффект при скроллинге мышкой, когда объекты на странице тоже движутся вслед за скроллом.

Регрессия кода — тестирование ради обнаружения ошибок в уже протестированных участках исходного кода.

Апдейт — обновление.

Водопад — водопадная модель процесса разработки, где процесс выглядит как последовательный поток: аналитика, проектирование, реализация, тестирование, интеграция и поддержка.

Часть 6



ДИЗАЙН. УПРАВЛЯЕМЫЙ КРЕАТИВ

В этой главе мы разберем управление креативным коллективом. Акцент сделаем на дизайн и дизайнеров. Однако будут и общие принципы и идеи управления. Типа устойчивости по предсказуемости или транзакционным издержкам на правочки.

Кажется, что управлять дизайнерами, этими креативными котятками, — невозможно. Либо все хорошо, либо ужасно!

Как разработать wow-дизайн?

1. Берем дизайнера, который рубит фишку. Где берем — не спрашивайте. Это проблема HR, а не руководителя проекта. В конце концов, позовем соцсети на помощь.
2. За пять минут объясняем задачу. Отдаем ему испачканную салфетку со своими каракулями. Отправляем рисовать.
3. Он приносит результат. Мы ему — правки. Он — результат, мы — правки! Намылить, смыть, повторить.

Если получилось — все молодцы. Ну, или как минимум — менеджмент. Ведь только благодаря ПРАВКАМ (!) удалось достигнуть нужного результата. Если не получилось — значит, дрянного дизайнера подсунули. Замените.

Работает? Безусловно! Будни мелкого фриланса.

Минус — непредсказуемость и неуправляемость. Плюс: для менеджера — прикрыты тылы и минимум трудоемкости. Сиди да картинки смотри — нравится/не нравится. Наш мозг любит всякие такие штуки с оценкой чужих стараний на субъективном уровне. А еще любит лениться и избегать ответственности.

При этом все прекрасно понимают важность дизайна. Если дизайн и интерфейсы «не очень» — можно даже не запускаться. И, особенно на больших проектах, хотелось бы сделать процесс дизайна более управляемым и предсказуемым.

В нашем арсенале десятки инструментов, подходов, приемов и техник, которые помогают этого достичь. Обратная сторона — инструментами надо РАБОТАТЬ. А мозг не любит, ну вы помните.



В этой главе разберем **экстремумы**: минимальный процесс организации дизайна и дизайнеров (готово, разобрали чуть выше) и навороченный. Такой, который позволяет не только долго работать над большими проектами, но и управлять всем тем, что не приколочено. После этого останется комбинировать методы и подходы в зависимости от задачи и уровня самостоятельности дизайнера. В конце концов это наша работа — строить процессы.

6.1 АРТЕФАКТЫ ДИЗАЙНА. КРАТКИЙ СПРАВОЧНИК ПО ПРИМЕНЕНИЮ

Давайте введем дюжину понятий из мира дизайна. Пригодится для построения процессов и разговоров с дизайнерами на одном языке.

Бриф — краткая декларация о намерениях. В ней прописываются базовые параметры будущего программного продукта, графического дизайна или иного проекта. Как правило, информация в нем очень общая, размытая. Нужен для «поговорить». В работе практически бесполезен.

Референс (англ. reference — справка, сноска) — изображения, рисунки, примеры других проектов, сайтов, мобильных приложений и интерфейсов, фотографии и т. д. Загружается в мозг участников перед началом работы, чтобы точнее передать детали, получить дополнительную информацию, сгенерировать идеи. Референсы используются как примерное представление того, что хотелось бы видеть. Или как попытка предоставить собеседнику на чужих примерах свое представление.

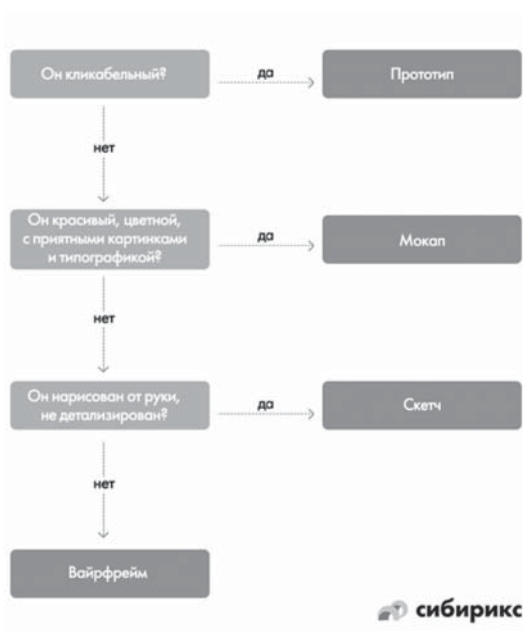
Мудборд (англ. mood board — доска настроения) — подборка референсов, шрифтов, иконок и т. д., выполненных на одном листе или экране. Необходим, чтобы определиться со стилистикой, направлением и настроением, которые должен передавать будущий продукт. Удобно собирать в Miro или графическом редакторе.

Фирменный стиль — набор элементов (шрифты, логотипы, цветовая палитра, текстуры, паттерны и т. д.) и правил их применения. Что можно, а что нельзя. Как правило, является частью **брендбука**, в котором, кроме фирменного стиля содержится маркетинговая составляющая бренда. Иногда выносится в отдельный документ — **гайдлайн** (гайд, мини-гайд), содержащий только правила и примеры использования.

Скетч (Sketch) — набросок от руки. Карандашом, на планшете или на маркерной доске. Нужен для быстрой передачи идеи из головы дизайнера (менеджера, аналитика) в головы окружающих. Когда дело касается визуальных фишек, намного проще сделать набросок карандашом, чем объяснять идею на пальцах.



Скетч для «Молочной Культуры». Карандаш, бумага.



Вайрфрейм (Wireframe) — черно-белый (как правило) план страницы. Здесь намечается расположение элементов: кнопок, изображений, текстов. Нужен, чтобы определить, где какой контент будет размещаться. Вайрфрейм и комментарии к нему можно использовать для составления ТЗ.

Прототип (Prototype) — интерактивный вариант вайрфрейма. На прототипе уже не нужны пометки, что и как работает. Чтобы понять это, нужно просто кликнуть по месту, к которому у вас вопрос.

Нужен, чтобы согласовывать с заказчиком расположение блоков и кнопок и проводить юзабилити-тестирования. А еще по прототипу удобнее, чем по вайрфрейму, писать постановки, ТЗ и бэклоги.

Мокап (Москир или mock-up) — красивый вариант вайрфрейма. Тут уже появляются цвета, подбираются изображения, продумывается типографика. Получается реалистичная картинка приложения или сайта. Нужен, чтобы создать стиль элементов, продумать визуальные мелочи и согласовать их с заказчиком.

Макет — графический документ, готовый к дальнейшей верстке разработчиками. В идеале должен быть удобно организован для дальнейшей работы. Все отступы по шнуркам, слои подписаны, и т. д. В реальности за это придется побороться.

Есть мнение, что более привычный синоним мокапа — макет. Но это не точно. Если мокап — это, чаще всего, красивая картинка (иногда — анимированная), то макет — это исходный файл для дальнейшей технической работы.

Компонент — элемент интерфейса. Кнопки, выпадающие списки и т. д., оформленные в графической программе так, чтобы изменение компонента в одном месте меняло его внешний вид на всех макетах, где он применяется. Может иметь несколько состояний: нажатая кнопка, эффект при наведении, выпавший-свернутый список, форма без ошибок/с ошибками и т. д. Может иметь правила и принципы **кастомизации**. Например, ширина кнопок или полей ввода подстраивается под введенный текст. Далеко не любая дизайнерская программа поддерживает компонентный подход.

Дизайн-система — набор компонентов и правил их применения. Что-то типа гайдлайна для программного продукта, только гораздо шире. Весь интерфейс строится из компонентов, как из кирпичиков. Компоненты дизайна могут быть синхронизированы с компонентами верстки (frontend). Несмотря на привлекательность и перспективность такого подхода, дизайн-системы появляются только в зрелых проектах. Где идеология, стилистика и структура компонентов уже устоялась. Трудоемкая, но неизбежная в больших и долгоиграющих командных проектах штука.

6.2 О КРЕАТИВНОСТИ

Креативность, создание необычных идей — трудно прокачиваемый навык.

Складывается ощущение, что креативность либо есть, либо нет. Прокачивается долго и сложно — написанием текстов, чтением, самостоятельным просмотром десятков работ в день. Иногда помогают креативные методики (о них чуть позже). Остальное не очень-то легально.

Под видом креатива часто пытаются пропихнуть бездарную дичь (скрещивание ежа и носорога, см. стили «кубизм», «экспрессионизм», «примитивизм» и еще какой-нибудь «ИЗМ») с криками «я так вижу». **КГ/АМ.** Отличить такой «креатифф» от сильной работы очень просто: по вниманию к деталям.

Продуманность, аккуратность, качественные идеи нужно поощрять. Дизайн ради дизайна, текст ради текста, минимализм ради сокрытия лени, неумения рисовать, очередной шаблон вордпресса или унылый лэндинг пейдж — выявлять и не допускать.

Креативность легко убить критикой на неправильных этапах и невниманием к деталям.

6.3 БУДНИЧНАЯ САКРАЛЬНОСТЬ

В первую очередь давайте разберемся с сакральностью, которая исторически нагромоздилась вокруг дизайнерских «трудовых будней». Ниже — сборник высказываний, мнений. Как дизайнеров, так и клиентов. Часто — противоречивых и не очень-то логичных.

«Я профессионал, мне виднее». Почти «я художник, я так вижу». Дизайнер может неожиданно резко начать реагировать на критику его работы, называя вас, менеджера или заказчика — непрофессионалом, который мешает сделать «нормально». Во многом тут играют роль когнитивные искажения, которые мы с вами разбирали: критика работы воспринимается как личное оскорбление. Понять по резкости высказываний, прав на деле дизайнер или нет — та еще задача.

«Нам нужна фишка, wow-эффект. Мы вас не ограничиваем». Как правило, за этим желание снять с себя ответственность, а вовсе не желание дать зеленый свет свободе творчества. Например, если компания продает посредственный продукт или услугу, ей нечем гор-

даться, нечего показать и рассказать про себя — именно такими фразами маркетологи, директора стараются перевалить ответственность на дизайнера (внезапно — именно на дизайнера) за свой угрюмый продукт. Ответственность за имидж компании или даже за продажи.

«Дизайн должен быть незаметным и не отвлекать». Если мне нужен интерфейс управления спортивным самолетом — мне не до wow-эффектов. Но это противоречит пункту **«Нам нужна фишка, wow-эффект»**. И попробуй эти требования подружи в одном проекте. Вспотеешь!

«Пусть дизайнер подумает». Это попытка манипуляции. Вот пускай нам чего-то покажут, а мы уже скажем, нравится нам или нет. Угадывайте. Не угадал — значит, плохо подумал.

«Время дизайна — прошло. Всех интересуют конверсии». Речь идет о том, что не нужен дизайн в стиле «дорого-богато», лучше, чтобы кнопка «звонябля» побыстрее нападала на пользователя и отбирала все деньги.

«Предусмотрите в дизайне повышение продаж и зафиксируйте это в контракте». Тут логическая цепочка такая:

1. UI+UX должны обеспечивать целевые действия.
2. А это — продажи.
3. Профессионалы? Обеспечивайте!

Дизайнер магическим образом компенсирует дрянной продукт. Неадекватные цены. Нецелевой трафик. Топорный контент. Совковый клиентский сервис. И правочки от жены заказчика. Обмечтаться!

Из диалога:

— Ок, если за результат отвечаем мы — значит, вы с правками не вмешиваетесь?

— Нет! Правки будут. Будут!

— Эээ...

Внятного алгоритма, кроме тестирования и замеров конверсий на реальном трафике, пока еще никто не придумал. Эту технику разберем в параграфе о Data Driven Design.

«Для любой задачи правильный дизайн может быть только один. Задача имеет только одно решение. Либо она решена, либо нет». Такое тоже встречается, но уже у дизайнеров. Говорит о зашоренности мышления, догматизме или хитрожопости. В научной литературе зафиксировано не менее 400 доказательств теоремы Пифагора, например.

«В дизайне можно все, что нельзя» — это уже Тема Лебедев. В противовес предыдущему высказыванию.

Основная сложность организации дизайн-процесса в том, что мы должны синтезировать из, грубо говоря, ничего что-то хорошее. Да еще и удовлетворяющее несформулированным требованиям случайных людей. Требования часто нерабочие и на уровне вкусовщины. Например, мне попадалось такое: «фиолетовый — цвет самоубийц, его использовать нельзя».

Или маркетинг может хотеть креатив и фишку, с анимацией вжух-вжух. А отдел продаж хочет товары на главной странице и ничего кроме. А директору важнее видео о компании с ним на заставке и что компании 20 лет. Видели старый ролик «Make my logo bigger»? Обязательно посмотрите (и почитайте параграф 6.13.2 об этом).

Вкусовщина и личные цели у дизайнера? А как же! Дизайнеру до лампочки конверсия. Ленточку с AWWARDS подавай! Важно, что скажут другие дизайнеры из тусовки.

И у вас, как у руководителя проектов, будет много работы, чтобы учесть и примирить высказанные или подразумеваемые интересы всех сторон. И выпустить при этом годный продукт.

6.4 КАЧЕСТВО ДИЗАЙНА

Что такое «качественный дизайн»? С учетом субъективности и са-
кральности — сразу и не скажешь. Наведем ясности и четкости.



Оказывается, «качество» хорошо раскладывается по семи слоям: техническому, правовому, экономическому, психологическому, технологическому, этическому и эстетическому. Сами слои придуманы и описаны для подготовки к сложным переговорам Владимиром Константиновичем Тарасовым в книге «Искусство управленческой борьбы». Однако разбиение мутного параметра «качество» по этим слоям — ключ для управления дизайном через управление качеством.

Слой	Что подразумевает	Что означает в контексте дизайна	Как управлять
Технический	Что (объекты). Где (какие особенности местности, пространства, территории). Когда (какое время года, какая погода, как это влияет на ситуацию). Кто (кто принимал участие, кто что слышал, какой информацией обладает). Законы физики.	Макеты, исходники. Внутренняя техническая организация макетов, слоев, компонентов, отступов. Контент, предоставленный до начала работ или разработанный во время выполнения дизайна. Правила хранения и передачи макетов и организации доступа к ним. Состав команды (кто дизайнер, кто арт-директор). Формальные и неформальные требования к макетам (мудборды, брифы и т. д.). Кто из команды	Технические чек-листы и требования к макетам, карты контента. Ревью (просмотр) исходников. Арт-дирекшн. Вычитка копирайтером, UX-копирайтинг. Регламенты хранения исходников и резервных копий, проведения брейнштормов и т. д.

Слой	Что подразумевает	Что означает в контексте дизайна	Как управлять
		был ознакомлен с требованиями. Какие мероприятия проводились (брейн-штурмы, внутренняя приемка макетов и т. д.). Кто участвовал?	
Правовой	Нормы права и обычая. Все, что связано с юридической стороной вопроса. Какие правовые взаимоотношения существуют между лицами? Какими документами регламентируется происходящее (договоры, положения, законы)? Какие последствия могут наступить? Кто действует в соответствии с нормами, а кто нет?	Авторские права и права на использование макетов, шрифтов, иконок, фотографий, видео, изображений, 3D-моделей и т. д. Какие последствия могут наступить при использовании этих макетов и изображений? Переданы ли макеты (исходники) в должном виде? Юридическая достоверность в текстах, использованных в макетах.	Контракты, лицензионные соглашения. Регламенты покупки и использования изображений, иконок, видео, 3D-моделей и шрифтов.

Слой	Что подразумевает	Что означает в контексте дизайна	Как управлять
Экономический	Что выгодно, а что нет. Все материальные ресурсы, прибыль, активы. Какую выгоду получает каждая из сторон? Каков предел, ниже которого сделка становится невыгодной? Какие материальные последствия могут наступить в том или ином случае? Какие есть ресурсы у сторон?	Количество часов, заложенных в бюджете на работу дизайнера. Предусмотренное количество внесения правочек. Помогает ли дизайн обеспечить конверсии. Бюджеты для покупки или создания видеороликов, оригинальных иллюстраций, 3D-моделей.	Система управления проектами. Контракты. Data Driven Design.
Психологический	Мотивы, повлиявшие или влияющие сейчас. Восприятие ситуации сторонами. Чего хотели и чего боялись участники процесса?	Референсы и примеры, на основе которых разработаны концепции. Акценты и ощущение, которое требовалось донести в дизайне (безумный креатив, спокойствие, теплоту,	Брифование, мудборды, предварительные демонстрации, подача готовой работы, мокапы.

Слой	Что подразумевает	Что означает в контексте дизайна	Как управлять
	Под чьим влиянием находились? Какие чувства и эмоции могут испытывать участники процесса? По поводу чего? Кто и от кого зависит в психологическом плане? Как отреагируют люди, если ситуация изменится в том или ином направлении?	технологичность и т. д.). Внутренние мотивы каждого из участников и заказчиков. Скрытые участники процесса, влияющие на него, но не участвующие напрямую. Степень эмоциональности обратной связи, постановок и проводимых демонстраций.	Степень подготовки к демонстрациям и харизматичность их проведения.
Технологический	Технология производства, продажи, принятия решений, поставки, обработки претензий и т. д. Также речь может идти и о бизнес-процессах, IT-технологиях.	Выбранные форматы графических файлов, систем разработки дизайна и систем верстки. Соответствие дизайна техническим требованиям. Системы отступов, модульные сетки. Готовность макетов к дальнейшей верстке: системы	Чек-листы с перечнем необходимых состояний компонентов. Тестирование дизайна на уровне QA и на уровне frontend-разработчика. Доводка дизайна во время верстки.

Слой	Что подразумевает	Что означает в контексте дизайна	Как управлять
		отступов, предусмотренность адаптива и всех необходимых состояний управляющих элементов, форм, сообщений об ошибках. Правила комментирования макетов и обработки комментариев.	UI/UX-тесты, коридорное тестирование. Совместные ретроспективы дизайнеров и frontend-разработчиков.
Этический	Кто и что ждал друг от друга? Ожидания могли быть созданы устным обещанием или молчаливо, вследствие принятой на себя роли. Были ли нарушения ожиданий какой-либо из сторон?	Предоставленные материалы, референсы, обратная связь. Точность и понятность обратной связи. Соответствие выполненной работы ожиданиям (явным и неявным). Этичность применения конкретных изображений, фраз, слоганов или демонстрации конкретной продукции.	Брифование, мудборды, предварительные скетчи, правочки.
Эстетический	Что красиво, а что нет.	Субъективные ощущения от дизайна.	Правильный подбор дизайна под задачу

Слой	Что подразумевает	Что означает в контексте дизайна	Как управлять
		Аккуратность. Соответствие ожидаемому уровню, выбранному стилю.	и требуемый стиль. Брифование, тестирование макетов, в том числе — на аудитории. Правки.

Итак, в дизайне в слове «качество» зашито много субъективных критериев. Но они поддаются управлению. Где-то лучше, где-то хуже.

Мы почти соскоблили налет сакральности и стратегически понимаем, как и чем контролировать технические аспекты качества. Давайте теперь посмотрим и разберем фундаментальный конфликт, зашитый в природу дизайнерской работы. Креативность и управляемость.

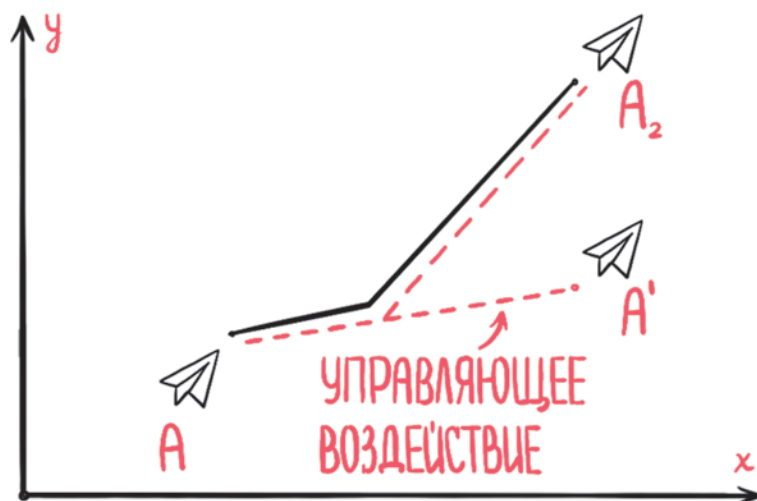
6.5 УПРАВЛЯЕМОСТЬ И КРЕАТИВНОСТЬ. САМОДВИЖЕНИЕ. УСТОЙЧИВОСТЬ ПО ПРЕДСКАЗУЕМОСТИ

В работах Щедровицкого по оргуправленческому мышлению (многokrатно переиздаваемых студией Лебедева) есть крутой тезис:

Управление возможно только в отношении объектов, имеющих
самодвижение.

Например, автомобилем, который стоит на месте, управлять нельзя. Стулом управлять нельзя. Его можно поставить, передвинуть или сломать. Это некая преобразующая деятельность, но это не управление.

Это очень важный момент, потому что если такого самодвижения нет, то нужно организовать первую деятельность как самодвижущуюся, чтобы потом ей управлять. Например, мы даем задачу дизайнеру, он начинает что-то рисовать, начинается самодвижение, и вот этим как-то можно управлять.



Допустим, у нас есть какой-то объект A , имеющий самодвижение. То есть, через какое-то время он перейдет в состояние A^1 , как бы сам по себе.

Допустим, нас A^1 не устраивает, нам бы хотелось другого состояния. Тогда мы делаем некое управляющее воздействие (даем пенделя, например), и в итоге объект переходит в некое состояние A^2 , которого мы и добивались. В этом, собственно, вся идея управления и состоит.

С креативными задачами (а дизайн именно такой) мы имеем здесь целых два «не слава богу».

Во-первых, у нас нет целевого состояния A^2 . Точнее, как правило, оно определено очень размыто. «Дизайн должен быть такой, чтобы няшно-секси-ми-ми-ми», что-то вроде того. Мы сами точно не знаем, чего именно хотим. И только посмотрев на результат (A^1), можем сказать:

- Не то!
- Можно ли точнее?
- Нет!

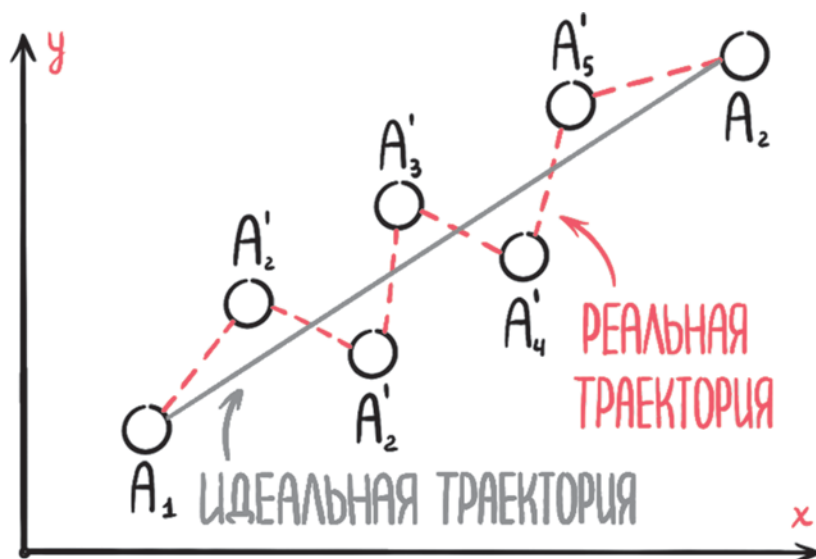
Кстати, обратите внимание, дизайнеры, услышав обратную связь в духе: «Ну, что-то не то», сразу пытаются выяснить: «А что конкретно не так?» (на самом деле, это очень плохой вопрос). И если на этот вопрос приходится отвечать вам — это очень парит.

Заказчики тоже ненавидят этот вопрос. Почему? А потому что поиск креативного решения вместе со всей неопределенностью перелетает к ним. Неопределенность бьет по башке. Лучше уж: «Пусть дизайнер подумает. Нам же нужна фишка, wow-эффект, мы вас не ограничиваем». Ну это как бы нормально, иначе бы задачу не называли «креативной».

Во-вторых, у нас все плохо с **устойчивостью по предсказуемости**. Представьте, вы собрались ехать на работу, садитесь в автомобиль, нажимаете на педаль газа. А автомобиль превращается в радужного единорога, пробегает вокруг вас четыре круга и улетает в сторону Ямайки. И это все без LSD.

Можно ли управлять таким автомобилем? Нет, **объектом, не обладающим устойчивостью по предсказуемости, управлять нельзя**.

Теперь вспомните какую-нибудь творческую, креативную личность. Которая может выдать что-то вау-супер, просто первое место на Каннском фестивале. А может впасть в депрессию, уехать в Кебезень и две недели пить арахиску, потому что поссорилась с подружкой. Много вы таким коллективом науправляете?



Понятно, что, если объект управления обладает устойчивостью по предсказуемости, эта устойчивость всегда лежит в каком-то диапазоне. Мы делаем прогноз в рамках этого диапазона, намечаем следующую контрольную точку. Затем собираемся в ней, проверяем, идем ли мы к нашей цели и, если нужно, — делаем управляющее воздействие. И так до следующей контрольной точки.

Например, на посадке самолета, на глиссаде, я мысленно выбираю точку метров до 10 от торца полосы и веду самолет туда. Но не стараюсь сразу выставить ручку управления самолетом (РУС) и ручку управления двигателем (РУД) так, чтобы можно было замереть и ничего не делать. Я смотрю **тенденции** — скорость-направление-высота. Может, ветерок где-то дунет, может — еще чего. И ручки управления постоянно в миллиметровом движении, **на упреждение**, — взял-отдал, взял-отдал.

То есть, чем меньше диапазон устойчивости по предсказуемости, тем больше мне нужно контрольных точек и управляющих воздействий. Чем больше у меня мастерства, тем больше я могу работать на упреждение, а не на компенсацию, больше следить за тенденциями, а не за ситуацией. И то, это если у меня цель перед глазами. А если прощелкал — уходи на второй круг: поздно будет пить зеленку, когда дырка в голове.

Помните, мы разбирали критерии качественного дизайна? Ведь там же куча потенциально-конфликтных требований. Например, чтобы получить качественный дизайн, нужно:

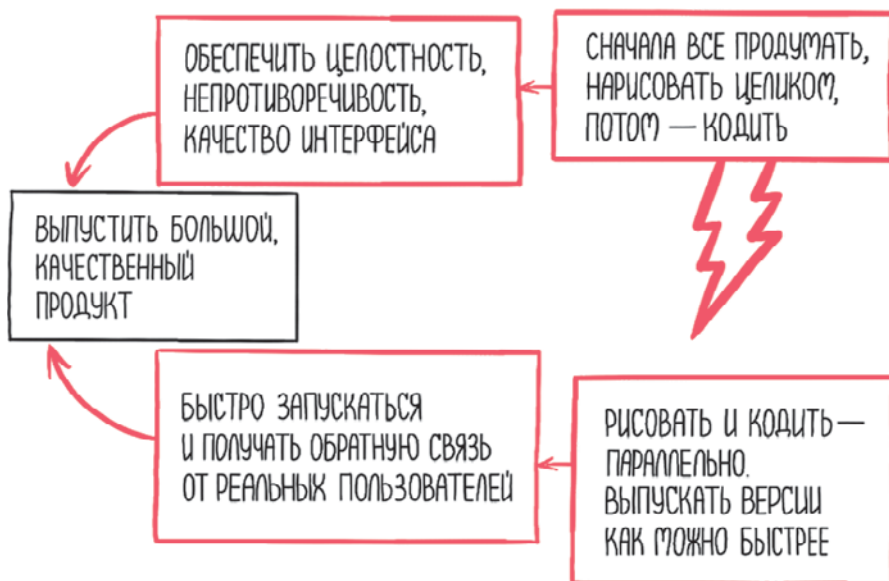
1. Выполнить креативную работу.
2. Обеспечить сроки, соответствие ТЗ и техническим требованиям верстальщиков.

Тут пока все ОК, конфликта нет. Однако! (я для наглядности построю грозовую тучку по методу ТОС Элияху Голдратта).

Чтобы выполнить креативную работу, нужно дать свободу творчества дизайнеру (если не давать — придется делать креативную работу за него, а это неопределенность, которая опять бьет по башке).

А чтобы контролировать сроки, соответствие ТЗ и технические требования, работу дизайнера надо регламентировать и загнать в рамки.

Вот тут уже конфликт есть.



Конфликт целостности интерфейса и итеративной разработки.

Но ведь совершенно понятно, что с этим делать, если мы знаем диапазон устойчивости по предсказуемости. Нужно:

1. Инициировать самодвижение дизайнера (дать ему все нужные материалы, провести брейншторм и т. д.).
2. Оставить его в покое на какое-то время (зависит от устойчивости по предсказуемости).
3. Контрольная точка, управляющее воздействие. Go to 2.

Это так и работает. Даже если вы просто берете дизайнера, объясняете задачу на салфетке, ждете готового результата, а потом мучаете его бесконечными правочками и «не то!». Никакой Америки тут не открыть.

Однако, и ради этого так подробно все на модельках и общих принципах расписывал, если нам нужна управляемость + креативность, важно:

- ▶ насколько у нас креативно-управляемый дизайнер, его технический уровень и область его ближайшего развития (бесполезно прыгать выше головы);
- ▶ как именно вы иницилируете самодвижение (начальный импульс);
- ▶ ваш опыт и область ближайшего развития: как расставляете контрольные точки и что именно в них делаете.

Слишком часто — это не решение. Креатив должен успеть образоваться (знаете поговорку «дуракам полдела не показывают»?). Слишком редко — это не управление. Это лотерея. Давайте попробуем нащупать этот баланс.

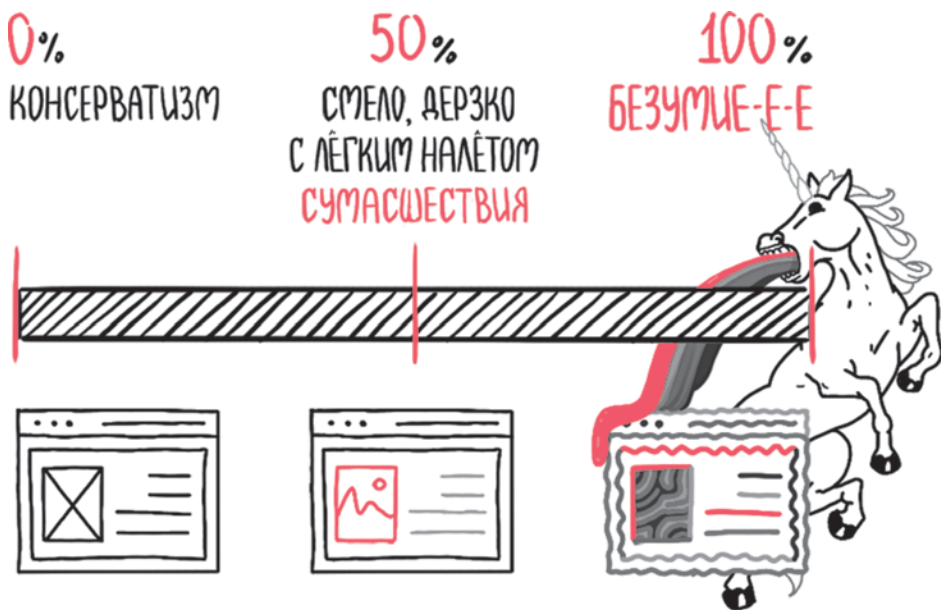
Кстати, запишите тезис себе на видное место:

Ценность сотрудника определяется кругом вопросов, которые он способен решить самостоятельно.

Это не только про дизайнеров. Это и про вас. По-моему, и зарплата должна от этого зависеть, и доносить это до всех нужно, и постепенно расширять эту зону компетенций.

6.6 БЕГУНОК КРЕАТИВНОСТИ

Давайте введем такое понятие как «бегунок креативности». Менеджеру, дизайнеру и арт-директору важно понимать, какой уровень креатива необходим и достаточен на проекте.



Определить это можно субъективно:

- ▶ 0% — консерватизм;
- ▶ 50% — смело, дерзко, с легким налетом сумасшествия;
- ▶ 100% — безумие.

В зависимости от того, как выкручен бегунок, надо по-разному строить рабочий процесс.

0% — типовые интерфейсы, B2B, личные кабинеты

Тут главная задача — не отвлекать пользователя и сэкономить бюджеты. Берем какой-то набор шаблонов, собираем проект из готовых элементов, как из конструктора. Кстати, сейчас полно таких шаблонов и библиотек компонентов (Bootstrap, Ulkit и т. д.) с разными темами, которые позволяют строить достойные интерфейсы без дизайнера. В тех случаях, когда под «креативностью» клиент подразумевает просто аккуратность интерфейсов — можно подумать про этот вариант.

Все как бы стандартно. Но удобно. Отличное решение для личных кабинетов, например.

Замечу, что, если требуется скомпоновать большое количество разрозненной информации так, чтобы было удобно пользоваться, придумать новые элементы управления, дашборды — задача сразу переходит в раздел креативных. Хотя может маскироваться под типовую (подумаешь, циферки и кнопочки на экран вывести).

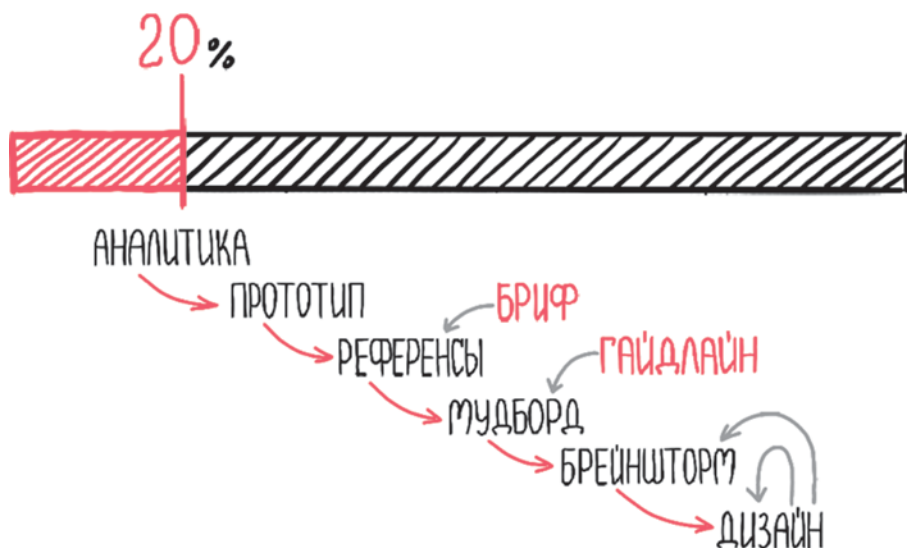
Например, интерфейсы в навигационных приборах самолета не выглядят креативными в том плане, как это понимается в дизайнерских конкурсах. Но их разработка — безумно сложная задача. Фестивальный дизайнер не справится, его туда просто не пустят. Да и большинство дизайнеров не справятся.

20–30% — мобильные приложения, e-commerce, корпоративный сегмент

В e-коммерсе, мобильных приложениях или в проектах корпоративного сегмента креативность 20–30% — оптимально. При первом знакомстве с проектом должно быть четко понятно, о чем он. И для чего пользователь сюда пришел. Какую задачу тут решают. Упор на UX/UI. Серьезная часть работы падает на аналитика. Тексты будут

важнее красоты. От дизайна нужны польза и функциональность, без отвлечений на «вау».

И сам рабочий процесс по дизайну получается управляемый, предсказуемый. Без большого количества регрессий.



Обратите внимание, при таком подходе сначала появляется прототип. Черно-белый интерактивный чертеж вашего проекта. Прототип нужен детальный. А затем, по сути, дизайнер его аккуратно красит и добавляет изюму (на основе референсов, мудборда и того, что придумали на брейншторме). Брифа может и не быть в явном виде — по мне, так это лишний, несчастный документ. Иногда нет фирменного стиля (с этим сложнее, но дальше посмотрим, как выкрутиться).

Здесь дизайнеры могут ходить с недовольным лицом. Мол, прототип и фирменный стиль слишком сильно нас ограничивают. Ну ничего страшного, такой уж тип проекта. Креативность там можно поискать в каких-то тонких деталях, мелочах и фишках. Микроанимациях или баннерах, например.

50–60% — промо-страницы и wow-эффекты

Если креативности нужно больше (бегунок в 50–60%), во-первых, у нас гарантированно появляются регрессии, возвраты на предыдущие этапы.



Во-вторых, нам понадобится минимум два брейншторма и карандашный скетч. То есть, сразу после агрегации требований рабочая группа придумывает концепцию, идею. Возможно, опираясь на мудборд, а возможно, генерируя мудборд на ходу. Идея может быть зафиксирована в карандашном скетче, и только затем перенесена на прототипы. Возможно, на прототипе просто оставлено место под блок, где написано «Креатив, см. скетч».

Такой подход предполагает довольно врубного Product Owner-а или заказчика, который способен включить воображалку, понять идею, изложенную на салфетке в виде скетча, и дать вменяемую обратную связь (именно идее). Ну, либо очень много работы может быть отправлено в мусорную корзину.

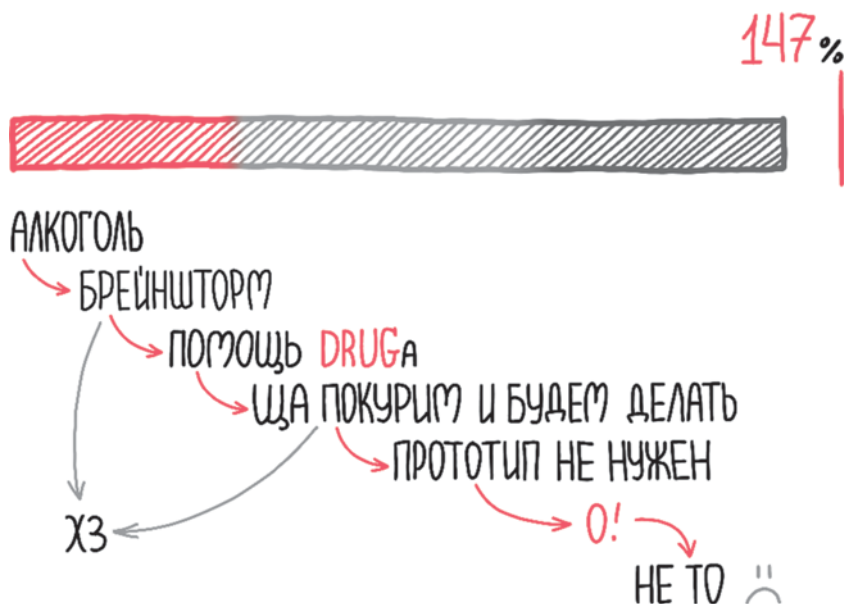
В этом, кстати, существенная разница в ментальности и подходах в американском и нашем дизайне. На западе превалирует практичность, и нет смысла двигаться к отрисовке на чистовую, пока сама идея не одобрена в виде эскиза. В наших, отечественных реалиях чуть ли ни кино снимаешь о том как проект будет работать — и то заказчики не врубаются.

Затем нужен еще один брейнштурм перед дизайном. Вполне возможно, что понадобится обновить мудборд. Параллельно с этим появляется или дополняется карта контента — тексты, которые нужно написать,

фото, которые нужно сделать, видео и т. д. Если на начальных этапах это могут быть «рыбные» тексты и картинки, по мере движения вперед буквы должны встать на свои места.

100–147% — прикольно, но ничего не понятно

Тут практически невозможно построить процесс и контролировать сроки-бюджеты. Надо ехать в Кебезень, за тем креативным парнем, возвращать его в муки творчества. И 99% его работы придется выкинуть. Регулярный бизнес на этом не построить.



Кстати, комфортный уровень креативности для заказчика, как правило, ниже, чем он сам думает и говорит.

Креативный креатив по креативной цене

Как-то мы делали проект по доставке питьевой воды в солнечном Баку. Основное преимущество — скорость доставки. Заказчик попросил максимум креатива, мол, деньги не проблема.

Окей, сели думать. Изучили доступную информацию. Придумали 3 идеи: с бегунком креативности на 30%, 50% и 147%. Расскажу про самую трудоемкую.

Оказалось, через пару месяцев в Баку планировали впервые проводить соревнования Формулы-1. Трасса проходила прямо посреди города. Как мы потом узнали, богатые люди специально скупали квартиры с видом на трассу. Цены взлетели в космос.

До соревнований еще пара месяцев. Идея созрела такая: берем дрон, снимаем гонки с высоты птичьего полета. Так, чтобы была панорама Баку и несущиеся болиды, с разных ракурсов. Затем снимаем пару десятков видео, как в болиды ставят бутылку воды (на место пилота) и как вытаскивают.

Дальше все просто: человек заказывает воду на сайте, мы с помощью видео и карт делаем имитацию, как будто ему болидом воду доставляют. Пилоты-водовозы. Все привязываем к GPS-трекерам реальных машин-доставщиков, чтобы видеонарезка играла ровно столько, сколько идет реальная доставка. Идет обратный отсчет, сколько минут-секунд осталось до доставки, болид загружается, едет, все видно по карте. Знакомые пейзажи.

Понятно, что смотреть это вряд ли до конца кто-то будет больше одного раза, но у конкурентов точно такого нет! Короче, весело и организационно-технически безумно сложно. Но ведь «деньги не проблема».

Излагаем идею. Заказчик загорается. Идет выяснять, как организовать съемки с дрона. Оказывается — никак. Все полеты будут запрещены, а снимать имеет право только одна компания. Видео у нее потом можно будет купить, но стоимость будет раза в три превышать стоимость разработки проекта.

«Ну так давайте покупать, деньги же не проблема?» — делая наивный вид, говорим мы. «Да», — говорит заказчик. — «Деньги не проблема. **НО НЕ ТАКИЕ ЖЕ!**»

Взяли вторую идею, получилось как надо.

На самом деле мы не сильно-то и рассчитывали, что будет выбран «безумный креатив». Ставки были на второй вариант. Но психологически действовали грамотно: не предлагать вариант 147% в таких ситуациях нельзя, вызывает конфликты. Держите в запасе и варианты попроще. Те, на которые на самом деле готов клиент. **Итак, комфортный уровень креативности для заказчика, как правило, ниже, чем он сам думает и говорит.**

Но вернемся к процессам.

6.7 ЭТАПЫ ЛИНЕЙНОГО ДИЗАЙН-ПРОЦЕССА

Мы пока разбираем тривиальный случай, когда на проекте один дизайнер, без тараканов, никаких параллельных работ не ведется. Сначала нарисовали концепцию. Утвердили. Потом на ее основе уже делаем остальные экраны. Так же, последовательно, не распределяем на других дизайнеров и не ведем верстку и программирование параллельно.

Так же не разводим штат лишних людей. Если нет у нас арт-директора — пусть руководитель проекта выполняет его задачи. Нет аналитика — аналитим за него. Вообще, привыкайте к реалиям профессии:

Руководитель проекта делает все то, что надо делать, но не делает никто другой.

Скуотища, короче. Потом уже научимся делегировать часть функций арт-директору, параллелить работу на дизайнеров и программистов, а также делить на итерации.

Как мы уже с вами знаем, в зависимости от бегунка креативности процессы строятся несколько по-разному. Более того, процесс нелинеен, неизбежны регрессии: возвраты на предыдущие этапы. Но этапы процесса хорошо очерчиваются.

6.7.1 АНАЛИТИКА

По-хорошему, аналитика должна быть проведена примерно в том виде, как мы это обсудили в главе 4. По-плохому — хватит с дизайнера перечня экранов и их состава. То, что в агрегации требований мы вынесли во вкладку «Структура».

Но это прямо по-плохому. Аналитика — это входные данные для дизайнера. Да, для погружения в задачу ему зачастую необходимо познакомиться с целевой аудиторией, бизнес-задачами проекта, рыночным анализом. Понимая глобальный замысел, он будет принимать лучшие микрорешения.

6.7.1.1 Прототипирование. Подари ему салфетку!

Входные артефакты: агрегация требований.

Возможные дополнительные входные артефакты: салфеточные прототипы, мудборды с примерами интерфейсов.

Выходные: прототип.

Цель: удобно скомпоновать информацию и интерфейсы, продумать юзабилити.

Инструментами прототипирования должны уметь пользоваться и аналитики, и дизайнеры, и руководители проектов. Благо, они довольно просты в освоении. Не видел никого, кто не осилил бы прототипирование в Figma или Moqups дольше, чем за несколько часов (Axure, конечно, я тут безбожно слил).

Для дизайнера будет полезным погрузиться в проект именно с прототипирования. Но есть нюансы, где от дизайнера на этом этапе больше вреда, чем пользы.

Аналитик рисует прототип		Дизайнер рисует прототип	
 Плюсы	 Минусы	 Плюсы	 Минусы
Будет точнее учтена структура и функциональные требования.	Функциональное решение вместо эстетического. Компенсация: 1) Смириться 2) Тестирование прототипов	Дизайнер заранее погружается в проект.	Высокая вероятность пропустить функциональные требования (как по творческой невнимательности, так и по профанации: «Да нафига так сложно?») Компенсация: 1) Смириться 2) Тестирование прототипов аналитиком

Аналитик рисует прототип		Дизайнер рисует прототип	
👍 Плюсы	🗨 Минусы	👍 Плюсы	🗨 Минусы
		Прототипы получаются более няшными.	Дизайнер может уйти в проработку деталей и нарисовать типмакет вместо прототипа. Излишние затраты. Эффект утенка (о нем чуть ниже).



Большинство дизайнеров плохо работают с текстовыми документами. Особенно, где много букв, цифр, фактов. Где пункты и подпункты. Сорри, если вы дизайнер, к вам это, конечно, не относится. Я про других 70–80% ребят из профессии.

Вообще, это их головняк. Но по итогу — ваш. Как решать?

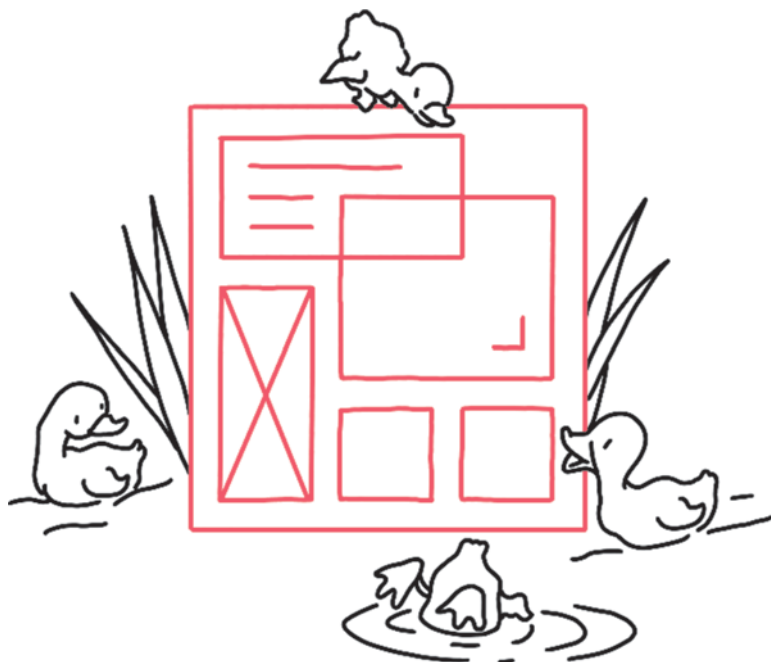
1. Проще прокачать аналитика рисовать удобные прототипы, чем дизайнера связывать большим количеством мелкодисперсной информации. Проектировать авионику дизайнерами — гиблая затея. Кого выбрать на конкретный проект — определите опытным путем.
2. Если нужно проектировать много экранов — работаем итеративно. Даем пачку в один-три экрана, в зависимости от сложности (ограничиваем Work In Progress).

3. Перед стартом очередной пачки даем время почитать структуру экранов, посмотреть референсы и сформулировать вопросы («полученное задание должно быть проанализировано», помните, да?).
4. Обсудить кратенько голосом. Разобрать вопросы. Почиркать на салфетке сложные места. Салфетку — подарить исполнителю.
5. Попросить красить светофором в исходных постановках готовое. (Белое — еще не трогал; Зеленое — готово; Желтое — сомневается или сделал как-то по-своему; Красным — покрасите вы, если будут косяки).
6. Планировать время на приемку и обратную связь. По возможности не давать следующую пачку, пока вас не устраивает предыдущая.

Кстати, примерно так же строится работа по дизайну внутренних экранов.

6.7.1.2 Эффект утенка

Утята не особо заморачиваются: кого первого в жизни увидели, тот и мама. И ничто, даже разумные доводы и демонстрация настоящей мамы-утки, не переубедит их разлюбить первого встречного.



В разработке похожая любовь наблюдается у заказчиков к прототипу. Могут начаться игры со шрифтами на чертежах. А могут продолжиться: встретившись с готовым макетом, заказчик попросит сделать шрифт (размеры блоков, отступов и т. д.), как в прототипе. Пообвыкся.

Эффект утенка провоцирует излишняя полировка графической части прототипа.

Готовьте схематичные прототипы, обязательно черно-белые, не вставляйте реальный фотоконтент. А вот реальные тексты (вместо Lorem ipsum), данные, цифры, суммы, названия товаров, параметры, названия колонок, фильтров, кнопок и вкладок — наоборот, хорошо!

Помните о цели прототипа: удобно скомпоновать собранную аналитиком информацию, акцентировать важное, спрятать второстепенное, продумать юзабилити, навигацию и пути пользователя.

На этом и сосредоточьтесь.

6.7.2 СБОР РЕФЕРЕНСОВ. ТРЕНДВОТЧИНГ

Референсы — примеры сайтов, приложений, интерфейсов, графических решений и всякие картинки, которые нужны для выработки правильной стилистики и настроения проекта.

У нас три ключевых источника референсов.

Аналитик. Ожидаем от него: сайты и интерфейсы конкурентов, зарубежных аналогов в данной тематике, разбор трендов отрасли, ссылки и примеры из смежных отраслей.

Дизайнер, арт-директор и вы лично. Очень важна насмотренность. Ваша личная и вашей команды. Дизайнеры вообще должны пропускать через себя десятки разных работ в день и следить, куда ветер дует (он, кстати, не очень-то дует). Для этого даже слово придумали: trend watching — буквально «наблюдение за трендами».

Заведите хорошую традицию: пусть кто-то из членов вашей команды всю неделю поставляет сайты или мобильные приложения по заданной тематике. Три-пять штук в день. Смотрите их вместе, например, с телевизора на утренних планерках. Разбирайте, критикуйте, изучайте фишки. Это обязательно должны быть Site Of The Day с AWWWARDS. Разберите узкоспециализированные, скучные темы. Например, «вывод ошибок в формах», «**контролы** выбора цветов товаров» или «**процедуры checkout** в B2B». Их гораздо сложнее искать

(особенно всю неделю, три-пять штук в день), но в этом и смысл. Пока ищите, на такое посмотритесь...

Время от времени ищите такие штуки сами — запоминаться вам будут, в основном, те приемы, которые вы нашли и показали другим, а не то, что просто посмотрели с экрана. Короче, повышайте насмотренность — пусть это будет отдельная задача в вашем ежедневном планировщике.

Бизнес-заказчик. В заказной разработке это наиболее важный источник информации. Обязательно запрашивайте и добивайтесь того, чтобы у вас были ссылки от него. Есть несколько классических ловушек.

- «Мне ничего не нравится», — говорит заказчик. Вариант: «Мы вас ничем не ограничиваем». Скорее всего, у вас будут очень большие проблемы со сдачей дизайна. Там тоже будет «ничего не нравится». А количество правок тоже будет «неограниченным».

Иногда за этим стоит страх и нежелание брать на себя даже частичную ответственность. Иногда — лень искать и думать. Иногда — аллергия на проекты конкурентов. Чисто потому, что это конкуренты.

Если поступил такой звоночек — нужно продлить картину мира заказчика в будущее. Расскажите технологию, как именно вы будете использовать эти референсы. Рассказывать нужно подробно, показывая примеры мудбордов других проектов и получившийся результат.

Если не выходит каменный цветок — попробуйте обойтись своими референсами. Обсудите их на мудборде. Как это сделать — разберем ниже.

Но если и из них «ничего не нравится» — двигаться дальше нельзя.

- Противоречивые референсы. Бывает, что список референсов большой, но выделить какую-то стилистику или тенденцию невозможно. Все противоречит всему. Тут и мелкое, и крупное, и яркое, и тусклое, и безумное, и скучное.

Так бывает, если референсы собираются у разных людей. Или, когда собираются референсы, оторванные от решаемой задачи (прислали все, что сочли креативным; не помогали решать задачу, а самоутверждались).

Очень важно поймать это противоречие, выявить и показать его заказчику. Разбить референсы на группы:

- основные: две-три стилистически близких, которые точно ОК;
- дополнительные: есть какие-то конкретные фишки, которые по возможности можно уложить в стилистику тех двух-трех основных референсов;
- остальное выкинуть, как бы жалко ни было. Не бывает автомобиля сразу и большого, и маленького, и гоночного, и внедорожника, и мощного, и экономичного, и грузового, и легкового, и розового, и зеленого.
- Референсы подобраны на вкус того человека, который не принимает решения по концепции. Причем, ему самому может казаться, что он «влияет». Но по факту просто «передает». В итоге сразу задано неверное направление в сторону долгоиграющего геморроя.

Поначалу все может идти хорошо. Вам кажется, что концепт восприняли благосклонно. «Передаст» — доволен, вносятся пара мелких правок. Маячит финишная прямая. Потом возникает какая-то странная заминка. Динамика и характер проекта резко меняются: то суета, то игнорирование. Это произошла вторая как с маком: показали концепт тому, кому надо, без вашего участия. И он сказал: «Фи, не угадали, ха-ха-ха!» Недогадливым и виноватым, естественно, назначат вас. Очень жаль!

6.7.3 МУДБОРД (ВИЗУАЛЬНЫЙ БРИФ, ДОСКА НАСТРОЕНИЯ)

Входные артефакты: агрегация требований, прототип, брендбук, структура главной страницы, референсы от бизнес-заказчика.

Возможные дополнительные входные артефакты: референсы конкурентов, зарубежные тренды, смежные отрасли.

Выходные: мудборд.

Цель: считать настроение, которое должен передавать будущий проект.

Мудборд (англ. Mood board — дословно «доска настроения») — условное полотно, на котором собраны различные визуальные элементы. Выполняется дизайнером. Планируйте порядка двух часов на эту операцию. Задачу на мудборд удобно проверять чек-листом.

Ниже — примерный чек-лист, который вы можете переработать под ваши задачи или сделать шаблонной задачей для разработки мудборда. Жирным выделены пункты, которые я считаю обязательными. Обратите внимание: изменение состава чек-листов и контроль за их соблюдением — это, по сути, управление через качество.

6.7.3.1 Чек-лист для составления мудборда (пример)

Состав мудборда:

- ▶ Шапка документа: название клиента + логотип разработчика.
- ▶ **🐼 Референсы, предоставленные клиентом.**
- ▶ **🐼 Референсы из агрегации требований.**
- ▶ **🐼 Сайты-референсы в разных графических стилях, предложенные дизайнером.**
- ▶ **🐼 Иконки в разных графических стилях (три-пять направлений).**
- ▶ **🐼 Подборки цветовых схем (три-пять направлений, если нет фирменного стиля).**
- ▶ **🐼 Подборки шрифтов для текстов и заголовков.**
- ▶ Элементы фирменного стиля и брендбука (опционально, если есть), на которые будете обязательно опираться.
- ▶ Элементы интерфейсов, форм, дашбордов.
- ▶ Ролики из Instagram с примерами интерфейсов, динамические заставки.
- ▶ Варианты стилистики иллюстраций/картинок/фотографий/3D-моделей. Графический контент.
- ▶ На референсы, которые лучше смотреть в отдельных окнах, проставлены ссылки в комментариях.

Процессные, дизайнеру:

- ▶ **🐼 Каждый пример в мудборде пронумерован, нумерация сквозная.**
- ▶ **🐼 Мудборд выполнен из тех элементов, стилистику которых действительно можно использовать в конечном продукте.** То есть, на выходе у нас получается не нарезка из разрозненных, оторванных от практического применения картинок, кочующих из проекта в проект, не топчик AWWWARDS, а набор образов, подобранных специально для этого проекта.

- ▶ Мудборд сдан арт-директору.
- ▶ Мудборд сдан руководителю проекта.

Процессные, менеджеру:

- ▶ Проведена демонстрация мудборда.
- ▶ Написан митинг-репорт.
- ▶ Однозначно ясно, что нравится.
- ▶ Однозначно ясно, в каком стиле и настроении должна строиться дальнейшая работа.

6.7.3.2 Примеры мудборда



Для примера я собрал несколько мудбордов в один pdf-файл. Скачать и посмотреть можно [здесь](#).

6.7.3.3 Типовые ошибки и косяки

1. В мудборд не включили какой-то пример, который ранее обсуждался с заказчиком (например, из агрегации требований или который прислал сам заказчик). Это грубый косяк, будет война...
2. В мудборд не включили свои примеры, которые раньше не видел заказчик. Это тоже косяк. Будет претензия: «А что мы по пять раз одно и то же обсуждаем».
3. В мудборде насыпана каша из элементов разных стилей, не относящихся к задаче. Например, собран самый топчик с AWWWARDS при склонности клиента к консервативным решениям. Или делается универсальный мудборд, примеры из которого кочуют из проекта в проект. Как потом использовать — непонятно. Это больше похоже на ошибку дизайнера (недальновидность). Но если ошибку разбирали, а тенденция «я мудборд сделал, а что там дальше — узнаем потом» не уходит — это уже грубый косяк. Акцентируйтесь на этом и здесь, и в чек-листе, поскольку это самый непрозрачный пункт.
4. Некоторые заказчики некорректно воспринимают слово «мудборд» (слышат ругательные коннотации). Используйте понятие «визуальный бриф» — им это проще. Просто наблюдение.

6.7.3.4 Демонстрация мудборда. Фиксация. Митинг-репорт

Мудборд нужно обсудить с бизнес-заказчиком. Типичный состав встречи: ответственные лица на стороне бизнес-заказчика (в правильном варианте — лицо, принимающее решения по дизайну, по-другому — ЛПР), руководитель проекта, дизайнер, арт-директор. Продолжительность — порядка часа. Обычно на это все идут очень охотно. Смотри картинки, говори свое мнение. А работа — идет...

Структура встречи:

1. Напоминаем цель и процедуру. Рассказываем, что будем смотреть мудборд (что это такое), и что задача — понять настроение, которое должен передавать будущий проект (а не скопировать что-то из этого). Предупреждаем, что будут уже известные примеры и новые.
2. Последовательно смотрим пронумерованные примеры (с общего экрана). Обсуждается каждый элемент. Обсуждайте детали и нюансы. Например, не весь сайт конкурента целиком, а конкретные блоки и формы. Если видите, что отторжение вызывает не сам референс, а просто что это конкурент — обсудите примеры из других сфер.
3. Собираем комментарии (нравится-не-нравится, про-нас-не-про-нас, скучно-креативно, в-самый-раз). Дизайнеры могут хорошо дополнить такую презентацию, комментируя стили (вроде «это баухаус, тут форма следует за функцией...») и приемы (вроде «Это цветовое кодирование, хотя акцентных цветов в этом примере слишком много...»).
4. Слушаем очень внимательно, что говорит заказчик, и не пытаемся его интерпретировать. Если показали пять примеров и про четыре из них он сказал «не нравится» — это не значит, что нравится пятый. Это значит только то, что вам не дали информации про пятый.
5. В итоге с такой встречи вы должны выйти с полным пониманием, куда выставить бегунок креативности и какое настроение нужно передать. Мы получаем годную обратную связь: по шрифтам, стилистике иконок, фактурам, общему настроению проекта.
6. Всенепременно фиксируем и отправляем митинг-репорт (что понравилось, что нет).
7. Типовые ошибки аналогичны тем, что допускаются на сборе референсов (разобрали с вами в отдельном параграфе). Самые грубые — не отработано «ничего не нравится» и отсутствие фиксации.

6.7.4 БРЕЙНШТОРМ

Если надо объяснять, то не надо объяснять.

З. Гиппиус

Мозговой шторм (брейншторм, мозговая атака) известен с 30-х годов прошлого века. Метод хорошо активизирует перебор вариантов. Придуман парнем из Нью-Йорка, Алексом Осборном. Его хорошо помотало: успел поработать на стройке, курьером в отеле, клерком, полицейским репортером, преподавателем, управляющим завода и в конце концов одним из основателей агентства BBDO.

Существует несколько десятков разновидностей этого метода. Однако все они не так изящны, как чистый мозговой шторм. Основная идея:

Процесс генерации идей должен быть четко отделен от процесса их оценки.

Иначе участники не решаются высказывать смелые, дерзкие идеи, опасаясь троллинга, критики и подколов.

С методом сначала связывались большие надежды. Однако попытки его применения в решении изобретательских и конструкторских задач не дали значимых успехов (как пишет Альтшуллер в ТРИЗ — теории решения изобретательских задач). Тем не менее, для рекламы и поиска креативных идей — самое то.

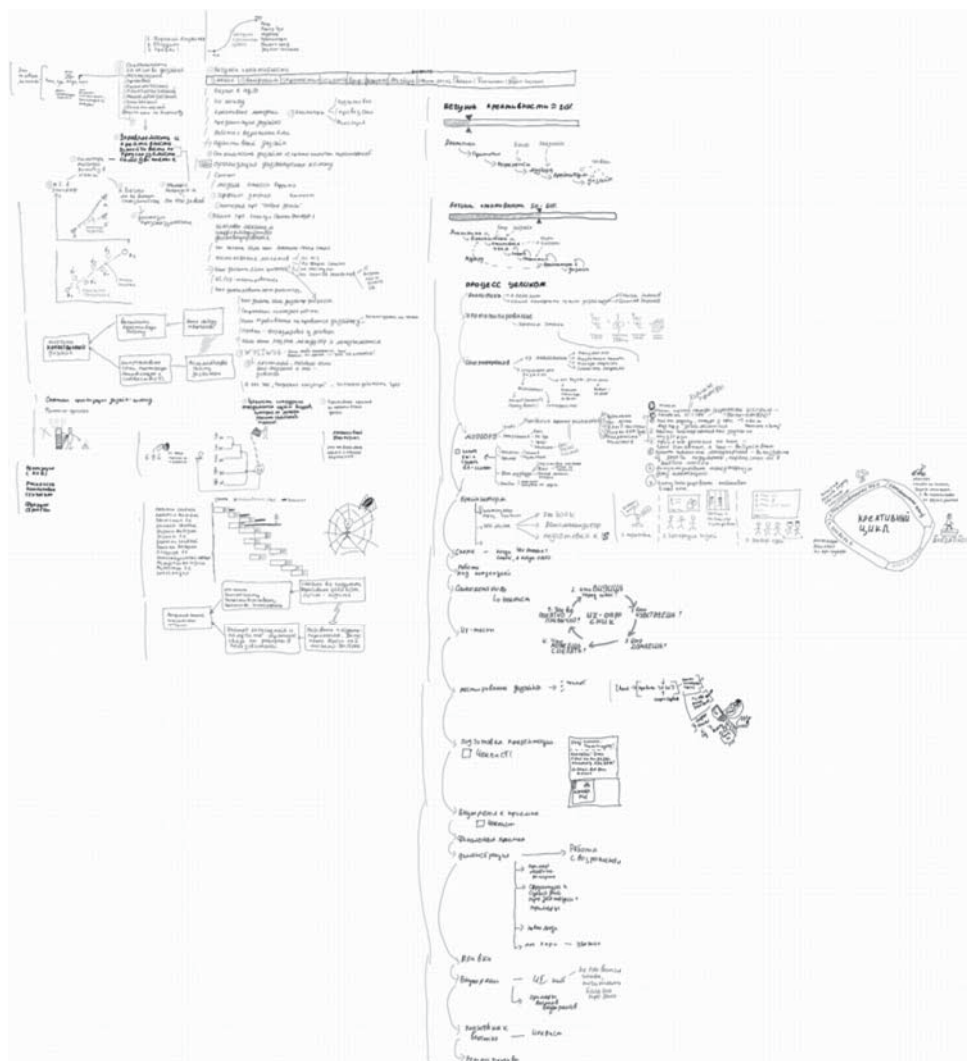


Основные виды:

1. **Индивидуальный мозговой шторм.** Хорош, когда вам нужно обдумать проблему самому.
2. **Письменный (брейнрайтинг).** Выписываем все подряд. Все связанные объекты, мысли, идеи, ассоциации. И строим связи между ними. Метод похож на интеллектуальные карты (или метод факт-карт, в терминах Курпатова). Когда на большой лист бумаги выписываем все интеллектуальные объекты, хоть как-то связанные с проблематикой. Специализированных программ по составлению мозговых карт тут лучше не использовать. Они слишком формальны. Хотя есть легкие приложения для iPad для вайтбордов. Мне нравится писать ручкой или стилусом; например, приложение Nebo — мой постоянный инструмент для письменных брейнштормов.
3. **Корабельный совет** — на флоте принято высказывать идеи от младшего чина к старшим. Аналогичный формат хорошо работает как при сборе идей, так и при сборе обратной связи. Нивелирует авторитет старших, дает высказываться младшим. Разбирать нужно сами идеи, а не их источник.
4. **Массовый мозговой шторм.** Классическая техника.
5. **Двойной мозговой шторм.** Фаза сбора идей и выбора решения отделяется во времени или даже проводится разными группами людей.
6. **Синектика.** Мозговой шторм в постоянной (сработавшейся) группе, где допустима критика, которая видоизменяет и развивает идеи.

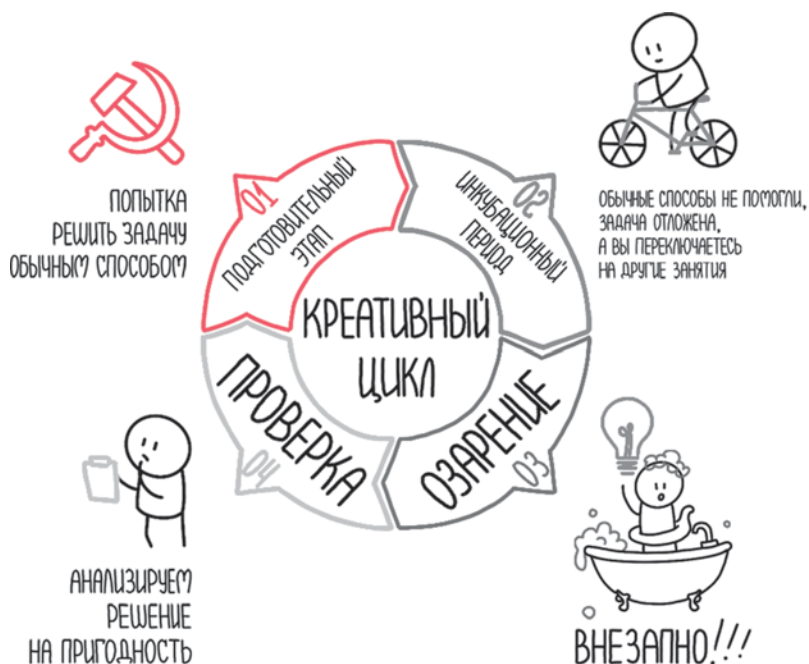
6.7.4.1 Подготовка к брейншторму

Это наиболее важно для заказной разработки, когда есть переключения между проектами и информация успеваешь выветриться. Но и в продуктовой разработке не повредит. Лучше вообще избегать переключений: провели демонстрацию мудборда, получили обратную связь, сразу пошли на брейншторм. Контекст свеж. Но увы, бывают паузы, и контекст выветривается за три-четыре дня. Чистые потери? Не факт.



Пример фрирайтинга: вся эта глава была
сначала просто картинками на iPad

Креативный цикл выглядит таким образом:



1. Подготовительный этап: попытка решить задачу обычным способом.
2. Инкубационный период: обычные способы не помогли, задача отложена, а вы переключились на другие занятия.
3. Озарение: ВНЕЗАПНО!
4. Проверка: анализируем решение на пригодность.

Таким образом, небольшой перерыв может породить озарения. А вот длительный — чистые потери.

В таких случаях мы ставим отдельную задачу на дизайнера и арт-директора по подготовке к брейншторму. За 30–60 минут до брейншторма, в зависимости от объема материалов. Входными материалам будут: агрегация требований, прототип, мудборд с зафиксированной обратной связью, контент, брендбук, фирменный стиль (если есть). Любые материалы, которые помогут дизайнеру понять концепцию заказчика. Например, фирменные буклеты, видео, изображения продукции.

В процессе подготовки мы ожидаем, что контекст проекта будет полностью восстановлен (чтобы не тратить на это групповое время брейн-

шторма). А также дизайнер и арт-директор принесут с собой несколько стартовых идей фишек, чем конкретно этот проект будет отличаться, выделяться или запоминаться. Такие идеи резко активизируют динамику брейншторма в самом начале. Это прямое применение **индивидуального мозгового штурма** и может включать в себя частицы **брейнрайтинга** (записываем все идеи на бумаге и строим связи между ними).

6.7.4.1.1 Сторифрейминг

Еще один подход загрузки проекта в мозг — сторифрейминг. Storyframing, или словесное прототипирование. Это обобщенное представление пользователей о том, какое поведение они ожидают от будущего программного продукта. Пошаговая история ментальной модели пользователей. Сначала я это, потом это, а оно мне — того! Тык-тык, и в дамки.

1. Налейте себе кофе. Откройте чистый лист текстового редактора. Отрадите словами все, что можете сказать о продукте или идее сайта. Как будто делитесь этим с хорошим другом.

Представляйте, что каждая новая мысль — это графический модуль, а предложение — элемент будущего дизайна.

Накидать все мысли в кучу обычно быстро и легко. Чашки кофе как раз хватит.

2. Теперь задачка усложняется. Глубоко вздохните, освежитесь, прежде чем вернуться к тексту второй раз. Сейчас нужно будет нещадно вырвать ненужное. Сделать рассказ лаконичным и емким, насколько возможно. Чтобы не зависнуть на этом слишком долго, можно опираться на такие вопросы:

- ▶ Что нужно знать пользователю об этом товаре или продукте?
- ▶ Как он попадет на страницу, и что уже ему может быть известно об этом?
- ▶ Какой информации ему не хватает, чтобы выполнить запланированный разработчиками сценарий поведения на сайте?

Убираем сомнения будущего посетителя страницы. Откидываем возможные вопросы.

3. Следующий пункт предполагает очередную издевку над текстом. Из оставшихся ранее кусков надо сложить историю. А лучше сразу несколько, комбинируя смысловые элементы в разном порядке.

4. Сторифреймы субъективны. Поделитесь своими наработками с коллегами. Возможно, они покрутят пальцем у виска, может, поднимут пальцы вверх или внесут ряд своих предложений относительно этого.

6.7.4.2 Генерация идей

Брейншторм разбит на две фазы: генерация идей и выбор идеи. Во время генерации идей предлагаемые идеи критиковать нельзя. Так же, как и обсуждать возможность их технической реализации.

Состав участников: дизайнер, арт-директор, руководитель проекта (модерирует). Аналитик и другие заинтересованные лица — могут приглашаться, но должны знать правила. Группы больше шести человек слишком дороги и шумны. Посторонний народ будет помехой.

Собираем всех в одной комнате. Нужны будут доска (или флипчарт), где можно рисовать + плазменная панель, подключенная к компьютеру, чтобы можно было быстро посмотреть какие-то материалы и ссылки. Мобильные телефоны оставляем за дверью. Проводить лучше стоя — выше динамика группы.

Продолжительность: от десяти минут (короткий шторм, обсудить идеи) до одного часа (что-то серьезное). Больше — непродуктивно.

Примерный сценарий для модератора:

1. Здороваемся. Напоминаем цель брейншторма (допустим, придумать фишку проекта) и задачу проекта. Кратко. Контекст в голове у всех есть. Если нет (пропустили фазу подготовки) — восстановите контекст:
 - a. Дать голосом вводную о проекте.
 - b. Обсудить «бегунок креативности». Даже если информации от заказчика нет, то у вас уже к моменту дизайна на основании общения с заказчиком должно сформироваться видение, к чему он готов.
 - c. Пройтись по референсам.
2. Собираем и фиксируем идеи. Либо маркером, либо буквами на ноутбуке, но так, чтобы все видели. Попробуйте оба варианта. Помогут стартовые идеи, с которыми пришли подготовленные дизайнеры. Рисуйте скетчи (например, для промо-блока) на листочках, маркерной доске, черновиках.

3. В самой острой фазе генерации идей вы можете словить, что они сыпятся по две-три штуки в минуту — только успевай записывать. Это хорошо. Гораздо хуже, когда все сидят тухлые, и нужно раскачивать.
4. На этой фазе модерируем критику. Не отбрасывать идеи только из-за того, что «мы так никогда не делали» или «это слишком сложно». Поощряем каждую идею. Даже если это шутка и нелепица. Не критиковать. НЕ КРИТИКОВАТЬ. НЕ КРИТИКОВАТЬ!!!

По согласованию с командой попробуйте другие креативные методики типа «метода шляп» и т. п. Экспериментируйте. Подборку креативных методик с фильтрами по времени и составу участников найдете по ссылке справа. Книжки по другим техникам поиска креативных решений вы найдете в конце главы.



Подборка креативных методик

6.7.4.3 Что делать с тухляком?

Да, бывает. В основном в несработавшихся командах или если кого-то «душного» позвали (выгнать и больше не звать).

Другие барьеры:

- ▶ Самоограничения: «Я так не умею», «Я так не нарисую», «Нам не хватит бюджета», «Это для меня слишком круто».
- ▶ Преклонение перед авторитетами: «Лебедев сказал, что так делать нельзя».
- ▶ Запреты, основанные на ложных аналогиях: «Это похоже на тот случай, когда все пошло через плохо».
- ▶ Общественный ограничитель: «Нельзя такое использовать — это сексизм».
- ▶ Стереотипы.

Для начала попробуйте раскачать группу на эмоциональном уровне. В списке литературы есть несколько старых книг по управлению динамикой группы и поднятию групповой энергетики. Берите на вооружение.

Допустим, тухляк не эмоциональный. Ищем, что мешает. Проверочные вопросы:

Чтобы решить творческую задачу, надо:

1. Хотеть ее решить. Хотим?
2. Верить, что решение возможно. Верим?
3. Понять, что мешает решению. Поняли?
4. Увидеть в помехах путь к решению. Протрите очки :)

Проверьте, не было ли ошибки раньше (например, невнятная обратная связь на мудборде). Попробуйте ее устранить, компенсировать или явно обозначить: «Да, вот есть такая печалька — клиент просит показать видимость крутого продукта, не имея крутого продукта и желательно его не показывая. Давайте подумаем, как выкрутиться на этот раз».

Что помогает раскатать вялый брейншторм, или какие качества модератора нужно в себе развивать:

- ▶ Ассоциативное мышление. Неформальная логика. Гибкость мышления. Дивергентное мышление (отход от основного направления). Конвергентное мышление (сосредоточенность на цели, сужение спектра возможностей).
- ▶ Визуализации, быстрые чертежи, схемы и другие символы.
- ▶ Умение задавать вопросы, проникающие в суть вещей (например, убийственный вопрос для любого маркетолога: «Как можно в трех-пяти словах сформулировать ваше главное конкурентное преимущество для шапки сайта?» — осторожнее с этим вопросом, в меня за него телефоном кидались).
- ▶ Умение наблюдать и экспериментировать. Внимание к деталям.

Итак, не тухлите сами и раскачивайте энергию группы. Хвалите больше и чаще.

6.7.4.4 Выбор решений

Когда идей достаточно, настала пора их просеять. Потому что в работу пойдут не все решения: некоторые могут быть альтернативой. Некоторые будут отброшены совсем. Чем руководствуемся:

1. Соответствие условиям задачи (агрегации требований, бегунку креативности, функциональным требованиям и т. д.).
2. Соответствие внешним условиям (законодательство, техническая выполнимость).

3. Практическая применимость (юзабилити, эргономика, пути пользователей).
4. Экономичность (хватит ли бюджетов и времени, можем ли добавить?).
5. Точность, простота, функциональность. Красивые решения всегда функциональны.
6. Интуиция, здравый смысл.
7. Доверие внутри команды. Желание сделать что-то более крутое, чем раньше.

При выборе решений хорошо работает метод «корабельный совет» из § 6.7.4. Хотя и дебаты, и споры — это нормально, но старайтесь вести обсуждение конструктивно. Модерируем. Помним, что наша цель — сделать офигенно! Остальное — вторично.

6.7.4.5 Фиксация решений и следующих шагов

По итогу брейншторма нам нужно:

1. Сформулировать фишку (прямо проговорите: «В этом проекте фишка заключается в ...»).
2. Придумать анимацию (какой элемент анимируется и как именно).
3. Зафиксировать, с какого референса и на что вы будете опираться. Какие приемы будете использовать.
4. Доску с брейншторма лучше сфотографировать и скинуть в групповой чат.
5. Обсудить, сколько времени тратим «на попробовать», получится ли реализовать ту или иную идею.
6. Определиться, что из того, что обсудили, можно взять на альтернативу.
7. Определите следующие шаги и следующую контрольную точку:
 - a. Рисуем скетч-черновик?
 - b. Рисуем на чистовую?
 - c. Рисуем методом прогрессивного джипега (подробнее в параграфе 6.7.6) или прорабатываем до упора какой-то один блок?
 - d. Рисуем несколько скетчей от руки и идем показывать их клиенту?
 - e. Идем добывать контент?

- f. Идем шерстить фотостоки для подбора иллюстраций?
- g. Идем искать иллюстратора в нужном стиле?
- h. И т. д. — четко понимайте следующий шаг.

Результаты брейншторма скидываем в групповой чат проекта и в постановку задачи дизайнеру.

Если все прозрачно и осталось только фигачить — получите у дизайнера декомпозицию работы с оценкой времени на каждый блок. Запланируйте ежедневный арт-дирекшн (стендапы). Иногда для малоопытных дизайнеров нужно две контрольные точки в день. Больше — нет.

6.7.5 СКЕТЧ



Карандашный скетч от руки. Нужен для обсуждения идеи: или когда запутались, и ничего не понятно, или для выбора между несколькими альтернативами. Трудоемкость: 5–20 минут.

Напомню, метод предполагает довольно врубного заказчика/Product Owner-а, способного понять идею на салфетке, совместить ее с референсами на мудборде и представить итоговый результат.

Однако даже самому тугодуму из скетча должно быть однозначно понятно, что это промежуточная картинка, а не финальный результат. Линии должны быть карандашные. Иконки и изображения — абстрактные.

6.7.5.1 Что должно быть вначале: скетч, мудборд, прототип или что?

Индивидуально.

Мы много экспериментировали с этим. Мудборд, похоже, лучше делать ДО прототипа. И, возможно, повторно делать его еще и перед дизайном.

- ▶ **Бегунок креативности < 20%:** В проектах типа e-commerce, где карандашная концепция неуместна, начинайте с прототипа.
- ▶ **Бегунок креативности > 50%:** На проектах, которые требуют креатива, делать мудборд, проводить брейншторм, придумывать идею и рисовать скетч нужно до того, как начнем делать прототип. Иначе у заказчика возникает эффект утенка, и вы идете красить прототип. А все ваши креативные идеи в лучшем случае сводятся к обсуждению фотки на слайдере.

Креативные идеи неплохо бы согласовать в карандаше (на скетче) с клиентом, и потом уже начинать делать прототип. Не надо извращаться и пытаться повторить карандашную идею в прототипах Ахиге. Достаточно будет нарисовать блок с надписью «креатив» или вставить в прототип кусок карандашного рисунка (справедливо только в случаях, если креативная идея уже придумана, а не для отмазки и откладывания головняка на потом).

6.7.6 РАБОТА НАД КОНЦЕПЦИЕЙ

В главе 5.5 мы говорили про самодвижение и устойчивость по предсказуемости. Так вот. К этому моменту самодвижение уже должно быть организовано. Дизайнер откусил кусок. Осталось только жевать.

Используйте метод прогрессивного джипега Артемия Лебедева. В любую секунду любой проект готов на 100%, хотя проработанность может быть и на 4%.



Метод прогрессивного джипега

Сначала накидайте структуру, потом проработайте детали. Не надо сразу полировать пиксели. Я встречал дизайнеров, распыляющихся в разные стороны: то один кусок поделают, то другой. Потом все выкинут и начнут перерисовывать с нуля. Потом полдня просидят над иконкой гамбургера в меню. Их можно попробовать скоординировать: выдать четкий список блоков, попросить оценку времени по каждому и показать как работать по прогрессивному джипегу. Но в целом ~~вн~~ из профессии! такие ребята слишком неэффективны, медлительны и энергозатратны, чтобы иметь с ними дело. Экологичнее иметь дело с людьми, с которыми можно обсудить правила игры и оставить их делать свое дело. А потом прийти в оговоренное время и увидеть что-то предсказуемое, а не мучать друг друга бесконечным контролем. Исключение можно сделать для джунов, пока учатся.

1. Договоритесь о контрольных точках. Стендапа на пять-семь минут один раз в день в большинстве случаев достаточно. Опционально может потребоваться вторая контрольная точка за день. Технику проведения стендапов можно взять из главы 3.1.
2. Уже знакомый нам принцип WYSIWYG (***What You See Is What You Get***). Если кажется, что получается дрянь — тебе не кажется. Если видите, что получается «не очень» — вмешайтесь. Используйте техники из инфографики «Что делать, если исполнитель упоролся» (ищите в списке литературы в конце главы). Привлекайте арт-директора на помощь. Он поможет идеями и заметит технические ошибки.
3. Подбадривайте дизайнера. Стимулируйте самоконтроль и желание сделать хо-ро-шо.
4. Если есть сомнения или альтернатива — просите сделать оба варианта. Экспериментируйте. Выбирайте уже из них. Требуйте сохранения промежуточных вариантов — они понадобятся при сдаче проекта, подготовке презентаций или если упретесь в тупик.
5. Принимайте, только если сделано хорошо, и вы сами довольны. Вы не сможете защитить макет, если он вам не нравится. Мужики не могут драться, если они не правы. При этом должно нравиться именно вам. Арт-директор и дизайнер с этим могут быть не вполне согласны, но у них могут быть другие штуки в приоритете: как это будет воспринято дизайн-сообществом, как это будет смотреться в личном портфолио, дадут ли ленточку AWWWARDS, достаточно

ли минимализма и т. д. В дизайне много субъективного, и не факт, что то, что вам нравится, — это лучшее решение. Но в любом случае у вас больше контекста по проекту, и ответственность за него тоже на вас.

Принимайте, только если это офигенно и вы довольны.

6.7.7 САМОКОНТРОЛЬ

Предлагаю внедрить и использовать две техники самоконтроля дизайнеров.

1. Покраска постановки — светофором. Про этот метод мы уже говорили в параграфе 5.8.1. Во-первых, метод позволяет дизайнеру проверить и визуально убедиться, что не пропущен никакой из блоков. Во-вторых, покрашенные постановки позволяют вам проверить сам факт самопроверки (не берем тут случай, что вас намеренно обманывают — с таким работать не стоит вовсе).
2. Чек-листы к задачам на дизайн. В них храним общие требования к формату макетов, наименованию слоев, объектов, файлов и т. д. Чек-листы должны регулярно пересматриваться и обновляться. Это инструмент обеспечения качества.

6.7.7.1 Квантовый скачок. Стоп-стоп-стоп! Смирись

Если шаг за шагом формально расписать весь процесс дизайна, мы обнаружим в нем квантовый скачок. Вы заметили?

Квантовый скачок — это момент между тем, когда все-все-все уже обсуждено, запрототипировано, согласовано, пробрейнштормлено, и тем, когда дизайнер показывает миру свой первый (пусть даже еще и не полностью готовый) макет страницы.

Именно тут и происходят сюрпризы.

Все, что делается до этого, — это попытка прицелиться. Все, что после — попытка компенсировать недолет или перелет.

Казалось бы, стоит только навести точнее: провести больше исследований, выяснить требования лучше, сделать более въедливый бриф, опросить всех стейкхолдеров, согласовывать концепцию как можно

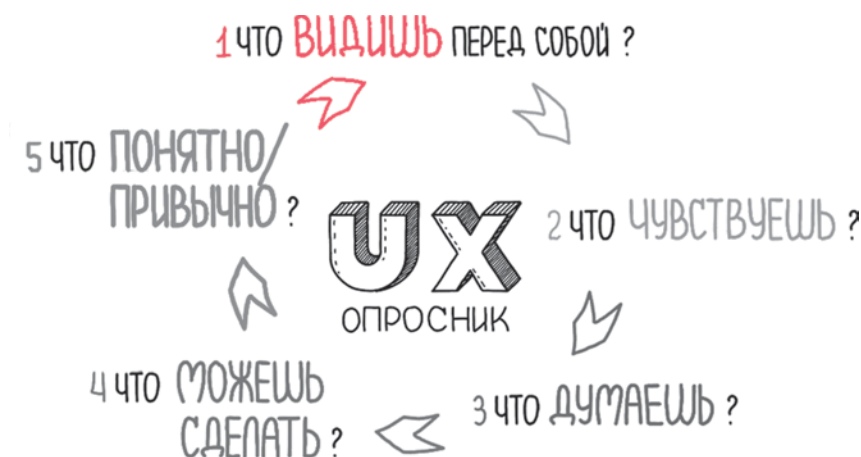
раньше, активнее, провести брейнштормы и применить креативные методики. Возьмите правильного дизайнера. Затем правильно постройте процесс презентации и сбора обратной связи — и все будет хорошо.

А вот и нет.

Исключить квантовый скачок невозможно. Можно улучшить устойчивость по предсказуемости. В определенном диапазоне. И научиться получать удовольствие от встречи с неожиданностью.

В общем, решение на букву «С» — Смирись.

6.7.8 UX-ТЕСТИРОВАНИЕ ДИЗАЙНА. 5 ВОПРОСОВ



Эти вопросы хорошо задавать и себе при приемке дизайна, и пользователям, проверяя конкретные сценарии работы системы. Или просто провести коридорное тестирование макетов. Слушаем внимательно, не перебивая, все, что говорит пользователь (или внутренний голос — помним, про «если кажется, то не кажется»). Задаем вопросы по кругу, для каждого сценария или экрана.

1. **Что видишь перед собой?** Интересна первая реакция: как быстро и точно пользователь разобрался. Если слушаем внутренний голосом — спросить: «Что я **на самом деле** вижу перед собой?»
2. **Что ты чувствуешь?** Обработка субъективного уровня. Особое внимание обращайте на «сложно», «запутанно», «лень думать» или какие-то экстремумы («этот красный разрывает мне глаза» или вроде того).
3. **Что думаешь?** Переключаемся в зону рационального.

4. Что можешь сделать? Убеждаемся, что все основные операции пользователь разглядел и знает, где куда дальше тыкать. И у него хватает всех нужных кнопочек и менюшек.

5. Тебе все понятно/привычно? Не изобрели ли мы неведому зверушку?

6.7.9 ТЕСТИРОВАНИЕ ДИЗАЙНА

Точно так же, как тестирование готовых приложений или сайтов, можно протестировать макеты. Точно так же это можно делегировать тестировщику.

Когда — очень опциональный этап. Имеет смысл на большом количестве страниц или на сложных страницах с большим количеством блоков.

На что тестировать (примерный чек-лист для QA):

Соответствие макетов постановкам:

- ▶ Текстовым. Использовать метод светофора.
- ▶ Прототипам.
- ▶ Агрегации требований.
- ▶ Брендбуку, фирменному стилю.
- ▶ Другим документам (перечислить в задаче).

Контент (фотографии, тексты, продукция, надписи на кнопках):

- ▶ Грамматика, орфография, пунктуация.
- ▶ Использован ли реальный контент (если имеется).
- ▶ Рыбный контент. Уместность, возможность заменить на реальный.
- ▶ UX-тексты. Ясность сообщений. Политкорректность.

Элементы управления:

- ▶ Наличие раскрытых **стейтов**.
- ▶ Стейтов при наведении.
- ▶ Заполненных стейтов.
- ▶ Вывод длинных ошибок.
- ▶ Отступы. Выравнивания. Соблюдения сеток. На всех макетах нужные элементы остаются на своих местах.
- ▶ Проверка чек-листа самоконтроля дизайнера на фактическое исполнение.
- ▶ UX-тестирование (пять вопросов).

Каждый пиксель должен быть поставлен обоснованно. В первую очередь с точки зрения удобства, бизнес-логики, интерфейсов. Во вторую — с эстетической.

По опыту, лучше всего QA справляется с поиском расхождений с постановками. Хуже всего с поиском ошибок:

1. в отступах (вылавливаются самые очевидные отклонения);
2. в тонкостях применения брендбука.

Чудес от таких тестов ждать не стоит. Делегировать приемку макетов QA и не вмешиваться не получится. Смотрите все сами.

6.7.10 ПОДГОТОВКА ПРЕЗЕНТАЦИИ



Этот этап имеет смысл:

1. в заказной разработке;
2. если макет вам нужно защищать перед кем-то еще, кто не погружен глубоко в ход работ по дизайну;
3. вам нужно защищать макет перед кем-то, кто пойдет его защищать еще перед кем-то.

Ни в коем случае нельзя отправлять макеты по почте.
Ни в коем случае нельзя отправлять макеты по почте.
Ни в коем случае нельзя отправлять макеты по почте.

И в чатики — тоже.

Состав и структура презентации может сильно варьироваться от ситуации и отношений между людьми. Общие рекомендации:

- ▶ Те ребята, кто смотрят презентацию, необязательно что-то помнят про целевые аудитории, прототипы, мудборды и т. д. Восстанавливайте контекст.
- ▶ У вас есть три секунды на восприятие. Потом интерес к слайду резко угасает.
- ▶ Ритм. Сложно акцентироваться на постоянно ярких или постоянно монотонных слайдах. Разбавляйте. Например, один яркий, три-пять спокойных.
- ▶ Яркие слайды должны взорвать голову. По возможности используйте анимацию.
- ▶ Опирайтесь на тренды и **бенчмаркинг**.
- ▶ Дайте альтернативу. Дайте сравнивать.
- ▶ Чем больше вы предусмотрите заранее, тем проще будет на защите. Развернутые состояния меню, карточки товара, мобильные версии — все, где чувствуете, что может возникнуть вопрос, — лучше добавить в презентацию, чем рассказывать на пальцах.

6.7.10.1 Чек-лист состава презентации (пример)

Примерный чек лист по подготовке презентаций, который можно взять за основу (у нас он толще, процентов на 40, но это наши тараканы).

- ▶ Вводный слайд. Наименование компании заказчика. Наименование и логотип разработчика. Год создания.
- ▶ Слайд с визуальным образом и наименованием целевых групп.
- ▶ Скриншот прототипа.
- ▶ Визуальный бриф.
- ▶ Мини-гайдлайн.
- ▶ Макет главной страницы в полный рост.
- ▶ Мобильный или адаптивный макет.
- ▶ Слайд со **стейтами меню** всех уровней (если предполагается концепцией). Развернутые меню. Меню мобильной версии. Фильтры.
- ▶ Механика анимации интерфейсов.
- ▶ Сравнение новой и старой версии.

6.7.11 ВНУТРЕННЯЯ ПРИЕМКА

Этот этап имеет смысл в заказной разработке, или если макет вам нужно защищать перед кем-то еще, и велика цена ошибки.

Цель: провести репетицию демонстрации макета финальному заказчику; найти недочеты и мелочи; подготовить руководителя проекта к возможным вопросам и возражениям; убедиться, что макет по креативным и эстетическим качествам соответствует или превосходит уровень макетов команды и требования заказчика.

Длительность: 30–60 минут.

Иницируется: руководителем проекта.

Состав участников: руководитель проекта, дизайнер, арт-директор — обязательно. Кроме того, соберите небольшую группу людей с незамыленным взглядом, которые могут вам оппонировать:

1. С точки зрения юзабилити, бизнес-требований и здравого смысла: руководитель смежного проекта (который не побоится сказать «все чушь, надо переделывать») или ваш руководитель (если он не является конечным заказчиком, но его шкура тоже на кону вашего проекта).
2. С технической точки зрения: программист, который потенциально будет заниматься фронтендом.
3. С точки зрения качества и юзабилити: аналитик, QA.
4. С эстетической точки зрения, вкусовщины, или отработки возможных провокаций: любые люди со своим мнением (например, PR-менеджер, который будет готовить кейс по этому проекту).

Группы в пять-семь человек более чем достаточно. Мероприятие по численности и длительности дорогое. Поэтому нет смысла звать туда людей, ценность и конкретика мнения которых близка к нулю. «Мне все понравилось», или «Ну, что-то не то», или фразы невпопад — серьезная заявка, чтобы на следующей внутренней приемке такого человека не было. Хотя попытки «прилипнуть» к приемке могут продолжаться: гораздо интереснее вместо своей работы смотреть еще несогласованный макет, высказывать свое мнение, влиять, но не отвечать.

Рекомендуемый способ:

1. Личная встреча участников у плазменной панели, либо интернет-конференция с демонстрацией экрана.
2. Руководитель проекта проводит презентацию макета. Дизайнер и арт-директор могут помогать, поясняя какие-то моменты. Или

даже проводить демо самостоятельно, в зависимости от их уровня (это хорошая точка роста для дизайнера).

3. Один из участников заполняет чек-лист встречи и фиксирует критику (правки).
4. После демонстрации каждый из участников должен высказаться. Используйте принцип корабельного совещания: собирайте обратную связь от менее опытных к более опытным. Сначала говорит стажер, в конце — ваш босс. Критика собирается по трем аспектам:
 - a. Качество самой работы (любые критерии из тех, что мы обсуждали в главе 6.4).
 - b. Качество презентации. Логичность, полнота и т. д.
 - c. Качество проведения презентации (менеджер мямлил, нес пургу или не знал материал — отправлять его на реальную защиту нельзя).
5. По итогу встречи участник, заполняющий чек-лист, отправляет список собранных замечаний всем остальным участникам. Участники могут уточнить какие-то пункты.
6. Руководитель проекта дорабатывает макет, презентацию (с дизайнером) или самого себя. Решение, какие именно доработки вносить в макет, принимает руководитель проекта (т. к. ему этот макет потом сдавать). В случае фиаско на внутренней приемке вполне возможны регрессии по всей цепочке процессов.

6.7.12 ФИНАЛЬНАЯ ПРИЕМКА

Финальное решение о том, что можно идти сдавать макет (все готово!), в зависимости от компании, могут принимать разные люди. Это может быть и арт-директор, и руководитель проекта, и (в случае заказной разработки) директор студии. Определите здесь четкие правила игры: за кем последнее слово.

6.7.13 ДЕМОНСТРАЦИЯ КОНЦЕПЦИИ

Этап больше характерен для заказной разработки. Очень похож на внутреннюю приемку, но неожиданностей может возникнуть на порядок больше.

Ответственный: менеджер проекта.

Может привлекать: дизайнера, арт-директора.

Способ: личная встреча или видеоконференция.

Продолжительность: 30–60 минут.

Макеты сдавайте только голосом, на видеоконференции или личной встрече. По электронной почте и чатиками макеты не отправляйте. Принимающая сторона не должна оставаться один на один с макетами, не имея возможности задать вопросы. Вопросы все равно появятся, а вместе с ними придумаются и ответы. Причем, самые невыгодные для вас. Сидят, смотрят картинки и всякую фигню про вас думают.

Старайтесь провести презентацию именно тому человеку, который принимает решение. Нет смысла делать хорошую презентацию менеджеру среднего звена, который пошлет потом письмо с макетом своему директору с припиской: «Вот, нарисовали. Покритикуйте!»

Примерный порядок:

1. Согласуйте время демонстрации с заказчиком. Сообщите продолжительность. Планируйте порядка часа.
2. Поставьте себе правильную цель: сдать качественно сделанную концепцию (а не «получить правки»). И да, работа должна быть действительно качественной. Помним принцип: принимать, только если это офигенно, и вы довольны.
3. Настройте клиента на позитивный лад и принятие макета. Волнительно. Интрига!
4. Демонстрируйте презентацию со своего экрана. Не отправляйте заранее ссылку или презентацию.
5. Восстановите контекст. Первыми кадрами должны идти уже знакомые клиенту артефакты. Кусочки прототипа, аналитики, мудборд и скетчи. Важно, чтобы клиент посмотрел и вспомнил принятый скетч. Это необходимо, чтобы у вас была какая-то опора.
6. Если был скетч — вкратце расскажите, какую именно концепцию собирались нарисовать («пересказать» скетч словами). Стараться использовать те же (один к одному) слова и выражения, которые клиент использовал во время брифинга. Акцентироваться на том, что клиент уже говорил эти слова и рассказать, как они соотносятся со скетчем и дизайном.
7. Расскажите правдивую историю работы над макетом. Обоснуйте свои идеи. Расскажите про ход ваших мыслей и о командной

работе над проектом. Дайте понять, что над макетом работало несколько человек, очень серьезно.

8. Расскажите про конфликт внутри команды (если он был) или про единодушие (если оно было). В общем, расскажите побольше правды и деталей о ходе работы.
9. Расскажите, какая фишка (уникальный элемент) использован на сайте и придуман специально для клиента.
10. Цифры. Факты. Связь вашей концепции с пользовательским опытом, путями пользователей. Обоснование UX, тесты. Расскажите и покажите, как все это связано и продумано. В конечном счете вы делаете продукт для людей, а не для рабочей группы.
11. Если в результате внутренних дебатов макетов получилось несколько — показывайте страницы/макеты по одному. Начиная с наименее удачного. Рассказывайте, что там будет и почему. Самый удачный макет показывайте последним. Расскажите, почему именно он. В заключении покажите все варианты рядом, на одном экране.
12. Если задумана анимация — покажите и обсудите, какая именно. Если анимация потребует увеличения бюджета — предложите варианты без увеличения бюджета и с увеличением.
13. Задавайте вопросы. Получайте обратную связь. Закончите разговор на позитиве.
14. Сразу напишите митинг-репорт. Если клиент просит время подумать — дайте ему осмыслить увиденное в одиночестве и принять решение. Не давите.

6.7.13.1 Что делать, если на демонстрацию приглашены новые важные люди

Ситуация. Вы работаете с руководителем проекта на стороне клиента. Демонстрация концепции запланирована. Руководитель проекта на стороне клиента пообещал провести эту встречу вместе с ЛПР-ами. Это очень хорошо.

Рекомендую провести еще одну, предварительную демонстрацию, с рабочей группой клиента, но без ЛПР-ов. Целей тут несколько:

1. У ЛПР-а может быть аллергия на какие-то дизайнерские ходы или контент. Вы про это не знаете. А рабочая группа на стороне клиента может быть в курсе. Пример: компания «Благодать» пишет

свое название с заглавной буквой Д посередине: «БлагоДать». Причем, не только в логотипе, но и в контенте. Такая пропущенная мелочь где-нибудь посреди макета может сильно раздражать во время демонстрации и осложнит вам жизнь. Рабочая группа на стороне клиента тут может хорошо подсказать.

2. Заручиться поддержкой рабочей группы. Палка о двух концах, конечно. С одной стороны, если вы все сделали правильно, рабочая группа будет на вашей стороне и помогать вам. С другой стороны — рабочей группе будет скучновато смотреть макеты второй раз. Будет неправильный настрой. Решение: либо на финальной презентации добавьте что-то, чего не видела рабочая группа, либо не показывайте рабочей группе часть презентации. В любом случае должно быть что-то новенькое.

По итогам такой встречи в макет могут быть внесены корректировки. На всякий случай напоминаю: ссылки на макеты не отправляем — начнется «коллеги, покритикуйте».

6.7.13.2 Что делать, если к ЛПР-у вам не попасть

Ну, во-первых, это очень плохо. Миллион причин, почему это плохо. Во-вторых, это почти всегда искусственная ситуация. Если цена вопроса высока (а при выборе концепции это почти всегда так), ЛПР-ы с удовольствием подключаются к таким встречам.

Искусственную ситуацию «не пущать» часто создает рабочая группа на стороне клиента. Мне много раз доводилось расфакапливать ситуации, решая вопрос с ЛПР-ом за три минуты о том, что на среднем уровне не могло сдвинуться с места неделями. «Где ж вы раньше-то были?» — говорят ЛПР-ы.

Приходится «продавать» такую встречу.

1. Для начала поговорите с рабочей группой клиента. Расскажите риски, что будет, если ЛПР-а не будет на встрече. Примеры аргументов:
 - a. Вообще это нормальная и правильная практика, что ЛПР смотрит концепции вместе с рабочей группой. Тиньков, например, так делал, глубоко погружаясь в свои продукты.
 - b. Без этого будет искажение информации, акцентов и обратной связи. Искажение порождает недопонимание. Непонимание рождает недоверие. Недоверие рождает факап.

- с. С нашей стороны будут дизайнер и арт-директор. Они смогут профессионально ответить на вопросы и обосновать решения. В противном случае ЛПР останется один на один со своими вопросами и придумает на них худшие ответы.
 - d. Мы знаем, как презентовать наш проект. У нас гораздо больше контекста, мы этим две недели жили. Сами вы такую же презентацию провести не сможете — просто не хватит инсайдерской информации.
 - e. Если на демке что-то пойдет не так, то виноваты будем мы. Если демку проводим не мы, значит вся ответственность на вас. Вы готовы ее принять?
2. Если ни в какую — подчеркнуто передайте ответственность за это решение — рабочей группе клиента. «Точно уверены, что берете на себя всю ответственность? Точно-точно? Прямо вот всю ответственность?» Вопрос, повторенный трижды, заставляет сильно задуматься. Но если вам твердо ответили три раза «да» — уж потом можно этим пользоваться на полную катушку.
3. Лучшее, что можно сделать, — записать видео с презентацией макета, так, как это рассказываете вы сами. Это может быть видеозапись с демонстрации макетов рабочей группе. Или просто отдельная видео-презентация. На втором месте — подготовить менеджера на стороне клиента проводить демо. Снабдите его фактами и аргументами. Худшее — махнуть рукой.

6.7.13.3 Если ЛПР убежал с демки

Это фиаско. Либо вы уж совсем не то нарисовали. Либо нарисовали то, что хотела рабочая группа (предварительно вам сказали «ВАУ»), но это совсем не то, что хотел ЛПР. Так бывает, когда менеджеры среднего звена воплощают свое видение, считая, что знают как надо. Но рабочая группа биться за вас не будет — козлом отпущения сделают вас. Либо такой неусидчивый ЛПР: то у него завод горит, то экстренно-забытое совещание, то срочные звонки прямо во время демки, то бабы рожают, то инопланетяне прилетели — водки просят.

Даже не знаю, что хуже.

6.7.14 ПРАВОЧКИ!

Ну куда без них. Тысячи мемасиков про это. Поиграть со шрифтами, вот это вот все...

У правочек очень высокие транзакционные издержки. По сути, каждая порция правок — это весь цикл разработки макета, от брейншторма до демонстрации, но в миниатюре. Именно это и больно.

Давайте договоримся, что правки у вас — конкретные, выполнимые и непротиворечивые. Если неконкретные, невыполнимые и противоречивые — это либо к психиатру, либо к главе «Работа с возражениями по дизайну».

1. Организуйте итеративную работу: собрали всю обратную связь; проверили на конкретность, выполнимость и отсутствие явных противоречий; выполнили; провели демо. Если обратная связь все еще поступает — не надо кидаться что-то переделывать. Пусть слегка отлежится.
2. В заказной разработке при фиксированных бюджетах ограничивают количество итераций (две-три), и все итерации проводят в рамках концепции. При работе по **Time & Material** или разработке in-house ограничителями будут время и бюджеты. В любом случае нужно двигаться в сторону выпуска проекта в свет.

Как-то к нам пришел запрос на отрисовку карточки товара для одного винного интернет-магазина. В смысле, алкашка, с миллионными оборотами. На стороне клиента управлял проектом талантливый парень, назовем его ПареньОтДизайна. Он хотел вникать во все и двигать-двигать-двигать пиксели. За деньги же отвечал очень прагматичный и бизнесовый дядька, с которым мы сразу нашли общий язык. Назовем его ПареньОтБизнеса.

Как оказалось, бесплатные концепции уже успели порисовать с десятков студий из top-a рейтингов. Была куча макетов. Но, по словам заказчика «все они не то».

Мы отказались работать без денег: работать бесплатно — грех! И выставили, как нам казалось, очень серьезный прайс. Клиент сначала отказался. Через полгода пришел и попросил-таки порисовать. Уже с деньгами. На тот момент количество макетов, которые были «не то» по одной странице у него превышало 20 вариантов. Мы законтрактованы и пошли работать. Выложились на 200%. «Да, это прорыв!» — сказал нам заказчик на демке.

А потом пошли правки. Порядка 500. По одной внутренней странице. И очень серьезный прайс превратился в смешной.

Принципиально ничего не менялось. Но пиксели двигались. В конце концов у нас все приняли и подписали акты. С комментарием «ну мы тут сами еще немного ваш макет покрутим». ПареньОтБизнеса понимал, что надо идти дальше. Но ПареньОтДизайна был слишком харизматичен и убедителен!

Прошел год. Несколько раз переговоры по проекту возобновлялись. Дело почти доходило до контракта на новые страницы и даже верстку. И каждый раз мы видели новые версии, слегка улучшенной страницы карточки товара или страниц чекаута. В конце концов всем это надоело, махнули рукой. А на сайте — все по-старому до сих пор.

Кажется, мы были единственными, кому заплатили за эту работу. Но дело не в этом. Если на стороне заказчика есть важная персона, влияющая на решения, но заинтересованная полировать микроны, а не запускать проект — в унитаз сливаются годы и миллионы рублей. У меня инженерное образование, я посчитал.

3. Если в процессе работы над правками вы понимаете, что просят сделать «не очень» — сделайте два варианта: так, как просят, и так, как было бы по красоте. Внимательно следите за дизайнерами: иногда им не нравится сам факт правок, и они делают так, чтобы доказать, что без правок было лучше: огрубляют, делают более гротескно, неаккуратно. Внутренняя мотивация — доказать, что правки — говно, а я — молодец, с первого раза все хорошо сделал. Лечится разговором.
4. Запутались — перештормите. Или обсудите правки голосом. Текст часто искажает смысл.
5. Очень много мелких правок? Сделайте дополнительный тест. Среди большого списка (из 30 пунктов, например), очень легко потерять пару мелочей. Для дизайнера это действительно может восприниматься как мелочь («Ну я же 99% сделал, подумаешь один пункт пропустил»). Для автора списка это повод к раздражению: я им пишу-пишу, а они такие невнимательные. В данном случае дизайнер неправ.
6. Доводка. Если в ходе работы выявили противоречие или невыполнимое требование, или просто возникла тенденция переставлять

пиксели с места на место — нужно садиться с автором правок и дизайнером за один экран и доводить макет в реальном времени.

Про критику Стива Джобса по поводу вида и характеристик продуктов Apple ходят интернет-легенды, и особенно канонической считается история с внешним видом калькулятора. Создание его внешнего вида Джобс поручил одному из самых юных сотрудников компании. Вскоре тот показал результат, на который получил резкий ответ: «Это только начало, но сейчас все выглядит отстойно. Цвет фона слишком темный, некоторые линии имеют неправильную толщину, а кнопки слишком большие» — припечатал его Стив. Сотрудник продолжил работу и показывал новые варианты практически ежедневно. И ежедневно получал новую пачку критических замечаний от Джобса.

Тогда, вместо очередной итерации, создатель изменил подход и сделал «Конструктор калькулятора для Стива Джобса». В нем можно было выбирать фон, размеры кнопок, толщину линий и все такое прочее. Джобс поковырялся в этом конструкторе десять минут и настроил параметры на свой вкус. Так и появился идеальный с его точки зрения калькулятор.

6.7.15 ДИЗАЙН-СПРИНТЫ

Дизайн-спринты все чаще и чаще применяют в компаниях при разработке программного обеспечения. Вы можете без проблем сделать все то же самое. Общая канва — та же, что и в классическом Scrum, который мы подробно разобрали в главе 3. Здесь я кратко перечислю некоторые отличия.

1. Планирование спринта занимает от полудня до дня. Здесь мы готовим и обсуждаем постановки, формируем задачу, проводим брейншторм, накидываем какие-то идеи. В общем, делаем все то, что нужно сделать до того, как начнем непосредственно рисовать.
2. Длительность спринта, как правило, не превышает неделю.

3. Нулевая итерация — это подготовка концепции. Когда концепция «устаканилась», можно выстроить стабильные спринты. Особенно хорошо это работает при проработке пользовательских историй на внутренних экранах.

6.7.16 ВНУТРЕННИЕ ЭКРАНЫ

Предполагаю, что концепцию мы прорабатываем **на самом сложном экране**. Это необязательно главная страница. Для магазина это вполне может быть карточка товара. Однако, если у нас есть концепция, есть прототипы, есть понимание, куда двигаться — почти всегда дальше идет спокойная, методичная, техническая работа. Которую просто надо сделать аккуратно. Исключения, естественно, бывают.

- ▶ Новые, почти самостоятельные разделы продукта. Например, добавляется личный кабинет.
- ▶ Технически сложные страницы. Попробуйте спроектировать и нарисовать что-то типа Google Calendar — воспользуйтесь от мелочей, деталей и частных случаев.

На простых страницах вместо брейншторма можно использовать короткое обсуждение, на 5–10 минут. Без обсуждения задачи голосом результат будет хуже на 20–30%. И будет больше переделок. На этом не сэкономить.

В начале работы нужно составить **таймлайн** отрисовки внутренних страниц. Пусть дизайнер самостоятельно оценит объем работ по каждому экрану. Один экран — один тикет. Можно использовать канбан-доску (Trello, Jira и т. д.). Но сгодится и файл со списком экранов в Google Docs. Если экраны очень сложные — декомпозируйте. Ежедневно проводите короткие стендапы (см. главу 3.1).

В заказной разработке запланируйте промежуточные демонстрации клиенту. Простые экраны лучше демонстрировать партиями, штук по пять. Это примерно на час. Сложные (объемные, с большим количеством элементов) экраны лучше сдавать по одному. Процедура аналогична демонстрации концепции (о ней мы говорили в 6.7.13), но чуть проще: в презентации достаточно оставить только новые экраны.

Не комплектуйте слишком больших партий: это приводит к потерям. Спринты по дизайну оптимальны длиной в неделю. Это не только мой

опыт, такая же рекомендация у Google Design Sprint. Берите классический Scrum, ничего замысловатого придумывать не надо. Кстати, это точка для распараллеливания работ по дизайну и верстке.

Все, что касается UX-тестирования, тестирования дизайна, скетчей и т. д. для концепций — применимо и для остальных экранов. Правки, исправления ошибок вносите как можно оперативнее. Иначе возникшая ошибка будет скопирована по десяткам макетов, и исправить ее будет уже крайне сложно. Особенно если не применяете компонентный подход.

6.7.16.1 UI-кит, компонентный подход и дизайн-системы

После отрисовки и согласования двух-трех внутренних экранов (один из них обычно — какая-то форма, второй — текстовая страница со всеми стилями, заголовками, отступами, списками и таблицами) можно готовить UI-кит. Это специальная страница, где мы собираем все элементы интерфейса. Ховеры, отступы, размеры заголовков, кнопки и другие элементы форм, фильтров, меню.

Собирать UI-кит с первой страницы смысла нет: нужно набрать критическую массу элементов и дать стилю устояться. Посмотреть, насколько удачно он работает на разных экранах. В общем, сначала преодолеваем порог чувствительности, потом готовим UI-кит.

Можно использовать компонентный подход таким образом, что изменения элемента в UI-ките приведет к его изменению на всех экранах. Подготовка хороших компонентов занимает время. Примерно +30% к трудоемкости. Годится для зрелых проектов с устоявшимися правилами применения интерфейсов. На старте подготовка полноценной дизайн-системы себя, скорее всего, не окупит. Однако если проект активно развивается, постоянно, спринт за спринтом добавляются новые экраны, и их уже десятки, а дизайнеров уже не один и не два — без компонентов и дизайн-системы будет тяжело.

6.7.16.2 Системные дизайнеры начинают и выигрывают

Это ребята, которые держат все по-полочкам:

- ▶ способны сразу грамотно организовывать слои и компоненты;
- ▶ видят сетку, а не расставляют элементы «на глазок»;

- ▶ используют компонентный подход;
- ▶ используют общую палитру для всех экранов и стили шрифтов, а не примерно пипеткой выбирают цвет;
- ▶ рисуют страницы последовательно (а не сегодня две этих, потом две тех, потом вернулся назад).

В разы продуктивнее творческого беспорядка. И творчества у них больше.

От несистемных, неаккуратных дизайнеров слишком много косяков, чтобы долго с ними возиться. Но пару вещей сделать можно:

1. Объясните, почему несистемный подход и мусор в макетах создают проблемы. Люди не дураки, и все прекрасно понимают.
2. Пару раз возьмите их на ретроспективу с frontend-разработчиками. Несистемный дизайнер должен узнать все о боли!
3. Посадите на несколько часов в пару с системным дизайнером. Пусть сидит, смотрит, наматывает на ус. Затем спросите, какие выводы он сделал. Если выводы правильные — хорошо. Если выводы: «Ну круто, но я такой человек, который сначала <отмазка>, а потом <что-то позитивное в свой адрес>», толк вряд ли будет.
4. Если выводы правильные — проследите, что дизайнер начал работать в новой парадигме. Со временем он оценит плюсы, и аккуратность может стать его привычкой.

6.7.16.3 Чек-лист подготовки UI-кита (пример)

Для всех интерактивных элементов отрисованы состояния по умолчанию и при наведении:

- ▶ Кнопки;
- ▶ Ссылки на внешние страницы;
- ▶ Ajax-ссылки;
- ▶ Пункты меню;
- ▶ Хлебные крошки.

Элементы для текстовых страницы и блоков.

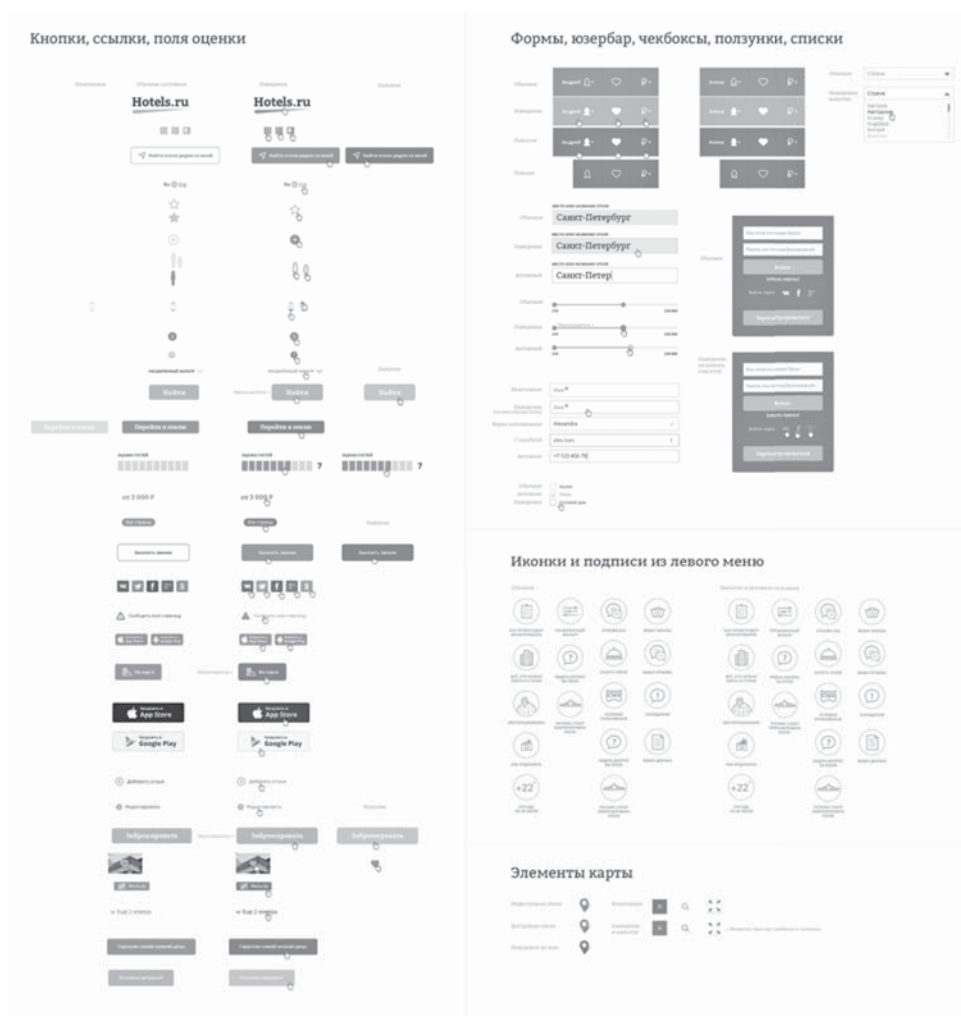
- ▶ Заголовки H1-H6, (размер шрифта, отступы) и основной текст (абзац), ,<i>, параграф.
- ▶ Маркированный список.

- ▶ Нумерованный список.
- ▶ Таблица.
- ▶ Цветовая палитра с подписанными **HEX-кодами** цветов.
- ▶ Слайдеры. Элементы переключения слайдов.

Формы и фильтры:

- ▶ Выбор времени: не выбрано, выбрано, состояние — **disabled**, открыт контрол выбора времени.
- ▶ Выбор даты: дата не выбрана, дата выбрана, поле состояние — **disabled**, открыт календарь для выбора даты.
- ▶ Выбор диапазона значений (ползунков с подписями границ от-до и текстовые **инпуты**): без наведения, наведение, **дизейбл**.
- ▶ Контрол выбора цвета в фильтре (при наличии такого в проекте).
- ▶ Иконка подсказки с **тултипом** (без наведения, с наведением).
- ▶ **Радиобаттон**, **чек-бокс**: не выбран, выбран, **ховер**, **дизейбл**.
- ▶ Переключатель (тумблер): без наведения включен, без наведения выключен, наведение, **дизейбл**.
- ▶ Выпадающий список с множественным выбором: показать все **стейты**, как и для диапазонов значений, **плюс стейт**, когда список развернут и выбрано несколько значений, когда свернут и выбрано несколько значений (с возможностью сброса каждого).
- ▶ Выпадающий список: не выбрано значение, выбрано, развернут с **ховером**, **ошибка валидации**, **дизейбл**.
- ▶ Поле ввода текста: пустое, с курсором внутри, с текстом внутри, **disabled**.
- ▶ Текстовая область (textarea) со скроллом: пустая, с текстом, **disabled**.
- ▶ Поле ввода пароля (с глазом): пустое, с введенным паролем, **ховер на глаз**, после клика на **глаз**, **disabled**.
- ▶ Ошибка валидации для поля. Предусмотрите длинные сообщения об ошибках (особенно во всплывающих окнах).
- ▶ **Стейт шапки** для авторизованного и неавторизованного пользователя для десктопа и адаптива.

Примеры UI-kit:



6.7.17 ПОДГОТОВКА МАКЕТОВ К ВЕРСТКЕ

Практика показывает, что на подготовку макетов к верстке, если их была пара дюжин или работа шла параллельно, нужна отдельная задача. Особенно если вносилось много правок. Какой бы ни был системный и аккуратный дизайнер — все равно он где-то действует «на глаз». А степень энтропии с правками будет расти. Нужно перепроверить выравнивания и наименования компонентов, артбордов, слоев и т. д.

Особенно много мороки с подготовкой иконок для мобильных приложений. Тут проблема может быть не только в дизайне, но и в конвертации и оптимизации: добавляются искажения и «мыло». Чек-лист по подготовке иконок можно найти по ссылке справа.



В заказной разработке есть смысл следить, какие моменты чаще всего упускают дизайнеры, и группировать их в чек-листы.

6.7.17.1 Где хранить чек-листы

Чек-листы на UI-киты, подготовку внутренних страниц, формы, а также тестирование макетов очень близки по составу. Держать их нужно в том месте, где:

1. Их точно увидит дизайнер, и во время работы они будут под рукой.
2. Их точно увидит руководитель проекта и сможет отследить состояние.

Напомню общий принцип из четвертой главы: держи документацию максимально близко к месту ее использования.

Я не рекомендую выносить чек-листы в отдельные файлы. За исключением, когда с этими файлами работаем как с распечатками — физический чек-лист с прокрашиванием пунктов.

Вынесите чек-листы в шаблонные задачи, если ваша система управления задачами умеет работать с шаблонами. Или сделайте шаблоны для макетов, содержащие чек-листы. Первый метод более удобен для менеджера. Второй — для дизайнера.

6.7.18 РЕТРОСПЕКТИВЫ

Реальность такова, что, когда твоя команда добивается успехов, ты тоже добиваешься успехов. Поэтому важно разбирать ситуации и помогать команде расти. Не только кодерам. Но и дизайнерам. Ретроспективы и разборы полетов необходимы как воздух.

Если дизайнеры ваши, «штатные», а не разовые фрилансеры на одну ночь — время от времени проводите с ними ретроспективы. Основная цель — сбор обратной связи и улучшение процесса. Формат ретро-

спектив можно посмотреть в главе 3.8. Празднуйте успехи. Хвалите больше и чаще. Помогайте решать проблемы.

Для заказной разработки (агентская схема) ретроспективы удачно проводить со всеми дизайнерами. Заодно можно устроить общий смотр работ за месяц и обмен опытом.

Время от времени дизайнеров можно брать на ретроспективы разработчиков, особенно если к макетам были претензии. Устраивайте совместные митапы и обсуждения макетов с дизайнерами и разработчиками. Холивары и драки. Гораздо лучше, если дизайнеры и программисты будут открыто и напрямую, глаза в глаза, между собой обсуждать технические проблемы макетов, чем разобьются на два лагеря «мы и они» и будут пассивно-агрессивно друг друга подкалывать в чатиках. Устанете менеджерить такую дрим-тим.

Для продуктовой разработки острой проблемы нет. Просто берем дизайнера на общую ретроспективу команды.

6.7.19 ВОЗРАЖЕНИЯ ПРИ ДЕМОНСТРАЦИИ ДИЗАЙНА

Вероятнее всего, руководителю проекта нужно будет защищать разработанную концепцию. Либо перед своим руководителем. Либо перед заказчиком. Либо перед Внеземным Иван Ивановичем.

Итак. Вы построили процесс. Создали с дизайнером концепцию. Вам все нравится. Идеально провели презентацию. Но вам начинают возражать. Причем как-то некорректно и невнятно. Самый дурацкий, но самый частый вопрос в этой ситуации: «А что конкретно не так?» Он открывает врата ада.

Поскольку материал очень большой, я покажу здесь общую схему и оставлю ссылки на две статьи, где вы найдете примеры отработки конкретных возражений и видеоролики.



Дизайн: работа
с возражениями (часть 1)

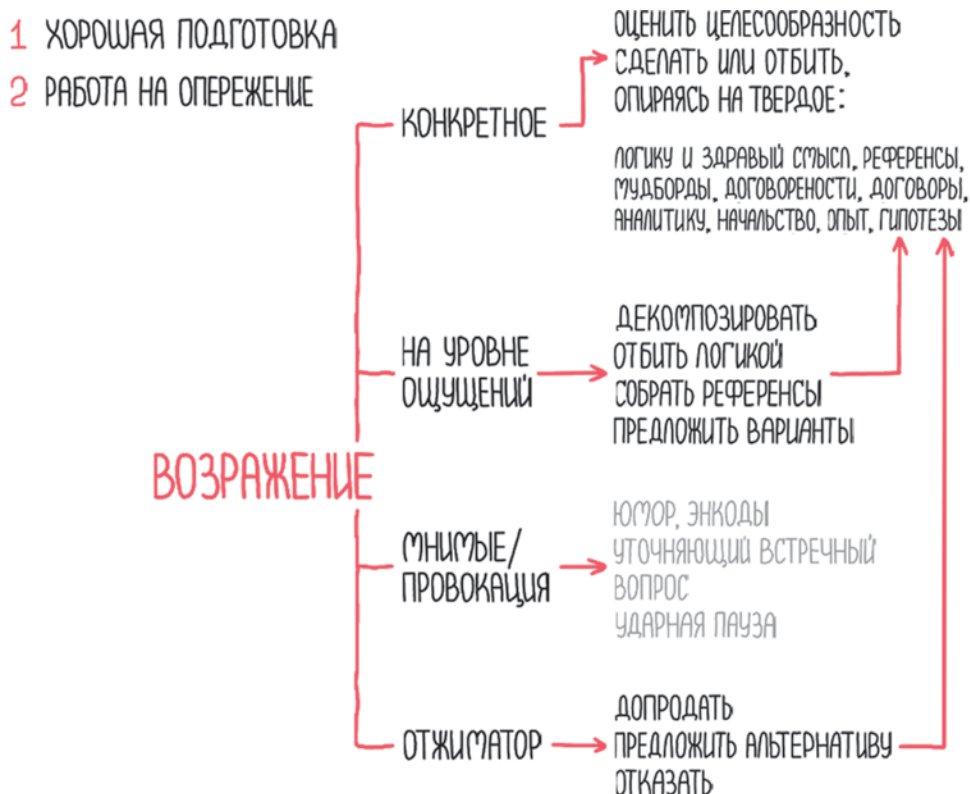


Дизайн: работа
с возражениями (часть 2)

Во-первых, я заранее согласен с тезисом: «Если нормально делаете — работа с возражениями не понадобится». Речь не идет о том, чтобы применить боевое НЛП и втюхать макет, от которого вас самого воротит. Ничего не сработает, если дизайн тухлый, презентация сырая, а менеджер не заинтересован в успехе проекта.

Речь идет о том, что, если все сделано офигенно, вы сами довольны. И вопрос в том, как при возникшем возражении вывести диалог в адекватное русло и решить ситуацию конструктивно. Помочь запутавшемуся клиенту понять, что именно ему не нравится. И в соответствии с этим вносить конструктивные изменения. А не ударяться в бессмысленные бесконечные правки.

Основное, что тут нужно, — профилактика. Хорошая подготовка. И работа на опережение. Заранее предвидеть, какие вопросы могут возникнуть. И заранее предусмотреть на них ответы.



Адекватная реакция на возражения по дизайну

Во-вторых, классифицируйте возражение. Самое главное — отделить **ощущения** и **конкретику**.

Если возражение **конкретное** — нам очень повезло. С этим можно работать. Обсуждаем, понимаем логику. Либо вносим правку. Либо отбиваем, опираясь или на логику, или на аналитику, или на тесты, или на опросы пользователей.

Если **«на уровне ощущений»** — тут сложнее, так как в дизайне все равно много субъективного. Самое важное — понять, из-за чего создается неправильное ощущение. Возможно, не нравится размер логотипа, а критикуют весь макет. В большинстве все решается несколькими локальными правками. В худшем случае нужно будет откатиться к этапам референсов и мудбордов. В катастрофических — сменить дизайнера. Но если вы все сделали правильно и сами довольны работой — до такого не должно дойти. В особо запущенных случаях надо менять не только дизайнера, но и заказчика. Или собирать требования с нуля. Посмотрите примеры отработки таких возражений в двух статьях выше. Они самые сложные.

Мнимые возражения и провокации можно купировать переговорными техниками. Желание получить больше, чем оговаривали на берегу, не проблема, а возможность выставить дополнительный чек, предложить альтернативу или отказать.

§ 7.8.19.1 Тирада. Не ссать и не лениться!

Почему не креативим? Ссым и ленимся. Тут много проявлений.

1. Критика не к месту

Как работает: во время генерации идей начинаем думать, что идею невозможно реализовать. «Невозможно» — чаще всего означает «Я не могу». В итоге идею не предлагаем.

Или хуже того — критикуем предложенную кем-то идею, объясняя ее негативные последствия или трудности, с которыми придется столкнуться. Или делаем постную рожу: «Ну-ну, продолжай, продолжай. Трынди-трынди, приятно слушать».

Что делать: такие ситуации нужно модерировать. Не только на брейнштормах, но и на ретроспективах. Объяснять сразу про две стадии брейншторма: генерацию идей и фильтрацию. И останавливать критику и обсуждение идей, пока сами идеи не сформированы.

2. Отсутствие контроля

Как работает: часто боимся и ленимся подойти и дать обратную связь. Особенно если она негативная. И особенно если могут что-то сказать в ответ. Или сделать.

Почему? В первую очередь мы не любим делать то, что не умеем или делаем плохо. Возможно, мы не умеем грамотно давать обратную связь:

отделить хорошее и плохое;

похвалить за хорошее;

объяснить, в чем ошибка;

инициировать повторное обсуждение или брейншторм по уже пройденным вопросам;

попросить о помощи;

настоять на переделке плохого;

закрепить выученные уроки.

А, возможно, мы видим, что сделано плохо, но не знаем, как это исправить. Поэтому не рискуем критиковать (хотя тут-то самое время эскалировать!). Тут огромный пласт для прокачки себя, своих коммуникаций.

Или боимся, что потребуется переделка, и понадобится больше ресурсов, которые придется выпрашивать и обосновывать их необходимость.

Что делать: контролировать и эскалировать.

3. Невнимание к деталям

Мелочь ничего не прощает большому. Давайте обратную связь и заставляйте быть внимательными!

Видите, что что-то пошло не так, — спросите себя, а не было ли где-то лени или боязни? Разберитесь и ответьте себе честно.

Итак, какие выводы по работе с возражениями:

1. Нормально делай — нормально будет. Если пошли возражения, скорее всего, где-то уже был косяк. Иногда он в ДНК. Но чаще всего — в процессе. Лени. Или страхе.
2. Типовые реакции на возражения: «А что конкретно не так?», можно и нужно купировать тренировками. Для этого проводите внутренние приемки, как это описано в § 6.7.11.
3. Коллекционируйте возражения и провокации. Собирайте новые. Отрабатывайте их в тишине, заранее. Изучите примеры типовых возражений в видео по ссылкам.
4. Отделяйте ощущения и конкретику.
5. Не говнитесь. Оставайтесь положительным героем.

6.7.20 ГОСПОДИ, ГОСПОДИ, ГОСПОДИ!

Да что ж все так сложно? И это мы еще ничего не распараллелили. Может все же: «Пусть дизайнер подумает, а мы ему — правки-правки-правки»?

Это как стройка: строите шалаш — пройдет и такой подход. Строите Москва-Сити — господи, господи, господи!

6.8 ГДЕ СИДИТ ДИЗАЙНЕР, ИЛИ МОДЕЛИ ОРГАНИЗАЦИИ ДИЗАЙН-КОМАНД

«Сидит» дизайнер, конечно, в компьютере. Удаленно или в офисе, или два дня так, три — так — пока не слишком важно: больше интересуют информационные потоки.

Мы подходим к распараллеливанию работ. Этим довольно сложно управлять, и, если у вас мало опыта, лучше действуйте последовательно.

Давайте рассмотрим несколько моделей организации дизайнеров. Плюсы, минусы, когда какую применять.

6.8.1 ФРИЛАНСЕР-ОДИНОЧКА

Минусы: заслужили имидж ненадежных ребят (теряются, бабушка заболела, факапят сроки и качество); склонны набирать заказы, не закончив предыдущие (текущие работы выполняются за счет будущих);



большинство их работ отвратительного качества; нацелены поскорее сделать заказ и взять следующий.

Плюсы: цена; автономность; легкая заменимость; попадаются звезды; не волнует, если они простаивают; если сработались и все устраивает — удобно строить длительные отношения.

Где применять: мелкие, разовые проекты; рутина; редкие работы, для которых внутри команды нет смысла выращивать компетенции (например, разработка шрифтов, иллюстрации в специфическом стиле, 3D-графика и т. д.) — если это не основная, а эпизодическая работа.

Распараллеливание: скидываем мелкие работы и рутину, результат которой встраивается в общий результат основной команды. Пример: баннеры, тексты, иконки.

Как не надо: брать нескольких фрилансеров, давать им рисовать макеты разных экранов по одному проекту (ну вы там как-нибудь самоорганизуйтесь).

6.8.2 АРТЕЛЬ. АГЕНТСКАЯ МОДЕЛЬ

Самая простая модель. По типу артели. Есть дизайнеры. Есть работа. Берем и распределяем эту работу в соответствии с загрузкой дизайнеров, их опытом и срочностью. Вполне может быть, что сегодня дизайнер рисует главную страницу сайта для тапочек, завтра — внутренние экраны приложения заказа такси.

Если дизайнеров несколько — выделяется арт-директор как самый авторитетный из них. Роль менеджмента — распределять задачи. Роль арт-директора — помогать проталкивать работу на нужный уровень по качеству.

Плюсы: высокая **утилизация**; разносторонняя прокачка дизайнеров — получается довольно универсальная команда; быстрый рост портфолио и собственных скиллов.

Минусы: низкая концентрация на продукте; хаотичность и высокая турбулентность; не все дизайнеры любят такую модель; потери на переключениях; косяки в макетах (одинаковые элементы могут выглядеть по-разному).

Где применять: заказная разработка «на потоке».

Распараллеливание: сложно из-за высокой турбулентности; требует мастерства в координации со стороны руководителя проекта.

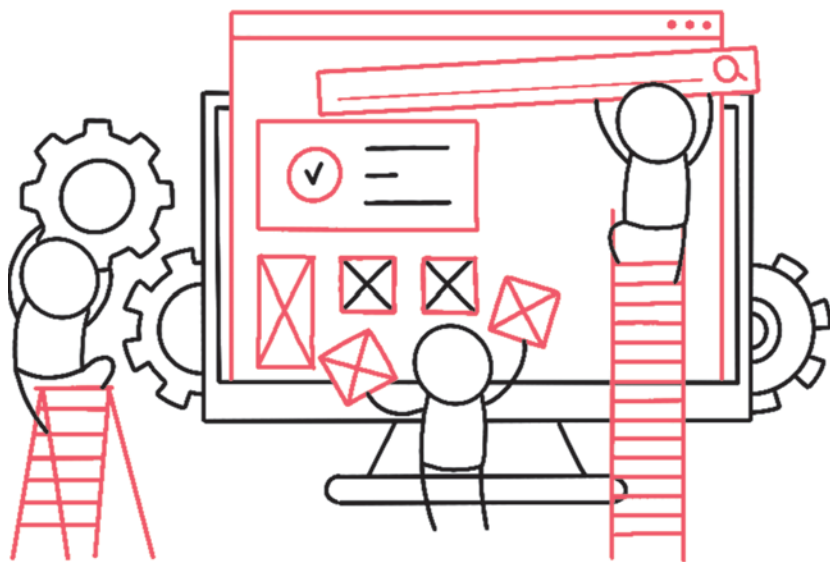
6.8.3 ПРОДУКТОВАЯ МОДЕЛЬ

Дизайнер, или группа дизайнеров, являются частью команды разработки. Сконцентрировано работают над своим проектом (или группой проектов). Зоны ответственности поделены более-менее внятно. Регулярно получают обратную связь от команды разработки (напрямую), так как «сидят» с ними плечом к плечу.

Плюсы: высокая концентрация; командная работа.

Минусы: дизайнер «вырван» из дизайнерской тусовки; прокачка и рост уровня замедляются или останавливаются (пять лет рисуешь баннеры для прокладок, в этом ты когда-то был круче всех, но тренды поменялись, а ты — нет).

Где применять: крупные, долгоиграющие, зрелые продукты, типа Mail.ru. Постоянный дизайнер на фултайм или команда для большинства коммерческих проектов — это чересчур.



Распараллеливание: довольно неплохо — можно как распараллеливать работы между дизайнерами, так и стартовать разработку, не дожидаясь финального согласования всех страниц; требует высокого мастерства в координации со стороны руководителя проекта и зрелости команды.

6.8.4 МАТРИЧНАЯ МОДЕЛЬ

Комбо агентской модели и продуктовой.

За конкретным проектом закрепляется конкретный дизайнер (или группа). Все дизайнеры в одной тусовке. Выделен арт-директор (как правило, рисующий) для помощи, прокачки и подталкивания работ на нужный уровень качества. Одновременно дизайнер концентрируется только на конкретном проекте. В течение дня (или нескольких дней) его стараются не дергать на другие проекты.

Плюсы: высокая утилизация; разносторонняя прокачка; высокий уровень концентрации на проекте; быстрый рост портфолио и собственных скиллов; универсальность команды.

Минусы: хаотичность и турбулентность остаются. Ниже, чем в агентской модели, но уровень высокий. Мелкие-срочные правки приходится пропихивать; требует высокого уровня менеджмента и инфраструктуры; слабая обратная связь от разработчиков к дизайнерам (и наоборот).

ПРОЕКТ 1		ПРОЕКТ 2		ПРОЕКТ 3	
					
					
					

Компенсации минусов: разделение рабочего дня на основной проект и мелочевку. Например, шесть часов в день — основной проект; мелкосрочная фигня — два часа. Регулярные совместные ретроспективы и митапы между командами разработки и дизайнерами. Регламентация, чек-листы, регулярный контроль.

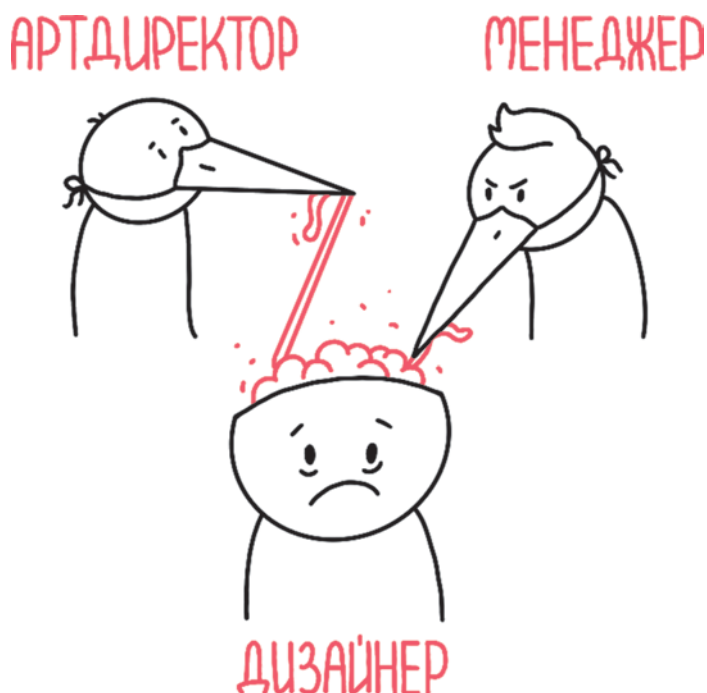
Где применять: это развитие продуктовой и агентской модели. Применять в зрелых, крупных командах с построенными процессами.

Распараллеливание: аналогично продуктовой модели.

6.8.5 ДВОЕВЛАСТИЕ

Практически при любом раскладе у дизайнера будет два начальника. Один — по задачам, руководитель проекта. Второй, по «красотишке» — арт-директор. Важно на берегу четко очертить правила: кто за что отвечает и кто главнее. За кем финальное «да» или «нет» по макету.

Лично я за руководителя проекта, поскольку именно он отвечает за весь проект целиком. Вспомним фанатично-дотошного Джобса, который сам выбирал из предложенных вариантов радиусы скруглений у экранов. В этой модели арт-директор — это как сервис, помогающий сделать все лучше. Но знаю коллег, которые со мной не согласятся. Окей. Главное, просто четко поделить зоны ответственности. Раз и навсегда. Иначе будут бесконечные споры и обидки. Причем, споры



не про то, как сделать проект лучше (это хорошие споры, особенно если есть конструктив, идеи и варианты). А про то, кто здесь альфа, кто здесь бета и чье мнение важнее.

6.9 СТРАТЕГИИ ПАРАЛЛЕЛЬНОЙ РАБОТЫ

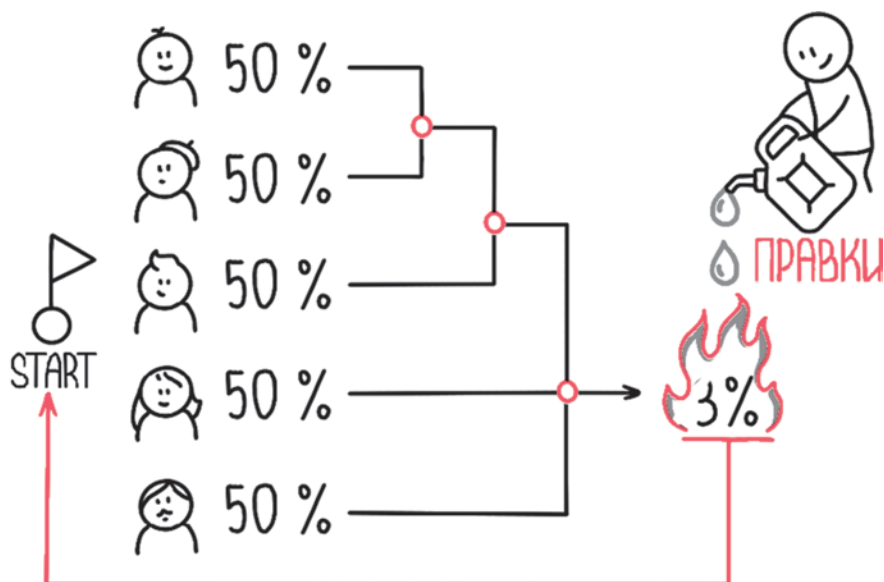
Добавление рабочей силы к опаздывающему проекту программного обеспечения задерживает его выход еще больше.

Закон Брукса

Для ускорения общего хода работ мы делаем некоторые задачи параллельно. В том числе в дизайне. При этом суммарно сгорает больше ресурсов на подготовительных этапах и синхронизации. Резко, по экспоненте, возрастают трудозатраты руководителя проекта.

Чувствительность к ударам Мерфи повышается в разы — становится еще больнее.

Наиболее хрупкие точки — это интеграция результатов работ нескольких специалистов. Допустим, у вас пять исполнителей. Вы обез-



Стратегии параллельной работы

жирили оценки и используете довольно оптимистичные метрики (см. главу 5 об оценках).

Скажем, оценки даются с вероятностью 50%. Это значит, что в половине случаев ваши задания будут делаться дольше. Какова вероятность успеть сделать всю работу в срок? Всего лишь 3%. Подсказка для тех, кто не учил математику: вероятности нужно перемножать; интуитивная подсказка: если кидать по-честному монетку — вероятность орла будет 50%; если кинуть две монетки — вероятность двух орлов одновременно будет 25%.

Теперь добавим к этому ПРАВОЧКИ. Пусть небольшие. Но так, что придется вернуться к началу. Дезинтегрировать весь проект. Сделать правки. И потом собрать все заново. Каждый такой цикл чуть ли не удваивает календарные сроки конкретного этапа.

Пример, понятный бизнесменам: вас попросили заменить одну пружину в двигателе внутреннего сгорания. Разобрали. Пока двигатель был разобран, вам почти без разницы, одну пружину менять, или все. Поменяли одну. Все по плану. Собрали. Проверили. Готово! Тут прилетели правочки: поменяйте еще одну! Ну что вам стоит, вы же профессионалы, и в рамках бюджета, пожалуйста. Ваши чувства?

Мало где можно «в открытую» в смете показать рост стоимости работ за счет распараллеливания. Особенно в заказной разработке. Тот, кто реально заложит нужную ему подстраховку, никогда не получит заказ. Это портит рынок.

По факту (не на словах, а именно по факту) сами заказчики в 99% случаев не готовы к параллельной работе. Чаще всего выигрыш, полученный за счет параллельных работ, будет разбазарен на внутренних согласованиях и получении нужных, компетентных данных, бюрократических процедурах и т. д.

...Но все же. Но все же. Вести параллельные работы нужно. В том числе в дизайне.

В каких случаях это оправдано:

1. Сама работа достаточно объемная и понятным образом делится на максимально независимые части. То есть, это точно не отрисовка концепции!
2. Бутылочное горлышко проекта — дизайнеры. У вас много экранов. Вы проглотили кусок. Теперь нужно жевать.
3. Выигрыш от параллельной работы не будет разбазарен на этапах согласования, ожидании лица принимающего решения, бесконечных итерациях правок, ожидании материалов и т. д.
4. Есть ресурс на управление параллельными потоками работ и интеграцию.

В этой главе мы рассмотрим несколько стратегий организации таких работ. Удачных и откровенно дрянных. Поехали.

6.9.1 ПОЕХАЛИ, ПОТОМ ЗАВЕДЕШЬСЯ!

Будь мужиком — верстай без макета!
Слоган agile-программистов

Дизайн пока не нужен, давайте программировать. Потом уже придумаем, как все будет выглядеть.

Пока берем библиотеку готовых компонентов. Вроде Bootstrap или UI-kit или Google Material Design. И делаем «программистский» дизайн. Потом дизайнер покрасит код. Или так и оставим.

Нельзя сказать, что это совсем не работает. Работает, и еще как!

[illegible]

Отрисовать или спроектировать ее «на чистовую», без потери смысла и нюансов — это был бы проект на тысячи часов. Причем, четверть из них ушла бы на управление проектом: разжевывание дизайнеру нюансов, корректировки того, что он сделал, воспоминание, почему мне надо именно так, доводку и полировку. Любой дизайнер без миллиона итераций и пристального внимания тут только все испортит.

Другой пример: система подачи авиационных планов полетов ivprf.ru.

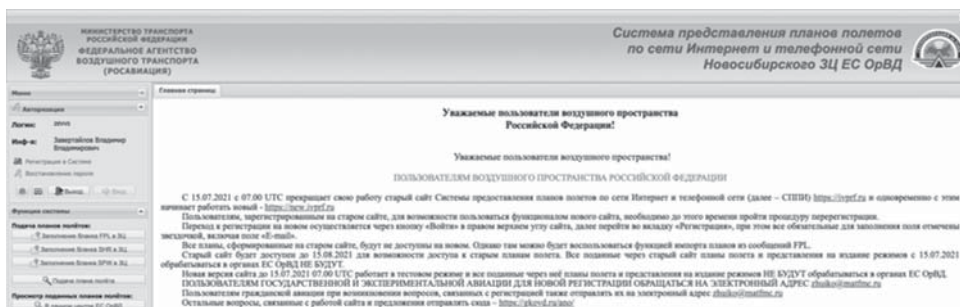
The screenshot shows a web application for flight plan presentation. At the top, there are two time displays: 06:13:03 and 23:13:03. The title bar reads 'Система представления планов полетов по сети Интернет и телефонной сети Новосибирского ЗЦ ЕС ОрВД' (Flight plan presentation system via the Internet and telephone network of the Novosibirsk ZTS ES OVD). The interface is divided into several sections:

- Left sidebar:** Contains navigation links like 'Главная страница', 'Справка', 'Обратная связь', and 'Информация о работе ЦСМР'.
- Main content area:** Displays a table of flight data with columns for flight number, date, time, and status. Below the table, there are several input fields and buttons for flight details, including '3 Тип сообщения', '7 Операционный индекс ВС', '8 Правила полетов', '10 Информация о полете', '11 Информация о полете', '12 Информация о полете', '13 Информация о полете', '14 Информация о полете', '15 Информация о полете', '16 Информация о полете', '17 Информация о полете', '18 Информация о полете', '19 Информация о полете'.
- Right sidebar:** Contains a list of flight numbers and their corresponding times.

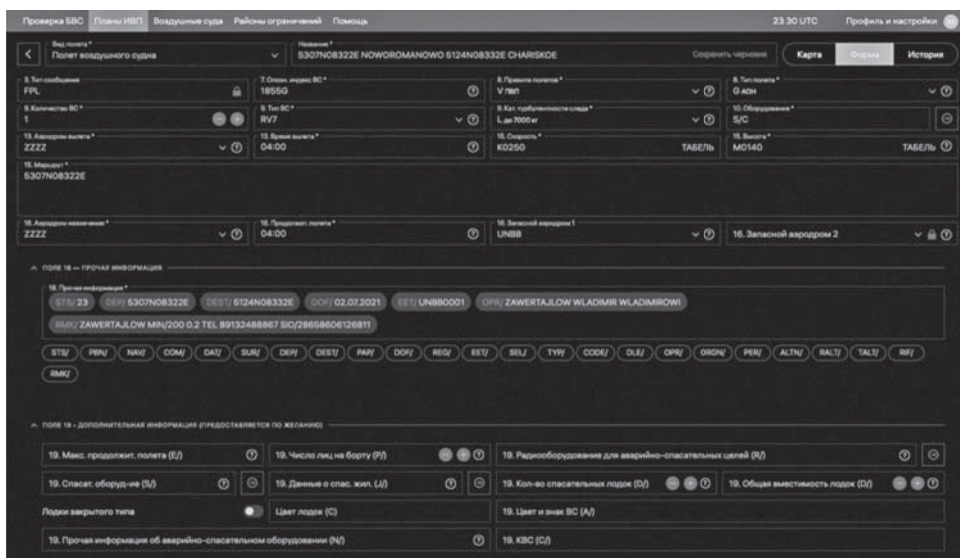
Та еще «громадзилла» на [Ext JS](http://ExtJS). Вроде как, были картинки, где Студия Лебедева ее перерисовала. Принципиально там ничего не поменялось: те же формы, те же поля, те же списки. Только стало посимпатичнее. Да и не сильно там дадут дизайнеру разгуляться: набор, порядок и нумерация полей закреплены в федеральных законах.

Но два года прошло после этих картинок, а внедрения все нет. Думаю, что для того, чтобы перекрасить формы и кнопки, придется все переписать с нуля. Как минимум — весь frontend. Тысячи часов и высокая стоимость ошибок.

Upd 2021.07.02 Пока книга готовилась к печати — новый дизайн таки выкатили. Выглядит шикарно. Глючит и тормозит невероятно. До 2021.08.15, декларативно, всех пилотов заставят подавать планы в ней. Импортировать старые планы можно только вручную, и то, если повезет.



А вот и первый план.



Итак, подход «сначала программируем» подходит для:

- 1. Работающих прототипов и автоматизации рутины.** Когда интерфейсы на начальном этапе не так важны, как проверка гипотезы «заработает или нет». Вам скажут «спасибо» только за то, что вообще удалось что-то автоматизировать. При этом для вывода проекта на рынок в виде SaaS или «коробки» придется все переделывать и делать еще 90% работы.
- 2. B2B.** В промышленной разработке интернет-проектов этот подход также применяется для оперативной разработки всевозможных личных кабинетов. Как правило, с использованием типовых компонентов. На выходе получается рабочая система, но с «программистским» дизайном и юзабилити. Многие подобные системы

в таком состоянии живут годами. Редизайнить их довольно сложно: обычно это требует ресурсов, сопоставимых с разработкой с нуля.

Даже если код системы хорошо структурирован, представление (фронтенд) изолировано — вообще нет гарантии, что в результате работы UI/UX-дизайнера не появятся основанные на реальной аналитике рациональные предложения по изменению бизнес-логики или переконпоновке модульности приложения. И будет конфликт между «удобно» и «привычно».

Еще одно ограничение, причем довольно существенное. Библиотеки компонентов могут в принципе не содержать нужного элемента или очень плохо компоноваться под нужды конкретного проекта. Например, если в качестве основы вы возьмете Google Material Design, а на проекте вам потребуется диаграмма Ганта — вы ее там, скорее всего, не найдете, и придется вставлять что-то инородное. Или городить программистский дизайн. Или, как вариант, — дорисовать конкретный компонент.

Конкурировать с приложениями с «программистскими интерфейсами», если в вашей нише есть только такие, довольно просто. Нужно сделать то же самое, но начать с проработки интерфейса. Те же самые планы полетов, но гораздо человечнее, хоть и за деньги можно вести на fpln.ru.

Весь мой опыт говорит о том, что **распараллелить** дизайн и код (в том смысле, что программисты пишут код, дизайнеры в отрыве от всего или «на живую нитку» рисуют дизайн, а потом, волшебным образом одно совмещается с другим) — утопическая затея. Сами дизайнеры и программисты между собой не договорятся. Будут тысячи нестыковок и конфликтов.

В итоге и в дизайне будут косяки. Пляшущие отступы, неудобный UX, никогда не актуальные макеты. И в коде будет мешанина стилей и костылей. Все устанут, перегрызутся. Потратят в два-пять раз больше ресурсов и времени, чем если бы действовали последовательно или итерационно. А результат будет дрянь.

И как быть? Две стратегии:

1. Лучше сделайте MVP целиком на библиотечных компонентах. Запустите. Отладьте. Обкатайте на реальных пользователях. Если

все ОК, через пару лет аккуратно начнете проект редизайна. И это будет большой проект!

2. Если набор библиотечных компонентов чуть-чуть не устраивает (например, ну очень хочется шрифтами поиграть) — сначала отрисуйте компоненты (UI-кит и пару сложных экранов). Затем собирайте на них всю остальную систему.

Итак, подход в принципе рабочий для стартапов, B2B или внутренних проектов. Но чреват большими затратами на редизайне. Параллелить без синхронизации дизайн и разработку — плохая затея.

6.9.2 ВСЕ ДОЛЖНО БЫТЬ ОДНОЙ РУКОЙ

Противовес. Другая стратегия. А давайте сначала все продумаем, нарисуем все состояния, все экраны. Утвердим. Все предусмотрим заранее. Потом будем что-то программировать. Годится, особенно для небольших проектов. Управлять — просто. Но никакого параллелизма нет. Зато есть персонало-зависимость, особенно с сильно-творческими, невротическими личностями. Готовыми в любой момент потерять интерес к проекту. Все бросить. И уехать пить водку в Кебезень.

6.9.3 НАПРАВЛЕНИЯ ДЛЯ ГРАБЕЖЕЙ

Дели отряды для грабежа, чтобы не было беспорядков.

В. К. Тарасов

Общая идея в том, чтобы выделить максимально независимые подпроекты в проекте и раздать их разным командам. При этом может быть общий стиль или даже фреймворк, который объединяет все подпроекты в одну канву.

Так, при разграблении города, гораздо практичнее распределить районы города между отрядами, чем собирать все награбленное в одну кучу и пытаться вознаграждать на свое усмотрение (*Это тебе, это мне, это снова мне, это опять мне. Как я, себя не обделил?*). Будет меньше обиженных. Влияет фактор удачи и фактор личной смелости, доблести, инициативности.

Пример 1. Дисней под каждый из мультфильмов делает мини-сайт с активацией. Механики могут быть самые разные: фоторедактор, где ты фотографируешь себя и наряжаешь в костюмы героев мультфильма, голосовалки, анкеты и квизы. Мы делали несколько таких проектов, и каждый раз дизайн утверждался заново. Вся стилистика задавалась конкретным мультфильмом, ничего не повторялось. Даже кнопки. Однако при программировании нужно было разместить общие коды подключения шапки и футера, которые собирали мини-сайты в одну экосистему. Понятно, что работу над сайтом каждого мультфильма можно было поручить отдельной команде.

Пример 2. Дизайн-системы больших сервисов вроде Mail.ru. Мало того, что у них десятки (если не сотни) подпроектов, так еще и некоторые web-проекты существуют в виде отдельных мобильных приложений. Для стандартизации тут точно понадобится хорошая, компонентная дизайн-система. Но если она есть — подпроекты можно поручить разным дизайн-командам. И все будет получаться в едином стиле.

Пример 3. Распараллеливание по компетенциям и типам работ. Интерфейсы рисует один человек. Иллюстрации и баннеры — другой. Фото-контент и ретушь готовит третий. Очень частая история.

Итак, мы не сваливаем всю работу в общую кучу и не зовем нескольких дизайнеров эту кучу разгребать. Мы выделяем направления, причем такие, что можно действовать независимо. Некоторые правила взаимодействия (вроде общих фреймворков, UI-кита, размеров баннеров, примеров размещения фото) будут нужны, но обычно, — это небольшие и простые для исполнения документы. Управлять довольно легко.

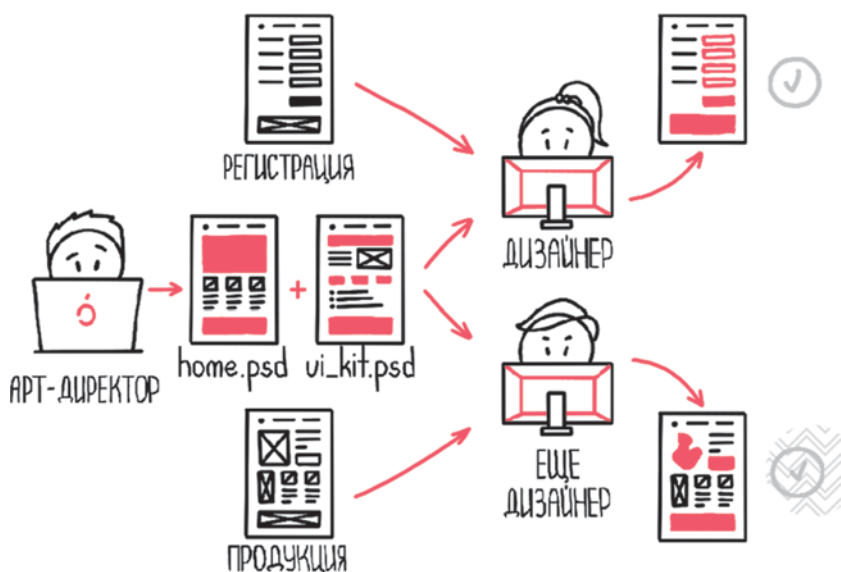
Хорошо работает на проектах большого размера.

6.9.4 ДИКТАТУРА-БАЗАР-ДИКТАТУРА

Концепция и UI-кит готовятся кем-то одним, опытным, на нулевой итерации. Например, арт-директором. Утверждаются. После этого составляется детальный бэклог со списком всех экранов.

Далее берем небольшую артель дизайнеров (два-три человека). Они самостоятельно определяют, кто и что будет рисовать в данный

конкретный момент. Кому что ближе. Работают, как правило, в одном проекте, все плотно общаются, видят результат друг друга. Обсуждения — громкие и каждый день. В общем, базар.



Арт-директор делает концепцию и UI-кит, а дизайнеры по ним рисуют страницы

Раз в неделю (допустим) базар заканчивается, начинается диктатура. Результаты сводятся. Тестируются. Проверяются отступы, одинаковость компонентов. Утверждаются арт-директором (или автором концепции). Проводится демонстрация стейкхолдерам. Вносятся нужные правки. Обновляется UI-кит.

Плюсы. Хорошо работает в слаженной команде. Действительно можно получить хороший, почти пропорциональный количеству дизайнеров рост скорости этапов. Можно комбинировать с «направлением для грабежей».

Минусы. Не будет работать, если дизайнеров более трех (слишком высокие накладные расходы на коммуникацию, синхронизацию, проверки-перепроверки и исправление ошибок). В случае простоев (например, на согласование следующего куска работ) вхолостую сжигается очень много времени. Мелких косяков, которые придется решать на верстке, накапливается довольно много. Даже с тестированием. Даже с компонентным подходом. Пуканы разработчиков могут гореть.

6.9.5 ВЕДУЩИЙ + ВЕДОМЫЙ ИЛИ НАСТАВНИК + ПАДАВАН

Работа в паре. Наставник + Падаван. Наставник — безоговорочный авторитет. Делегируются очень небольшие работы. С постоянным контролем и проверкой. По мере роста компетенций Падавана задания могут становиться все больше.

Подходит, скорее, не для запараллеливания, а для прокачки и погружения в проект юных дарований. На начальных этапах проект такой подход не ускоряет, а даже наоборот.

Если Падаван сопротивляется авторитету Наставника, на все имеет свое мнение или делает очень много ошибок (чем раздражает наставника) — ничего толком не получится.

Может сработать в долгосрочной перспективе. При выращивании команды из стажеров. Получаем больше обученных людей, погруженных в проект.

6.9.6 ОПАЗДЫВАЕШЬ — ТОРМОЗИ!

Никто не виноват, а морды у всех битые.

Михаил Евдокимов

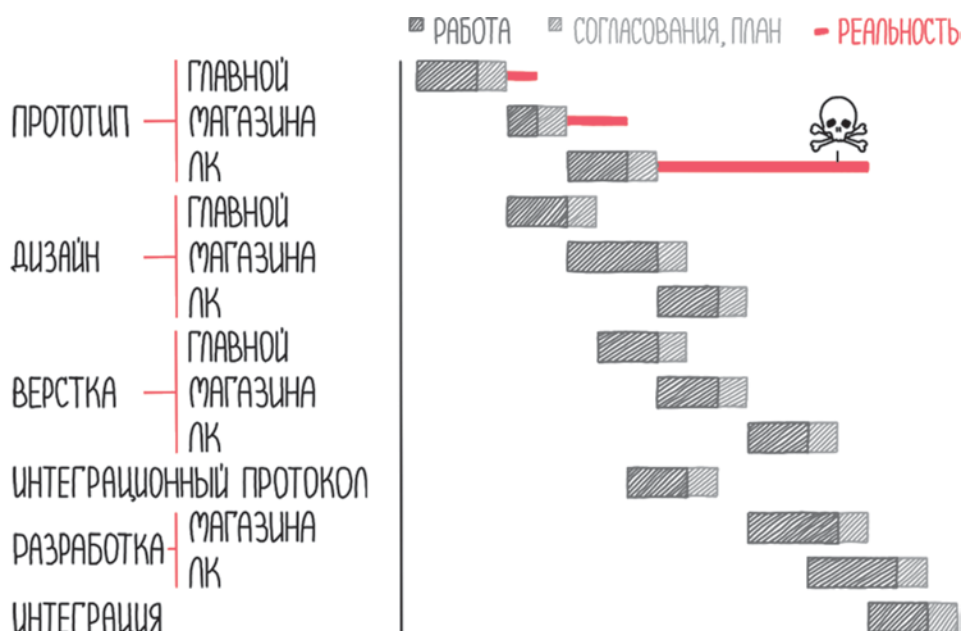
Миллион раз такое видел. На входе заказчик обещает максимально быстро все принимать и предоставлять.

— Да мы! Да легко! Да сколько угодно! Да я лично! Да в любое время суток! 24/7! Или даже на 8!

Просит максимально распараллелить этапы работ. И также быть 24/7 в онлайн (это тревожит, ибо обещает работу вместо жизни). Больше людей, больше! Таймлайн составлен так, что все с колес, все с колес.

Пять-семь потоков работ. Параллельно. С десятков человек. Все что-то хотят, требуют внимания. Еще только прототипы рисуются, уже дизайн пары экранов нарисован. И верстать-программировать пора. Но еще ничего не принято, не сдано окончательно. Везде правки, правки, правки. И тут — опаньки...

Начинаются раздутые и долгие согласования. Отпуска. Потом — лето. Кто-то заболел. Потом выясняется, что есть еще один ЛПР. Очень важный. Хренпоймаешь Сергей Эдуардович. В бухгалтерии задержали очередную оплату. Здравствуй, Мерфи, давно не виделись.

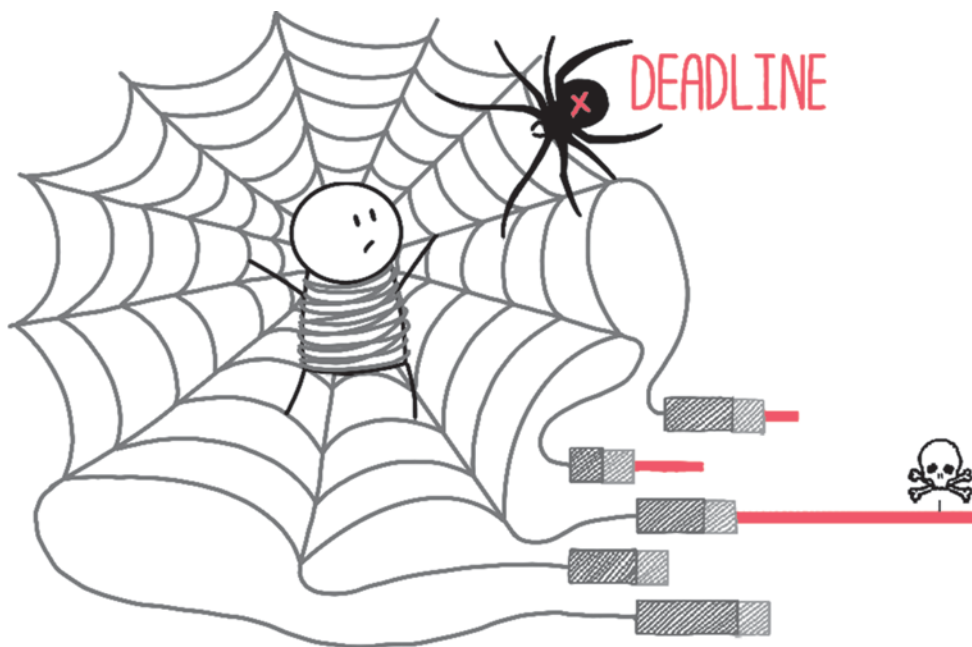


Ожидания от распараллеливания и жестокая реальность

Менеджеры (как заказчика, так и агентства) очевидно не справляются с потоком информации и взаимных обещаний. Агентство пищит: «Ну вы же обещали согласовывать быстро». Заказчик манипулирует: «Но мы же не знали, что будет ТАКОЙ УЖАС!» Хренпоймаешь Сергей Эдуардович вечно занят и ничего не принимает. Хотя в субботу прислал голосовое сообщение: «Друзья! Концепция изменилась! Пока нет времени рассказывать детали, надиктую позднее. Начните-ка все с начала. И в рамках бюджета, пожалуйста». Атмосфера полна любви и обожания. Сроки уже пылают. Хотя проект идет полным ходом.

Добавлять рабочую силу, очевидно, смысла нет. Это как паутина. В которую поймали менеджера. Чем больше он дергается, тем больше запутывается. Чем больше к нему ниточек, тем сильнее он увяз, вместе с проектом.

Спешка приводит к ошибкам. Ошибки приводят к недоверию. Недоверие приводит к катастрофе. Вот где-то тут все и выгорают. Или особенно творчески развитые успевают наделать хвостовых рисков (подробнее о них в части 12) и уволиться, пока не долбануло.



Есть целая куча причин, почему происходит именно так. В этой главе мы их разбирать не будем (см. часть 12). Дам только отсылку к Теории Ограничений: вероятнее всего, менеджмент (как агентства, так и заказчика) стал бутылочным горлышком проекта.

Что делать?

Нужно сбросить ритм. Сесть за стол переговоров. Открыть сетевой график работ. Плановый и фактический. Измерить реальную скорость согласований и выполнения заданий. И обязательно — сокращать количество параллельных работ. Резать паутинки.

Магическим образом это ускорит календарный срок всего проекта. Кроме того, менеджменту нужно сконцентрироваться на критическом пути проекта. Применить управление **Барабан-Буфер-Канат** из теории ограничений. Легализовать и визуализировать проектный буфер. И отслеживать его расход. Но самое главное — убрать параллельные работы. Итак.

Опаздываешь — тормози!

6.9.7 ПРЕДВАРИТЕЛЬНЫЕ ВЫВОДЫ

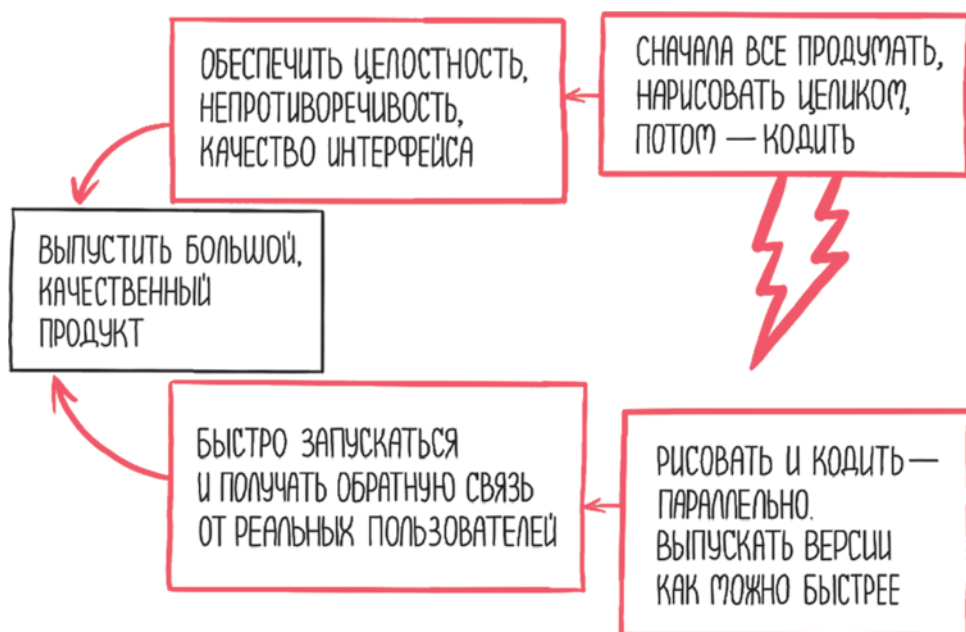
На практике я вижу, что дизайн почти никогда не является бутылочным горлышком проекта. Параллелить есть смысл на очень небольших участках. Либо по принципу «направления для грабежей», либо по «диктатура-базар-диктатура». В продуктовой модели гораздо лучше организовать дизайн-спринты.

6.10 AGILE + UX. ДИЗАЙН-СПРИНТЫ

Пора рассмотреть итерационный подход в дизайне. Он самый сложный в организации. И если вы с ним разберетесь, значит, и со всем остальным справитесь.

В первую очередь обратите внимание на этот подход в **продуктовой разработке** или в **разработке крупных проектов**.

Мы будем исходить из того, что нет каких-то аксиом и теорем в дизайне. В лучшем случае есть гипотезы, опыт и результат альфа-бета тестирования на целевых группах.



Уже знакомый нам конфликт целостности интерфейса и итеративной разработки

В Agile разработчики предпочитают создавать проект по частям, отдельными итерациями. И таким же образом выпускать продукт на рынок. И тестировать гипотезы.

UX-дизайнеры предпочитают продумывать и разрабатывать концепцию целиком. Более того, дизайнер вообще не поймет требования «разрабатывать интерфейс по частям». И будет крутить пальцем у виска.

Намечается два пути: оставить дизайнеров в покое или вовлечь их в Agile. Притом стараясь никого не покалечить.

Итак. Наша с вами задача — построить такой рабочий процесс, в котором дизайн получился бы целостным. Кроме того, мы на берегу считаем, что проект будет развиваться, получать обратную связь от пользователей, проходить валидацию рынком и его разработка пойдет итеративно. По Agile. Задача звучит так:

Как обеспечить параллельную работу над дизайном, написанием кода, выпуском новых версий продукта, стратегией и сбором обратной связи, оставляя дизайн качественным и целостным?

6.10.1 ПРОБЛЕМА ВОСПРИЯТИЯ ПРОДУКТА

Допустим, заказчик (или Product Owner) захотел создать клевую машину. В голове у него есть какой-то яркий образ, как эта машина поможет достичь его целей. Формулировки в виде «быстрая», «стильная», «красная», например. Объем двигателя, степень компрессии и Pantone краски спойлера в его картине мира вряд ли фигурируют. С программным обеспечением все то же самое. «Хочу социальную сеть для котиков», например.

Кто-то умный должен все это переварить и спроектировать. Сделать спецификации на разных уровнях. От концепции до чертежей конкретных узлов и агрегатов. Еще до того, как будет строиться первый рабочий прототип. В реальном мире это десятки инженеров. В digital хватит одного аналитика. И мыслят они больше схемами: «экран авторизации котика, экран профиля, а тут котик может аватарку поменять».

Теперь нам нужно придать этому какую-то видимую форму. Дизайнер делает макет, например, 1:5 из, скажем, пластилина. Все это дорабатывается-перерабатывается. И в какой-то момент появляется целостная картина! Полноразмерная модель, в которую влюбляется Product Owner. Там есть эмоция и драйв! Примерно так же идет работа над прототипами и концепцией в digital-продукте. Пиксели — материал очень податливый.

Дальше приходят инженеры. Смотрят на спецификацию. Смотрят на макет. И начинают брюзжать. Это сделать невозможно, это будет слишком дорого, тут будет слишком сильное лобовое сопротивление, тут недостаточно жесткости, бла-бла-бла. Короче, макет никуда не годится. И так во всех сферах.

Так же ведут себя программисты, когда к ним приходят с бредовыми идеями или слишком сложными задачами. Отличить две ситуации невозможно. Инженер воспринимает проект как набор каких-то технологий, деталей, шестеренок, которые нужно подружить между собой. Целостная картинка его мало волнует — можно годами играть в этот конструктор. Лишь бы не мешали. Может ли он показать часть работы? Запросто! «Вот, смотрите, какую я гайку сделал!» Или: «Я ускорил выборку новостей из базы для котиков на 0.0002 миллисекунды! Хвалите меня!»

А потом мало создать машину. Нужно ее обкатать, протестировать, сертифицировать, наладить производство и рекламу. Загрузить в AppStore, пройти модерацию, запустить лендинг.

Для конечных пользователей все промежуточные стадии — от идеи Product Owner-а до наладки производства — имеют такую же ценность, как автомобиль без колес. Более того, пользователи очень избалованы (особенно в B2C), и в абы-какую машину не сядут. Покритиковать, поинтересоваться и дать обратную связь — это запросто. Высказывать мнение — это почетно. Но и то не все готовы время тратить. А уж заплатить за незаконченное — только если сильно прижало. Или очень громкий хайп.

Выводы такие: управляя проектом, время от времени смотрите на него с разных позиций. И с инженерной, и с дизайнерской, и с пользовательской.

6.10.2 МИНИМАЛЬНЫЙ ПРОДУКТ ДИЗАЙНЕРА, LEAN И ЗОЛОТАЯ СЕРЕДИНА

Приходит покупатель в магазин ковров и говорит:
— Мне нужен ковер длиной пятьдесят метров,
а шириной один метр.

Продавец удивился, но отрезал. Покупатель просит:

— А теперь порежьте его по одному метру.

Продавец:

— Вы что, псих?

Тот радостно:

— Да! У меня и справка есть!

Минимальный продукт дизайнера, который можно отделить и отдать в дальнейшую разработку — это один экран. Технически возможно проектировать и раскрашивать экран за экраном и отдавать их сразу на разработку. Однако вероятнее всего будет очень много регрессий (возвратов, переделок) и связанных с этим накладных расходов.

Давайте подсветим этот момент. Классически, по философии бережливого производства LEAN, считается, что комплектовать большие партии убыточно. Например, завод, большая часть продукции которого простаивает «в запасе» на складе, по сути, занимается заморозкой своих активов. А еще товар портится и морально устаревает.

В идеале нужно перейти на принцип вытягивания и все производство организовать «с колес». Покупатель заказал iPhone — мы тут же запустили производство, бац-бац — сделали поставку. Покупатель взял пакет молока с полки — корова по цепочке поставок произвела молоко. Ха-ха! Просто представьте, как вы «вытягиваете» молоко из коровы, когда будете покупать его в магазине в следующий раз. Сложно развидеть.



Однако есть затраты на транспортировку, организацию закупок и прочее. То есть существует какая-то минимальная партия, уменьшать которую экономически нецелесообразно.

В промышленном производстве можно получить эффект масштаба, когда мы производим одну и ту же деталь в одних и тех же условиях миллион раз. Эффект масштаба съедается за счет времени переналадки оборудования между операциями. Поэтому, опять же, есть какая-то минимально оправданная партия, которую нужно сделать за раз, не переключаясь на производство продукта другого типа. Иначе переналадка съест все деньги впустую.

Теперь, что мы наблюдаем в разработке программного обеспечения и дизайне? Переключение между задачами (переналадка) все так же сжирает время. Однако никакого «эффекта масштаба» у нас нет! Каждый проект делается в уникальных условиях и всего один раз. В следующий раз, даже если будет тот же самый проект, условия изменятся (например, у команды будет другой опыт). Каждый раз все по-новому.

Любая поправочка (даже игра со шрифтами) — это возврат товара по цепочке поставок на завод. С дальнейшим разбором и перезапуском конвейера. С переналадкой оборудования и т. д. И чем меньше у нас партии, чем больше правок, тем выше накладные расходы.

Интуитивно это все понимают. Однако часто делают вид, что таких накладных расходов нет. «Ну что вам стоит, тут дизайнеру пять минут поправить» — говорит заказчик, а по факту, с учетом переналадок и «правок на правки» незаметно улетает два-три часа. Экономика не то что не сходится, она в ж-пе. Это называется «транзакционные издержки». И это основной вид потерь в digital.

Так вот. Самый нежелательный тип заказчика — этот тот, кто готов бесконечно лезть в процесс, все обсуждать, пересогласовывать, сыпать правками, менять все к лучшему миллион раз. Но не готов понимать и оплачивать огромные итерационные и моральные издержки, которые он создает таким поведением. Каким бы милым и душевным такой человек ни выглядел — съест мозги за три копейки. Ну его нафиг.

Теперь. Как определить оптимальный размер партии?

У нас три варианта:

1. Продуктовая разработка in-house-командой

Тут самая низкая цена транзакционных издержек. Можно сесть с дизайнером за один монитор и водить его рукой. Это подкупает. Однако бюджет проекта расходуется при каждой такой транзакции. И эти затраты ощущаются всеми участниками процесса.

Если человек с кошельком напрямую вовлечен в этот процесс — это часто его фокусирует на главном в ущерб второстепенному. Он тратит не только деньги, но и свое время, а оно часто очень дорогое. Ну и правками можно всех задолбать, в том числе in-house-команду. Этого никто не отменял.

В случае продуктовой разработки можно попробовать сделать минимальным продуктом дизайнера один экран.

2. Заказная разработка по Time & Material

Заказчик оплачивает все транзакционные издержки и все правки. Это как наемное войско: в определенном диапазоне готовы слушаться, но счетчик тикает.

Digital-наемники. Замучаете их своими придирками и бесконечными требованиями — ничего хорошего не будет. Задолбают вас своими счетами на каждый чих (счет за то, что выставили счет и т. д.) — тоже ничего хорошего не будет.

Оптимальный объем партии: один экран в случае проработки концепции или сложных экранов. От трех до пяти для простых экранов или экранов второго уровня.

3. Заказная разработка по Fixed Price

Тут ограничением будет количество правочек. Пара итераций. Еще в каком-то диапазоне, иногда, по доброй воле и в мелочах, могут пойти навстречу. Но как экономика начнет расходиться — это или доп-косты или конфликтные переговоры. И то, и то всех нервирует.

Транзакционные издержки делают очень дорогими мелкие правочки, особенно в сравнении с большой уже проделанной работой. «Как же так, тут пару блоков поправить, а стоит столько же, сколько полсайта нарисовать?» Ну да, разукomплектации партии, потеря эффекта масштаба и т. д. Но на пальцах это не объяснить. Звучит неубедительно.

В случае заказной разработки оптимальный объем партии — также один экран для проработки концепции (или сложных экранов) и три-

пять для простых экранов или экранов второго уровня. Однако на верстку и разработку чаще всего пойдет не один экран и не пара экранов, а какая-то партия, обеспечивающая запуск минимального работоспособного продукта. Первая версия, вторая и т. д. Это может быть и десяток экранов.

Еще раз. Во всех трех случаях меня сейчас интересуют не моральные аспекты — соотношение клиентоориентированности и границ разумного — а оптимальный размер партии. Только бизнес, ничего личного.

Бытовой пример: допустим, разрабатываем интернет-магазин с личным кабинетом и мобильным приложением. Необходимый минимум страниц: карточка товара, чекаут, список товаров и главная (перечислены в порядке приоритетов с точки зрения бизнеса). На самом деле экранов гораздо больше, но с таким набором уже как-то можно выходить на рынок.

На нулевой итерации минимальным продуктом от дизайнера будет одна ключевая страница. Либо главная, либо карточка товара. Берите самую сложную. Это не зависит от того, Fixed Price, T & M или in-house-команда у нас. А вот дальше — интереснее.

Допустим, концепцию согласовали. В случае in-house-команды мы можем сразу отдавать ее в верстку. Главное — не запутаться потом с мелкими правочками (об этом ниже).

В случае Fixed Price я бы собрал партию из всех нужных для MVP страниц и сделал UI-кит. Затем отдал бы все это в разработку и параллельно приступил к отрисовке тех экранов, которые нужны для второй версии проекта (например, экранов личного кабинета).

В случае Time & Material я действовал бы так же, как при Fixed Price — готовил бы партии макетов, нужные для выпуска версий продукта. Почему? Дело в том, что скорость разработки, как правило, значительно ниже скорости отрисовки дизайна. Особенно на внутренних и интерфейсных экранах вроде личных кабинетов. Один дизайнер за неделю может запросто нарисовать объем макетов, достаточный, чтобы загрузить команду из пары программистов на месяц-два вперед.

Все, конечно, зависит от команды и задачи, и тут нужно подстраиваться. Но опыт показывает, что дизайнер не будет бутылочным горлышком, если концепция уже согласована, и остается только фигачить. Поэтому размер партии нужно подбирать экспериментально.

Итак:

1. Регрессии (правочки) — это основной вид издержек в digital. Избежать части издержек позволяет комплектование партий. Согласовали партию дизайна — отдали в разработку. Размер партии зависит от типа контракта, проекта и команды. И подбирается экспериментально.
2. На нулевой итерации достаточно устаканить один, самый сложный, экран и проработать UI-кит.
3. Далее работу лучше организовать дизайн-спринтами в одну-две недели. За это время успевать готовить и согласовывать от трех до пяти экранов. Очень желательно, чтобы эти три-пять экранов создавали новую версию продукта. Например, отрабатывали какой-то сценарий поведения пользователя, которого раньше в системе не было.
4. За исключением нулевой итерации, дизайнер очень редко бывает бутылочным горлышком проекта. Один дизайнер, как правило, может надолго загрузить команду разработки. Бутылочным горлышком, скорее всего, будут согласования и «устаканивания».

6.10.3 UX И СЛОЕНЫЙ ПИРОГ

Разобьем опыт взаимодействия пользователей и интерфейса на несколько слоев. Модель называется «пирог Джесси Гаррета». Нижние слои — стратегия. Мы идем от нижних слоев к верхним. От основ к деталям. И чем выше, тем конкретнее.

В основании лежит стратегия. Хотя бы в общих чертах мы должны понять, что создаем или какую задачу хотим решить.

Функции. Или требования. За счет чего конкретно мы решаем стратегическую задачу? Что приоритетно, а что — подождет. Это то, что мы собрали в агрегации требований в главе 4. Эти же функции у нас попадают в бэклог продукта.

Структура. Какие должны быть экраны, чтобы реализовать задуманные функции? Как они связаны? В агрегации требований это последняя вкладка. Тут, в отличие от бэклога, который мы видим просто списком, нам важнее связи между компонентами системы.

Уровень компоновки — когда продумывается расположение конкретных элементов. Это наши прототипы.



Пирог Джесси Гаррета

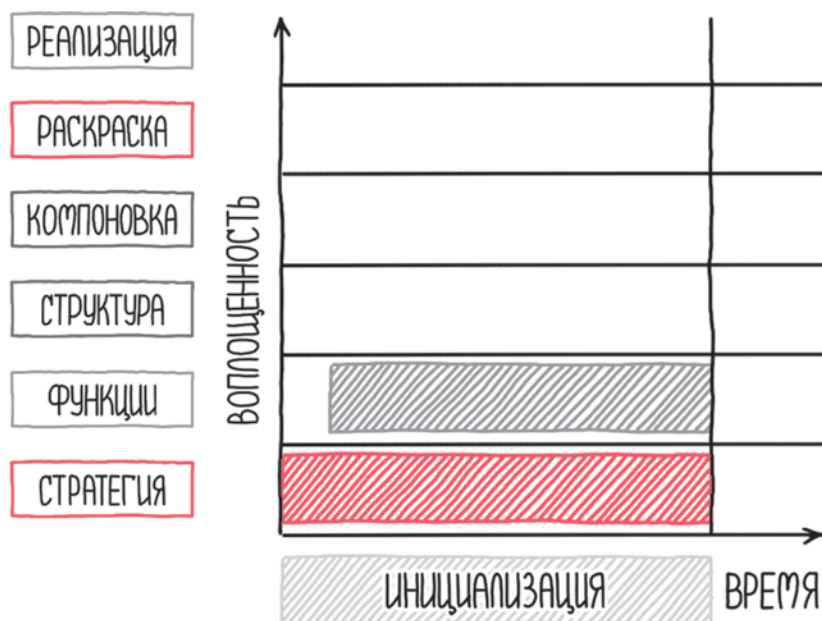
Уровень раскраски — когда элементы визуально оформляются. Играем со шрифтами.

Уровень реализации — это мы уже все запрограммировали. Интерфейс интерактивно реагирует на поведение пользователя и работает как надо.

За стратегию и функции у нас отвечает руководитель продукта (Product Owner). За структуру и компоновку — аналитик. За раскраску — дизайнеры. За реализацию — разработчики. Каждый делает продукт, который уходит на уровень выше, а оттуда — еще выше. Чем дальше мы зашли, тем выше цена ошибки и переделки.

6.10.4 ПУЛЬС ПРОЕКТА. ИНИЦИАЛИЗАЦИЯ. ИТЕРАЦИЯ № 0

Теперь давайте посмотрим, как развивается проект во времени. У нас будет две оси: воплощенность и время. А работа будет разбита на спринты.



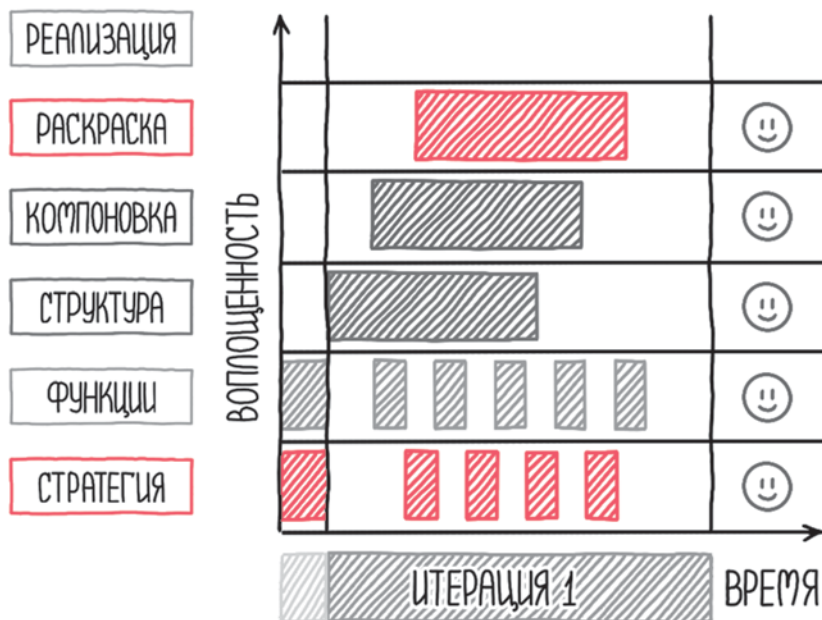
Этап инициализации

На этапе инициализации большую часть времени уделяем стратегии. Здесь много абстракций и допущений. Далее переходим к формированию требований (функций).

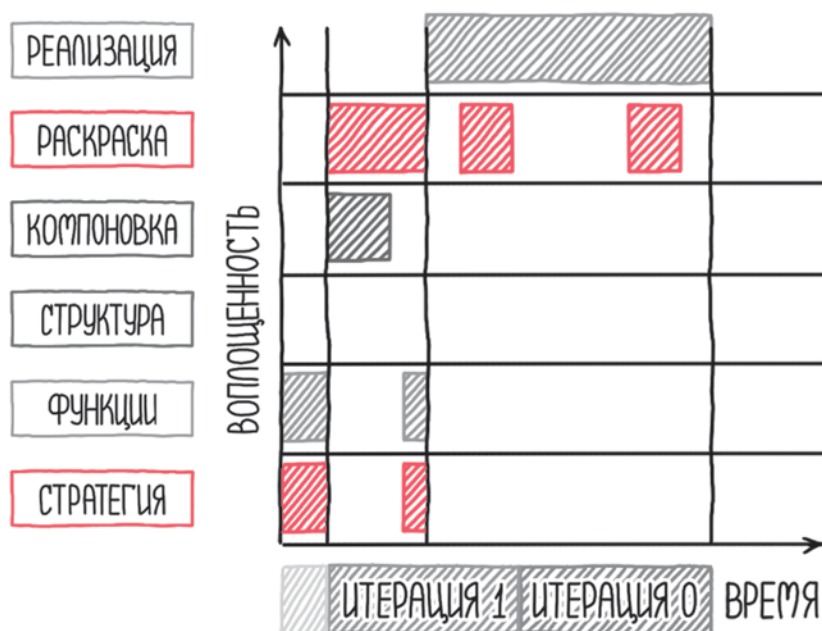
Нулевая итерация. Прорабатываем ключевые экраны. Работаем сначала над структурой. Затем — компоновкой, прототипами. И уходим в раскраску. При этом получаем обратную связь по интерфейсам. Отслеживаем счастливые (или угрюмые) лица потенциальных пользователей. И, возможно, вносим какие-то корректировки в стратегию. Изменение стратегии может повлиять на набор функций. Тут все связано.

Экранов берем самый-самый минимум. Но полноценно, со всеми функциями, которые войдут в MVP. То есть при подготовке интерфейсов мы не «кусочничаем». Нет задачи сразу нарезать проект на двухнедельные спринты для разработчиков. Но и не рисуем миллион экранов.

Чем выше мы продвигаемся вверх по «пирог», тем больше конкретики добавляется в проект и тем дороже обходятся изменения. Поэтому так важны промежуточные демонстрации и проверки макетов. И только затем, когда концептуально наш интерфейс устоялся, мы переходим к первой итерации разработки.



Нулевая итерация



На этапе Итерации № 1 уже можно приступить к реализации

По-любому, программисты найдут нестыковки на макетах. Что-то нарисовано косовато или нелогично с точки зрения реализации. Поэтому прямо в рамках итерации разработки небольшие возвраты к дизайну допустимы. Иконки могут забыть нарисовать в векторе, например. Или мелкие элементы интерфейсов и их стейты при наведении. Куски из адаптива. Пустую корзину, страницу успешного выполнения заказа и так далее.

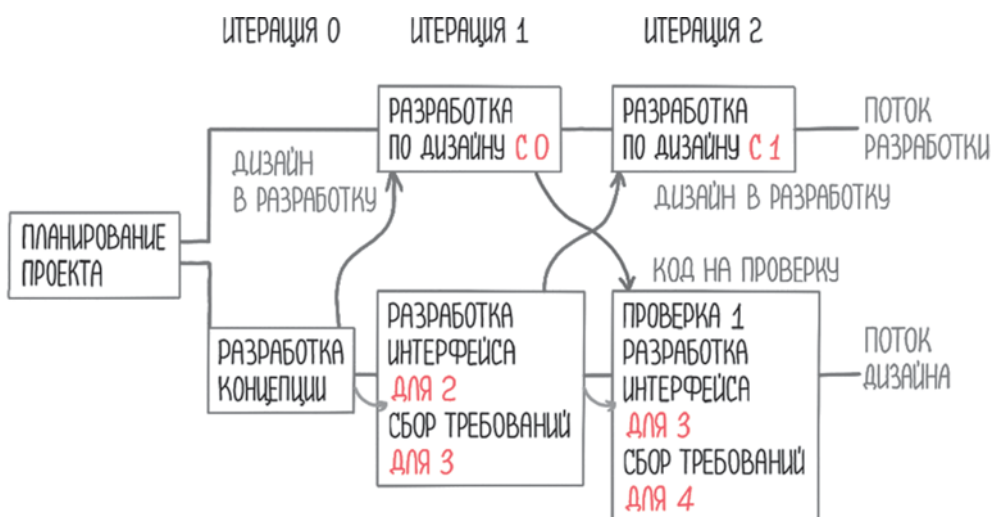
Допустим, разработка первого спринта закончилась. Что остается? Провести демонстрацию, проверить гипотезы, получить обратную связь от пользователей. Возможно что-то поменять на уровне стратегии. А это неминуемо повлечет изменения и в функциях, и в структуре, и в компоновке. И в раскраске, и в коде.

Осталось только замкнуть круг.

Кстати, необязательно релизить результаты первых спринтов наружу. Их может быть недостаточно для минимального жизнеспособного продукта. Тут нужен баланс: с одной стороны, вредно затягивать запуск проекта и обкатку на реальных пользователях. С другой — релиз должен быть клевым, выглядеть няшно.

6.10.5 ВСЕ ПЕРЕПЛЕТЕНО

Итак, идея очень простая. Мы стратегически спланировали наш проект. Выделили MVP. На нулевой итерации нарисовали концепцию. Устаканили ее. Далее отдаем это в разработку.

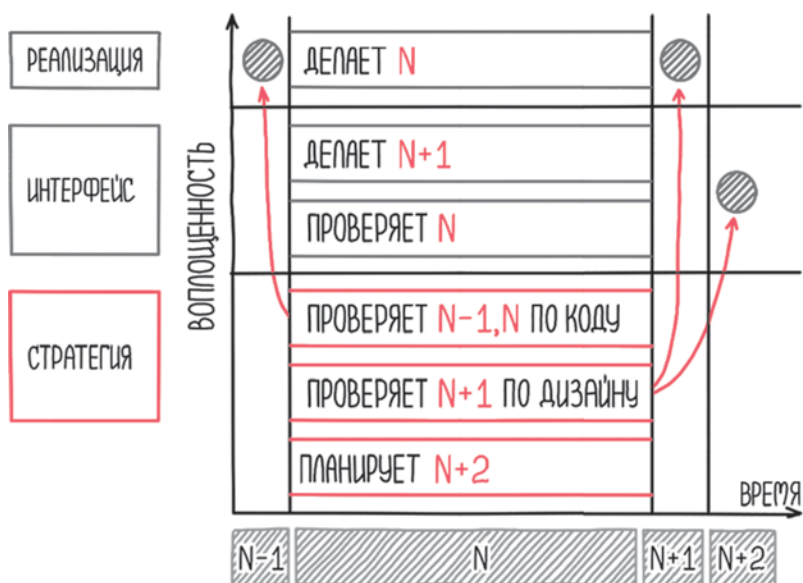


Пока кодеры трудятся, мы с дизайнерами:

1. рисуем интерфейсы для следующих итераций;
2. дорисовываем и исправляем мелочи и ошибки, которые нашли в макетах программисты;
3. проверяем и тестируем на реальных пользователях предыдущую итерацию разработки, собираем обратную связь, обдумываем, как это может повлиять на продукт.

То есть, нам в фокусе нужно держать как минимум четыре итерации: результат предыдущей, текущую и минимум две следующих.

Итого, еще раз, на чем концентрируемся:



1. Кодеры работают над итерацией N.
2. Дизайнеры бегут чуть вперед и работают над N+1. Параллельно помогают разработчикам с их заданиями: дорисовывают то, чего не хватает для спринта, осуществляют авторский надзор.
3. Стратегия, функции, структура формируют N+2-ю итерацию.
4. В фокусе проверки у нас:
 - а. N-1 итерация — то, что только что выпустили в свет;
 - б. N — то, что делают кодеры;
 - в. N+1 — то, где трудятся дизайнеры.

6.10.6 UX-BACKLOG

Итак. В фокусе приходится держать несколько потоков работ. Вам может помочь в этом UX-Backlog. Он очень похож на те бэклоги, которые мы с вами разбирали в главе 3 про Scrum. Но есть нюансы.

1. User Story в традиционном смысле этого слова дизайнерам усваивать сложно. Дизайнер мыслит макетами. Поэтому мы делаем в UX-бэклоге разбивку User Story на список страниц или экранов. Например, User Story — «Покупка товара» разбивается на экраны «Корзина», «Оформление заказа», «Оплата», «Страница успешного оформления заказа». Их удобно сгруппировать в один дизайн-спринт. Личный кабинет может содержать несколько User Story, например: возврат товара, повторный заказ или управление персональными скидками. Что-то можно сделать в рамках одного дизайн-спринта. А что-то — отложить и сделать позднее.
2. Технически UX-Backlog удобно собрать в Google Docs или в канбан-доске.
Бэклог в Google Docs более прост в реализации, можно начать с него.

Влияние на бизнес (приоритет)	User Story	Оценка, Story point	Примечания	Как проверить	Сформирована	Дизайн готов	Тестирование пройдено
1000	User Story 1	4			да	да	80%
980	User Story 1.1	2					
970	User Story 1.2	2					
900	User Story 2	3			да	да	30%
880	User Story 2.1	2					
870	User Story 2.2	2					
Спринт 2:							
400	User Story 3	8			да	нет	нет
350	User Story 3.1	6					
330	User Story 3.2	2					
Спринт 3:							
400	User Story 3	2			да	нет	нет
350	User Story 3.1	3					
330	User Story 3.2	4					
300	User Story 4	1			нет	нет	нет
250	User Story 4.1	1					
230	User Story 4.2	1					

Или работать в Trello. Каждый экран — отдельная карточка. Карточки можно связать тегами по User Story. Стадии прохождения карточек по доске формируются из того бизнес-процесса, который вы построите по

дизайну. Готовы ли постановки, прототипы, макеты? Протестировали ли макеты и т. д. На каждый дизайн-спринт удобно заводить отдельную доску. Тут же удобно использовать чек-листы.

По ходу работы отслеживаем степень готовности дизайна каждой User Story.

Я не вижу смысла делать одну общую доску с дизайнерами и программистами и пытаться синхронизировать спринты. У них разные ритмы, Burndown, разные критерии готовности (DOD или Definition Of Done).

Это как два разных станка, которые из-за разной скорости работы не получится поместить в конвейер. Нужен будет склад готовой продукции (буфер запаса) для программистов. Это наши готовые, но несверстанные макеты. Слишком большой буфер не нужен: макеты могут «испортиться» из-за изменившихся требований. Слишком маленький — тоже плохо, поскольку может привести к простоям команды разработки.

В общем, UX-бэклог ведем отдельно. И время от времени пополняем из продуктового.

6.10.7 ПАРАЛЛЕЛЬНЫЕ ПОТОКИ. ПОЧЕМУ НА ПРАКТИКЕ ЭТО СЛОЖНО?

На картинке и в теории все понятно. Но как доходит до дела — за- просто забуксовать. Сценарии такие:

1. Технические ошибки в дизайне

От них никто не застрахован. Однако, если таких ошибок много — разработка забуксует. И чем отчаяннее попытки исправить все на ходу, тем больше застревает команда.

Решение мы разобрали в § 6.9.6. Опаздываешь — тормози. Сбросить скорость. Аварийно остановить разработку. Провести ретроспективу. Вернуться к предыдущим этапам. Сделать все на совесть: оттестировать макеты, провести разбор типовых ошибок, дополнить и выполнить чек-листы.

2. Перфекционизм

С заказчиками-перфекционистами параллельных работ вести нельзя. Кстати, не факт, что они вообще собираются выпускать продукт на рынок.

3. Хотелки, требования, обратная связь

Хотелки появляются как во время работы с реальной системой, так и просто по ходу времени.

Попытка внедрять правки одновременно и в дизайн, и в тут же разрабатываемый код приведет к катастрофе. Все запутаются, будет десять тысяч ошибок.

Стоимость исправления растет чуть ли не по экспоненте в зависимости от того, сколько уровней затронет переделка. Строительные примеры — менять планировку здания после чистового ремонта. Или добавить лифтовую шахту после стройки.

Что делать?

1. Собирать хотелки в backlog, как при классическом скраме. Дать разработчикам закончить текущую версию. Оттестировать и принять все по старым требованиям. И только после этого запустить итерацию по изменениям.
2. Увеличить объем партии на дизайне. Так, чтобы дизайн успел устанавливаться. То есть, по сути, снизить степень параллелизма и тратить чуть больше времени на согласование и тестирование макетов.

4. Сломанная коммуникация

Дизайнеры косо смотрят на менеджеров и заказчиков. Заказчики недовольны недодуманностью («а почему вы сразу не предусмотрели?!»). Программисты недовольны всем на свете, потому что стоят в конце конвейера и исправляют косяки за всех. Пользователи ворчат, минуют рейтинг и не платят.

Важно, чтобы программисты и дизайнеры могли наладить между собой прямой контакт. Встречались, обсуждали макеты. Удобнее всего делать это, если все люди in-house. Когда можно собраться и оперативно переговорить у монитора либо разработчика, либо дизайнера. Поэкспериментировать. И сразу сделать микроизменения, принять микрорешения. Все под одной крышей. Удаленное общение работает сильно хуже. Но, в принципе, тоже возможно.

Итак:

1. Чем больше параллельных дел, тем выше сложность в управлении, больше потерь на синхронизацию и коммуникации и выше стоимость доработок.

2. Если чувствуете, что все начинает выходить из-под контроля — сбрасывайте скорость и снижайте количество параллельных работ. Сокращайте work in progress.
3. Настраивайте прямое общение между участниками проекта и пользователями. Заставляйте дизайнеров общаться с программистами напрямую, а не через бэклоги, тикеты и чатики. Никакие процессы не заменят прямого общения людей.

6.11 СТРОИМ ДИЗАЙН-КОМАНДУ

Если речь идет не про разовый проект на одну ночь, а о долгосрочной совместной работе — придется заниматься выращиванием дизайн-команды. Подробно про развитие команд поговорим в главе 7. Здесь отмечу нюансы, касающиеся дизайнеров. Я считаю, что с технической точки зрения здесь больше ответственности у арт-директора.

Однако у руководителя проектов могут быть полномочия на подбор. Или, как минимум, если на вас лежит ответственность за проект, вы можете получить право не работать с теми дизайнерами, которые вам не нравятся. Как получить такое право — можно почитать в книге Павла Сивожелезова «Сложные переговоры с подчиненными». Допустим, оно у вас есть. На что нужно обратить внимание?

6.11.1 РЕЗЮМЕ. ПОРТФОЛИО. СОБЕСЕДОВАНИЯ. ТЕСТОВОЕ ЗАДАНИЕ. ОНБОРДИНГ. ИСПЫТАТЕЛЬНЫЙ СРОК. СТАЖИРОВКИ И ДЖУНЫ

В резюме я обращаю внимание на профильный опыт и грамотность. Приоритет — это портфолио. Диплом можно выкинуть. Онлайн-курсы ни о чем не говорят. Пол, возраст, религия и т. д. — не волнуют.

В портфолио смотрю релевантность работ нашим задачам. Общий уровень, аккуратность, масштабируемость макетов (в том плане, что можно будет с ними потом работать).

В портфолио юные дизайнеры пытаются выехать на сочных картинках. Для заказчиков это по праздникам прокатывается. Для подбора кадров — нет. Отделите контент от самой работы.

На собеседованиях с дизайнерами одна проблема: непонятно, о чем их спрашивать. Я смотрю на общий уровень адекватности, эрудиро-

ванность, позитивность, энергетику. Сработаемся-нет. И мы вместе обсуждаем портфолио: что можно было бы улучшить, какие ошибки видит кандидат.

Хорошо, когда макеты есть не только в виде картинок, но и в уже работающем виде: интересно посмотреть, какие изменения были сделаны в процессе и почему. «Ну вот тут заказчик правками все испортил» или «Ну вот тут программисты сделали не так, как я хотел» — готовьтесь, на ваших проектах будут похожие обвинения и жалобы. Все идет через задницу почему-то у одних и тех же людей.

Считаю, что тестовые задания нужны. Релевантные тем задачам, которые будем решать. Они не должны быть слишком большие. Два полноценных вечера — примерно так.

Давать ли тестовые задания до собеседований — зависит от многих факторов. Если желающих много — да. Если не очень — лучше сначала встретиться-поговорить.

Но и тестовые задания, и портфолио, и подробное интервью не гарантируют, что, когда дело дойдет до дела, все будет как надо. На испытательном сроке старайтесь как можно быстрее подключить дизайнера к реальным процессам и задачам.

Хорошо, когда есть ясная процедура онбординга. Покажите и расскажите, как и что у вас устроено. И вперед.

Важны первые успехи. Чтобы с первых дней начало получаться. Первые работы и макеты удавались. Давайте реальные, но небольшие задания. Выделяйте время на обратную связь и брейнштурмы. Бросить просто так в новичка салфеткой с каракулями, сказать «ну пусть дизайнер подумает» и ждать от него успехов — ну такое... Как притретесь, поймете, что ожидать друг от друга.

Следите не только за качеством работы, но и за дисциплиной. Если на управление дизайнером нужно сильно больше времени, чем в среднем, — это повод поговорить. Попробовать исправить. По идее время на управление дизайнером должно сокращаться по мере того, как он глубже погружается в процессы. Если этого не происходит, надо разбираться, искать причины. Если ничего не помогает, есть закидоны и неуправляемость — легче вовремя расстаться.

Стажировки работают. Входные требования к стажеру ниже, чем к кандидату. Но это дорогое мероприятие. Нужно больше времени на кураторство. И задачи нужно давать такие, которые безопасно профа-

капить. Но можно вылепить дисциплинированного и четкого дизайнера. Кроме того, работая со стажерами, прокачиваются и сами кураторы. Двойная выгода. Если политика компании это позволяет — составьте программу прокачки дизайнеров и стажировок.

6.11.2 АРТ-ДИРЕКТОР ≠ МЕНЕДЖЕР

Как только артель дизайнеров перерастает два-три человека, нужно одного из них назначить главным. Во избежание споров. самого толкового, авторитетного и опытного. Его и назначаем арт-директором. Говорят, в армии по двое до ветру ходят, и то главного назначают.

Руководитель проекта отвечает за весь проект целиком. Арт-директору можно делегировать:

1. Помощь дизайнеру. В любых ситуациях: когда не хватает технического уровня, знаний, примеров, не получается по уровню креатива и т. д.
2. Проверки и помощь по эстетической стороне вопроса. Что красиво, а что — нет. Эстетическое качество управляется через примеры. Поэтому так важна насмотренность.
3. Проверка по техническому качеству макета.
4. Брейнштурмы в части придумывания креатива.
5. Расфакапливание в критических ситуациях (подмена дизайнера, отрисовка макетов его рукой и т. д.).

Что нельзя делегировать артдиру:

1. Контроль сроков.
2. Полноту выполнения спецификаций.

Обратите внимание: менеджер делегирует часть функций арт-директору, а не арт-директор использует менеджера как говорящую прокладку между дизайнером и клиентом.

Конечные решения — можно показывать дизайн миру или нельзя — остаются за руководителем проекта. Ответственность на нем. Ему и решать.

Да, вопрос «кто главнее» — щепетильный. И многое зависит от самой компании. Исторических факторов. Культуры. Регламентов. И людей. Но моя религия говорит, что у кого выше ответственность,

у того и должно быть больше полномочий. Любители «влиять, но не отвечать» — вредный балласт. За борт.

Относитесь к арт-директору как к эксперту. Помощнику. Консультанту, если хотите. Используйте его знания и опыт, чтобы получалось делать руками других дизайнеров всякие крутые штуки и прокачивать их уровень. Но не превращайте арт-директора в еще одного менеджера.

6.11.3 ДИЗАЙН-БАТТЛЫ И СТЕРОИДЫ

Работа занимает все запланированное время (закон Паркинсона). И немного больше (закон Мерфи). Следствие: любую задачу можно сделать за час. Шутка. Почти.

Это проверялось неоднократно. Но в это сложно поверить. Можно нарисовать дизайн за один час. От постановки задачи до «пошли посмотреть».

Да, в нем будут косяки, огрехи, и это вряд ли поедет на Каннский фестиваль. Фишка не в этом. А в том, чтобы дизайнер посмотрел, попробовал сам и понял, что есть другая парадигма: не полировать микроны по две недели. А сделать работу целиком за один час. Кстати, об этом же говорит метод прогрессивного джипега Лебедева (о нем мы говорили в пункте 6.7.6).

Что предлагается. Раз в неделю (лучше в пятницу вечером) даем дизайнерам произвольную тему. Нарисовать сайт ко Дню космонавтики, например. И час времени. Через час ребята презентуют работы. Голосуем, выбираем лучшую. Только заранее определите критерии. Иначе побеждать будут самые шибанутые.

Сначала будет казаться, что нереально сделать что-то за час. Потом будет очевидно, что успеваем и придумать, и что-то нарисовать, и даже тексты написать.

6.11.4 ОБУЧАЛКИ И СМОТР РАБОТ

Используйте карты компетенций, чтобы понимать и прокачивать уровень своих ребят. Пример такой карты вы найдете в Приложении 1, а о принципах работы с ними мы поговорим в главе 7.11. Отмечу здесь нюансы.

Прокачивать владение разными инструментами по принципу «чем больше, тем лучше» смысла нет никакого. Общее представление получить

можно. Но все быстро забывается без практики. Например, если ваш дизайнер доблестно осваивал 3D-Max в течение пары месяцев и даже сделал что-то коммерческое, а затем полгода не практиковался — это, скорее всего, значит, что он не в теме. Стерто-стерто-стерто. То же самое с моушн-дизайном. Есть смысл расти в глубину, а не хватать разные методики и техники, пробовать их и бросать.

Заведите обучающие мероприятия для дизайнеров. Смотрите видео, доклады, тренды. Организуйте обмен опытом. Дайте дизайнерам подготовить доклад по недавно изученной теме. Пусть расскажут, над каким проектом работали. Покажут. Обсудят, как все шло. Это можно чередовать с дизайн-баттлами, просто выделив час времени в пятницу вечером.

Из навыков, которые имеет смысл дизайнерам прокачивать всегда: навыки презентации, самоорганизацию, публичные выступления. То, что называют Soft Skills. Поможет на демонстрациях макетов.

Вообще же, чем начитаннее, насмотреннее и опытнее дизайнер, тем шире круг вопросов, которые он может решить самостоятельно. А этим и определяется ценность сотрудника.

6.12 DATA DRIVEN DESIGN: КАК ИСПОЛЬЗОВАТЬ ДАННЫЕ В ДИЗАЙНЕ И МОЖНО ЛИ ПЕРЕЛОЖИТЬ ОТВЕТСТВЕННОСТЬ ЗА КРАСОТУ В МАКЕТАХ НА ЦИФРЫ

Каждую секунду в 2020 году среднестатистический пользователь создавал 1,7 Мб информации — представляете, сколько всего можно «вынуть» из этих данных!

Маркетологи довольно потирают руками, а дизайнеры-то тоже не отстают: с 2015–2016-х годов тема Data Driven-дизайна, заряженного данными, будоражит умы творцов. Больше никаких правок из-за вкусовых разногласий — только цифры, только хардкор. Но вообще-то в гонке за проверкой дизайн-решений цифрами можно увязнуть и выгореть дотла.

В этом параграфе поговорим о том, что такое Data Driven Design, когда он пригодится, а когда лучше довериться дизайнерской интуиции, какие есть нюансы и как вообще собирать данные и строить на их основе рабочие гипотезы.

6.12.1 ЧТО ТАКОЕ DATA DRIVEN DESIGN И КОГДА ОН ПОЯВИЛСЯ

Data Driven Design — дизайн на основе аналитических исследований, когда нельзя просто так взять и предложить цветовую схему или расположение элемента на странице: сначала это нужно проверить и подкрепить цифрами. Такой дизайн тесно связывают с конкретными метриками: если они достигаются, дизайн хорош, если нет — надо что-то менять (и опять исследовать и доказывать это цифрами).

До его появления дизайнеры принимали решения, исходя из своего опыта, насмотренности, своих представлений о дизайне, своего дизайнерского «почерка» или из интуиции, в конце концов. Частая проблема такого подхода в том, что дизайнеры страдают от эффекта ложного консенсуса и проецируют свои реакции и поведение на странице на потенциальных пользователей — то есть, принимают решения на основе собственного опыта. Хотя далеко не факт, что все юзеры будут делать и думать так же.

Отсюда растут корни понятия «вкусовщина». Клиент мог принимать дизайн, а мог говорить, что «что-то не то» — и пойдешь объяснять, почему кнопки такие, а блоки располагаются так-то: железных аргументов, кроме «я художник, я так вижу», не было.

И тут на сцене появляется Data Driven Design, призванный навсегда уничтожить споры о дизайне между клиентом и дизайнером, а еще подать на блюдецке желания пользователей. Сначала вы исследуете, что вообще нужно юзерам, потом получаете конкретные данные — останется только делать так, как они велят.

Звучит заманчиво, ведь при таком подходе дизайн всегда обоснованный: никакой интуиции не надо, никаких правок из-за разницы вкуса тоже. Но это только на первый взгляд. На второй — это дорого для заказчика: каждый штрих нужно подкреплять исследованиями, и не всегда достаточно какого-нибудь A/B-теста. А поскольку улучшения непрерывны, исследования становятся постоянной статьей расходов. Другая проблема касается уже команды дизайнеров — если слепо верить цифрам, можно расслабиться, снять с себя всякую ответственность или потерять всякий вкус к своей работе. По интернету гуляет цитата ведущего дизайнера Гугла, который не выдержал этих бесконечных проверок по каждой мелочи и уволился:

«Да, это правда, что команда Google не могла выбрать между двумя оттенками синего, и поэтому тестировала 41 оттенок, чтобы увидеть, какой из них лучше. У меня недавно была дискуссия о том, какой должна быть обводка: 3, 4 или 5 пикселей, и мне было предложено доказать мой выбор. Я не могу работать в такой среде. Я устал обсуждать такие незначительные дизайнерские решения. В этом мире есть более интересные проблемы с дизайном».

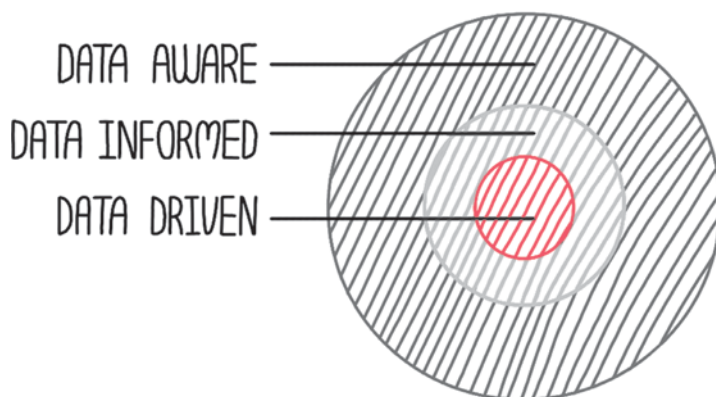
6.12.2 НЕДОСТАТКИ DATA DRIVEN ПОДХОДА

1. Данные не скажут вам «делай так и так, чтобы решить проблему» — и это самое главное разочарование для тех, кто хотел переложить ответственность за свои решения на цифры.
2. Чтобы собирать данные, нужна инфраструктура для их сбора: сотрудник, который этим занимается, ПО для сбора, какое-то ПО для систематизации полученной информации, регулярная аналитика и выработка гипотез. А еще тесная работа с командой: потому что зачем вам данные, если вы никак их не применяете. Готовьте кошелек, иными словами.
3. Сотрудники, способные адекватно интерпретировать данные, на рынке труда стоят дорого (а еще их нужно постоянно обучать, потому что сфера стремительно развивается — один только ИИ насколько влияет).

6.12.3 ОТКУДА ВЗЯЛСЯ ТЕРМИН

О подходе Data Driven стали говорить еще в 90-е, и тогда он казался программированием — это была альтернатива **функционального** и **объектно-ориентированного программирования**. С ростом объема данных, развитием аналитических систем и рынка digital в целом подход Data Driven проник и в другие сферы: маркетинг, менеджмент, обучение и дизайн.

Подходы к использованию данных в дизайне хорошо описаны в книге Рошель Кинг, Элизабет Черчилль и Кейтлин Тан «Дизайн на основе данных».



Три подхода к использованию данных в дизайне

Data Driven — дизайн, управляемый данными

Когда решения принимаются исключительно на основе количественных данных — то есть данные имеют первостепенное значение. Подход годится, если нужно что-то оптимизировать: понять, какой вариант целевой страницы эффективнее или какие метрики дают прирост конверсий.

Data Informed — дизайн с учетом данных

Более гибкий подход, чем предыдущий: мы смотрим на данные, но при этом можем с ними не соглашаться, потому что также доверяем своему опыту, интуиции и качественным данным (о том, как собирать количественные и качественные данные, поговорим ниже).

Data Aware — дизайн, осведомленный данными

Когда полученные количественные данные — не единственный фактор принятия решения, и команды работают в совокупности с другими факторами. То есть, у вас есть широкий диапазон данных, вы понимаете ограничения для их сбора и выбираете ту методологию, которая лучше всего подходит под конкретную задачу.

6.12.4 DRIVEN, INFORMED, AWARE — КАКОЙ ПОДХОД ВЫБРАТЬ?

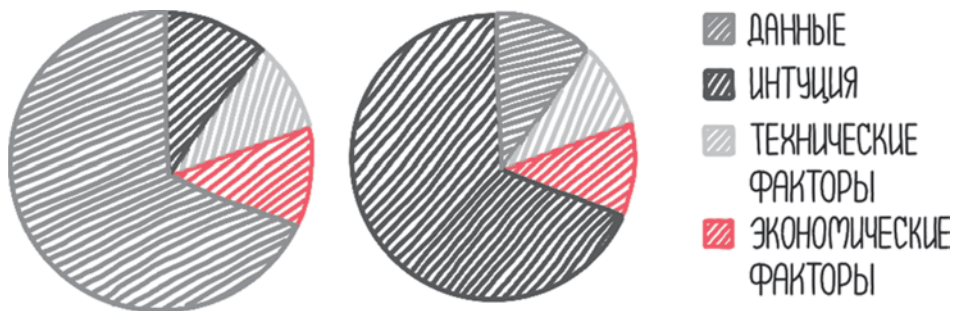
Какой из подходов лучше, однозначно не скажешь — как всегда, все зависит от задачи и обстоятельств. На самом деле, чтобы определиться, достаточно ответить на три вопроса:

1. Каков контекст?

Если вы внедряете что-то инновационное — подойдет подход Data Informed, потому что создавать новое, опираясь на данные прошлых периодов, сложно. Просто потому что непонятно, что брать за ориентир. Если же вы проверяете гипотезу, используйте Data Driven — подход четко покажет, «зашло» ваше решение пользователям или нет.

2. Какова глубина решаемой проблемы?

Хотите обновить дизайн целевой страницы? Или чувствуете, что достигли потолка с текущим товарным ассортиментом и пора менять что-то глобально? В первом случае неплохо бы начать с анализа данных: что там происходит с воронкой продаж, как люди заходят на страницу, регистрируются и совершают целевые действия, и есть ли вообще проблема с конверсией.



Разные соотношения факторов при принятии решений в разных ситуациях: слева — основная опора на данные, справа — на интуицию (по Крису Ли).

Данные или интуиция?

- ▶ Если вы выбираете между двумя или тремя вариантами — смотрите на данные: в помощь старый добрый А/В-тест. Только убедитесь, что вы смотрите на долгосрочные показатели ценности, а не только на просмотры и клики.
- ▶ Если качество и эстетика продукта страдают, доверьтесь интуиции дизайнера, ведь общее ощущение качества — это совокупность сотен микрорешений, согласованность и точность исполнения. Пользователи не эксперты по дизайну, поэтому их мнение будет слишком субъективным.

- ▶ Если есть сомнения в поведении пользователя — используйте данные, а не спрашивайте мнение юзеров. Люди склонны делать то, что вы от них ждете, когда вы их спрашиваете об этом напрямую. Доверяйте тому, что они делают, когда думают, что их никто не видит.
- ▶ Если вам нужно создать бренд или повысить репутацию, данные почти бессильны в измерении этих параметров. Доверяйте интуиции и опыту дизайнера и не смотрите на краткосрочные результаты: это игра в долгую.

3. Какова макроцель?

Всегда стоит соотносить затраты на получение данных с их ценностью при принятии решения. Да, чем больше данных, тем обоснованнее становятся решения, но и стоимость этих решений возрастает. Вам нужно определиться, сколько данных будет достаточно, чтобы достичь цели.

Например, ваш сайт с мобильного выглядит отвратительно — и, если какой-то влиятельный клиент поделится своим негативным опытом его посещения с подписчиками в соцсетях, ваша репутация пострадает. Можно судорожно собирать всевозможные данные, чтобы разобраться в проблеме (и потерять за это время всякое доверие), а можно постараться найти рабочее решение как можно скорее, пусть оно и будет пока интуитивным.

Но это не значит, что в другой ситуации вам тоже придется пренебречь данными — возможно, риск и последствия ошибки будут настолько высоки, что лучше семь раз отмерить, как в поговорке.

6.12.5 СБОР КОЛИЧЕСТВЕННЫХ ДАННЫХ

A/B-тест

Классический способ сравнить, какой из двух вариантов лучше: вы просто делите аудиторию на две приблизительно равные группы и одной показываете одну версию, а другой — другую. У какой будет выше конверсия, та и работает эффективнее.

Многовариантный тест

Здесь меняют не один элемент, как в A/B-тесте, а сразу несколько, например, всю шапку страницы. Такое тестирование определяет, какая комбинация элементов работает лучше остальных.

Аналитика

Уже привычные всем Google Analytics и Яндекс.Метрика расскажут, куда люди кликают, откуда приходят, как долго остаются на сайте и как часто на него возвращаются. Но при этом можно пойти дальше и смотреть внутри аналитических систем и дополнительные параметры, вроде глубины и типов скrolла (горизонтальный или вертикальный). Настройку целей тоже никто не отменял: действия и бездействия, взаимодействия с формами и контентом, скачивание файлов, просмотры видео и все такое прочее.

Перед созданием прототипа и этапом дизайна наши аналитики разговаривают с заказчиком о пользователях, погружаются в веб-аналитику, изучают конкурентов — все это дает полную картину поведения пользователей и трендов отрасли, которую дизайнеры берут за основу в своей работе.

Тепловые карты

Вы можете заморочиться и не только изучать данные яндексовского Вебвизора с картами кликов, но еще и подключить айтрекинг (сразу предупреждаю, что адекватное оборудование — это очень дорого).

Опросы

Опросы используют для сбора и количественных, и качественных данных. Но для качественных их использовать все-таки проще, потому что во многих случаях вопросы, которые приведут к количественным, а не к качественным данным, подготовить проще.

6.12.6 СБОР КАЧЕСТВЕННЫХ ДАННЫХ

Количественные данные дадут понять, что именно люди делают. А вот качественные расскажут, почему они делают именно так.

Анализ конкурентов

Пока вы постоянно улучшаете продукт, конкуренты тоже не бездельничают. Поэтому за ними нужно периодически подглядывать. А вот слепо копировать не стоит — крадите как художники :)

Интервью

Это тоже недешевое удовольствие: нужно потратить время своих специалистов на поиск людей, составление адекватных вопросов, на сами беседы, на обработку результатов. Но оно того стоит, потому что такую глубокую информацию сложно получить из простого онлайн-опроса.

Customer Journey Map и User Flow

Customer Journey Map помогает осмыслить визуально, как пользователи взаимодействуют с продуктом. User Flow — это то, как пользователи достигают своей цели на сайте: от момента первого скролла до ухода. Обычно вы более-менее предполагаете шаги вашего User Flow, но фактические потоки могут сильно отличаться от ожидаемых, и это может указывать на проблему с пользовательским интерфейсом. Самый простой способ это проверить — попросить группу пользователей выполнить конкретную задачу и наблюдать, как те будут действовать. Эта информация поможет определить потенциальные слабые места и даст задел для дальнейших исследований с помощью A/B-тестов или интервью.

Отзывы из соцсетей

В соцсетях люди любят описывать свой опыт взаимодействия с продуктом, особенно если тот был негативным. Для вас это больно, но полезно — так можно определить узкие места, понять ожидаемые требования к продукту и улучшить его. А заодно повысить NPS — индекс приверженности потребителей товару или компании.

6.12.7 КАК ИСПОЛЬЗОВАТЬ ПОЛУЧЕННЫЕ ДАННЫЕ

Собираемые данные могут помочь в четырех направлениях:

1. проверка — удостовериться, что вы на верном пути;
2. предвосхищение — узнать о потенциальных возможностях и болевых точках;
3. исследование — открыть новые паттерны поведения, тренды и предпочтения;
4. улучшение — совершенствовать дизайн, опираясь на объективность и итеративный подход.

Исследование MIT Technology Review показало, что компании, стоящие в первой трети рейтинга в своей отрасли, используют данные при принятии решений — и за счет этого они в среднем на 5% более продуктивны и на 6% более прибыльны, чем конкуренты, которые так не делают.

Качественные исследования отлично подходят для понимания контекста, но гораздо хуже — для сегментации пользователей или выявления тенденций в больших объемах данных. Хорошая новость в том, что их можно объединить, чтобы получить более полную картину по целевым персонам.

Чтобы создать качественные целевые персоны для своего продукта, вам нужно:

- 1) Провести интервью — запланируйте короткие интервью с несколькими клиентами, чтобы выяснить контекст: что для них важно, как они себя ведут в продукте, какая мотивация его использовать. Пока не сегментируйте, а просто собирайте данные. На основе интервью составьте список из 15–25 атрибутов, характерных для групп.
- 2) Составить опрос — сформулируйте атрибуты в виде вопросов. Для атрибута «зарплата» вопрос может быть: «Какова ваша зарплата?», а для «поездка в супермаркет» — «Как часто вы посещаете супермаркет?». Старайтесь соблюсти баланс между поведенческими и мотивационными вопросами, задайте также два-три открытых вопроса о болевых точках и оставьте поле для адреса электронной почты. Чтобы проверить адекватность вопросов, можно почитать о CustDev (см. 4 главу) или использовать модель BRUSO.

BRUSO — модель, в основе которой пять параметров:

- ▶ Brief — краткость;
- ▶ Relevant — актуальность;
- ▶ Unambiguous — однозначность;
- ▶ Specific — конкретность;
- ▶ Objective — объективность (соответствие задаче).

- 3) Отправить опрос целевой аудитории — вставить в рассылку по электронной почте, разместить баннер на вашем сайте или просто подкинуть задачку по поиску людей вашему HR-менеджеру :)
- 4) Сегментировать клиентов.
- 5) Снова провести интервью — но теперь уже выбрать собеседников из числа тех, кто прошел опрос и попал в какую-то группу пользователей. Спрашивайте об их поведении и мотивации и докапывайтесь до первопричин, почему они действуют так, а не иначе. Полученные ответы станут базой для создания Data Driven персон.

6.12.8 КАК ПРОВЕРИТЬ ГИПОТЕЗУ С УЧЕТОМ ДАННЫХ — МОДЕЛЬ ИЗ ПЯТИ ШАГОВ (ПОДХОД DATA AWARE)



Модель из пяти шагов для проверки гипотез, ориентированных на данные (оригинал схемы принадлежит Аарону Гитлину, Interaction Designer в Google)

1. Цель

Конкретная и четкая — например, увеличить доход, не прибегая к серьезной доработке функционала.

2. Проблема или возможность

Вероятно, на достижение цели повлияет сразу несколько факторов. И для этого надо понять, какие параметры на нее влияют, и нет

ли с ними проблем? Например, где люди «вываливаются» из воронки продаж. Или как всегда используют ваш сайт. Сюда же опросы, заявки в техподдержку и прочие имеющиеся количественные и качественные данные.

Техника для исследователей 80/20: искать такие данные, которые доказывают, что ваши предположения неверны.

3. Гипотеза

Хорошая гипотеза на дороге не валяется — у нее должна быть четкая метрика для оценки и видимый путь к этому измеримому результату. Гипотеза строится по принципу:

Для [группы пользователей], если [изменение], то [эффект], потому что [обоснование], которое повлияет на [метрика], где:

- ▶ [группа пользователей] — сегмент оцениваемых пользователей;
- ▶ [изменение] — то, что вы планируете сделать;
- ▶ [эффект] — каким будет результат по вашему мнению;
- ▶ [обоснование] — почему вы думаете, что эффект вообще будет;
- ▶ [метрика] — измеримый результат, который вы ожидаете увидеть.

Пример гипотезы: если для всех посетителей сайта, просматривающих страницу продукта, выделить красную кнопку «Оформить заказ», это увеличит клики на нее, потому что красный цвет ассоциируется с ощущением срочности, а это приведет к увеличению конверсии.

Дополнительно вам стоит подумать и зафиксировать, что вы узнаете, если гипотеза окажется неудачной, как этот опыт можно применить дальше и сколько ресурсов стоит вложить в этот эксперимент.

4. Тест

Адекватный эксперимент — это тест, где:

- ▶ вы контролируете переменные, чтобы понимать разницу между вариантами;

- ▶ варианты четко представляют гипотезу;
- ▶ предмет эксперимента является значимым и этичным (проверяем реальную гипотезу и не тестируем то, что возмущает пользователя — скажем, скрытые расходы или доплаты).

5. Результат

Измерьте результаты, сделайте выводы. Если гипотеза не оправдала себя, что полезного вы вынесли? Если оправдала — как это использовать дальше?

Итого. Данные не панацея. Некоторые цели трудно измерить, а некоторые данные сложно собрать. Если задавать неправильные вопросы, спрашивать не в то время и не тех людей, можно сделать неверные выводы. Хорошо зная свой продукт, вы перестаете видеть его объективно.

Но принимать проектные решения с данными — всегда проще. Конкретные цифры и гипотезы на основе качественных и количественных исследований полезны для постепенных, тактических изменений — правда, только если они проверяются и уравниваются дизайнерским инстинктом и здравым смыслом.

Данные — это, конечно, хорошо, но они никогда не скажут вам, в чем именно проблема и как ее решить. Но они точно помогут выявить узкие места и оценить эффективность решений.

6.13 ЛАЙФХАКИ, ВОПРОСЫ И ОТВЕТЫ

У меня осталось несколько советов и рецептов по ситуациям, которые возникают в работе с дизайнерами. По большей части они касаются заказной разработки. Их сложно структурировать. Поэтому просто поделюсь рецептами.

6.13.1 ФЛАГМАНСКИЕ ПРОЕКТЫ. КОНТРОЛИРУЕМЫЙ ФАКАП

Мы покупаем целлофан.
джаст фо фан

В работе на потоке заказов есть один минус. Как ни крути — это конвейер. Сложный. Хитрый. Но конвейер. Вообще любой отлаженный бизнес — это конвейер. И это — нормально. Но иногда мы его специально ломаем. Берем какой-то проект. И внутри себя решаем, что на этом проекте мы можем себе позволить бесконечное количество ресурсов. Зеленый свет на все. Любые технологии. Если надо — любое количество итераций. Полировок. Подгонов. Надо выложиться.

Это как отказной сет в качалке. Делать так на всех проектах не получится. Не хватит ресурсов. Сдохнете. Но иногда...

Какие проекты брать? Те, которые нравятся лично вам. Могут быть конкурсными, зрелищными, фестивальными. Те, что прокачивают команду. И где сложились правильные отношения с заказчиком. Что значит «правильные»? Скажем так: он не портит правочками работу.

Может ли заказчик сказать, что именно ему нужен именно такой подход? Да. Но это так не работает. Звезды должны совпасть.

Итак. Время от времени делайте флагманский проект. Сильно выходящий за границы компетенций и привычное качество работы команды.

6.13.2 MAKE MY LOGO BIGGER!

Кто не видел — поищите ролик «Make my logo bigger» на видеохостинге.

Можно ли «улучшать» логотип при разработке концепции? Я имею в виду ситуацию, когда при разработке, например, сайта, дизайнер решил немного заапгрейдить логотип клиента: написать другим шрифтом, убрать устаревшую херабору, улучшить логотип на 147% и т. д. Возможно, клиент на берегу, при обсуждении мудборда, сам был не против таких улучшений.

Так вот. Изменить логотип — это лучший способ провалить концепцию. У собственников к этому очень часто трепетное отношение. И не надо лишний раз провоцировать.

Что делать, если очень хочется? Сделайте это на вторичном, альтернативном варианте. И показывайте ненавязчиво. Обязательно

в презентации первым должен идти вариант с оригинальным, «не улучшенным» логотипом.

У меня были случаи, когда сначала на изменение логотипа было категорическое «НЕТ!!! МЫ РАДИ ЭТОГО ЛОГОТИПА 25 ЛЕТ РАБОТАЛИ!!!». А потом, через год, по-тихому, логотип менялся на то, что было в альтернативном варианте. Но это год прошел. Надо было свыкнуться.

Итак. Хотите «улучшить» логотип — делайте это на альтернативном варианте. И как-то ненавязчиво.

6.13.3 ДИЗАЙН ИЛИ ФИРМЕННЫЙ СТИЛЬ: ЧТО РАНЬШЕ?

Что угодно, только не параллельно и то, и то!

Если у клиента нет, например, логотипа — перспективы такие:

1. Пойти и сделать фирменный стиль. Но это может быть достаточно дорого. И, с учетом согласований, занимает три-шесть месяцев. Иногда — дольше. Хотя на берегу кажется «чего там? За пару недель все согласуем». Практика показывает иллюзорность такого прогноза. Там куча факторов, почему происходит именно так. Но происходит именно так.

Есть ли резон замораживать разработку на такой срок? Не всегда. Надо считать деньги. Я видел конфликт коммерческого директора с директором по маркетингу в духе «пока вы там цветочки причесываете — я миллионы теряю». В общем, посчитайте, сколько и чего вы потеряете, если застрянете на выборе шрифта в логотипе на полгода. Устраивает? Тогда да.

2. Разработать концепцию с текстовым логотипом. Просто аккуратно, текстом, написано название компании. Гармонично вписано в концепцию. Ну, может быть, какой-то значок добавлен. Все понимают, что это «рабочка», MVP. Денег дополнительно за это брать не надо. Но и «правочек» не будет.

А затем, по желанию, разрабатывать фирменный стиль **на основе концепции**. Если будет разработка нового логотипа — **рисовать его сразу с примеркой в дизайн**.

Подход работает хорошо. Он дешевый, быстрый и дает возможность почти мгновенно запустить продажи. В моей практике был случай, когда в рамках отрисовки дизайна сайта был придуман не только логотип, но и название, которое очень понравилось заказчику. С этим и жили. Хотя это, скорее, курьез.

Когда может не сработать? Когда ушли разрабатывать логотип, и забили (забили) сразу примерять его в концепцию. Вот тут уже можно влететь на серьезные переделки.

Итак. Если тема «логотипа» очень щепетильна — это ставит под угрозу все сроки проекта. И часто весь проект — на паузу. Если полгода ждать не хочется — делаем текстовую версию логотипа и живем с ней. Утверждаем концепцию. Запускаем в разработку. Затем, если будет желание, можно отдельно разработать логотип и фирстиль. Но так, чтобы он вписывался в уже выполненную концепцию. Иначе можно застрять на месяц-другой на переделках и проверках.

6.13.4 ФОТОСТОКИ, ЛИЦЕНЗИИ И ПРАВА

По количеству пиратов Россия обгоняет Сомали — еще бы, у нас пиратом быть намного проще и приятнее. Жмешь «Скачать», и вжух — ты уже нарушил авторские права.

— Почему меня это касается?

Потому что у каждого изображения есть автор, а у него — авторское право. Оно появляется вместе с изображением и исчезает только через 70 лет после смерти своего создателя. Тогда картинка становится общественным достоянием, которым можно беспрепятственно пользоваться.

Пока автор жив, вам нужно договариваться с ним, чтобы использовать изображение. Если не сделаете этого, автор имеет полное право обратиться в суд, который будет на его стороне. Даже если речь идет о фоточках из соцсети.

В 2013 году издательский дом «Аргументы и факты» использовал фотографии пяти блогеров без их разрешения. Блогеры обратились в суд с требованием заплатить за каждый использованный ИД «АиФ» снимок 20 000 рублей. А всего их было 207 штук. Блогеры выиграли дело и даже получили бонус: компенсацию морального ущерба. «Аргументы и факты» выплатил пострадавшим более 2 000 000 рублей.

Чтобы не платить за нарушения при использовании изображений, нужно знать свои возможности. Для этого либо обсудите их с автором, либо смотрите на лицензию и радуйтесь, если там Creative Commons.

Наличие Creative Commons означает, что изображение можно распространять без переговоров с автором. Но с некоторыми ограничениями, которые зависят от вида лицензии.

CC0 — самая желанная пометка. Означает, что автор отказался от авторских прав, и изображение стало общественным достоянием. Его можно использовать в коммерческих целях, менять и распространять без указания авторства.

CC BY — позволяет редактировать изображение, брать за основу других произведений. Подходит для коммерческого использования. Работает так же, как CC0, только обязывает указывать авторство.

CC BY-SA — вид лицензии, который позволяет все вольности CC BY, но с одним условием. Творения, в основе которых лежит изображение с лицензией CC BY-SA, автоматически приобретают такую же лицензию. Это значит, что вашу картинку могут взять, отредактировать и, например, поставить в рекламу. Потому что имеют полное право.

CC BY-ND — лицензия, которая разрешает распространять изображение, в том числе в коммерческих целях. Но запрещает редактировать. Обязательно указание авторства.

CC BY-NC, CC BY-NC-SA, CC BY-NC-ND — изображения с этими лицензиями вряд ли пригодятся дизайнеру, потому что запрещают коммерческое использование. Максимум, куда можно их пристроить — в презентацию для корпоративного капустника. Так что не будем подробно на них останавливаться.

— А что с платными фотостоками? Купленные там изображения можно использовать как угодно?

Нет. На платных фотостоках самый распространенный вид лицензии — Royalty-Free. Она позволяет использовать изображение неограниченное количество раз, но не более 500 000 на разных носителях. Получается, чтобы использовать фотографию в печатной продукции с тиражом более 500 000 экземпляров, придется доплатить за расширенную лицензию (Extended Royalty-Free).

Также изображение с Royalty-Free нельзя использовать на майках, кружках, ковриках для мышки — в общем, на всем, что продается. В книге в качестве иллюстрации — можно, на обложке блокнота для продажи — нельзя. Понятненько?

И еще один нюанс: покупая изображение с лицензией Royalty-Free, вы приобретаете право на его использование. Если вы делаете работу для заказчика, который в будущем хочет где-нибудь еще применить заужанную фотографию (визитки, реклама, фон рабочего стола и т. д.) — пусть сам ее и покупает. Иначе получится, что вы незаконно передали фотографию и нарушили договор с фотостоком. Чтобы этого избежать, выбирайте другой вид лицензии. Получится дороже: например, на iStock за эту возможность придется доплатить 18 долларов. У каждого стока — свои условия и заморочки по этому вопросу.

Вообще покупать фотографии со стоков руками исполнителя не всегда выгодно. Если фото покупается за валюту, то делается это в 99% с банковской карты сотрудника компании (никто не хочет заморачиваться с валютными счетами). Чтобы попасть на карту, деньги заказчика снимаются с расчетного счета, переводятся в рубли, перечисляются сотруднику — и на каждом шаге налог отъедает от них по кусочку. В итоге из-за снятий и конвертаций цена фотографии может вырасти на 50%.

— Как скачать фотографию и не нарушить закон?

Для этого есть простой, но верный алгоритм.

1. Если ищете картинку в Гугле — ставьте соответствующую галочку.

2. Но лучше — берите изображения со специальных сайтов. Мы как-то делали подборку платных и бесплатных фотостоков, изображения с которых можно качать и без опасений использовать.



30 фотостоков
на все случаи
жизни

3. При покупке или скачивании фотографии всегда смотрите на лицензию: убедитесь, что она разрешает использовать изображение так, как вам нужно.

6.13.5 ФОТО-ВИДЕО-3D-СТОКИ. КАК ДАВАТЬ ССЫЛКИ КЛИЕНТУ

Еще один тонкий момент, на котором молодые руководители проектов могут накосорезить. Итак. Клиенту в целом все нравится. Осталось поменять вон ту кошечку на более симпатичную.

И тут открываются врата ада. Все не то. Все не так. Вот уже и концепт не нравится. Очевидно, агентство — му-да-ки!

В чем косяк № 1? Скорее всего, угадать, что такое «более симпатичная кошечка» (или «более инновационный вид из окна») в голове клиента, вы не сможете. Потому что пока он не увидел, у него самого нет четкого представления. Это как стрелять по мишени, которой нет.

Нужно дать много вариантов. Чтобы человек сам смотрел и подбирал. Причем, лучше — не один, не два, не десять. А несколько сотен. Где есть несколько сотен картинок? Правильно, на фотостоках.

И тут случается косяк № 2. Менеджер может отправить клиенту ссылку на фотосток. Мол, вон там фотки, сходи поищи. Последствия обычно настигают и больно дают по попе.

Мало того, что это звучит, как предложение поискать себе еду в помойке. Это еще и показывает крайнюю незаинтересованность.

Что рекомендую сделать я. Мы будем давать ссылки на фотостоки. Но предварительно постараемся разобраться, что означает для клиента «симпатичная кошечка». И сделать соответствующие подборки (выборки) в фотостоках, с которыми у нас налажена работа. И дальше отправляете (или смотрите вместе с клиентом) не просто ссылки на фотосток (мол, вон там фотки, сходи поищи). А уже подготовленные, отфильтрованные подборки. Дескать, мы искали по таким-то и таким-то фильтрам. Нам нравятся вот эти и вот эти фотографии. Выбирайте.

Тут и выбор есть. И рекомендации. И клиентоориентированность.

6.13.6 РАЗМЕР НЕ ИМЕЕТ ЗНАЧЕНИЯ

Иногда маленькая картинка занимает по времени отрисовки, трудоемкости и потраченной энергии больше, чем весь остальной макет. Да, она маленькая. Но времени убито вагон. Аргумент «она маленькая» тут не работает. Хотя, конечно, досадно угробить на какой-нибудь баннер столько же сил, сколько на главную страницу. И надо ли?

6.13.7 НУЖЕН ЛИ ИЛЛЮСТРАТОР ИЛИ 3D-ШНИК В ШТАТЕ

Выделенный — почти всегда нет. Существуют десятки стилей иллюстраций. А иллюстратор, как правило, работает в одном-двух. Такое может пригодиться в продуктовой разработке, если вы сотнями рисуете баннеры про монопродукт. В мультипроектной среде так вряд ли получится.

В идеале нанимайте дизайнера, который умеет чего-то два. Например, делать UX (основное) и иллюстрации в нужном нам стиле (бонус). Если иллюстраций становится много — уже думаем, как разгрузить этого дизайнера, убрать с него UX. Та же стратегия про 3D.

6.13.8 ГДЕ ВОВРЕМЯ ВЗЯТЬ РЕАЛЬНЫЙ КОНТЕНТ, ЕСЛИ ЕГО НЕ ПРЕДОСТАВЛЯЕТ ЗАКАЗЧИК. КАРТЫ КОНТЕНТА

НЕ-ЗНА-Ю!

Ничего не работает как надо. Ни звонки маме. Ни жалобы начальству. Ни угрозы. Ни подкуп. Ни шантаж. Ни взятка борзыми щенками. Ни-че-го.



Контент — импотент!

Это боль всей заказной разработки. Сначала не до него. Потом — «пусть дизайнер подумает». Потом — совсем не до него. Потом запускаемся с рыбным контентом. Вплоть до штампованного слогана на печатных баннерах (я не шучу). Или дядькой из клипарта вместо фото генерального директора. Несколько раз обнаруживал себя на каких-то сайтах: успел побывать и прорабом на стройке, и директором производства сыра; кругом сплошной фейк и надувательство. Или даже фейковый номер телефона в шапке сайта. Про отзывы и «факты о компании» я вообще молчу.

№	А	В	С	Д	Е	Ж	З	И
1	Провести диктант текста: http://ru.yandeximg.ru/							
2	Дата проведения урока	Лекция	Комментарий	Требования к материалам	Ссылка на материал (обязательное поле)	Файл заметки	Ссылка на материалы (обязательное поле)	Комментарии
3	01/11/2017	Первый урок истории	Текст письма	— Email письмо — Значения словарных словосочетаний — Синонимы без пробелов: 40-70	http://ru.yandeximg.ru/yandeximg/			Есть, флэшкарточки. Но материал уже использовался
4	01/11/2017	Первый урок истории	Фото тестуры для увеличения по ширине на 100%	— Фото тестуры для каждого из предметов — Для каждого из предметов	http://ru.yandeximg.ru/yandeximg/			Есть, в формате интерактивных заданий
5	01/11/2017	Первый урок истории	Тесты при перемещении текста	— Тесты для каждого из 4-х паров — Вопросы, для каждого из паров увеличить	http://ru.yandeximg.ru/yandeximg/			Есть, Лекция в Google Drive. Материал уже использовался, но для работы есть
6	01/11/2017	Первый урок истории	Лекция, текст презентации	— 2 видео — Текст презентации	http://ru.yandeximg.ru/yandeximg/		https://yadi.sk/i/4wR5P936KQ0Q	Есть, но ресурс не имеет даты
7	01/11/2017	Презентация	Описание	— Тесты для каждого из каждого из предметов — Математический синоним без пробелов: 80	http://ru.yandeximg.ru/yandeximg/			Сопоставления
8	01/11/2017	Презентация	Мифология	— Картинки для каждого из предметов — Общий текст для всех предметов	http://ru.yandeximg.ru/yandeximg/			Сопоставления
9	01/11/2017	Презентация	Презентации по цветам	— Математический синоним без пробелов: 80	http://ru.yandeximg.ru/yandeximg/			Сопоставления
10	01/11/2017	Презентация	Фотографии черепицы	— Материал не в перемещении, а в видео, мы тоже	http://ru.yandeximg.ru/yandeximg/			Есть, Отправленные письмом презент. Фотоны через интернет, добавлены
11	01/11/2017	Написание эссе	Тестуры слова	— Фото тестуры для каждого из предметов — Тесты для каждого из предметов	http://ru.yandeximg.ru/yandeximg/			Самые фотографии тестуры являются перемещенными картинками. Присланы.
12	01/11/2017	Написание эссе	Синоним и описание слова	— Описание в каждую группу — Иллюстрация в каждую группу	http://ru.yandeximg.ru/yandeximg/			Сопоставления
13	01/11/2017	Написание эссе	Тест слова	— Синоним перемещается	http://ru.yandeximg.ru/yandeximg/		https://yadi.sk/i/4wR5P936KQ0Q	

Карта контента. 2017–2021 г. Гуглдокс. Буквы. Меланхолия.

6.13.9 ПРАВКИ, С КОТОРЫМИ НИКТО НЕ БУДЕТ СПОРИТЬ

1. Конкретные, непротиворечивые, технически выполнимые и однозначно сформулированные.
2. Не противоречат техническому заданию, прототипу и визуальному брифу.

Вряд ли кто-то будет настаивать на берегу, что нужно предусмотреть неконкретные, противоречивые, технически невыполнимые и размыто абстрактные формулировки. А также противоречащие всем тем договоренностям, от которых отталкивались на берегу. Однако в реальности такие правки все равно будут поступать. Какие-то вы сможете обработать в режиме эксперимента. Какие-то просто выкинете. Вот тут уже не будет алгоритма. Нужно мастерство.

Итак. С принципами никто не будет спорить. Но вот пролезть между ними попробуют обязательно.

6.13.10 ЧТО ДЕЛАТЬ, ЕСЛИ ДИЗАЙНЕРУ НЕ НРАВЯТСЯ ТРЕБОВАНИЯ И ПРАВКИ

Понятное дело, что правки ни у кого не вызывают восторга. Однако это неизбежная часть процесса.

Дизайнер может сознательно или подсознательно профанировать или делать правки плохо. Используя в том числе фразы «это невозможно», «так никто не делает», «будет херня».

Мотив — доказать, что он прав (а автор правок, соответственно, — нет). «Ну я же говорил, что будет херня. Смотрите!» А вот это уже косяк.

Разъясните своим ребятам, что вы **обязаны** (по контракту, по договоренностям, чисто по-человечески) сделать то, что просит заказчик. И сделать это на уровне. Но если сама правка кажется нелогичной — вы **вправе** сделать рядом альтернативный вариант, чтобы показать свое видение. Более того, заказчик действительно может ошибаться, или его правка может носить экспериментальный характер. В случае ошибки вы **обязаны** ее выявить, показать наглядно и предложить альтернативу.

Итак. Чувствуете, что дизайнер профанирует правки — напомните, что он может оформить свои предложения в альтернативе. Но обязан сделать, как просят, и сделать хорошо.

6.13.11 ВОДИМ РУКОЙ ДИЗАЙНЕРА

Я уже говорил про это в параграфе, посвященном правкам. Но тут акцентирую отдельно.

Ближе к концу работ по концепции правочки становятся все мельче. Но от этого не становится сильно легче. Помним про транзакционные

издержки. Больше раздражает не «одна маленькая правочка», а сам факт очередной итерации. «Re: Fwd: Правки по 15-й, точно финальной версии» или типа того.

Если все запутались или (особенно!) застряли на последних этапах доводки макетов, нужно сесть за один экран с дизайнером. И шлифовать макет в реальном режиме времени.

Формулировать мелкие правки и плодить еще десять итераций невыносимо долго и дорого. Просто сядьте рядом или расшарьте экран. Проведите вместе час, доводя последние штрихи. И дизайнеры, и заказчики это адекватно воспринимают.

Итак. Если тяготит бесконечное количество мелких правок — самое время сесть вместе и смотреть на экран. И вместе думать.

6.13.12 ТЕНДЕРНЫЕ КОНЦЕПЦИИ.

БЕСПЛАТНО РАБОТАТЬ — ГРЕХ

Я считаю, что начинать надо с аналитики. С изучения задачи. С агрегации требований. Бесед со стейкхолдерами. А не с картинок.

На картинках можно сделать мудборд. Показать свои кейсы. Причем, экстремумы — вот самый навороченный, вот самый примитивный, а все, что между ними, — мы умеем.

А рисовать концепции в унитаз — потеря денег, времени и самоуважения.

6.13.13 ДИЗАЙН, КОТОРЫЙ ПРИШЕЛ СВЕРХУ, ИЛИ ЗА ВСЕ ОТВЕЧАЕТ ТОТ, КТО СТОИТ В КОНЦЕ КОНВЕЙЕРА

У нас было с десятков проектов, по которым дизайн достался от других команд. Это никогда не проходит гладко.

Несколько раз в работу приходили макеты, разработанные полиграфистами. Тогда еще в Corel Draw. Как правило, это макеты «дорого-богато», с шикарными градиентными заливками, уникальными отступами в каждом блоке.

Все без сеток, все на глаз, все выравнивания — по чувству прекрасного, без математики. Заказчикам эстетически все нравилось. Верстальщики же приходили в ужас.

Как правило, дело заканчивалось перерисовкой макета, что называется «по мотивам». Но уже с учетом технических и UI-UX требований.

Часто такое разделение «на команды» приводит к потерям важных сценариев. Например, логика доставки не продумана до конца. Ситуация может усугубляться тем, что нет возможности влиять напрямую на дизайнеров (у них свое начальство). Или что по ходу разработки прилетает «поправленный макет, который нужно срочно учесть в разработке». Или правочки вносятся втихаря. Поменяли пару артбордов в Figma и делаем вид, что так и было.

Плохо, что дизайнер не до конца вовлечен в продукт. А все косяки почему-то расхлебывать программистам.

И хотя полноценного рецепта тут нет (нет управляемости), план действий такой:

1. Проверить макеты визуально. Если они ничего, кроме рвотных позывов, не вызывают — продать отрисовку с нуля. Или отказаться от проекта. Если все четко — можно попробовать с этим жить.
2. Проверить типовые сценарии: нет ли дырок. Посмотреть визуально: нет ли грубых ошибок. Например, нет никаких сеток, все на глаз, вставленные тексты никак не продуманы с учетом, что их может быть больше или меньше и т. д. Принять решение: берем эти исходники и работаем с ними или рисуем «по мотивам». Если макеты рисовали полиграфисты — значит, исходники брать в работу нельзя.
3. Провести тестирование макетов. Можно использовать чек-лист. Смотрим как юзабилити, так и покрытие сценариями. Проверяем сам макет на качество. К сожалению, полноценно такие проверки не гарантируют 100% решение вопроса. Что-то да будет пропущено. Иногда довольно значимые вещи.
4. Договориться, кто и как будет вносить правки по итогам тестирования: ваша команда или изначальные авторы. И то, и другое требует времени, коммуникаций и повторных тестов.
5. Внедрить нужные технические правки. Причесать и принять макеты.
6. Договориться, кто и как будет вносить изменения в макет во время разработки. Удобнее забрать макеты себе и мелкие огрехи чинить самостоятельно. Ужаснее заводить тикеты на каждый чих и неделями ждать исправлений.

7. Договориться, что пока идет разработка, изменения в макеты не вносятся. Это точно всех запутает и поссорит.
8. Авторский надзор по итогу верстки. Возможно, тут разгуляется дополнительный аппетит у дизайнера.
9. Хорошо, если есть возможность обсуждать макеты с их автором. Хуже, если он в отпуске, заболел, уволился и умер.

Итак. Явных плюсов для заказчика в такой связке я не вижу. Но раз уж так вышло — с этим можно работать. Хотя накладные расходы высокие.

6.13.14 МОУШН-ДИЗАЙН. КОГДА ДЕЛАТЬ АНИМАЦИЮ

Обычно мы готовим анимацию при подготовке презентации, но после внутренней приемки макета. Однако анимация анимации рознь. Где-то достаточно собрать два-три кадра. Где-то нужно долго мучать After Effects. Точно не нужно кидаться делать анимации, пока не согласована статика.

Если на анимацию нужно больше четырех-восьми часов — выносим ее в отдельный подпроект. Там может быть отдельный брейншторм, отдельная отрисовка раскадровки, отдельное демо и даже отдельный мудборд.

Естественно, если нужно делать такое кино, на это нужен дополнительный бюджет. Но в большинстве случаев четырех часов достаточно, чтобы показать механику анимации.

6.13.15 БЕЗЫСХОДНОСТЬ. ГДЕ ХРАНИТЬ МАКЕТЫ

Здесь я только обозначу проблему, поскольку не в праве говорить о ее решении. Мне не нравится ни как сделано у нас, ни как у коллег, ни как это описано в статьях зарубежных агентств. Везде в итоге бардак, путаница.

Итак. Когда проектов, макетов, артбордов и версий становится слишком много, возникает резонный вопрос, как организовать хранилище. Причем так, чтобы было понятно, какие из файлов актуальные, а какие — уже не очень.

Казалось бы, у программистов уже есть инструменты — системы контроля версий. Бери и делай то же самое. Но нет. С дизайном это

так не работает. Особенно, когда проект долгоиграющий, с сотней экранов, которые делаются-переделываются. Что-то ушло на A/B-тест, но в итоге пришел директор и сказал: «Фиг с ним, с тестом. Кнопка будет черной!» Где-то файлы бинарные, очень тяжелые. Сравнение версий толком не работает. Про **merge** я вообще молчу.

Можно придумать систему имен макетов. Систему папок. Наименование артбордов. И это обязательно нужно сделать. И закрепить в регламентах. Однако проблема не в том, как придумать правила. Проблема в том, как заставить их соблюдать. Мантры «команда должна» или «менеджер должен проверять» работают на очень короткой дистанции.

Дизайнеры зачастую сами плодят хаос, в котором не могут разобраться. Увеличить нагрузку на менеджера дополнительной проверкой — не вариант; у него и так слишком много дел. Делегировать проверку QA не получится: он не в контексте, какие макеты последние, а что устарело. Да и система наименований файлов не гарантирует, что в них будет действительно самая последняя версия.

Облачные редакторы типа Figma еще больше усугубляют ситуацию. Кажется, что макет есть у всей команды. Но по факту его нет нигде. Были случаи, когда на платных платформах макеты просто пропадали. И даже техподдержка ничего не могла сделать. Регулярное падение облачных сервисов или безбожные тормоза (той же Figma) при их бодрящих ценовых политиках вгоняют в тоску зеленую.

Мы организовали автоматизированную выгрузку файлов из Figma, хоть это было и нетривиально. Сейчас он доступен в открытом доступе. Ссылка по QR-коду. Но никто не гарантирует, что редактор через год будет открывать свои же *.FIG-файлы. Офлайновые редакторы в этом плане несколько лучше, но ненамного. Зато хуже в командной работе. Но макет — это не единственный артефакт от дизайнера. Есть еще 3D-модели, моушн, ролики и т. д. Куча всего, на самом деле.



В общем, безысходность.

Что можно сделать, так это придумать рациональную, поэтапную систему наименования папок и макетов. Делать регулярные бекапы. Добавить в чек-лист пункты по выгрузке исходников в нужное место в нужном виде. Dropbox, сетевой диск, Google Drive — тут без разницы, как решите, так и будет. И делегировать кому-то регулярную сверку макетов. У нас сейчас так. Но я все еще недоволен :)

6.14 ДЛЯ ТРЕНИРОВКИ

Итак. Мы подробно разобрали, из чего и как строить процесс работы по дизайну. У вас полно кирпичей для этого, лайфхаков и готовых схем.

Задания будут такие:

1. Распишите для себя, какие шаги в дизайне вы сейчас применяете. Какие — нет. Какие хотели бы? Составьте план внедрения. Внедряйте.
2. Организуйте регулярную процедуру трендвотчинга, по типу того, как это описано в § 6.7.2. Следите за трендами сами.
3. Изучите видео и примеры по работе с возражениями. В течение четырех недель собирайте и записывайте возражения, которые возникли с вашими клиентами. Проанализируйте, что могло привести к таким возражениям? Можно ли было что-то сделать на опережение?

Успехов!

ЛИТЕРАТУРА

Подробнее о дизайн-системах:



Групповая динамика:

- ▶ Жанна Завьялова, Елена Фарба, Екатерина Каденильяс-Нечаева, Мария Авдюнина, «Энергия бизнес-тренинга. Путеводитель по разминкам».
- ▶ Ирина Авидон, Ольга Гончукова, «100 разминок, которые украсят ваш тренинг».
- ▶ Андрей Моисеев, Жанна Завьялова, «Работа с возражениями и сопротивлениями».
- ▶ Анна Гуревич, «Ролевые игры и кейсы в бизнес-тренингах».
- ▶ Ирина Абрамова, Надежда Мизина, Мария Небытова, Ольга Пыхова, Ольга Хромова, «Тренинги на все случаи бизнеса».

Книжки по работе мозгофф и поиска решений:

- ▶ Владимир Тарасов, «Искусство управленческой борьбы».
- ▶ Георгий Щедровицкий, «Оргуправленческое мышление. Идеология, методология, технология».
- ▶ Генрих Альтшуллер, «Найти идею. Введение в ТРИЗ — теорию решения изобретательских задач».
- ▶ Даниэль Канеман, «Думай медленно, решай быстро».
- ▶ Джона Лерер, «Как мы принимаем решения».
- ▶ Дрю Бойд, Джекоб Голденберг, «Творчество в рамках».
- ▶ Дэвид Майерс, «Интуиция».
- ▶ Кейт Кинан, «Решение проблем».
- ▶ Максим Поташев, Павел Ершов, «Путь решения».
- ▶ Сергей Елкин, «Инженерная эвристика».
- ▶ Стивен Левитт, Стивен Дабнер, «Фрикомыслие».
- ▶ Павел Сивожелезов, «Сложные переговоры с подчиненными».
- ▶ Питер Друкер, «Эффективный управляющий».
- ▶ Юрий Лапыгин, «Управленческие решения».
- ▶ Эдвард де Боно, «Гениально».

Инфографика «Что делать, если исполнитель упоролся».



ПОЯСНЕНИЯ

Экстремум — если говорить о математике, это максимальное или минимальное значение функции на заданном множестве; для нас — просто максимальное и минимальное значение.

Артемий Лебедев — дизайнер, предприниматель, блогер, путешественник и автор «Ководства» — руководства по веб-дизайну. Основатель, совладелец и генеральный директор Студии Артемия Лебедева.

Кастомизация — от английского custom, что в переводе означает «сделанный или изготовленный на заказ»; в разработке и околоразработочных областях кастомный — значит настроенный вручную под свои требования.

КГ/AM — согласно portalу Lurkmore (электронной энциклопедии современной культуры, фольклора, субкультур и всего прочего), сокращенная форма от фразы «креатифф говно, афftar мудаk».

AWWARDS — онлайн-площадка конкурса профессиональных веб-дизайнеров и разработчиков, где те размещают свои работы ради признания в профессиональной сфере и ради поощрения самых лучших, самых клевых и самых инновационных решений в веб-дизайне.

QA — сокращение от Quality Assurance engineer, это специалист по обеспечению качества, который занимается улучшением процесса разработки ПО, предотвращением дефектов и, главное, выявлением ошибок в работе цифрового продукта.

Арачка — средне-алкогольный напиток (25–30°) на основе сброженного молока.

Контролы — элементы управления в интерфейсе (например, выпадающие списки или стрелочки).

Процедура checkout — процесс заполнения формы во время оформления покупки в интернет-магазине быстро, без лишних форм и переходов.

Бенчмаркинг — сравнение своей работы с лучшими представителями на рынке и в отрасли с целью перенять фишки («кради как художник» и вот это вот все).

Стейты меню — разные состояния меню: свернутое/развернутое, раскрыты подкатегории/не раскрыты и так далее.

Time & Material — оплата результата, исходя из трудозатрат.

Таймлайн — временная шкала, хронология или попросту визуальное отображение списка событий или задач в хронологическом порядке на графике/диаграмме.

Аjax-ссылки — такие, которые загружаются в фоновом режиме, не заставляя браузер ждать и не отдавать страницу полностью, пока эти ссылки не подтянутся с сервера.

HEX-коды — коды цвета в шестнадцатеричной системе счисления, где для обозначения цвета используются шесть символов: цифры от нуля до девяти и латинские буквы от А до F. Так, например, 000000 — это черный, а FFFFFFFF — белый.

Инпут — поле для ввода.

Тултип — всплывающая подсказка.

Радиобаттон — круглый элемент интерфейса, на который можно нажать и тем самым выбрать какую-либо опцию; радиобаттоны — угрюмые одиночки, поэтому нельзя выбрать сразу два радиобаттона, в том вся их суть.

Чек-бокс — квадратный товарищ радиобаттона; обычно это квадрат, в котором можно поставить галочку (радикальное отличие от радиобаттона в том, что можно выбрать сразу несколько чекбоксов, и они не будут против).

Дизейблед — отключенный, недоступный.

«**Мыло**» — если иконку сохранить неправильно (например, в растровом формате вместо векторного), при увеличении масштаба ее очертания будут замыливаться, терять свою четкость. Идеальные иконки четки до последнего пикселя, что бы с ними ни происходило.

Утилизация — отношение коммерческого времени к рабочему, аналогия соотношения «план/факт».

Merge — слияние объектов, в нашем случае — разных версий дизайна.

Ext JS — библиотека JavaScript для разработки веб-приложений и пользовательских интерфейсов (сейчас существует как отдельный фреймворк).

Барабан-Буфер-Канат — метод ТОС для планирования и управления производством при наличии внутреннего ресурса-ограничения, состоящий из трех элементов: 1) барабан — ограничение или ресурс с ограниченной мощностью, который выполняет работу в определенной последовательности на основе требуемого клиентом срока выполнения заказа и предельной мощности ресурса; 2) буфер — буфер времени, который защищает ограничение и график поставки от влияния неопределенности; 3) канат — механизм, чтобы приостановить запуск сырья в производство для соответствия потреблению этого сырья на ограничении.

Функциональное программирование — написание программ по типу, как в школе задачи через формулы решали. $S(a, b) = a \times b$ — типа, площадь посчитали. И так вся программа — как одна больша-а-а-я формула.

Объектно-ориентированное программирование — когда программа описывает объекты и что с ними можно делать. Типа «кнопку» можно «нажать», «человек» может «гулять». Остается связать одно с другим, написать, что делать, когда нажимается кнопка «Гуляй, Вася». Сумбурно, но идея примерно такая.

Часть 7



СТРОИМ И РАЗВИВАЕМ КОМАНДУ

А зачем вам команда? Зачем людям вообще понадобилось объединяться, испытывать дискомфорт от необходимости договариваться и подстраиваться друг под друга? Терять время на синхронизацию. Что-то кому-то объяснять, доказывать, терпеть? Какой профит? И когда он возникает?

В этой главе мы поговорим про командообразование. Разберем несколько моделей и посмотрим, на чем фокусироваться руководителю. Поговорим про цели команд, KPI (ой), OKR. Ритуалы, в том числе геймификацию. Выдачу обратной связи.

Помчались.

7.1 АНАТОМИЯ КОМАНДООБРАЗОВАНИЯ

Детский пример: допустим, вы пещерный человек. Очень хотите кушать. И чего-нибудь на себя надеть. И тут неподалеку ходит саблезубый мамонт. Большой, сильный, быстрый, кровожадный и, сука, умный. Ну такая порода. Итак, появляется задача: навалить саблезубому мамонту.

В одного вам не справиться. В лучшем случае уйдете голодным. В худшем — сами получите звездюлей. А вот вдесятером у вас уже есть шанс. Правда, мой знакомый историк утверждает, что шкурка у мамонта была семь сантиметров, и пробить ее можно было разве что из калаша, 7.62, но у нас же детский пример.

И вот вы как-то договариваетесь на своем пещерном наречии. Формируете большую цель: навалить этому ублюдку и откусать мамонтины. Без этой цели я бы вообще с половиной этих грязных пещерных

макак не общался. Но жрать хочется. Распределяете обязанности: ты — справа, ты — слева, ты — сзади, а ты кидай в него булыжники вот с той скалы. Заарканивай. Пиф-паф-пуф. Получилось!

Осталось справедливо поделить мясо и шкуру (та еще задача). И можно расходиться. Хотя в общем ребята оказались нормальными, никакие не макаки, хорошие мужики. Завтра повторим.

Так что же вас объединило? Заставило вылезти всей толпой из пещеры? Договариваться? Действовать? Правильно — **Большая Цель** (саблезубый, которого вы прямо сейчас уже жуετε и перевариваете). Причем, такая цель, которую невозможно достичь в одиночку. И все это понимают.

Итак. Команды формируются тогда, когда они **осознают**, что у них есть **Большая Цель**, которую невозможно достичь в одиночку. Возникает эффект синергии.

Синергия — эффект, при котором итоговый результат выше (иногда в сотни раз), чем сумма эффектов от каждого персонального участника. Собственно, из-за этого эффекта и имеет смысл сотрудничать. Виды синергии:

- ▶ Механическая: пять человек могут поднять большой камень. Один — потратит время, но ничего не сделает. Одинаковые способности складываются, а эффект получается больше, чем сумма эффектов каждого по отдельности.
- ▶ Профессиональная: один может делать то, чего не могут делать другие. Этот метко камни кидает. Этот — копье. Один — сильный, другой — быстрый. А в пещере еще женщины остались — они вообще могут делать то, чего мужики делать не могут. Детей рожать, например.

Итак, сотрудничать и строить команды нам нужно для Больших целей, ради эффекта синергии. Теперь посмотрим жизненный цикл команды.

7.1.1 МОДЕЛЬ БРЮСА ТАКМАНА

Брюс Такман, американский психолог, по заказу Министерства Обороны США исследовал сотни команд — экипажей атомных подводных лодок — с целью понять, как они себя ведут и что можно сделать, чтобы все не передрались в хламину. Оказалось, что даже пара ум-

ных человек обязательно будут выяснять отношения. Что неминуемо скажется на итоговой производительности.

Какие фазы проходят команды:



Расширенная модель Такмана

1. Формирование (Forming). Вы собрали несколько человек вместе. Дали им Большую цель. Начинается работа. Да, какая-то производительность есть у каждого из них. Но в самом начале будет небольшой всплеск: проект — новый, новизна — бодрит. Плюс в самом начале все стремятся показать свои лучшие качества. Все ведут себя хорошо. В этот момент неопытный менеджер может обрадоваться: вот, смотрите, какую супер-команду я создал! Хотя это еще не команда, это просто рабочая группа, у которой есть Цель.
2. Шторм (Storming). Начинаются разборки. Кто тут альфа, кто тут бета. Кто прав, кто не прав. Устанавливается социальная иерархия. Без этого мы не можем. Однако в один момент производительность резко падает. Кто-то что-то делает, но в целом дело уходит на второй план. В этот момент неопытный менеджер начинает сильно нервничать (например, применять репрессивные меры), бегать и решать конфликты, всех уговаривать. Нормальной командой

тут еще не пахнет. Бывает, что не запахнет никогда — так и будет штормить всю дорогу, или все окончательно разругаются.

3. Нормализация (Norming). Когда все роли поделены, можно наконец-то начать нормально работать. Вот тут и формируется команда. Здесь и начинает возникать эффект синергии, ради которого все и затевалось. Должны появиться первые успехи.

Команду формирует позитивный опыт совместного достижения цели. Если опыт не позитивный — все считают, что и ловить тут нечего: с этими людьми мамонта не завалить.

4. Производство (Performing). Команда вышла на свою оптимальную производительность. Время на срач не тратится. Высокая синергия, высокая эффективность. Так может продолжаться довольно долго. Год, два. Или пока не будут достигнуты цели, ради которых команда создавалась. Вот только достигнув этой стадии — менеджер мо-ло-дец!
5. Переформирование (Reforming). По естественным причинам состав команды время от времени будет меняться: кто-то будет уходить, и нужна будет замена. Кто-то устанет. Производительность будет постепенно снижаться. Короче, нужна новая кровь! И вот в эти моменты команду будет штормить (Storming) по-новому. С добавлением человек сначала обязательно будет просадка. Но если все в порядке — производительность со временем выйдет на прежний уровень или увеличится (за счет свежей крови, драйва и энтузиазма, который есть у новичков). Хотя команда может пережевывать и выплевывать таких новичков, в том числе новых руководителей проектов.

Прикольная модель. Все так и есть, проверял на практике многократно. Однако непонятно, что с ней делать. Тут самый опасный момент — Storming. Мысли такие:

1. Неправильный подбор людей ничем компенсировать нельзя. Ни обучением, ни уговорами. Будут срать до небес и никогда не станут командой.
2. В момент шторма будут те, для кого самое важное — выяснение отношений. А будут те, кто делает дело: пишет код, тесты, рисует дизайн. Руководителю проекта надо перестать тратить время и энергию на тех, кто не занят делом. А больше хвалить, помогать,

слушать, давать интересные задачи и т. д. тем, кто проявляет эффективность.

Двое, допустим, много выясняют отношения, решают, кто из них главнее, чей код круче, чья архитектура — правильнее. И от них много шума. А третий сидит тихонечко и по мере сил поставляет фичи. Надо дать ясный сигнал тем двоим: ребята, ваши разборки меня мало интересуют! Я буду тратить свое время и энергию на того, кто показывает результат. Сделать это можно только волевым усилием, ведь выяснители отношений слишком шумят и пытаются украсть внимание остальных ребят.

Итак. Пока будущую команду штормит, 80% времени и энергии тратим на тех, кто делает дело.

7.1.2 ВСТУПЛЕНИЕ В ДОЛЖНОСТЬ

Это все хорошо в новых командах. Но если вы пришли управлять в уже сложившийся коллектив? Упс. Тут все сложнее. Вообще, вступление в должность руководителя — тот еще квест.

Во-первых, мы должны знать, что случилось с предыдущим руководителем. Не съела ли его команда. Может, она менеджеров как перчатки меняет. Раз в полгода. Цап-царап, уносите, следующий.

Во-вторых, знакомимся с командой. Вообще с группой и с каждым наедине. Во время группового знакомства сразу делаем какое-то физическое изменение в окружающем мире: открываем окно (или закрываем), переставляем флипчарт и т. д. Главное, чтобы команда зафиксировала у себя в голове: этот чувак считает, что имеет право что-то менять и делает это, не спрашивая ни у кого разрешения. Только не облажайтесь на этом этапе! Не уроните флипчарт или не пытайтесь открывать заколоченное окно. Не отмоетесь потом никогда.

В-третьих. Сидим и наблюдаем. Кто что делает, как все устроено. Чертим у себя на планшете модели отношений: кто главный, кто за что отвечает, как работа делается на самом деле. Формируем у себя картину мира. Пока ни во что не вмешиваемся. Разбираемся с психотипами членов команды.

В общем, реально сидим у себя в кабинете, ничего не делаем, только наблюдаем и чертим картину мира. Находим объекты влияния (например,

самый главный и яростный программист). Строим с ними отношения. И потихонечку начинаем оказывать управляющие воздействия.

Итак. При вступлении в должность в уже как-то сформированный коллектив — важно не начать махать шашкой. Иначе получите командную аллергию: вас разжуют и выплюнут. А надо сформировать адекватную картину мира. Выявить агентов влияния. И выстроить с ними отношения. Завоевать авторитет и доверие. Укрепить поле власти. Спешка и импульсивность тут вредны. Однако, если были допущены грубые ошибки при подборе членов команды, — починить что-то будет крайне сложно (без замены части сотрудников тут не обойтись).

7.1.3 ДРУГИЕ КОМАНДНЫЕ МОДЕЛИ, О КОТОРЫХ СТОИТ ПОЧИТАТЬ

Здесь будут только отсылки к литературе и краткие концепции. Без спешки освойте их и в теории, и на практике.

1. Транзактный анализ по Эрику Берну. Кто там, в вашей команде, играет роль Родителя, кто — Ребенка, а кто — Взрослый. Изучите эти связи внимательно. Классическая книга — «Игры, в которые играют люди», рекомендую.
2. Модель Ицхака Адизеса о жизненном цикле организации и его классификация PAEI руководителей (производитель результатов — Р, администратор — А, предприниматель — Е и интегратор — I).
- 3.



Перспектива	Краткосрочная	Проблемы ванны Возможности не ванны	Не уверен в себе Беглец Производитель (P) Любит жаловаться/ныть Тороплив Страх быть на виду/объектом придирок Что делать Отвечает за все Требуется похвалы	Уверен в своих поступках Контролёр Администратор (A) Любит благодарность Нетороплив Страх наказания/ошибиться Как делать Наказать виновных Желает быть важным	Проблемы не интересны Возможности не интересны
		У него все плохо предпочитает конкретику	У него всегда кто-то виноват предпочитает точность	Опирается на факты	
Долгосрочная		Проблемы не ванны Возможности ванны	Зависимый Предприниматель (E) Любит лезть Нетерпелив Страх быть невидимым/быть игнорируемым Когда делать, как не сейчас Ни за что не отвечает Требуется восхищения	Мазохист Интегратор (I) Любит сплетни Терпелив Страх одиночества/быть отвергнутым С кем делать Виновных нет Желает быть нужным	Проблемы интересны Возможности интересны
		У него все виноваты предпочитает без конкретики	У него все хорошо предпочитает образность	Опирается на мнения	
			Результаты	Эффективность	

PAEI-классификация (типология лидерства) по Ицхаку Адизесу

Ключевые вопросы здесь:

- ▶ На какой стадии находится организация или команда?
- ▶ В какую стадию она стремится перейти?
- ▶ Какие сильные стороны у вас, как руководителя (пройдите тест PAEI на определение стиля менеджмента)?
- ▶ Чем вы можете помочь команде?

Книги Адизеса «Управление жизненным циклом корпораций» и «Идеальный руководитель» настоятельно рекомендую.

4. Модели рабочих групп по Михаилу Ефимовичу Литваку (книга «Командовать или подчиняться», также рекомендую к прочтению):
 - a. Карьеристы. В хорошем смысле этого слова. Нацелены на обучение и достижение результатов.
 - b. Культурники. Результат не так важен, важно тусоваться вместе.
 - c. Алкогольно-сексуальные группы. Это вообще не про работу.

То, что рабочая группа сложилась именно таким образом — это не ваша вина или заслуга. Это наследие от предыдущего руководства. Руководители строят такие группы, к которым относятся

сами: карьеристам-достигаторам хорошо с карьеристами, алкоголикам — с алкоголиками. Приглядитесь, туда ли вы попали? Кстати, про алкоголь.

7.1.4 ТИМБИЛДИНГ, АЛКОГОЛЬ, БИРЮЗА

Я очень неоднозначно отношусь к корпоративным мероприятиям. Особенно «бирюзовым» корпоративным тренингам вроде совместного прыгания на тарзанке или еще какой-то коучевской фигни. Вербочные тренинги. Иногда это прикольно, но как единственный инструмент построения команды — однозначно не на первом месте.

И очень неоднозначно отношусь к мероприятиям с применением алкоголя. С одной стороны, это часть культуры. Вроде как можно посмотреть на человека, какой он в жизни. С другой стороны, это может сильно не понравиться, и не надо на это смотреть.

У нас проходят виски-брейки — митапы, куда мы приглашаем докладчиков на digital-темы и где разрешен алкоголь. Долгое время на барной стойке был алкобар, которым можно было пользоваться по вечерам. Однако столько дерьма случилось из-за алкоголя, что никакая сомнительная польза его не перекроет.

Сейчас ничего на стойке бара не стоит и не будет. Раньше на день рожденья мы дарили бутылку шампанского и деньги. Сейчас — подарочную карту книжного магазина и деньги. Или дополнительный отгул. Никто не обиделся, наоборот — говорят спасибо: можно купить себе книгу, до которой не доходили руки.

Для себя я выработал такие принципы:

1. Не решать никаких дел с хоть сколько-то выпившим человеком. На мероприятиях будут пытаться подходить и, в порыве теплых чувств, дыша перегаром в лицо, пытаться решить какой-то вопрос. Например, повышения зарплаты. Говорю, что у меня принцип: не решать ничего с выпившими людьми, и предлагаю зайти ко мне в кабинет в десять утра в понедельник для обсуждения вопроса. Пока никто не дошел.
2. Человек, от которого на работе в рабочее время чувствуется перегар, отправляется домой. В первый раз — с разбором полетов. Второй раз — навсегда. Был случай, когда за пьянку и невыход на работу пришлось уволить четырех человек. Жестко. Но справедливо.

Другие мероприятия за рамками деятельности компании — вроде волонтерства, сбора мусора, макулатуры, очистки рек, походов в горы, донорства и тому подобного могут сильнее сплотить людей. По Литваку — это вотчина Культурников. Как по мне, есть — так хорошо, нет — ну и ладно.

Что мне нравится — это совместные обучающие мероприятия. Берем какие-то темы. Делимся опытом. Разбираем технологии с программистами (у нас это называется «холивар»). Читаем. Смотрим фильмы. Разбираем кейсы. Проводим дизайн-баттлы. Тренировки на моделях (пусть и в игровой форме, но тех навыков, которые нужны нам для дела). Вот это, по-моему, то, что обязательно должно быть.

А алкоголь не нужен. Совсем.

7.1.5 ПОДБОР КОМАНДЫ

Итак. Ошибки при подборе команды практически невозможно исправить обучением и воспитанием. Скорее всего, дело кончится заменой людей. На что следует обратить внимание и что можно попробовать:

7.1.5.1 Выпускникам

1. Если вы недавно окончили вуз, либо еще учитесь на старших курсах, скорее всего, ВЫ РАЗОЧАРОВАНЫ. Либо выбором (чаще всего не своим) скучной и не то чтобы перспективной специальности. Либо качеством образования. Вплоть до прямого обмана («при поступлении обещали практику программирования, а по факту этого не было, или было очень мало»).
2. Если да — успокою: время, конечно, потеряно. Но вы не одиноки. Можно ли с этим что-то сделать и как-то исправить? Боюсь, что не всегда. Выбирайте компанию, в которой построен процесс обучения на практике. Идите на стажировки. Рискуйте.

7.1.5.2 Новички, кураторы и стажировки

1. Сейчас очень много пишут про поколение Z. Какие они нежные и креативные, да к работе непригодные. Блин, ну не вижу я изнеженности. Или какой-то «нетакойтовости». Очень много хороших, адекватных ребят. Люди на самом деле очень хорошо

дисциплинированы и слушаются. Просто, может быть, не вас. А истинного лидера группы. А это можете быть не вы. И нефиг жаловаться. Качайте компетенции, что уж.

2. Понятно, что нужно проверять профессиональные навыки и уровень кандидата (программист под Android должен уметь программировать под Android).
3. Однако не надо бояться джунов. Если у вас есть ресурсы на их воспитание и прокачку, можно найти много толковых ребят с гибкими мозгами. Податливых, как пластилин. Берите их на стажировки. Выращивайте. Самых перспективных оставляйте себе. У нас за успешные стажировки полагается полет на спортивном самолете, например.
4. На каждого нового сотрудника нужно иметь двух, лучше — трех, опытных. Иначе размываются компетенции и связи внутри команды. Лучше расти медленнее.
5. Обязательно назначайте куратора (наставника) для новичка. Люди растут лучше и быстрее, когда за ними присматривают, подсказывают и направляют. Лучше, чем схема «мастер-подмастерье», пока ничего не придумали.
6. Для кураторов резервируйте время на обучение новичков. Примерно час в день. Структурируйте это время: например, три-четыре транзакции обучения, ревью и обратной связи за день, в определенные часы. Иначе новичок может разрывать мастера своими вопросами.
7. Должна быть внятная программа стажировок. Четкие шаги и задания. Именно по ней стоит отслеживать прогресс.
8. Время от времени спрашивайте у куратора, какой прогресс по стажеру. Важно, задает ли стажер вопросы. Если нет — он не вырастет.
9. Разговаривайте со стажерами, минимум раз в неделю. Какие проблемы, сложности, хераборы происходят. Бывает, что куратор со стажером не нашли общий язык или стажер понял, что тема не его. Нужно ли это чинить? Пожалуй, нет. Слишком хлопотно.
10. Переговорите с потенциальным куратором. Хочет ли он себе стажера? Если нет — заставлять не нужно. Хотя многие охотно берут и воспитывают. А в итоге начинают лучше и с большим интересом работать сами. Оплачивайте кураторам небольшие бонусы за стажеров.

11. Я сам не делал, но мне рассказывали про такую ошибку. Берут три-пять зеленых ребят, сразу со студенческой скамьи. Говорят: «Вы теперь команда! Погнали делать проект». Это не работает вообще. И даже если уровень ребят хоть маленько подрастет (например, вы их каким-то чудом доучите), они сбегут от вас при первой же возможности. Продукт джуниорами вы не постройте. Добавление людей в опаздывающий проект задерживает его еще больше. Закон Брукса. Работает на 100%.

7.1.5.3 Собеседования

Вы бы прошли сами у себя испытательный срок? А собеседование? Вот!

Я предпочитаю короткие личные собеседования в первый раз, и затем, если ОК — подробное, часовое. Первое собеседование и техническое интервью могу сейчас себе позволить делегировать. Но обязательно поговорю с потенциальным кандидатом.

Если опыта собеседований мало и есть возможность — пишите кандидата на камеру. Затем спокойно проанализируйте еще раз, какие ошибки вы допустили во время интервью.

Обычно на собеседованиях мы даем выполнить несколько заданий — о них подробнее поговорим чуть ниже. Так вот. Обязательно фиксируйте оценки по каждому из заданий по пятибалльной шкале. Формальная фиксация оценок позволит избавиться от ситуации, что кандидат вам лично понравился/не понравился и его ответ на ПЕРВЫЙ вопрос повлияет на то, как вы будете воспринимать ответы на остальные вопросы (см. книгу Канемана «Думай медленно, решай быстро» — там этому фактору посвящена отдельная тема).

Я использую пятибалльную шкалу:

5 — сделал сам или с минимальными подсказками;

4 — сделал, но за три-четыре итерации подсказок;

3 — сделал более чем за три-четыре итерации подсказок и с большим количеством ошибок;

2 — сделал с очень большим скрипом;

1 — все плохо. Не сделал, отказался категорически делать, даже при подсказках.

Плюс фиксирую текстом какой-то комментарий, который позволит вспомнить, а что вообще было-то (ваше мнение/впечатление).

Странно, что про это надо говорить. Начните собеседование с приветствия.

Поздороваться. Представиться. Рассказать, как будет проходить собеседование. Я, например, предупреждаю, что **буду задавать глупые, дурацкие вопросы**, но ответы хотел бы честные. Это как бы снимает с меня возможную претензию: спрашивали фигню какую-то. Такое можно услышать от кандидатов, которые завалили интервью. Да, спрашиваю. Но я предупредил заранее.

Рассказать, что после этого вопросы можно будет задавать вам. Предупредить, что собеседование носит предварительный характер и, если все ОК, затем нужно будет встретиться и поговорить подробнее, про жизнь. Не пропускайте вводную часть.

Если планировалась видеозапись: спросить разрешения на видеозапись для внутреннего использования. Включить камеру. «С вашего позволения, я бы хотел вести видеозапись нашей встречи для внутреннего использования. Вы не возражаете?», или что-то вроде того.

Первый вопрос — открытый. Попросить рассказать о себе. Что-бы не пугать кандидата, который подробно все расписал в резюме, можно добавить оговорку: «Расскажите, пожалуйста, о себе. Может быть что-то такое, о чем я не мог прочитать в ваших файликах». Не пропускайте эту часть. Следите за реакцией. Уверенность-неуверенность. Что говорит и как говорит. Попутно возьмите и углубитесь в какой-то пункт, который кажется кандидату очевидным, и попробуйте выловить в нем лажу. Пример: выяснить, по какой теме писал диплом, и подробно расспросить про эту тему. Какую задачу решал и **как конкретно делал**. В итоге можно выяснить, что диплом был не совсем его. Вообще вопрос **«как ты это делал»** меня всегда сильно интригует.

На технических собеседованиях я прошу писать код на бумажке. Немного простых задач на подумать. Это многим не нравится. Поэтому используем тот же самый прием, что и с «глупыми вопросами». Предупреждаем заранее в как бы шуточной форме: «Сегодня вам предстоит делать то, что, скорее всего, вы никогда не делали и, надеюсь, не будете делать в будущем: писать код на бумажке. Ручкой». На самом деле, многие писали, чего уж там.

Байки про то, что «если бы не бумажка, я бы в своем любимом текстовом редакторе, который у меня настроен под себя, — уж как-а-ак написал! А тут из-за этих бумажек ничего не решил» — полная лажа. Откуда я это знаю? Все просто. Крайние пару лет большая часть собеседований проходит по сети. И вместо бумажки прошу написать кусочек кода в своем любимом редакторе. Те, кто умеет программировать — справляются. Те, кто «копипаст-программисты», «stackoverflow-программисты» и «теоретические программисты» — садятся в лужу. Видел такое десятки раз. С бумажкой хотя бы можно сохранить лицо.

Я не буду приводить конкретные вопросы (иначе мне придется их поменять), однако вот на что я проверяю, и именно эти качества фиксирую в оценках по пятибалльной шкале:

Базовые качества:

- ▶ Честность.
- ▶ Интеллект.
- ▶ Зрелость кандидата.

Дополнительные (но обязательные для потенциальных руководителей проектов):

- ▶ Несет заряд положительной энергии.
- ▶ Способен побуждать к действиям других.
- ▶ Способен делать выбор/принимать решения.
- ▶ Исполнительность/добивается результатов.
- ▶ Энтузиазм.

У Сунь-цзы, великого китайского полководца, мыслителя и стратега, который не проиграл ни одной битвы, и которого я буду еще несколько раз вспоминать в этой главе, есть список требований к полководцу. Полководец — управленец высшего звена, от которого зависит жизнь и смерть, победа и поражение.

а. Ум. Макиавелли писал: *«Умы бывают трех родов: один все постигает сам; другой может понять то, что постиг первый; третий — сам ничего не постигает и постигнутого другим понять не может. Первый ум — выдающийся, второй — значительный, третий — негодный».*

- b. Доверие.** Беспристрастность. Здравый смысл. Нет зависимости от мистики.
- c. Гуманность.** Умение поставить себя на место другого.
- d. Мужество.** Самостоятельность в принятии решений. Физическая и эмоциональная смелость.
- e. Строгость.** Внимание к мелочам, деталям. Требовательность (в том числе к самому себе). Умение держать повестку дня.

Хороший, на самом деле, список.

Обязательно должен быть вопрос на «подумать». Включающий вторую систему (по Канеману). Что я имею в виду: если спросить человека, сколько будет 2×2 — он незамедлительно ответит: «Четыре». Если спросить: «Сколько будет 38×18 ?» — придется переключаться в другой режим. Это и есть вторая система. Медленная, ресурсоемкая. Люди не любят ее включать. Задач на ее проверку в интернете — куча. Подойдет что угодно из списка задач Google, например.

Мне важнее не то, что человек сразу даст правильный ответ. А как быстро он включит вторую систему и начнет думать и рассуждать логически. Бывают кандидаты, которые наотрез отказываются думать, вместо логики подсовывают свое мнение, и никогда не согласятся умножить два числа столбиком на бумажке. В последнее время таких все больше — деградация налицо. Увы, человечество глупеет.

Реже, но бывает, что кандидат как-то уж очень бодро решает ваши вопросы. Видимо, вам пора их поменять. Утекло. Имейте при себе запасные.

Серия профессиональных вопросов зависит от вакансии и требуемого уровня. Тут что-то рекомендовать сложно. Однако! Если у вас есть карты компетенций — о них мы еще подробно поговорим в этой главе — разговаривать нужно именно про те компетенции, которые вам нужны. Иногда я прошу прямо перед собеседованием заполнить карту компетенций — так можно быстрее понять сильные стороны и точки роста.

Заканчивая собеседование, есть резон рассказать про свою компанию, продукт и задачи, которые предстоит решать. **Стек технологий**, клиенты, какие вызовы сейчас есть и т. д. По сути, продать свою компанию. У меня, конечно, есть ожидание, что кандидат предварительно

почитал и изучил нас подробно. И многие так и делают. Но лишним это точно не будет.

Дайте позадавать вопросы. А, да. Ничего нет утомительнее, чем отвечать на длинный список вопросов «А какая у меня будет карьера?», «А сколько человек у вас в команде?», «А правда вы проводите стендапы и ретроспективы?» или еще более забавный «А как вы управляете проектами?». Особенно, по кандидату, которого вы внутри себя уже решили не брать. Эту пытку надо пережить стойко.

Сказать, к какому сроку ждать письма на почту. И напомнить, что, если ОК по тестам и ОК сейчас — позовете на повторное.

Если готовы озвучить решение здесь и сейчас и дать обратную связь о сильных и слабых сторонах, причем, сделать это необходимо — сделайте. Это требует некоторого мужества и навыка. «Чувак, ты завалил вот это, вот это и вот это. Вот тут ты молодец. А вот это стоит подтянуть» — разбор полетов, примерно в таком духе. Такую обратную связь в реальном режиме времени мало кто дает. Но для кандидатов она может оказаться максимально полезной. Все лучше, чем неделями ждать писем-отписок с отказами.

Бывает, что кандидат понравился, но на другую позицию. Пришел на QA, а стал копирайтером, или типа того. Возможно, что человека лучше сначала взять на стажировку. Возможно — дать платное тестовое. Или работать дальше как с внештатником. Это окей. Действуйте по ситуации.

После собеседования закройте глаза. И оцените в совокупности кандидата. Общее впечатление, от одного до пяти. Зафиксируйте. Затем посчитайте средний балл по качествам из списка выше и также зафиксируйте. Тут же или максимум до конца дня примите решение.

Если предварительное интервью вы делегировали — не смотрите на его результаты заранее. Сработает когнитивное искажение «эффект привязки». Сначала составьте свое мнение. Потом анализируйте результаты предыдущих интервью. Можете даже резюме не читать, если есть, кому делегировать первичный отбор.

Про мистику, гороскопы и прочую нумерологию. Как-то просматривали список сотрудников. Угадывали по характеру знаки зодиака. Сверяли с датой. «Блин! Вот это точно — Скорпион! Такой говнистый...» Все сходилось! Через час вскрылось — сверяли с датой трудоустройства. Это все, что надо знать про это мракобесие.

Если кажется, что с кандидатом что-то не то — вам не кажется. Не очаровывайтесь. Особенно мальчики — девочками. Родственники на работе — это слишком сложно. Ломается механизм информационного дистанцирования. Плывет дисциплина. Дома не остается общих тем, кроме как о работе. Знаю пару разводов на этой почве.

7.1.5.4 Тесты

Как-то, при сдаче экзамена на вертолетные права нас отправили к психологу. Я не ожидал подвоха, никак не готовился. Ну психолог, и психолог. Однако в его кабинете мы застряли часа на три. Психолог — барышня бальзаковского возраста — достала толстую, пыльную папку с тестами. Выдала нам их целую кипу. Сказала: «Ну, мальчики. Развлекайтесь. В 12 приду — проверю». И ушла пить чай.

Чего там только не было! От психологии до задач по математике. От аэродинамики до стрелок-кнопок-компасов. Через несколько часов мы были признаны адекватными людьми и пошли оформлять документы. А я, вернувшись в офис и побыв в шкуре кандидата, выкинул часть тестов, которые мы проводили перед собеседованием.

В данный момент мы используем несколько тестов. В основном, на внимание, грамотность и когнитивные способности. Что-то конкретное рекомендовать не могу. Ну допустим, 16-факторный опросник Кеттелла все еще не плох. Но есть один тест, который хочу НЕ рекомендовать.

У меня накопились статистические данные, что тест IQ — херня. Не чье-то мнение, типа «да это всем известно» и «а Гугл уже давно это понял» (такие инсинуации я часто слышал от желающих прикрыть свою рукожопость или неспособных обосновать свое мнение чем-то, кроме «читал в интернете»). А собственные, статистически-значимые данные, которые уже просто невозможно было игнорировать. И этот тест мы убрали.

То, что тест давал сильные сбои при повторном прохождении, было известно давно (выбросы в 140–160+ очков). Но это очень легко можно отсечь парой-тройкой вопросов на интервью. Сложнее было, когда человек с умными глазенками и явно шустрым умом показывал 97.

Причины понятны: тест строится на довольно простых типах задач. И если алгоритм тебе известен (понятна задача), найти решение — уже

не такая большая проблема. А вот если ты не понял смысл задачи из-за недостатка опыта решения именно таких задач — шансов у тебя не густо.

А далее, на интервью, уже могло сработать когнитивное искажение «эффект привязки» — особенность принятия численных решений человеком, вызывающая иррациональные смещения ответов в сторону числа, попавшего в сознание перед принятием решения. Мне было интересно собирать статистику и отслеживать за собой и этот эффект.

Выводы такие:

- ▶ Тест IQ точно херня. С реальной способностью решать реальные рабочие задачи корреляция хоть и есть, но слабая. Тест показывает только умение проходить этот тест.
- ▶ Выбросьте его.
- ▶ Общайтесь лично. Подстраивайтесь под реальную ситуацию. Давайте скидку на «нервяк».
- ▶ Отслеживайте свои когнитивные искажения.

И да, работать с умными людьми — огромный кайф! Дорого, и стоит того.

7.1.5.5 Располагайте людей по мощности

Это то, что до меня очень долго доходило. Но в очередной раз, перечитывая Сунь-цзы (то еще читиво), я обратил внимание на этот фрагмент:

Тот, кто ставит людей соответственно их мощи, заставляет их идти в бой так же, как катят деревья и камни. Природа деревьев и камней такова, что когда место ровное, они лежат спокойно; когда оно покатое, они приходят в движение; когда они четырехугольны, они лежат на месте; когда они круглы, они катятся.

Поэтому мощь того, кто умеет заставить других идти в бой, есть мощь человека, скатывающего круглый камень с горы в тысячу сажень.

«Вот она, моя ошибка! Я от всех стремлюсь требовать одинаково!» И перестал это делать. Ну не дано (или неинтересно) человеку быть фулл-стек разработчиком — не надо его туда насильно запихивать.

Не хочет QA расти до менеджера — ну и бог с ним. Пусть лучше будет хорошим QA, чем плохим менеджером. Люди, расставленные по мощности, работают в итоге лучше и мотивированнее. Чаще попадают в состояние потока. Действуют как бы сами. Остается отойти в сторонку и не мешать.

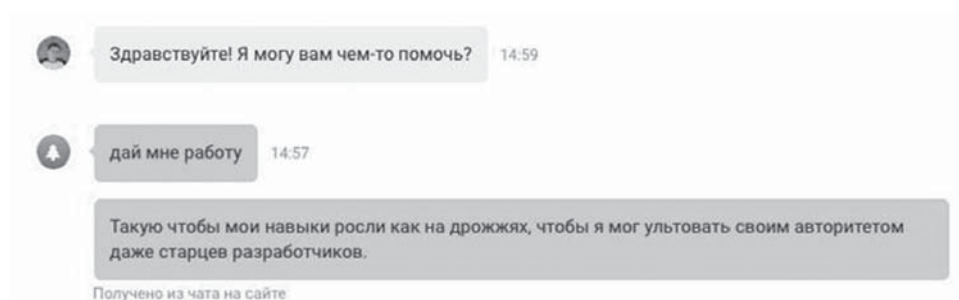
Еще доброго из трактата Сунь-цзы:

Если полководец разговаривает с солдатами ласково и учтиво, значит, он потерял свое войско. Если он без счета раздает награды, значит, войско в трудном положении. Если он бессчетно прибегает к наказанию, значит, войско в тяжелом положении. Если он сначала жесток, а потом боится своего войска, это означает верх непонимания военного искусства.

То есть выстраивать отношения с командой надо так, чтобы и в победе, и в кризисе вы вели себя одинаково и могли пользоваться как поощрениями, так и наказаниями.

7.1.5.6 Быстрое обучение, сложности и нытье

В чем измерять «быструю обучаемость»?
Быстро обучаться — не мешки ворочать!



Нашему мозгу нравится считать себя быстро обучаемым. Мозг льстит сам себе. Мы, наверное, навскидку можем вспомнить пару случаев, когда реально смогли разобраться с какой-то темой за пару дней. И как-то что-то начало получаться. Но как давно это было? Год назад? Два? В школе? Херня!

Типовая галлюцинация: «Отсутствие навыка перекрывается желанием осваивать новое и учиться». Ну-ну...

Да и чем кончилось это экспресс-обучение? Мы стали реально профи, гуру в этой сфере (признанным, лучшим в городе/стране/мире?), или стали лучшим программистом среди гуманитариев, научившись за пару дней с правильной стороны подходить к компьютеру (а потом и вовсе забросили, ибо стало скучно и появилась новая тема)? Если второе — херня два раза!

Быстро обучаться — не мешки ворочать! Прыгать с темы на тему, толком не сделав ничего, что можно без зазрения совести показать, ничего не достигнув, не получив реального результата — скверная затея. Это интеллектуальное бродяжничество.

Лучше учиться «в глубину». Получать хоть какой-то, но реально ощутимый, значимый результат. Что-то, что реально работает и приносит пользу (или радость). А вот для этого уже нужно себя заставлять пахать. Часто — через боль, через «не хочу», «не могу» и «ничего непонятно». Или чтобы кто-нибудь заставлял добиваться результатов (если не хватает своей заставлялки). Не секрет, что, делая реальные проекты в коллективе или с наставником, мы учимся быстрее, а наши навыки будут глубиннее и устойчивее.

Некоторые люди и даже целые организации становятся лучше, потому что завидуют вашей компании и хейтят лично вас.

Я видел десятки случаев, когда человек с огромным трудом и отчаяньем, слезами, матами продирался через «не хочу», «не могу» и «ничего не понятно». А потом полученный результат на несколько лет становился главным и самым ценным результатом его жизни.

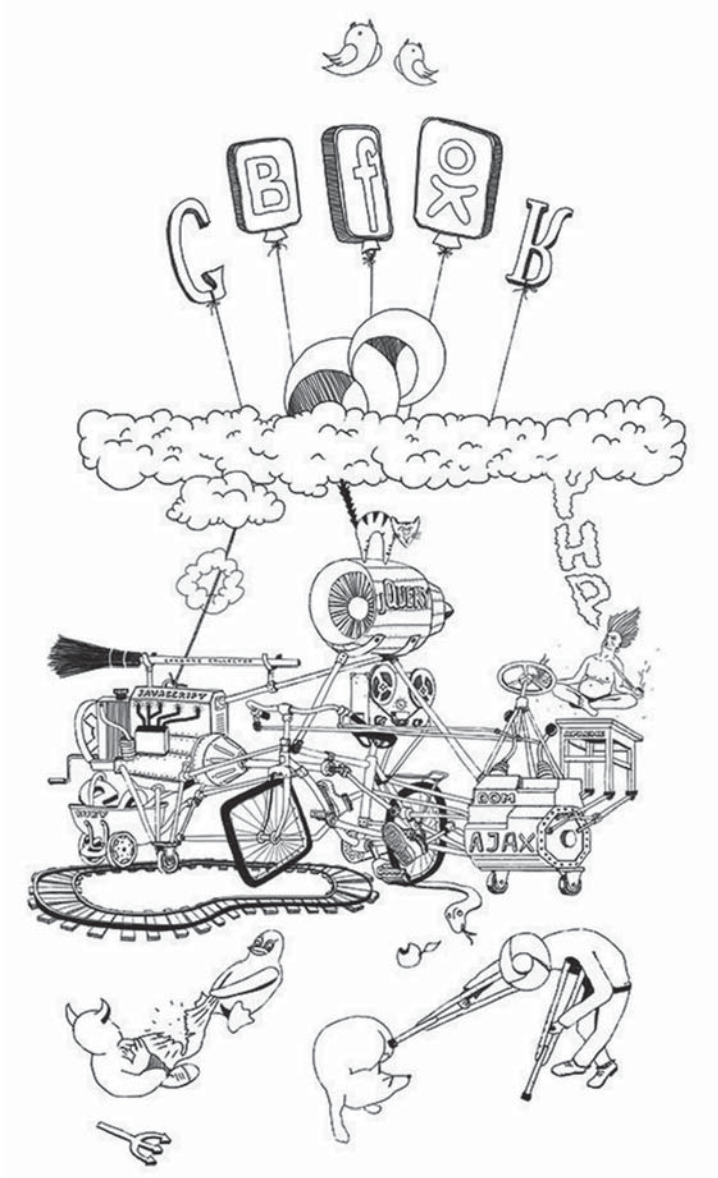
Почему? Да потому, что обучение — это награда за эффективность, нашу стойкость и терпение. А не подачка за впустую потраченное время.

Сначала они ноют, что им трудно, что они устали. Их приходится заставлять, подпихивать, подбадривать или идти на уловки. А потом, когда все получается, они считают это своим главным достижением в жизни. Всегда — своим. Черт, да мы все такие Хитрожопые!

Решение проблем ноющего показывает ему эффективность стратегии нытья и плодит новое нытье. Забивание на решение проблем ноющего ведет к дезорганизации. Учите приходить с решением. Показывайте путь. Запрещайте ныть. И не нойте сами. Тот, кто ноет — живет недолго.

**7.1.6 БЫТЬ НАСТОЯЩИМ И ПИСАТЬ КОД VS
ПЕРЕБИРАТЬ И КАЗАТЬСЯ, ИЛИ ВСЕ О НЕПРАВИЛЬНОЙ
МОТИВАЦИИ ПРОГРАММИСТОВ**

Отличный фреймворк!
Простой, чистый, производительный!
Так на его могиле и напишем...



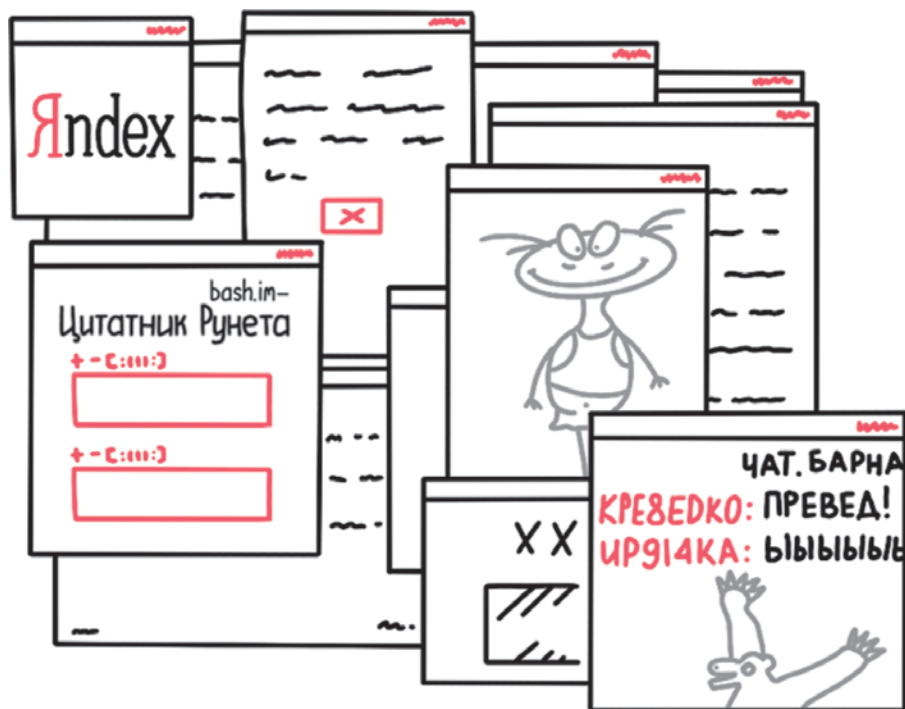
Полсотни интернет-зависимых не спят. Прижав тощие и не очень задницы к стульям, они флудят в самописном чатике. Удовольствие стоит шесть рублей в час (цена «городского интернета» в далеких 2000-х с одним сервером на город-полумиллионник).

На это хватало сэкономленного на студенческих обедах. На выход в Глобальный Интернет по 33 рубля (фигасе!) в час — уже нет. **Писк модема на 33 600.** Ночь.

Самое главное — на сервер можно зайти по FTP и выложить на всеобщее обозрение свое «ясделие». Свой сайт. Или чат. Или что ты там еще придумал. И кто-нибудь обязательно туда придет, посмотрит и оценит. И ты увидишь это на счетчике посещений (предтеча лайков размером 88×31).

В стеке технологий (правда, тогда слово «стек» употребляли с чем-то вроде «push ax») у нас было много всего:

- ▶ PHP3, который еще не успели осудить (или успели, но в «городском интернете» про это никто не знал).
- ▶ HTML4, который не во всех тогдашечных браузерах умел рендерить таблицы.



- ▶ CSS-1.0 — магия.
- ▶ JavaScript, который ламеры путали с Java.
- ▶ Шестая чашка дрянного нескафе-голд.

Эти технологии были потрясающими. В прямом смысле. Например, JavaScript мог трясти окнами браузера и открывать десять тысяч окон, отправляя в нокдаун неугодного пользователя чата. Нынешние фреймворки так не умеют.

Но через пару лет пришли хулителю из Java (которая не Script) и рассказали, что все, что мы делаем — дерьмо. И на сервере код наш — дерьмо. И на клиенте — дерьмо. И браузеры наши — дерьмо. И вся веб-разработка — дерьмо. Впрочем, доля горькой правды в этом была. Браузеры того времени и впрямь были удивительными механизмами.

7.1.6.1 Кладбище фреймворков. Начало

Глотком свежего воздуха стал Prototype (кто-нибудь помнит этот фреймворк?). Было так круто писать вместо `document.getElementById` что-то типа `$()`!

Вот только Prototype расширял функционал базовых объектов. А это посчитали дерьмовым. И старичок загнулся практически сразу, как появился jQuery.

Меня это бесило дня два. Мне нравилось писать

```
$("id").style.color = "#FFF";
```

вместо

```
$("id").css ({color: "#FFF"});
```

Но кто-то сказал, что это плохо, и я пересел на jQuery. Многие пересели.

Prototype оказался только шлюзом в мир, полный настоящего, вонючего дерьма! Оно хлынуло потоком. Маленькими кусками вроде MooTools, который реально где-то полгода делал вид, что он конкурент jQuery и Prototype. И большими кусками вроде Dojo (кто это помнит?!), Yahoo!, UI Library (в итоге — Ext JS).

Нам нужен был MVC (или HMVC?), и у нас появился выбор: Backbone, CanJS, Knockout и еще десяток новых каждую неделю. Нам надоел

CSS, и снова мы могли выбирать: Less, Sass, PostCss. Кто-то сказал, что JavaScript — дерьмо, и так появился CoffeeScript, и меня реально убеждали, что надо писать именно на нем, ибо прогресс! Блин, ну и где он теперь?!



TypeScript. ES6. Отлично, JavaScript стал типизированным, но перестал работать в браузере: теперь его приходилось **компилировать**. И не только его. Много-много рутинной магии на сервере.

Поэтому появились автоматизаторы рутинной серверной магии по типу Grunt, Gulp, Broccoli, Webpack. Меня выворачивало наизнанку от синтаксиса Angular, и я был рад сплясать на его могиле, когда появился React (к которому нужен был Redux). Даже Vanilla JS, фреймворк в 25 байт в сжатом виде, на фоне общего зоопарка выглядел не так плохо.

Каждый день появлялись новые фреймворки и библиотеки, которые решали проблемы, о которых мы даже не задумывались. Каждый день появлялось то, что можно поизучать. Мы читали Хабр, и у нас глаза блестели от счастья (или от ингибитора обратного захвата серотонина). Но не проходило и полгода, как новую хорошую технологию кто-то яростно порицал, и она становилась сначала отстоем, а потом — дерьмом.

Бывало, начинаешь проект с новыми и хайповыми технологиями и методологиями, а заканчиваешь уже с отстойными. Хотя, по сути ничего не менялось, разве что появлялась парочка статей на Хабре

«Почему Ангуляр — отстой» и «Scrum умер и больше не работает». Проект, работа и даже собственная компания казались отстоем просто потому, что кто-то сказал «MVC — для лохов!».

7.1.6.2 Не быть лохом

Правда в том, что большей части разработчиков фиолетово на сам проект. Куда важнее чувствовать себя «в тренде». Не быть лохом. Поэтому часто новомодные технологии — решающий фактор в выборе работы. И мало кто из работодателей осознается, что набирает людей для поддержки унылого проекта на фреймворках полувекковой давности.



В вакансиях студии выпячивают только те технологии, которые не успели обрасти разгромными отзывами. Мы тоже так делали и прятали под стол гадости вроде лучших отечественных CMS-систем. Да мы даже обернули ее прекрасным фреймворком Laravel, чтобы клиенты радовались маркетинговому трепу про мощность админки, а мы — пользовались MVC, не особо мучаясь.

Мы в студии не в худшем положении: можем экспериментировать и менять технологии как перчатки. У нас негласное правило: на новом проекте попробуй новую технологию. Стек всегда остается более-менее актуальным. Но глобально это не решало вопрос. Не все проекты заканчиваются сразу после релиза, ага.

Разнообразие мнений породило проблему выбора и обучения junior-ов. Я собеседовал людей, которые на полном серьезе говорили, что учили Angular, а не JavaScript. Ай хэйт ю, Ангуляр.

Я видел разработчиков, которые считали, что умеют писать «на jQuery» — точнее, они могут найти подходящий слайдер и воткнуть его на страницу. И (цитирую дословно) «настроить под себя». Эти люди гордо именовали себя «jQuery-интеграторы», но не могли отсортировать список значений в select-боксе. Они беспокоились, что разговор зайдет о чем-то, что может быть сложнее «интеграции плагинов» (точнее, занятий, которые они почему-то считают «программированием»). Когда их просили написать чуток кода на бумажке без копипасты с Гитхаба, их поры сочились гормонами сжиженного стресса.

Но черт возьми! Не они в этом виноваты!

7.1.6.3 Не читай срачи, а трогай ручками

Как-то вечером я захотел сделать небольшой проект для души: сайтик с простым бэкэндом и кое-каким интерактивом на Javascript. И решил предварительно посмотреть, на чем сейчас модно писать. Господи, зачем я это прочитал...



Та самая статья
из 2016-го

Мой проект не закончен до сих пор. Кажется, скоро я открою стартап или начну искать ко-фаундера.

Сейчас в арсенале нашей студии:

- ▶ Vue на одних проектах;
- ▶ React + Redux на других;
- ▶ Typescript, Babel + JSX + ES6 + Browserify + PostCSS + Webpack — повсеместно.

Но я не знаю, сколько времени осталось, пока они не вошли в категорию «X — дерьмо!».

Если ты талантлив и одарен, у тебя, скорее всего, есть и обратная сторона этих ништяков: ты слишком умный. Ты оптимизируешься, экономишь силы. Перестаешь изучать и пробовать технологии на практике. И все больше полагаешься на стороннее мнение. Становишься зашоренным. Изучение и применение технологии заменяешь чтением веселой статьи на Хабре про то, что она отстой.

В курилке можно назвать технологию X отстоем — это поднимет авторитет в глазах коллег и даст моральное право не изучать ее и не использовать. Со стороны выглядит так же круто, как если бы ты освоил технологию. Но это не одно и то же.

Сингулярность ближе, чем ты думаешь. Избегай зашоренности. Меньше полагайся на чужое мнение. Смотри сам. Изучай сам. И попробуй потрясти этот мир. В смысле —

```
window.moveBy (x, y)
:)
```

7.2 ТРИ СПОСОБА ПОСТАНОВКИ ЦЕЛЕЙ

Бирюзовые компании?
Делегировать сначала научиться!

Как в вашей компании ставятся цели? Например, для всей компании? Или для проекта? Или для вас лично? Цели исходят от руководителя или от рядовых сотрудников?

Одна из идей — определить, какой внутренний продукт компания покупает у каждого из своих сотрудников. Например, внутренний продукт у уборщицы — чистый пол. Сегодня-завтра-послезавтра — чистый пол. У программистов — код, делающий что надо по спецификации

и без багов. У копирайтеров — статьи, которые дочитывают до конца, на заданные темы, в заданном стиле, без фактических ошибок. Сегодня-завтра-послезавтра. До пенсии.

Хотя метода — рабочая. Но скуотища, для нашего человека! Хорошо, что в digital мы не делаем один проект миллион раз (надеюсь). Есть место для творчества. А значит, повторяемость процесса не такая, как на конвейере. Но как поставить цели такой организации? Сегодня-завтра-послезавтра?

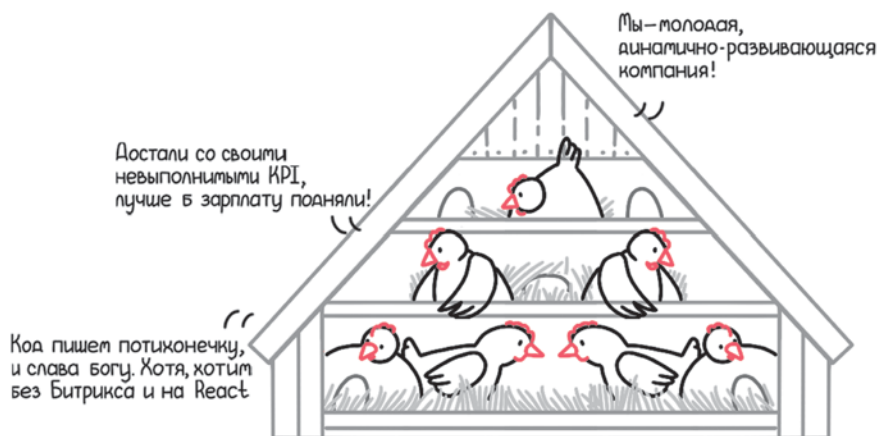
Если брать классические теории управления, тот же ТОС Элияху Голдратта, то цели должен определять руководитель. На самом деле, для чего существует ваша организация? Нормальный ли ответ: «Чтобы зарабатывать деньги, сегодня и в будущем», или так себе?

Если взять что-то более новомодное, про Бирюзовые компании — там в постановке целей надо звать уборщицу.

Так цели ставить снизу или сверху?!

1. ППР (Пришли-Поговорили-Разошлись) или **Бирюзовый ППР**. Какие-то цели обсуждаются. Но ни действий, ни реального прогресса по ним никто не отслеживает. Разговоры не превращаются в конкретные задачи. Добавление демократии и Бирюзовых соплей только усугубляет ситуацию. Разговоры разговариваем — да. Дело делаем — нет.

2. Деспотичный курятник. Гадь на нижнего, толкай ближнего, следи, чтобы сверху не прилетело. Смотри, чтоб не спихнули вниз и карабкайся вверх. Ничего не поделать, мы — социальные существа, у нас сильно выражен иерархический инстинкт.



- 3. Меткий техасский стрелок.** Идея такая: меткий техасский стрелок сначала выпускает весь барабан в стену. А затем вокруг того места, где кучнее всего попадания, рисует мишень. Часто встречается в отделах маркетинга, когда плановые показатели появляются (или подгоняются) после итоговых. Зато все — молодцы!

Итак, что же не так с целями?

1. Целей нет.
2. Цели несинхронны по всей организации.
3. Никто не отслеживает прогресс.
4. Все хотят «развиваться», изучать новое и интересное, но это не нужно компании.

7.3 ДОБАВИТЬ ЯСНОСТЕЙ, ПО СУНЬ-ЦЗЫ

Кто из государей обладает Путем? У кого из полководцев есть таланты? Кто использовал Небо и Землю? У кого выполняются правила и приказы? У кого войско сильнее? У кого офицеры и солдаты лучше обучены? У кого правильно награждают и наказывают?

По этому всему я узнаю, кто одержит победу и кто потерпит поражение.

Сунь-цзы, Искусство Войны

Итак. Нам понадобится навести и поддерживать в динамике несколько ясностей рационального уровня:

1. Ясность цели. Куда идем, чего хотим. Амбициозность и сложность задачи, вызов.
2. Ясность пути. Как достигаем? Например, за счет низких цен или за счет выхода в премиальный сегмент?
3. Ясность правил.
4. Ясность поощрений и наказаний. Мгновенная обратная связь.
5. Ясность образцов для подражаний. Посильность задачи. Важно знать, что такой путь возможен, что есть люди, которые, например, регулярно выполняют план продаж.

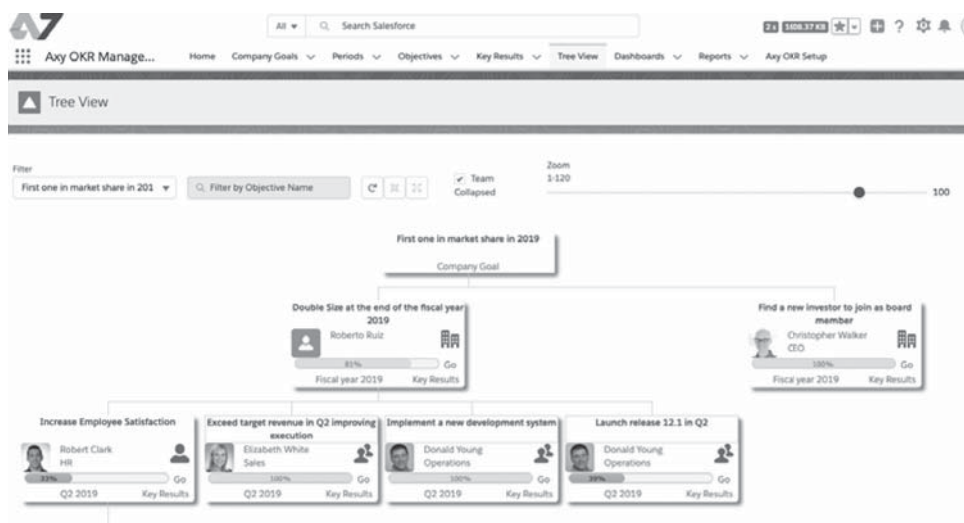
Ну и как это внедрить? Даже в вашей команде из семи плюс-минус двух человек, которыми вы управляете? А в компании вроде Сбербанка?

7.4 А БЫЛО БЫ НЕФИГОВО!

А хотелось, чтобы все было разнообразно...
А еще, чтобы по утрам не чувствовать себя безобразно.

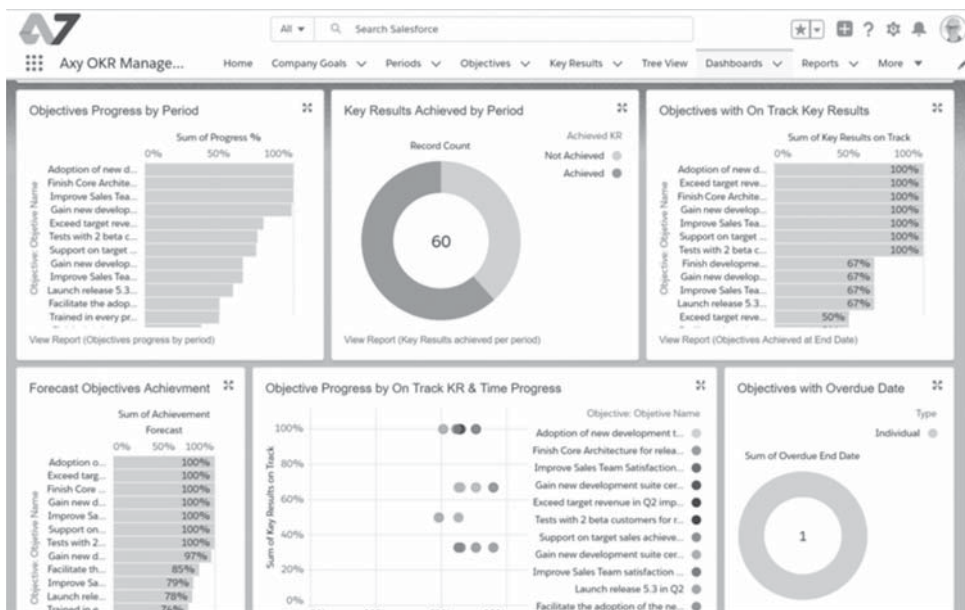
Znaki

А правда, почему нет! Хочу сидеть, смотреть в экран, как вся компания прошита целями сверху донизу. И как их в реальном режиме времени делают. И чтобы сразу было понятно, кто красавчик, а кто — не очень.



Софт на скрине — Аху OKR

Потом что-нибудь надоело — подкрутил пару параметров, вывел на дашборд и смотри-наблюдай, как все быстро переоделись и побежали в другую сторону.



Софт на скрине — Аху OKR

По-моему, идиллия. Ни с кем не надо разговаривать, никого ни в чем убеждать. Вон, все «ясности» — на дашборде. Я такую работу могу из отпуска делать!

7.5 РИТУАЛЫ, ПРЕДМЕТЫ, УДАЛЕНКА

Для того, чтобы человечество защитило себя от самих себя, издревле были придуманы РИТУАЛЫ. Не просто так, а с определенными целями. Свадебный пир и гости на свадьбе с обеих сторон — чтобы убедиться, что женятся именно те люди. Если свидетелей много — сложно сделать подлог. Горсть земли на похоронах — чтобы те, кто участвовал в погребении, были причастны (а вдруг заживо зарыли?). Игра в Planning Poker — чтобы каждый убедился в понимании задач и имел возможность высказать свое мнение (ну и был причастен к оценке проекта). Церемония награждения на рейтингах, дипломы, доски почета, благодарственные письма и медали — с кучей разных целей. И т. д.

Чтобы ритуал соблюдался правильно, он закреплялся ПРЕДМЕТАМИ. Канбан-доски, бокалы, кусочки ткани, баннеры партнеров, сцена... Все

не просто так. С предметами сложнее накосячить в ритуале. Вернее, косяк сразу становится заметным. Кто не использует предметы, многое теряет.

Кризис — это когда старые правила уже не работают, а новые еще не появились.

Один из минусов резкого перепрыгивания в удаленку (или в системы, подобные тем, что были на предыдущем скриншоте) — поломка ритуалов. Предметов становится очень мало, и большинство из них — виртуальные. Zoom, наушники, гуглдок, Фигма, Джера, Нотион и редактор кода. С точки зрения вашей бабушки — это все «в компьютере». Но чтобы сохранить корпоративную культуру (и защитить себя же) — ритуалы нужны.

Ищите и создавайте новые. Думайте.

7.6 KPI, ИЛИ ПОСОБИЕ ПО КОМАНДНОМУ САМОУБИЙСТВУ

Для написания этого параграфа в свое время было затрачено:

- ▶ 68 338 километров на поездки.
- ▶ 72 человеко-часа на почтовую переписку.
- ▶ 423 человеко-часа на эксперименты с коллективом в 30 человек.
- ▶ 88 часов на подготовку докладов и выступления на конференциях.
- ▶ 17 чашек кофе на беседу с мудрыми людьми на афтепати.
- ▶ Порядка 25 часов на набор этого текста и правку багов в нем :)
- ▶ До смерти замученный копирайтер, который был вынужден разбирать мои черновики, аудиозаписи, и вообще, спасибо ему.

Много денег и времени. Пожалуй, самым затратным (по нервам, времени и деньгам) был эксперимент над собственной командой, о котором мне безумно неловко вспоминать. Но об этом ниже.

Рано или поздно, наверное, у каждого директора возникает желание справедливо оплачивать выполненную работу. И очень многие сейчас пытаются внедрять KPI (ключевые показатели эффективности). Работает так: вы, как владелец бизнеса, назначаете конкретные цели для сотрудников. Они достигают или не достигают поставленных целей в процессе работы. Тем, кто достиг — выдается плюшка (денежная премия).

Смысл такого подхода: платить по справедливости. На сколько наработал — столько и получил. Это честно, это логично, это — прекрасно!

Ну, логично же, что:

- ▶ Продажникам нужно назначать процент с оборота. Волки должны быть голодными. (Да, есть альтернативное мнение, что применить такой подход — значит «обложить себя дополнительным налогом». Но как по мне — тут все справедливо :-))
- ▶ Офисному планктону — ставить оклад. Стабильность для них — о-о-очень важное условие существования.

А вот с творческими единицами (дизайнерами, программистами) все значительно сложнее.

Мы провели опрос руководителей ведущих диджитал-агентств и веб-студий страны на тему: «А как вы используете KPI по отношению к труду творческих единиц», в результате получили вот такую картинку:



Некоторые компании (15%) применяют KPI для оценки эффективности труда программистов и дизайнеров.

Около 25% компаний внедряют KPI в данный момент/встречают сопротивление внутри компании или же работают по упрощенной схеме.

Примерно 30% компаний производят оплату труда работников на основе субъективных оценок руководителей. Вернее, 30% сознаются в этом :-). Не сознаются оставшиеся 30%.

Самое интересное, что многие пытались внедрить KPI или пытаются сейчас. Причем, не очень успешно. Это не значит, что «KPI плохой». Плохо приготовленную пищу есть невозможно. Может, мы просто не умеем этот KPI готовить?

Но статистика говорит о том, что затруднения при внедрении есть у подавляющего большинства. И есть подозрение, что такая проблема у всех общая. Давайте попробуем разобраться.

Первое, с чем придется столкнуться при внедрении KPI, — сопротивление коллектива.

Возникает вопрос: что сильнее всего парит разработчиков при внедрении KPI?

Проведя несколько экспериментов и опросов среди коллег, мы выделили шесть основных причин:

1. Боязнь новизны. Все тотально боится нововведений, думая, что станет хуже (меньше денег, больше работы и т. п.).
2. Непрозрачная схема. Используя схему материальной компенсации со множеством параметров, мы повышаем риск того, что работники ее не поймут. Людей бесит и демотивирует, когда они не понимают, как именно им достигать наилучших результатов, или почему они вдруг получили меньше денег.
3. «А че так много?» Да, такое тоже бывает. Если схема построена таким образом, что результат этого месяца появится только через два-три. «В этом месяце я работал хуже, а получил больше. Значит, в прошлый раз мне недодали. Руководство — идиоты, ничего не понимают в моей работе!»

4. ЧСВ работника. Практически нереально попасть в самоощущение человека и выдать ему «справедливый» бонус.
5. Неполная зависимость достижения критерия от работника. От дизайнера, например, не совсем зависит, будет ли принят нарисованный им дизайн, или придется делать 50 правок.
6. Отчеты. Не знаю никого, кто любит писать отчеты, проставлять затраченное время, обещать «точные сроки».

Если посмотреть на этот список внимательно, можно обнаружить, что бо́льшая часть претензий связана с выбором, учетом, прозрачностью и адекватностью критериев.

ОК. Значит, нужно просто придумать Хорошие Критерии!

Ну такие, которые все поймут, которые не будут никого парить, которые будет просто объяснить даже на собеседовании. И чтобы было все честно, и хотелось работать еще и еще.

В общем, давайте попробуем найти Хорошие Критерии. (Кстати, «Хорошие» — для кого?) У нас есть три ключевых пострадавших стейкхолдера: владелец студии, заказчик и разработчики.

Что может быть Хорошим Критерием с точки зрения заказчика? Обычно все сводится к деньгам (ну либо каким-то фактическим результатам):

- ▶ ROI — грубо говоря, это «отдача от финансовых вливаний». Выведенный экономистами показатель не совсем применим к разработчикам: ведь они не могут контролировать отдачу от своей работы и на ходу измерять ее в деньгах. То есть не могут напрямую влиять на показатель.
- ▶ Низкая стоимость фичи. Для заказчика выгодно иметь низкую стоимость фичи. А для разработчика это — разрыв шаблона («Как это так: я получаю больше денег за то, что дешево работаю?»).
- ▶ Степень удовлетворенности. Не знаю, как ее считать, но если учитывать, что люди хотят счастья или хотя бы меньше париться (© Дмитрий Сатин), то можно предложить даже вот такую формулу:

$$\text{Удовлетворенность} = \frac{\text{Сколько фич приняли} \times \text{Коэффициент счастья заказчика}}{\text{Сколько фич сделали} \times \text{Коэффициент геморроя от разработчика}}$$

Однако реалии сейчас таковы, что прийти и предложить, например, дизайнеру зависимость его ЗП от эфемерной «удовлетворенности» заказчика — это гарантированный способ остаться без дизайнера. Нужен очень серьезный кризис, чтобы эта тема начала работать. Или очень много хороших лишних дизайнеров.

- Дата релиза. Вроде бы, все логично: сдаем проект вовремя — получаем много денег, сдаем досрочно — получаем еще больше денег. Показатель годный, но имеет уже обозначенную проблему: не все зависит от разработчика. Затык по срокам чаще всего возникает с клиенто-менеджерской стороны. (Отсюда справедливое: «Почему я должен терять в зарплате, хотя это менеджер не выбил с заказчика контент?»).

ОК. Эти Критерии, Хорошие для заказчика, очевидно, не будут Хорошими для разработчика. *(Я без иллюзий, сейчас можно запросто придумать еще 200 штук разных критериев, значимых для бизнеса).*

Но ведь можно мерить ПРОИЗВОДИТЕЛЬНОСТЬ! Это же так просто!

Или нет? В чем ее мерить? Если бы я красил забор — то все очевидно. Но есть заковырка. В нашей отрасли много мыслящих, творческих, талантливых людей, и заборы никто не красит. Давайте разберем на примере программистов. Итак, какие Хорошие Критерии оценки производительности приходят на ум?

- KSLOC. Знаете, что это? А что такое индусский код — знаете? Внедрите — узнаете. KSLOC — это количество тысяч строк кода. Если вы привязываете этот показатель к зарплате, то ждите тыщи строк копипасты. Один мой знакомый получил выполненный заказ где-то в Бангалоре — PHP-скрипт, всего за десять долларов, но на целых 20 Мбайт. И он работал!
- Количество какого-то дерьма в час (WTF/h). Количество нарисованных страниц в день, количество реализованных фич в час и т. д. Вроде бы, нормальная метрика — что-то можно реально подсчитать и использовать для раздачи плюшек. Однако возникает проблема, аналогичная предыдущему пункту: падение качества в ущерб количеству, рост технологического долга. Мотивация,

интерес, удовлетворенность — все стремительно падает вниз. Как результат, текучка и низкая квалификация.

- Количество багов. Чем меньше багов, тем больше платим. Все логично, не правда ли? На самом деле — нет. В вашей студии внедрен багтрекер? Если да — забудьте. Ваши тестировщики очень скоро договорятся с вашими программистами о том, сколько багов писать, а сколько — нет, чтобы это было не в ущерб обеим сторонам.
- Переработки. «Если ты задерживаешься на работе — ты плохо работаешь». Тоже ведь логично? Боремся с переработками: например, отключаем электричество после 18:00. Однако тут нужно помнить, что психология разработчика в корне отличается от психологии офисного планктона: если он сидит до вечера — значит, ему интересно (и это надо поощрять).

В нашей сфере люди работают, в основном, потому что им это интересно.

Не надо им мешать тупыми корпоративными правилами.

- Focus Factor. Эта метрика пришла к нам из любимого мною Скрама. Показывает, сколько задача должна была занять в идеале, а сколько вышло в итоге. «Концентрация» команды над проектом. Можно ли платить деньги на основе этого критерия? Вполне, но если ваши менеджеры не «технари», то программисты будут сознательно завышать оценки по времени, сводя к минимуму свои собственные риски. Следствие такого подхода: растягиваются сроки, заказчик негодует (или покупает не у вас). Да, и каждая планерка будет превращаться в склоки и споры за десять минут.
- Velocity. Тоже из Скрама. Пресловутая «производительность». Тут довольно неочевидно, гуманитарии могут пропустить абзац.

$$\text{Velocity} = \text{Focus Factor} \times \text{Оценка новых задач}$$

Позволяет предсказать, сколько задач команда сможет набрать в следующий этап в зависимости от того, сколько она выполнила в предыдущем. Проблемы такие же, как у фокус-фактора, плюс добавляется еще одна. Часто менеджер (особенно неопытный), почуяв-

ший, что производительность команды можно «измерить», начинает применять данный инструмент «в другую сторону»: не как измеритель успеха команды, а как инструмент принуждения.

Но Velocity не может быть точным критерием, поскольку показывает, сколько времени может занять та же самая задача, выполняемая той же командой при тех же условиях. Однако после выполнения задачи команда уже поменялась: у нее появился опыт того, как именно решать конкретно эту задачу. И метрика не сработает повторно.

- Cycle time. Время с момента, как возникла идея реализовать фичу на проекте, до момента, как она была сделана.

Мне лично очень нравится эта метрика. Одна из ключевых, которую стоит замерять и оптимизировать. Но разработчики не влияют на этот фактор напрямую. Это слишком высокоуровневая метрика. Если вы начинаете платить команде зарплату на основании того, какой у них Cycle Time, это значит, что вы как руководитель не стремитесь решить проблемы команды и разбираться в процессах, а просто переваливаете все на команду.

Попытка поставить зависимость зарплаты разработчика от высокоуровневой метрики — свидетельство менеджерской импотенции.

Итак, можно ли измерить эффективность команды? Да, можно, тем более что показателей для этого мы с вами понаписали с десяток. И еще десятка два можно придумать, лежа на диване. Другой вопрос: стоит ли делать зарплату разработчика зависимой от показателей? А вот это уже рискованно.

Я начинаю работать и делаю свою работу хорошо, потому что я профессионал, и мне это интересно. Но если меня начинают гнобить дурацкими метриками — я буду оптимизировать эти дурацкие метрики. Я буду писать тысячи строчек или рисовать десятки говнодизайнов в день. И мой интерес к работе очень-очень быстро иссякнет, я буду тупо хотеть бабла. Это называется подменой внутренней мотивации — внешней.

7.6.1 ИСТОРИЯ ОДНОГО БЕЗУМИЯ

Как-то раз «мой хороший знакомый», руководитель студии, загорелся идеей внедрить очень справедливую оплату труда, где бы учитывалась куча параметров. Естественно, к делу подошли с размахом. Написали как-то целую кучу критериев:

- ▶ ежемесячный план по отработанным человеко-часам и фактически отработанному времени;
- ▶ ежеквартальный план по сбыту;
- ▶ количество подопечных и их зарплаты;
- ▶ количество позитивной коммуникации от клиентов (удовлетворенность);
- ▶ количество повторных обращений клиента с новыми проектами;
- ▶ награды на профильных конкурсах;
- ▶ отрицательная коммуникация с клиентом;
- ▶ количество багов, найденных QA;
- ▶ рост дебиторской задолженности;
- ▶ количество багов, найденных клиентом после старта проекта;
- ▶ чтение книг, написание статей.

И еще штук 20 (полезный список, забирай ;-))

Все это было сведено в одну систему. Естественно, систему нужно было сбалансировать. Поэтому в первые несколько месяцев было решено откалибровать ее на виртуальных «фантиках». Была изобретена большая доска, на которой нарисовали список сотрудников. На доске вывешивались разные «фантики» — сразу же, как только поступал платеж, заканчивался проект или происходило какое-то хорошее (или плохое) событие, которое бы в будущем влияло на зарплату.

Буквально в течение одного часа лица сотрудников сделались сильно-сильно хмурыми. Через пару дней начались вопросы: «А почему мне меньше фантиков?» или «А почему мне не дали фантик — я же Васе помогал?»

Настроение становилось тревожным. Через неделю на оценку проектов стало уходить в четыре раза больше времени, чем уходило раньше, и каждая оценка превращалась в бесконечный спор между разработчиком и руководителем проектов. К концу месяца мало кто хотел помогать товарищу — объясняли тем, что «своей работы хватает». Вскрылось бесконечное количество ситуаций, которые невозможно было формализовать. Многие фантики выдавались по субъективным ощущениям.

Мало кто хотел работать без фантиков, напряжение росло. Производительность и мотивация — падала. Еще через месяц программу свернули. Еще через пару месяцев пропала тревожность.

7.6.2 ВЫВОДЫ О КРІ

Разные метрики стоит измерять и думать-думать-думать, как на них влиять. Но не переносить высокоуровневые метрики напрямую на разработчиков и дизайнеров. И еще.

«Разработчик состоит из четырех компонентов: тело, сердце, разум и душа.

1. Телу необходимы деньги и безопасность.
2. Сердцу — любовь и признание.
3. Разуму — развитие и самосовершенствование.
4. Душе — самореализация».

С. Архипенков

Уважайте других людей и давайте им возможность делать то, что им нравится.

И совсем последнее. Есть подозрение, что каждый руководитель должен сам понять, готова ли его организация к переходу на КРІ. Я надеюсь, эта небольшая подборка статей, которую нам удалось собрать, поможет в принятии верного решения.



Подборка статей
по теме КРІ

7.7 ОБЪЕКТ KEY RESULT (OKR) ДЛЯ СИНХРОНИЗАЦИИ ЦЕЛЕЙ ВСЕХ УРОВНЕЙ

OKR — это фреймворк для стратегического управления, придуманный умниками из Intel. OKR синхронизирует индивидуальные цели сотрудников с целями команды, а цели команд — с целями компании. В идеале должно получиться так, что сердечко каждого сотрудника бьется в ритме компании: «Тук-тук, я знаю, что я тут делаю, я понимаю, какой вклад я вношу в развитие компании».

7.7.1 ЦЕЛИ, КЛЮЧЕВЫЕ РЕЗУЛЬТАТЫ И ИНИЦИАТИВЫ

OKR расшифровывается как Objectives (Цели) and Key Results (Ключевые показатели). Кроме этого, нас будут интересовать конкретные Initiatives (Инициативы) по достижению наших целей.

7.7.1.1 Objective = Цель. Куда я иду?

OKR призывает ставить не просто цели, а амбициозные цели. Они должны быть азартными. Они могут быть даже сумасшедшими и фантастическими. Главное, чтобы они вдохновляли, заводили, всплывали перед сном. Это всегда цели на вырост — такие, которые не выполнишь на раз-два. Для достижения этих целей нужно проявить изобретательность и превзойти себя. Цели отвечают на вопрос «Куда я хочу прийти?»

Итак, Цель в OKR должна быть:

- ▶ простой и нескучной;
- ▶ мотивирующей;
- ▶ легко запоминающейся;
- ▶ амбициозной.

7.7.1.2 Key Results = Как я пойму, что достиг цели?

Ключевые показатели отвечают на вопрос: «Как я буду оценивать свое продвижение?» Убираем методологию меткого техасского стрелка. Нам понадобятся от двух до пяти конкретных метрик:

- ▶ количественные;
- ▶ измеримые;
- ▶ основанные на ценности, а не на действии.

Количество пользователей, показатель отказа и т. д. Очень похоже на KPI. Но есть одно маленькое НО! Деньги. OKR не привязаны к материальному поощрению.

Почему в этом есть смысл? Особенно в творческих коллективах — дизайнеров, программистов. А вот! Работает закон Гудхарта. Он формулируется довольно заумно: «Когда экономический показатель становится целью для проведения экономической политики, прежние эмпирические закономерности, использующие данный показатель, перестают действовать». Однако суть простая.

Если сумма значительная — результат подделывают. Какой бы хитрый KPI вы ни придумали, если речь будет идти про серьезные деньги — его хакнут.

$\forall \{KPI\} \exists B:$

Еще веселее это сформулировал Максим Дорофеев в Джедайских техниках:

$KPI(B) \in$ 

$Prj(B) \rightarrow ?$

«Для любой системы KPI существует такая стратегия, что показатели KPI при следовании этой стратегии находятся в зеленой зоне, но при этом сам проект через ж*пу идет в неизвестность».

Итак. Если бы за выполнение OKR давалась премия, а за невыполнение — не давалась, никто бы не ставил по-настоящему крутые и сложно достижимые цели, все занимались бы накруткой всякой хераборы.

Между использованием OKR и KPI нет существенных противоречий: одно дополняет другое. KPI работает в устоявшихся, регулярных, отлаженных процессах, где количество усилий пропорционально достижению результата. OKR лучше работает для чего-то нового, амбициозного.

7.7.1.3 Initiatives = Какие шаги я планирую предпринять?

Что мы должны для этого сделать? Какие проекты, задачи и фичи мы должны реализовать?

7.7.2 ДЕРЕВО ЦЕЛЕЙ

В идеале нам нужно синхронизировать цели сотрудника с целями компании. По всем уровням. Сотрудники что-то хотят для себя, плюс прокачать какие-то свои навыки, компетенции, стать в чем-то крутыми, востребованными, получить признание. У руководства и собственников свои цели, видение и задачи организации. От зарабатывания конкретной

суммы денег до помощи голодающим мартышкам в Африке — смотря какая организация. Что тут предлагает OKR?

1. Компания ставит от трех до пяти целей на год и на каждый квартал.
2. Сотрудники ставят три-пять целей на квартал.
3. Команда синхронизирует цели сотрудников с целями компании и ставит свои три-пять целей на квартал.

Например:

OKR компании

Цель: выпустить продукт на западный рынок, не привлекая сторонние инвестиции.

Ключевой результат 1: запустить новый сайт к 2020 году.

Ключевой результат 2: оборот 220 миллионов в год при сохранении рентабельности; не менее 25% — иностранные заказы.

Ключевой результат 3: выпустить востребованный SaaS-продукт на западный рынок.

OKR сотрудника

Цель: прокачаться в 3d-иллюстрации.

Ключевой результат: добавить в портфолио 3d-модель с детальной проработкой всех элементов.

OKR команды

Команда смотрит на OKR компании и OKR сотрудника и говорит: «Ага, ты хочешь прокачаться в 3d-иллюстрации, а компании нужно запустить новый сайт к 2020 году. Давай, ты предложишь концепцию сайта с 3d-иллюстрацией, и, если концепция будет одобрена, возьмем ее в работу».

И ставит свои цели:

Цель: утвердить концепцию сайта.

Ключевой результат: начать разработку с марта 2019.

На каждом уровне Сотрудник — Команда — Компания должна прослеживаться связь. Так, чтобы можно было посмотреть на цель и понять, кто над ней сейчас работает и каким образом. В данном случае мы четко видим, что над целью «выпустить новый сайт к 2020 году» работает вот эта команда, и, в частности, вот этот сотрудник.

7.7.3 СХЕМА ВНЕДРЕНИЯ OKR

- ▶ Сформулировать стратегические планы.
- ▶ Сформулировать годовые планы компании.
- ▶ Сформировать квартальные планы команды.
- ▶ Синхронизировать личные планы с планами отделов и компании. Оцифровать.
- ▶ Внедрить кураторов.
- ▶ Внедрить квартальные чеки.
- ▶ Синхронизировать квартальные планы с регулярной работой по спринтам.

Нужен ли для внедрения OKR какой-то специальный инструмент? Если пробовать это, внедрять и экспериментировать, то за инструментами лучше не гнаться, потому что и так есть очень много мест, где вы устанете внедрять. Лишний софт при таких манипуляциях только все усложнит. Однако вот вам табличка:

	Weekdone	Perdoo	Atiim	7Geese	15five
Ссылка					
Бесплатная версия	15 дней	14 дней	30 дней	14 дней	14 дней
Стоимость	От 7\$ за пользователя в месяц	От 9\$ за пользователя в месяц	От 10\$ за пользователя в месяц	От 10\$ за пользователя в месяц	От 7\$ за пользователя в месяц
Размер команды	S, M, L	M, L	M, L	M, L, XL	S, M, L
Уровни OKR	Компания Отдел Команда Персона	Компания Группа	Компания Отдел Команда Персона	Компания Отдел Персона	Компания Отдел Персона

	Weekdone	Perdoo	Atiim	7Geese	15five
Целе- полага- ние	Снизу вверх + Сверху вниз	Снизу вверх + Сверху вниз	Снизу вверх + Сверху вниз	Снизу вверх + Сверху вниз	Снизу вверх + Сверху вниз
Свод- ный отчет	HTML + PDF + XLS + по элек- тронной почте	PDF + CSV + по элек- тронной почте	HTML + по электрон- ной почте	HTML + PDF + XLS	CSV

7.7.4 А НАДО ЛИ? СКОЛЬКО СТОИТ? КОГДА НАЧИНАТЬ?

Тема с OKR похожа на то, что должно происходить в каждой компании на уровне здравого смысла. Это все очень понятные вещи про постановку целей, про обратную связь и, казалось бы, каждая компания должна это использовать.

И, конечно, есть десять тысяч причин, почему это может не сработать локально, с отдельно взятым сотрудником. Например, сотруднику интересно «расти и развиваться», но совсем неинтересно решать текущие задачи. OKR только обострит эту проблему и приведет к более быстрой ротации перспективных кадров.

OKR — это, скорее, тюнинг, чем обязательная схема. И это стратегический инструмент, который сработает на большом количестве людей. Но не гарантирует успеха в конкретном, отдельно взятом случае. Хотя шансы повышаются.

Можно про это начинать задумываться, если вас в вашей digital-компании человек 30–50 и серьезно задумываться — если 70–100 и более. Пока вы маленькие, пока вы у всех на виду — обычный Scrum с обычными ретроспективами справляется со своими делами. Небольшие спецназ-отряды делают свое дело.

OKR лучше сработает в продуктовой разработке, чем в заказной. Когда есть клиентская разработка — очень сложно организовать постановку целей по OKR. Слишком разбалансированный поток работ, слишком широкий нужно иметь кругозор, слишком сложно строить долгосрочные планы (или они содержат слишком много допусков).

Слишком много «слишком»... И, скорее всего, у большинства компаний, оказывающих высококомпетентные услуги, та же проблема.

Как только появляются короткие по циклу услуги, как таковые OKR сложно привязать к реальности. Как-то можно. Но это вопрос непростой, и он требует думалки в каждой конкретной компании и даже команде. А вот где это хорошо ложится — в продуктовой разработке.

Однако, если вы строите армию — похоже, что тут нужен другой подход. OKR используют гиганты вроде Intel, Uber, Dropbox, Oracle, Google, LinkedIn. И OKR позволяет не только регулярно мониторить уровень сотрудника, его удовлетворенность, планы и прогрессы, но и синхронизировать цели и задачи.

OKR, по сравнению с теми же самыми системами Waterfall, — экономически более дорогая штука. С каждым индивидуально составляя план работы, синхронизируй цели, обсуждай, спрашивай, наставляй, веди. Это ресурс. Для учредителей точно будет маргинальнее организовать все так, чтобы люди сидели в их угрюмой конторе и пилили беспросветно, годами, код по какой-нибудь ТЗ-шечке на технологиях из 90-х, не тратили время на обучение. Но это тлен, смерть и разложение. OKR помогает найти баланс между личными целями карьеристов (в хорошем смысле этого слова) и экономикой компании.

Итак. В спецназе энергичный менеджмент способен справляться с постановкой целей и помощью сотрудникам без каких-то фреймворков. В регулярной армии нужен регулярный подход, и OKR тут может хорошо сработать. Хотя это чертовски дорого, но, похоже, что, когда у вас человек 100 и больше, альтернатива OKR — хаос и мракобесие.

7.8 AGILE И ЦЕЛЕПОЛАГАНИЕ. ФОКУСИРУЕМ КОМАНДЫ НА ЦЕЛИ

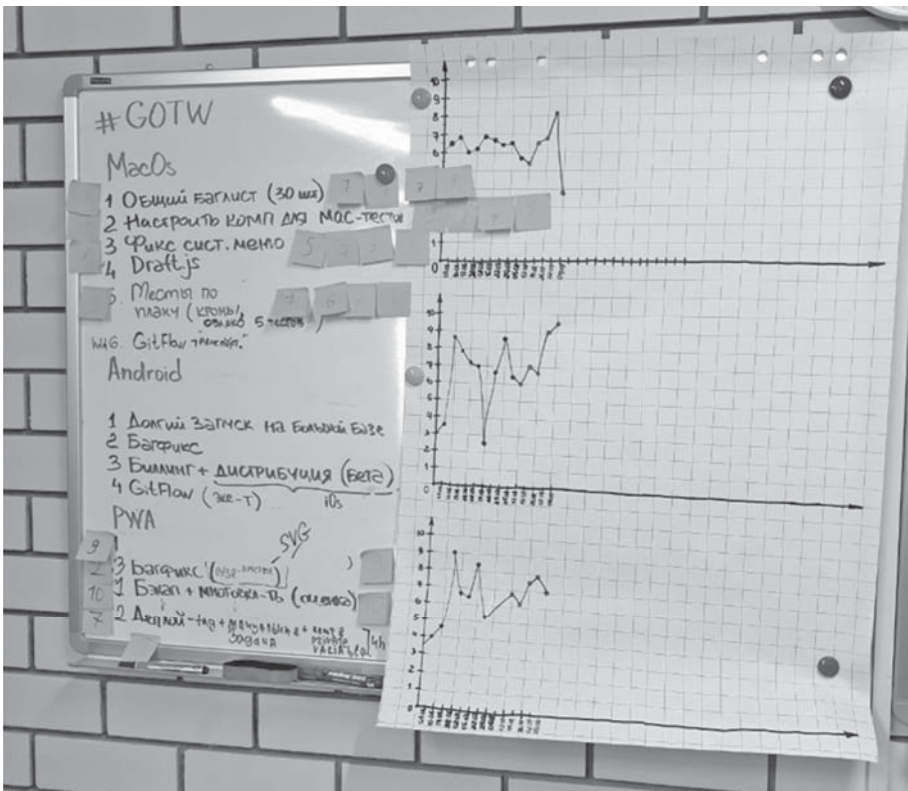
По сути, все, чем мы управляем — это внимание команды.

Поэтому есть смысл в начале каждого спринта (на этапе планирования) сформулировать одну-три емкие цели и зафиксировать ключевые показатели так же, как это делается в OKR. Повесьте эти цели на доску или на стену, возле которой ежедневно собирается команда. Об эти цели нужно постоянно спотыкаться глазами.

Далее. Нам нужно сфокусировать команду на целях спринта. Проще всего это делать на стендапах. Каждый день. Вместо классических трех вопросов для стендапов — используем эти:

- ▶ Что ты **сделал для достижения целей** спринта вчера?
- ▶ Что **мешает** нам **достичь целей**?
- ▶ Что мы **можем с этим сделать сегодня**?
- ▶ **Признать усилия и достижения (даже мелочи).**

Далее. На ретроспективе мы проверяем достижение целей спринта. Есть несколько методик это сделать. Попробуйте такую: раздайте каждому члену команды липкие стикеры по количеству целей. И дайте проголосовать, от одного до десяти, насколько он считает, что мы достигли цели.



Доска с целями трех команд.
Стикеры — с оценками команды, достигнуты ли цели.
График — среднее достижение целей (за несколько спринтов).

Нормально будет, если мы достигаем 70–80% целей. Если достигаем 100% — значит, цели были не слишком амбициозные. Если 40–50% — это злит, бесит и приводит к выученной беспомощности.

Замечу, что ЦЕЛИ — это не ФИЧИ СПРИНТА. А достижение целей — это не Burndown Chart, о котором мы говорили в главе 3. Это еще одна метрика, более верхнеуровневая. Burndown показывает объем поставки за один спринт. Давайте заострим внимание на этой разнице: какими могут быть цели спринта?!

На основе фич:

- ▶ Запустить раздел «монетизация».
- ▶ Разработать новую поисковую систему.

На основе ценности:

- ▶ Увеличить число еженедельных посещений активными пользователями в среднем до 3,3.
- ▶ Снизить отток доходов до 1%.
- ▶ Увеличить вовлеченность (пользователи, заполняющие полностью профиль) до 75%.

В заказной разработке у вас есть только первый вариант. Однако в продуктовой у вас развязаны руки, и команда, сфокусированная действительно не на клепании задачек, а на развитии продукта, способна творить чудеса.

Итак. Вы можете добавить в классический Scrum или Канбан фокусировку команды. Фокус на бизнес-результате лучше, чем на предсказуемости объема поставки. Если вы все сделаете правильно — команда возьмет обязательства по цели спринта.

7.9 КАРТЫ КОМПЕТЕНЦИЙ. АТТЕСТАЦИИ И ЧЕКИ. ЯСНЫЙ КАРЬЕРНЫЙ ТРЕК СОТРУДНИКА

Мы используем карты компетенций, чтобы сфокусировать развитие сотрудника в нужном нам направлении и мониторить его прогресс.

Наиболее полной у нас получилась карта руководителей проектов. Наименее полной — карта для дизайнеров: действительно, не очень

понятно, как оценивать уровень их мастерства. Кроме того, с определенного уровня ценность дизайнера определяет не то, насколько красивые картинки он научился рисовать, а его образ мысли, способность решать бизнес-задачи, самостоятельность, переговорные навыки и навыки публичных выступлений. Soft Skills, те же навыки, что важны руководителям проектов. В общем, так получилось, что эти карты во многом пересекаются.

Дизайнер	Куратор	Аналитик	Проектант	Наименование	Р	К	М	Л	Д	К
					0 — совсем 1 — мало 2 — средне 3 — много 4 — очень много 5 — отлично	Для оценки компетенции необходимо по 5-балльной шкале: 0 — совсем не умею, 1 — мало, 2 — средне, 3 — много, 4 — очень много, 5 — отлично, 6 — отлично, 7 — отлично, 8 — отлично, 9 — отлично, 10 — отлично				Какая в следующий раз
0	0	0	1	— Ретроспективы менеджеров. Проведение и контроль внеочередных	7			1		
0	0	0	1	— Проведение обучения менеджеров	6			1		
0	0	0	1	— Разбор факторов и выдача обратной связи исполнителям с применением техники успешного послания и наведения	5			4		Провести курс по SEO. Метрики, инструменты, как запускать, отслеживать, как карта кликов работает и т.д.
0	0	0	1	— Управление процессами на внутренних проектах компании (все этапы, включая подготовку контента)	5			1		
0	0	0	1	— Проверка логов у менеджеров	6			5		
0	0	0	1	— Проверка бюджетов и выполнения планов менеджерами	7			0		
0	0	0	1	— Контроль ставок менеджеров и приема дипломных работ	6			4		
0	0	0	1	— Внедрение согласованных изменений в процессы компании. Проверка результатов	7			1		
0	0	0	1	— Контроль офиса во время отсутствия генерального и финансового директора (контроль нагрузки РМ, офис-менеджеров, управление ресурсами филиалов (покупки, аренда помещений), отпуски/больничные/отпуска, форс-мажоры)	7			1		

Карта компетенции. Фрагмент

Раз в полгода имеет смысл садиться с каждым сотрудником и обновлять его карту компетенции:

- ▶ Что прокачал с прошлого раза? Какие прогрессы? Что не получилось? Почему? Что потрогал на практике, а что — только в теории.
- ▶ Что прокачивать в следующий раз.
- ▶ Оценка куратора (в свободной форме).
- ▶ План прокачки с конкретными тикетами в рабочее время (чтение, внутренние митапы, семинары, внутренние проекты).
- ▶ Рекомендации для прокачки в личное время (сильные, слабые стороны).
- ▶ Текущий грейд и понятные перспективы роста до следующего. Можно привязать рост грейда к росту компетенций, но обязательно через достижение результатов и роста зоны самостоятельности и ответственности сотрудника. Иначе можно сформировать ожидание: грейд растет из-за того, что я изучаю что-то новое, а не из-за практических достижений и ответственности.

Плохие новости: это довольно ресурсоемкая операция, требует много времени и эмоциональной энергии куратора. Больше пары-тройки таких разговоров в день вы вряд ли проведете. Кроме того, может

оказаться, что сотрудника некуда дальше выращивать — у вас просто нет задач, которые ему интересны.

Более того, с точки зрения большинства бизнесов, бесконечный рост сотрудников не нужен и даже вреден. Это неприятно. Но так есть. Нужно выполнение какой-то функции в заданных параметрах. А рост компетенций свыше необходимых приведет только к ротации кадров и денежным потерям.

В общем, вы вскрываете эту проблему стеклянного потолка. И либо ищете новые вызовы. Либо получаете головняк. Хотя, с другой стороны, это дает время на маневр, вроде своевременного начала поиска замены.

7.10 360

Created: 17 Sep 2019 Updated: 27 Sep 2019

Обратная связь по аттестациям

Ирина

Были проблемы со шрифтами (не те). Контентные картинки не были выгружены. Приходится дергать [redacted]. Блоки не выровнены по центру.

Ольга [redacted] Понравилась дотошность в тестах и что он всп очень понятно разъяснил. Доступно. Помог разозраться\

Ольга [redacted] Нравится работать. Видит проблему и предлагает решение. И все объясняет подробно.

Ирина [redacted] Нравится с ним контактировать, т.к. он не просто берет и что-то делает за тебя, но показывает первый враз как, причем так, что второй раз вопрос не возникает. нравится подход к оперативным вопросам.

Поработала с [redacted] Очень понравилась ее оперативность и готовность помочь. Сразу оценила, сказала что сейчас готова сделать задачу.

На этих же аттестациях, где идет работа и обсуждение карты компетенций, можно каждого сотрудника попросить и зафиксировать обратную связь о коллегах, с которыми он взаимодействовал. Просто пишем это в свободной форме и смотрим за тенденциями. К кому-то, возможно, нужно присмотреться внимательнее, кому-то дать премию.

Затем можно донести обратную связь, которую вы собрали, до адресатов. Можно делать это анонимно.

Как-то я собрал такую обратную связь и сделал распечатки по каждому человеку. Убрал имя автора, но оставил имя адресата обратной связи. Был конец года, и у нас намечался корпоратив. Я скинул распечатки в коробку и предложил вытянуть две-три и зачитать вслух. Мероприятие вызвало бурный интерес. Однако, стоило мне отвернуться и отойти от коробки — тут же на нее накинудись всем офисом. Каждый искал, что написали про него. И с живым интересом читал это. Кто-то вслух. Кто-то был очень доволен. Но точно не осталось неравнодушных. И да, это было весело!

Итак. Во время обновления карт компетенций соберите обратную связь 360, каждый о каждом (с кем взаимодействовал).

- ▶ Открытый разговор со ВСЕМИ тет-а-тет.
- ▶ Донесение выданной обратной связи.

Проблемы, с которыми вы можете столкнуться:

- ▶ очень ресурсоемко (ментально выматывает);
- ▶ не все умеют давать обратную связь;
- ▶ может всплыть пара душных людей, и это будет большая проблема.

7.11 ОДИН НА ОДИН

По-хорошему, вы должны разговаривать с каждым членом команды один на один. Снижать температуру, выяснять настроение, помогать и синхронизировать мысли и планы. Собирать и выдавать обратную связь. Особенно в период, когда команда только формируется.

Лучше это делать раз в неделю. Обычно с подготовкой такой разговор занимает в районе часа. Остается только посочувствовать руководителям, у кого в прямом подчинении человек сорок. Так бывает при плоских оргструктурах. Или у руководителей, которые мало кому доверяют. Или у руководителей, которые не обучили выдаче обратной связи линейных руководителей.

Мало кто из руководителей готов тратить столько времени и энергии на это. Итог: сотрудники чувствуют себя брошенными. В долгосрочной перспективе это становится проблемой. Собственно, самая частая жалоба сотрудников — недостаточная или плохая обратная связь.

7.12 РАЗБОР ПОЛЕТОВ

Разгром полетов.
(для насыщенных ретроспектив)

Когда я учился летать, после каждого полета, уже на земле, мой учитель отводил меня в сторонку и выдавал обратную связь. Указывал на ошибки. Говорил, как действовать. Подбадривал, если я сделал что-то особенно хорошо. Отвечал на вопросы.

Пока ты в воздухе, такой разбор сделать затруднительно. Однако каждый раз на земле разбирался каждый полет.

Такие же разборы после групповых полетов, тройками. Технология была одна и та же. Полет окончен — идем в сторонку и без посторонних разбираем, что и как. Часто это была жесткая обратная связь с матерными терминами. Но всегда по делу. И это помогало расти быстрее.

Заведите себе привычку делать разбор полетов после каждого взаимодействия с вашим сотрудником. Провели совместную с дизайнером демонстрацию макетов заказчику — дайте обратную связь дизайнеру, как он держался на демке. Сразу же, пока ощущения свежи.

Делайте это наедине, без посторонних ушей. Делайте это откровенно. Смотрите в глаза. Но не переходите на личности. Разбор полетов может стать одним из самых ценных ритуалов для сотрудника.

7.13 ОБРАТНАЯ СВЯЗЬ

Итак. Есть проблемы с выдачей обратной связи. Ее либо нет, либо мало, либо она негативная. Ругать ругают, а хвалить не хвалят. Ключевых причин тут две:

1. Боимся:

- ▶ «А я скажу, а он ответит...» — боимся реакции сотрудника на критику.
- ▶ «А чтоб не расслаблялись...» — я их похваляю, а они — расслабятся.
- ▶ «Я такой человек, который не хвалит» — встречается и такое. Часто.

2. Ленимся. Это занимает кучу времени, и всегда найдутся дела поважнее.

Обозначу четыре типа обратной связи. Позитивная, Негативная, Хреновая и Нейтральная. Давайте их разберем.

7.13.1 НЕЙТРАЛЬНАЯ. КВИТАНЦИИ. НАУЧИТЕСЬ ГОВОРИТЬ «СПАСИБО!»

Опять же, пример из авиации. При радиообмене летчик должен повторить те параметры, которые он услышал от диспетчера с земли. Высота такая-то, давление такое-то установил. Иначе диспетчер не понимает, правильно ли его поняли и услышали ли вообще. Без эмоций, просто перечень фактов и параметров. То же самое вы можете просить от сотрудников при устном делегировании (говорили об этом в пункте 1.2.2). Но и вы сами можете выдавать такие квитанции.

К нейтральной обратной связи можно отнести «Спасибо», когда вы получили результат или ответ от сотрудника. Приучите себя говорить или писать «Спасибо» на каждую транзакцию. Без этого нет законченности. В чатах для этих целей лучше работают эмодзи, стикеры и лайки на сообщения. Фраза «ОК» вместо «Спасибо» — тоже ОК. Слова вроде «Принято», «Услышал», «Я вас понял» — скорее раздражают, чем закрывают транзакцию, хотя служат тем же целям. Уберите их.

Итак. Закрывайте каждую транзакцию общения с сотрудником — квитанцией. Научитесь говорить «Спасибо» вместо «Услышал».

7.13.2 ПОЗИТИВНАЯ

Позитивной связи должно быть много. Хвалите больше и чаще. Хвалите прилюдно: и в присутствии человека, и когда его нет рядом. Говорите офигенным людям, что они — офигенные! Необязательно словами — это может быть теплый взгляд, улыбка, смех и т. д.

Это творит чудеса. Для этого нужно развивать наблюдательность и иметь запас личной энергии и времени. Вы вряд ли сможете кого-то похвалить, если задержаны между десятком проектов и бегаєте от сотрудника к сотруднику. Хочу выделить четыре вида позитивной обратной связи:

- ▶ **Благодарность.** У вас искренние чувства благодарности к человеку. Не сдерживайте их. Хочется сказать человеку что-то хорошее — сделайте это незамедлительно. Не для него. А для себя. Найдите самые простые слова и сделайте это. И совершенно без разницы, как это будет выглядеть со стороны, как и ктоотреагирует и кто и что подумает. Вам по кайфу — расскажите про это миру.

- ▶ Похвала. Хвалим человека либо за полученный результат («Какой классный макет! Ты — молодец!»), либо за те качества, которые он проявил во время работы с вами («Ты — крутой!»).
- ▶ Признание. Признаем усилия человека, которые он приложил для достижения результата. Разница с предыдущим пунктом тонкая, но есть. Похвала как бы говорит: «Ты молодец, что сделал это для меня» и как бы намекает на то, что, если бы ты не сделал или сделал бы не так, ты бы молодцом не был. Хотя говорящий может не иметь в виду такую конструкцию, но она может быть так услышана. Признание фиксирует усилия человека, которые он приложил. Факты и конкретику. Пожалуй, это наиболее ценная обратная связь.
- ▶ Манипуляции. Позитивная обратная связь дается не с целью выдачи обратной связи, а с какой-то другой, корыстной целью. Например, лесть. Голодные на обратную связь люди проглотят и такое. Разборчивые — отнесутся с недоверием. И правильно сделают. Люди не любят смешанные сигналы. Обман — путь войны. Не хотите воевать — не обманывайте.

Итак. Хвалите больше и чаще!

7.13.3 ХРЕНОВАЯ

Наверное, каждый получал обратную связь, которой лучше бы не было вовсе:

- ▶ Пассивно-агрессивная. Ты получаешь обратную связь и такой сидишь и не понимаешь, а что с ней сделать. «Ну ваще...», «Ой, все», «Все с тобой ясно», «Вова в своем амплуа», «Сеошники — такие сеошники», «Ну что с вас, программистов, взять...», «Могли бы и сразу все предусмотреть». Вот это вот все. Не делайте так.
- ▶ Классический коучевский «Сэндвич с Дерьмом». Сначала похвалили. Потом наругали. Потом опять похвалили. Булочка, кашка, булочка. Кроме странного послевкусия от такой обратной связи мало чего остается. Люди не любят смешанные сигналы.
- ▶ Письменная критика (даже благая, даже дела ради). Хотите получить жесткий конфликт или увольнение сотрудника с бесконечной ненавистью к вам — напишите ему длинное сообщение с критикой.

Вполне можно с искренней, по делу. Подробно опишите, где, как, почему он не прав. Кроме желания поскорее от вас избавиться ни к чему обычно это не приводит.

Итак, хреновая обратная связь не достигает никакой цели, кроме как поскорее от вас избавиться. Если вам нужно именно это — пожалуйста. Но нужно ли? Может, вы просто хотели выдать Негативную обратную связь?

7.13.4 НЕГАТИВНАЯ ОБРАТНАЯ СВЯЗЬ

Еще Александр Сергеевич Пушкин говорил:
«ДУШИ прекрасные порывы».

Надо правильно расставлять акценты, товарищи офицеры.

Контр-адмирал В. Г. Доброскоченко

Это самый сложный вид обратной связи, который только может быть. Именно поэтому тут много букв. Однако!

Количество негативной обратной связи, при ее качественном использовании, должно постоянно сокращаться. Иначе вы делаете что-то не то.

Собственно, цель негативной обратной связи — сделать так, чтобы в будущем не было подобных проблем и проступков.

Пока вы, как руководитель, только учитесь, вы неминуемо надеетесь здесь ошибок. Будут обиженные люди. И даже несправедливо обиженные. Но это не должно вас останавливать. Почему все, кроме вас, имеют право на ошибки, а вы — нет? Почему программист может наделать сотню багов, а вы не можете ошибиться в формулировании обратной связи? Можете. Но вопрос в том, анализируете ли вы эти ошибки, учитесь ли на них?

Итак, в процессе взросления вас как руководителя ошибки неизбежны. Анализируйте их. Особенно, если довели кого-то до крайностей. Это — обратная связь на вашу обратную связь, если хотите.

Негативная обратная связь требует мужества и много подготовительной работы. Я настойчиво рекомендую изучить книгу **Павла Сивожеlezова «Сложные переговоры с подчиненными»** — особенно главы, касающиеся разбора ошибок и проступков. Здесь будут только краткие конструкции, которые помогут строить такие разборы.

Итак. Негативная обратная связь может появиться в трех случаях:

Ошибка подчиненного. Подчиненный действовал в рамках своих компетенций (регламентов, знаний, полномочий и т. д.), но допустил ошибку. **За ошибки наказывать нельзя!** Однако нужно провести разбор полетов и сделать выводы на будущее. Повторные ошибки, которые были подробно разобраны — уже являются проступком (косяк).

Проступок. Было какое-то нарушение (регламентов, правил, права обычая). Тут все сурово. Проступки надо пресекать, причем так, чтобы в будущем они не повторялись.

Иррациональное поведение руководителя. Тут два момента:

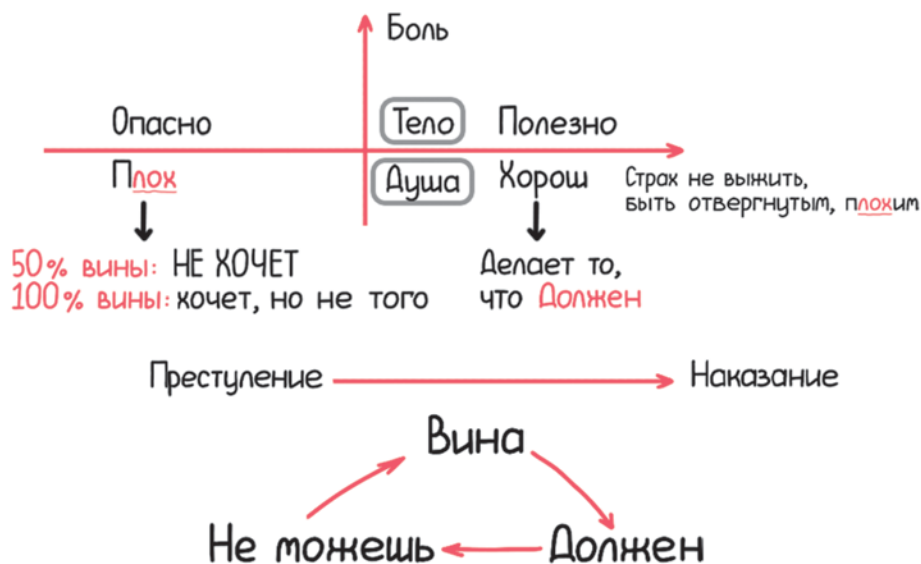
- Эмоциональный. У меня хреновое настроение, поэтому все огрели. У меня хорошее настроение — всем премия. Это почти всегда плохо работает и снижает авторитет руководителя. Проблема в том, что со своими эмоциями вы мало чего можете сделать в моменте. Максимум — отрефлексировать и проанализировать последствия. Поймали себя на этом — отрефлексируйте.
- Иррациональное управление. Это высший пилотаж, из области искусства. Лучше всего про иррациональный уровень написано в книгах Владимира Константиновича Тарасова. Приведу цитату:

«Иррациональное управление означает, что руководитель говорит своим сотрудникам примерно следующее: нет никаких ясностей, но вы люди взрослые, понимаете, что именно нужно делать. Работайте. Кто не понимает, с теми прощаемся. Но, чтобы выйти на этот уровень, сперва нужно выстроить все предыдущие ступени! А сами ценности не просто декларируются, а задаются через различные прецеденты и поступки первого руководителя».

*Владимир Тарасов, «Социальная технология
в вопросах и ответах».*

С ошибками и проступками нам всем очень сильно засрали мозги и многим привили чувство вины.

7.13.4.1 Как нам засрали мозги чувством вины



Как нам засрали мозги чувством вины

Любое живое существо стремится увернуться от Опасного в сторону Полезного. Опасное — это Плохо. Полезное — Хорошо. В обществе Плохих людей изгоняли. И был реальный страх не выжить. Собственно, изгоняют и сейчас: увольняют из компании, изолируют в специальных заведениях и т. д. Плох = лох. А если ты пЛОХ — у тебя есть реальный шанс быть отвергнутым из общества и не выжить. Побьют, съедят, опустят по социальной пирамиде.

О том, что мы делаем что-то Плохое — у нас есть ряд сигналов. Например, Боль. Боль — это сигнал об Опасности: осторожно, тело разрушается.

Чтобы не доводить до физического разрушения тела — у нас есть шикарная машинка по предсказанию будущего: наш мозг. Он без перерыва анализирует текущий контекст и на основе прошлого опыта делает прогноз опасности. Наверное, все в детстве хватало что-то горячее и обжигались, прежде чем поняли, что такое «горячо».

Прогноз опасности называется Страх. Страх помогает выжить. Люди, у которых нет страха (вообще), не доживают и до 12 лет. Пер-

вые системы управления в человеческом обществе использовали эти два механизма: Страх и Боль. Методы никуда не делись и поныне. Их используют государства. Но в современном мире, полном возможностей, с сытыми, умными, креативными людьми это такое... Более того, всегда был класс людей высокого уровня духовности, на которых эти методы не работали в принципе. Однако человечество не уgomинилось и придумало кое-что похуже. Чувство вины. И с этим попали практически все.

Вина — это боль души. Раз ты Виноват — значит, Должен. Если Должен, то, как правило, не можешь (например, не можешь вернуться в прошлое и исправить ситуацию). Значит, виноват еще сильнее. Круг замкнулся. Чтобы избавиться от Вины за свое Преступление, ты Должен понести Наказание. Если ты этого не хочешь (не хочешь делать то, что Должен) — к твоей Вине добавляется еще 50%. Если (не дай Бог) ты хочешь, но хочешь не того, чего от тебя требует манипулятор («Ах ты вон чего захотел?!») — к Вине добавляется еще 100%.

И вот в этой чехуе мы с вами все живем. Подробнее про эти механизмы можно посмотреть в книгах Алексея Капранова и в его аудиокурсе «33».

Сейчас давайте сойдемся вот на чем. Перманентное чувство вины — это очень вредная штука. Она срезает под корень яйца мужикам и целым организациям. Управлять через чувство вины, которое постоянно давит на людей, — очень плохая идея. Появляется выученная беспомощность. Имитация работы вместо работы. Перекалывание ответственности. Ничего никто не решает без главного руководителя. В креативных, быстрых, зрелых командах такое мракобесие процветать не должно.

7.13.4.2 Карма. Хансей. Индугьгенция

Очевидно, что за сотрудником может тянуться хвост проблемных проектов и задач. Чем дольше работает сотрудник, тем хвост длиннее. Карма копится.

Самая большая проблема — то, что карма мешает двигаться и развиваться дальше. Коллеги могут припоминать что-то типа: «Э... это Васька. Он, знаешь, уже сколько проектов завалил». И очень хочется карму почистить...

В любой момент можно вспомнить, что Вася когда-то накосячил. И сказать, что он всегда так делает. Это — манипуляция в чистом виде. И этот хвост проблем необходимо отрубить. Чистить карму.

Люди часто увольняются, чтобы отрубить этот хвост. Приходишь на новое место. Садишься, включаешь компьютер. И ты еще пока ничего не накосячил, ничего не должен. С тебя будут мало спрашивать и, скорее всего, будут учить. Вот именно ради этих первых нескольких дней все и затевалось. А дальше, если человек не менял своих привычек и не работал над собой — все повторится, но на новом месте.

Карма копится у всех: у рядовых сотрудников, у руководителей проектов, у клиентов и у самих digital-компаний. Для ее чистки нужно регулярно и системно разбирать возникающие проблемы и больше к ним не возвращаться. Разобрали, приняли меры, поехали дальше. Карма обнулилась, больше к этой претензии не возвращаемся. Делать это нужно как на ретроспективах (командная карма), так и на разговорах один на один.

Карма может быть элементом геймификации или даже реальным параметром, который можно посмотреть в профиле каждого сотрудника. В этом случае должны быть Ясные правила начисления и чистки кармы. Про пять Ясностей по Сунь-цзы мы уже говорили в главе 7.5 — Ясность правил, Ясность поощрений и наказаний, Ясность образцов для подражаний — это, как минимум, то, что нам потребуется. Можно обнулять карму раз в год, выдавая какие-то материальные плюшки или дополнительные дни отдыха за позитивную карму.

Более жесткий вариант кармы — количество жизней. Например, девять (как у кошек) или три. Жизни могут списываться за какие-то жесткие проступки. Восстанавливаться за время безошибочной работы, трудовые подвиги или решение форс-мажоров. Тут также важны негласные правила. Мне не очень нравится этот метод, хотя я применял его точно (например, если человека приходилось переводить на повторный испытательный срок в случае чего-то ужасного).

В японском менеджменте мне попадалось понятие Хансей (см. Джеффри Лайкера «Дао Toyota»). Они там ребята замороженные, нам их понять сложновато. Означает оно примерно следующее: «посто-

янный самоанализ». Что-то типа стыда, но конструктивное. Это когда Вася накосячил, ему говорят «пойди испытай хансей». Сходи, подумай, что за дичь ты сотворил. Вася пошел, отрефлексировал это дело, сам себе почистил карму, и пришел бодрый, веселый, с решением всех проблем наготове.

В нашей ментальности это мало с кем работает. Скорее, вы получите встречное обвинение или пачку оправданий. Нам чертовски важно быть правыми, даже когда мы не правы. И чтобы к нам отнеслись с пониманием.

Тем не менее, формула: «Ребята! Случилось дерьмо. Вот оно, смотрите (показать пальцем)! Давайте подумаем, как его убрать/исправить/избегать в будущем», без обвинений и наездов на конкретных людей — рабочий механизм в зрелых коллективах, где есть кредит доверия и позитивный опыт совместного решения проблем.

Итого, что требуется:

- ▶ Заниматься непрерывным самоанализом. Разбором ошибок и проступков.
- ▶ Быть искренне заинтересованным в решении проблемы.
- ▶ Быть искренним с собой, коллегами и клиентами.
- ▶ Подходить к анализу систематично.
- ▶ Как элемент геймификации можно внедрить понятие «кармы» (или количества жизней, но это более жесткий ход, и он уже не про геймификацию). И правила, по которым меняется карма.
- ▶ Чистить карму и не возвращаться с претензиями по вопросам, которые были разобраны.
- ▶ Следить за тенденцией: если никакие разборы вопросов и меры не помогают и проступок повторяется из раза в раз, возможно, пора с этим завязывать.

7.13.4.3 Разбор проступков

Сложность выдачи обратной связи для руководителей проектов в том, что во время этой процедуры мы должны, как минимум, не проиграть по трем параметрам: Власть (в идеале после разбора она должна укрепиться), Дело (как минимум, не должно пострадать) и Отношения (не должны испортиться, а в идеале — укрепляться).

Слишком мягкое поведение руководителя, который многое терпит, закрывает глаза, ведет к потере власти и расшатыванию дисциплины.

Слишком жесткое ведет к плохим отношениям в коллективе, упадку морального духа и вредит делу.

Что выбрать? Жесткость или мягкость по отношению к проступкам?

Точность. Негативная обратная связь, чтобы она сработала, должна быть очень точной. Гораздо точнее, чем позитивная. Согласитесь, «Я даю тебе премию, потому что ты — молодец», может быть хоть как-то нормально воспринято. А вот: «Я наказываю тебя, потому что ты — говнодел» — какая-то жуть за гранью добра и зла.

Шаги к разбору проступков:

- ▶ Эмоциональная подготовка. Успокоиться, убрать эмоции. Особенно, если цена ошибки была высока.
- ▶ Рациональная подготовка. Собрать факты.
- ▶ Уединиться с сотрудником. Негативная обратная связь или разбор ошибок должны проводиться только один на один.
- ▶ Выслушать первую версию.
- ▶ Задать вопросы. Расспрашивать подробно, в деталях.
- ▶ Классификация поступка. Это ошибка? Или это проступок? **Ошибки разбираем, но за них не наказываем.** Проступки будут иметь последствия (например, кармические). Ошибка, которую уже разбирали подробно, является проступком.
- ▶ Формулирование ошибки/проступка. Четко, **точно** сформулировать, что именно было нарушено и в какой мере.
- ▶ Вывод на признание ошибки/проступка.
- ▶ Установить моральные причины. Внутри проступка всегда лежит моральная причина (лень, невнимательность, халатность, заносчивость и т. д.). Доводить разбор нужно до моральной причины.
- ▶ Извлечение уроков. Какие выводы сделаны на будущее? Что будет изменено?
- ▶ Завершение разговора. Фиксация мер и договоренностей.
- ▶ Совершенствование. Проведение мероприятий по системному устранению ошибок в будущем. Это может быть как изменение рабочего процесса, регламентация, так и дообучение сотрудника необходимым навыкам.
- ▶ Анализ. Это делает руководитель наедине с собой. Насколько получилось сохранить или улучшить параметры Власти, Отношения

и пользы для Дела? Удалось ли выйти на моральную причину? Удалось ли не сорваться на эмоции? Такой анализ улучшает процедуру обратной связи.

Подобный разбор занимает от 20 минут до часа и довольно сильно выматывает. Очевидно, затевать его ради мелочи, придираться и лезть под кожу всем каждый божий день — плохая затея. Достаточно сделать замечание или «разбор полетов», убедившись, что все все правильно поняли. Однако серьезные ошибки и проступки нужно разбирать именно по такой схеме. Фигни становится меньше. За подробностями еще раз отправлю к книге Павла Сивожелезова и к старой, но все еще актуальной статье о наказаниях в digital (QR-код со ссылкой на нее).



7.13.4.5 Наказания. Чек-лист и пособие для молодых руководителей и интровертов

Следует с осторожностью пользоваться методом кнута и пряника. Особенно по отношению к мазохистам и диабетикам.

Наказания не любят: о них не говорят на тренингах, в книгах пишут только об успехе и плюшках. Но сами наказания есть в любых коллективах.

Я тоже чертовски не люблю и не хочу мусолить эту тему, и оставляю только ссылку, факультативно, на подробный материал и краткий чек-лист.

- ▶ Профилактика вместо наказаний.
- ▶ Задача — сделать человека лучше.
- ▶ Редко, но грамотно.
- ▶ Моральные наказания вместо материальных.
- ▶ Руководитель обязан наказывать подчиненного, если тот нарушил установленные и доведенные до него правила.
- ▶ За ошибки наказывать нельзя! Ни взглядом, ни жестом, ни кривым выражением лица.
- ▶ В сложных случаях проводите root cause.
- ▶ Прощение и амнистия — разные процедуры.
- ▶ Взаимозачет косяков недопустим!
- ▶ Не торопитесь наказывать. Разберитесь в ситуации и соберите факты.

- ▶ Шкала наказаний — дистанцирование и приближение к себе.
- ▶ Наедине. Ни в коем случае не на людях!
- ▶ С холодной головой.
- ▶ Задать вопрос. Спокойным тоном.
- ▶ Сделать паузу. Дождаться ответа. Морали не читать!
- ▶ Не дайте увильнуть. Отсекайте отмазки. Задавайте повторные вопросы.
- ▶ Включение дурака и поиск дыр в регламентах за рамками здравого смысла — повод для отдельного разговора или расставания.
- ▶ Заставьте увидеть ситуацию под правильным углом. Лицом к зеркалу!
- ▶ Докопайтесь до моральной причины.
- ▶ Закончите на позитиве.
- ▶ Обнулите карму.
- ▶ Тренируйтесь на тех, кого не жалко.
- ▶ Некоторых людей наказывать нельзя.

Негативная обратная связь. Чеклист

Подробнее: blog.sibirix.ru/2016/07/26/punishment

- | | |
|---|--|
| 1. Профилактика вместо наказаний | 13. С холодной головой |
| 2. Задача — сделать человека лучше | 14. Задайте вопрос. Спокойным тоном |
| 3. Редко, но грамотно | 15. Сделать паузу. Дождаться ответа. Морали не читать! |
| 4. Моральные наказания вместо материальных | 16. Не дайте увильнуть. Отсекайте отмазки. Задавайте повторные вопросы |
| 5. Руководитель обязан наказать подчинённого, если тот нарушил установленные и доведённые до него правила | 17. Включение дурака и поиск дыр в регламентах за рамками здравого смысла — повод для отдельного разговора или расставания |
| 6. За ошибки наказывать нельзя! Ни взглядом, ни жестом, ни кривым выражением лица | 18. Заставьте увидеть ситуацию под правильным углом. Лицом к зеркалу! |
| 7. В сложных случаях проводите root cause | 19. Докопайтесь до моральной причины |
| 8. Прощение и амнистия — разные процедуры | 20. Закончите на позитиве |
| 9. Взаимозачёт косяков недопустим! | 21. Обнулите карму |
| 10. Не торопитесь наказывать. Разберитесь в ситуации и соберите факты | 22. Тренируйтесь на тех, кого не жалко |
| 11. Шкала наказаний — дистанцирование и приближение к себе | 23. Некоторых людей наказывать нельзя |



7.13.5 СООТНОШЕНИЕ ПОЗИТИВНОЙ, НЕГАТИВНОЙ И НЕЙТРАЛЬНОЙ ОБРАТНОЙ СВЯЗИ

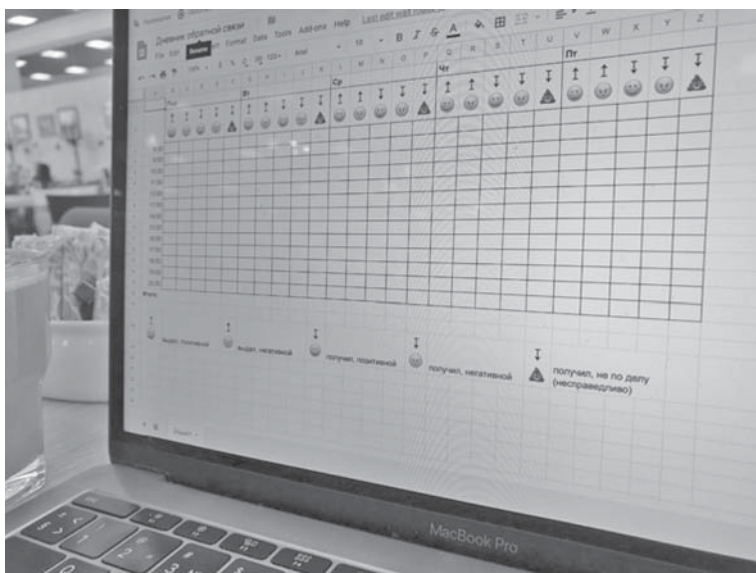
Британские ученые подсчитали, что для компенсации одной негативной обратной связи (ее послевкусия) нужно целых шесть позитивных транзакций. Михаил Литвак, известный российский психолог, к моделям которого мы еще обратимся в этой главе, считает, что выгоднее вообще ни на кого не ругаться. Все плохое произойдет и без вас. А вот хорошее — не факт.

Однако как понять, соблюдается ли соотношение шесть к одному?

7.13.6 ДНЕВНИК ОБРАТНОЙ СВЯЗИ. ПРАКТИКА

Я предлагаю в течение пары недель поиграть в игру и вести табличку-дневник обратной связи. Фиксируйте, сколько за день:

- ▶ Выдали негативной обратной связи.
- ▶ Выдали позитивной.
- ▶ Выдали нейтральной.
- ▶ Получили негативной.
- ▶ Получили позитивной.
- ▶ Получили хреновой (не по делу, например).



Дневник обратной связи. Пример

По итогам пары недель посмотрите, соблюдается ли у вас здоровое соотношение позитивной/негативной обратной связи? Достаточно ли обратной связи вы выдаете? А получаете? Нет ли душнила? Не являетесь ли душнилой вы сами? Смогли ли вы разбирать ошибки и проступки полноценно? Стало ли проступков меньше? Улучшилась ли ваша Власть? Отношение? Польза для Дела? Отрефлексируйте это.

7.14 СТРАТЕГИЧЕСКИЕ СЕССИИ

Время от времени нужно посмотреть на все, что вы делаете, со стороны. OKR предполагает и рекомендует проводить такие сессии раз в полгода. Лучше за рамками стен компании. В идеале — на два дня. Вот такое расписание мне попало для подобных стратегических сессий.

День 1		День 2	
8:00-9:00	Бизнес-контекст	8:00-9:00	Изменения планов
9:00-10:30	Концепция продукта	9:00-11:00	Работа команд
10:30-11:30	Архитектура и практики разработки		
11:30-13:00	Контекст планирования и обед	11:00-13:00	Презентация итоговых планов и обед
13:00-16:00	Работа команд	13:00-14:00	Риски программы
16:00-17:00	Рецензирование черновиков планов	14:00-14:15	Голосование
17:00-18:00	Рецензирование руководством и решение проблем	14:15-???	Переделка планов, если нужно
			Ретроспектива

К сожалению, такую роскошь мы себе позволить не можем. Дорого. Предпочитаю ретроспективы и проработку конкретных проблем, оставляя стратегию за собой.

Однако не проводить стратегические сессии было бы ошибкой. Сейчас я делаю это отдельными группами (например, работаю с продуктовой командой SingularityApp или только с руководителями проектов). И компактно сжимаю стратегическую сессию до одного дня. Команды это действительно бодрит, и появляется чувство перспективы. Пробуйте.

7.14.1 СИНХРОНИЗИРУЕМ ЦЕЛИ: МЕТОД ДЖОБСА

Что делать, если вы сами себе стратег (или даже диктатор), а мнение и идеи команды выслушать хочется? Ну так выслушать, чтобы и их не обидеть, и не быть обязанным действовать не по своему плану? Вообще, конечно, игры в демократию — это всегда риск. Капитан у корабля должен быть один. Корабль, где каждый себе капитан, будет гнить на рифах. Но есть прикольный метод. Говорят, им пользовался Стив Джобс, про тяжелый характер которого тоже много чего говорят.

Итак:

1. Проводим стратегическую сессию.
2. Команда вырабатывает десять целей, которые она хотела и могла бы достичь.
3. Джобс вычеркивает три, которые ему не нравятся.
4. Команда оставляет три, которые будет достигать.
5. ...
6. Profit!

7.14.2 ПАЛЬЧИКОВОЕ И СТИКЕРНОЕ ГОЛОСОВАНИЕ



Интересный метод — голосование на пальцах.

Пальчиковое голосование позволяет быстро оценить идею целей по пяти или десятибалльной шкале. Если идей или проблем, которые нужно решать, много — раздайте каждому из участников по два или три стикера. Попросите наклеить на те идеи, которые он выбирает. Где больше стикеров — та идея и победила.

7.14.3 ВРЕМЯ НА УЛУЧШЕНИЕ КОМАНДЫ, КОДА, ПРОЦЕССОВ, СЕБЯ

Может ли инициативный работник быть дисциплинированным?

Да, может. Причем, сразу в двух аспектах:

1. Проявлять инициативу в рамках своих компетенций (выделенных ему ресурсов, бюджетов). Таким образом, не нарушая дисциплины, повышать эффективность, производительность труда и добиваться новых целей. Например, у директора по продажам может быть право менять кадровый состав. И, хотя в целом его все может устраивать, он может проявить инициативу и заменить пару среднячков на одну «звезду».

2. Наиболее интересный пример: в рабочем процессе специально остаются легальные технологии для продвижения своих инициатив. Например, резервы времени на улучшение кода. Резервы времени на самообучение. Возможность обращаться с инициативой к руководству в специально отведенное для этого время и т. д. В небольших боевых группах это может быть сформулировано так: «приказы могут обсуждаться, но выполняются».

Сколько времени на это отдать — зависит от ситуации на предприятии. Фазы проекта, дедлайнов, бюджетов.

Обязательно ли это? Нет. Если смотреть только на потолок — пол вообще мыть не надо. Но рано или поздно вы споткнетесь. И рассмотрите пол о-о-очень подробно.

7.15 ПРОФИЛАКТИКА ЭМОЦИОНАЛЬНЫХ ВЫГОРАНИЙ

Терпеть не могу тему эмоциональных выгораний. Мало того, что вокруг нее нереально много всякой хераборы: сектантства, шарлатанства, балаболии и высшей математики. Хуже! Как только про нее говорят или думают слишком часто — тут оно и происходит.

Это как читать медицинский справочник болезней с симптомами: можно случайно найти у себя все болезни. Самосбывающееся пророчество, старт которому дает жалость к самому себе. Поэтому дисклеймер: **не зацикливайтесь на эмоциональных выгораниях.**

Безопаснее считать, что их нет. Если уж совсем подгорели, словили депрессию (а от нее люди в окно выходят) — не занимайтесь само-

лечениями по книжкам и видео из интернета. Сходите к специалисту: психотерапевту. Хорошему! Он имеет право выписывать медикаменты. Не к психологу, а к психотерапевту. Психолог — это, как правило, девочка, чтобы поболтать ни о чем.

7.15.1 КРИСТАЛЛ ПЕЗЕШКИАНА

Нельзя себя сохранить.

Важно грамотно себя растратить.

На что вы тратите свою энергию, время и внимание? Интересную модель предложил немецкий невролог Носсрат Пезешкиан. Разбив сферы активности человека на четыре сектора: Тело, Деятельность, Контакты и Смыслы.

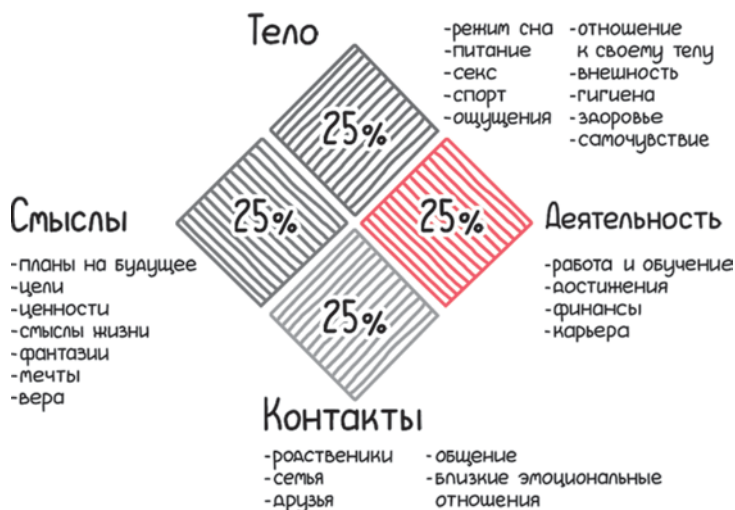
Тело — это Режим сна, Питание, Секс, Спорт, Ощущения, Отношение к своему телу, Внешность, Гигиена, Здоровье, Самочувствие и т. д.

Деятельность — Работа и обучение, Достижения, Финансы, Карьера.

Контакты — Родственники, Семья, Друзья, Общение, Близкие эмоциональные отношения.

Смыслы — Планы на будущее, Цели, Ценности, Смыслы жизни, Фантазии, Мечты, Вера, Хобби, Путешествия, Приключения и т.д

Сколько процентов ВАС вы тратите на каждую из этих областей? Прежде чем читать дальше, ответьте себе на этот вопрос. Я подожду.



Кристалл
Пезешкиана
здорового
человека

Как-то возле метро мне встретился парень-инвалид. Какой-то весь перекошенный. У него на шее висела картонка с крупной надписью маркером: РАБОТА. Шел мелкий, мерзкий дождь и толпа угрюмых прохожих. А парень стоял без зонтика, без навеса, со своей табличкой, и без перерыва повторял:

РАБОТА-РАБОТА-РАБОТА-РАБОТА

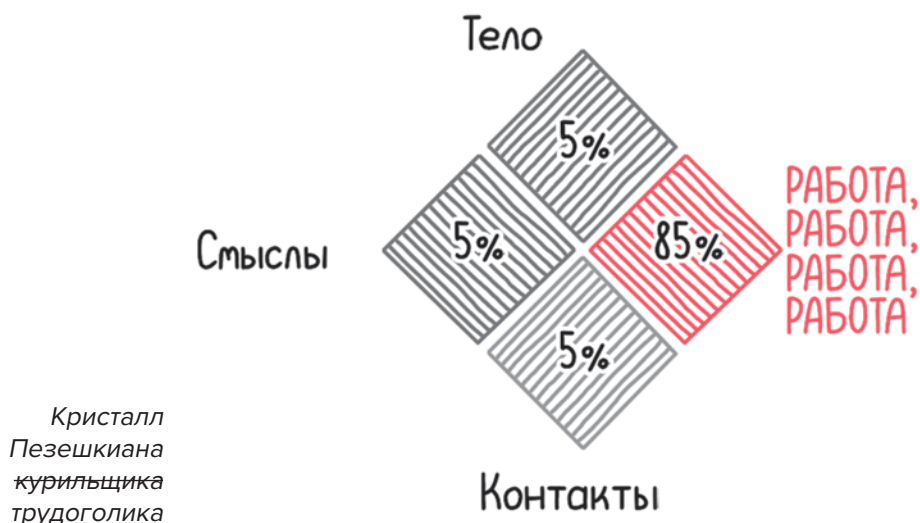
РАБОТА-РАБОТА-РАБОТА-РАБОТА

РАБОТА-РАБОТА-РАБОТА-РАБОТА

Через полтора часа я возвращался со встречи с клиентом. Нужно было ехать на следующую станцию, уже третью за день, на другой конец города, куда-то в Бирюлево. Яндекс-такси тогда еще не придумали. У метро все так же шел дождь и ~~рота красноармейцев~~. И перекошенный парень-инвалид все так же стоял со своей дурацкой табличкой и как заведенный повторял: РАБОТА-РАБОТА-РАБОТА-РАБОТА.

«Да ужжж...» — подумал я. И нырнул в метро.

Проверьте, не перекошило ли вас как-то так?



У количества работы есть естественный ограничитель: будете много работать — заболаете и будете не работать. С этим чаще всего сталкиваются молодые предприниматели, которые все берут на себя. Но стоит ли доводить до кризиса? Иногда можно рискнуть. Цель оправдывает средства, и все в таком духе. Чаще — нет.

7.15.2 ХОРРОР-РЕЙТИНГ

Хоррор-рейтинг профессий:

- ▶ Шахтеры (8,3 балла).
- ▶ Полицейские (7,7 балла).
- ▶ Строители и журналисты (по 7,5 баллов).
- ▶ Актеры (7,2 балла).
- ▶ Политические и общественные деятели (7 баллов).
- ▶ Врачи (6,8 баллов).
- ▶ Водители общественного транспорта (5,4 балла).
- ▶ Фермеры и дипломаты (4,8 балла).
- ▶ Сотрудники библиотек и музеев (2,8 балла).

Я думаю, фактор рисков у руководителей проектов сопоставим с врачами. Та же ответственная работа с людьми, к которым надо проявлять эмпатию. В заказной разработке, вероятно, он еще выше, чем в продуктовой и управлении in-house командой на стороне клиента.

Хоррор-рейтинг факторов:

- ▶ Личностные. Бросила подружка. Кошка заболела. Дома какая-нибудь хрень творится.
- ▶ Статусно-ролевые загоны. Установки, с которыми работают психологи. Вроде того, что мы точно знаем, как должно быть. А оно почему-то не так.
- ▶ Корпоративные загоны. Всякая хрень на работе.

РБК давали статистику по России: 79% сотрудников испытывали синдром профессионального выгорания. Чуть позже, во время массового перехода людей на удаленку из-за самих знаете чего, появилась статистика по Калифорнии. Также, порядка 70%. Боже, это в Калифорнии, где солнце-пальмы-океан круглый год, Голливуд, охренительные зарплаты и легалайз. Люди, вы чего? Успокойтесь... Но нет!



Статистика
РБК

Хоррор-рейтинг корпоративного выгорания:

- ▶ Высокая нагрузка/дефицит времени (работа-работа-работа-работа вместо жизни).

- ▶ Высокая самоответственность.
- ▶ Содержимое работы.
- ▶ Работа с людьми.
- ▶ Текучка или нестабильность на предприятии.
- ▶ Самостоятельность/несамостоятельность в работе.
- ▶ Обратная связь (или ее отсутствие — а это мы уже разбирали).
- ▶ Условия работы (стул неудобный, офис далеко, удаленка или ее отсутствие).

Милота какая. Ребят, мы в зоне риска. Особенно «эффективные» менеджеры!

— Что может эффективный менеджер за два дня такого, что у других занимает неделю?
 — Задолбаться.

Хоррор-рейтинг ядовитых мыслей:

- ▶ Я не имею права на ошибку.
- ▶ Я обязан сдерживать себя в любой ситуации.
- ▶ Я не могу себе позволить предвзятое отношение.
- ▶ Я обязан являть собой пример для подражания.

Есть у вас такие мыслишки? Очень жаль! Скоро появятся ядовитые чувства.

Хоррор-рейтинг ядовитых чувств (растут из мыслей):

- ▶ Вина перед собой и другими за неделанное, невыполненное (а это мы с вами уже проходили).
- ▶ Стыд за ошибку или промедление в помощи.
- ▶ Обида от недооцененности.
- ▶ Страх перед неудачей.

Пора к доктору? Минуточку! Есть тест!

Найдите в интернете опросник «Работоголизм» (автор Е. П. Ильин). После этого вам мало не покажется. Вот ни разу нельзя узнать себя в этих фразах:

Хоррор-рейтинг из опросника «Работоголизм»:

1. Организационные недостатки на работе постоянно заставляют нервничать, переживать, напрягаться.

7. Когда я прихожу с работы домой, то некоторое время (часа два-три) мне хочется побыть наедине, чтобы со мной никто не общался.

11. Я откровенно устал от проблем, с которыми приходится иметь дело на работе.

21. Иногда самые обычные ситуации общения на работе вызывают раздражение.

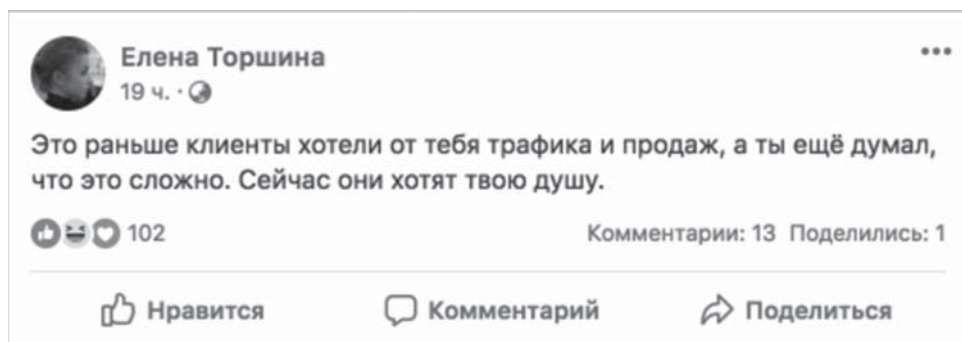
24. При воспоминании о некоторых коллегах или партнерах у меня портится настроение.

27. Обстановка на работе мне кажется очень трудной, сложной.

Это даже читать больно. Но я продолжу. Итак, digital-продюсеры, особенно в заказной разработке, по степени риска близки к врачам.

«Зачастую негативные эмоции, проявляющиеся у медицинского работника при работе с пациентами, возникают из-за негативных чувств самого пациента. Ухудшение в отношении медицинского работника к пациенту является следствием вникания в негативные переживания пациента. Это своеобразное “заражение”, а именно переход эмоций от пациента к медицинскому работнику» — пишут врачи сами про себя.

Токсичные клиенты? Никогда такого не было, и вот опять!



Ну и напоследок, есть же еще коллеги. Программисты, со своими умностями, подколками, неуважением к труду менеджеров, бесконечным чувством собственной важности и претензиями, что им все все должны, например. Отсюда целых две загадки:

Про Бога:

- Чем Бог отличается от программиста?
- Бог не считает себя программистом.

Про Мужика:

- Что общего между руководителем digital-проектов и Настоящим Мужиком?
- Он всем все время ДОЛЖЕН. И по-любому ВИНОВАТ!

Да, да, все те же засранные с детства мозги. Чувством вины, которое самоиндукцией медленно, но верно убивает нас, спиливая мужикам руководителям яйца под корень.

У меня плохие новости для миллениалов: кажется, выгорание неизбежно. Готовьте огнетушители.

7.15.3 СИНДРОМ МЯСОРУБКИ

Глаза безумные. Красные. Слюнка изо рта течет. Охота молча сидеть и смотреть в стену. Или молча ненавидеть людей.

Примерно так заканчивается рабочий день менеджера проектов, который решил (вернее, оно само так получилось) дофига эффективно планировать рабочее время.

7.15.3.1 Запуск мясорубки

Причем начинается все хорошо. В лучших традициях тайм-менеджмента.

Ты планируешь неделю. Срочное-важное. Матрица Эйзенхауэра. Матрица — главное кидалово гуру тайм-менеджмента, которые сами не способны внятно, четко и однозначно ответить на вопрос: «Как же спроецировать эту матрицу в линейный список дел?!» Подсказка: с точки зрения математики, однозначно — никак. Решений множество, выбирай любое. Думай головой второй раз (первый был, когда ты сочинял эту матрицу или вспоминал, как она называется и что это за дерьмо вообще). Короче, ну эту матрицу и этих гур.

Сначала расставляешь в план крупные дела. Потом расписываешь мелочевку. Оставляя время, чтобы отследить тот момент, когда все пойдет через жопу. Ты ли им управляешь?

Материшься, плачешь или закуливаешь, потому что один хрен все не успеть. Опять. Ну и ладно, это и так было понятно. Сабай-сабай, как говорят в Таиланде (означает нечто среднее между «и пусть весь мир подождет» и «авось прокатит»).



И ведь катит! Классика тайм-менеджмента, работаем. Неделя, вторая, третья. Может быть, даже несколько месяцев это будет получаться. Пока не раскрутится мясорубка.

7.15.3.2 Мясорубка набирает ход

Это происходит незаметно. Появляются дела, которые ты не предусмотрел. А мог бы. Опережающий звоночек клиенту. Дополнительная контрольная точка для разработчиков. Небольшой брейншторм на десять минут по дизайну иконки.

Мелочь ничего не прощает большому. Не проверил, что клиент прислал тебе акт (который так громко обещал подписать вчера) — лови головняк с переделками и грязными инсинуациями на следующей неделе («А мы думали, что вы сделаете не так, как мы вас просили, а как-нибудь еще. Вы же профессионалы»).

Но мы же взрослые люди, мы учимся на своих ошибках. И чтобы не промотать мелкие дела в следующий раз — планируем и их. Звонки.

Пропинки. Контрольные точки. Время на постановку задач. Время на проверку задач. Время на планирование времени на планирование. Все это дерьмо.

Проблемы с ним две. Первая. И вторая.

Большая часть мелочевки должна делаться строго в определенное время. И в ней задействованы другие люди, нифига не приучаемые встраиваться в наш график.

- ▶ 16:10 — Проверить, что Вася не упоролся (или что он эскалировал, что упоролся).
- ▶ 16:15 — Вася упоролся, но эскалировал (ибо в прошлый раз, когда он этого не сделал, его сильно били). Теперь надо проверить Васю через полтора часа.
- ▶ 17:25 — Звонок Иван Ивановичу Внеземному на пять минут (потому что позже у него рейс в Берлин, он летит на выставку на неделю — хрен его потом вообще выцепишь).
- ▶ 17:26 — Ой, не дозвонился. Внеземной не отвечает.
- ▶ 17:40 — Внеземной позвонил сам. САМ! Первый раз за три года! И просил перенести разговор на 18:32. Точность — вежливость королей, епрст.
- ▶ 17:40 — Кажется, тут должна была быть контролька упороида-Васи... Фак!
- ▶ 18:32 — Внеземной не смог. Улетел.



Ты берешь кота за рога (так говорят в Чернобыле) и пытаешься предусмотреть и сделать все это. В конце дня из списка запланированных задач готово дай бог 50%. Баю-бай, завтра будет день сурка.

Короче: чем больше и точнее планируешь, тем быстрее твой список дел превращается в ФАРШ.

Осознание этого наступает в момент, когда надо поработать над чем-то большим, четырехчасовым. Но такого большого и цельного куска времени нет в твоём графике. Все разбито офигенно важной мелочевкой, с которой, если её не запланировать, будешь иметь последствия.

Безумные красные глаза. Слюнка. Смотреть в стену, ненавидеть людей.

7.15.3.3 Дело не в инструментах

Еверноут, гуглкалендари, омнифокус, напоминания, ту-ду-лист, помодоро... Аппстор ломится от программ, грозящих превратить вас в полностью эффективного сапиенса. Дело не в инструментах. Они есть на любой вкус и цвет. Стать полностью затраченным можно в любой программе. Или даже на бумаге. Дело в отношении. В голове.

7.15.3.4 Индуцированный самозатрах

Поражает та скорость, с которой менеджер стремится обвесить себя делами. Нам важно быть занятыми, потому что так мы чувствуем себя защищенными. У нас появляется алиби. К нам теперь на козе не подъедешь.

Мы начинаем формулировать дела в виде процессных задач («Позвонить Ивану Ивановичу», «Ответить на вопросы», «Пропнуть должников») вместо того, чтобы итогом задачи стал какой-то артефакт («Получить акт», «Согласовать приложение», «Получить оплату»). Формально мы справляемся с задачей «Позвонить Ивану Ивановичу», даже если просто дозвонились, и он послал нас по матери вместо того, чтобы прислать акт. Мы полдня можем быть заняты тем, что слайдим встречи и задачи на следующий день. Еще и умудряемся уставать от этого.

7.15.3.5 Опаздываешь? Крути ручку против часовой!

Именно так. Важно поймать тот момент, когда не мы планируем дела, а мелочевка планирует нас. Опаздываешь — тормози, помните? Инстинкты будут сопротивляться. Искушение создать себе алиби заня-

того человека или переделать еще больше дел будет очень высоко. Но именно этот момент нужно отследить. Остановиться. И конкретно отфильтровать список дел.

Кому-то — переписать процессные постановки на результативные. Сократить количество дел, запланированных в течение дня. Кому-то поможет метод Айви Ли (шесть дел в день и минимум дерготни). Кому-то — день-два поработать с выключенным телефоном. Кому-то — просто не путать работу с разговорами о работе. Самое сложное — вовремя спросить себя: «А не херню ли я делаю?» И остановить мясорубку.

7.15.4 СТРЕСС И МЕХАНИКА ВЫГОРАНИЙ

За что мне, такому хорошему, такая хреновая жизнь?

Алексей Капранов

Но ведь можно же просто убрать все стрессовые факторы, и все? Ага, щаз! Обмечтались! Во-первых, не получится (вы даже интересные задачи не сможете всем обеспечить: интерес самая неустойчивая эмоция). Во-вторых, не сработает.

Вот вам схема. Ее придумал и прекрасно объяснял Алексей Капранов в своих книгах и курсе «33» (к сожалению, [§]). Итак.

Во-первых, человек не может без стресса. Откуда мы это знаем? В экспериментах с камерой сенсорной депривации, изолирующей человека от любых ощущений, человек испытывает состояние невесомости и полной свободы от беспокоящих воздействий извне. Никакого стресса. Сначала — эйфория. Однако потом начинаются глюки, страхи и кошмары. Выходит, если человек в сознании и кругом нет стресса, — он его себе придумает.

Об этом позаботилась природа. Для выживания было важно быстро отличить Опасное и увернуться в сторону Полезного. Но за последнее время все сильно поменялось. И если вы не чудите в баре для байкеров, не перебегаете МКАД и не тусуетесь в маргинальных средах — физическая смерть вам вряд ли грозит. Реальных опасностей стало меньше.

Вас скорее оденут в рубашку с длинными рукавами, обколят галоперидолом, изолируют и долго будут лечить, чем вы реально получите все те несовместимые с жизнью проблемы, для решения

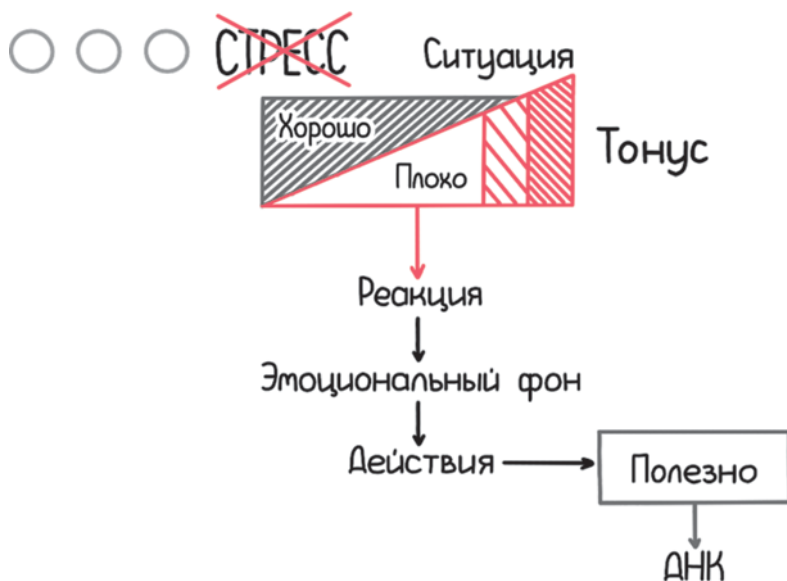
которых природа позаботилась. Инстинкт самосохранения остался без работы. И теперь он придумывает себе стрессы сам. Начальник плохо посмотрел — стресс. Не поздоровался — вообще кошмар, скоро уволят, наверное. Ага.

Как говорит Антон Петряков (автор канала Freshlife28), «любая неизвестность страшит наш инстинкт самосохранения — этого пьяного десантника, который оказался в XXI веке без работы».

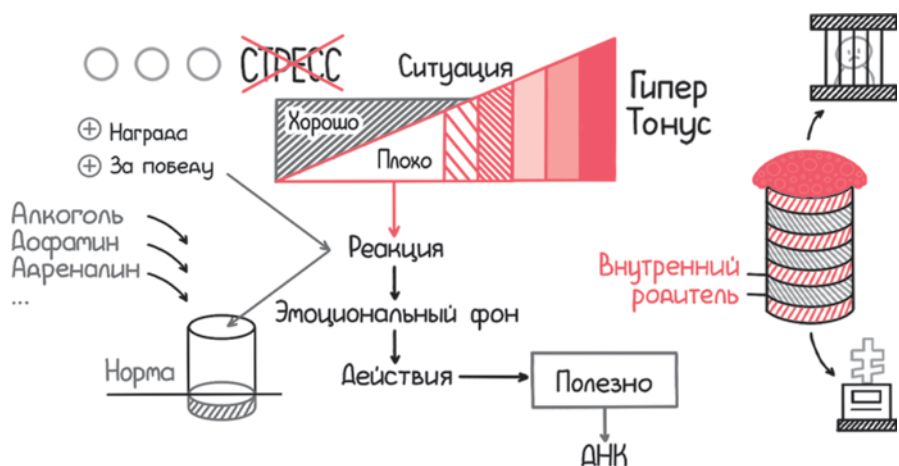
Итак. Стрессов с угрозами для жизни у нас негусто. Есть Ситуации. Наш мозг быстро просчитывает, Опасные они или Полезные.



Большинство Полезных ситуаций мозг игнорирует как несущественные. Ну хорошо, и хорошо. Нормально, и нормально. Но чутко реагирует на Опасное. Собственно, только Плохое и Опасное заставляет нас хоть что-то делать и как-то двигаться. Иначе бы никто с дивана не вставал.



Итак. Получили ситуацию. Классифицировали ее как Хорошо или Плохо. На Плохо — идет реакция. Вырабатывается кортизол, адреналин и т. д. Тело напрягается, мышцы приводятся в тонус. К бою готов!



Но тут вот такая штука: есть Нормальный уровень гормонов радости. А если мы закачиваем «Кайфец» извне — норма начинает расти.

Допустим, мы кайфуем от хорошо сделанной работы, позитивной обратной связи и интересных задач (пока не будем брать химические зависимости, но там та же механика).

Первый день — приходите на работу. Состояние — нормальное. Сделали задачу — похвалили себя. Всплеск радости. Но постепенно все приходит к норме.

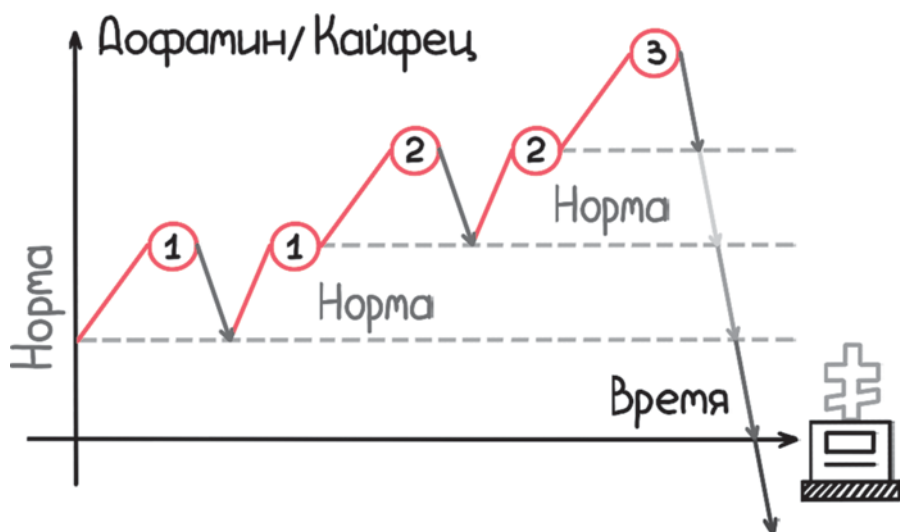
Теперь. Вы сделали задачку. Кайфанули сами, еще и начальник похвалил. Двойной кайф. Норма выросла.

Нужна задачка посерьезней. Не два проекта одновременно, а 15. Не 8 часов на работе, а 24 (или сколько вы там работаете?). И не только начальник похвалил — еще и клиенты, и премию дали. Вот теперь точно Кайфец. Норма выросла еще сильнее.

Но ты ли управляешь тем моментом, когда все пойдет через жопу?

Тут что-то задачки интересные кончились. Остались сложные, геморройные, непонятно с какого боку подходить. Начальник не хвалит. Может, забыл, а может, у самого настроение хреновое. Клиенты ножками топают. Программисты бухтят. Дома еще «выбрось елку» да «выбрось елку». Тьфу!

И тут вы упадете не на ту Норму, которая уже успела устаканиться. А ниже плintуса. Очень похоже на уже известный вам график из первой главы с кривой роста мастерства, в той точке, где мы все дружно попадаем в яму.



Кривая роста мастерства. Внимание на провалы и ямы.

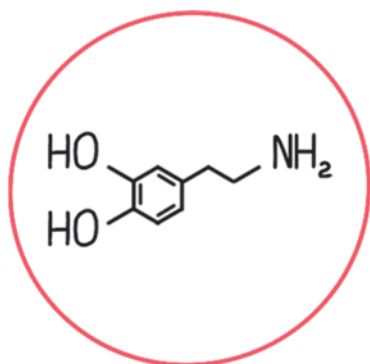
Добавление других «радостей» вроде алкоголя, секса, йоги с медитацией проблему завышенной нормы не решает. Будет новый виток, норма еще сильнее вырастет. А потом — опаньки. Кровь, ошметки.

7.15.5 БИОХИМИЯ ПРОЦЕССА

Кстати, о гормонах радости. Дофамин. Эндорфин. Серотонин. Окситоцин. Адреналин. Без них — никуда. Для продуктивной работы и обучения самым важным считается Дофамин.

Дофамин

Мотивация
Любознательство
Память
Скорость и точность
Концентрация
Экстраверсия
Творческое мышление
Формирование
привязанностей
Удовольствие

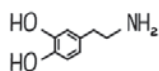


Стимуляция

- соцсети
- шоппинг
- телевизор
- сладкое
- кофе
- курение
- секс
- алкоголь
- наркотики¹
(кокаин/
амфетамин)

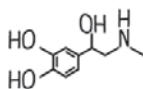
Отвечает за мотивацию, любопытство, память, скорость и точность, концентрацию, экстраверсию, творческое мышление, формирование привязанностей, удовольствие. Короче, делает из вас рок-звезду!

Как его подкрутить в моменте? Соцсети и игры (построены на дофаминовых циклах), шопинг, телевизор, сладкое, кофе, курение, секс, алкоголь. Выглядит как коварный план.



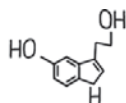
Дофамин

ОТДЫХ
НОВАЯ ЦЕЛЬ



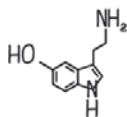
Адреналин

волнующие
активности



Эндорфин

силовые
упражнения
секс



Серотонин

свежий воздух
прогулки
сон
питание
физ. активность

Окситоцин

ОБНИТАШКИ
 ЗНАКОМЫЕ
 ПРИЯТНЫЕ ОБРАЗЫ
 АЕТИ
 СОБАКИ

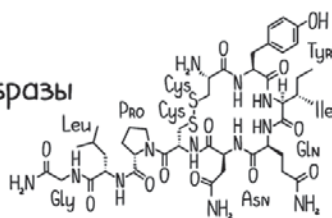


Схема восстановления гормонов радости

¹ Запрещенные наркотические вещества на территории РФ.

Что у нас там есть еще среди гормонов радости? Как их восстанавливать или добавлять?

Адреналин. Это для всяких экстремалов, вроде катания с гор или пилотажных самолетов.

Эндорфин. Качалка и секс.

Серотонин. Гуляем, дышим воздухом, спим, правильно кушаем, не забываем про упражнения.

Окситоцин. Больше для девочек. Обнимашки, дружьяшки, дети, собаки. Плохие новости для любителей кошек: нет, только собаки.

Мальчикам важнее тестостерон. Это не про радость, но с ним они горы могут свернуть. А без него превращаются в плаксивых ноющих старичков. Качалка, похудеть, убрать жир. Жир, грубо говоря, делает из мужика — бабу. Если уже возраст — подумать про ГЗТ. В биохимии все довольно сложно устроено, за подробностями в доступной форме рекомендую посмотреть книгу Александра Сашанова («Sashan») «Стероиды. Разумный подход».

Словом, ничего такого, чего вы раньше не знали. Ешь, спи, занимайся спортом, сексом и любимым делом. Но это все не решает проблему на 100%. Банка со стрессом-то никуда не делась. Вон она, красненькая и через край льется.

Трудоголики держатся дольше всех. Годами не выгорают. Потому что хоть что-то делают, разгребают свой стресс и видят результат. Остальные — хуже.

И чего делать-то?

В тяжелых случаях депрессии — топать к доктору. Хорошему доктору. А в целом — не надо добавлять «кайфеца». Нужно выгребать из банки со стрессом. Идти в проблему. Брать самую болящую. И решать, одну за другой. Год решать. Два решать. Пока не решишь — решать.

Только этот совет не поможет, если человек довел себя до кондиции «не могу с дивана встать, могу молча смотреть на стену и ненавидеть людей».

7.15.6 НАМ НАДО СЕРЬЕЗНО ПОГОВОРИТЬ... ИЛИ ВЫВОД ЛЮБИМОГО СОТРУДНИКА ИЗ ШТОПОРА

Наверное, каждый руководитель рано или поздно получал в чатик вот такие сообщения.



Ну понятно, что это будет за разговор. И что это не на пять минут. И если еще не очерствели — ножки становятся ватными, сердечко бьется, а в голове одна мысль: «Черт, еще и это! Не хочу, не хочу, не хочу».

К сожалению, в этой точке уже очень мало что можно сделать. Это значит, вы пропустили значимые сигналы, которые вам подавал человек.

В авиации есть такая фигура — Штопор. Это критический режим полета. Снижение по крутой спирали малого радиуса. Вращение относительно всех трех осей. Неуправляем. Обычно это результат грубых ошибок пилота, например, потери скорости. Преднамеренный штопор обязательно отрабатывается в спортивной авиации, с хорошим запасом высоты. Однако на гражданские самолеты на такое не рассчитаны. Штопору предшествует характерная тряска. Времени на решения и действия предельно мало.

Что мы с ним делаем:

- Распознать штопор/сваливание.
- Остановить вращение.
- Набрать скорость.
- Вывести самолет в нормальное положение.
- Контролировать параметры.
- Избегать ошибок.



Предштопорная тряска выгорающего

Вы пропустили эту «тряску». Например, резкие перепады настроения, падения или резкие всплески производительности, жалобы и нытье, которого раньше не было, красные глазенки, понурый вид, которого раньше не наблюдалось, закрытость, ругань в отношении проектов, клиентов, технологий, обсуждение зарплат и условий в других компаниях. В общем, человека трясет по эмоциям. Именно чтобы не пропустить (а точнее, не допустить) критический момент, важно регулярно проводить один на один и следить за тенденциями. Контролировать параметры.

Дальше. Ну допустим, сообщение «нам надо серьезно поговорить» — получено. Что имеет смысл попробовать:

- Принять решение: бороться за человека или нет. Восстановление подгоревшего человека — это сложно, и тут много совместной работы. Есть ли у вас на это ресурс? Готовы ли вы на это, ведь

там нет гарантий? А может быть, вы намеренно довели человека (я не про вас лично, я про других, плохих менеджеров)?

- ▶ Первый разговор. Внимательно выслушать. Задавать вопросы. Договориться о попытке совместных действий. Если человек идет навстречу и скорее просит помощи, чем уже все за себя решил — договоритесь два-три месяца пожить без резких движений.
- ▶ Найти настоящие причины. Выработать меры. Убедиться, что меры помогут.
- ▶ Планомерно, уверенно, но без спешки — внедрить меры.
- ▶ Системно контролировать эмоциональное состояние сотрудника. Избегать ошибок.

Может сработать составление Адской Таблицы. Выпишите все, чем занимается ваш сотрудник (до уровня операций) и все, что НЕ ТАК. Оцените, насколько его раздражает каждый конкретный пункт. Можно использовать стобалльную шкалу или привычную со школьной скамьи — пятибалльную: 5 — нравится; 4 — норм; 3 — такое себе; 2 — ненавижу; 1 — ад, тлен, страдания, депрессия. Может так оказаться, что проблема всего в одном проекте или клиенте и человек сам эту проблему решить не может, варится в собственных проблемах и безысходности. А у вас хватит опыта и полномочий разделаться с ней в два счета. Так бывает. Чаще, чем хотелось бы.

1	Активность	Оценка (1-5)	Исключения
2	Тест верстки	3	
3	Тест сборки	3	
4	Подготовка аналитики	4	
5	Подготовка ТЗ	3	
6	Изучить инфу по проекту	5	
7	Подготовить и обсудить вопросы с аккаунтом	3	
8	Транзит + брифование заказчика	4	

Адская Таблица

Если человек идет на контакт — значит, еще не все потеряно. Ищем меры, внедряем. Типовые проблемы и решения (вместо многоточий, уверен, вы сможете придумать еще с десятков вариантов, это уже не так сложно):

Высокая нагрузка Дефицит личного времени	Выгонять домой вовремя. Планировать на шесть часов в день (два — на форс-мажоры). Отправить в отгул принудительно. Делегировать проекты и полномочия. Перевести на удаленку или частичную удаленку. Частичная занятость. ... 40-часовая рабочая неделя — решает многие проблемы!
Высокая само-ответственность Синдром мясорубки	Разрешить/заставить намеренно зафакапить несколько задач и показать, что ничего страшного не будет. Проредить контакты и клиентов по принципу Парето (80/20). Проговаривать, что дома нужно играть/гулять/заниматься сексом/читать, но НЕ РАБОТАТЬ. Запретить на три недели сидеть в телефоне по вечерам. ...
Не нравится содержимое работы	Выявить адские места с помощью таблицы оценок. Делегировать, что можно, другим сотрудникам. Научиться подавать себя в аренду. Позиция «БУДУ» вместо «ДОЛЖЕН» и «ХОЧУ». Научиться выполнять операции профессионально (нам не нравится делать то, что мы делаем плохо). Сменить часть задач. Творчество дома. Сменить отдел или проект. ...

Работа с трудными людьми	Прокачивать переговорные навыки. Не работать с мудаками. На трудных этапах руководителю самому подключаться в коммуникацию «чисто послушать». Спорт. ...
Низкий уровень личной энергии	...

Типовые проблемы с типовыми же векторами решений

А вообще, профилактика, профилактика и еще раз профилактика.

7.15.7 ПРОФИЛАКТИКА

Я трудоголик, но уставший...
Прекрасное, из резюме

Помните Кристалл ~~курильщика~~ Пезешкиана? Если там одна работа-работа-работа (85%) — банальное убирание 15% работы и перераспределение освободившейся энергии на другие сферы улучшает их вдвое! Это очень серьезно и может предотвратить разговор из серии «нам надо серьезно поговорить».

Восстановление	Профилактика
Отдых Отдых. Длительный саббатикал).	Встречи, ретроспективы Мониторинг состояния. Опросы. Работа с мыслями и установками.
Детокс Изоляция от соцсетей.	Регулярный отдых Отслеживать, кто сколько отдыхает. Заставлять отдыхать. Ограничивать количество проектов/клиентов и токсичных контактов.

Восстановление	Профилактика
Новая цель Волнующие активности.	Рассказывать сотрудниками про выгорания Спорт, правильное питание, медитативные практики, Mindfulness.
Обратитесь к специалисту Философские таблетки — к доктору!	

Векторы профилактики и восстановления

Вредный совет

Напишите своему руководителю сообщение по типу того, что в начале главы. А во время разговора просто попросите отгул на пятницу. Посмотрите за реакцией. Отгул вы почти гарантированно получите (если, конечно, он вам не проломит голову).

Шучу-шучу. Не делайте так. Руководитель — тоже человек, а это довольно злой ход.

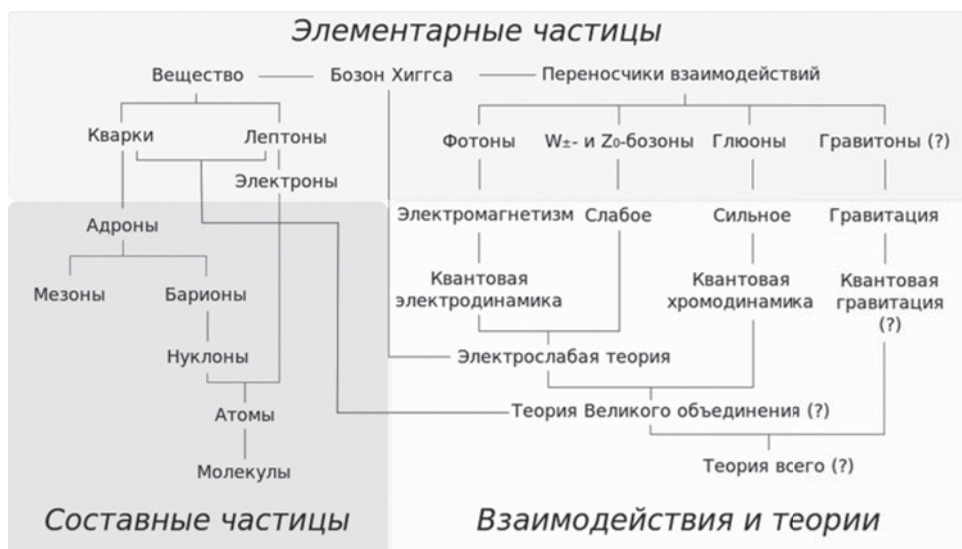
7.15.7.1 Спать!



Самые большие управленческие ошибки, провальные переговоры и глупости за последние годы у меня были с лютого недосыпа. Я люблю вставать рано, но это не повод насилловать себя и не высыпаться. Сон реально влияет на все: от ваших когнитивных способностей и самочувствия до позитивного отношения к миру, людям и себе. Посмотрите это видео и спите хорошо!

7.15.8 ЛИЧНАЯ ЭНЕРГИЯ И САМОПОМОЩЬ

Вокруг темы нереально много всякой ереси: сектантства, шарлатанства, мистики и высшей математики. Книг по самопомощи — пруд пруди. А физики, например, имеют очень богатое представление об энергии, как и что устроено.



Современные представления об энергии. Картинка из Википедии

На доступном уровне рекомендую к совместному просмотру фильм «Теория Всего. Бог в нейронах» (доступен по ссылке справа).



Видео
«Теория Всего.
Бог в нейронах»

Также рекомендую изучить работы Татьяны Черниговской по работе мозга, книги и видео Андрея Курпатова на эти же темы. А также курс Алексея Капранова — «33» (осторожно: мат, подколки, черный юмор, упражнения).

7.15.9 ВСЕ БУДЕТ ХОРОШО. ДАЖЕ ЕСЛИ НЕ БУДЕТ

Вы еще не подгорели? Надеюсь, нет. Тема выгораний провоцирует жалость к себе. Чем больше вы ее мусолите, тем больше впадаете в ступор. Поменьше надо о ней думать. Побольше — действовать. И все будет хорошо. Даже если не будет.

7.16 ГЕЙМИФИКАЦИЯ. ТЮНИНГ. КОРПОРАТИВНАЯ КУЛЬТУРА

На работе человек может пребывать в двух состояниях. Играть и страдать.

Геймификация, применение игровых механик и какие-то необязательные для работы активности являются тюнингом. Когда основной контур уже отлажен, есть ресурсы на подобные штуки — можно попробовать с чем-то поэкспериментировать в сторону игровых механик.

Однако не стоит думать, что, если у вас унылая каторжная галера, вы такие: «Хоба! С завтрашнего дня — геймификация!», и все у вас получится. Не получится. Сначала делаем то, что должны. Потом занимаемся тонкими настройками.

Ниже я приведу несколько приемов, которые в разное время применялись (и применяются) в нашей команде. Часть приемов отжила свое. А часть вошла в привычку, традиции и используется уже много лет. Помните, я говорил про ритуалы? Так вот. Это — оно.

Однако не стоит к этому относиться слишком серьезно. Так... большие дяди и тети дурачатся. Контролируемая глупость.

7.16.1. ДЯТЕЛ-БОАРД

«Принимай задачу, только если это ОФИГЕННО и ты ДОВОЛЕН» — записал Антон на желтый стикер и повесил на специальную доску. Стикер провисит на ней четыре недели — примерно столько, по мнению британских ученых, нужно для формирования устойчивой привычки (ха-ха). И каждый день коллеги напомнят тебе, что где-то ты смалодушничал, попробовал скруглить углы, принял работу, которая не нравилась даже тебе, и пошел сдавать ее заказчику. Фу так делать.

Эта доска со стикерами у нас завелась лет много назад. Мы называем ее «Дятел-board» или «Дятел», поскольку техника мозгоклюская: «Мэ-мэ-мэ-мэ-мэ, Мэ-мэ-мэ-мэ-мэ... Тьфу, блин!» Кто-то утром зачитывает вслух все стикеры. Ставит палочку для учета дней. Убирает отработанные стикеры.



Решает несколько задач за раз:

1) Лишение права на незнание

Допустим, мы меняем какой-то регламент. Решили разрабатывать мудборды перед прототипами, например. Вправе менеджер не знать про это изменение? Ну, вообще, да. Мы же не госкорпорация: бюрократию не жалуем. Изменение регламентов под поспись не доносим. Да и сами регламенты все никак не угонятся за реальностью. Попал стикер на доску — и все, сложно будет сказать, что ты про это ничего не слышал, когда человек восемь тебя видели.

2) Напоминашки

Менеджер — он как настоящий мужик — все время что-то ДОЛЖЕН. Даже если это хрупкая девушка (хотя сейчас некоторые хрупкие девушки по характеру больше мужики, чем некоторые нехрупкие с виду мужики, но это так, лирика). Дело, нюансов, техник, подходов, которые лучше бы знать и применять — ну очень много. Даже если они формализованы в регламентах и чек-листах, даже если вспоминаются на внутристудийных обучениях. То, что применяется не каждый день — все равно забывается.

3) Хитрожопые скруглятели углов

Ну, есть такое явление. Хочется тебе сделать побыстрее и попроще. Сэкономить силы, так сказать. Например, не проводить демонстрацию спринта клиенту, а сделать все изменения за две недели прямо на бо-

евом сервере. Желательно во время самой важной для клиента акции в году. Чик-чик — и в продакшн. Кровь, ошметки... Нет, понятно, что за такие выходки чаем с козинаками не напоят. Как минимум, поговорят о добре и зле. И дальше по справедливости — в зависимости от регулярности профанаций, степени тяжести и тенденции к исправлению. Может, например, именной стикер про тебя появится. Слава богу, явление довольно редкое.

4) Ретроспективные советы, которые непонятно куда зацепить

Есть менеджеры, есть команды. Регулярно заканчиваются спринты и проходят ретроспективы. И там бывают какие-то интересные нюансы, техники, советы или факапы, про которые хорошо бы всем рассказать.

Например, «делаешь редизайн проекта — проведи Code Review» — ну что это, правило что ли? Понадобится в следующий раз через полгода, может. Делать из этого отдельные регламенты — утонем в бумаге. А вот обсудить такую штуку, почитать по утрам — глядишь, кто-то и вспомнит, что перед тем как запланировать этап программирования, программисту нужно дать пощупать сайт, познакомиться с ним — сделать Code Review.

Решили завести «дятла» себе? Смотрите, как надо: рисуете «дятлу» рабочую область, разделив ее на четыре части. Одна часть — одна рабочая неделя. Всего 20 рабочих дней. Каждый день на тикете делается отметка о прочтении, как зарубка.

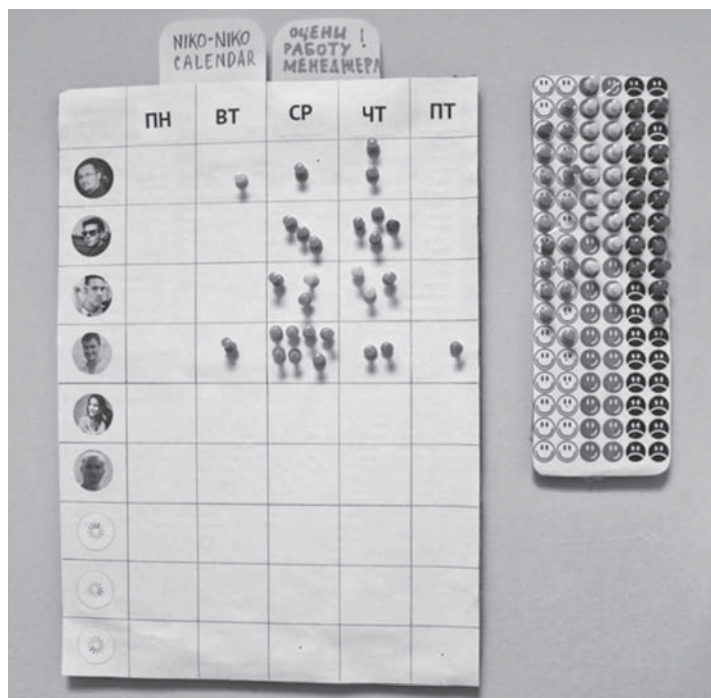
Тикет с новой привычкой крепится на неделю в первый раздел, затем на неделю — во второй, потом — в третий, а на последнюю неделю кочует в четвертый и затем остается в архиве на веки вечные.

Сделали? Отойдите на пару шагов, дабы взглянуть на результат работы со стороны. Отлично? Вы довольны? Берите в работу;)

7.16.2. НИКО-НИКО-КАЛЕНДАРЬ

У многих, думаю, бывало: разработчик отмалчивается на ретроспективах, а потом берет и внезапно увольняется. Потому что вы все его достали.

Игровое противоядие: в хорошо проходимом и в то же время укромном месте размещается календарь (японцы называют его *Niko-Niko*):



На календаре — фото руководителей проектов и пять рабочих дней. Справа — трехцветные кнопки. Любой желающий в любое время может остаться со своим руководителем «один на один» и оценить свои впечатления от его работы.

Плюсы: обратная связь очевидна.

Минусы: ну вот воткнули в тебя красную в начале недели. И она тебя бесит всю неделю. А сделать с этим ты ничего не можешь. Демотивация. Отклеивать фотографии в случае ротации кадров — тоже деморализует. Попробуйте: может, зайдет, может — нет.

7.16.3. ПРАВИЛА В ВИДЕ СТИКЕРОВ ИЗ СЕРИИ LOVE IS...

У нас была проблема с офисным холодильником. Время от времени там обнаруживался какой-нибудь продукт далеко не первой свежести. Найти владельца, естественно, было невозможно.

Ввели правило: в пятницу уборщица вправе выкинуть все из холодильника. Использована игровая механика «обратный отсчет».

Кроме того, сами правила можно сформулировать в игровой форме, а не в том страхоужасе, как это делают в госучреждениях.



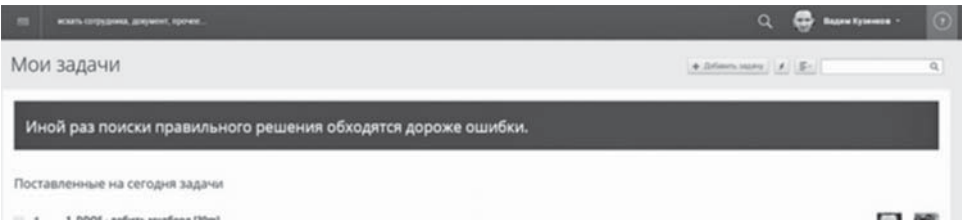
7.16.4. СОВЕТ ДНЯ

Эта тема была навеяна сайтом fucking-great-advice.ru.

Сначала советы мы писали на стикерах и разносили людям. Кто-то хранил целые пачки таких стикеров за несколько лет. Потом это переехало в корпоративный портал. А потом появился телеграм, где можно закреплять сообщения.



Тот самый сайт с советами



Пример совета дня в корпоративном портале

7.16.5. ТРЕНДВОТЧИНГ. САЙТЫ ДНЯ

Молодых аналитиков и дизайнеров мы просим делать ежедневные подборки новинок сайтостроения на заданные темы. Смотрим их на менеджерских планерках. Черпаем идеи, прокачиваем насмотренность.

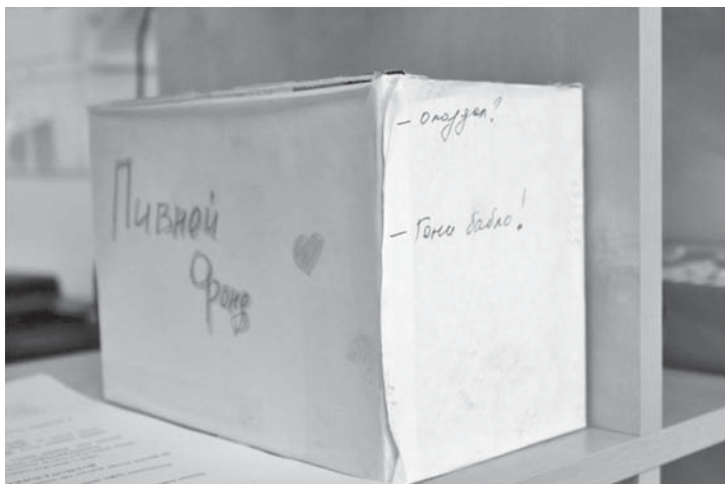
7.16.6. ЖЕЛТЫЕ ПИСЬМА

Эта идея по мотивам господина Чичваркина, экс-владельца «Евро-сети». Книжки, кстати, у него — отличные. Так вот. Он в свое время писал яркие, эпатажные письма своим сотрудникам. На актуальные темы компании. Стиль и подача были безумными, в чем-то даже злыми. Но точно не могли оставить равнодушными. Время от времени пишите своим ребятам письма на актуальные темы. Делайте их яркими, интересными к прочтению. А не «бубубу-бубубу, завтра переезд в новый офис, всем прийти в девять утра».

7.16.7. ПИВНОЙ ФОНД

Вот с этим прям очень осторожно! Еще Макиавелли завещал: «Чтобы избежать ненависти, государю необходимо воздерживаться от посягательств на имущество граждан и подданных и на их женщин».

Однако есть такая игровая механика: «избегание наказаний». Конкретно — об опозданиях.



Что сделали мы: договорились между собой, что каждый опоздавший менеджер расплачивается за свою недисциплинированность символическим взносом в копилку «пивного фонда». Номинал взноса — сто рублей. И механика «избегания» начала работать.

Важно, чтобы все добровольно на это пошли. Любая принудилровка здесь будет серьезно снижать боевой дух.

У нас это в итоге не прижилось. Да и нечего покушаться на имущество, пусть и символическое. Есть более правильные методы. Да и критичность опозданий в нашей сфере — весьма сомнительная. Только если ты реально заставляешь себя ждать. Гибкий график рулит!

7.16.8. СУББОТНИКИ

Да-да-да. Те самые, на которых дедушка Ленин таскал надувное бревно. Только в пятницу, вечером и раз в год (ибо нефиг воровать личное время сотрудников). Просто возьми и разгребь пару квадратных метров вокруг себя. Вы не представляете, сколько хлама можно накопить за год. И с каким удовольствием его можно выкинуть. Хотя девчонки куда-то забрали всю макулатуру — говорят, деревья надо беречь.

7.16.9. КОРПОРАТИВНАЯ БИБЛИОТЕКА

Это сейчас у многих есть. Но на прошлом субботнике я повыбрасывал несколько десятков киллограмм книженций (особенно про программирование: они устаревают слишком быстро, чтобы тратить на них деньги).

Вообще, штука прикольная. Выглядит интересно. На гостей впечатление производит. Провоцирует что-то почитать на обеде. Но должен быть кто-то, кто за ней следит. Иначе — пылица. И книжки не возвращают. Я задолбался «Пиши-сокращай» покупать — шестая за год уходит в никуда!

Мы использовали стикеры с уникальными номерами на книгах и доску с фотографиями. Взял книгу, отлепил стикер, приклеил на свой портрет. Прочитал, вернул стикер на место, поставил книгу в соответствующую ячейку. Сразу видно, кто и что читает в данный момент, а книги возвращаются. Минусы: должен кто-то следить за этим. Ну и деревья жалко. Цифра рулит, однозначно.



7.16.10. ЛИДЕРБОРДЫ И КАРМА

Лидерборды — особенно публичные писькомерки — категорически не советую. Смотри главу о KPI и «Пособие по командному самоубийству». Карма — ну такое. Может в итоге сработать как катализатор думанья не про дело, а про карму. Особенно, если начнут ей меряться.

7.16.11. ПЯТНИЧНЫЕ ДАЙДЖЕСТЫ

А вот это — прикольная штука. По пятницам руководители проектов скидывают в общий Телеграм-канал краткий дайджест за неделю по своим проектам. Ребята пишут это в веселой форме, приятно читать. Вошло в традицию. Помогает знать, чем занимаются другие команды. Кстати, если о ком-то долго нет новостей, это индикатор проблемки.

7.16.12. «СПАСИБЫ» В КОНЦЕ МЕСЯЦА

В ту же копилку, что и пятничные дайджесты.

Каждый месяц я прошу ребят написать мне в личку, кого бы они хотели отметить. Кто — молодец. Кто помог. Сделал что-то прикольное. Прорывное. Или просто, кого бы хотелось похвалить, с кем приятно работать. Дальше эта обратная связь летит в общий канал. Своеобразный опрос 360, но не 360.

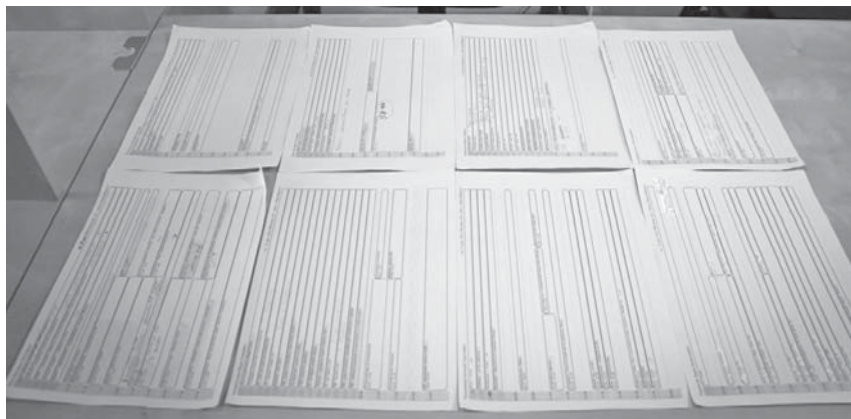
7.16.13. ЛОТЕРЕЯ

Игровая механика «случайное событие». Также в конце месяца. Можно рискнуть ста рублями, положив их в общий банк. Далее по какой-то механике определяется победитель (мы использовали как генератор случайных чисел, так и бутылочку и игру на выбывание — тут полный полет фантазии). Компания удваивает банк. Порой это сотка баксов на карман — ребятам нравится.



7.16.14. РАСПЕЧАТКИ РАБОЧИХ ПЛАНОВ НА ДЕНЬ И МАРКЕРЫ

Это, конечно, не про геймификацию. И сейчас к этому редко прибегаем: все в единой системе. Но на начальных этапах, когда человек только-только учится планировать свое время или очень сильно перегрузил себя задачами — хорошо срабатывает распечатка плана дня. С возможностью красить его по ходу работы. Вдобавок, издалека видно его прогресс.



7.16.15. ПЕСОЧНЫЕ ЧАСЫ, ВМЕСТО/ВМЕСТЕ С ТАЙМЕРОМ ПОМОДОРО



В ту же степь, что и распечатки. Таймер Помодоро придуман, чтобы сосредоточенно работать минут 20–25, затем сделать перерыв, минут на 5. Продуктивность с ним вырастает раза в два. Почему?

Чтобы выполнить задачи дня качественно и аккуратно, нужно, чтобы не дергали. Поэтому надо отключиться от внешнего мира, чата, команды. Сконцентрированно поработать в потоке. Это с одной стороны.

С другой, чтобы команда и сотрудники могли быстро решать возникающие вопросы, нужно быть на чате, телефоне и т. д. Быть готовым принять коммуникацию.

Одно из решений проблемы — квантовать время небольшими блоками (второе — правильное планирование). Сконцентрированно работать над задачей, затем проверять коммуникации, затем переходить к следующей. Мне удобны 20-минутные кванты, например.

Сложность была в том, чтобы не дергали посередине кванта. Компьютерные таймеры не видно со стороны. Таблички «Занят» и «Не беспокоить» работают плохо, поскольку нужно все время следить, в каком состоянии табличка. Индикатор типа «если у меня наушники на голове, то я занят» — часто игнорируется. Можно, конечно, сбежать в переговорку (благо, их хватает) или поработать из дома. Но что, если не получается? Кроме того, правильно, чтобы сотрудники сразу понимали, через какое время можно подойти.

Решение тривиально:



Экономика: гаджет стоит 10–15 евро, в зависимости от величины кванта. Рекомендую попробовать, если вы часто вынуждены переключаться между задачами.

7.16.16. ПРАЗДНУЙТЕ СОВМЕСТНЫЕ СОБЫТИЯ

Я думал, это есть у всех. Но люди на собеседованиях рассказывают, что нет. Говорят, даже чай вместе пить не дают. А в некоторых госкомпаниях нет мыла и туалетной бумаги. Короче, мракобесие.

Отмечайте совместные события. Дни рождения. Выпуск больших проектов. Рождения детей, свадьбы. Любые успехи и достижения. Не представляю, сколько пиццы было схомячено за все время.

7.16.17. ШАШЛЫКИ И ВЫЕЗДЫ НА ПРИРОДУ

Ну а как без этого?

7.16.18. ХАКАТОНЫ

Берем, придумываем и делаем проект. За день. Пока никто не мешает. Так мы сделали:

1. Хелпдеск, который отдали в open source и используем сейчас сами на своих продуктах.
2. Корпоративный парольник.
3. Planning Poker Online.
4. Пару мобильных приложений.

Тут можно попробовать разные технологии, с которыми вы раньше не сильно работали. Это драйвово. Однако надо понимать, что сделать полноценный продукт за это время не получится. Получится MVP, функциональный прототип и куча эмоций. А для полноценного, оттестированного продукта понадобятся еще сотни часов. Это скорее не про создание коммерческого продукта за один день. Это про фан, обкатку технологий и идей.

7.16.19. КОНКУРСЫ ПРОЕКТОВ

Придумываем задачу. Назначаем призовой фонд. Разрабатываем правила. Назначаем судейство. Набираем участников. Понеслась. Классная, на самом деле, штука. Позволяет приобщить людей к каким-то технологиям, которые вы бы хотели прокачать в своей компании.

7.16.20. ДИЗАЙН-БАТТЛЫ

Уже говорили о них в главе об управляемом креативе. Напомню концепцию: дается тема и час времени. Внезапно оказывается, что макет можно нарисовать за час. Далее — дизайнеры презентуют перед

всеми свою работу (прокачка навыков презентации). Голосуем (стикерами). Выбираем победителя. Плюсы: ускоряет дизайнеров и повышает их веру в себя. Минусы: большинство работ не имеют эстетической ценности, а побеждает, как правило, самая безумная.

7.16.21. ХОЛИВАРЫ

Это митапы от программистов для программистов. Внедряем новую технологию — устраиваем обсуждение, показываем плюсы и минусы. Вышла новая версия чего-нибудь — изучили, попробовали, рассказали всем. Упоролись с чем-то, но разобрался — тоже поделитесь этим.

7.16.22. REVIEW ПРОЕКТОВ

Также командная активность. Руководители проектов презентуют готовые проекты, которые они сделали со своими командами. Рассказывают историю: как и что было. Вовлекается вся команда. Можно сделать голосование за лучший проект. Такая активность позволяет держать все команды в теме, чем занимались другие. Можно совместить с шашлыками и выездом на природу. Раз или два в год будет достаточно.

7.16.23. ПОНЕДЕЛЬНИЧНЫЕ ОБУЧЕНИЯ

Это обязательная активность для руководителей проектов. Разбор кейсов, конкретных ситуаций, подготовка к переговорам. Разбор технологий, сложных случаев. Тренировки. Изменения в регламентах. Утро понедельника у нас в обязательном порядке уходит на эту активность.

7.16.24. ВЕЧЕРНИЕ, ПЯТНИЧНЫЕ ОБУЧАЛКИ

Периодически пробуем в разных форматах. От совместного просмотра обучающего видео, уроков по графике (для дизайнеров) до уроков рисования с профессиональным художником. Если так уж по-честному, вечер пятницы — не лучшее время для ударного труда (и деплоев). А вот расширить кругозор — самое оно.

7.16.25. УРОКИ АНГЛИЙСКОГО

Набираем группу, находим препода — пошли учиться. Проводим во внерабочее время, обычно — вечером. Нюансы: группа может не сработаться из-за разного уровня (кто-то только читать учиться, кто-то уже шпарит). Второй нюанс, и он фатальный для таких занятий: сначала набирается группа человек в 10–15. Через месяц остается пять. Через полгода остается парочка самых упорных, но и те ходят нерегулярно. Приходится прекращать занятия. Потом опять кто-то поднимает вопрос, набирает группу, и виток повторяется.

7.16.26. ЧТО ЕЩЕ?

Я однозначно что-то забыл: уж больно много разного мы пробовали, пробуем и будем пробовать. Что-то приживалось, что-то умирало, что-то воскресало вновь. У меня нет задачи сделать из компании игровую комнату. Каждый ритуал придумывался под конкретный запрос, для решения конкретной проблемы. А не просто чтобы было.

Далеко не на все вещи хватает энергии и ресурсов. Но я хочу, чтобы на работу было приятно ходить. Чтобы кругом не было тухляка. А люди кругом росли и развивались.

Итак. Мы с вами посмотрели, как, для чего и каким образом создавать и развивать команды. Ставить цели. Синхронизировать их. Давать обратную связь. Не выгорать, а наоборот — создавать корпоративную культуру. Кроме вас, руководителей проектов, командой вряд ли кто-то будет заниматься. Это ваша вотчина. Формируйте ее такой, чтобы вам самим нравилось в нее возвращаться.

Хвалите больше и чаще. Отмечайте любые достижения. Создавайте ритуалы.

7.17 ДОМАШНЕЕ ЗАДАНИЕ

- ▶ Пройдите тест PAEI на определение стиля менеджмента.
- ▶ В течение двух недель ведите дневник обратной связи.
- ▶ Проведите «один на один» с каждым членом своей команды. Проанализируйте.

- ▶ Проведите опрос 360.
- ▶ Подумайте, какие меры вы предпримете, если ваш разработчик не укладывается в оценки задач. Придумайте не менее пяти разных мер. Чем вы можете дополнить этот список из статьи по ссылке ниже?



ЛИТЕРАТУРА

- ▶ Сунь Цзы, «Искусство войны».
- ▶ Антон Макаренко, «Педагогическая поэма».
- ▶ Эрик Берн, «Игры, в которые играют люди» и «Люди, которые играют в игры».
- ▶ Ицхак Адизес, «Управление жизненным циклом корпораций», глава 12 о PAEI.
- ▶ Ицхак Адизес, «Идеальный руководитель».
- ▶ Михаил Литвак, «Командовать или подчиняться».
- ▶ Даниэль Канеман, «Думай медленно, решай быстро».
- ▶ Павел Сивожелезов, «Сложные переговоры с подчиненными».
- ▶ Владимир Тарасов, «Социальная технология в вопросах и ответах», «Технология жизни: книга для героев», «8 ступеней управленческого искусства».
- ▶ Алексей Капранов, «За что мне такому хорошему такая хреновая жизнь? Креативный антивирус для мозга» и «МЖ. Как научиться жить душа в душу».
- ▶ Алексей Капранов, Семинар личностного роста «33» (аудиокурс).
- ▶ Джефффри Лайкер, «Дао Toyota».
- ▶ Александр Сашанов (Sashan), «Стероиды. Разумный подход».
- ▶ Татьяна Черниговская, «Чеширская улыбка кота Шредингера. Язык и сознание» и другие книги о работе мозга.
- ▶ Андрей Курпатов, «Красная таблетка», «Красная таблетка-2», «Троица» и «Чертоги разума», «Счастлив по собственному желанию».
- ▶ Джен Синсеро, «НИ СЫ: будь уверен в своих силах и не позволяй сомнениям мешать тебе двигаться вперед».

По OKR хороших книг мне не попадалось. Все годные материалы по этой теме собраны под этой ссылкой.



Подборка материалов по OKR

ПОЯСНЕНИЯ

Стек технологий — комбинация языков программирования, фреймворков и программного обеспечения, используемых при разработке продукта.

Писк модема на 33 600 — модем, который мог передавать 33 600 бит в секунду, имел характерный звук при работе.

Ламер — на компьютерном сленге это человек, который плохо умеет обращаться с компьютером (неспособен или принципиально не желает освоить этот навык).

Компилировать (в программировании) — транслировать программу, составленную на исходном языке высокого уровня, в эквивалентную программу на низкоуровневом языке, близком машинному коду.

Хабр — русскоязычный сайт тематических коллективных блогов с элементами новостного сайта, созданный для публикации новостей, аналитических статей, мыслей, связанных с информационными технологиями, бизнесом и интернетом.

Гитхаб (Github) — крупнейший веб-сервис для хостинга IT-проектов и их совместной разработки, основанный на системе контроля версий Git.

Root Cause — поиск основной причины наступления события. Принцип простой: постоянно задавать вопросы «почему?», «почему это произошло?», «по какой причине?» К каждому ответу снова задавать вопросы, а на основе всех ответов построить дерево решений.

ГЗТ — гормонозаместительная терапия.

Саббатикал — долгосрочный неоплачиваемый отпуск.

Часть 8



ВРЕМЯ И ЭНЕРГИЯ РУКОВОДИТЕЛЯ. ЭТЮДЫ

Мы уже затрагивали тему важности личной энергии руководителя в главе 7. Книжек и материалов про это очень много. Не вижу смысла повторять подходы GTD, Дэвида Аллена, Джедайские техники Дорффеева, планирование Хаоса Фридмана. Но настоятельно рекомендую освоить эти техники, выбрать себе инструмент (подход, лайфхаки и софт) по вкусу и приучить себя к дисциплине. Здесь я расскажу свой подход.

Я использую недельные циклы планирования. Утро воскресенья, как правило, уходит на планирование недели.

Порядок действий

1. Список Хвостов:

- a. Ретроспектива. Пройти по списку сделанных задач. Активных проектов. Людей, с которыми я взаимодействую. Выписать плюсы и минусы: что было удачно, а что — нет. По минусам — придумать список мер. Пока просто Список Хвостов, ничего не планируя.
- b. Просмотреть, что на прошлой неделе я не смог закончить. Добавить это к списку мер. Таким образом я собираю все хвосты по прошлой неделе.
- c. Пройтись по источникам входящих. Почта, чаты и т. д. Если что-то требует обработки — также добавить в список хвостов.
- d. Прочитать список Спусковых Крючков. Это mindmap-карта с перечнем всех основных сфер, объектов, людей, на которые я трачу внимание. Все, что вспомнил, заносу в Список Хвостов.
- e. Есть в интернете хороший список «21 вопрос, который стоит

задавать себе каждую неделю». У меня — свой, но этот тоже хорош. Я чередую его и Спусковые Крючки (на оба сразу мне жалко времени).

- f. Все, Список Хвостов готов. Пока ничего не планирую. Мне хватает от 25 минут до часа в зависимости, насколько сильно я застрял на Ретроспективе. Сосредоточиться помогает таймер Pomodoro. Переходим к Главному.
2. Главное. Главное в идеале должно быть одно. Из Годового плана (списка активных проектов) я записываю и планирую, расставляя уже на конкретный день и время — конкретные мероприятия. Не более семи, больше все равно не успею.
3. Ненавижу торопиться.
4. Вставать в 4:44 проще, чем лечь спать в 22:00.
5. Утренние задачи. До того, как начнется основная работа, я успеваю сделать какое-то одно большое дело из списка Главного. Либо просто им позаниматься. Большая часть этой книги была написана именно в утреннее время. Этот режим точно подойдет не всем. Но мне нравится именно ранний подъем. Ощущение сделанного большого дела с утра дает энергию на весь остальной день.
6. Рутинa. Примерно 50% времени у меня рутинизировано. Не требует какого-то специального планирования. Это все понедельники (уходят на обучение руководителей проектов и собеседования), пятницы (уходят на недельные планерки, демки и ретроспективы команды) и утро каждого дня, с 8 примерно до 12–14-часов. Это время уходит на мелкие задачи, встречи, оперативку, спорт. Все то, что **нужно** делать. На это же время я расставляю из Списка Хвостов какие-то мероприятия, которые реально нужно сделать. Одно-два в день. Большую часть стараюсь делегировать, планируя себе только контрольные точки. Остальное — выбрасываю. Либо совсем. Либо раскидываю по проектам «когда-нибудь, может быть» в своем планировщике. Тут самое главное — не переживать, что куча времени уходит на рутину. После рутины нужен небольшой перерыв, чтобы включиться в спокойный рабочий ритм.
7. Остается пара-тройка часов, чтобы заниматься важными задачами. Или поучиться. Или придет какой-то пушной зверек по работе, скажет «здрасьте» и заставит заниматься именно им. Или плюнуть на все, и поехать летать на самолете. Как повезет.

8. Время после 17. Обычно там уже нет мозга. Я стараюсь расставлять туда всякие не особо важные звонки и встречи. Там, где можно сильно не думать.
9. В конце дня корректирую план на следующий день, если в этом есть необходимость.
10. Вечер. Гулять, отдыхать, кататься, читать книжки. Тут главное — не перегрузить себя информацией. Иначе мыслемешалка в голове не даст толком выспаться. Отбой в районе 22:00.
11. В примерно таком же ритме получается жить на два города.
12. Поездки, командировки, конференции, особенно если это США, ломают график. Ничего не поделаешь. Стараюсь, по возможности, сохранить ту же канву: раннее время подъема и ранний отбой.

Итого. На планирование недели уходит от 25 минут до часа. На ежедневный пересмотр планов — 5–10 минут. Утро, со свежим мозгом, уходит на главное. Рутинa сейчас занимает примерно половину дня. Остальное дневное время — какие-то крупные задачи. Годовые планы пересматриваю и корректирую два раза в год.

Похожая схема используется у руководителей проектов (естественно без раннего-раннего подъема):

1. Недельное планирование — по пятницам. Составляется недельный план мероприятий по каждому активному проекту. Подбиваются хвосты.
2. Понедельник-утро — обучалки.
3. Утром каждого дня — актуализация своего плана на день. Стендапы с разработчиками, дизайнерами. Крупные активности вроде брейнштормов, планирования спринтов.
4. День. Обычно уходит на активности с клиентами.
5. Вечер — актуализация планов разработчиков.

По сути, единственное чем вы управляете — это ваше внимание. Вместо поиска волшебной «мотивации» есть более мощный инструмент — дисциплина. Есть план — работаю по плану. Не жду, пока «захочется» чего-то сделать. Мотивация — либо придет сама, либо не придет — как повезет. Это на сладенькое.

Итак. Дисциплина важнее мотивации. А дальше будут несколько этюдов, напрямую имеющих отношение к управлению временем и энергией руководителя. Этюды

ЭТЮД #1. МИКРОПЕРЕКЛЮЧЕНИЯ КОНТЕКСТОВ

То, что меня все время бесило — это когда во время сконцентрированной работы над каким-то запланированным большим делом приходится чего-то ждать. Например, пару секунд, пока запустится программа. Пять-десять секунд, пока скачается файл. Или 30 секунд, пока ответит собеседник. Или какой-нибудь архив распакуется. Или смотреть на очередной «красивый прелоадер». Да мать его!

Эти небольшие паузы не так велики, чтобы успеть сделать что-то толковое. Но вполне достаточны, чтобы сделать себе гадость. Например, обнаружить себя через полчаса за чтением очередной пачки новостей. Или листанием социалочек. Или открыванием очередного таба с очень интересной ссылкой или очередным параллельным скайп-чатиком. Какая уж тут к черту концентрация. Какой нафиг «поток»?

В общем виде задача получилась такая: чем бы полезным занять мозг во время вынужденных микроожиданий, чтобы не было переключений контекста с текущей задачи.

Я много экспериментировал. И делать надо так: как только возникает такое микроожидание — берем и начинаем обдумывать ваши следующие шаги по текущему делу. Вот прямо с момента, как это ожидание закончится. Во всех подробностях и мелочах. Вплоть до тактильных ощущений и жестов. Вплоть до кнопок, которые будем нажимать. Можете даже глаза закрыть. (Хватит на прогресс-бары смотреть, верно говорю?!)

Бонусы такие: концентрация и поток остаются, прокрастинация, раздражительность и усталость снижается, дело делается ловчее и быстрее. Enjoy.

ЭТЮД #2. ПУШИНКИ

Это был девятый класс. Мы с ребятами гоняли балду, программировали и играли на гитарах.

И тут родители позвали меня с их друзьями съездить на выходных на озеро. Им — выпить-погудеть в палатках. Ну и чтобы был кто-то с гитарой.

Я, естественно, полез переплыть озеро от берега до берега (и сейчас так делаю: если есть хоть какой-то адекватный водоем — надо от берега до берега).

Озеро большое. Плыву. Нормально. До того берега доплыл. Надо возвращаться.

Гребу. Понимаю, что сил обратно может и не хватить.

На трети обратной дороги понимаю, что сил точно не хватит. Даже на спине. Потону.

Смеркалось.

И тут я вижу, что на воде лежат пушинки. Как раз тополя дали пух. Какая-то в паре метров.

Думаю: ну до нее то точно догребу. Потом уж хрен с ним, и потону, чо.

Доплыл. Смотрю — вторая, метрах в пяти. Ну поди до нее-то смогу, а там уж и неважно. Дальше — третья. Еще. Дальше легче.

Минут через сорок, выбравшись на берег, замерзший, с дрожащими от усталости руками, получив обратную связь от родителей («ты чего ж, блин, так долго, мы же волнуемся...»), никому ничего не рассказав, я сидел и глупо улыбался. Получилось!

В тот вечер я никому не играл на гитаре. Но, единственный раз в жизни радовался, что есть на свете тополиный пух и можно напрычься и доплыть от пушинки к пушинке.

ЭТЮД #3. ПЕРИОД ПОЛУРАСПАДА ИНФОРМАЦИИ

Как-то в очередной раз пересматривал список проектов и задач. Чистил.

Натыкаюсь на список: «Задачи/Неразобранное 2013». Большой, зараза, тыщи полторы строк. А там ссылки всякие «☒прочитать умную статью такую-то» да «☒посмотреть умное видео такое-то». Ладно, думаю, протыкаю. Может, что ценное.

Итог: 50% ссылок — дохлые («Это видео было удалено, ай-ай», — пишет видеохостинг, и все в таком духе).

Я, конечно, упорный: могу упороться, поискать через веб-архивы. Но не буду.

Факт, что люди большую часть времени занимаются digital-херней и вся их «важная работа» будет удалена из этого мира года через три-четыре максимум, можно считать доказанным.

Исключение: социальные сети (тут вообще как в мусорное ведро писать).

А незадеплоенный код протухает со скоростью роста функции Аккермана (самая быстрорастущая функция).

А, да. Если нашли что-то достойное в сети — **делайте локальные копии и бекапы**. И не жалеите удалять старые задачи.

ЭТЮД #4. ГРЫЖА

Вот представьте себе, что у вас в руках баскетбольный мяч.

И на нем — грыжа. И вы ее так пальцем внутрь — ХОБА!

И держите...

Все, типа грыжи нет. И тут она такая: РАААЗ, и с другой стороны вылезла. А вы ее вторым пальцем. А она опять, в третьем месте... И так, пока у вас пальцев не хватит.

Примеры:

Ваши планы дня. Вы запикиваете в них целую кучу. Давление возрастает. Бац — где-то вылезает ТАКОЕ, на что вы просто не успеваете реагировать. Собираетесь духом, затыкаете эту грыжу, добавляя рабочее время. Вылезет что-то еще, в другом месте. Затыкаете своими выходными. Отпусками. Ночными работами. Но это явно не поможет. Грыжа будет вылазить раз за разом, в самых неожиданных местах. А пальцев вам точно не хватит.

Клиенты: когда клиент хочет чего-то больше, чем в договоре, вы ему не даете, а он начинает делать какие-нибудь очень странные штуки.

Сотрудники: когда они хотят расти в каком-то векторе, который вам не актуален (или который вы не можете обеспечить).

Сдувать давление? Менять мячик? Давить сильнее? Ваши варианты?

ЭТЮД #5. ПРИЧИНЫ ИЛИ СЛЕДСТВИЯ?

«Сначала мы поработаем с последствиями. А потом займемся причинами» — услышал я отрывок разговора.

«Эй! Ты что несешь!» — чуть было не вырвалось у меня в адрес едва знакомого человека. Это же все в корне неправильно! Давно всем известно, что надо не бегать и последствия устранять! Вспотеешь. Надо причину искать и ей заниматься!

Такой стереотип мышления очень популярен. Борись с причиной проблемы, а не со следствием — все талдычат.

Однако что происходит по факту?

1. Допустим, потек радиатор. Мы СНАЧАЛА подставляем тазик, а ПОТОМ звоним в ЖЭК, ждем угрюмых мужиков в ватниках.
2. Или, допустим, заклинило тебе спину после тренировки. Перекосежило так, что ходить не можешь. Так тебе невролог СНАЧАЛА обезболивающего пропишет, на капельницу пошлет. А уж как сможешь не орать на каждом шаге — займется причинами.
3. Или пожар случился. Его СНАЧАЛА тушат. А ПОТОМ уже ищут, кого бы вздрючить.

И, похоже, что все так.

Понятно, что стереотип «сначала — причина!» полезен для тех, кто решил годами менять тазики, жрать обезболивающее и все в таком духе.

Однако, даже если мы такие «правильные» и у всего ищем причины — глубоко ли нам это удастся?

Тот же радиатор. Тойотовский Lean говорит нам, что надо задавать вопрос «почему» минимум пять раз. Ну, допустим, мы упорные, что нам пять! Давай хотя бы десять!

Почему потек? А потому, что образовался свищ. А почему? Истончился чугун. А почему? Началась коррозия. А почему? Из-за образования воздушных карманов. А почему? Радиатор перекошило на несколько миллиметров. А почему? Радиатор просел под собственной массой. А почему? Действует сила тяжести. А почему? Гравитация! А почему? А вот тут даже физики отвечают как-то невнятно и крайне неуверенно. И, потом, много вы с гравитацией наберетесь?

В общем, выводы пока такие:

1. Ни с какими «причинами» никто не борется. Только со следствиями. Вопрос только — на каком уровне. Дай бог, хотя бы на пятом.
2. «Сначала — причины», если не врать себе, — просто популярный стереотип.

И да. Во всем виновата гравитация.

ЭТЮД #6. ЛУЧШИЙ МЕНЕДЖЕР СВОБОДНОГО ОСТРОВА

Куба. Кайо-Коко. Отель Melia. Только-только начали пускать туристов после полуторагодового перерыва. На входе группы встречает лично директор, лично раздает велком-дринки.

Как потом оказалось, это не единственная его причуда. И днем, и ночью этого парня видели лично контролирующего и работу бара, и кухни, и бассейна, и стойки ресепшна. Даже артисты пели громче и веселее свое «Гуантанамера» в его присутствии. Парень засветился везде. И на раздаче лобстеров, и лично интересующимся «все ли у вас okay» у посетителей новогодних вечеринок.

«Ха! Этому парню явно нечем заняться» — саркастически хмыкнул я. А как же делегирование, доверие к сотрудникам, свобода действий для раскрытия своего потенциала в конце концов?! Бирюзовые компании, или как их там?

Кстати, про лобстеров. Кто не в курсе, на Кубе нет еды. Совсем. В Гаване ты можешь зайти в магазин и купить, например, ни-че-го. Сахар, рис, фасоль и 15 яиц в месяц местные жители получают по талонам. Со средней зарплатой в 17–20 долларов много и не купишь. Хотя встречаются длинные очереди за пивом. Кафе, где любил пить мохито Хемингуэй (да и вообще, любые кафе), закрыты на клюшку из-за Ковида. Когда-то там было красиво, но сейчас почти везде в городе — разруха. Менталитет местных — отдельная тема.

И тут этот парень, неуловимый. Со своим тотальным контролем, то там, то сям. Лобстерами. И, похоже, самым высоким рейтингом из отелей в округе (ладно-ладно, рядом есть еще Pullman, где можно с детьми, примерно с таким же рейтингом, но по отзывам ребят-соседей, там не так нажористо).

Ну и кто я такой, чтобы саркастически хмыкать по поводу того, как надо управлять кубинским отелем на Кубе с персоналом из местных жителей??? Их, кстати, привозят из-за дамбы, за 17 километров; рядом никто не живет. Понятно, что за ним стоит логистика и большие дяди с материка (это испанские сетевики). Но ты пойдی попробуй, перезапусти всю эту машинку после закрытия на Ковид и обеспечь хорошие отзывы от русских (основной контингент, наравне с канадцами). Вспотеешь.

ПО РЕЗУЛЬТАТУ: похоже, парень на своем месте, гармоничен и знает, что делает!

ЭТЮД #8. УВЕДОМЛЕНИЯ, ЧАТЫ, ПОЧТЫ...

«Рванок» — неожиданный телефонный звонок, порвавший выполнение текущей задачи.

* * *

Если отключить вообще все уведомления, пуши и прочую мерзость — через неделю начнут бесить сноски в книгах.

* * *

Не стоит проверять почту сразу после того, как спланировал неделю. Дай своим планам хоть до завтра побыть актуальными.

* * *

От любителей писать в чат «Привет» и пропадать на пару дней:

— Владимир, здравствуйте. Могу я Вам задать вопрос?

— Вы только что один задали. Вам нужен еще один вопрос?

* * *

Сегодня мы возьмем интервью у человека, который все успевает.

— Скажите, как вы все успеваете?

— Идите нафиг, мне некогда.

* * *

Чаты — зло!

— Я в Нельзяграме писал. Чего не отвечаешь?

— Ты б еще в Тиндер написал.

ЭТЮД #9. ПЕРФЕКЦИОНИЗМ, СРОКИ, ПЛАНЫ

Перфекционист по срокам? Не встречал.

* * *

Он был жуткий перфекционист: просил, чтобы за ним перепроверили сразу два тестировщика!

* * *

Сроки сами себя не продолбают!

* * *

План работе не мешает. (Растаманский тайм-менеджмент.)

* * *

~~Гори в аду!~~

Пиши ТЗ.

ЭТЮД #10. ТЕНДЕРНОЕ

«Вы уверены, что адекватно оценили уровень нашей бюрократии, когда называли нам цену...?»

* * *

«Мы будем вам помогать» = «Мы скушаем вам мозг, почти задаром».

* * *

— Некогда разбираться. Делай херню!
(будни маркетолога)

ЭТЮД #11. СУПЕР-БАЗОВЫЙ НАВЫК

Если в одном письме (или сообщении в Скайп) задается несколько вопросов — приучите себя нумеровать. Иначе вам так и будут отвечать только на самый удобный вопрос, игнорируя под эту дудку все остальные.

ЭТЮД #12. СКОВОРОДКА

А вот есть ли в русском языке какое-то специальное (отдельное) название человека, который... Вот допустим, стоит целая сковородка еды. Он придет, все сожрет, но оставит ма-а-а-ленькую ложечку. И ходит с таким видом «я вам там поесть оставил», хотя всем ясно, что это чтобы сковороду не мыть. И вот во всем так...

ЭТЮД #13 ЗОЛОТЫЕ СЛОВА РУКОВОДИТЕЛЯ

«Твоя ошибка <вот в этом>.

Иди и переделай <вот так-то>.

Ага, вот теперь хорошо».

Нет, блин, мы так не хотим. Нам надо, чтобы сам догадался. Осознал. Проникся всей глубиной глубин.

Вопросы наводящие ему задаем. Намеки намекаем. Иронию иронизируем. Сократы недоделанные, блин.

А чтобы получить конкретику, нас надо вывести из себя. Злость срывает крышку с чайника. С паром и свистом вырывается и четкая

критика, и конкретные распоряжения. Но это ж ой как постараться надо.

Порой так мы корчим из себя Очень Важных. Чешем ЧСВ об подчиненного.

Порой уклоняемся от ответственности. Банально — сами не знаем, что делать. Нет ни ответа, ни решения, ни регламента, но хочется, чтобы кто-то дотумкал сам и сделал ровно так, как бы поступили мы сами. Но только еще лучше и дешевле.

Порой просто компостируем мозг: вот, мол, какие у нас подчиненные недогадливые. Это прямо плохо-плохо.

Порой думаем, что если мы будем говорить четко и конкретно, то сотрудники сядут на шею и полностью откажутся думать. Порой пытаемся научить человека действовать правильно, тыкая носом в лоток. Это уже лучше, но тоже так себе.

Идеально уметь и то, и то. И вопросами подталкивать. И конкретику формулировать. Четко и без мозготраха. Редкое сочетание.

ЭТЮД #14 ДИНАМИЧНО РАЗВИВАЮЩЕЕСЯ DIGITAL-АГЕНТСТВО

Однажды молодое креативное диджитал-агентство начало динамично развиваться.

Программисты начали писать код без багов. Нагрузка распределялась на всех равномерно: не было ни простоев, ни перегрузок. Никто не болел, не выгорал, не попадал в армию. Клиенты все прислали вовремя (а акты и контент — даже раньше времени). Креатив заходил с первого раза, даже без брейншторма. Дизайн сдавался без правок. Сервера не падали. Интеграции не отваливались. Тендеры сами выигрывались.

Такое динамичное развитие всего за год позволило выйти из оперативки сначала собственникам, затем — менеджменту, потом — всем остальным, и вернуться обратно, на фриланс.

ЭТЮД #15 НЕ ПРОДОЛБАТЬ МОМЕНТ!

Никогда бы не подумал, что можно испытывать радость от ответственности. Мало того, от ответственности за ВСЕ.

В момент доклада спикера починить его презентацию (хотя на

это есть чувак за пультом, но он не справился). Дать бутылку воды задыхающемуся от кашля докладчику... В момент разгона, на взлете самолета отстегнуться, в прыжке закрыть внезапно открывшийся через проход отсек под ручную кладь (хотя на это есть стюарды, а я ЗНАЮ, сколько перегородок я пробью башкой в случае экстренного торможения; но стюарды уже не успеют). Но летящие чемоданы на голову — совсем плохо.

Боже, дай мне мудрости в каждый момент понять, что для меня такое «нести ответственность за ВСЕ», сил и намерения действовать. И не продолбать момент!

ЭТЮД #16 СОЦИАЛОЧКИ

Полезны ли личные аккаунты политикам и управленцам в работе?

Все не так однозначно. Здесь есть выбор. И здесь есть сложности.

Выбор: вести аккаунт самостоятельно или поручить его пресс-службе.

Вариант с пресс-службой — более соблазнительный и более системный. Меньше времени, меньше личных усилий, не нужно читать и разбираться с негативными комментариями и комментаторами (троллями и нытиками).

Однако, во-первых, это уже не личный аккаунт, а ширма — фальшивая попытка сыграть в открытость. Увы, это за версту чуют читатели.

А во-вторых, за год происходят десятки громких ляпов, где пресс-служба опубликовала то, что как бы противоречит «политике компании» или даже «политике государства», или как-то жестко отработала с обратной связью, подмочив репутацию. Часто сотрудник пресс-службы становится козлом отпущения.

Вариант самостоятельного ведения аккаунта крупным политиком или бизнесменом тоже не без сложностей. Обязательно найдутся чем-то недовольные граждане, нытики или просто тролли. На которых нужно будет как-то реагировать. В публичном поле. И мало того, что это трудозатратно, портит настроение, отвлекает от важного. Руководители или политики, привыкшие к власти, находящиеся в режиме цейтнота, вряд ли будут сюсюкаться с нытиками. Могут ответить резко. И получить еще больше обиженных. А это вредно для имиджа и для дела.

Итого. Соцсети — это просто еще один канал, где есть люди, которым можно что-то рассказать. Мало чем принципиально отличается от

журнала, телевизора, блога. Ключевое отличие — работа с обратной связью читателей. Ну и конечно, нужно успевать готовить интересный и полезный контент. А это порой не самая простая задача.

ЭТЮД #17 ХАОС-ПОРЯДОК-ХАОС. СИНУСОИДА

Мне нравится, когда создается хаос. Запускаете вдолгую много-много процессов. Оставляете себе ниточки для их контроля. И смотрите, как все складывается, как бы само собой.

В какой-то момент кажется, что хаос побеждает. Событий одновременно происходит больше, чем ты можешь не то чтобы на них реагировать, а даже осознать. Пропадают нужные люди, мандят и теряются подрядчики, профукались сами (перед собой и не только), вылазят доп-кости, с кем-то ругаетесь навсегда, что-то ломается, валится из рук, портится, рушится. Почтовый ящик разрывается, телефон не умолкает, в телеге 100 500 сообщений.

Сингулярность ближе, чем вы думаете.

А потом вдруг внезапно пазл начинает собираться. Как бы сам по себе. Там — заканчивается ремонт, там — стройка. Вовремя привозят мебель, которую делали на заказ за 3 000 километров. Разрисовывают стену, обновляют договоры, запускают сайты, приходят нужны новые люди, знакомства и связи, 17 тонн металла, топливо, запчасти, документы, деньги, книги, стройматериалы...

Потом все затихает. Обычно — к началу августа. Но это только кажется.

Я не знаю, как это работает, но это всегда волна. Синусоида хаос-порядок-хаос.

ЭТЮД #18 СКОРОСТЬ ПРИТИРКИ. ПЕРЕВОДИ НЕПЕРЕВОДИ

У кого же это из юмористов-классиков было?

Как ведет себя человек, который немного знает язык, но разговаривает через переводчика:

— Переводи.

- Зачем ты это переводишь, я это понял...
- Ну че ты замолчал? Почему не переводишь? Это я не понял!
- Да это я понял, молчи.
- Что молчишь? Ничего ж не понятно!

Порой именно так менеджеры с программистами обсуждают постановки задач.

П: Да тут все понятно, зачем столько писанины?!

П: Да тут вообще непонятно, как это должно работать. Опять требования — говно!

П: Где было написано про это? Здесь? Ну так здесь столько букв, фиг все запомнишь.

М: Это же очевидно, что нужно было везде сделать, а не только здесь! Посмотри вокруг-то!

М: Откуда я должен знать, что у тебя с ЭТИМ проблемы могут быть.

и т. д.

На третьем спринте, как правило, все друг к другу привыкают. Уровень безобразия выравнивается. И дальше спокойно общаются на одном уровне детализации и технологического жаргонизма. Специально делать ничего не надо.

ЭТЮД #19 ТИПОВЫЕ ЗАДАЧИ

Люди стебуются над типовыми задачами для собеседований, потому что не умеют на них отвечать. Предложите кому-нибудь продать ручку. Сначала он нервно хихикнет. Потом скажет: «Боже, никогда не думал, что еще где-то это осталось. Это же было на Пикабу». А потом зафейлит продажу.

То же самое почти с любой классикой. От перестановки элементов в списке до крышек от люков. Не знаю, как решать, но над этим стебались — значит, не спросят.

Еще интереснее понаблюдать, как человек пытается в реальном режиме времени привести аргументацию, почему он считает, что клас-

сическая задача — глупая и спрашивать ее не надо. Это легко делать, сидя в интернете: погуглил умников, поискал мемасиков, закидал оппонента смешными картинками и доводами. Когда вопрос стоит «здесь и сейчас» — уже хрен.

На Пикабу не было про то, что, поугарав над мемом с Пикабу, ты не стал круче в том вопросе, по поводу которого был мем?

ЭТЮД #20 НЕВОЗМОЖНЫЕ ЗАДАЧИ

«Сообщаю! Решение задачи придумать не могу. Это сделать НЕВОЗМОЖНО!»

К сожалению, среди программистов это частое явление. Заявить, что что-то сделать невозможно и перевесить головняк с клиентом или поиск альтернативных путей решения на начальника или менеджера. На языке менеджмента это звучит как: «Отстань со своей задачей, мне лень думать». Но от этой беды людей довольно легко отучить, возвращая им подарок «НЕВОЗМОЖНО» И «ЛЕНЬ ДУМАТЬ» — обратно.

Обычно я прошу человека рассказать, какие методы решения он применял, с каким конкретно сложностями столкнулся, какие плюсы, минусы у каждого из применяемых методов. Очевидно, если программист вел себя добросовестно — он расскажет все конкретно и покажет, какие были артефакты его поисков. Покажет код, покажет изученную документацию.

Хитрец попыбует юлить. «Да я не помню», «Ну там какие-то проблемы». В этом случае управленческую борьбу (о сотрудничестве тут речи уже не идет) переносим в «здесь и сейчас». И уточняем, что за явление с человеком такое произошло: амнезия по поводу только что решаемой задачи? Мерцающий идиотизм? Ну-ну...

Программисты — очень умный народ с очень серьезным чувством собственной значимости. Один раз вернете им подарок «мне лень думать», второй раз попадать на него им очень не захочется.

Хотя был у нас такой упорный парень — назовем его Леха. И вот, как только он начинал «ну это невозможно...», ему это обратно возвращалось. Раз, другой, третий. На четвертый приходит и начинает: «ну это невозможно...». Я на него взгляд поднимаю. Ловлю глазами глаза. Он тупится. Уводит взгляд. Роняет на ходу: «Через час будет готово» и уходит делать. И ведь справился :)

ЭТЮД #21 ЗАВАЛЕННЫЕ СИГНАЛЫ ИЛИ ДЕЛЕГИРОВАНИЕ ЗАДАЧ САМИМ СЕБЕ

У нас в университете читали азы микросхемотехники. Нолики, единички, транзисторы Шоттки, мультиплексоры. Вот это все.

Сигналы от схемок было принято рисовать в виде таких графиков _Г_Г_Г_ (внизу — нолик, прыгнул вверх — единичка). И архиважно было, чтобы сигнал был именно прямоугольный.

Но в реальных системах есть наводки, взаимное влияние, и вообще физика сильно против прямоугольных сигналов. В итоге сигналы начинают заваливаться. Девайсы — глючить.

.^_^.^_.

Приходится учитывать не только логическую сборку схемы, но и как ее, зараза, физически так собрать, чтобы шумы не убивали сигнал.

В управлении проектами такой же эффект наблюдается, когда люди сами себе фиксируют задачи. Со временем задачи становятся все менее и менее конкретными.

0. Иван Иванович. Звонить. Выяснить: сроки и стоимость.

1. Позвонить Иван Ивановичу, выяснить сроки и стоимость (изменен порядок; задача из «контекстной» формулировки стала событийной).

2. Позвонить Иван Ивановичу, обсудить проект (потеряли конкретные вопросы к обсуждению).

3. Иван Иванович — запросить статус (потеряли, что это должен быть именно звонок).

4. Иван Иванович (сохранен только контекст, а остальные артефакты потеряны).

Все, задачу можно трактовать как угодно, зато проверить человека (что он реально сделал, а что — имел в виду) — невозможно.

Требуй четкости во всем. Сам действуй четко. Это экономит время.

ЗАКЛЮЧЕНИЕ. RIGOR

RIGOR — суровость по отношению к самому себе.

Я не знаю ни одного навыка важнее планирования. Планирование, дисциплина, следование плану дает энергию и мотивацию. А не

наоборот: бесполезно ждать, пока снизойдет какая-то там мотивация, и тогда типа вы начнете что-то делать. Черт с ней, с мотивацией. Начните планировать и ставить цели. Сначала через силу. Потом смысл появится. Успехов!

ЛИТЕРАТУРА



- ▶ Максим Дорофеев, «Джедайские техники» и «Путь джедая».
- ▶ Александр Фридман, «Вы или хаос».
- ▶ Дэвид Аллен, «Как привести дела в порядок».

Блог SingularityApp

Часть 9



СТРАТЕГИИ ЗАКАЗНОЙ И ПРОДУКТОВОЙ РАЗРАБОТКИ

Это было ужасное утро, 26 декабря. Мороз на улице был за 30. Я всю ночь колесил по кварталам в поисках нашего программиста, который ушел с корпоративной вечеринки с двумя детскими подарками подмышкой. И не вернулся. Сашка погиб.

Месяц назад с проекта, которым он занимался большую часть времени — последние лет пять, — уволился его напарник. А нового парня еще не успели толком ввести в курс дела. Отличные новости для клиента под конец года.

Звоню, рассказываю «как есть». Обещаю подумать, что можно тут сделать и как будет оптимально. Для меня «подумать» — это не вежливая форма «отстань». А абсолютно формальные техники. С выписыванием фактов, поиском связей между ними, вариантов решений и оценки последствий. В этот раз потребовалось почти двое суток.

«Владимир! — так зовут директора заказчика, с которым у нас сложились доверительные отношения, — Вам эта мысль понравится. Вы ее только сразу не отбрасывайте. С ней нужно переспать. Давайте строить внутреннюю команду. In-house. А я помогу, чем могу». Понадобилась пара дней, чтобы пройти все стадии: от отрицания до смирения. Но в конечном счете мы с клиентом пришли к единому мнению: «Да, это круто! Это сработает лучшим образом в нашей ситуации». И перешли к конкретным шагам.

В этой главе мы разберем нюансы организации команд в продуктовой и заказной разработке. Какую модель лучше выбрать. И как организовать заказную разработку. В том числе техническую поддержку.

9.1 ЗАКАЗНАЯ ИЛИ ПРОДУКТОВАЯ КОМАНДЫ?

В чем разница в организации и управлении. Формирование продуктовой команды. Когда? Где? Как? Из кого?

Разработка софта — это сложно и дорого. Всегда было сложно и дорого. Особенно в современных системах, с кучей микросервисов, интеграций и взаимосвязей. И если есть шанс не писать код, а взять готовое решение или конструктор — берите конструктор.

Мы уже не в песочнице. В картине мира digital-проектов — десятки и сотни артефактов, за которыми нужно следить, чтобы выпускать и поддерживать достойный продукт. И, тем не менее, в какой-то момент конструктора станет мало. Да и конкуренция в основном идет на поле технологичности. И придется с этим разбираться.

Когда ваши задачи переросли коробочные решения, по большому счету, есть три варианта: заказная разработка под ключ, аутсорсинг и in-house команда.

9.1.1 ЗАКАЗНАЯ РАЗРАБОТКА

Вы пишете в агентство. Обсуждаете задачу. Агентство проводит аналитику, собирает требования. Рисует прототипы, дизайн. Пишет код. Запускает проект. Занимается технической поддержкой. Множество технических и организационных вопросов и рисков агентство берет на себя.

Такие команды лучше всего годятся для запуска продукта с нуля, поскольку имеют опыт, компетенции, их не надо собирать, собеседовать и тратить время на притирку. Быстрее всего выпустить и развивать MVP именно в заказной разработке.

Какие плюсы и минусы в этом подходе?

Плюсы:

- 1. Фиксированная или гибкая стоимость контракта/спринта.** В зависимости от задачи и стадии проекта вы можете использовать как фиксированную стоимость контракта, так и перейти к гибкой модели, оплачивая фактически затраченное время. Многие клиенты предпочитают работу по фиксированной стоимости из-за

ее предсказуемости. И это правда хорошо работает на запуске среднего проекта. Однако после запуска, когда новые задачи и хотелки в проект летят со всех сторон, и некогда не то чтобы формализовывать задачи и оценки, но и просто вникать в суть, единственный способ сохранить высокую скорость — Time & Material, с оплатой фактического времени команды. Можно комбинировать эти модели.

2. **Не беспокоят простои команды.** Балансировка нагрузки — один из самых сложных головняков в агентстве. С одной стороны, нельзя допускать простоев команды: это чертовски дорого, расслабляет и приводит к ненужным мыслям. С другой стороны, нужно иметь запас мощностей, чтобы быть готовым быстро включиться в задачи клиента. Тем не менее, высокая нагрузка очень вредна.

Если начальник хочет, чтобы у всех подчиненных всегда была работа отсюда и до пенсии, с максимальной интенсивностью — вселенная подкинет такому начальнику работу без выходных и бессонницу в придачу.

Впрочем, теперь это головная боль агентства.

3. **Не надо думать о найме, обучении, мотивации.** На что уходит масса энергии.
4. **Можно ТРЕБОВАТЬ, а не пасти котов¹.** Котики — умные, сытые, независимые IT-шники, с хорошим запасом чувства собственной важности. С ними порой сложно. А тут — можно требовать. Это, в конце концов, — почетно!
5. **Всякие интересные штуки с оплатами.** Ну вы знаете эти игры. Печать затерялась, бухгалтер счет на столе забыл, или: «У нас платежный день — каждую четную среду месяца, с 11:00 до 11:30. Очень жаль, приходите через месяц».

¹ «Как пасти котов. Наставление для программистов, руководящих другими программистами» — книга Дж. Ханка Рейнвотера.

Минусы:

Главный минус агентства — **расфокусированность**. Скорее всего, менеджер в течение дня переключается несколько раз с проекта на проект. Разработчики тоже могут быть расфокусированы. На этой неделе — одно, на следующей — другое. Это, кстати, неплохо для их роста как специалистов и поддержания интереса к работе, если задачи интересные. В агентствах специалисты растут быстрее, и выбор задач для них больше. Расфокусированность — следствие рваного потока работ, характерного для заказной разработки.

9.1.2 СВОЯ КОМАНДА (IN-HOUSE)

Своя команда оправдана, если у вас от тысячи часов в месяц стабильной загрузки. И видно, что такая нагрузка — это не временное явление, а так будет еще долго.

Плюсы:

- 1. Сфокусированность.** Если специально не раздвигаете команду криками «Важно! Срочно! Внезапно!», а фокусируете ее внимание на действительно важных задачах, помогаете команде расти и решать проблемы — вам покажут наилучшую результативность. Сфокусированность творит чудеса. Однако есть минус: без переключения на разные задачи программистам и дизайнерам может стать скучно. Согласитесь, это маленький ад: несколько лет подряд заниматься отрисовкой каких-нибудь баннеров для бульонных кубиков или поддержкой дремучего кода на технологиях, которые устарели пять лет назад. Рутинная, скука, бесперспективность, декаданс.
- 2. Скорость и гибкость** — пока не обленились и не разжирили. Тут не так однозначно. По моим наблюдениям, у многих программистов, которые поработали на стороне клиента, — вырастает нимб. Особенно у тех, кто почувствовал свою незаменимость («Я — властелин 1С, и фиг вы со мной что сделаете»). Чувство собственной важности зашкаливает до небес. А делать они толком ничего не делают (только инфантильно ноют, брюзжат, все запутывают, убегают от ответственности). В агентствах таких не терпят. А вот в клиентских командах — обычное явление.

Нюансы:

Еще раз: оправданно при тысяче часов в месяц **стабильной** загрузки. Ключевое слово — стабильной!

В команде может быть дизайнер, несколько программистов, руководитель проекта, аналитик-тестировщик. Тысяча часов — это небольшая спецназ-команда в пяти-семи человек. Как раз норма управляемости для руководителя проекта. При хорошей сработанности — может творить чудеса.

Минусы:

1. Простой, найм, обучение, мотивацию, косяки **приходится оплачивать**. Тут есть риски и очень много работы, которую приходится делать каждый день для формирования команды и поддержания их в форме. Много возни по типу воспитателя в детском садике вундеркиндов. Есть масса хороших практик для формирования таких команд, о некоторых мы говорили в главе 7.
2. **Руководитель — расфокусирован**. Тут вот такая штука. В любом проекте самые интенсивные действия по организации идут в начале проекта. Нужно все предусмотреть, продумать, всех организовать. И еще раз — ближе к концу работы, когда нужно все принять, проверить или отправить переделывать к чертовой матери. Однако посередине всегда есть довольно большое плато, где руководитель не обязан бегать в мыле. Все происходит как-то само по себе.



Однако современное программное обеспечение обновляется постоянно. По некоторым проектам мы делаем обновления чуть ли не несколько раз в день. Но по большинству работаем недельными или двухнедельными циклами, спринтами. То есть каждую, допустим, неделю у нас новая версия. И нужно обеспечить «старт», «нормальную работу» и «запуск».

Тут пока все нормально. Однако циклы накладываются. Руководитель должен отслеживать несколько потоков разработки одновременно: продумывать стратегию и приоритеты на пару спринтов вперед, обеспечивать постановки и дизайн для следующего спринта, проверять новые интерфейсы, проверять и помогать команде разработки с текущим спринтом, собирать обратную связь от пользователей по только что запущенному спринту и т. д.

В общем-то, можно распределить работу со стратегией и работу с командой по двум ролям (руководитель продукта и руководитель проекта, либо вырастить внутри команды программистов Scrum-мастера, который будет следить, чтобы игра шла по правилам). Так и делают в зрелых командах. Но кто-то из них будет все равно делать все то, что надо делать, но не делает никто другой.

Как вы уже знаете, самоорганизующимся командам нужен самоорганизатор. В общем, у руководителей тут очень много работы и сильная расфокусировка.

9.1.3 ВЫДЕЛЕННАЯ КОМАНДА (АУТСОРС)

Гибридный вариант, когда под конкретного заказчика резервируется команда внутри агентства. Команда выкупается полностью (реже — частично, ведь тут сразу возникает расфокусировка). Как правило, оплачивается фактически затраченное время. Стоимость задач зависит от их срочности: работа спринтами, как правило, дешевле. А вот «все прям все сейчас бросили и идите перекрашивать кнопку с красной на зеленую» — дороже.

Плюсы:

1. Сфокусированность команды выше, чем в заказной разработке (но не такая, как можно обеспечить в своей команде).
2. Скорость, гибкость.
3. Простои, найм, обучение, мотивация, косяки — не ваша забота.

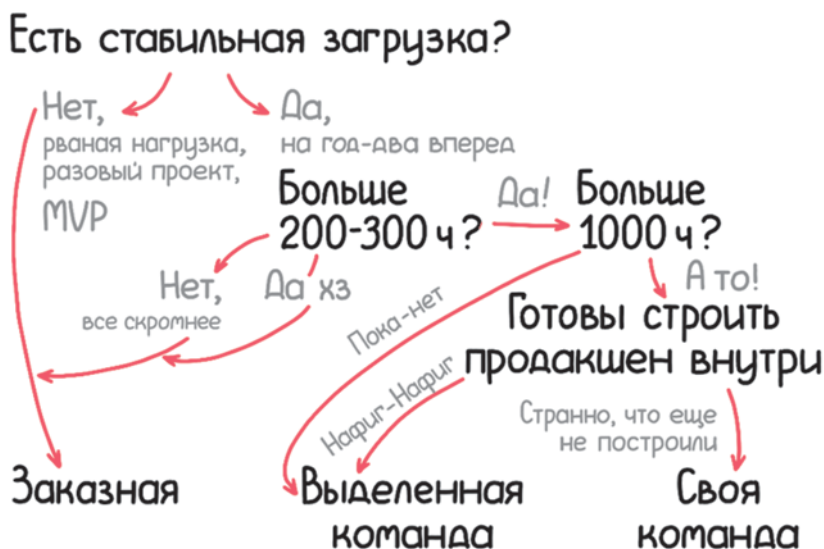
Нюансы: Оправданно от 200–400 часов в месяц. Это пара разработчиков, кусочек руководителя проекта и кусочек тестировщика. Время от времени — дизайнер, если это нужно для задачи.

Минусы:

1. Проджект-менеджер, скорее всего, будет заниматься еще тремя-четырьмя проектами.
2. Норма управляемости — семь плюс-минус два. Это естественный ограничитель размера команды.

9.1.4 КАКУЮ МОДЕЛЬ ВЫБРАТЬ

Итак, если у вас разовый проект, рваная, нестабильная нагрузка или нужно выпустить продукт (или MVP), но нет времени, сил, компетенций и желания возиться и пасти котов, однозначно, заказная разработка — правильное решение.



Если у вас идет стабильное развитие разработанного решения, если у вас планы на год-два вперед, причем, нужно часов 200–400 в месяц, скорее всего, в агентстве под вас будет выделенная команда. Внутри нее могут быть некоторые ротации (увы, средний срок работы сотрудника даже в известной соцсети — чуть более двух лет). Но это не ваша проблема.

Наконец. Если весь бизнес строится вокруг программного продукта, в месяц у вас стабильно задач на тысячу и более часов, а компетенций и энергии для построения внутренней команды хватает, то, скорее всего, своя команда — это ваш вариант.

9.2 DIGITAL КАК УСЛУГА

Частый сценарий запроса на диджитал-услуги — сайт нужно переделывать **ВОТПРЯМОСЕЙЧАС**.

Поводов для переделки может быть много: от банального устаревания дизайна и недотягивания по технологиям (отсутствие адаптива или черепашья скорость загрузки) до желания интегрировать сайт с внутренней инфраструктурой компании.

Обычно инициирует заказ таких услуг какой-то большой-большой человек в компании. У него есть потребность освежить сайт и есть условный подчиненный Леночка, которой задача ставится примерно так: «А не сделать ли нам новый сайт?» Леночка в ответ скажет: «Конечно!», но про себя подумает: «Черт, еще и это!» У ее босса, естественно, будут железные аргументы, поэтому Леночке придется молчать и терпеть. И побыстрее решать эту задачу.

Часто в компании заказчика руководителем проекта становится маркетолог — специалист, который и так сильно перегружен другими смежными проектами: от листовок о сборе макулатуры и благоустройстве прилегающей территории до ведения соцсетей, отслеживания продаж и организации конференций. Эдакий многорукий бог Шива. И вот, ко всему многообразию его задач добавляется еще одна срочная — сделать новый сайт. Причем, параллельно от босса сыплются разного рода вводные вроде «в рамках бюджета, пожалуйста». В переводе это порой значит «бесплатно» или «почти бесплатно».

Леночка — как раз такой человек, и у нее в подобной ситуации всего три альтернативы:

- 1) Конструктор. Это отлично для простых проектов. Но если дело сложное — этот вариант отпадает.
- 2) Поисать внутри компании бородатого эксперта из IT-отдела (который в 99% случаев пошлет с этой задачей на три буквы);
- 3) Найти стадо ~~стадо-бурундуков~~ исполнителя на диджитал-рынке.

Я считаю, что те, кто занимается разработкой диджитал-проектов внутри своей компании на базе имеющихся экспертных групп, либо заказывают услуги на аутсорсе — герои нашего времени. Чаще всего они зовутся digital-продюсерами и играют ключевую роль в диджитал-проектах. Увы, Леночка, скорее всего, не обладает нужными компетенциями диджитал-продюсера. Поэтому она будет руководствоваться несколько иными принципами выбора подрядчика, чем опытный специалист:

1. Цена вопроса

Вопрос «Сколько стоит» — первый, с которого начинается общение агентства с потенциальным клиентом почти в 100% случаев. Почему?

Во-первых, цена — понятный критерий, который можно измерить сразу у нескольких подрядчиков и сравнить. И самое главное: заказчику не надо разбираться, разжевывать каждому разработчику все требования и нюансы, понимать, чем они отличаются (и отличаются ли) друг от друга. В отличие от мутных требований, которые, скорее всего, заказчик пока не может четко сформулировать для себя сам («сделай то, не знаю что», «ну, как у Apple» и все в таком роде), цена — это конкретное число.

Во-вторых, цену можно отжать. После этого какое-то время заказчик чувствует себя героем. А заодно думает, что «раз ребята прогибаются — с ними приятно иметь дело».

В-третьих, названная цена снимает с него ответственность за формулирование требований и переносит их на студию-разработчика. Потому что та обещает сделать проект за конкретные деньги. Кроме того, выбор «самого дешевого» — это тоже снятие ответственности. На вопрос «Почему ты выбрал этих?» с ответом «Самая низкая цена» — невозможно предъявить претензии. А обосновать выбор — «Потому что у них лучшее качество» — уже гораздо сложнее. Нужна смелость брать ответственность за такие решения.

У нашей Леночки была постановка «в рамках бюджета» — поэтому и она будет смотреть на цену в первую очередь.

2. Существенные критерии

Какие еще существенные критерии, кроме цены, могут быть? Обычно клиенты называют многое: сроки, качество, компетенции... Иногда — портфолио. Да, с ним можно посмотреть на уровень работ агентства

и на уровень его клиентов. Правда, Газпром может быть в числе клиентов у большинства, но этот факт ничего не скажет о том, каким будет процесс работы, и не гарантирует успеха в финале.

Ремарка про сроки

Одна из характерных черт большинства клиентов — горящие сроки. Как обычно стартуют проекты: приходит Большой Босс с паяльником и говорит: «Нам срочно нужен проект X. И Мы УЖЕ опаздываем!» А паяльник такой большой, красный. И тут все начинают бегать-суетиться. Общая идея: если не сказать людям, что они уже опаздывают, если не загнать их в нереальные временные рамки — они расслабят булки и будут делать проект годами.

Чем ответит разработчик на требования нереальных сроков?

«Честный» ответит: «Не-е-е, это невозможно». И не получит контракт. А если вдруг получит, будет периодически раздражать заказчика нытьем «мы не успеваем», «мы опаздываем». Получит общественное презрение, порицание и дурную репутацию.

«Опытный» ответит: «Сделаю». Заберет контракт. Потом по ходу работы будут бодрые рапорты: «Все по графику!», «Все по плану», а в последний день: «Извините, прилетели марсиане, увезли на Марс главного программиста, все сервера и исходный код. Нужно еще пару месяцев на доводку вашего проекта». У нас неоднократно повторялся сценарий, когда наш прогноз по срокам был шесть месяцев, клиент выбрал тех, кто обещал за три. Через девять приходит и говорит: «Надо довести проект до ума. Он еще не запущен, но нам скоро его отдадут». Внутри проекта обычно ужас и кошмар.

Замечу, что «Опытный» получит по башке всего один раз, а «Честный» будет получать каждый день. Большой красный паяльник у босса, желание показать конкурентоспособные сроки и возможность «потерпеть боль всего один раз» провоцируют разработчиков сразу говорить нереальные сроки. Ничего удивительного, что их часто срывают.

Для заказчика, чаще всего, важны даже не сами сроки, качество и компетенции, а то, как разработчики РАССКАЗАЛИ о них. Это называется «зажигательность обещаний».

Почему это работает? Потому что на этапе продажи клиент не хочет слышать о потенциальных проблемах проекта — он хочет услышать, что все будет хорошо, что проект сдадут в такие-то сроки, за такой-то бюджет и что «все будет в лучшем виде». Если же подрядчик имеет определенный опыт и знает, что на этом проекте будут сложности, скажем, с интеграцией, он наверняка честно об этом расскажет. И заказчик станет напрягаться. Или даже выберет другую студию, которая о таких нюансах не говорила. Что он с ней будет потом делать, когда все эти проблемы всплывут, — другой вопрос.

3. К черту критерии, «просто пришлите КП!»

Ладно, когда сбор коммерческих предложений поручают маркетологу а-ля Леночка — это полбеды. Если та не первый год на месте, то хотя бы может иметь представление (пусть и смутное) о задаче. Хуже, когда сбор КП доверяют тому, «кого не жалко». Замечу, что так происходит не всегда: в ответственных компаниях, которые понимают важность проекта, сразу выделяют толкового и ответственного человека и дают ему достаточно ресурсов и полномочий.

Тем не менее, ситуация, когда «собрать цены» поручают зеленому сотруднику две недели от роду — очень распространена. Такой сотрудник, как правило, не знает ни планового бюджета проекта, ни целей, ни технических деталей. Он не обладает достаточным опытом, авторитетом и компетенциями. Его мнение вряд ли будет учтено начальством при выборе подрядчика. Его личные перспективы на проекте — отдал КП и свалил в сторону.

Как вы думаете, будет ли такой человек париться из-за проекта? Будет ли он досконально прорабатывать и продумывать с вами, как с потенциальным подрядчиком, оптимальную структуру и вдаваться в технические тонкости? Будет ли приглашать к обсуждению SEO-шника или специалиста по информационной безопасности внутри компании?

Скорее всего, нет. Скорее всего, как только в агентствах начнут задавать ему много вопросов (а многие начнут, если есть желание подготовить почтенным людям почтенное коммерческое предложение), он переключит фокус на тех подрядчиков, которые таких вопросов не задают и просто молча присылают КП.

Итого — среднестатистические критерии выбора digital-подрядчика для Леночки:

Цена: вы должны хотеть мало денег.

Прогибаемость: вы должны прогибаться.

Зажигательность обещаний: вы должны красиво петь (просто хор Турецкого!).

И еще — не задавать неудобных вопросов.

ПРИВЕТ ПОСЛУШНЫМ ИДИОТАМ!

А если серьезно, клиентам действительно проблематично выбирать и ранжировать подрядчиков для своих проектов. Чтобы хоть как-то решить эту задачу, иногда те идут на разного рода ходы. Например, спрашивают кейсы в их конкретной тематике: медицинской, производственной или еще какой-нибудь. С одной стороны, если у подрядчика есть специфический опыт, тот понимает нюансы и подводные камни. «О, у вас в портфолио нет проектов зубоврачебных кабинетов Липецкой области? Значит, вы не знаете нашу специфику. До свидания!» или «О, вы те самые, кто делал сайт нашим конкурентам через дорогу? А вдруг вы все наши идеи своруете! До свидания!» — хорошая такая западня.

Другая «хитрость» — максимально прописывать условия и требования в контракте, чтобы потом можно было прикрываться этим документом в случае возникновения проблем. Увы, мнение: «Контракт — гарантия ВСЕГО!» — это большое заблуждение. Как минимум, по трем причинам:

1. «Поехали, потом заведемся»

История, когда проект начинают, не до конца понимая, что именно хочется получить на выходе, — довольно частая. В общем-то, это нормально. Именно поэтому так хорошо работает Scrum, когда требования конкретизируются по ходу.

Однако цену-то клиент хочет зафиксировать сразу. А требования — нет. Интереснее ведь добавлять по ходу.

Более того, в нашей культуре вполне себе нормой является отношение к подрядчикам «мне плевать на их прибыль» и «хочу результат за их счет». По опыту работы с западными компаниями могу сказать, что в США и Европе это не так.

Вопрос: как должна ответить команда разработки, если поймет, что не сможет на клиенте заработать из-за постоянно растущих требований? Правильно! Плохим внутренним качеством. Качеством кода. Костылями.

Особенно больно это аукнется на крупных проектах, когда клиент попытается их развивать и поддерживать. Все будет тормозить, глючить и ломаться. И никто не захочет брать такой проект на техническую поддержку за вменяемые деньги.

2. «Но мы же договорились!»

Обычно со стороны и подрядчика, и заказчика контракт согласовывают первые лица компании. В общих чертах им все понятно:

— Петрович, сделаешь?

— Да!

Дальше Петрович приходит к программистам, и там происходит примерно такой диалог:

— Программисты, делайте!

Программисты честно спрашивают:

— Что?

— Ну вот ТЗ. Будет. В следующем месяце...

Нормально. И потом, когда проект готов:

— Логисты, вот вам новый сайт! Следите теперь за заказами в админке. Вот доступы!

— !@#%!!

Проблема в том, что договаривались одни люди, делать будут другие, а пользоваться — третьи. Буквально с первых секунд происходит разрыв ожиданий. Даже если есть подписанный контракт, он от этого вряд ли спасет.

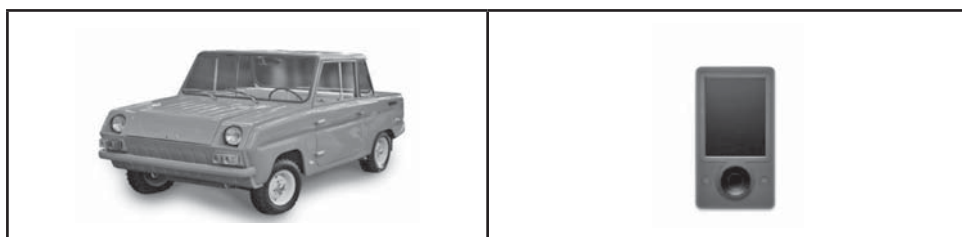
3. «Но мы же можем зафиксировать все требования!»

Ага, щас. В техническом задании зачастую невозможно (или сложно) прописать все-все-все требования. Вы либо скатитесь к инструкциям вроде «не сушить котов в микроволновке», либо у вас всегда будет повод для двойной трактовки требований. И чем детальнее заказчик постарается их расписывать, тем сложнее будет с ними работать. Но от двойных толкований это не избавит.



Скорее всего, на проекте появятся какие-то переделки. И тут-то подрядчик, которого прогнули в цене, точно захочет отыгаться. Особенно, если стало ясно, что долгосрочные отношения с заказчиком у него не получатся. Привет, многочисленные часы на препирательства, было это в ТЗ или нет (позиция такого разработчика очевидна: «Ну, если не на самом контракте, то хоть на переделках-то я заработаю!»).

Увы, реалии таковы, что подписанный контракт, по которому заказчик «договорился» со студией, не гарантирует ему легкой жизни и успешного проекта. Самое грустное, что ни одно ТЗ или подробный контракт не смогут учесть НЕфункциональные требования. И если тупо следовать этим документам, можно получить что-то такое:



Слева — чумовой автомобиль С-ЗД («эс-три-дэ»), или по-другому «Инвалидка». Справа — «убийца» iPod от Microsoft. Скорее всего, изделия ПОЛНОСТЬЮ соответствуют поставленным требованиям. Вроде как и функции все на месте, только вот пользоваться результатом не очень хочется.

В ответ на подобный результат работ вы, скорее всего, услышите: «Вы сделали то, что мы просили, но абсолютно не то, что нам надо». Допиливать «неведому зверушку», которая не оправдала ожиданий, заказчику окажется очень больно и дорого.

Такая ситуация случается чаще всего при работе по водопадной модели, когда на проекте не предусмотрены промежуточные релизы и демонстрации результатов разных этапов работ заказчику. К счастью, в диджитал по этой модели работает все меньше и меньше людей.

9.2.1 ОТКРЫТОСТЬ

Мы уже давно не в песочнице! И заказать диджитал-проект — это не то же самое, что заказать двадцать кубов бетона. Хотя, говорят, и там есть нюансы. Вариант «он вам денег — вы ему счастья» — не работает в принципе. Вариант «он вам денег и ТЗ, вы ему — счастья» — не работает тоже.

На технически сложных проектах, которые глубоко завязаны на бизнес-процессы заказчика (или могут их даже в корне изменить), необходимы:

- ▶ Деньги (да, конечно).
- ▶ Актуальные требования и ожидания (вместо один раз на коленке написанного ТЗ). Тут опять поможет Scrum с возможностью менять требования на ходу.
- ▶ Поддержка в поле власти: проект должен получить добро на самом высшем уровне, а не быть инициативой мальчика-из-маркетинга-вторую-неделю-на-испытательном-сроке. Это нужно, чтобы вы имели потенциальную возможность получать нужную информацию от заинтересованных в проекте лиц на стороне клиента. Вспомните, как на аналитике мы выявляли и опрашивали стейкхолдеров, выявляя внутренние конфликты. Чтобы их решить — нужна власть.
- ▶ Перспектива долгосрочных отношений — чтобы обе стороны знали, что проект поддерживать и развивать еще до-о-о-лго.
- ▶ Адекватное время на разработку системы. И адекватное время, которое руководитель проекта со стороны заказчика будет выделять на решение ключевых вопросов по проекту.
- ▶ Достоверная и непротиворечивая информация. До старта проекта важно учесть требования всех потенциально заинтересованных

сторон: логистов, бухгалтерии, сэйлзов, маркетинга и т. д. Обязательно донесите эту мысль до заказчика, чтобы потом не было «сюрпризов»!

Если клиент вас не выбирает по какой-то причине (не сошлись по бюджету, например), сделайте доброе дело: расскажите, на что стоит обратить внимание при выборе другого подрядчика. И пусть в мире станет еще на один адекватный диджитал-проект больше. Ну и плюсики в вашу карму никто не отменял;)

Что стоит требовать от подрядчика?

Открытости. Разработчики должны объяснить, КАК именно они собираются добиться результатов. Не просто обещания, а четкий и понятный план, как именно требуемый результат появится в нужные сроки. Какие техники будут применяться на прототипировании, дизайне, планировании итераций, или планировании сроков всего проекта, разработке, ежедневном контроле, тестировании, отчетности или технической поддержке.

И только в такой конфигурации на проекте у заказчика может наступить простое человеческое счастье :)

9.2.2 «ХОРОШИЙ-ПЛОХОЙ-ЗЛОЙ»: КРИТЕРИИ ПОДХОДЯЩИХ КЛИЕНТОВ

Чтобы понять, что перед вами потенциально достойный клиент с классным проектом, достаточно трех параметров:

1. Высокая вероятность сделки — означает, что клиент быстро движется по воронке продаж и быстро переходит от стадии к стадии. Чем быстрее клиент делает шаги навстречу, тем выше вероятность сделки и выше его приоритет.

2. Адекватная стоимость или маржинальность контракта — чем крупнее бренд, чем дороже контракт и чем он интереснее с точки зрения финансов, тем выше приоритет у клиента.

3. Беспроblemность — не самый простой критерий, который складывается из кучи факторов. Оговорка: если клиент кажется сложным, далеко не факт, что проблема в нем. Возможно, лично вам просто пока не хватает опыта и компетенций для ведения переговоров с ним. Наверное, именно поэтому на этапе продаж сложные переговоры могут успешно провести только первые лица компании, а на уровень менеджеров отдаются какие-то исполнительские задачи, предварительные переговоры или фиксация договоренностей.

Давайте разберем хорошего, плохого и злого потенциального клиента под микроскопом.

9.2.2.1 Хороший

Скорее всего, проект будет успешным с минимумом сложностей по ходу разработки, если:

- ▶ проект имеет шанс выжить (стать прибыльным, № 1 или № 2 в своей отрасли);
- ▶ сходу есть ощущение, что вы значительно компетентнее клиента, можете и хотите ему помочь;
- ▶ в разговоре чувствуется взаимное уважение;
- ▶ проект технологически крут, интересен и может создать повод для PR;
- ▶ клиент готов вносить предоплату;
- ▶ клиент согласен на **копирайт** или +20% стоимости за NDA (иначе агентство будет без работ в портфолио и бесславно загнется);
- ▶ клиент быстро принимает решения по ходу переговоров;
- ▶ клиент готов выделить 30–60 минут два-три раза в неделю на работу над проектом;
- ▶ есть контакт с ЛПР;
- ▶ клиент имеет четкий дедлайн и намерен его придерживаться;
- ▶ клиент в течение рабочего дня отвечает на письма, а в рабочее время доступен в мессенджере или по телефону;
- ▶ клиент готов на Scrum;
- ▶ у клиента есть деньги на раскрутку проекта.

9.2.2.2 Плохой

От некоторых потенциальных заказчиков проще аккуратно отойти в сторонку и не трогать их. Нужно принять, что вы не сможете сделать все-все-все проекты в мире, и лучше искать именно те, которые подходят вам по духу и комфорту работы.

Так, проект вряд ли получится завершить без крови, пота и слез, если клиент:

- ▶ придирается к работам;
- ▶ намекает на откат;
- ▶ не дает точной информации при брифовании, но торопит заключить договор;
- ▶ имеет штатного или удаленного админа или другого технического спеца, с огромным ЧСВ, который планирует вмешиваться в ход программирования и код проекта или «дописывать» модули прямо по ходу работы;
- ▶ имеет юридический отдел, который хочет выслужиться за счет тонны правочек в договоре;
- ▶ хочет мелкую техподдержку и ничего кроме;
- ▶ хочет почесать свое ЧСВ об ваше и сделать из вас никчемность (а проект — лишь повод для этого).

9.2.2.3 Злой

Также повод задуматься, а стоит ли вообще продолжать общение по проекту, если ваш собеседник:

- ▶ «Передаст»: назначен менеджером, но ничего не делает, кроме копипасты сообщений.
- ▶ Чайка: прилетел-наорал-нагадил-улетел.
- ▶ Фантазер: «сделай то, не знаю что». Проекты космического масштаба и космической же глупости.
- ▶ Пробивальщик цены или тендер.
- ▶ Хулигатель: ругает всех предыдущих подрядчиков, никогда ничем не доволен. Скорее всего, вы попадете под раздачу, что бы ни делали.
- ▶ Гопник: мат, ненормативная лексика, грубые выражения, криминальный или военный сленг.
- ▶ Невежа и хам: допускает неуважительные высказывания и грешит непартнерскими (барскими) повадками.

Если какой-то из этих критериев сработал, — это не значит, что стоит тут же отказаться от работы с таким клиентом. Если признаков набралось больше одного — значит, на проекте есть определенные риски, и с ними либо нужно смириться, либо (что гораздо лучше) заложить эти риски в смету. Либо отойти в сторону. Моя религия говорит, что не надо тратить энергию умных людей, своей команды, ради удовлетворения ЧСВ глупых и токсичных людей на мелких, бесперспективных проектах.

9.2.3 ОПРОСЫ УДОВЛЕТВОРЕННОСТИ

В ходе работ нужно регулярно собирать обратную связь от клиента. Проще всего спрашивать общую оценку **по методу NPS**, от нуля до десяти. Если у вас восемь-десять баллов — все в порядке. Некоторые клиенты принципиально не ставят ничего выше восьми, чтобы вы не расслаблялись. Однако, если несколько этапов подряд вы получали восемь-десять баллов и вдруг внезапно вам ставят четыре — нужно анализировать небольшой период и искать проблему в нем. И исправлять. Это проще, чем получить четыре балла по итогам года.

Цели опросов удовлетворенности:

- ▶ выяснить степень удовлетворенности клиента — чтобы понимать его настроение, впечатления от работы;
- ▶ получить оценку работы менеджера — его проактивности и оперативности;
- ▶ сделать анализ качества работ студии: можно найти узкие места, доработать и улучшить какой-то процесс;
- ▶ предупредить и нейтрализовать возможные негативные реакции: лучше узнать о них на первичных этапах, чем эта бомба замедленного действия рванет в конце проекта перед самым релизом;
- ▶ напомнить клиенту о своих компетенциях: показать, что работа шла прозрачно, четко, с высоким качеством и высокой степенью его удовлетворенности;
- ▶ что-то допродать.

О чем стоит спрашивать?

- ▶ о рекомендациях — порекомендовал бы вас клиент своему партнеру, и насколько по шкале от одного до десяти;
- ▶ о сроках: был ли этап окончен в оговоренные сроки;

- ▶ о работе менеджера: насколько прозрачно и регулярно информировал о ходе работы по проекту, как быстро отвечал и проявлял проактивность;
- ▶ об общей степени удовлетворенности: попросить оценить по шкале;
- ▶ о возможных улучшениях в ходе работы.

Итак. Регулярно, на протяжении всего проекта, снимайте обратную связь и реагируйте, если есть проблемы.

9.2.4 ПЕРЕДАЧА ПРОЕКТОВ

Передача проектов — например, от продавца к руководителю проектов или между руководителями проектов — связана с потерей информации. Потери неизбежны. Далеко не все отражено в письменных документах. Плюс есть личные отношения, договоренности и ожидания, которые не передать никак. С одной стороны, это плохо. С другой стороны, позволяет перезапустить отношения и начать с чистого листа. Это такой кайф, иметь право ничего не знать о проекте и начинать с нуля!

В зависимости от структуры и бизнес-процессов агентства, человек, который занимается ведением потенциальных клиентов от момента обращения до момента подписания контракта, может называться по-разному. Продажник, Sales (сэйлз) аккаунт-менеджер (эта роль вообще сильно размыта) или аккаунт-директор. Далее по тексту, для избежания путаницы, будем называть его «Продажник».

Чек-листы и **контрольные карты** позволяют минимизировать потери передачи информации. Давайте рассмотрим два сценария: передача от продавца — руководителю проекта, в разработку. И передачу проекта от одного руководителя проекта — другому.

9.2.4.1 Передача проекта от Продажника — Руководителю Проекта

У продавцов и руководителей проектов есть скрытый конфликт интересов. Продавцу важно продать как можно больше, и он может пообещать больше, чем реально готова обеспечить компания. Руководителю проектов не нужны проблемные клиенты, горящие сроки, работа по ночам без выходных или невыполнимые технические требования.

Хороший продажник проводит с потенциальным клиентом много времени, выясняя задачу, мелочи, нюансы. И чем дольше, тем обиднее, когда при передаче проекта от продавца к руководителю проектов, который рулит разработкой, теряется часть информации.

Когда заказчик слышит от руководителя проектов вопросы, на которые точно отвечал продавцу пару месяцев назад, в нем начинают шевелиться неприятные мысли. «А в этой компании люди друг с другом совсем не общаются? Мне теперь что, все сначала объяснять?!»

Чтобы информация передавалась по-максимуму, предлагаю несколько этапов:

1. Продажник собирает всю информацию по проекту. Прямо всю. Отчеты, резюме встреч, файлы от заказчика, визуальные материалы, брендбуки, выгрузки, чек-листы по выгрузкам и т. д. И складывает в папку проекта.

2. Продажник составляет **файл передачи** (или **дневник проекта**). Фиксирует в нем, что где хранится. Где лежат договоры и приложения, смета и контакты заказчика. Вспоминает и описывает нюансы проекта.

3. Продажник передает файл руководителю проекта. Назначает дату и время разговора — пока внутри студии, без участия заказчика.

4. Вместе с менеджером обсуждает голосом все вопросы по проекту, отсекает те, которые уже были рассмотрены с клиентом. Особенно важно досконально изучить смету. Каждую строчку, каждую оценку.

5. Продажник, руководитель проекта и заказчик созваниваются вместе, чтобы обсудить оставшиеся вопросы и уточнить нюансы. Обязательно посмотрите вместе смету, этапы и тайминг. Тут не должно быть сюрпризов.

«Может возникнуть ситуация, что на эти вопросы вы уже отвечали. И это нормально. Часть информации при передаче может теряться» — об этом стоит предупредить в начале такого разговора.

Важная часть передачи — правильно представить заказчику руководителя проекта. Рассказать о его достижениях и плюсах для проекта заказчика без слащавых комплиментов и вранья. Клиент должен понимать, что попал в надежные руки.

6. После того, как звонок состоялся, руководитель проекта пишет резюме разговора, отправляет всем участникам. И дальше — хозяин проекта уже он.

9.2.4.2 Чек-лист передачи проекта (пример)

- ▶ Все документы (договор, приложения, счета) сохранены на Google Диск в папке «Проекты» у того менеджера, который прописан в договоре и ведет проект.
- ▶ Коммуникации на этапе пресейла зафиксированы в резюме и постоянно отправляются на email заказчику. Продажник должен быть готов по запросу менеджера скинуть всю переписку или дать ссылку на записи переговоров (а также презентации смет, информацию о встречах, промежуточных обсуждениях).
- ▶ В папке «Проекты» создана папка с названием проекта. В ней — файл-таблица «Дневник проекта», где есть следующие вкладки: смета, таймлайн, график выплат.
- ▶ Прочие материалы хранятся в папке «Материалы по проекту»:
 - ☐ коммерческое предложение;
 - ☐ бриф;
 - ☐ визуальный бриф;
 - ☐ скетч;
 - ☐ фирменный стиль;
 - ☐ прочие файлы, поступающие от заказчика на этапе пресейла.
- ▶ Договор с заказчиком подписан. Выставлен счет и получена платежка.
- ▶ В календаре запланировано обсуждение проекта внутри студии (заказчик не привлекается).
- ▶ Продажник обсудил проект с руководителем проекта. Руководитель проекта сформулировал вопросы, на которые у продавца точно нет ответа/информации.
- ▶ Запланировано знакомство и разговор с руководителем проекта, продавцом и заказчиком. Важно вежливо представить и описать достижения руководителя проекта, обозначить дальнейшие действия и передать ему слово. Параллельно необходимо фиксировать результаты обсуждения.
- ▶ Составлено резюме разговора. Продажник вместе с руководителем проекта проверил, что ничего не упущено — после этого резюме отправляется заказчику руководителем проекта.
- ▶ Проект можно считать официально переданным после передачи проекта менеджеру по чек-листу, получения оплаты и скана подписанного договора и приложения.

9.2.4.3 Передача проекта на время отпуска менеджера

- ▶ Подготовить транзишн-файл — таблицу с данными:
 - Название проекта.
 - Контакты: полный список контактных лиц со стороны заказчика с указанием кто за что отвечает (e-mail, телефон, скайп).
 - Статус — где сейчас находится проект: описать в свободной форме и не забыть про важные детали.
 - Документы — показать, где можно найти:
 - договоры и приложения;
 - актуальную смету на текущий этап (спринт) и на следующие (если есть);
 - счета и акты;
 - таймлайн, график платежей;
 - бэклог (на текущий спринт и общий на проект);
 - техническое задание и прототип;
 - дизайн (если готово не все, то список отрисованных и согласованных макетов надо подготовить отдельно);
 - верстка (если сверстано не все, то список сверстанных страниц надо подготовить отдельно);
 - багрепорт;
 - инструкция по работе с админ-панелью и т. п.
 - Состав команды: список разработчиков, участвовавших в проекте, с указанием, кто и что делал.
 - Кому проект переходит — главное, не забыть предупредить его.
 - Список задач на время отпуска, либо ссылка на нужную страницу бэклога.
- ▶ Уведомить клиента о передаче проекта на время отпуска.
- ▶ Обсудить голосом по каждому проекту с новым ответственным задачи и нюансы по проекту, а также линию поведения в ответ на те или иные действия заказчика.
- ▶ Отправить уведомления заказчику (временно ответственный ставится в копию):
 - в очередном еженедельном отчете;
 - в письме-напоминании перед самым отпуском.
- ▶ Пока менеджер в отпуске:
 - ставить его в копию всей переписки с заказчиком;
 - ставить ему в календарь (или другую тикет-систему) вопросы, которые он должен будет решить по проекту после возвращения.

9.3 ОРГАНИЗАЦИЯ РАБОТЫ ТЕХНИЧЕСКОЙ ПОДДЕРЖКИ В ЗАКАЗНОЙ РАЗРАБОТКЕ

Техническая поддержка — это боль и слезы рынка digital-разработки. Декларируют техподдержку многие. Делать системно и рентабельно умеют единицы. Организовать работы грамотно, четко и рентабельно здесь **ОЧЕНЬ** сложно. В первую очередь потому, что требуются высокие и разносторонние компетенции руководителей проектов и разработчиков. Сегодня вы обсуждаете, как интегрировать супер-пупер систему по **шине обмена данными**. Завтра — учите нового человека на стороне клиента, как попасть в админку сайта. Оперативность, многозадачность и частые переключения. Много-много рутины. И это все на фоне высокой конфликтности атмосферы. А еще — в противовесе с возможностью спокойно делать новые и интересные проекты на новых неизведанных технологиях.



Грозовая тучка для владельца студии веб-разработки согласно Теории Ограничений Систем (по Э. Голдратту)

Обычно над каждым проектом работает КОМАНДА разработчиков. Это важно, потому что после запуска проекта как минимум два-три человека причастны к коду и могут его поддерживать. На практике

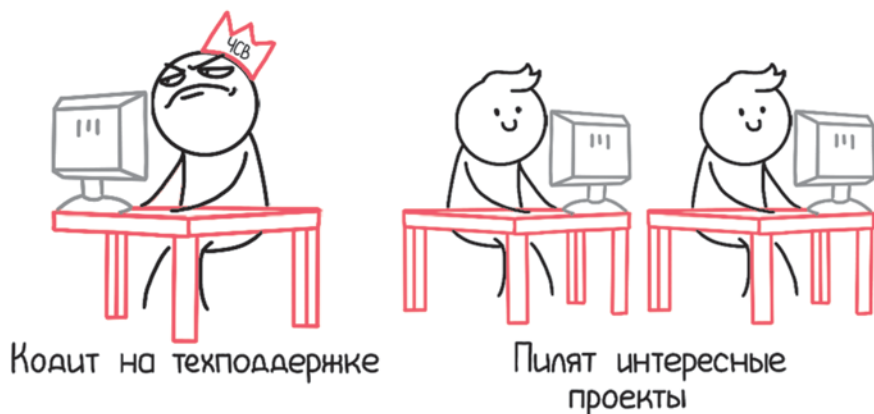
это сильно лучше, чем если бы всю разработку делал один «незаменимый» гений. С незаменимостью также помогают бороться две вещи: строгие стандарты кодирования и регулярные `code review`, которые мы проводим на своих проектах.

Временную поддержку дополнительных программистов, не участвовавших в реализации проекта, я не очень люблю: высок риск запустить прогрессирующее отравление проекта. Это явление — проявление человеческого фактора и, кажется, во многом обусловлено русским менталитетом. Желание творить крупными мазками, а не кропотливо доводить детали.

Нежелание нового разработчика лезть в чужой код и его понимание, что проектом он занимается временно — приводит к тому, что в код втыкаются костыли. После появления в коде критической массы костылей, даже команда, которая изначально разработала проект, начинает считать код чужим и решает задачи с помощью новых костылей. Через некоторое время проектом заниматься никто не хочет, и все ноют, что он — говнокод. Купировать отравление можно, но довольно затратно: часто требуется провести глубокие код-ревью и `рефакторинг`. Иногда два-три раза подряд.

В общем, добавление новых людей в поддержку я считаю оправданным только в трех случаях:

1. Мне нужно ввести нового человека в проект, чтобы он занимался им дальше на постоянной основе. Например, если чувствуется, что предыдущая команда устала от проекта — либо им надо давать новые задачи и вызовы, либо мы их потеряем.
2. Для развития новичков под присмотром куратора.
3. Очень-очень редко (и рискуя получить за это минусов в карму), но все же: как мера воспитания слишком честолюбивого разработчика с задатками гурмана. Я имею в виду тех редких людей, которые на любую задачу начинают разглагольствовать, что она им неинтересна, все технологии — говно, весь чужой код — вообще говно и даже свой, который он писал неделю назад — тоже устарел. Он вполне может схлопотать себе на ближайшие два-три месяца (или пока не уgomонится, или пока не станет ясно, что не сработаемся) проекты только на техническую поддержку. Это в то время, как его коллеги получают новые, интересные проекты.



При планировании нагрузки на разработчиков мы рассчитываем, что их рабочий день состоит из двух частей: работы над основным проектом (закладываем шесть часов) и технической поддержки (оставшиеся два часа). Перекуры, технические перерывы и кофе размазаны по всем часам и никак не фиксируются.

Как правило, часы техподдержки наступают с 16:00 и до 18:00. Это время хорошо подходит для решения мелких и простых задач. Обратная сторона медали: деплой на продуктивные сервера происходит в конце рабочего дня, и в случае чего, целую ночь на продуктиве может висеть «кака». На критичных проектах деплой или часть саппорта мы переносим на утро. Если в какой-то день нет задач на техническую поддержку — программист занимается основным проектом.

В перерывах между спринтами (например, когда идет интенсивное тестирование и заниматься проектом нельзя), мы можем ставить больше техподдержки, если это целесообразно. В утренние часы разработчиков стараемся не дергать (без крайней необходимости). Важно, чтобы разработчики работали в потоке и как можно меньше отвлекались и переключались. Уменьшаем энтропию. Кроме того, это позволяет разработчику не заикливаться только на саппорте старого, но и пробовать новые технологии на новых проектах.

Итак, мелкие и срочные задачи мы вклиниваем в эти два часа в день, с 16:00 до 18:00 (ну фактически, с учетом командной работы это может быть и четыре, и шесть часов — в зависимости от того, параллелятся ли задания). Крупные задачи, которые можно делать в спокойном режиме, мы запускаем в работу спринтами — минимум по 60 часов —

в основные часы. Работа спринтами в пересчете на стоимость часа для клиента дешевле, чем экстренные и срочные задачи, которые нужно сделать еще вчера.

За коммуникацию с клиентом на технической поддержке в большинстве случаев отвечает тот же менеджер, что и работал над проектом. К сожалению, это не всегда рационально с точки зрения использования дорого-щ-щ-щего времени менеджеров (дорогого — не только в смысле затрат, а в смысле Теории Ограничений: менеджеры очень часто становятся бутылочными горлышками системы, а час простоя в бутылочном горлышке равен часу простоя всей системы).

Если на менеджера за счет технической поддержки чрезмерная нагрузка (мы называем это «рвет радугой») и потери на передачу проекта другому менеджеру принципиально окупятся — я могу поменять менеджера. Часто проекты на поддержке попадают подрастающему поколению для прокачки собственных сил и набора клиентской базы. Крайне редко менеджера меняем, если на проекте возник какой-то конфликт с клиентом и нужно начать отношения с чистого листа.

В исключительно редких случаях нехватки менеджерского времени с формулировками клиента напрямую работает программист. Однако мы стараемся минимизировать такие коммуникации, поскольку нечеткость формулировок крайне нервирует программистов, а ответы программистов на вопросы клиентов без предварительной вычитки могут конфузить клиента наповал.

9.3.1 КАКОЙ БЫВАЕТ ТЕХПОДДЕРЖКА

Бесплатная гарантийная

Для ориентира: у нас она такая для всех проектов на этапе ввода в эксплуатацию (обычно это три месяца) и в гарантийный период (12 месяцев). Это закреплено договором. В других компаниях может быть по-другому, но год, кажется, становится стандартом отрасли.

В первом случае менеджер проекта — на горячем старте и готов ответить на любой возникающий вопрос у клиента.

Во втором — мы готовы поправить любые найденные клиентом скрытые дефекты, которые не были обнаружены при запуске проекта. Но здесь речь только про явные баги. В спорных случаях мы оставляем за собой право требовать, чтобы клиент обосновал нам,

почему это — баг. Если на диагностику требуется время и по факту баг оказался не багом, — мы также оставляем за собой право выставить счет на затраченное на диагностику время. Мера вынужденная и редкая, поскольку неизбежно приводит к конфликту. Порой проще пойти навстречу и сделать лишнего, чем возиться со счетами и бумажками.

Развитие проектов, разработанных у нас

Проекты часто остаются с нами на многие годы, развиваются и эволюционируют. Обычно мы добавляем новые функции спринтами. Нам удобнее и дешевле делать один большой блок работ, чем 100500 мелких срочных задач: экономится время менеджера, меньше рисков допустить ошибки, значительно меньше требуется контроля, и процесс идет более гладко. Однако срочные и мелкие задачи, когда ГОРИТ, клиент не должен накапливать. Это плохо для его здоровья. Поэтому такие задачи тоже могут попасть в разработку, но с наценкой за срочность.

Проекты, которые делали не мы

Таких запросов все больше и больше, но с ними очень сложно работать. Большинство из них не соответствуют глобальным целям и идеологии нашей компании — жизнь за счет техподдержки, на мой взгляд, более стабильный, но скучный бизнес. Нет драйва и надрыва, что ли. Поэтому сторонние проекты берем крайне редко и неохотно. Критерии здесь такие (порядок важен):

- ▶ проект должен быть (или потенциально может стать) № 1 или № 2 в своей нише или он должен понравиться нам;
- ▶ код проекта должен быть грамотным или получен зеленый свет (в виде времени и денег) на полный рефакторинг;
- ▶ по проекту есть достаточный объем задач;
- ▶ клиент адекватно понимает наши принципы и ограничения технической поддержки и принципиально согласен с ними.

Эти критерии мы отобрали абсолютно сознательно и отсеки очень много заявок. Если вы готовы работать на поддержке сторонних проектов с кучей мелких, разнопрофильных задач и сумеете это грамотно организовать — у вас, я уверен, выстроится большущая очередь. Мы же заточены под крупные проекты, а много маленьких расфигачат нам весь конвейер.

9.3.2 КАК ОРГАНИЗОВАТЬ ТЕХПОДДЕРЖКУ ФИЗИЧЕСКИ

Есть два возможных варианта организации технической поддержки. Первый — **многоуровневая ТП**.



Сначала отправляем пользователя читать документацию и всячески скрываем контакты технической поддержки. Далее на первой линии его ждет специалист с низким или средним уровнем технических компетенций. Он может решать какие-то простые проблемы и отвечать на простые вопросы. Если ему становится сложно — он переводит обратившегося клиента на технического специалиста, который разбирается в проблеме намного лучше. Причем иногда не сразу, а назначает на какое-то конкретное время. Наверняка сталкивались: так делает любой провайдер или банк.

Эта схема подходит, когда у вас всего один большой продукт, он технически сложный и по нему есть регулярное большое число вопросов и обновлений.

Второй вариант организации технической поддержки — **принцип одного окна**.

Здесь есть человек, который принимает задачи у заказчика. Затем распределяет их по специалистам. Принимает задачи и контролирует ход работ. Получившееся — отдает заказчику.



Такая схема больше подходит для поддержки проекта в заказной разработке. То есть, агентство запустило проект заказчика и далее берет его на обслуживание. К сожалению, менеджер на стороне агентства — бутылочное горлышко всей системы. Чтобы обеспечивать в таких условиях оперативность, особенно если поток задач огромен, нужно очень постараться.

9.3.3 ЧЕГО ОБЫЧНО ХОЧЕТ КЛИЕНТ НА ТЕХПОДДЕРЖКЕ

Проактивных предложений по улучшению проекта

По проектам в активной фазе поддержки проактивную позицию должен занимать менеджер проекта. Он замотивирован в получении процента от проектов. Проекты, по которым нет никаких работ, мы системно фильтруем несколько раз в год и стараемся их «разбудить». Это хорошая практика, приносящая свои плоды. Будит спящих клиентов, как правило, аккаунт-менеджер. Но не всегда спящего клиента нужно будить :)

Ставить задачи «как придется»

Это важный момент, который касается ответственности. Ответственность — штука обоюдоострая. Здесь есть несколько принципов, о которых мы договариваемся с клиентом. Как правило, с самими

принципами клиент не будет спорить (они понятны и логичны), однако будет пытаться найти лазейку между принципами. Менеджер проекта, кстати, тоже.

- 1) *Дерьмо на входе — дерьмо на выходе.* Плохо поставленная задача с высокой степенью вероятности будет сделана неправильно.
- 2) *Никакой телепатии.* То есть, вообще считаем, что по умолчанию телепатии нет. «Это же было очевидно» или «Я подразумевал, что это будет само собой» — к сожалению, не аргумент. Принцип верный, но в нем есть изъян: это очень большая лазейка для недобросовестного менеджера проекта бесконечно тянуть деньги и делать одну и ту же задачу. Но клиент всегда интуитивно чувствует, когда его доят, ему это очень сильно не нравится, и он вам обязательно про это скажет.
- 3) *Полученное задание должно быть проанализировано с точки зрения выполнимости, целесообразности, полноты и ресурсов.* Наверное, 70–80% рабочего времени менеджера тратится на то, чтобы въехать в задачу, уточнить формулировки, понять, на что она может повлиять, предложить более эффективные ходы, предусмотреть все риски или оценить задачу. Вполне возможно, что вместо того чтобы отдать задачу в работу, менеджер будет названивать и задавать вопросы. Или вообще отговорит клиента делать ее. Да, вам не заплатят за то, что вы отговорили. И иногда очень сложно подобрать аргументы, чтобы отговорить. Но так — кармически правильнее. Впрочем, если аргументы не помогут — у вас не будет выбора, кроме как заткнуться и сделать, как просили :)
- 4) *Двойных трактовок не избежать.* Опять же, тут есть лазейка как для менеджера, так и для клиента: «включить дурочку». Но если работать системно, то любителей шнырять в лазейки можно отловить и наказать, применить управляющее воздействие.
- 5) *Иногда формулировки нужно уточнить.* Аналитикам или разработчикам. В основном это касается больших задач или тех, где нужен эксперимент или рисерч. Или совсем уж запутанных постановок и явной дислексии.

Итого: у вас есть право уточнять и переформулировать постановки клиента в более четкие, уточнять и дописывать детали. И даже отговаривать клиента от каких-то задач (если, на наш взгляд, они

сделают проект хуже). Однако **за финальные формулировки несет ответственность клиент**. Зафиксированные вами формулировки один в один будут направлены в разработку. И по ним же будет проверять тестировщик. И по ним же вы будете сдавать задачу.

Ставить задачи «где придется»

Сегодня менеджеру прилетит постановка «ВОТ ТУТ БАГ» в Телеграмм, завтра — «СЛОМАЛОСЬ!!!» в Ватсапп, послезавтра — он вам позвонит, пока вы сладко спите ночью. Желание понятное, но вредное. Создает неразбериху и делает любой проект технической поддержки убыточным. Чтобы такое обрабатывать, менеджеру проектов нужно ничего не делать, но все время быть на стреме. А это экономически нецелесообразно и морально тяжело.

Решение: на все проекты по техподдержке завести единую систему хранения задач клиента — таск-менеджер. Там же идет их уточнение, оценка, и проставляются статусы. Самая популярная на западе система — Zendesk. Можно использовать и неспециализированный софт: Trello и Jira. Или даже организовать оперативную работу в Google Docs и в портале Битрикса. Но первая задача — свести все каналы коммуникации в одно четкое место. Именно в этой системе вы работаете по задачам и проблемам средней срочности

Самое первое, что требуется, чтобы менеджер и клиент притерлись, — это настойчиво, но вежливо научить клиента писать и отвечать в таск-менеджер. Понятно, что все равно в каких-то случаях клиент будет писать в Zoom, звонить по телефону или писать 50 писем в течение часа в почту. Задача менеджера здесь: вежливо попросить поставить задачи в нужном таск-менеджере. Потому что в Телеграме задача потеряется (да, это аргумент). Поначалу это может раздражать клиента, однако со временем плюсы такого подхода станут понятны и ему. И да, для срочных форс-мажорных задач можно все бросать и решать проблему по телефону. Это нормально.

Возможно, если это предполагает специфика бизнеса, вы организуете отдельный канал для экстренных вопросов, например телефонную линию дежурного менеджера.

Обсуждение крупных задач лучше вести в Скайпе, Zoom или по телефону, с обязательной фиксацией результатов разговора в конкретные задачи. После него клиенту останется только проверить и утвердить правильность формулировок и дать добро в работу.

Точных оценок

За большинство проектов технической поддержки оплата идет по фактически затраченному времени. В задачи менеджера входит:

- ▶ Дать предварительные оценки трудоемкости.
- ▶ Выяснить, делать задачи или нет.
- ▶ Если есть отклонения от оценки (например, нашлась какая-либо особенность у проекта, которая не дает закончить работу в срок), предупредить заказчика и сообщить новые прогнозы и риски.
- ▶ По окончании работ зафиксировать реально затраченное время и сообщить заказчику итоговую стоимость. Раздувать сроки и оценки особого смысла нет, поскольку клиент это чувствует и выскажет вам это на опросе удовлетворенности.

Замечу, что работа по Time & Material на практике для клиента быстрее и дешевле, однако возможна, только если клиент доверяет и доволен. Вам это тоже выгодно: так сокращаются затраты на менеджмент и бюрократию (а менеджер, напомню, у нас всегда бутылочное горлышко системы). Об оценках мы подробно говорили в главе 5.

Ответственности за сделанное

При схеме работы с финализированными оценками все риски по кодированию лежат на стороне разработчика. В студиях такое не любят, но иногда приходится.

При схеме Time & Material, по сути, все риски лежат на клиенте. Если ошибок много и их исправление выставляют клиенту, — он будет недоволен и вам про это скажет. Как правило, экономически выгоднее не вступать в спор и поправить свои ошибки за свой счет, не нагнетая обстановку. Даже несмотря на то, что согласована схема работы по фактическим часам. Я придерживаюсь того, чтобы исправлять ошибки за свой счет, если:

- ▶ проект на поддержке;
- ▶ понятно по шагам, как ошибку воспроизвести;
- ▶ обеим сторонам очевидно, что это — ошибка разработки (реализации).

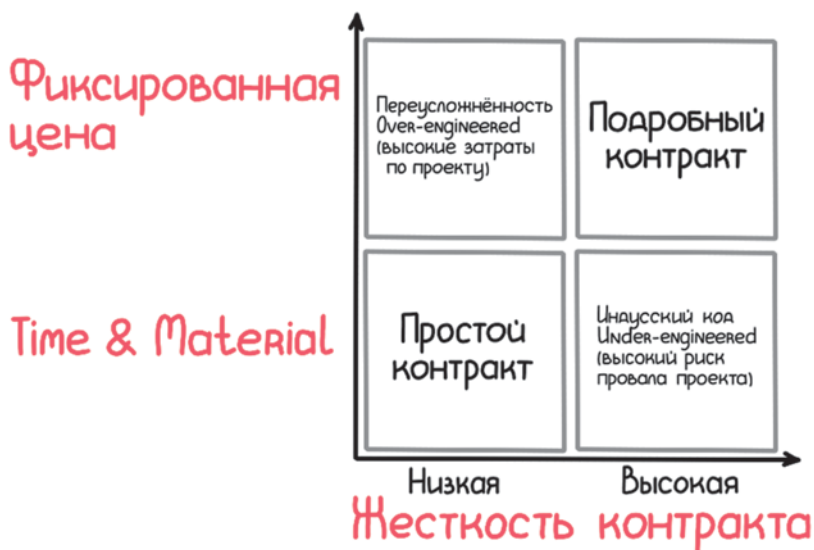
Последний пункт дает повод для инсинуаций как заказчика, так и разработчика, поэтому не может быть описан формально. Например, если причиной ошибки послужила неполнота постановки задачи — **ответственность за это лежит на клиенте** (ведь именно он отвечает за приемку финальных формулировок). Более того, это значит, что с клиента не брали денег за реализацию задачи в более расширенной трактовке. Если же косяк откровенно ваш — исправляйте, не компостируя мозги. Так в итоге дешевле.

Исключение: очень сложные системы с большим количеством зависимостей. Поменяешь запятую в карточке товара — сломается какой-нибудь хитрый импорт, который запускается раз в месяц. Здесь либо нужен `code freeze` + полный тест системы на каждое изменение (что очень дорого), либо полное покрытие кода автотестами (что тоже очень-очень дорого), либо смирение, что такие ситуации возможны.

Замечу, чтобы схема работала, нужно, чтобы спорные ситуации возникали как можно реже. Иначе клиент будет недоволен поддержкой, или поддержка будет убыточной для агентства.

И да. Агентства-разработчики, как правило, не берут на себя ответственность за третьих лиц (например, за хостинг) или упущенную выгоду. Это нормальная сложившаяся практика на рынке, и обычно клиентами это воспринимается адекватно.

Рекомендую не увлекаться с клиентом обсуждением темы техподдержки, а просто обрисовать две схемы работы: с оценками, в которые заложены агентские риски, и оценками, где выставляются фактически понесенные затраты, и дать выбор. В конце концов агентства должны понимать, что бюджеты клиента ограничены, а клиент — что агентствам нет смысла работать без прибыли. К сожалению, в нашей культуре результат за чужой счет — это норма.



Компромиссным может быть вариант с абонентской платой, в которой предусмотрено N часов технической поддержки в месяц, которые сгорают, если их не выбрали за определенный период — считается, что специалист был зарезервирован на эти часы за заказчиком. Реже неистраченные часы полностью или частично переносятся на следующий отчетный период, но, как правило, такая схема не интересна агентству и очень сложна в документообороте.

Оперативности

В наших штатных договорах мы гарантируем время реакции 24 рабочих часа. В 97% случаев это — годный вариант. Но кому-то нужен 24/7, причем нужен реально, а не «что бы вдруг если что» — тут не обойтись без абонентской платы. Обращаю внимание, что речь идет именно о времени реакции, а не о времени готовности любой поставленной задачи.

В основном реальная скорость работы зависит от того, как быстро задача попадет на разработку.

Форс-мажоры. Понятно, что там, где форс-мажор, все сломалось и нужно все бросить и починить — бросаем и чиним. Упал хостинг, оплата

не работает. Это критично, и тут нельзя тянуть, даже на выходных. Мы используем Zabbix для мониторинга таких событий (чтобы узнать о возможных проблемах на серверах раньше клиентов) и уведомления в Телеграм. Если уровень форс-мажора выше 3–5% от всех задач — это повод искать огрехи в организации самого проекта или процесса работы с ним. Но многие реально экстренные вопросы можно решить на опережение.

0–8 часов — на приемку и уточнение задачи. Менеджер получает уведомление о создании новой задачи. Банальный деловой этикет и принцип пустого ящика требует, чтобы в течение суток все такие обращения были обработаны. Далее, в зависимости от срочности, — задача может быть либо отложена до формирования спринта, либо экстренно запущена в работу, либо запланирована в плане работы на программиста на следующий день. Либо, если формулировки не ясны, — нужно их выяснить. Формально здесь у менеджера есть лазейка «включить дурочку» и долго-долго чего-то не понимать и уточнять. Но клиенты всегда это чувствуют.

8–16 часов — обычно через сутки (либо в тот же день) после того, как задача попала менеджеру проекта, и все формулировки понятны, она попадает на разработку. Это связано с тем, что мы планируем, какими проектами будет заниматься каждый программист на утренних планерках (в 8:45) и затем уже стараемся не отклоняться от этого плана. Как правило, задача от клиента попадает в план на разработку в тот же или на следующий день после постановки. Зависит от ясности постановок и плановой загрузки.

16–24 часа — в большинстве случаев мелкие задачи выполняются и отгружаются клиенту за первые 8–16 часов. Еще восемь часов — резерв для выравнивания нашего потока работ или тестирования. По большим и трудоемким задачам либо формируется спринт, либо — график выполнения, в зависимости от их срочности и приоритетов клиента. Абсолютно непродуктивно делать 30–40 часовые задачи урывками по два часа в день.

«Ну и чтобы недорого»

Я рекомендую завести минимум две тарифные сетки:

- ▶ обычная — для задач, которые не горят. Их можно набрать в спринт или делать в рамках графика.
- ▶ срочная (когда «бросаем все, делаем любой ценой») — по повышенному тарифу.

Увеличение стоимости срочной техподдержки связано с тем, что мелкие и срочные задачи способны сломать нам весь конвейер и ставят под удар сроки по другим проектам. Мы стараемся свести срочные задачи к минимуму. Клиенты тоже не любят повышенный тариф, но у них есть сознательный выбор. Задача менеджера проектов — предложить клиенту эту альтернативу.

9.3.4 ЕСЛИ В РЕЗУЛЬТАТЕ ТЕХНИЧЕСКОЙ ПОДДЕРЖКИ ВОЗНИКЛИ ДОПОЛНИТЕЛЬНЫЕ ПРОБЛЕМЫ

Это очень нервная ситуация, она не нравится ни клиентам, ни агентствам. Часто причиной становится спешка.

Поэтому, банально, не стоит пренебрегать тестированием. Тестируем изменения очень внимательно и, как минимум, дважды:

- 1) в первый раз — когда выполнили их на тестовом сервере;
- 2) во второй раз — когда перенесли на боевой сервер.

Но! Программный код — множество нелинейных взаимодействий. И изменение даже, казалось бы, небольшое в одной его части может повлечь за собой изменения, проблемы и БАГИ в другом месте. Иногда — в совершенно неожиданном. Например, изменили верстку карточки товара — сломали корзину. Упс!

Что делать в таком случае:

- 1) *«И так сойдет», «Херак-херак, и в продакшн».* Плохой вариант. Запускаем изменения. Вылезут какие-то проблемы в другом месте — поправим, как найдем. Или когда их найдет кто-то другой. Такой подход на практике довольно часто можно встретить в веб-разработке — особенно, у незрелых агентств или при очень большой спешке. С ним можно как-то жить, но только если скорость устранения ошибок значительно выше, чем скорость их проявления.
- 2) *Каждый раз проверять проект целиком после всех изменений.* Очень экономически невыгодный вариант. Полный тест сложного сайта или приложения может занимать дни. При этом даже самый кропотливый тест не дает 100% гарантии, что внесенная правка не повлекла за собой регрессию кода: какие-то повторяющиеся проблемы и ошибки.

- 3) *Автоматические тесты.* Критические пути пользователей по сайту лучше всегда покрывать тестовыми сценариями. А сценарии — автоматизировать. Не гарантия, что ошибок вообще не будет, но повышает вероятность, что не будет регрессий. Автотесты — это скрипты, воспроизводящие поведение пользователя на сайте. Например: просмотреть список товаров, отфильтровать каталог, перейти в карточку товара, положить товар в корзину и оформить заказ. Эти скрипты автоматически запускаются с заданной периодичностью или во время деплоя, или по ходу разработки. Если что-то пошло не так, оповещение уйдет руководителю проекта на стороне студии и клиенту. Одна проблема: автотесты — это очень дорого.
- 4) *Проверять только изменения:* делать небольшой смоук-тест по основным функциям проекта и основным пользовательским маршрутам.
- 5) *Периодический пересмотр.* По истечению определенного периода времени проверять проект целиком. Разумеется, по согласованию с заказчиком и с выставлением счета.

Чаще всего заказчики готовы терпеть некоторое количество некритичных багов, которые агентство готово оперативно устранить. Причем воспринимается и скорость, с которой было принято обращение, и скорость, с которой проблема была устранена. Менеджер проекта должен обеспечить прозрачность по срокам готовности задачи.

9.3.5 ПОДДЕРЖКА САЙТОВ С БОЛЬШИМ ОБЪЕМОМ ДОРАБОТОК

Если у клиента активно работает отдел маркетинга, тот генерирует множество задач. Задач очень срочных, которые напрямую влияют на продажи с сайта. Добавили на сайт особенные скидки к 8 Марта, запустили рекламу — продали на 80% больше, чем в прошлом месяце.

Поэтому такие доработки часто выполняются лоскутно. Зачастую — разными командами и разработчиками с разной квалификацией. Задача отдается тому, кто быстрее готов взять ее в работу. Искажения и нестыковки накапливаются, приходится постоянно заниматься тушением пожаров.

В таком случае процесс техподдержки будет таким:

1. Аудит текущего состояния

От клиента нужны доступы к коду и админ-панели CMS. От разработчика — клевый матерый специалист, который сделает аудит кода — код-ревью. Что он будет смотреть:

- ▶ само качество кода: как он структурирован, есть ли история изменений, соблюдаются ли гайды CMS-системы;
- ▶ сколько и каких сторонних модулей CMS используется;
- ▶ безопасность кода (статический анализ уязвимостей);
- ▶ параметры базы данных;
- ▶ современность используемых технологий;
- ▶ использование системы контроля версий (да или нет);
- ▶ скорость генерации страниц, объем используемого JavaScript;
- ▶ реализацию адаптивной версии;
- ▶ наличие аномалий.

2. По итогу — диагноз: утешительный или не очень. Утешительный: где-то что-то можно отрефакторить (переписать, почистить, освежить). Не очень утешительный: придется все переделывать.

Сценариев для переделки два:

- 1) Полный цикл разработки проекта с нуля — старый проект в это время уходит на поддержку в режиме «тушения пожаров»: правятся критические проблемы, но новые функции не добавляются. Клиенту хорошо: проект продолжает приносить заказы. Как только готов новый проект, он «пересаживается» на место старого. Если это сайт — сохранив все старые ссылки, чтобы поисковым системам не к чему было придраться.
- 2) Если у клиента есть горящие доработки, их можно начать делать на старом коде, параллельно постепенно переписывая, рефактора его. Этот вариант дороже (строить новый дом всегда проще, чем реставрировать руины) и более рискован. Если старый код откровенно плохой, сайт может «лечь» от любых изменений.

Маркетолог спрашивает программиста: в чем сложность поддержки большого проекта?

Программист: ну представь, что ты писатель и поддерживаешь проект «Война и мир». У тебя ТЗ — написать главу, как Наташа Ростова гуляла под дождем по парку. Ты пишешь: «шел дождь», сохраняешь, вылетает сообщение об ошибке: «Наташа Ростова умерла, продолжение невозможно». Почему умерла? Начинаешь разбираться. Выясняется, что у Пьера Безухова скользкие туфли, он упал, его пистолет ударился о землю и выстрелил в столб, а пуля от столба срикошетила в Наташу. Что делать? Зарядить пистолет холостыми? Поменять туфли? Решили убрать столб. Получаем сообщение: «Поручик Ржевский умер». Выясняется, что он в следующей главе облакачивается о столб, которого уже нет...»

9.3.6 САМАЯ ВАЖНАЯ ЧАСТЬ СИСТЕМЫ ТЕХНИЧЕСКОЙ ПОДДЕРЖКИ

В долгосрочной перспективе бухтение клиента практически неизбежно (ну или я не знаю случаев, когда клиент был бы целиком доволен двух-трехлетним итогом поддержки, не имел каких-либо нареканий и время от времени не смотрел бы на сторону).

Довольно правильно делать с клиентом пересмотр сделанного за два-три месяца. В Скраме это называется ретроспективой. В русской ментальности — разбором полетов.

Вы садитесь и формально анализируете список задач и смотрите причины возникновения ошибок. Это могут быть формулировки, и в этом случае вы совместно должны решить, как вам улучшить постановку задач. Или могут быть слишком частые ошибки программирования, и тут нужно искать причины на своей стороне. Наконец, клиент может быть недоволен тем, что его не отговорили от каких-то его (!) идей и это привело только к лишней трате денег.

*Как-то мы рефакторили большой проект. Часть кода переписали с **jQuery** на **Vue**. Часть еще осталось в старом коде. Параллельно шла работа над новыми задачами и мелкосрочной поддержкой. Команда*

была недовольна, что приходится работать и со старым, и с новым кодом. Все нервничали. Два фреймворка на проекте замедляли его работу. А рефакторингу не видно было конца, поскольку его делали урывками. Пришлось сесть и всем вместе хорошо подумать. В итоге было принято решение сфокусироваться на рефакторинге, добиться его и временно не брать в работу новые спринты. Причем проблемы с производительностью решить за свой счет. Клиенту рассказали в открытую всю логику: какие есть проблемы, почему надо делать так и почему мы готовы часть работ сделать бесплатно. Реакция? «Вот теперь я вижу, что мы действительно с вами работаем в одной команде», — сказал клиент.



Руководитель проекта после руткуз-анализа проблем на проекте.
Считает, что меры по устранению проблем — придуманы.
И меры — помогут.

Вы должны системно проверять, доволен ли клиент работой с вами. Встречаться лично, разговаривать, обсуждать перспективы проекта, перспективы бизнеса клиента и его планы на ближайший год. Особенно осторожным следует быть при смене менеджера на стороне клиента. Важно рассказывать клиенту, где вы можете пойти ему навстречу и где нет, и почему. Это добавляет открытости и управляемости. Без этого в долгосрочной перспективе вас сольют.

9.3.7 ЧТО ДЕЛАТЬ, КОГДА СЛИШКОМ МНОГО МЕЛКОЙ ТЕХНИЧЕСКОЙ ПОДДЕРЖКИ, И ВАС РАЗРЫВАЕТ В ХЛАМ?

Повышать цены и отказаться от наиболее токсичных клиентов (сегменты С и D по ABCDX-сегментации из главы 4). Совет — жесткий. Но и выбор у вас — сдохнуть на проекте, спалить команду. Или привести работу в спокойное русло. Денег будет больше. Проблем — меньше.

9.3.8 КРАТКИЕ ВЫВОДЫ

Итак, техническая поддержка — это довольно сложная штука в организации. Давайте еще раз разберем основные рекомендации по ее организации:

1. Работайте на опережение. Мониторинг серверов или оперативные предложения по улучшению проектов с вашей стороны позволят более грамотно и равномерно распределять нагрузку на разработчиков и устранять инциденты.
2. Приучите заказчика записывать все задачи в определенное место. Согласовывайте формулировки по мере поступления задач.
3. При необходимости организуйте несколько линий технической поддержки. Оберегайте основную команду разработки от «СРОЧ-НЯКОВ» и пожаров.
4. Изначально сообщайте вилочную оценку: от и до. Если превышаете вилочную оценку — сообщите заказчику и уточните, все равно ли делаем или пора остановиться.
5. Установите время реакции на заявки и соблюдайте его. Ваши мощности должны реально быть способны. Не обещайте того,

чего вы не в состоянии обеспечить. Например, 24/7 реально потребует посменной работы.

6. Объединяйте несрочные задачи в спринты, чтобы разгрузить «бутылочное горлышко» всей системы — вас.

9.4 ТАКТИКА ОРГАНИЗАЦИИ IN-HOUSE-КОМАНДЫ

Общий тон настроений в коллективе должен быть мажорный.

А. С. Макаренко

Довольно сложно давать конкретные советы, не зная реального проекта, архитектуры и культуры компании. Однако если вы решили строить in-house команду, эти идеи и принципы вам помогут.

1. Поставьте сильного руководителя проекта. Лучше с агентским опытом. Еще лучше — с сочетанием технического бэкграунда и хорошими переговорными навыками. Все крутится вокруг этих двух компетенций. Знания в маркетинге будут плюсом.
2. Выбирая между двумя руководителями проектов, берите самого позитивного и энергичного. Никакие процессы не заменят энергию и драйв. Унылый руководитель проекта гарантированно сделает унылой всю команду и унылый продукт.
3. Поддержите его властью. Руководитель проекта не должен быть мальчиком для битья, а должен иметь право формировать команду. Кроме того, у него должен быть доступ к ресурсу, открывающему любые двери внутри компании. Иначе проект забуксует во внутренней бюрократии. Итак, руководитель проекта должен иметь доступ к полю власти и поддержку со стороны высшего руководства.
4. Стройте команды. Не один программист, а минимум — два. Иначе есть риск остаться с неподдерживаемым кодом.
5. Первую версию продукта или большую автономную разработку лучше отдать агентству. Так будет быстрее. Когда проект будет вызревать (достигнет какой-то стабильной точки), можно начать формировать свою команду.

6. Используйте классический Scrum или Канбан для организации разработки. Не изобретайте колесо. Проводите стендапы. Демонстрации. Планинги. Обсуждайте задачи. Команда должна чувствовать свою причастность к важному.
7. Используйте тикет-системы. Снижайте до минимума СРОЧНО-ВАЖНЫЕ задачи в мозг.
8. Проводите ретроспективы. Следите за удовлетворенностью команды. Прислушивайтесь и решайте проблемы.
9. Подбадривайте. Хвалите больше и чаще. Общий тон настроений в коллективе должен быть мажорный.
10. Фокусируйтесь. Минимизируйте переключения команды. Дерготня, суета, спешка приводят к ошибкам и выгораниям.
11. Заведите и отслеживайте несколько простых метрик, говорящих об успехе команды. Визуализируйте эти метрики. Это могут быть оценки задачи в Story Point, которые вы отгрузили за очередной спринт (план-факт) и количество дефектов. Три таких метрики (план, факт, дефекты) уже достаточно.
12. Решайте проблемы команды. Медленное железо, слабые сервера, неудобные стулья или необходимость отгулов — за этим всем нужно следить.
13. Проводите раз в полгода стратегические сессии с участием всей команды и заинтересованных в проекте лиц. Формируйте планы. Прозрачные и понятные для команды. Должна быть ясность целей и ясность пути.
14. Ищите вызовы и интересные задачи. Давайте время на изучение новых технологий. Работа не должна превращаться в рутину.
15. На зрелых проектах — документируйте и стандартизируйте разработку. Покрывайте проекты автотестами.
16. Имейте в виду, что статистически рано или поздно у вас из команды будут отваливаться люди. И приходиться новые. Это неизбежно, что бы вы ни делали. У вас, по сути, есть два года, чтобы раскрыть потенциал человека. Если все сделаете грамотно — то больше, но реже. Постройте процесс онбординга (погружения новых людей в команду). Избавляйтесь от токсичных людей, которые тормозят команду.
17. Постройте карьерный трек по каждому члену команды. Проводите аттестации. Внедрите карты компетенций. Давайте время на учебу. Внедрите OKR.

18. Сами много читайте и развивайтесь. Пока есть развитие — не будет застоя. Обязательно почитайте старые книги Макаренко (например, «Педагогическая поэма»).
19. Снижайте персонало-зависимость. Стремитесь к совместному владению кодом. Не должно быть укромных уголков.
20. Делайте хороший продукт для хороших целей. Не тратьте время на разработку ерунды, от которой всех воротит.
21. Все будет хорошо. Даже если не будет!

ЛИТЕРАТУРА

- ▶ Дж. Рейнвотер, «Как пасти котов. Наставление для программистов, руководящих другими программистами».
- ▶ Антон Макаренко, «Педагогическая поэма».
- ▶ Ашкеназ Рон, Мэнвилл Брук, «Настольная книга лидера. Как управлять собой, вдохновлять команду и вывести компанию на новый уровень».
- ▶ Алексис Филлипс, Филлип Санда, «Потенциал команды. Как добиться максимальной эффективности командной работы».
- ▶ Адизес Ицхак, «Идеальный руководитель: Почему им нельзя стать и что из этого следует».
- ▶ Саймон Синек, «Лидеры едят последними: как создать команду мечты».
- ▶ Павел Сивожелезов, «Мягкий босс — жесткий босс. Как говорить с подчиненными: от битвы за зарплату до укрощения незаменимых».
- ▶ Роберт Чалдини, «Психология влияния».
- ▶ Фурнье Камиль, «От разработчика до руководителя. Менеджмент для IT-специалистов».
- ▶ Дейл Карнеги, «Как завоевывать друзей и оказывать влияние на людей».
- ▶ Алексей Ходорыч, Вадим Петровский, «Энкоды: Как договориться с кем угодно и о чем угодно».
- ▶ Гэвин Кеннеди, «Договориться можно обо всем».
- ▶ Марк Гоулстон, «Как разговаривать с мудаками».

ПОЯСНЕНИЯ

Копирайт — знак авторского права, в веб-разработке реализуется упоминанием названия и логотипа студии в подвале сайта.

Контрольные карты Шухарта — визуальный инструмент, применяемый в управлении производством и бизнес-процессами, график изменения параметров процесса во времени для осуществления статистического контроля стабильности процесса.

Метод NPS — оценка удовлетворенности и лояльности клиента с использованием индекса NPS. Это индекс определения приверженности потребителей товару или компании/бренду (индекс готовности рекомендовать), который обычно используется для оценки готовности к повторным покупкам.

Шина обмена данными — элемент IT-архитектуры, который отвечает за интеграцию разрозненных информационных систем в единый программный комплекс с централизованным управлением передачей информации.

Code Review — рецензирование кода, обзор кода, ревизия кода или инспекция кода. Процесс систематической проверки исходного кода, чтобы найти и исправить баги, которые могли остаться незамеченными в начальной фазе разработки.

Рефакторинг — перепроектирование или переработка кода, изменение его структуры, которые не производят функциональных изменений, но существенно облегчают его работу.

JQuery — популярная библиотека JavaScript, языка программирования.

Vue.js — прогрессивный фреймворк для создания пользовательских интерфейсов с открытым исходным кодом, который легко интегрируется в проекты с использованием других JavaScript-библиотек.

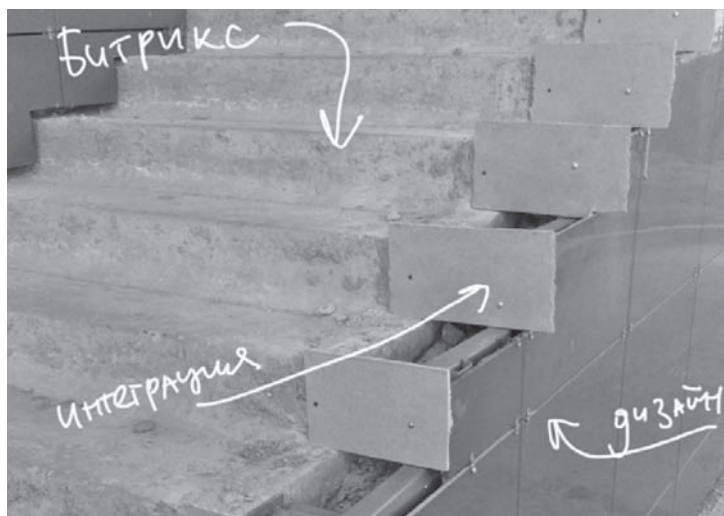
Code freeze — замораживание в разработке на срок, при достижении которого существенно ужесточаются правила внесения изменений в исходный код.

Часть 10



ИНТЕГРАЦИЯ. РИСКИ И НЕМНОГО РЕАЛЬНОСТИ

Интеграция — это объединение разнородных приложений и систем в единую среду. Нас в первую очередь будет интересовать веб-среда для транспортировки данных. Редкий проект сейчас обходится совсем уж без интеграции систем. Большинство популярных CMS-систем имеют штатные возможности интеграции с популярными ERP, CRM, платежными шлюзами, sms-гейтами и сервисами доставки. Однако, как только какая-либо из систем будет серьезно кастомизирована, интеграцию также нужно будет кастомизировать или организовывать заново. Кроме того, в мире полно экзотических систем. И с каждой из них придется разбираться отдельно.



Чаще всего, интеграция — это довольно просто.

1. Берем, например, сайт или мобильное приложение. Внимательно смотрим на него, страница за страницей. Для этого хватит прототипов. И для каждого элемента выясняем, откуда это на странице. Как это сделать, мы уже разбирали в главе о декомпозиции.
2. Выясняем, есть ли эти данные в донорской системе. Смотрим, в каком виде. Выясняем, пригодны ли они к использованию в целевой системе. Если да — все в порядке. Если нет — нужно будет доработать донорскую систему или даже вбивать информацию. Бывает, что это очень большая ручная работа.
3. Описываем точки контакта двух систем. Кто, от кого, чего ждет? Когда? В каком виде? Какой транспорт используется? Это называется «интеграционный протокол». Снабжаем это дело примерами и тестовыми сценариями (mock-объекты). Так, чтобы можно было вести разработку без необходимости иметь обе системы под рукой и ждать друг друга.
4. Реализуем протокол со стороны обеих систем.
5. Включаем рубильник.
6. Если все сделали правильно — все заработает.
7. На самом деле, так не бывает. Потребуется время на доводку, отладку, исправление ошибок. Отработку сценариев, которые невозможно было предусмотреть заранее.
8. Если интеграция двусторонняя (например, в одну сторону летят товары, в другую — заказы), делаем все то же самое, но в обратную сторону. Можно проверить оба контура одновременно.

Заметьте, в этой схеме абсолютно неважно, что за системы мы интегрируем: 1С, SAP, Ахapta, АМО, Битрикс. Непринципиален и транспорт: ведем ли мы обмен на файлах, веб-хуках, REST API, на основе JSON, используем ли шины данных, бинарный gRPC или передаем данные почтовыми голубями. Главное, чтобы совпали входы и выходы.

Здесь и далее, для простоты текста, за донорскую систему возьмем 1С. За целевую — сайт. Хотя, повторяюсь, нет никакой принципиальной разницы, что там на обоих концах.

Почему это сложно? Дьявол, как обычно, в деталях. И коммуникациях. Бога там точно нет. Но давайте разбираться.

10.1 ПОЧЕМУ НЕ ХВАТАЕТ ШТАТНЫХ ИНТЕГРАЦИЙ?

Фэйлом кончаются от 30% до 50% попыток внедрить штатную интеграцию сайта с 1С. Это коллеги рассказали. То есть в трех случаях из четырех придется что-то подкручивать напильником, а в одном — вообще вызывать эвакуатор или реанимацию. И чего бы это, ведь...

...Топовые производители современных отечественных систем управления контентом в один голос заявляют, что умеют интегрироваться с 1С. Самая красивая интеграция, естественно, у 1С Битрикс. Нужно сделать пару настроек, и товары полетят в Битрикс, а заявки — обратно, в 1С-ку. Когда она так работает — я ее люблю.

С другими системами дела ненамного лучше.

ERP-системы — они же Enterprise Resource Planning, планирование ресурсов предприятия, — часто уже были дописаны какими-то программистами, а штатные протоколы не в состоянии обеспечить все хотелки бизнес-клиента.

Что же может пойти не так? Ответ: все. И самая главная проблема отнюдь не техническая. А вполне себе человеческая.

10.2 ЧЕЛОВЕЧЕСКИЙ ФАКТОР

В реальности сценарий продажи и внедрения интеграции может превратиться в АД!:

Фаза пресейла

Менеджер проекта: Нужна ли будет в Вашем интернет-магазине интеграция с 1С?

Клиент: Да, посчитайте, пожалуйста, может понадобится.

Менеджер проекта: А какая версия 1С у Вас используется?

Клиент: Я не знаю, там же все просто, из коробки, да?

Менеджер проекта: Штатная интеграция не во всех случаях может Вам подойти. Давайте либо оставим вопрос интеграции на последующие этапы, либо, для более точной оценки, нам потребуется более детальная информация по Вашим настройкам 1С. Есть ли у Вас специалист, который занимается 1С?

Клиент: Да, приходит мальчик, что-то дописывает... Какая разница? Давайте сейчас без этих подробностей. Мне нужна примерная цена.

Менеджер проекта: Ок, штатный вариант интеграции будет стоить N долларов, нетиповая интеграция — порядка МММ долларов, но возможно и более.

Клиент (мимо трубки): X#&@се...

Анализ диалога:

Менеджер — уже тертый калач и задает правильные вопросы. Но на фазе пресейла такие детали, как правило, неизвестны заказчику, а выяснять их долго и мучительно. Проще выбрать того подрядчика, который не станет задавать таких вопросов. Попытка менеджера проекта перенести интеграцию на дальнейшие этапы способствует тому, что фаза пресейла пройдет более гладко.

Но на практике это может привести к большим проблемам уже после запуска системы, когда выяснится, что структура каталога товаров в 1С и его структура на сайте — принципиально разные. Информация о том, что 1С кто-то дописывает — очень тревожный сигнал, равно как и то, что клиент планирует менять версию 1С.

Как правило, это гарантирует, что штатной интеграции не хватит.

Фаза утверждения технического задания

Менеджер проекта: Давайте согласуем структуру вашего каталога. Какие разделы ориентировочно там будут?

Клиент: <такие-то>

Менеджер проекта: Это соответствует структуре базы вашей 1С?

Клиент: Нет, но на сайте у нас будет другая структура, не такая, как в 1С.

Менеджер проекта: Нужно ли будет обновлять цены на товары из 1С?

Клиент: Да.

Менеджер проекта: Тогда, скорее всего, штатной интеграции не хватит. Можете ли **прислать вашу текущую выгрузку из 1С?**

Клиент: Нет, я без понятия.

Менеджер проекта: Ок, а можете ли **предоставить временный удаленный доступ к 1С?**

Клиент: Нет, это запрещено корпоративными правилами, и администратор не даст доступ.

Менеджер проекта: Для правильной организации структуры каталога нам нужно посмотреть вашу выгрузку и текущее состояние информации.

Клиент: Наш программист говорит, что сделает вам выгрузку в таком формате, как вам будет надо. Пришлите нам описание, что именно вам от нас нужно.

Менеджер проекта: В идеале нам необходима выгрузка из 1С в формате CommerceML 2, причем еще до начала работ по программированию.

Клиент: Ок, я передам нашему программисту.

Анализ диалога:

*Непредоставление выгрузки или доступа к 1С, к сожалению, — частая проблема. На этом же этапе работ я бы порекомендовал начинать взаимодействие с программистом заказчика (вполне возможны разного рода неожиданности). Порой это самый дешевый и реальный вариант. Договоритесь с заказчиком, чтобы он организовал вам звонок с программистом. Добейтесь, чтобы программист либо дал вам доступ к выгрузке, либо 100% согласился обеспечить выгрузку и настройки под ваши требования. **Зафиксируйте договоренности письменно.***

Этап разработки

Программисты студии: Нам нужна выгрузка, чтобы правильно сделать структуру инфоблоков.

Менеджер проекта: Ок, запрошу у клиента.

Клиент: Мы, вроде бы, обо всем договорились? Выгрузка будет позднее, наш программист вам все сделает, скажите только, что конкретно надо.

Менеджер проекта (программистам): Ребят, выгрузку сделают такую, как вам надо. Напишите спецификацию, что вам сделать...

Программисты студии: Нам бы стандартную...

Менеджер проекта: Сделайте пример, как вам надо, мы отправим ее заказчику.

Программисты студии: ОК. Тогда мы делаем структуру каталога на основе ТЗ и отправляем ее клиенту, верно?

Менеджер проекта: Да.

Менеджер проекта (клиенту): Вот наша спецификация с примером. Пожалуйста, подтвердите, что все ОК.

Клиент: Да, все ок, не волнуйтесь.

Анализ диалога:

Практически гарантированный фэйл. Программистам нужно спроектировать структуру каталога, но, если выгрузка из базы будет принципиально другой, — это нужно учесть заранее. Пожалуй, тут еще все можно было бы и спасти, но...

Этап сдачи проекта

Менеджер проекта: Готово, вот ваши разделы каталога. Теперь необходимо его импортировать из вашей базы в том формате, который мы запрашивали, и у вас будет заполненный каталог.

Клиент: В каком формате?

Менеджер проекта: Я высылал вам пример, такого-то числа...

Клиент: Пришлите еще раз, пожалуйста. И поставьте в копию нашего программиста.

Анализ ситуации:

*Ну все, фэйл случился. Теперь проджект-менеджер будет пытаться «рулить» удаленным программистом, **который не стоит у него в подчинении и которого он не нанимал**. Все зависит от того, какое чувство собственной важности у программиста на стороне клиента... И не дай бог оно будет >9 000:-)*

Фаза приемки проекта

Программист на стороне клиента (ЧСВ9000+): Я не могу так сделать, потому что у нас нетиповая конфигурация.

Менеджер проекта: Да, но вы говорили, что сможете обеспечить поддержку формата.

ЧСВ9000+: Я же уже объяснил, у нас — НЕТИПОВАЯ!!! конфигурация. Вы должны переделать на своей стороне!!!

Менеджер проекта: В каком формате вы можете сейчас сделать выгрузку?

ЧСВ9000+: В каком вам надо?

Менеджер проекта: Мы скидывали вам пример выгрузки уже дважды. Нам нужен формат CommerceML, с присланной нами структурой инфоблоков.

ЧСВ9000+: Нет! Так нельзя сделать! Этот формат нам не годится!

Менеджер проекта (клиенту): Ваш программист говорит, что не может сделать выгрузку в требуемом формате.

Клиент: Нет, **это вы не можете предоставить ему нормальные требования к формату.** Вы срываете нам сроки контракта!

Менеджер проекта: Давайте соберем звонок на троих и обсудим этот вопрос.

Далее события развиваются по любопытному сценарию:

Менеджер проекта: Добрый день. Цель нашего сегодняшнего разговора — прийти к однозначному формату выгрузки. **Мы настаиваем на нашем формате, CommerceML 2, так как он является стандартом отрасли,** де-факто, хорошо документирован, и вся работа по его интеграции уже выполнена.

ЧСВ9000+: Мы не можем использовать ваш формат, потому что он не соответствует представлению наших данных в 1С. Кроме того, Commerce ML — это проприетарная разработка, предназначенная для обмена коммерческой информацией с топовыми продуктами 1С. В нашем случае идет речь о кастомной интеграции сайта, который вы обязаны нам сделать, с уникальной и **заточенной под нас** — базой 1С. Поэтому ваш формат не годится!

Менеджер проекта: В каком формате вы сможете сделать выгрузку?

ЧСВ9000+: В каком вам надо?

Менеджер проекта: Клиент, вот смотри — мы ходим по кругу.

Клиент: Я не понимаю технических деталей. **Вы можете общаться между собой?**

Менеджер проекта: Пусть пришлет хоть что-то. Мы посмотрим, насколько это удобоваримо...

Клиент (своему программисту): ЧСВ9000+, сможем им сделать?

ЧСВ9000+: Я не понимаю, что им от меня надо. Через неделю что-нибудь сделаю.

Анализ диалога:

Если давить на программиста заказчика — можно выяснить, что он либо не разобрался в спецификации, либо пробовал, но у него не получилось, либо уже нагородил какого-то своего говнокода, или какой-то свой «универсальный» формат, от которого теперь не хочет отступать. Особенная жесть начинается, если ваш клиент платит сво-

ему 1С-нику почасовую ставку, и 1С-ник утверждает, что работа с его стороны займет неделю из-за ваших «необоснованных» требований (или уникальности «нашей базы»).

Через неделю

ЧСВ9000+: Формат будет — три файла CSV: один с каталогом, второй со структурой разделов, третий с остатками на складе. Пока готовы только два файла (прилагаю). Картинки будем загружать в специальную папку по номерам артикулов, транслитом.

Программисты студии: У вас все характеристики товаров хранятся в одном поле. Фильтр не будет работать...

Клиент: Это не наша проблема. Фильтр должен работать!

Менеджер проекта: Проблема на вашей стороне. У вас в базе 1С. Нужно все товары вбить заново, в любом случае.

ЧСВ9000+: Это не моя проблема. Это база.

Программисты студии: Мля...

Менеджер проекта: Мля...

Клиент: Мля...

Анализ диалога:

Для краткости показаны только самые сильные ходы ПМ-а. На самом деле можно протрахаться значительно дольше. Вырулить можно, но о попадании в бюджет и сроки уже речи не идет. Виноват — клиент (в договоре формально закреплено, что **выгрузка будет предоставлена в требуемом формате**), но это неважно, поскольку цель менеджера проекта — запустить проект, а не доказать «виноватость».

Главное, не вестись на разговоры вроде **«вы же профессионалы, должны были предусмотреть»**. Вы предусмотрели и решили продолжить работы, имея открытый риск. **А риск, увы, — сработал.**

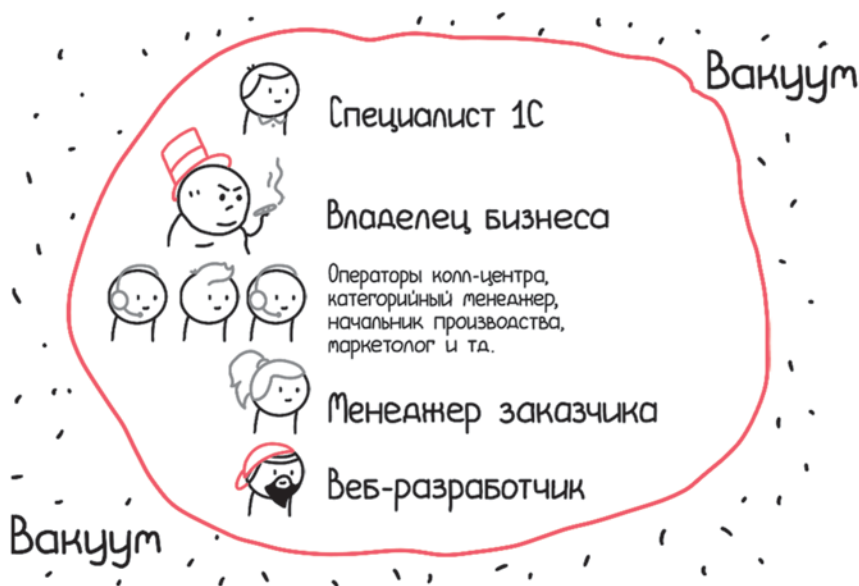
Лечится, как правило, **увеличением цены** на два-три-четыре человеко-дня со стороны программистов студии, и еще часов 8–16 нервных переговоров со стороны менеджера проекта и клиента. Собственно, отсюда и разница в цене — N долларов за штатную интеграцию, МММ долларов — за нештатную. Нервные клетки не восстанавливаются.

Итого, примерно такой расклад по основным рискам:

Риск/ситуация	Последствия	Противодействие
Выгрузка запрошена на этапе пресейла.	— Клиент поищет кого-то попроще. — Слишком много времени уйдет на пресейл, а проект — сорвется.	— Вынести интеграцию на отдельный этап. — Дать «вилку» на лучший и худший случаи. — Сообщить заказчику о возможных рисках.
Выгрузка не предоставлена на этапе составления ТЗ.	— Неправильно спроектирована структура каталога. — Срыв сроков и бюджета.	— Настаивать на предоставлении выгрузки. — Вынести интеграцию в отдельный этап. — Письменно сообщить заказчику о возможных рисках.
Интеграция вынесена в отдельный этап.	— Придется переделывать всю структуру каталога.	— Получить выгрузку до начала проектирования.
1С была модифицирована сторонним разработчиком, имеет устаревшую версию или плохо структурированный каталог.	— Невозможность выполнить штатную интеграцию. — Длительные переговоры с программистом заказчика, потеря времени.	— Настаивать на соблюдении подписанных требований. — Выполнить настройку выгрузки на стороне клиента своими силами.
Выполнение настроек 1С на стороне клиента своими силами.	— Непрогнозируемая трудоемкость и возможные сложности с нетиповой конфигурацией. Риск «закопаться» в проект.	— Настаивать на соблюдении подписанных требований к выгрузке.

Риск/ситуация	Последствия	Противодействие
	— Риск получить в нагрузку к сопровождению сайта — бесплатные консультации по 1С или попасть на исправление каких-то глюков в 1С, которых «не было до вас».	— Поручить настройку 1С надежному третьему лицу (к которому в случае чего будут все претензии). Кандидатуру согласовать с заказчиком.
Студия настаивает на соблюдении протокола.	— Риск разрыва отношений по причине отсутствия возможности у клиента реализовать требования самостоятельно. — Затягивание сроков сдачи проекта.	— Вынести интеграцию с 1С на отдельную фазу. — Выполнить настройки 1С самостоятельно. — Принять данные в том формате, в котором их способен предоставить клиент.
Студия прогнолась и согласилась изменить требования протокола под любой формат.	— Переделка структуры каталога, трудоемкое программирование по интеграции «за бесплатно», сорванные сроки.	— Закладывать на интеграцию очень много денег. — Запомнить полученный опыт и более не попадаться.
Программист на стороне заказчика — неуправляем.	— Длительные, тяжелые переговоры. — Срыв сроков.	— Организовать ежедневные трехсторонние созвоны с заказчиком, его программистом и студией. Решить проблему на более высоком уровне (эскалировать).

10.3 РАБОЧАЯ ГРУППА



Есть студия или агентство (разработчик внешней системы, сайта или приложения) — люди там как-то организованы, часто есть ответственный менеджер, за которым прячется его техническая команда. Хотя бывает, что и не прячется.

Есть заказчик. Большой босс, который хочет либо что-то заработать, либо сократить издержки. Заказчиком может выступать коммерческий отдел или отдел маркетинга — всегда по-разному и зависит от того, кому проект более выгоден.

Есть специалист или команда по ERP-системе, которые знают инфраструктуру. И лениво что-то там допиливают при необходимости. Всех несогласных шлют куда подальше. И да, они могут быть на аутсорсе, со своим графиком и сроками.

На стороне заказчика есть digital-продюсер, которому формально поручили управлять проектом, но реальной власти у него может и не быть (наихудший вариант). Хорошо, когда он может любую дверь с ноги открыть, имеет поддержку босса и, может, даже мафии.

А еще в цепочке взаимодействий могут быть всякие контентщики, категорийные менеджеры, армия маркетологов, сеошники, и прочие люди, которым с этим потом работать. И у всех есть свои ограничения.

Если посадить технических специалистов из разных компаний рядом, далеко не факт, что они договорятся.

Классически процесс построен так, что менеджер уполномочен принимать решения и организовывать ресурсы для выполнения задачи. В его голове единая картина по проекту, он отвечает за задачи и цели, за бюджет, за сроки. Хорошо, когда такой же уполномоченный человек есть и на стороне компании заказчика (иногда, в небольших компаниях или при очень высокой щепетильности, это даже сам владелец бизнеса): так проще договориться.

Но если такого человека нет — перед стартом работ по интеграции важно выяснить, кто будет принимать решения и отвечать за их реализацию, например, если потребуется доработка на стороне ERP. Идеально, когда это один человек, готовый в споре среди своих сказать: «Мы сделаем так, и точка».

10.4 ОРГАНИЗУЙТЕ ИТЕРАЦИИ ОБСУЖДЕНИЙ

Собирайте рабочую группу минимум два раза в неделю. Еще продуктивнее — раз в день. Вам будут нужны ответственные лица со стороны обеих интегрируемых систем, технические специалисты и представитель бизнес-заказчика, который сможет арбитрировать вопросы бизнес-логики.

Обсуждайте всего три вопроса:

1. Что было сделано вчера?
2. Какие планы на сегодня?
3. Какие есть проблемы?

Это проверенный рабочий Scrum-стендап, позволяющий сделать работу на 100%. Собрались, простендапились, разбежались. Сделали, согласовали, обозначили реальные сроки готовности к вводу в эксплуатацию и пошли допиливать все необходимое.

10.5 ФОРМАТ ДАННЫХ

Мы тут все про 1С, потому что это самый частый случай — ее, к тому же, иногда можно интегрировать с сайтом штатными средствами. Однако возможности гораздо шире. Веб-сервис можно интегрировать с ERP-системой любого плана. ERP подразумевает целое множество

самых разных программных пакетов. От дорогущих решений Oracle и Microsoft до условно бесплатной CRM-ки.

В зависимости от того, что и с чем мы хотим интегрировать, есть несколько сценариев обмена данными:

- ▶ текстовый файл/CSV;
- ▶ CommerceML;
- ▶ XML/JSON;
- ▶ промежуточная база данных (MySQL/MS SQL);
- ▶ web-сервисы (SOAP/REST);
- ▶ NoSQL-решения;
- ▶ Шины данных (вроде Apache Kafka) и микросервисные архитектуры.

10.5.1 CSV

Формат таблиц, содержащий данные в виде текста с символами-разделителями (например, точка с запятой) на ячейки и строки. Из него можно быстро импортировать данные в базу данных сайта, например, контакты клиентов или адреса филиалов. Этот файл формируется и складывается в определенное место, откуда сайт его регулярно забирает и загружает в соответствующую таблицу уже в своей базе данных.

	A	B	C
1	Имя	Фамилия	Телефон
2	Лена	Иванова	71234567890
3	Ирина	Смирнова	71234567891
4	Иван	Иванов	71234567892
5	Петр	Сидоров	71234567893
6	Владимир	Ленский	71234567894
7	Соня	Орлова	71234567895
8			
9			
10			
11			
12			
13			

В этом случае сторонам необходимо договориться, какие столбцы CSV соответствуют столбцам базы данных, и как они должны обрабатываться сайтом.

10.5.2 COMMERCEML

Стандартизированный подвид **XML**. Также используется для обмена данными в связке «сайт — ERP». Очень схож с тем, как работает предыдущий вариант: данные из реквизитов файла XML забираются сайтом в его базу данных, и наоборот. Это самый частый формат обмена с системой 1С, особенно для интернет-магазинов. Стандартом предусматривается использование схем XML, в частности, для обмена:

- ▶ каталогами товаров в системах управления каталогом;
- ▶ коммерческими предложениями (заказами);
- ▶ документами.

```
<?xml version="1.0" encoding="WINDOWS-1251"?>
- <КоммерческаяИнформация ФорматКоличества="ЧЦ=18; ЧДЦ=2; ЧРД=-;" ФорматСуммы="ЧЦ=18; ЧДЦ=2; ЧРД=-;" РазделительДатаВремя="Т" ФорматВремя="ДФ=ЧЧ:мм:сс; ДЛФ=Т" ФорматДата="ДФ=yyyy-MM-dd; ДЛФ=DD" ДатаФормирования="2017-06-19T10:50:36" ВерсияСхемы="2.05">
  - <Документ>
    <Ид>607</Ид>
    <Номер>...</Номер>
    <Дата>2017-06-19</Дата>
    <ХозОперация>Заказ товара</ХозОперация>
    <Роль>Продавец</Роль>
    <Валюта>руб</Валюта>
    <Курс>1</Курс>
    <Сумма>44990.0000</Сумма>
    - <Контрагент>
      - <Контрагент>
        <Ид>...</Ид>
        <Наименование>Екатерина</Наименование>
        <ПолноеНаименование>Екатерина</ПолноеНаименование>
        <Имя>Екатерина</Имя>
        + <АдресРегистрации>
        + <Контакты>
        - <Представитель>
          - <Контрагент>
            <Отношение>Контактное лицо</Отношение>
            <Ид>3d432e5d46b99fba6bb0322c5b6945a3</Ид>
            <Наименование>Екатерина</Наименование>
          </Контрагент>
          </Представитель>
          <Роль>Покупатель</Роль>
        </Контрагент>
      </Контрагент>
    <Время>01:01:15</Время>
    <Комментарий/>
  - <Товар>
    <Ид>379873d1-de0a-11e4-b4ba-005056ac3408#379873d1-de0a-11e4-b4ba-005056ac3408</Ид>
    <ИдКаталога>1c_catalog-c3505cf6-707a-41c3-b52c-cb62e30e0c4e#</ИдКаталога>
    <Наименование>ИНТЕР Диван с механизмом еврокишки</Наименование>
    <БазоваяЕдиница>МеждународноеСокращение="РСБ" НаименованиеПолное="Штука" Код="796">шт</БазоваяЕдиница>
    <ЦенаЗаЕдиницу>44990.0000</ЦенаЗаЕдиницу>
    <Количество>1.0000</Количество>
    <Сумма>44990</Сумма>
    - <ЗначенияРеквизитов>
      - <ЗначениеРеквизита>
        <Наименование>ВипНоменклатуры</Наименование>
```

Современные движки для сайтов и сама 1С обычно имеют встроенные модули обмена в таком формате. Обмен данными абсолютно под все задачи сайта не всегда получается решить штатно, но эти готовые решения сильно уменьшают объем кода, который пишут и проверяют разработчики с обеих сторон баррикад. Например, не надо писать скрипты старта импорта или экспорта, обработчики полученных паке-

тов из 1С: все это уже есть в готовом решении, но с определенными ограничениями. И, к сожалению, иногда довольно суровыми.

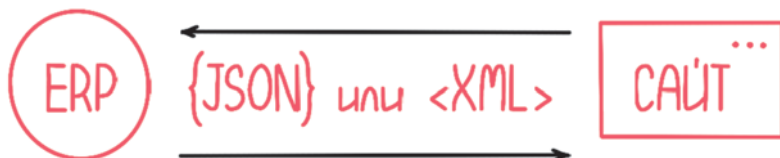
Чем хороша штатная интеграция:

- ▶ использует легкочитаемый стандарт CommerceML;
- ▶ дает серьезную прибавку к скорости разработки;
- ▶ позволяет экономить бюджет;
- ▶ позволяет изменять базовые настройки на уровне администратора, а не разработчика;
- ▶ дружит с нативными возможностями Битрикса, которые критически важны;
- ▶ содержит готовые скрипты для интернет-магазина: смарт-фильтр, каталог, заказы, кэширование.

10.5.3 XML/JSON

Сайт шлет в ERP запрос статуса заказа — по номеру и идентификационному паролю. В ответ получает кусок XML-кода, который содержит нужный массив данных. Далее он его обрабатывает и выводит пользователю на экран в виде, допустим, таблицы. Форматы XML и JSON, по сути, решают одну и ту же задачу, но JSON — более легковесный.

Из плюсов такой интеграции: в отличие от штатных обменов пакетами импорта здесь запрос-ответ могут происходить в режиме реального времени. Запросил — получил ответ — передал пользователю.



10.5.4 ПРОМЕЖУТОЧНАЯ SQL-БАЗА

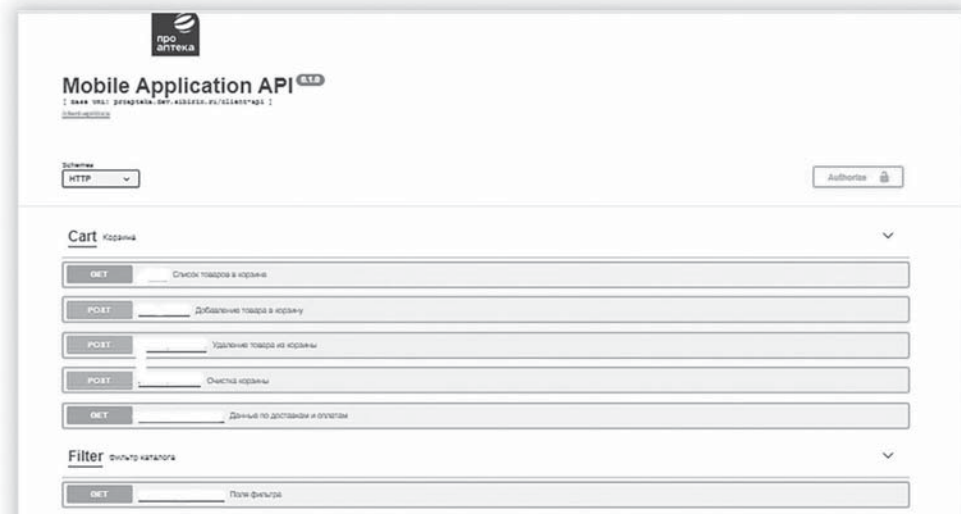
Редко, но бывает. Когда системы слишком разнородны, имеет смысл интегрироваться через промежуточное звено, где хранятся только релевантные для обмена данные. Я не вижу каких-то принципиальных плюсов в такой схеме по сравнению с альтернативными методами, кроме структуризации данных. Лучше посмотреть в сторону **NoSQL**-решений или общей шины данных вроде Apache Kafka.

10.5.5 ВЕБ-СЕРВИСЫ (SOAP/REST)

Частая история с внешними сервисами доставки, оплаты, расчета каких-то данных. Массивы данных по формализованному API шлются в одну сторону и получают от нее соответствующий ответ. Идеально подходят для быстрого запроса и получения информации «на лету». В случае протокола SOAP обмен идет в XML-формате. REST, как правило, — в JSON.



Есть отличная система Swagger — фреймворк для проектирования, создания, документирования и использования веб-сервисов RESTful. В итоге формируется единый документ API, который можно передать другому разработчику, и тот разберется за несколько минут.



The screenshot displays a mobile application interface with a list of expandable sections. Each section has a title, a subtitle, and a list of actions with 'GET' or 'POST' methods.

- Products** (Товары каталога)
 - GET: Получить список товаров
 - GET: Детали товара
 - GET: Получить список промо товаров
- Section** (Разделы каталога)
 - GET: Список разделов
- Delivery** (Работы доставки)
 - GET: Расчет стоимости доставки
- Feedback** (Обратная связь)
 - POST: Обратная связь
- Location** (Местоположения)
 - GET: Определение города по GPS
 - GET: Список всех доступных городов
- Notification** (Уведомления в приложении)
 - GET: Получить историю уведомлений
 - GET: Получить количество непрочитанных сообщений

Более того, многие мобильные приложения обмениваются со своей backend-системой именно в такой форме. Поэтому, скорее всего, с этими форматами вы столкнетесь.

10.5.6 БИНАРНЫЕ ПРОТОКОЛЫ (GRPC)

По тому же принципу можно организовать обмен в бинарном виде, по протоколам GRPC/GRPC-WEB. Они более формальные и строгие в отличие от бесконечно гибкого JSON, а значит — уменьшается риск допустить ошибку. Хорошо работают для организации взаимодействия мобильных приложений и бэкэнда. Данные передаются гораздо компактнее, чем в формате JSON. Это дает скорость. В web на данный момент требуется дополнительная прослойка, так как браузеры напрямую не поддерживают GRPC.

Из минусов: строгость протокола крадет часть гибкости. Протокол сложнее расширять. Кроме того, в нем есть небольшая неудобная штука, со значениями по умолчанию. Но это уже головная боль программистов.

10.5.7 NOSQL-РЕШЕНИЯ

Сервисы, вроде CouchDB, Redis, Apache HBASE и других. В NoSQL-базах структура данных не делится на строгие типы, поэтому, если нужно поменять модель данных, достаточно просто отразить изменение в коде приложения. Отличаются большой мощностью хранилища за счет объединения нескольких серверов быстрой сетью, где каждый сервер обрабатывает только свою часть данных.

10.5.8 ШИНЫ ОБМЕНА СООБЩЕНИЯМИ И МИКРОСЕРВИСЫ

Шины обмена сообщениями, вроде Apache Kafka, Active MQ, Rabbit MQ, Axelot, служат для гарантии транспортировки сообщений от одного адресата к другому. Кроме маршрутизации сообщений могут также менять сами сообщения, но это считается дурным тоном в программировании.

Применяются чаще всего в микросерверных архитектурах. Когда большой проект разбивается на подпроекты, как правило, с собственными базами данных и даже, возможно, написанными на разных языках программирования.

Например, сайты вроде Amazon.

Другой пример: SingularityApp — его серверная часть построена именно по микросервисной архитектуре. Есть сервис, отвечающий за синхронизацию пользовательских данных с облаками. Есть — отвечающий за проведение платежей. Есть сервис, отвечающий за интеграцию календарей. И так далее. Всего более дюжины подсистем. Если по каким-то причинам откажет один из сервисов — остальные могут продолжить свою работу. Однако это не 100%: сервисы обмениваются сообщениями между собой. И чтобы в случае падения сервиса не потерялись сообщения, которые ему были отправлены, как раз используются серверы очередей.

Хотя такая архитектура позволяет делать один проект несколькими независимыми командами, обеспечивает отказоустойчивость и масштабирование, тут очень легко нагородить сверхсложную, тормозную, зациклившуюся архитектуру. Получить ад микросервисов. Если вы еще не имеете хотя бы года опыта управления интеграционными проектами,

настоятельно рекомендую воздержаться от попытки построения таких систем. Либо вам нужен толковый и опытный архитектор, который интеграциями занимался еще тогда, когда вам интернет не провели.

Ту же 1С с сайтом можно интегрировать любым из перечисленных способов. Все зависит от сложности структуры данных, протоколов их обработки, стоящих перед разработчиком задач, и, конечно же, бюджетов.

10.6 ИНТЕГРАЦИОННЫЙ ПРОТОКОЛ

Итак. Если упрощать все до безобразия, работа по интеграции делится на четыре этапа:

1. Бриф.
2. Протокол интеграции.
3. Непосредственно разработка.
4. Ввод в эксплуатацию.

Делается это на разных стадиях относительно общего таймлайна проекта. Например, протокол, как показывает практика, хорошо делать после прототипирования, край — дизайна.

Чего не рекомендуется делать — так это внедрять на стороне клиента несколько систем одновременно. Например, внедрять за один подход ERP, CRM, сразу интегрировать с ними сайт и тут же привязывать к этому всему мобильное приложение. Утонете. Начните с одной, базовой, системы. Например, приведите ERP в порядок. Затем, по одной, подключайте остальные.

10.6.1 БРИФЫ НА ИНТЕГРАЦИЮ

Чтобы понять и вилочно оценить фронт работ, у нас с годами сформировался очень подробный бриф на интеграцию. Еще на этапе продажи просим его заполнить, указать основные требования и вводные, предоставить пример выгрузки (например, товаров).

Без этих данных невозможно дать информацию по стоимости и срокам интеграции — вилка получается такая, что никакой конкретики: от 8 до 80 часов на описание протокола и от 40 до 200 часов на реализацию интеграции.

Выглядит это примерно так:

Вопрос	Ответ
Какая будет использоваться складская система?	Управление торговлей 11
Если 1С	
Версия платформы 1С	8.3
Конфигурация 1С. <u>Дорабатывалась</u> или базовая	Конфигурация нетиповая
Есть ли специалист в штате компании, который занимается/может заниматься доработкой конфигурации?	Да
Структура каталога в 1С будет отличаться от структуры на сайте?	Да
Все ли описания товара (включая пути к файлам фото товаров) заполнены в 1С	Да
Как хранятся свойства товаров в 1С? Справочниками или строкой в описании?	Типы значений могут быть различны. В 80% случаев это справочники
Есть ли возможность организовать взаимодействие с 1С через веб-сервисы?	Да
Есть ли возможность предоставить выгрузку товаров из каталога в формате .xml или .csv до начала составления технического задания?	Да
Общие вопросы	
Как хранятся фотографии товара в 1С, как планируется их выгрузка на сайт? По какому критерию определяется основная фотография товара?	Выгрузку можем предоставить в двух вариантах: в двоичном виде (тип <u>хранилище значения</u>) в общем файле выгрузки, в виде отдельных файлов с привязкой к товару в основном файле выгрузки. Либо в случае работы через веб-сервис по согласованному протоколу обмена.
Есть ли база описаний и фотографий товаров? Какое там количество артикулов?	Базу можно сформировать при необходимости. Количество артикулов необходимо уточнить.

Но иногда бывает и так:

Вопрос	Ответ
Какая будет использоваться складская система?	1С
Если 1С	
Версия платформы 1С	не могу сказать, хотим поменять
Конфигурация 1С. <u>Дорабатывалась</u> или базовая	соответствует всем нашим потребностям
Есть ли специалист в штате компании, который занимается/может заниматься доработкой конфигурации?	Мой друг Дима, приходит, когда надо
Структура каталога в 1С будет отличаться от структуры на сайте?	Наверное нет
Все ли описания товара (включая пути к файлам фото товаров) заполнены в 1С	Почти все
Как хранятся свойства товаров в 1С? Справочниками или строкой в описании?	Выпадающие списки
Есть ли возможность организовать взаимодействие с 1С через веб-сервисы?	Не знаю, что это
Есть ли возможность предоставить выгрузку товаров из каталога в формате .xml или .csv до начала составления технического задания?	Отправил вам эксель файл с номенклатурой
Общие вопросы	
Как хранятся фотографии товара в 1С, как планируется их выгрузка на сайт? По какому критерию определяется основная фотография товара?	Не понятно. В 1С нет фотографий, они там не нужны
Есть ли база описаний и фотографий товаров? Какое там количество артикулов?	где-то 100 товаров, еще добавляем
Есть ли сотрудники для наполнения описаний товаров?	если Дима придет, он заполнит
Какова ценовая политика? Это несколько типов цен или система скидок? От чего зависит?	Не решили еще
Есть ли «Товарные предложения» (SKU) - товар один, но несколько размеров/цветов. Отличаются ли они при этом по цене?	Что это???

По итогам брифа уже можно представить, какие сущности откуда берутся, определить возможный объем работы и выдать ценник на выполнение данных работ.

Не стесняйтесь задавать вопросы на этом этапе. Смотрите глазами, что происходит в системах на самом деле. Смотрите выгрузки. Пару раз посмотрите в CommerceML-файлики, и они перестанут вас пугать.

10.6.2 ПРОТОКОЛ

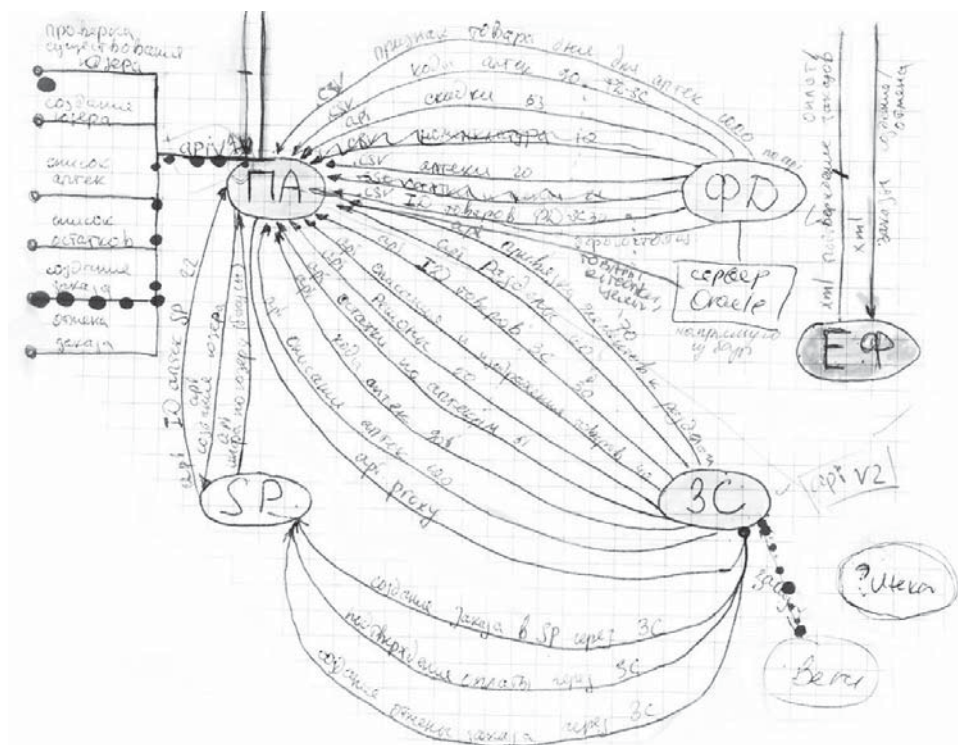
Чтобы выяснить, какие действия нужны от ERP-программистов и web-интеграторов, нужен протокол. Аккуратный, легко читаемый, четкий. Чтобы каждый мог взять и понять, откуда берутся данные для вывода любой буквы.

Бывает так, что даже для формирования одной страницы на сайте приходится консолидировать данные из нескольких подсистем.



На одной странице интегрированы данные из четырех внутренних систем.

Иногда, прежде чем писать формальное описание протокола, приходится изрисовать десяток салфеток. Это тоже нормально.



Салфетка с интеграционным протоколом в самом начале.

Протокол интеграции отвечает на ключевые вопросы: какие сущности и какие поля интегрируем, в каком направлении, с какой периодичностью и в каком формате. Также в нем прописывается, что является первичным: сайт, ERP или какая часть данных берется на сайте, а какая — в ERP.

Итак. Что должно быть в протоколе:

1. Транспорт и формат обмена. XML, JSON и т. д.
2. Точки входа. Кто и куда стучится? Как часто? По расписанию или по событию?
3. Для каждой точки входа описание формата передаваемых и возвращаемых данных:

- a. Перечень сущностей (например, товары, заказы, пользователи, скидки).
 - b. Для каждой сущности — перечень передаваемых полей и их формат (даты новостей, цены, название товара и т. д.). Имейте в виду, что сущности могут быть вложены друг в друга, как матрешки.
 - c. Перечень обрабатываемых ошибок. Коды ошибок и тексты сообщений.
 - d. Передача файлов (если таковая предполагается). Передаются ли сами файлы, или только ссылки на них? Должны ли вы их скачивать? Или можно использовать предоставленные ссылки?
4. Примеры использования. Недостаточно просто описать формат словами. Приложите тестовые примеры, опираясь на которые можно будет сделать приемочные тесты.

По готовности протокола у вас не должно остаться неведомых ссылок в никуда на странице, скажем, карточки товара. Каждый функциональный элемент имеет происхождение и задачу, которая с ним связана. Например, вес товара и объем, который должен использоваться в алгоритме расчета доставки. Объем мы считаем из трех параметров упаковки товара, которые должны импортироваться на сайт.

Здесь важно помнить, что даже одна маленькая ссылочка, пропущенная на данном этапе, может обернуться для вас большими проблемами на этапе программирования. Когда вы поймете, что только ради нее базу надо было проектировать совсем иначе. Поэтому тут надо включать всю свою дотошность и методично раскапывать истину.

Чтобы исключить возможные недоработки, отлично сработает метод светофора. Берем техническое задание, прототип и аккуратно прокрашиваем: что понятно реализуется, где есть вопросы, а что просто не заработает в текущих реалиях. Таким образом, все запланированные сущности в протоколе перечисляются, описываются и обрастают вопросами.

Имея на руках качественно проработанный протокол интеграции, вы сможете:

1. Гарантировать, что вся эта нужная информация на странице будет связана и будет работать.
2. Четко и однозначно поставить задачу разработчику.



3. Диагностировать и устранять возможные проблемы на этапе отладки.

Иначе будет что-то, как-то интегрированное с чем-то, которое как-то работает, но как — знает только программист Василий из деревни Малые Дубняки, который на связи в каждый второй четверг третьего месяца, после дождичка. А вам с этим жить и зарабатывать.

10.6.3 РАЗРАБОТКА

Обычно идет параллельно на стороне донорской и принимающей системы.

Однако, когда речь идет об интеграции, вам не избежать коммуникации со всеми сторонами. Могут выявляться проблемы и недоработки, организовать решение которых — задача менеджера. Если эти проблемы идут постоянным потоком, имеет смысл назначить регулярные стендапы, как мы делали на этапе составления протокола интеграции, чтобы контролировать результаты до тех пор, пока фонтан проблем не иссякнет.

Работоспособность системы можно сверить и по протоколу. Берем блок, тестируем и проверяем одну за другой сущности, отмечаем проверенное флажками. Аномалии отмечаем красным, фиксируем проблемы, на чьей они стороне, и ставим задачи на их устранение.

Разработчики обеих систем могут начать кивать друг на друга и ждать, пока один из них сделает свою работу до конца. Но хороший протокол, примеры (моск-объекты) и настойчивость позволяют запараллелить эти работы.

Если что-то пойдет не так — у нас всего четыре варианта:

1. Мы что-то не предусмотрели в протоколе. Бывает. Садимся, открываем протокол. Уточняем.
2. Что-то накосячили в донорской системе. Идут и делают по протоколу. Через «не могу».
3. Что-то накосячили в принимающей системе. Идут и делают по протоколу. Через «не могу».
4. Слишком дофига данных. Вообще это тоже проблема протокола. Нужно бить данные на части, организовывать инкрементальный обмен, дополнительные методы обновления только самых чувствительных и т. д. Поэтому важно заранее посмотреть объем данных для обмена. Загрузка ста тысяч **SKU** в CommerceML с обработкой картинок может длиться несколько часов, причем плотно нагрузит сервер.

Все. Других источников проблем нет. Теперь — детали и самые частые точки отказа.

Со стороны ERP

Некорректно передаются данные ERP, например:

- ▶ неверно передаются сущности;
- ▶ неверно настроено подключение;
- ▶ не передаются нужные сущности или, наоборот, передаются лишние данные.

Сверяемся с протоколом и отправляем доделывать. Когда мы имеем дело с системой, где большое количество сущностей и взаимосвязей, просто прогон обменов туда-сюда может давать неполную картину. В таком случае хорошо натравить на сайт тестировщика и сделать сверку того, что должно быть, с тем, что мы имеем по факту.

Например. У вас есть сущность на сайте. У нее должен быть указанный в протоколе набор полей. И вот, после прогонки наживую, она не работает корректно. В первую очередь надо смотреть, а корректно ли она приходит. Открываем админку, смотрим. Сверяемся с протоколом.

Смотрим выгрузку. Пришло не в том виде — вот и ошибка. А если все при этом пришло, как договаривались — поздравляем, у вас баг на стороне сайта или приложения.

Свойства	Назначение свойства в ТЗ	Есть в Базе данных информации (да/нет)	Как используется (данные + описание)	Статус	Комментарий КТО СДЕЛАТЬ	Комментарий ЧУ	Что сделано	Статус проверки готовности (данные/сделано)
Реквизиты		нет	не используется	не используется	В них прописаны стоимости оборуд. Должна прописана стоимость Цанс/Сборные http://net.kh.kh.kh.MMAZaib/ данные, если не используется	Разобраться Роман	Роман убрал все лишнее	
Платеж-назначение	Платеж/назначение	нет						НЕ СДЕЛАНО В выходные свойства исправил http://net.kh.kh.kh.MMAZaib/ Должен сделать если не убрать всё лишнее тогда исправил
Файлы		да	Можно прикрепить различные файлы, например инструкции	Присылает корректно	Должно приходить в поле файлы	Надд ДОБАВИТЬ Роман в ТЗ	Юле майлы товар с файлами файлы добавлены в товар БЕЗО Стол раздаточной Крюкова/Товарное договор 3П Стол фиксированный Роман сорти - http://net.kh.kh.kh.MMAZaib/	СДЕЛАНО
Характеристики		нет	не используется	удалено	удалить	Надд удалить Роман в ТЗ	Убрать только из выгрузки	СДЕЛАНО
Штрихкод		нет	не используется	удалено	удалить	Надд удалить Роман в ТЗ	Убрать только из выгрузки	СДЕЛАНО
Выгрузка на Сайт	Выгрузка на Сайт	нет	не используется	удалено	удалить	Надд удалить Роман в ТЗ	Убрать только из выгрузки	СДЕЛАНО
Выгрузка в Базу	Выгрузка в Базу	да	Для блока с описанием товаров	Присылает корректно	удалить	Надд удалить Роман в ТЗ	Убрать только из выгрузки	СДЕЛАНО
Гарантия	Гарантия	нет	Для блока с описанием товаров	Присылает корректно				СДЕЛАНО
Гарантия	Гарантия	нет	Текстовое поле для ограничения товара с гарантией. Может принимать значения Да/Нет	Присылает корректно				СДЕЛАНО

Лучший метод здесь — светофор. Что красить — надо смотреть по ситуации.

Со стороны принимающей системы

Полученные данные не обрабатываются или обрабатываются некорректно. В первую очередь проверяем полученные данные. Если там все okay, начинаем сверяться: все ли принимающая сторона делает по протоколу? Если нет, устраняем недочеты, проверяем еще раз.

Проблема в протоколе

Эта птица не полетит. Худший случай. Это значит, что вы что-то некорректно спланировали. Собираемся и согласовываем, как решить поставленную задачу так, чтобы система заработала.

Пример:

Нужен обмен пользователями и их бонусами между 1С и интернет-магазином, например, на Битриксе. Штатный модуль такого не умеет ни там, ни там.

Договариваемся, в каком формате будет обмен: пользователи отдельно и отдельно файл с бонусами. Хотим получать только обновления по бонусам. На сайте реализована именно приемка обновлений, как договорились. При запуске системы выясняется, что на стороне 1С выгрузка бонусов, в принципе, возможна, но только полная: отдельно

выгружать изменения она не умеет, а научить ее — очень трудоемко. Это условие не было выяснено на этапе написания протокола — в итоге получается, что все сделано вроде как договорились, но это не полетит.

Могут ли быть изменения в протоколе по ходу работы? Да, и это нормально. Сначала добавляем их в смету, затем в протокол.

10.6.4 ВВОД В ЭКСПЛУАТАЦИЮ

Вы думаете, после интеграции все волшебным образом начнет работать само? НЕТ.

Тут начинается самое интересное:

1. Организуем тестовые стенды обеих систем. Загружаем в них данные, максимально приближенные к боевым.
2. Выкладываем на сервера. Как правило — сначала на промежуточные. Затем цикл повторяем на боевых.
3. Собираем команду со всех сторон на день-два.
4. Методично гоняем обмены туда-сюда.
5. Прогоняем типовые ситуации.
6. Составляем список проблем.
7. Формализуем их в задачи.
8. Ходим по итеративному кругу: сделали задачу — проверили — сделали — проверили.
9. Проводим нагрузочный тест. Для этого есть специальные инструменты имитации пользователей. Но нам важна и сама скорость обмена. Может так оказаться, что узким звеном станет сеть, или процессор, или обработка картинок, или сама ERP-система. Общие рекомендации тут дать вряд ли получится. Соображайте.
10. Когда не осталось нерешенных вопросов, проект готов к релизу.

И так до тех пор, пока то, что вы делали последние месяцы, не начнет наконец-то работать. Это действительно может занимать от пары часов до нескольких недель, в зависимости от сложности системы. Садимся и гоняем импорты вживую. Фиксируем проблемы, на чьей они стороне. Исправляем, проверяем. Иначе можно бесконечно спихивать задачи друг на друга, пока проект стремительно протухает, деньги не зарабатываются, а нервы и ресурсы истощаются.

10.7 НЕ НАДО БОЯТЬСЯ ИНТЕГРАЦИЙ

Интеграция — это нормально. Современные проекты без нее вряд ли возможны. Важно отстроить конструктивную коммуникацию со всеми участниками и проделать четыре шага:

1. Узнайте, что нужно: заполните бриф на интеграцию, получите необходимые выгрузки.
2. Запланируйте: сделайте протокол интеграции.
3. Интегрируйте: системно и технологично.
4. Приготовьте к вводу в эксплуатацию: методичная двусторонняя отладка.

Всего-то :)

ПОЯСНЕНИЯ

Веб-хук — это способ передать в стороннюю систему информацию о каком-то событии, которое произошло в текущей системе.

REST API — REpresentational State Transfer, популярный архитектурный подход для создания API.

JSON-формат — текстовый формат обмена данными, основанный на JavaScript.

Бинарный gRPC — высокопроизводительный фреймворк, созданный Google для вызовов удаленных процедур (RPC).

XML-формат — формат файла с расширяемым языком разметки: текстовый документ, структура которого отображает его описание и пользовательские теги.

NoSQL — это общее название для нереляционных баз данных, которые отличаются от реляционных тем, что в реляционных SQL-базах есть таблицы с данными и связи между ними, а NoSQL-базах все иначе: там может не быть таблиц, а вместо них — свои модели данных.

SKU — идентификатор товарной позиции, единица учета запасов, складской номер, используемый, например, в ERP-системах.

Часть 11



РАБОТА С ФАКАПАМИ

В любой непонятной ситуации прокачивайте свои компетенции.

Когда вы растете как руководитель проекта — ошибки и факапы неизбежны. Будут падения на серверах. Будут упущенные задачи. Будут злые клиенты и потерянные деньги. Будут обиженные сотрудники. Вопрос — что вы будете с этим делать? Бежать из компании в компанию или из отрасли в отрасль (стратегия колобка, который плохо кончил)? Глушить боль вискарем? Бухтеть и жаловаться? Или продолжите преодолевать проблему за проблемой?

Единственный способ вырасти — не отступать во время ошибок. Системно (системно!) их разбирать. Не прятаться. Переживать всю боль, которую доставила вам конкретная ситуация. Рефлексировать. И решать проблемы.

Вообще, без эмоциональных качелей от «Я — ничтожество» до «Я крут! Моя команда — крутая» — вы не растете. А когда вы не растете — вы умираете.

В этой главе мы посмотрим на методики и инструменты системного разбора проблем. Суть этих инструментов в том, что ими надо **работать**. А когда факап и нет времени — хочется по-быстренькому как-то откидать проблему и забыть про нее.

Самое время сказать СТОП! Замереть. Остановиться. Достать планшет. И начать думать. Час — думать. День — думать. Надо — год думайте. В конце концов поймаете: «АГА! ВОТ ОНО». И решите вашу проблему. Такова цена роста, так растут специалисты, и такова жизнь.

Если конфликт скрытый — подрывай!

Бывает есть какая-то тлеющая проблема: конфликт или поведение сотрудника. Что-то не то, но пока обходим проблему вокруг да около: избегаем, терпим, закрываем глаза или жалеем сотрудника, которого давно пора взбодрить. Не надо.

Если есть скрытый конфликт, особенно этический, который рано или поздно все равно взорвется — подрывайте его сами: выведите наружу, сделайте очевидным. Поставьте вопрос ребром: меняйся или уходи. О таком методе решения конфликтов хорошо написано в книге Макаренко «Педагогическая поэма».

11.1 РЕТРОСПЕКТИВЫ

Мы уже разбирали методики проведения ретроспектив в главе про Scrum. Здесь хочу добавить пару моментов.



Командное
решение проблем

1. Проводите ретроспективы. Обязательно. Не игнорируйте рефлексию.
2. Не скатывайтесь в формальности.
3. Научитесь слушать то, что говорят люди на ретроспективах. Выработайте навык активного слушания. Критерий: вы способны дословно повторить последнее высказывание любого человека, с которым вы общаетесь.
4. Сложные проблемы требуют комплексных решений. И порой долгих раздумий. Даже не пытайтесь за полчаса решить, например, проблему низкой скорости команды и большого количества ошибок. Такие вещи требуют времени на обдумывание, моделирование процесса разработки и итеративного подхода.
5. Если нужно — меняйте формат ретроспективы. Вместо нее составьте факт-карты, проведите root-cause-анализ или постройте ТОС-диаграммы. Да, это требует времени. И мозгов. Но другого хорошего варианта просто нет.
6. Вместо того чтобы глушить проблему, испытайте всю связанную с ней боль. Боль и дискомфорт дают энергию к действиям. Пока все хорошо, человек хрен что будет делать. И не только человек — найдите описание эксперимента «Вселенная 25», где для крыс создали идеальные условия, а они в итоге перестали размножаться и вымерли. Вообще, порой чем хуже, тем лучше. Но не надо делать хуже специально. Из зоны комфорта не надо выходить, в нее еще надо умудриться попасть. А вот из зоны скверных привычек, кривых процессов и неадекватных паттернов поведения выходить надо обязательно. Боль полезна!
7. Добейтесь, чтобы с ретроспективы все выходили с ощущением, что:
 - а. меры придуманы;
 - б. меры помогут.
8. Внедряйте задуманное. Не тяните.
9. Обсуждайте решения, успехи и неудачи с прошлых ретроспектив.

Итак. Ретроспективы, рефлексия, разбор ошибок — это важно.

11.2 ВОСЕМЬ СОРТОВ МУДЫ В DIGITAL-КОМПАНИИ

Муда, что по-японски означает «потери», — это любая деятельность, которая потребляет ресурсы, но не создает ценности для клиента.

Хорошо систематизировали виды потерь на производстве ребята из Toyota: они выделяют семь-восемь видов муды. Посмотрим, есть ли аналогии между потерями в автомобилестроении и в работе digital-компании.

Муда № 1. Потери из-за лишних запасов

В digital-компании: вся незавершенная или несданная работа:

- ▶ несданный дизайн;
- ▶ недоверстаный макет;
- ▶ непротестированный код;
- ▶ незапрограммированные требования;
- ▶ незадеплоенный код.

Чем больше недоделанной работы в вашей компании, тем больше у вас дебиторки. Тем больше вы рискуете прогореть.

Опять же, если посмотреть с точки зрения заказчика, недоделанная работа не приносит никакой пользы. Она ему не нужна.

Что делать: всеми правдами и неправдами старайтесь сократить количество несданной, одновременно выполняемой работы. Лучше довести один проект до конца и затем приступить к следующему, чем взяться за четыре-пять и делать их параллельно.

Муда № 2. Потери при ненужной транспортировке

В digital-компании: повторное обсуждение одного и того же вопроса, по которому уже принято решение (Relearning).

Например, вы пришли к выводу, что неплохо бы использовать в студии какой-то современный фреймворк для фронтенда. Тщательно изучили вопрос. Собрались, обсудили плюсы и минусы. Похоливарили, что лучше — Vue или React. Реально полностью разобрались в вопросе. И как стандарт у себя решили принять Vue. Внедрили. Обучились. Начали использовать.

И вот через полгода какой-то умник говорит: «Ребята! А почему мы используем Vue, а не Angular? Vue же говно, потому что <аргумент>»,

а Angular — это круто, потому что <еще какой-то аргумент>». На самом деле через полгода никто уже не помнит, то ли эти аргументы уже рассматривались (и вы вместе пришли к выводу об их недостаточности), то ли эти аргументы вы пропустили при обсуждении. Приходится изучать вопрос повторно. Тратить время вместо того, чтобы делать деньги.

Что делать: фиксировать не только решения, но и контекст и аргументацию по принятым решениям. Здесь хорошо работают диаграммы (Root Cause-анализ, Mind Map, АЗ-анализ). Найти баланс между пересмотром своих процессов/средств разработки и соблюдением принятых стандартов.

Муда № 3. Потери из-за перепроизводства

В digital-компании: ненужная функциональность.

Так прикольно бывает запилить для клиента пару фишечек, **пасхалочек** или полировать фотографии на какой-то странице пятого уровня, на которую никто никогда не зайдет, или оптимизировать страницу «Наше руководство» для скорости загрузки в 0,005 секунд.

Общая идея: делать функции, которыми никто не будет пользоваться (и на которых ваш клиент не заработает) — это потери. Пример-лидер — это MS Excel: 95% пользователей постоянно используют только 5% функций. Нафига все остальное?

Что делать: критически оценивайте то, что вы делаете, с точки зрения пользы. Отмечайте и устраняйте работы, которые не приносят пользу клиентам и вам самим.

Муда № 4. Потери времени из-за ожидания

В digital-компании: затягивание согласований:

- ▶ ожидание согласования с заказчиком;
- ▶ внутренние согласования;
- ▶ бесполезные совещания, обсуждения и личные встречи.

Очень такая актуальная потеря — ожидания. Пока заказчик согласует макет на трех уровнях. Пока утвердит тексты. Пока юрист соизволит прислать правочки к договору.

Особенно много ресурсов улетает на личные встречи по пустяковым вопросам. Часто, чтобы прикрыть жопу и не принимать решений, на совещания по любому вопросу приглашается как можно больше лю-

дей. Перемещения тел по Москве на встречи для обсуждения «какого размера у нас будут буквы» — чрезвычайно дорогое удовольствие, но мало кто реально считает, сколько стоит собрать и провести одну такую встречу (подсказка: обычно несколько десятков тысяч рублей). Договоренности не фиксируются, обсуждения улетают в пустоту.

Что делать: сократить до минимума количество вовлеченных в проект людей. Принимать решения максимально оперативно. Сократить количество совещаний и личных встреч. Если уж и проводить собрание — иметь четкий регламент, фиксировать и соблюдать договоренности.

Муда № 5. Потери из-за выпуска дефектной продукции

В digital-компании: баги.

Что важно знать про баги: чем позже дефект найден, тем он дороже. Если вы, допустим, программист, и находите и фиксируете баг сами и сразу — это стоит дешево. А вот если его находит клиент, то это уже чревато. Особенно, когда баг найден через два года, уничтожил базу данных клиента, а программист, который писал код — в отпуске, в другом городе и умер.

Что делать: проводить как можно более раннее тестирование. На крупных проектах — автоматические тесты. Четкие стандарты кодирования. Регулярные практики code review. Наставники для новичков.

Муда № 6. Потери из-за ненужных перемещений

В digital-компании: потери передачи проекта:

- ▶ ответственности;
- ▶ знаний;
- ▶ проектов;
- ▶ действий;
- ▶ заявок.

Сколько промежуточных звеньев между тем, как идея клиента попадет из его головы к тому человеку, который будет конкретно ее реализовывать? Вполне возможно, что клиент сначала рассказывает свою историю аккаунт-менеджеру, затем — менеджеру проектов, затем менеджер проектов рассказывает все вводные аналитику, затем собранные данные передают дизайнеру.

Каждый раз при передаче проекта происходит потеря времени и информации: вербальной и невербальной. Потеря информации

в итоге приводит к ошибкам, переделкам и, опять же, к затратам времени. Особенно чувствительны потери при передаче проектов с одного менеджера проектов на другого, либо при смене программиста в проекте. И самый сложный случай — смена ответственного лица на стороне клиента.

Важная тонкость: когда проект изучается и анализируется в первый раз — это имеет ценность для клиента. Потерями будет только повторное размусоливание одних и тех же сведений.

Что делать: минимизировать количество вовлеченных в проект людей до необходимого минимума. Исключить передачу проектов между командами и ответственными лицами.

Муда № 7. Потери из-за лишних этапов обработки

В digital-компании: переключение разработчиков между большим количеством разных задач.

Человеческий мозг плохо приспособлен к многозадачности. Пока мы переключаемся с проекта на проект, мы теряем контекст задачи. На восстановление контекста нужно от 15 минут до получаса. Это чистые потери. Считайте, если вы дернули человека, — это вы слили 15 минут. Помножьте на ваш рейт, вычтите у себя из зарплаты. Если будет жадно — не дергайте. Если не жалко и вопрос того стоит — можно дернуть.

Кстати, если ваш программист способен переключать контекст быстро — это значит, что в нем сильны задатки менеджера.

Что делать: не дергайте людей. Сокращайте Work in Progress (WIP) — количество задач в одновременной работе.

Муда № 8. Нереализованный творческий потенциал сотрудников

В digital-компании: долбаная рутина.

Всегда есть выбор: поставить на проект того человека, который сделал 100500 аналогичных проектов, или поставить того, кто еще не разобрался с такой темой. Нагружая разработчиков рутинной и не давая им действовать самостоятельно, вы увеличиваете нагрузку на вашего HR-а. За вами выбор: вложить ресурсы в обучение или вложить в HR.

Что делать: стараться время от времени ставить задачи на пределе способностей программистов. Необязательно это делать все время, но иногда задача-вызов должна появляться у любого сотрудника.

11.2.1 В ПОИСКАХ МУДЫ. ИНСТРУМЕНТЫ

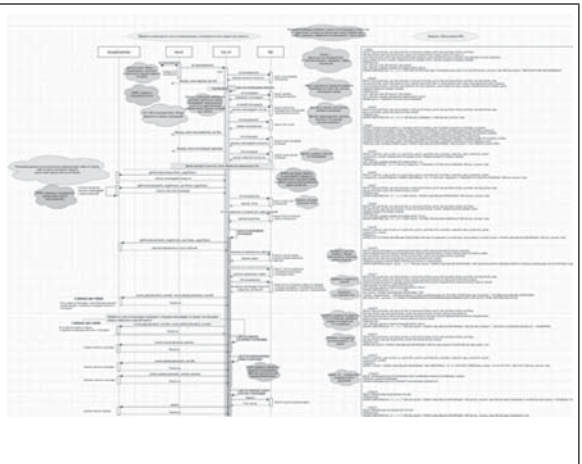
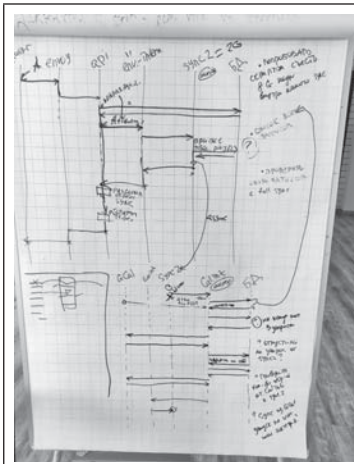
Пока проблема скрыта, она практически нерешаема.

- ▶ Пока проблема в запутанном взаимодействии микросервисов, вы как менеджер ничегошеньки с ней не сможете сделать. Даже своей команде не сможете помочь решить такую проблему (кроме как дать им вагон времени, отойти в сторонку и молиться).
- ▶ Пока проблема в стихийных бизнес-процессах, вы не сможете решить ее системно. Попытки решить точечно приведут к проблемам в другом месте. Или к куче регламентов и правил, которые невозможно не нарушить. Что, в свою очередь, приведет к еще большим проблемам и ненависти.

В общем, пока проблема под капотом, ее не решить. Надо открывать капот. Или даже разбирать двигатель.

Первое, что нужно сделать, — визуализировать предметную область. Нарисовать диаграммы взаимодействия микросервисов. Нарисовать диаграммы бизнес-процессов. Расписать связи. Визуализировать проблему с учетом всех связанных с ней объектов.

И только потом можно искать решение.

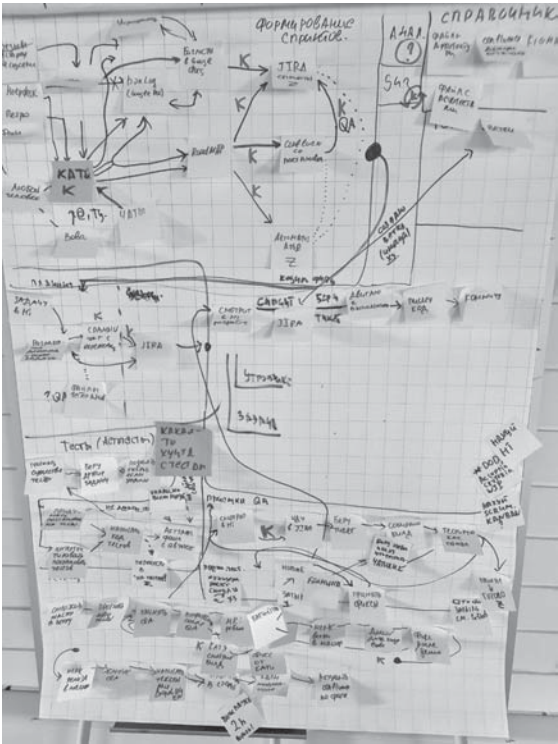


*Пример с ретроспективы: визуализация запутанных потоков взаимодействия микросервисов в поисках **deadlock**. Все началось с зависания сервера в неподходящий момент. Продолжилось поиском причин на флипчарте. А закончилось скрупулезным расписыванием всех взаимосвязей в коде. Бывает и так.*

Итак. Визуализируйте проблему. Уровень детализации должен быть достаточен, чтобы увидеть и саму проблему, и нужные взаимосвязи. Погружайтесь внутрь, залазьте под капот, разбирайте двигатель. Привлекайте к этому команду.

Какие инструменты помогут это сделать и где про них почитать:

- 1. **Диаграммы теории ограничений (ТОС).** Дерево текущей реальности (описывает текущее состояние дел), инъекции, дерево будущей реальности, грозовые тучи конфликтов. Изучите книги Голдратта по теории ограничений (только именно Голдратта — это клевые бизнес-романы, — а не его последователей: книгу Уильяма Детмера читать можно, только если плохо спится — нудятина редкостная).
- 2. **Факт-карты.** По сути, те же идеи, что и в интеллект-картах и деревьях ТОС, но менее формальные. О методе можно почитать, посмотреть видео или пройти по нему целый курс у Андрея Курпатова. Но вообще он интуитивно понятен и очень прост. Собираете все факты, выписываете, строите связи. В какой-то момент поймаете «АГА!» и сможете действовать.



Факт-карта с парадоксальными выводами

Как-то раз мы использовали этот метод для проведения брейн-шторма по запутанной проблеме: на проекте была почти нулевая скорость разработки. Понадобилось два часа, чтобы прийти к неожиданному выводу.

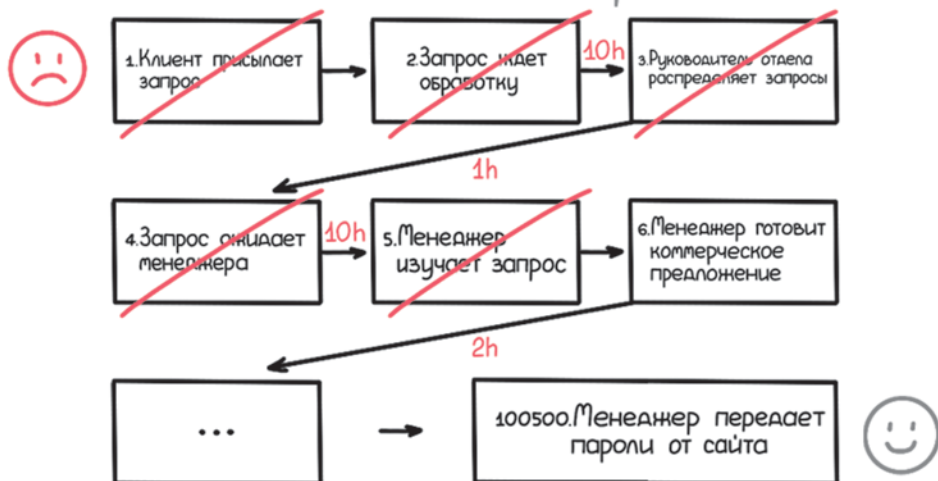
Нам мешает Scrum-подход, и нужно переходить на Канбан. Еще три недели ушло на переход на новые рельсы. А реальный эффект получили только через полтора месяца. Но через два часа мы уже понимали, что меры выработаны и меры помогут!

3. Value Stream Map. Визуализируйте поток поставки ценностей. Вся цепочку операций, которую вы делаете, от момента того, как к вам пришел клиент, до момента, как вы получили деньги, выполнив заказ и сделав клиента счастливым. Степень детализации зависит от задачи: иногда нужно идти глубоко, на уровень операций, вроде «менеджер пишет сообщение в скайп» — именно в этом и может быть проблема.

А дальше у каждой операции указываем затраты на нее. Смотрим, какие из операций не приносят ценности для клиента и продукта. И ищем способы их устранить. Время от времени нужно встряхивать так все процессы.

Ищем и устраняем потери

сделать клиента — счастливым — быстро!



4. АЗ-анализ. Используйте этот шаблон:

Описание контекста проблемы Plan АЗ-анализ (отчет АЗ)	АЗ: <суть проблемы> Владелец: Ментор: Дата:
Текущее состояние Plan	Контрмеры (эксперименты) Do
Цель/целевое состояние Plan	Проверка результатов Check
Поиск корневых причин Plan	Дальнейшие действия (список задач) Act

Шаблон АЗ-анализа от Хенрика Книберга и Тома Поппендика — внутри инструкция по применению.



5. Пять Почему (5 Why). Тойотовский метод: про любую проблему задавать вопрос «почему» несколько раз (иногда пять — недостаточно) и погружаться внутрь проблемы:
- а. Заказчик негодует. Почему?
 - б. Упал сервер. Почему?
 - в. Повис скрипт. Почему?
 - г. Бесконечный цикл в коде. Почему?
 - д. Вася выложил непротестированный код. Почему?
 - е. Вася — новый сотрудник, который не знал, что код нужно тестировать.

Далее ищем решения на каждом уровне. Обучаем Васю, чиним бесконечный цикл, поднимаем сервер, успокаиваем заказчика и т. д. Главное — не уходить в бесперспективные ветки («почему упал сервер — потому, что Вася — мудака»).

5 Почему (5 Why)

- упал сервер	поднять сервер
- повис скрипт	перезапустить скрипт
- бесконечный цикл в коде	убрать бесконечный цикл
- Вася выложил <u>непротестированный</u> код	протестировать код, научить Васю отдавать код на тестирование
- Вася – новый сотрудник, который не знал, что нужно отдать код на тестирование	внедрить обязательное обучение новых сотрудников

Бесперспективных веток, как минимум, три:

- Поверхностный анализ. Нашли проблему в человеке, все свалили на Васю.
- Философия. Решили, что вопрос философский и решения не имеет.
- Решение на букву «С». Смирение. Не наш метод :) В Lean целых пять решений на букву С. Так и называется, пять С.

6. Диаграмма Исикавы. Она же Fishbone-диаграмма (рыбий скелет). Попробуйте этот инструмент, он прост и должен быть в вашем арсенале.

Итак. Есть прекрасные инструменты системного разбора и анализа проблем. Чем больше у вас будет их в арсенале, тем легче вы сможете справляться с трудностями. Изучите их и попробуйте применять.

11.3 ХВОСТОВЫЕ РИСКИ В DIGITAL-ПРОЕКТАХ

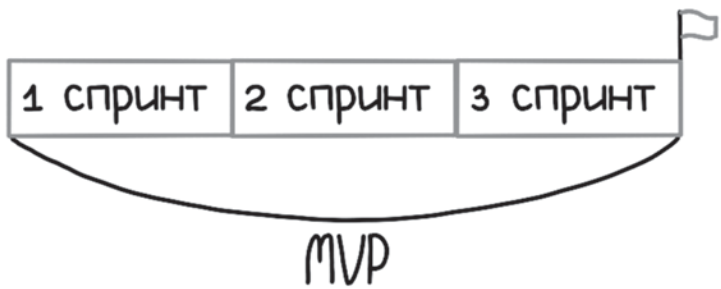
Приходит мужик в баню, забыл полотенце и шарит глазами по бане: чем бы вытереться. Видит, висит табличка: «Занавесками не вытираться!» Думает: «О, блин, идея!»

В статьях для продактов обычно сыплют советами: делай то, делай это, а еще вон то и то. Но в этом разделе мы поговорим о том, как делать НЕ надо. За это больно-больно бьют, из-за этого произошел

прошлый экономический кризис и, наверное, случится еще не один. Из-за этого вообще случается много-много нехорошего, поэтому нужно уметь бороться с этим явлением. Речь про хвостовые риски.

11.3.1 ОТКУДА ОНО?

Допустим, вы выпускаете какой-то MVP вашего продукта/сайта/приложения, который готовы отдавать на растерзание пользователям. Вы уже построили гипотезы, согласовали примерный объем работ, примерно понимаете продукт, который вам нужно выпустить — разобрали его на спринты и стартовали разработку. Сделали первый спринт, второй, третий: MVP готов.



Вы запустились, и все счастливы: пользователи приходят, вы смотрите, тестируете какие-то гипотезы — все хорошо и правильно.

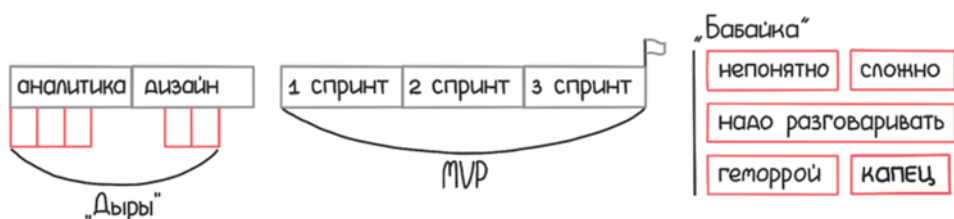
Но на каждом проекте перед спринтами есть какая-то предыстория: аналитика, дизайн и все прочее. И вот там бывают какие-то ошибки, недодделки — может быть, где-то аналитик что-то не додумал и просто оставил какие-то непонятные «дырки» или неконкретные требования (в ТЗ, в бэклоге).



Эти «дыры» всплывают, например, когда идет планинг — кто-то может сказать: «Ребята, ваши требования — говно» и потребовать уточнений. А вместо уточнений менеджер заводит своего рода «могильник», куда отправляет все задачи из разряда:

- ▶ «непонятно»;
- ▶ «сложно»;
- ▶ «надо разговаривать с клиентом»;
- ▶ «влетим по бюджету»;
- ▶ «геморрой».

«Бабайка», одним словом :)



Могильником пованивает. Лезть в него и разбираться ну очень не хочется. Но рано или поздно с ним придется что-то делать.

Допустим, у нас заказная разработка. Вы говорите клиенту: «Ура, все готово!» Подразумевая, что готово все интересное, большое, героическое и понятное. Только там кое-что нужно будет сделать при деплое... (настроить платежи на боевом сервере, например, или родить на страницу «О нас» текст получше, чем «мы молодая, динамично развивающаяся компания»). В итоге к концу трех спринтов вылезает дополнительный скоуп работ, которые неравномерно размазываются по времени, и конца им не видно. А к изначально «успешному» по времени и бюджету проекту (студия заработала денег, клиент доволен сроками, менеджер — красавчик) прилетает еще пачка гадких задач, сжигающих весь бюджет, сроки и отношения.



Но это еще цветочки. Когда у менеджера «все закончено», для клиента «все только началось». Это тонкий момент, который, к сожалению, не до всех доходит. Команда и менеджер ожидают плюшек, похвалы, бонусов, передышки и перезарядки своей батарейки перед новыми подвигами. Клиент начинает тестировать продукт на реальных пользователях, генерирует новые идеи или натывается на «бабайки» из могильника.

Что в этот момент делает умный, но хитрожопый менеджер? Сваливает: в отпуск, на больничный или насовсем. Главное — получить свои бонусы до того, как могильник раскопали. И быть в этот момент вне зоны досягаемости.

Очень редко так делают дизайнеры (практически никогда), редко, но бывает — программисты (в «бабайку» складываются «костыли» и «велосипеды» — технический долг). Среди менеджеров, к сожалению, такая ситуация встречается почаще. Прекрасно про это явление написано у Талеба в «Рискуя собственной шкурой. Скрытая асимметрия повседневной жизни».

11.3.2 КАК НЕ ДОПУСТИТЬ ОБРАЗОВАНИЯ ХВОСТОВЫХ РИСКОВ

Потенциально «бабайка» может возникнуть на любом проекте. Но даже если она там появится, хороший менеджер не позволит ей превратиться в бомбу замедленного действия. Что для этого можно сделать?

1. Повышать квалификацию

Чаще всего «бабайки» выползают, когда не хватает знаний и навыков. Мы откладываем что-то в сторону, если не знаем, что с этим делать. Более зрелое поведение — действовать с точностью до наоборот. Я чего-то не знаю — значит, это повод разобраться. Обязательно стоит прокачивать и себя, и других.

2. Легализовать могильник

Частая история, когда сложности, непонятки и прочие тонкости на проектах хранятся у менеджера на стикерах, в ежедневниках, в голове — где угодно еще, но вне доступа для других. Чтобы из них не

получился могильник, стоит записывать все в одном месте, например, в том же бэклоге. Пусть могильник будет, но он будет понятен по размеру, легализован и визуализирован.

3. Сложные задачи — в начало

Я рекомендую в первые спринты разработки брать самые сложные задачи. Мотивация:

- ▶ в начале проекта всегда больше энтузиазма (энергии, драйва — как угодно), и сложности решать проще;
- ▶ чем раньше вылезут сложности и вопросы, тем больше времени будет принять по ним решение (можно будет увидеть прогресс потери времени, если используете Burndown Chart или метод освоенного объема из § 5.9).

Это не всегда возможно, но стоит попробовать.

4. Шкура на кону (по Н. Талебу)

По древним вавилонским законам Хаммурапи, если строитель построил дом, а тот упал и убил хозяина, строителю грозит смерть — он отвечает собственной шкурой. Схожую историю про строительство паровозного моста в царской России вы уже знаете из главы 3 (при завершении строительства моста по нему пускали паровоз, а всю команду строителей загоняли под мост).

Когда менеджер сам отвечает за проект, сам горит за него и нацелен довести его до конца, а не просто получить медальку — вот тогда шансы на успех проекта значительно выше. Очень хорошо это ложится на Scrum, когда вся команда участвует в демке спринтов, рассказывая, что и как было сделано. И получает прямую обратную связь.

5. «А не херню ли я делаю?»

Вопрос, который надо задавать себе почаще. Если поймали себя на этом — значит, скорее всего, «бабайка» уже рядом.

6. Метод «Приблизиться к оленю» (по В. К. Тарасову)

Представим ситуацию: охотник хочет убить оленя — прицеливается, но стреляет мимо. Какие последствия? Во-первых, падает его репутация среди других охотников. Во-вторых, страдают материальные ресурсы

(одну стрелу уже потерял). В-третьих, он останется голодным. А все потому, что он был от этого оленя далеко. Чтобы гарантированно убить оленя, нужно приблизиться к нему на такое расстояние, где ты точно не ошибешься.

В практическом плане это значит: все, что несложно проверить, нужно проверять самому. Если говорят, что сделано, проверить, что это реально так. Открывать файлы, проверять, трогать, тыкать, а не слепо доверять это дело другим.

Как побудить другого человека проверять выполнение задачи так же тщательно, как ты сам?

- ▶ Попросить физический артефакт проверки (фото, документ, таблица), а не просто слова.
- ▶ Задавать конкретные вопросы (вместо «оплатил ли клиент?» спрашивать номер платежного поручения).
- ▶ Повторять свой вопрос — это порождает сомнение. Например: «Ты положил макеты в нужную папку? Точно положил? В ту самую папку, где остальные хранятся?»

11.3.3 ЭКСТРЕННЫЕ МЕРЫ, ЕСЛИ МОГИЛЬНИК ВСКРЫЛСЯ ПОСРЕДИ ПРОЕКТА

1. Проинформировать всех заинтересованных лиц о сложностях и согласовать решения. Например, включить описание проблем в еженедельный статус-репорт.

2. Спланировать очередность решения проблем

Если могильник есть и в нем слишком много затыков и сложностей, стоит распланировать, в какой момент какую штуку оттуда достать и разобраться с ней.

Как правило, такие вещи всплывают во время выполнения спринтов. Первый звоночек бывает на планинге, потом еще что-то во время выполнения задач, потом — при приемке. В итоге за один спринт накапливается какое-то количество спорных элементов. По моему опыту, нормальная практика, когда в конце спринта мы с клиентом обсуждаем их и решаем, что мы берем в новый спринт — тогда это не становится могильником.

11.3.4 СИМПТОМАТИКА ХВОСТОВЫХ РИСКОВ

Симптомы полезно знать, чтобы отловить, не плодишь ли ты сам как менеджер хвостовые риски и не плодят ли их члены твоей команды.

- ▶ «Потом», «в следующем спринте...», «когда получим доступы...» — либо сам менеджер так говорит, либо слышит это от исполнителей;
- ▶ туман, отсутствие конкретики, байки;
- ▶ «если бы...» — перекалывание ответственности с себя на других («если бы дизайнер нарисовал по-другому, клиент бы...»);
- ▶ усреднение: как только начинается разговор про какие-то средние величины, про то, что в целом все хорошо — в целом, в среднем, в общем — это уже очень тревожный сигнал, что человек «зарывает хвост». Внешне у него все хорошо, но никто не знает, какой у него могильник и сколько в нем спрятано. А если долго складывать в могильник проблемы, рано или поздно это все вскроется и погребет менеджера под собой.

Пример усреднения

Русская рулетка: каждому выжившему дается по тысяче долларов. Эксперимент проводится на тысячу испытуемых. В среднем выигрыш составит 860 баксов. Если же испытуемый один, а выстрелов тысяча — проигрыш (и место в гробу) гарантированы.

Итого: даже простая легализация могильника делает проект управляемым. Шкура на кону, высокая квалификация и акцент на сложных вещах работают еще лучше. Успехов!

11.4 АЛГОРИТМ РАЗБОРА ПРОБЛЕМ

«Этот проект похож на вечный огонь! 🔥»
(подслушано; слишком много смысловых слоев...)

Независимо от выбранного инструмента, шаги следующие:

- ▶ На первом этапе нужно выявить проблемы. Нежелательные Явления (НЖЯ, в терминах ТОС). Зафиксировать их. Визуализировать. Мне удобно это делать в майндмэпе или на маркерной доске (ну, или ватмане).

- Построить диаграмму причинно-следственных связей по выявленным проблемам. Для этого нужно по каждой проблеме задавать вопросы: «почему?», «почему это произошло?», «по какой причине?» Искренне, как бы ни было неприятно это сознавать — сообщать о причинах проблем. Записывать причины на диаграмме, строить причинно-следственное дерево.



*Мы собрали НЖЯ,
Получилось до... очень много*

- Не уходить в неконструктивные ветки («В программе баг, потому что Вася — мудака»). Исходить из того, что все люди хорошие, но в рабочей системе есть какие-то ограничения, которые мешают

им правильно выполнять процесс. Стараться найти такие ветки, которые являются истинными причинами и ограничениями.

- ▶ После того как дерево проблем построено, пройдите по всем веточкам и всем узлам, предложите противодействия, которые нейтрализуют проблему (одно или несколько конкретных действий, инноваций или изменений рабочего процесса).
- ▶ Убедитесь, что изменения помогут. Убедитесь, что эти изменения можно внедрить и это будет рационально. Поговорите об этом с вашим начальником. Согласуйте с ним, какие именно изменения вы будете внедрять и каким способом. Запланируйте работы в свой календарь или в календари тех сотрудников, которых планируется привлечь к внедрению изменений.
- ▶ Внедрите изменения. Проанализируйте последствия. Сообщите о результатах вашему начальнику. Без внедрения вся проделанная ранее работа — бессмысленна.

11.5 РИСК-МЕНЕДЖМЕНТ. ЛЖЕНАУКА?

Многие мои коллеги тут не согласятся, но я не люблю выделять большое количество времени и сил на попытки предусмотреть и регламентировать все возможные проблемы в будущем.

Да, есть сферы, где нужно заранее предусмотреть риски и реакцию на них. Но практика показывает, что самые веселые риски вы все равно не предусмотрите и огребете по полной. Черный лебедь вылетит из-за угла и устроит вам ататуй. Единственное, в чем мы можем быть уверены — что Мерфи работает, даже когда вы отдыхаете. Особенно когда вы отдыхаете!

И тем не менее!

Тем не менее, управление рисками принято выделять в отдельную дисциплину. Поэтому тут будет пару страниц про эту штуку.

11.5.1 РИСК-МЕНЕДЖМЕНТ

Два главных риска любого проекта: не уложиться в сроки и бюджет. Остальные нужно продумать и принять решение, что с этим риском делать: игнорировать и надеяться, что вам прилетит в меньшей

степени, или управлять ими. Риск-менеджмент — это процесс из пяти этапов:

- 1. Поиск основных рисков.
- 2. Оценка их важности.
- 3. Поиск способов снижения риска.
- 4. Оценка стоимости этих мер.
- 5. Оценка целесообразности мер при данной степени риска.

Информация, полученная на каждом этапе, вносится в реестр рисков.

№	Причина	Описание риска	Важность (рейтинг)	Вероятность	Последствия	Оценка потерь	Стратегия	Основной сценарий	Запасной сценарий	Ответственный	Случился / не случился
1	Факт охвата	Выдачей разработанной заявки на дополнительный контракт увеличивается на одного человека, проект не будет сдан в срок.	20	5	7	5000 рублей	Принятие	Заполнить дополнительный арочный размер в 5 дней	-	Менеджер Маша	
2	Человеческий фактор	Менеджер вносит ошибку, чтобы поговорить с рабочими своей группы	8	8	1	3 рубля	Принятие	Установить индивидуальную связь на телефоне, чтобы она всегда была на связи. Поговорить с менеджером о своем. Предложить варианты: 1. Пусть договориваются с парой из команды, чтобы переписать данные на листе. 2. Если не смогут договориться, пусть установит или резервный источник питания. 3. Если 1 и 2 не работают, пусть вынудит учителя из школы временно отлучиться.	-	Менеджер Маша	
3	Оценочный риск	Потребовалось сделать из-за ошибок на первом этапе. За последние недели переделано было 6 раз по 30 минут. Если так пойдет дальше, не хватит денег на проект, на этот до 5 часов для каждого разработчика, т.е. 10 часов в общей.	64	8	8	10000 рублей	Парадокс	Найти и самостоятельно установить дополнительный источник питания	Найти и установить дополнительный источник питания	Эксперт Евгений Аветисович	

В результате вы получите таблицу со следующими полями:

- ▶ причина;
- ▶ описание риска;
- ▶ важность;
- ▶ вероятность;
- ▶ последствия;
- ▶ оценка потерь;
- ▶ стратегия;
- ▶ основной сценарий;
- ▶ запасной сценарий;
- ▶ ответственный (за управление риском);
- ▶ случился/не случился (заполняется постфактум).

Риски бывают известные (контролируемые) и неизвестные.

Известные риски бывают там, где есть информация: если вы уже работали в похожих условиях и знаете, где ружье может выстрелить. Или слышали об этом. И понимаете, что делать с последствиями.



Неизвестные риски (в некоторых источниках — неконтролируемые) — те, которые вы даже не можете представить из-за недостатка информации. Они появляются на новых проектах и при работе с незнакомой командой. Их сложно найти, к ним трудно подготовиться.

11.5.2 ОЦЕНКА ВАЖНОСТИ РИСКОВ

Важность рисков определяется, чтобы иметь представление, на что реагировать в первую очередь. Ее можно получить несколькими способами. Первый и самый простой — перемножить вероятность и последствия риска.

Вероятность и последствия оцениваются по десятибалльной шкале. Делает это руководитель проекта, опираясь на свой опыт и знания.

Если их не хватает, можно обратиться к команде, но на данном этапе это необязательно.

Когда вероятность и последствия оценены, руководитель перемножает их и получает показатель важности риска. Максимум получится 100 баллов.

Второй способ рекомендуется в PMBoK 5 — это матрица вероятности и воздействия. Выглядит она так.

Вероятность	Угрозы					Возможности				
0,90	0,05	0,09	0,18	0,36	0,72	0,72	0,36	0,18	0,09	0,05
0,70	0,04	0,07	0,14	0,28	0,54	0,54	0,28	0,14	0,07	0,04
0,50	0,03	0,05	0,10	0,20	0,40	0,40	0,20	0,10	0,05	0,03
0,30	0,02	0,03	0,06	0,12	0,24	0,24	0,12	0,06	0,03	0,02
0,10	0,01	0,01	0,02	0,04	0,02	0,08	0,04	0,02	0,01	0,01
Воздействие	0,05	0,10	0,20	0,40	0,80	0,80	0,40	0,20	0,10	0,05

В матрице нужно определить не только вероятность возникновения риска по шкале от одного (100% произойдет) до нуля (не случится никогда), но и тональность последствий: негативную или позитивную. И найти соответствующий показатель на нижней шкале.

После нужно определить уровень воздействия риска: очень низкий, низкий, средний, высокий или очень высокий. Найти соответствие на шкале слева. И после — найти ячейку на пересечении уровня воздействия и вероятности. В ней-то и будет показатель важности риска. Чтобы вы сразу понимали, хороший этот показатель или плохой, ячейки раскрашены в три цвета. Серый означает низкий уровень риска, светло-серый — средний уровень, и красный — высокий уровень.

11.5.3 ПОИСК СПОСОБОВ СНИЖЕНИЯ РИСКА

Есть четыре стратегии работы для снижения рисков.

Уклонение от риска. Цель: уклониться от угрозы и полностью исключить ее. Реализовать уклонение бывает невозможно, а если возможно, то сложно технически. Да и не факт, что принятые меры сработают. Наверное, единственный стопроцентно работающий вариант уклонения — полное прекращение проекта.

Передача риска. В случае передачи вы делите последствия риска (как плохие, так и хорошие) с другой стороной. Как вариант, с заказчиком или подрядчиком. Так можно поступить в случае, если повлиять на риск невозможно. Таким может стать, например, приостановка проекта из-за несвоевременной оплаты счетов.

Чтобы передать риск, нужно договориться об этом с принимающей стороной и закрепить решение в договоре или соглашении.

Снижение риска. Цель этого способа — уменьшить вероятность возникновения рисков и смягчить их последствия. Начинать снижение рисков лучше на старте проекта. Потому что чем дальше, тем дороже и сложнее управлять риском.

Вот как снижают риски при строительстве больших пассажирских самолетов. Существует риск, что оборудование перестанет работать. Чтобы последствия при этом были максимально благополучные, производители самолетов используют принцип дублирования. Если отказ системы может привести к катастрофе даже при правильных действиях пилота — система дублируется трижды. Если отказ системы приводит к катастрофе при неправильных действиях пилота — то дважды. Если отказ не приводит к катастрофе — система не дублируется.

В случае с digital-проектами можно проводить больше тестов, добавив резервное копирование и т. д.

Принятие риска. Те, кто выбирают это способ, смирились с риском и не делают ничего активного, чтобы его предотвратить. Вот если он наступит — тогда они начнут работать с последствиями. Принятие подходит в том случае, если уклониться от риска нельзя, а передавать его — дорого и неоправданно.

В принятии можно действовать активно или пассивно. Пассивно — просто зафиксировать, что делать, когда угроза превратится

в реальность. Активно — установить резерв на возможные потери: дополнительное время или деньги.

В реестре рисков в столбец «Стратегия» впишите способ снижения. Сделать это нужно для каждого риска. И, исходя из стратегии, придумайте и запишите основной и запасной сценарий действий на случай, если риск сработает.

И не забудьте назначить ответственного по каждому риску, иначе толку от сценариев будет мало. Обязательно должен быть тот, кто будет следить за их реализацией.

11.5.4 ОЦЕНКА СТОИМОСТИ МЕР ПО СНИЖЕНИЮ РИСКА И ИХ ЦЕЛЕСООБРАЗНОСТИ

Один из методов, по которому можно оценить риск — анализ ожидаемого денежного значения. Чтобы рассчитать его, нужно умножить значение каждого возможного результата на вероятность его наступления, а затем сложить вместе полученные значения. Возможный результат выражается в деньгах. Для неблагоприятных рисков показатель будет отрицательным, для благоприятных — положительным.

$$XY + X_2Y_2 + X_3Y_3 + \dots$$

X — вероятность наступления риска

Y — убыток (прибыль) в случае наступления риска

В случае, если проект небольшой и не предполагает серьезных рисков, можно не заморачиваться с формулами, а проставить экспертные оценки.

Итак. Среди матерых менеджеров есть мнение, что риск-менеджмент несостоятелен — поэтому к нему редко прибегают. Действительно, на небольших проектах вести подобные таблицы и учет возникших рисков избыточно. Но на проектах критичной инфраструктуры — штука оправдана. Фиксируем риски, фиксируем стоимость, фиксируем меры в случае возникновения.

11.6 ЗАКЛЮЧЕНИЕ

Итак. Факапы неизбежны. Боль полезна для обучения. Главное — не сдаваться, продолжать учиться и решать проблемы. В какой-то момент наступит ясность мысли, и придет сила. Об этом хорошо написано у Кастанеды, глава про четырех врагов.

Не бойтесь проблем, идите внутрь, разбирайтесь и решайте их. Успехов!

11.7 ДОМАШНЕЕ ЗАДАНИЕ

Изучите инструменты из § 11.3. Начните применять их на практике в своих проектах. Попробуйте каждый из них. Выберите те, которые понравились.

ЛИТЕРАТУРА

- ▶ Михай Чиксентмихайи, «Поток. Психология оптимального переживания».
- ▶ Джеймс П. Вумек, Дэниел Т. Джонс, «Бережливое производство. Как избавиться от потерь и добиться процветания вашей компании».
- ▶ Хенрик Книберг, «Scrum и XP: заметки с передовой», «Scrum и Kanban: выжимаем максимум».
- ▶ Мэри и Том Поппендик, «Implementing Lean Software Development: From Concept to Cash».
- ▶ Элияху Голдратт, «Цель», «Цель 2», «Критическая цепь», «Сателлитная программа» (видеокурс).
- ▶ Джеффри Лайкер, «Дао Toyota. 14 принципов менеджмента ведущей компании мира».
- ▶ Эрик Рис, «Бизнес с нуля. Метод Lean Startup для быстрого тестирования идей и выбора бизнес-модели».
- ▶ Том Демарко, «Deadline. Роман об управлении проектами».
- ▶ Руководство РМВОК — «Руководство к своду знаний по управлению проектами».
- ▶ Нассим Николас Талеб, «О секретах устойчивости. По следам “Черного лебедя”».
- ▶ Питер Бернштейн, «Против богов: укрощение риска».

ПОЯСНЕНИЯ

Пасхалка — секрет в компьютерной игре, фильме или программном обеспечении, заложенный создателями. Обычно его находят самые внимательные пользователи, а выглядит он как шутка или прикольный эффект.

Deadlock, или Взаимная блокировка — ситуация в многозадачной среде или СУБД, при которой несколько процессов находятся в состоянии ожидания ресурсов, занятых друг другом, и ни один из них не может продолжать свое выполнение.

Пять решений на букву С — в философии Lean: сортировка, соблюдение порядка, содержание в чистоте, стандартизация и совершенствование.

Часть 12



ОСНОВЫ ТЕХНИЧЕСКОЙ ГРАМОТНОСТИ

Я очень долго не хотел писать эту главу. Хотя хорошо разбираюсь в предмете и слежу за тенденциями. Однако технологии до сих пор быстро меняются. Не так быстро, как во времена дикого интернета. Но рассказывать про конкретные фреймворки все еще нет смысла. Все поменяется завтра. Кроме того, очень сложно подстроиться под уровень читателя: с одной стороны, нужно изложить все просто. С другой, есть масса нюансов и частных случаев, которые будут упущены. Так вот.

Я буду упрощать картину мира до безобразия.

Здесь мы рассмотрим базовые технические понятия, которые лежат в фундаменте современных технологий. Глава будет полезна начинающим руководителям digital-проектов, у которых еще не встал на место пазл: как же все это устроено внутри на самом деле.

И да. Хотите быть первоклассным руководителем — повышайте свою техническую грамотность. Постоянно.

12.1 ЯЗЫК ОТРАСЛИ

Трудно стало работать.

Развелось много идиотов, говорящих правильные слова.

Штирлиц, Семнадцать мгновений весны

Сфера IT пронизана профессиональным сленгом, который очень раздражает тех, кто не в теме. Но руководителю проекта приходится с этим сленгом сталкиваться ежедневно. Чтобы общаться с командой

программистов, нужно говорить с ними на одном языке. В любой отрасли — банковской, юридической, криминальной — есть такой язык.

Нужен он для нескольких целей:

1. Определение «свой-чужой».
2. Грузить и запутывать чужих.
3. Умничать или доминировать друг перед другом.
4. Договариваться в коротких терминах и решать вопросы по делу.

В общем, если вы хотите быть «своим», чтобы вас не грузили собственные разработчики и не доминировали над вами, а также, чтобы было проще решать вопросы, придется освоить язык отрасли.

Однако бойкое владение языком отрасли не гарантирует, что перед вами профессионал. Профи вообще, как правило, немногословны. И способны сложные вещи донести в простой форме.

К сожалению (или к счастью), отрасль довольно сложная. Термины, как веревки в плетеном коврикe, цепляются один за другой. А еще содержат кучу исключений и частных случаев, где все работает совсем по-другому, чем обычно. Именно поэтому опытные программисты говорят и пишут с большим количеством оговорок.

Как освоить язык отрасли?

1. Так же, как учили иностранный язык. Выписывайте все непонятные слова. Ищите значения. Через пару месяцев картинка встанет на место. Ничего там сложного, уверяю, нет.
2. Найдите «словарь терминов IT». Однако не пытайтесь прочитать и заучить его за раз. Зубрить бесполезно. Надо действовать.
3. Переспрашивайте непонятные слова до полного понимания предмета. Не обращайте внимание на гримасы программистов, их закатывания глаз при попытке объяснить какой-то термин. Профессионал может объяснить сложную концепцию простыми схемами и словами. Дилетант будет раздувать из гондона дирижабль.
4. Почти шуточный совет девушкам, но я видел десятки примеров, где это работает. Девушки, охмурите программиста. Отмойте, оденьте, воспитайте и все такое. Будет вам источник всех благ и ценной информации.



Неплохой пример такого словаря

Итак. Изучайте язык отрасли.

12.2 КАК РАБОТАЕТ ИНТЕРНЕТ

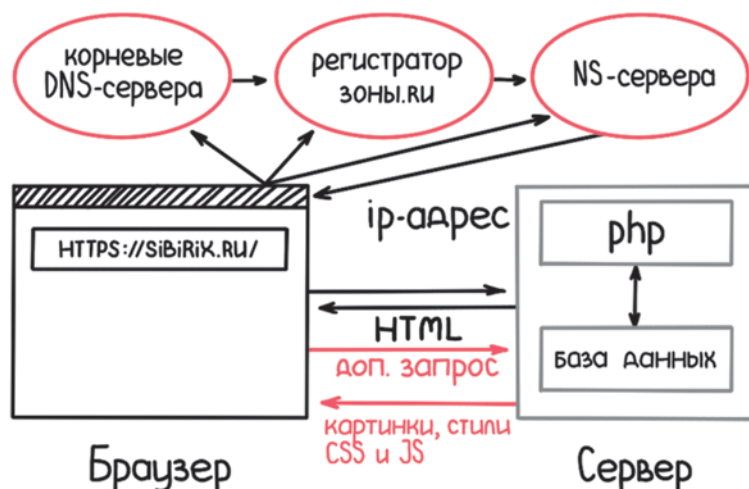
12.2.1 АДРЕСА

У каждого устройства в сети есть IP-адрес. Это уникальный номер в рамках подсети, типа 127.0.0.1 или fe80:0:0:0:200:f8ff:fe21:67cf. Это как ваш номер в паспорте. Да, его можно поменять, но нужно обращаться в органы. Да, может быть несколько паспортов: отечественный и пара загранных.

12.2.2 ДОМЕНЫ

Адреса набирать неудобно. Чтобы не сильно мучаться, были придуманы Доменные Имена (домены): `www.myhomesite.ru` или вроде того. Это как фамилия-имя в паспорте. Только уникальное в рамках всей сети — двух Ивановых.Иванов.Ивановичей.ru быть не может. За этим следят.

Домен можно привязать к IP-адресам. Закрепить паспорт и его номер за конкретным Ивановым. За это отвечает служба DNS (Domain Name System — «система доменных имен»). Это распределенная система, размазанная по тысячам серверов, чтобы обеспечить устойчивость к отказам. Если совсем просто, DNS — это система, которая позволяет найти тот самый сервер, на котором лежит сайт, который вы вбиваете в строку браузера. То есть перевести символьное имя сайта в конкретный сетевой адрес.



Домены первого уровня (RU, COM) вам создать не получится. Домен второго (myhomesite.ru) нужно покупать за деньги и потом не забывать продлевать. Домены третьего, четвертого и какого угодно другого уровня можно настроить самостоятельно.

12.2.3 ПАКЕТЫ ИНФОРМАЦИИ

В сети устройства обмениваются пакетами информации. Это как посылать письма в конвертах. Причем еще и сами конверты могут вкладываться друг в друга. На внешнем конверте может быть адрес получателя «Москва». На вложенном — «Бауманская, 7». На следующем — «Офис 215». На самом внутреннем — «Отдать Вове лично в руки!»

Думаю, у всех на кухне есть пакет с пакетами, кастрюля с кастрюлями или матрешка с матрешками. Интернет от этого недалеко ушел. Но зачем вкладывать пакет в пакеты? Почему нельзя вот так вот сразу и куда надо?

Так нужно, чтобы одна конкретная программа на вашем компьютере или телефоне достучалась до другой конкретной программы на сервере, попутно не думая, по проводам, воздуху или спутнику идет сигнал. Каждая «обертка» изолирует пакет информации от среды передачи. Физически сигналы идут, например, по проводам. Долетели до коммутатора — нужно направить их в правильную подсеть. Долетели до нужного компьютера — нужно отдать данные нужной программе. И везде свои тонкости и нюансы.

12.2.4 ПРОТОКОЛЫ

Чтобы системы понимали друг друга, придумали страшное слово «протокол». Означает просто договоренность о режимах запросов и ответов. Меня можно спрашивать о том-то и том-то, в таком-то формате, а я буду отвечать в таком-то формате. Все.

Стандартизация форматов сообщений, протоколы — это не изобретение программистов. Похожие техники используются во всех средах, где может случиться потеря информации, но важно ее

донесение. Например, в авиации. *«Восемнадцать пятьдесят пять гольф, тысячу двести по тысяча тринадцать гектопаскаль над точкой занял, задание»*. Гыр-гыр-гыр, быр-быр-быр. Со стороны ничего не понятно. Но ооочень интересно. Профи же поймет сразу, что к чему.

Протоколов существует много, на разные случаи жизни. Из того, что вы, скорее всего, услышите — FTP (протокол передачи файлов), SSH (протокол удаленного доступа к серверам), POP3/IMAP/SMTP — почтовые протоколы, IP и TCP/IP — низкоуровневые протоколы, отвечающие за доставку пакетов с пакетами. Ну и конечно, HTTP/HTTPS.

12.2.5 HTTP, HTTPS, SSL-СЕРТИФИКАТ

Сайты открываются по протоколу HTTP или HTTPS. HTTP не шифрованный — я могу поймать ваш трафик и посмотреть все ваше порно. Как сосед по квартире может подглядеть в ваш телефон.

HTTPS — шифрование. Просмотр чужого порно сильно усложнен. Ваш сосед будет видеть только рябь на экране.

Чтобы шифровать трафик, на сервер нужно установить SSL-сертификат. Как правило, — платный (но есть бесплатные варианты). SSL-сертификат говорит, что вы действительно обмениваетесь трафиком с тем, кем хотели. Говорит, но не гарантирует.

12.2.6 СЕРВЕРНЫЙ КОД. BACKEND

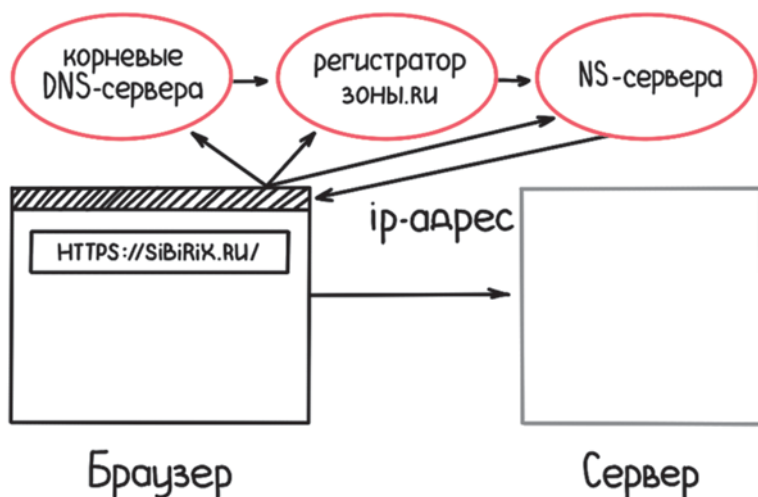
Когда вы послали запрос на сервер, там начинает работать программный код. Его пишут программисты. Как правило, на **PHP**, **Python**, **Node.js**, но вообще выбор у них богатый. Код, который работает на сервере, называется Backend.

Вообще на серверах может быть много всего веселого (сервисы очередей, базы данных, поисковые движки, почтовые и FTP-сервера, микросервисы, цветные фантики и прочая лабуда). Причем, по принципу «матрешка в матрешке», сервер в сервере. Но мы так глубоко в дебри не полезем.

В простом случае ваш Backend-программист написал какой-то **скрипт** (сценарий), типа выборки свежих новостей из базы данных, сформировал из этого страничку и отдал в ваш браузер.

12.2.7 HTML

Когда к серверу приходит запрос от браузера, он, как правило, отвечает сгенерированным HTML-кодом — Hypertext Markup Language (язык разметки **гипертекста**). Фактически это структура документа, например: шапка сайта с конкретным текстом, статьи с конкретным текстом, футер. Кроме HTML-документов по сети могут передаваться графика, видео, файлы.



HTML — это только структура страницы, без какой-либо визуальной составляющей, CSS — уже стили (например, что шапка сайта определенной высоты и синего цвета, футер внизу, статьи сделаны карточками и т. д.). JavaScript (JS) — это поведение элементов на странице, то есть обработка реакций на действия пользователя (при наведении на карточку вывести ее в расширенном виде, при клике добавить что-то в корзину и так далее).

Все, что работает на стороне сервера (выполняемый программный код при обращении к базе данных) — это бэкенд. Все, что выполняется в браузере (HTML, CSS, JS) — фронтенд.

12.2.8 COOKIE И СЕССИИ

Общение браузера с сервером, как вы уже знаете, идет по протоколу HTTP (Hypertext Transfer Protocol) — протоколу для передачи гипертекста. Он по умолчанию не содержит состояний. А значит, не помнит, какие страницы вы запрашивали до этого. То есть, если вы три часа ходили по сайту и запрашиваете какую-то следующую страницу, для сервера это выглядит, будто кто-то пришел и просит страницу в первый раз.

Для авторизаций, запоминания (кликнутых ссылок, паролей, номеров карт) и сохраненных корзинок придуман специальный механизм — сессии. Это период времени, в который сервер «помнит» про вас, что это именно вы.

Обычно он реализован через cookies. Когда к серверу приходит первый запрос от браузера пользователя, то помимо HTML, приходит уникальный идентификатор — cookie (строка из 32 символов, перемежку буквы и цифры). Этот код будет только у вас, и при любом последующем запросе браузер добавляет в него эту cookie — так сервер может понять, что это вы, это ваша авторизация, это ваша корзина и так далее.

По умолчанию сессия — это визит длиной 30 минут. Если страница за это время не обновляется и с ней не происходит никаких действий, то сессия считается завершенной. Хорошая новость: длину сессии можно изменить вручную в настройках сервера.

Но если не сделано какой-то дополнительной обработки, протокол HTTP по умолчанию не помнит, какие страницы вы посещали до этого.

12.2.9 ИТОГО

В сети есть устройства со своими адресами. Как-то соединенные (кабелями, по воздуху, спутником, голубиной почтой — неважно). И, как-то настроенной маршрутизацией между ними. Адреса запоминать неудобно, поэтому есть доменные имена. За сопоставления домена и адреса отвечает DNS. Домены обычно стоят денег. Место на сервере для вашего проекта называется хостинг, и он тоже, как правило, стоит денег.

Между устройствами ходят пакеты информации. Причем для разных задач могут быть разные протоколы (форматы обмена сообщениями). Основным протоколом интернета является HTTP(s). S говорит, что трафик

шифрованный. На сервере лежит код, который обрабатывает запросы пользователей: например, лезет в базу данных, выбирает из нее свежие новости и отдает пользователю в формате HTML. Чтобы работали авторизации, пользователей отмечают с помощью cookie и используют механизм сессий, в течение которых сервер что-то про вас помнит.

Это самая простая схема, но для понимания общих принципов пока достаточно.

12.3 ЯЗЫКИ ПРОГРАММИРОВАНИЯ И ФРЕЙМВОРКИ

Язык программирования — это то, как пишется программа, и состоит он из двух частей. Первая — синтаксис: то, что пишет программист. Вы сейчас представили циферки, как в фильме «Матрица», но нет. Программист пишет не в машинных кодах — он пишет более-менее понятный человеку код. «Если случилось это — делай вот это. Повторить два раза» и все в таком духе.

Вторая часть языка — транслятор. Компилятор или интерпретатор, который анализирует написанный программистом код и переводит его в машинные инструкции — инструкции для процессора на выполнение. Компилятор делает «перевод» заранее. Интерпретатор — на лету. Первый дает более быстрые программы, но требует предварительной подготовки. Второй работает медленнее, но более гибкий, с ним проще вносить изменения в код (кстати, это не всегда хорошо). В вебе победили интерпретаторы. В мобильных или десктоп-приложениях — пока что компиляторы.

Версии языков — это добавление к ним новых возможностей. Что-то устаревает, что-то добавляется — языки программирования развиваются, как и обычный естественный язык: добавляются новые термины, какие-то слова приходят из других языков, какие-то устаревают.

12.3.1 ВЫБОР ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

Если хотите разжечь небольшую священную войну — спросите у программистов, какой язык программирования лучше. Пожалуй, нет вменяемого ответа на этот вопрос, и все зависит от множества факторов. Есть традиционные сферы применения языков. Типа латы-

ни — для медиков и умных цитат. Можно и не на латыни поумничать, но с ней как-то больше доверия.

На текущий момент распределение примерно такое:

1. PHP — для сайтов (в том числе таких больших, как Facebook) и серверного API мобильных приложений. Не самих приложений, а их серверной части.
2. JavaScript — проник почти всюду. От сайтов до разработки мобильных и desktop-приложений (вроде Notion). Но больше всего используется для работы на фронтенде, в web-программировании.
3. Node.JS — тот же JavaScript, только на сервере. Для web-сервисов, динамичных сайтов, чатов, API и тому подобного. А еще, чтобы писать на бэкенде и фронтенде на одном языке.
4. Python — для сайтов и математики (вроде машинного обучения).
5. Java (не перепутайте с JavaScript перед своими программистами) есть везде, но в вебе ее мало и дорого. Много ее в банковских сферах и мобильных приложениях под Android (наряду с языком Kotlin).
6. Dart/Flutter — технология от Google, выглядит очень перспективной для портативных мобильных приложений.
7. Swift используется для приложений под iOS.
8. На C или C++ сайты в здравом уме никто не пишет. Но на них полно критичного по скорости кода, вроде операционных систем, драйверов и тому подобного.
9. Asp.net, Perl и еще десяток — сдохли. Аминь.

12.3.2 BACKEND. ФРЕЙМВОРКИ И CMS

Серверный фреймворк — это фактически каркас, чтобы быстрее писать код (например, PHP) на сервере (на бэкенде) и чтобы его проще было потом поддерживать. Обычно фреймворки содержат в себе заготовки типовых операций, модульную структуру и провоцируют писать весь проект в определенной парадигме. Понятно, что можно обойтись и без фреймворка, но написанный с нуля код вряд ли будет поддерживаемым: часто другой программист не хочет разбираться в том, что написал кто-то без использования стандартных подходов. Стандартизация рулит.

Поэтому в любой адекватной разработке используется либо какой-то фреймворк, либо какая-то CMS. В чем их отличия:

12.3.2.1 Фреймворки

Фреймворк — это штука, созданная программистами для программистов. С ним проще разрабатывать, но чаще всего в фреймворках нет мощной **админ-панели**, в которой было бы удобно, например, вести обработку заказов. Это не очень большая проблема, но иногда мешает.

Большая часть фреймворков — модульные (набор необходимых модулей, отвечающих за разный функционал, которые можно устанавливать отдельно по мере необходимости) и практически все бесплатные (Open Source). Часто используемые фреймворки: Laravel, Symfony, Yii, Zend Framework (Laminas). Они решают одни и те же задачи, у них разный, но в целом близкий, порог входа: некоторые написаны проще, некоторые — сложнее. Разобравшись в любом из перечисленных выше фреймворков, программист, как минимум, сможет с ходу понять код, написанный для другого (не считая внутренней сложности проекта, логики работы).

12.3.2.2 CMS (системы управления контентом)

CMS (Content Management System, системы управления контентом) сделаны программистами для НЕ программистов (контент-менеджеров, продавцов и так далее). Их отличает юзерфрендли (тут все относительно): внутри, как правило, есть админка и базовые бизнесовые штуки типа управления заказами, интернет-магазинами, скидками и так далее. Минус: кодить в них не так гибко, поскольку есть ряд ограничений, диктуемых архитектурой CMS-системы.

На западе победили бесплатные CMS, у нас пока еще держатся коммерческие (их подъедают конструкторы типа Tilda). Наиболее используемые CMS, кроме Битрикса: Joomla, Wordpress, Drupal, NetCat, ModX, Magento.

12.3.2.3 CMS или фреймворк?

Программировать на чистом языке без фреймворка — это как строить дом с нуля и без инструментов: рыть котлован, заливать фундамент, подводить коммуникации, строить стены и так далее. Шалаш или землянку построить еще как-то можно.

Использовать фреймворк — это как взять уже построенное здание без внутренних перегородок, к которому уже подведены коммуникации:

можно спроектировать внутренности, как надо и как будет удобно. При этом инструментов полно, и они очень надежные.

Кодить с CMS — это как построенное здание, но уже с перегородками, витринами, складами, которыми можно пользоваться для типовой задачи. Но для НЕтиповой придется мучиться и сносить стены внутри под свою задачу. Иногда при этом есть риск обвалить все здание.

В общем, если у вас что-то похожее на типовой интернет-магазин — берите CMS. Если социальная сеть или мощные личные кабинеты, или реально большие нагрузки — фреймворк.

12.3.2.4 Что такое «самописный сайт»

Как правило, таким грешат студенты, пишущие «за еду». Самописный сайт — это код, в котором в девяти случаях из десяти программист не следует ни одному из принципов программирования, а пишет так, как ему захочется.

Плюс фреймворков — жесткие правила: как идет обработка страниц, как идет работа с базой данных, как построены какие-то бизнес-кейсы, какие цепочки действий там происходят. И если вы разобрались с условным Laravel, то вы в любом сайте на Laravel за 30 минут поймете, что там и как работает (опять же, кроме бизнес-логики, которая может быть настолько замудренная, что без литра не понять).

Но если взять самописный сайт, понимания будет ноль: неизвестно, как идет обработка запроса, как идет работа с базой данных, где искать конфигурации... Приходится читать код «от» и «до» и разбираться. И даже если код написан хорошо и следует каким-то правилам, все равно придется погружаться в него полностью. Самописный — это еще и значит, что нет поддержки сообщества, нет доработки модулей, нет закрытия каких-то уязвимостей при обновлениях. Поэтому это один из худших вариантов, который только может быть.

12.3.2.5 Итого

Для разработчиков соблазн написать проект на фреймворке высок. Профессиональный код на фреймворке получается более красивым.

Зачем использовать условный Битрикс, когда можно сделать с точки зрения программиста почти идеальный код? Вариант имеет место быть для сложных задач, которые на CMS делать было бы дольше и доро-

же. Но иногда подход «фреймворк ван лав» оборачивается сложной поддержкой и ограничениями в дальнейшем. Опять же, зависит от разработчиков, следования стандартам кодирования, документации и аккуратности ведения проекта.

Критерий выбора фреймворка — тот, с которым вы имеете больше опыта. По большому счету, их мало что отличает: нет разницы в простоте кода, производительности или скорости разработки (последняя зависит только от скилла кодеров). На момент 2021 года у нас основной — Laravel.

12.3.3 Фронтенд

Фронтенд — код, который выполняется на стороне клиента. Грубо говоря, в вашем браузере. Анимации, проверки форм на лету и тому подобное. Как правило, пишется на JavaScript.

Мы говорили о фреймворках на стороне сервера. Так вот. Фреймворки бывают не только на бэкенде, но и на фронтенде. Разделяют реактивные и нереактивные (классические) фреймворки.

Реактивные работают по типу формул в Excel: поменяли значение в одной ячейке — поменялись значения во всех связанных формулах. Сейчас это модно. Самые популярные Vue, React, Angular, которые облегчают написание кода для взаимодействия с пользователем. Но ситуация может быстро поменяться.

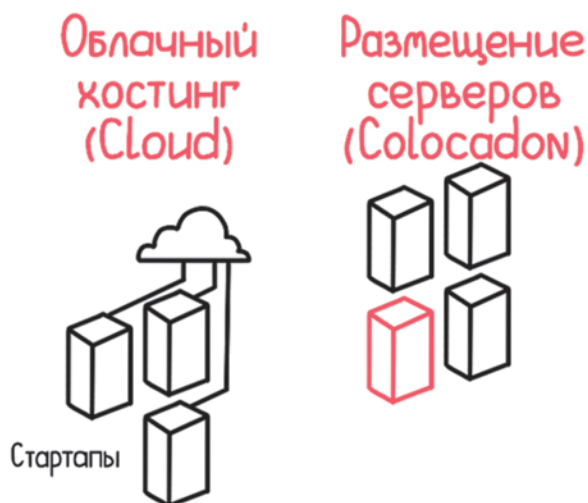
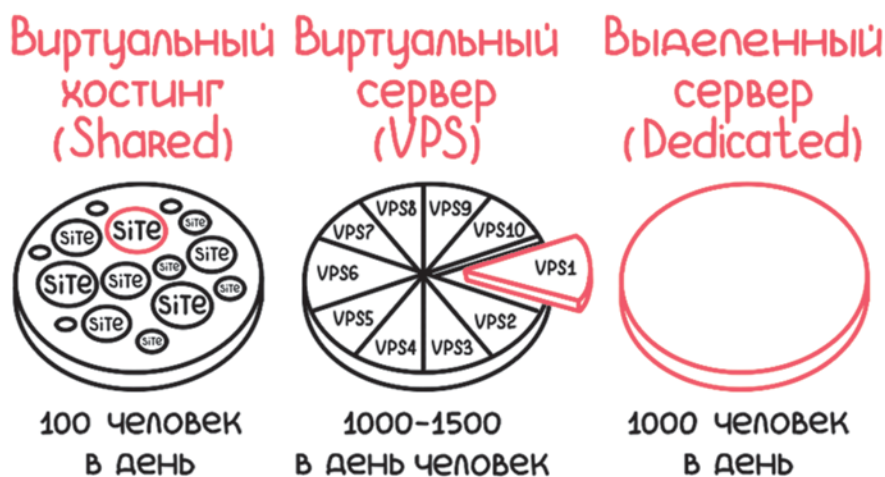
В классическом подходе с сервера прилетает HTML-страница, на элементы которой (на кнопки, например) навешиваются скрипты. Типа «при клике на кнопку открываем окно авторизации».

В реактивных фреймворках сама HTML-страница передается в очень урезанном виде. К ней же передаются нужные данные (как правило, в формате JSON). Не верстка карточки товара, например, а только его наименование и цена. И уже на стороне клиента идет отрисовка этих данных в красивый, пригодный для пользователя вид. Такой подход позволяет легче менять данные на лету, без перезагрузки страницы. И полностью разделить разработку на фронтенд и бэкенд, а затем использовать готовый бэкенд для мобильного приложения. Но его все еще не очень любят поисковые системы (хотя ситуация стремительно меняется, и существуют методы обхода этой проблемы типа SSR — Server Side Rendering).

12.4 ХОСТИНГ. СЕРВЕРНОЕ ХОЗЯЙСТВО

Хостинг — это услуга по предоставлению ресурсов для размещения файлов на сервере, постоянно находящемся в сети. Подразумевает, что на сервере, где хранятся файлы, запущено необходимое для обработки запросов к ним ПО.

А вот на каком именно сервере — есть варианты: Shared-хостинг (виртуальный), VPS (Virtual Private Server) — виртуальный частный сервер и Dedicated Server — выделенный сервер.



12.4.1 SHARED-ХОСТИНГ

Shared-хостинг — сервер, на котором размещено множество сайтов от разных клиентов. То есть, на одном сервере может лежать и ваш сайт, и сайт вашего любимого конкурента. Это как квартира в коммуналке: за вас все настраивает хостер, и от его доброй воли зависит практически все, что вы можете там делать. Разрешает что-то настраивать самостоятельно — вы настраиваете, не разрешает — ну извините.

12.4.2 ВИРТУАЛЬНЫЙ ИЛИ ВЫДЕЛЕННЫЙ СЕРВЕР

Виртуальный сервер, или VPS — когда один физический сервер распределяется между несколькими клиентами, но более изолированно. Выделенный сервер (VDS, или Dedicated) — это полное владение сервером с отдельной операционной системой и ПО.

По возможностям настройки виртуальный и выделенный сервер (VPS и VDS) — это практически одно и то же: можно поставить нужный софт. На Shared-хостинге, если вам потребуется, например, поставить генератор PDF-документов на сайт — не выйдет, если захотите там сменить поисковый движок на Sphinx — тоже, скорее всего, не прокатит. Зато на VPS и VDS — легко.

Отличия двух последних видов: VPS это как квартира в многоэтажке, а Dedicated (VDS) — как собственный дом. Dedicated-сервер — большой и мощный по ресурсам, но просто так расширить их нельзя (отжать территорию у соседа не получится — только покупать новый участок для застройки).

С VPS-сервером за счет того, что он виртуальный, практически всегда можно переключиться на более дорогой тариф с большим количеством ресурсов, при этом на самом сервере не придется ничего перенастраивать (пополнение в семье — снимите квартиру в той же многоэтажке, только побольше). Хотя бывают ограничения: наращивание мощности можно делать только в рамках одной линейки тарифов и без смены технологии виртуализации (это движок, который позволяет имитировать на одном сервере несколько операционных систем). Чаще всего VPS используют на мелких и средних проектах, а на масштабных (вроде больших федеральных интернет-магазинов) — лучше Dedicated.

Если у вас есть собственный сервер, его нужно размещать в дата-центре хостера, который будет заниматься обслуживанием и обеспе-

чивать бесперебойную работу и сохранность оборудования, а также постоянный доступ к интернету. Этот вид хостинга называется colocation. Я видел, как директор одной компании наотрез отказывался размещать сервер с сайтом за пределами офиса. Он в своем праве, и все бы хорошо, если бы не уборщица, которая по пятницам, чтобы помыть пол, выдергивала его из розетки. На все выходные.

Виртуальный выделенный сервер — это когда берется один большой и очень мощный компьютер, и его мощность делится на несколько частей: одна часть продается одному клиенту, другая — другому, третья — третьему. Если кто-то захотел увеличить мощность, он доплачивает, и его часть увеличивается.

Dedicated-сервер — это реальное «железо»: компьютер, который стоит у хостера в его дата-центре. Хостер обеспечивает ему бесперебойное питание, у него нет проблем с интернетом, и этот сервер используете только вы. А значит, просто так увеличить его ресурсы уже не получится.

Чтобы выбрать подходящий сервер, нужно оценить примерное количество посетителей и примерное количество ресурсов, которое потребуется для работы сайта. Исходя из этого можно выбрать подходящий тариф. Если у вас около 2 000 посетителей в сутки, хватит хорошего мощного VPS-сервера. Если больше 10 000 — берите Dedicated.

12.4.3 ОБЛАКА

Кроме того, очень популярны облачные серверы — объединенная система серверов, на которых располагается проект: выделяемые мощности не ограничиваются одним сервером, а распределяются сразу между несколькими серверами. Их обычно содержат крупные, технологичные игроки рынка, вроде Amazon или Яндекс.Облака.

Плюс облачного сервера — чаще всего с ним можно работать через API: он предоставляет интерфейс, и можно написать нужный скрипт, чтобы регулировать нагрузку на сервер. Например, если количество

юзеров сайта сильно разнится в течение дня, скрипт будет в зависимости от времени суток автоматически запускать дополнительные один-два-три сервера для более быстрой обработки запросов пользователей.

Минус облачных серверов: они дороже, чем VPS. Тарификация часто почасовая, и стоимость работы 24 часа в сутки за месяц выходит в два-три раза дороже, чем у аналогичных по ресурсам VPS-серверов. Зато когда вам не нужны дополнительные ресурсы, вы их и не оплачиваете. Но в целом, облако по опыту получалось в два-три раза дороже, чем выделенный сервер (речь про довольно мощные конфигурации с большим количеством трафика).

12.4.4 НАРАЩИВАЕМ МОЩНОСТЬ. СЕЙЧАС? НЕТ! КОГДА, КАК?

По мере роста и развития проектов бывает, что ресурсов сервера начинает не хватать: страницы тормозят, в пиковые часы сервер не справляется с нагрузкой и может быть недоступен. В этом случае смотрим, из-за чего конкретно тормозит сервер. Если упираемся именно в ресурс — выбираем, что дешевле: купить новый более мощный сервер, либо оптимизировать код проекта. Ищем бутылочное горлышко, пытаемся его расширить. Если уже все, что могли, оптимизировали — остается наращивать ресурс только за счет дополнительного «железа».

При переезде на новый виртуальный сервер все просто: вы покупаете более мощный тариф, и вам добавляют ресурса. С Dedicated такой номер не пройдет: придется покупать новый отдельный сервер, настраивать его с нуля, копировать данные со старого сервера на новый (а это уже не тестовый, а весь реальный контент с текущего сайта: тысячи картинок, огромная база данных и так далее). А ведь все это требует времени.

Есть показатель SLA — процент времени доступности сервера. Для сайтов он, как правило, не сильно критичен: если интернет-магазин будет недоступен полчаса в месяц — это неприятно, но не критично. Но если среди требований к проекту есть 99,97%

доступности (а это всего несколько минут простоя в месяц) даже при относительно низкой загрузке нужно предусматривать дополнительные серверы для обеспечения надежности. Лучше к этому подойти комплексно и продумать все возможные сценарии развития событий сразу: например, вместо двух VPS-серверов у одного хостера взять два у разных.

Существует несколько принципиально разных стратегий наращивания серверных мощностей. Можно выносить отдельные сервисы на отдельные серверы (например, вынести базу данных на отдельный сервер, но тут важно не проиграть по каналу связи до остальных серверов). А можно вести обработку пользователей на разных серверах, используя механизм балансировки. Кроме того, если мощностей сервера базы данных станет не хватать, ее также можно распределить по нескольким серверам (например, используя механизм шардирования — разнесения базы данных на несколько серверов).

На что еще обратить внимание при выборе хостинга:

- ▶ Отзывы — проверьте в нескольких источниках, но будьте скептичны: это могут быть проделки маркетологов или ботов.
- ▶ Работа службы поддержки (обязательно круглосуточно) и надежность хостинга.
- ▶ Место на диске: чем его больше, тем дороже будет стоить хостинг. Заранее прикидывайте, сколько будет контента на сайте, его тип (планируется ли видео) и как быстро этот объем будет расти.
- ▶ Оперативная память (оценивайте из расчета, сколько посетителей онлайн могут быть одновременно на вашем сайте, средняя оценка с потолка — 50 пользователей в онлайне заберут на себя 2Гб оперативки, но все зависит от множества факторов).
- ▶ Количество сайтов и баз данных: если вы планируете размещать несколько сайтов, проверьте, что в выбранном тарифном плане их количество равняется количеству баз данных.
- ▶ Лицензия хостера: у надежного провайдера она должна быть (и он, в свою очередь, вправе потребовать от вас документы при регистрации аккаунта, это нормально).
- ▶ Использует ли провайдер быстрые диски.

- ▶ Предоставляет ли провайдер SSL-сертификат (лучше, если бесплатно, хотя, бесплатный на VPS или выделенном сервере можно получить и самостоятельно).
- ▶ Возможности масштабирования (если предполагается рост проекта, выбирайте варианты подороже: они более гибкие).

Совет

Если вы собираете персональные данные на своем сайте (а так делает каждый, у кого есть даже самая простая форма для обратной связи с полями под ФИО и телефон), убедитесь, что данные хранятся на российских серверах. Это необходимо, чтобы не противоречить ФЗ-152 «О персональных данных».

Запросите у хостинг-провайдера места расположения серверов (на которых ваш сайт находится) и заключите с ним договор, прописав их адреса внутри. Адреса мест также пригодятся при подаче заявления в РосКомНадзор. И да, храните все документы на собственном сервере (и документы от хостинг-провайдера) — а то мало ли когда РКН решит проверку устроить.

Как вариант, попробуйте трансграничную передачу данных: у хостинг-провайдера должно быть две базы данных для хранения персональных данных: одна за рубежом, другая на территории РФ (причем, российская БД обязательно должна быть первоисточником). Посоветуйтесь со своим юристом на всякий случай :)

12.4.5 ПАНЕЛИ УПРАВЛЕНИЯ СЕРВЕРОМ

Панели управления хостингом облегчают элементарную настройку серверов. Но сильно мешают тонким настройкам. Чаще всего, установка программного обеспечения на сервере происходит с помощью конфигурационных файлов (текстовых файлов с конфигурацией разных программ). Графический интерфейс (панель управления сервером) — это прослойка, которая якобы облегчает отправку этих конфигурационных файлов. Но обычно «облегчает» — это значит «скрывает непонятное». Да, новичкам нужен простой инструмент. Для профи этого уже мало.

Если вам нужны какие-то тонкие настройки, их наверняка не будет в панели управления. Разово поправить конфигурационные файлы будет можно, но при любом изменении в панели эти изменения, скорее всего, пропадут.

Если по какой-то причине панель все-таки нужна клиенту — это нужно уточнять заранее. Понять целесообразность: если нецелесообразно — обговорить и убедить, что такой необходимости нет. Некоторые заказчики не глядя покупают хостинг и сразу панель управления к нему. Важно, чтобы потом заказчик случайно в нее не зашел и не стер одним кликом все настройки.

12.4.6 НАСТРОЙКА СЕРВЕРА. ДОСТУПЫ

Для настройки сервера нужен полный административный доступ к нему. В Linux администратор называется root-ом или root-пользователем. Соответственно, заказчик при покупке сервера должен передать разработчику логин и пароль от root-пользователя и адрес самого сервера, чтобы разработчики знали, куда подключаться. Или настроить весь софт самостоятельно по заданию разработчиков и выдать им только необходимый минимальный доступ. Доступ к серверу может быть по паролю и по ключу.

Допустим, вы получили чистый сервер. Во-первых, нужно сделать так, чтобы он откликался на какой-то домен и при обращении к нему открывался сайт. При настройке обычно устанавливается все необходимое программное обеспечение, подготавливается база данных, выставляются оптимальные настройки для системы управления или фреймворка, а также создаются отдельные доступы: доступ по SSH, чтобы подключиться и выложить программный код, и доступ к базе данных для выкладки данных сайта.

Примерно в 50% проектов в заказной разработке заказчик делегирует настройку серверов подрядчику. Тут важна порядочность и аккуратность обращения с паролями. Это чувствительное место. Позаботьтесь, чтобы доступы были у минимального количества людей. И только те, которые нужны. Используйте специальные парольники с распределением прав доступа. Используйте сложные пароли. Регулярно их меняйте. Да, это хлопотно.

12.4.7 БЭКАП СЕРВЕРОВ

Бэкап — это создание резервной копии. Обычно резервные копии серверов делаются по трем причинам:

- ▶ защита от неумышленного повреждения: например, когда кто-то случайно удалил **инфоблок** или заказ;
- ▶ защита от умышленного повреждения: когда сайт взломали и намеренно удалили с него что-либо;
- ▶ защита от выхода из строя оборудования: компьютеры все-таки физические объекты, и у них иногда умирают жесткие диски со всей информацией.

Одно из преимуществ Dedicated-серверов перед VPS: у первых обычно не один жесткий диск, а два, которые являются полной копией друг друга (зеркальный raid). Если вдруг откажет один, со второго данные можно восстановить.

Если мы говорим про защиту от умышленного и неумышленного повреждения, бэкапы можно делать на том же сервере: сервер раз в сутки полностью копирует и архивирует базу данных и настройки всех установленных программ, а затем складывает их в отдельную папку у себя же. Если речь идет о защите от поломки оборудования (по нашему опыту, раз в год на каком-то проекте «падает» жесткий диск), и проект при этом достаточно крупный — имеет смысл купить место на отдельном сервере для резервных копий.

В моей практике какое только оборудование не горело на серверах. Включая два data-центра. Делайте бекапы и копируйте их в разные дата-центры, господа!

Чаще всего бэкап может развернуть только администратор сервера, а админ сайта (заказчик) — нет. Хотя есть бэкапы на уровне систем управления контентом, и их тоже рекомендуется настроить. Особенно, если контент-менеджеры очень талантливые и готовы с добавлением пары новостей снести половину сайта. Так бывает.

Если на сервере много места, можно и «поиграться» с частотой резервного копирования. Если проект крупный, и в нем используется

более одного сервера (отдельные серверы для базы данных и отдельные для выполнения кода), можно настроить их так, чтобы они друг на друга делали резервные копии — и тогда дополнительное место для бэкапов не понадобится. Следите за местом.

12.4.7.1 Кот Бэкап Шредингера

Посадим кошку в стальной сейф вместе с адской машиной (защищенной от кошки). В счетчик Гейгера положена крупинка радиоактивного вещества, столь малая, что за час может распасться один из атомов, но с такой же вероятностью может не распасться ни один. Если атом распадается, счетчик через реле приведет в действие молоточек, который разобьет колбу с синильной кислотой. Предоставив эту систему самой себе в течение часа, мы скажем, что кошка еще жива, если за это время не распался ни один атом. Первый же распад привел бы к отравлению кошки. ψ -функция всей системы выразила бы это тем, что живая и мертвая кошка (с позволения сказать) смешаны или размазаны в одинаковых пропорциях.

Классический мысленный эксперимент про Кота Шредингера.

Допустим, ваш админ настроил бэкап. Гарантирует ли это, что в случае выхода из строя какой-нибудь железки у вас все будет хорошо? Неизвестно! Пока не развернете, не узнаете. Причин тому может быть масса:

1. Неправильная настройка бэкапов. Что-то бэкапится, но не все. Не полностью или не так. Например, нет файлов кода. Или файлов данных. Или картинок.
2. Бэкапы делаются, но в них не хватает конфигурационных файлов серверов. В итоге данные восстановить можно, но это займет много времени.
3. Бэкапы не распаковываются. Бывает.
4. Бэкапы делаются, но не передаются по сети на запасные сервера. Например, потому, что там пару месяцев назад кончилось место или поменялись доступы.

Из курьезов, часть 1. Как-то для одного внутреннего сервиса я попросил написать документацию, по которой можно было восстановить его работоспособность на другом узле в случае аварии. Программист отчитался о готовности. Я говорю «ок» и выключаю сервер. Мол, вос-

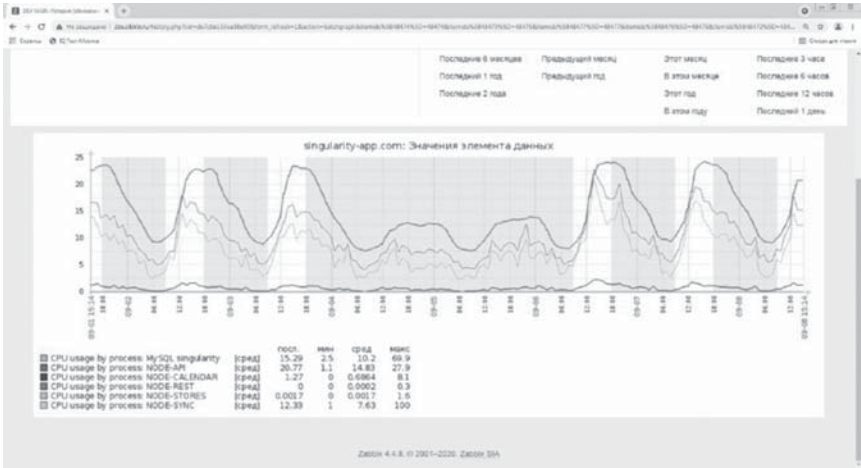
становливай. Программист смотрит растерянными глазами и говорит: «А документация-то на сервере». Упс.

Из курьезов, часть 2. Через несколько лет я в очередной раз, проделывая такой же фокус (документация → выключили → восстанавливай), засекаю время на восстановление системы. Получилось за пару часов. Долго, но жить можно. Была пятница, мне было не до того, а на выходные я про это благополучно забыл. Через пару недель у нас начались странные аномалии: пользователи время от времени получали письма от нашего сервера, которые мы не отправляли. Долгие поиски источника проблемы выявили следующее. Систему-то из бэкапа мы развернули. Целиком. А вот выключить ее забыли. Она себе тихонечко работала с устаревшими данными пользователей и так же тихонечко отправляла несколько сотен писем в день с неактуальными уведомлениями. Упс.

В общем, бэкапы должны быть. Их нужно периодически проверять (учебные тревоги). Иначе, когда дело дойдет до дела, бэкап будет то ли жив, то ли мертв. Как тот котик.

12.4.8 СИСТЕМЫ МОНИТОРИНГА СЕРВЕРОВ

Zabbix — самая популярная система мониторинга серверов. Бесплатная и простая. Можно настроить мониторинг разных событий (вроде всплеска нагрузки или кончающегося места на дисках). Вывести на плазменную панель. Загорелось красненьким или желтеньким — надо что-то делать.



Как выглядит Zabbix изнутри

По критичным событиям (вроде проблем в модуле оплаты) можно добавить уведомления в Телеграм. А также прописать и отработать регламент по реакции на такие инциденты. Назначить ответственных. И провести пару военных тревог. Первого января, в пять утра, например. Шутка.

12.4.9 КОНТЕЙНЕРЫ. DOCKER, KUBERNETES

Очень популярны системы контейнеризации, позволяющие изолировать в рамках одного сервера установленное на нем программное обеспечение. Если продолжать аналогию с домом и жильцами — мы даем каждому жильцу по отдельной комнате и возможность общаться между собой. Причем, если этого захочется — мы берем и целиком переносим квартиру в другой дом. Жители особо ничего не заметят. Или можем скопировать квартиру вместе с жителем и перекрасить в ней стены. Почти такая же квартирка, только стены — зеленые.

Контейнеры имеют смысл, и это удобно. Особенно для конфигурирования и масштабирования. Самые популярные системы контейнеризации: Docker (он работает на одном узле) и Kubernetes (предназначен для работы через кластер серверов).

12.4.10 БАЗЫ ДАННЫХ

Никакая система не имеет смысла без данных. Для их организации используются системы баз данных. Самые популярные движки баз данных в web на данный момент:

1. Реляционные — данные хранятся в структурированных таблицах: MySQL, PostgreSQL
2. Нереляционные — данные хранятся в виде ключ-значение: MongoDB, Redis.

Выбор конкретной базы и ее внутреннее устройство нужно делать в самом начале разработки проекта. Самым опытным кодером в команде.

Как? Вот есть у вас техническое задание: в нем товары, покупатели, справочники, магазины, города и так далее. Но пока здесь не выделены конкретные сущности, связи между этими сущностями не расписаны, поля для них тоже не указаны. А это все требуется. Поэтому одна из первых задач проектирования структуры базы данных — составить

с помощью разработчика схему со структурой сущностей, их полей и взаимосвязей. В первую очередь, чтобы понимать, как это происходит внутри (и чтобы потом не оказалось внезапно, что у остатков товара есть зависимость от магазина, а это не учли).

Фактически, чтобы создать структуру базы данных, разработчику нужно прочитать всю спецификацию, загрузить проект в голову, понять, что там есть, предположить, какие там есть связи, задать менеджеру проекта дополнительные вопросы и только потом нарисовать и создать таблицы. Да, что-то можно доработать и дописать потом. Не проблема. Но если что-то не учли по-крупному, это придется, как минимум, переделывать.

Итак. Выбор движка базы данных зависит от задачи. Выбор внутренней структуры хранения данных — тоже. Доверьте это дело самому опытному разработчику в команде.

12.4.11 КРОНЫ. ВЫПОЛНЕНИЕ ПО РАСПИСАНИЮ

Для многих digital-проектов надо периодически выполнять какие-то задачи: например, провести обмен с 1С-кой, увеличить какой-нибудь счетчик на главной странице, собрать еще откуда-то данные. Cron в помощь. Это специальная программа, запущенная на стороне сервера, которая выполняет периодические задачи.

Оговорка для Битриксойдов: Битрикс придумал свой механизм для периодических задач — «Агенты». Может работать как на механизмах Cron, так и на хитах — когда формируется страница, тут же проверяется, не надо ли выполнить какое-то задание по расписанию. Ну такое... Время выполнения не гарантируется, скорость работы страниц падает, долгие задания так не запускать.

12.4.12 ЭЛЕКТРОПОЧТА

Электронная почта работает через три ключевых протокола: POP3, IMAP, SMTP.

POP3 используется для загрузки сообщений на локальный почтовый клиент с последующим удалением с сервера.

IMAP так же, как и POP3, используется для получения сообщений электронной почты. Но POP3 получил сообщения и разорвал соеди-

нение, а IMAP продолжает общаться с сервером во время работы почтовой программы, синхронно. Сами тексты писем в IMAP загружаются только по требованию.

SMTP — протокол передачи почты, используется для отправки сообщений с локального компьютера на удаленный сервер и далее для обмена почтой между серверами, для доставки почты в ящик получателя.

Есть тысячи причин, почему отправленное письмо может не дойти до адресата: от неправильной настройки серверов до попадания в черные списки из-за слишком активных и бездумных маркетологов. Контур очень чувствительный для заказчика. Если не хотите себе долгоиграющий геморрой с разбором «почему письма не доходят», рекомендую подключить к домену почтовые сервера Яндекс.Почты (бесплатно) или у Gmail (за деньги). Там с доставкой и управлением почтовыми ящиками все более-менее в порядке.

Еще один вариант — установить на сервере специальные программы для обслуживания почтового сервера и настроить их. Но это занимает много времени, и объективно не стоит потраченных усилий. Другие почтовики больше доверяют письмам, отправленным от Яндекса или Гугла, чем письмам от какого-то mx.vasiapupkin.ru.

12.4.12.1 Вы попали в спам? SPF и DKIM

Это DNS-записи, обеспечивающие безопасность почты.

SPF описывает, кому разрешено отправлять письма от этого домена — в этой записи указывается либо IP-адрес сервера, либо IP-адрес каких-то сторонних почтовых служб (сервисов для рассылок вроде Unisender).

Когда почтовый сервер получает письмо, где указан какой-то домен, он идет на этот домен и проверяет, есть ли там SPF-запись. Смотрит, кому разрешено отправлять письма, «прикидываясь» этим доменом. Если источник попадает в этот список — гуд, письмо «хорошее». Если SPF-запись не настроена, либо там не указан IP-адрес — очень высокая вероятность, что письмо попадет в спам.

DKIM — это криптографическая подпись письма. Каждое отправляемое сервером письмо шифруется криптографическим ключом. В этой записи содержится публичная часть ключа: почтовик получил письмо, проверил подпись публичной части, если она совпадает — значит, это точно тот сервер.

Бывает, что на проекте SPF-запись сразу не настраивают: спам никто не рассылает, письма шлют изредка, почтовики сайту доверяют. К заказчику приходят SEO-шники и говорят, что хотят настроить регулярные рассылки через сервис — и для этого сами настраивают SPF, где прописывают нужный сервис для рассылок. Из-за этих изменений доверие почтовиков к письмам падает, и все любовно приготовленные рассылки начинают попадать в спам.

Поэтому, если есть доступ к **NS-серверам**, лучше SPF-записи настраивать сразу. В любом случае необходимо настраивать SPF корректно, причем вместе с DKIM и, возможно, с DMARC (технической спецификацией, предназначенной для снижения количества спамовых и фишинговых электронных писем).

12.4.13 SSL-СЕРТИФИКАТЫ. НЮАНСЫ

Сейчас массовый тренд на то, чтобы сайты без шифрования помечались как небезопасные. Но это не значит, что все такие сайты могут украсть ваши данные. Браузерам проще запугать пользователя, чтобы тот не ходил на незашифрованные сайты, чем спровоцировать ситуацию, когда у кого-то однажды реально украдут данные на таком сайте. Превентивная мера, если хотите.

HTTPS-шифрование — это end-to-end несимметричное шифрование. Когда пользователь отправляет запрос, на его стороне тот шифруется, и далее на сервер передаются зашифрованные данные, где они расшифровываются. Никто (по идее) не сможет вклиниться в процесс передачи этой информации и украсть ее. Это особенно полезно, когда вы сидите через открытый Wi-Fi в какой-нибудь кафешке. Также это защитит от недобросовестных действий со стороны провайдера либо государства. По HTTPS можно понять, к какому домену вы обращаетесь, но неясно, какую страницу вы запрашиваете и какие данные вы передаете. Так вот, за HTTPS-шифрование как раз отвечают SSL-сертификаты.

Сертификаты бывают платные и бесплатные, на один домен или на поддомены (**wildcard-сертификат**). Раньше получение сертификата было достаточно сложным процессом: приходилось подтверждать владение сайтом, для владения крутым сертификатом требовались паспортные данные, данные об организации — вплоть до выписок из банков. Все это мешало и сильно замедляло процесс внедрения шифрования.

Несколько лет назад появился сервис Let's Encrypt, который предоставляет бесплатные сертификаты почти кому угодно. Для большей части сайтов достаточно простого шифрования без какого-либо подтверждения. Подтверждение раньше бывало разным: либо в адресной строке появлялся зеленый замочек, либо высвечивалось зеленым название организации плюс замочек. Сейчас от этого постепенно отказываются: теперь любой шифрованный сайт помечается иконкой замочка.

SSL-сертификат, по большому счету, — это торговля ~~воздухом~~ доверием. Есть центры сертификации — большие организации, которым доверяет большинство крупных компаний. Сами они сертификаты не выдают, а лишь сертифицируют более мелкие региональные организации, которые выдают SSL конечным пользователям. Такая иерархия позволяет быстро отзывать сертификаты, если компания, их выпускающая, окажется «нечистой» — все выданные ею SSL сразу же окажутся невалидными.

Сам SSL-сертификат — это просто сгенерированный ключ: текстовые файлы с набором букв и цифр, из двух половинок. Отличное приобретение за тысячи и десятки тысяч рублей в год. По факту вы платите только за то, что сделали это через организацию, которой все как бы доверяют. При этом Let's Encrypt — тоже доверенная компания, но она выпускает их бесплатно. Но есть и минусы. Например, ограниченный срок годности сертификата (три месяца) — частая смена обусловлена безопасностью.

И все-таки в Let's Encrypt сертификат выдается не совсем просто так. Сначала на сервер устанавливается специальная программа, указывается конкретный домен, а затем отправляется запрос на серверы Let's Encrypt, и сервер «стучится» к этому домену. Если по этому домену ответит установленная на сервере программа — отлично, владение сервером подтверждено. Также у Let's Encrypt есть процедура проверки владения доменом.

Для получения сертификатов есть несколько разных программ: сами Let's Encrypt просто опубликовали протокол работы с ними, а кто захотел — написал свою программу, реализующую данный протокол. Самая популярная из программ для получения сертификатов — CertBot.

Если проект сначала запустился на одном домене, а потом заказчик внезапно понял, что ему нужны поддомены по городам, ему, скорее всего, придется покупать новый сертификат. Так называемый wildcard-сертификат, распространяющий свое действие на поддомены. «Расширить» сертификат нельзя, только выпустить заново.

12.4.14 ДЕПЛОЙ. РАЗВЕРТЫВАНИЕ КОДА НА СЕРВЕРЕ

Деплой — процедура развертывания кода на сервере.

Что нужно для деплоя в самом простом случае:

- ▶ Название домена.
- ▶ SSH: логин, пароль, адрес и порт для подключения.
- ▶ MySQL: логин, пароль, адрес для подключения, имя базы данных.
- ▶ Управление NS-серверами: адрес панели, логин, пароль.
- ▶ На всякий случай: панель управления хостингом — адрес панели, логин, пароль (заменяет FTP/SSH и MySQL).

Для начала, конечно, нужен настроенный сервер. Далее программист копирует на рабочий сервер код и **ядро сайта**, выкладывает базу данных (код и база данных выкладываются, как правило, специальным скриптом деплоя). При этом создается резервная копия сайта, которая переносится на настроенный сервер, где автоматически разворачивается.

Затем программист проверяет, что все корректно перенесено на рабочий сервер, проверяет все интеграции и работу установленных модулей — и, в общем-то, первичная выкладка проекта на этом заканчивается.

С дальнейшими выкладками все немного сложнее. Во время доработок чаще всего вносятся изменения в базу данных: добавляются новые таблицы или поля к заказу, настраиваются интеграции с какими-то сервисами. При этом база данных на уже выложенном сайте не статична: заказчик загружает контент, а если это интернет-магазин — там появляются реальные заказы. Эти данные ни в коем случае нельзя потерять, поэтому нельзя просто взять и скопировать базу данных с тестового сервера на рабочий.

Спасает механизм под названием «миграция базы данных»: программист кодом должен описать, какие изменения следует внести в базу данных. Для создания миграции по таблицам базы есть автоматизированные инструменты. Но с миграцией не все так просто. Например, настройки вроде платежных систем сложно или невозможно перенести механизмом миграций. Будет ручная работа.

В общем, при деплое есть риск что-то потерять, и обязательно нужен смоук-тест (тест самого базового функционала, с которым юзер может исполнить свой пользовательский сценарий без проблем). Он нужен, чтобы убедиться, что не отпало что-то, что было задето изменениями.

При частых и ответственных деплоях процедуру тестирования лучше вообще полностью автоматизировать. Для этого придуманы подходы `continuous integration` и `continuous delivery`. Важно автоматизировать рутину, чтобы исключить человеческий фактор. Покрывать автоматическими тестами критические части проекта. Однако этот подход целесообразно использовать на очень крупных и постоянно меняющихся проектах, так как он влечет серьезные транзакционные затраты (это дорого).

12.4.15 АДМИНЫ И ДОСТУПЫ

Верить в наше время нельзя никому, даже себе. Мне — можно!

Мюллер

Администратор, делающий все нужные вам настройки, к сожалению, сам по себе является точкой отказа. В том числе по безопасности. Увы, в моей практике и в практике моих клиентов были случаи, когда обиженный админ вредил, угрожал, портил информацию и шантажировал заказчиков. И даже оказывался внештатным сотрудником ФСБ. Мерзко, низко, противно, коварно. Но на сегодняшний день — это реальность.

Совет

Чем больше проектов, тем больше всяких паролей и доступов. Надеяться на свою память, — как минимум, безответственно. Два-три пароля вы, может, и запомните, но, когда их несколько

десятков, — надо обеспечить их надежное хранение и управление доступом к ним. Персональных менеджеров паролей тут будет недостаточно. За это отвечают корпоративные менеджеры паролей.

Кстати, о доступах. Для обычных действий разработчикам и сотрудникам компании достаточно ограниченных доступов к серверу, только к папке проекта и небольшому набору служебных команд. Главные root-права от сервера должны быть максимально ограничены и выдаваться только самым доверенным и надежным людям, поскольку с таким доступом можно как все починить, так и все поломать.

Итак. Для управления серверным хозяйством, особенно если проектов много, нужен квалифицированный, выделенный человек с очень большими правами доступа к чувствительным для бизнеса местам. Где будете такого брать — не знаю. Но по объявлению в газете я бы не рискнул.

12.5 СКОРОСТЬ ЗАГРУЗКИ

У Гугла рейтинг сайта в поиске уже несколько лет зависит от параметров самого сайта: скорости его загрузки, адаптации для мобилок, общего веса и отсутствия лишнего контента при загрузке.

Проверить скорость загрузки сайта легко с помощью сервиса Google PageSpeed (далее по тексту — GPS), который оценивает скорость в баллах:

- ▶ 0–49 — очень медленно (красная зона);
- ▶ 50–89 — медленно (желтая зона);
- ▶ 90–100 — быстро (зеленая зона).

Что может снижать скорость загрузки страниц:

1. Много тяжелых картинок. К сожалению, неопытные контент-менеджеры часто грешат загрузкой фотографий внутрь страницы без их сжатия.

2. Общий объем скриптов JavaScript.
3. Сторонние скрипты, вроде счетчиков или онлайн-чатов.
4. Использование Google Tag Manager и встраивание дополнительных скриптов через него. Частный случай пункта № 3.
5. Добавление видео с **видеохостинга** через `<iframe>`.
6. Плагины, многочисленные слайдеры, анимации, 3D-модели. Красота требует жертв. Но когда дело доходит до жертв, часто становится не до красоты.
7. Плохой код.
8. Плохой хостинг (сервер и канал).

Даже если на десктопе у вас все хорошо, для сайта и мобильных совсем другие правила. Жестче и строже.

Об оптимизации скриптов должны думать программисты, начиная уже с этапа верстки. Но бездумная вставка всяких сторонних кодов мгновенно убивает все оптимизации.

12.5.1 КАК УСКОРИТЬ РАБОТУ СТРАНИЦ

Тут, опять же, все зависит от множества факторов. Нужно искать причину тормозов. Бутылочное горлышко. И решать конкретно. Но вот несколько точек, которые можно проработать.

12.5.1.1 Использовать CDN

CDN, или Content Delivery Network, — служба быстрой доставки контента по всему миру. С ее помощью московский сайт загрузится одинаково шустро на соседней улице или за океаном. На всех континентах рассредоточено огромное количество серверов, которые могут хранить закешированные (сохраненные) копии вашего сайта. Когда пользователь запрашивает необходимый контент, данные моментально загружаются из ближайшего дата-центра, а не с основного далекого веб-сервера. Нет необходимости каждый раз пересылать большие массивы информации на другой конец Земли.

12.5.1.2 Использовать AMP и турбо-страницы Яндекса

Очень спорные технологии, превращающие ваши сайты в доноры контента. Есть мнение, что Гугл специально сделал ставку на скорость загрузки, чтобы продвигать свое детище Accelerated Mobile Pages — или просто AMP — (где может крутить только гугловскую рекламу). Но это не точно :) Российский аналог — Яндекс.Турбо (Яндекс тоже учитывает скорость загрузки при ранжировании). Зачастую страницы по этим технологиям получаются убогими.

12.5.1.3 Сжимать изображения

Базовое правило контент-менеджеров, но многие им все равно пренебрегают. Сжатие уменьшает вес изображений без потери качества — для этого можно использовать сервисы, вроде TinyPNG или Optimizilla.

12.5.1.4 Использовать изображения в формате WebP

Помимо известных форматов для изображений .png и .jpg, в 2010-м благодаря Google появился формат .webp, который поддерживался не всеми браузерами и не всеми их версиями. Долгое время эппловский Safari его игнорировал (с 2020-го стал допускать). Поэтому приходится хитрить: олдскульным браузерам отдавать картинки в старых форматах, а тем, кто уже в теме, — отдавать в .webp. Приятная новость: этот процесс можно автоматизировать с помощью модуля ngx_pagespeed.

Из плюсов формата: он легче, чем PNG и JPEG. Это значит, что на большей части картинок он экономит немного места, но на некоторых графических файлах (одноцветных, с прозрачными областями) экономия получается значительная. Например, png-файл с прозрачностью, который весит 200 Кб, в формате .webp может весить всего 20 Кб.

Конвертировать изображения в .webp можно с помощью сервиса Online-Convert.

12.5.1.5 Установить модуль ngx_pagespeed

Для своих проектов мы всегда устанавливаем на сервера модуль для автоматической оптимизации картинок, поскольку основная проблема с GPS, из-за которой сайт не попадает в зеленую зону, — слишком большой объем картинок в передаваемом трафике.

Но он работает не только с изображениями — модуль ускоряет сайт и сокращает время загрузки страниц, автоматически применяя передовые методы веб-производительности к страницам и связанным ресурсам (CSS, JavaScript, изображениям). При этом не требуется менять существующий контент или рабочий процесс.

Помимо оптимизации изображений, модуль может переписать HTML, отсрочить загрузку JavaScript и изображений, продлить «живучесть» кэша и понять, какой перед ним браузер, чтобы на лету конвертировать картинку в формат .webp, сделав ее меньшей по весу.

12.5.1.6 Оптимизировать серверный код

Почистить и оптимизировать запросы, добавить индексы, положить в кэш то, что должно там лежать, поджать картинки, разгрузить канал. Общий принцип: профилируем, замеряем скорость работы отдельных фрагментов кода. Если какое-то место вызывает вопросы — целенаправленно его оптимизируем до достижения целевых показателей. **Сначала зафиксируйте текущие показатели и обозначьте целевые.** Просто так сказать программисту: «Ну ты там пооптимизируй чего-нибудь» — это практически гарантированно слить рабочее время в пустоту.

12.5.1.7 Взять более мощный сервер

Тоже решение, причем иногда оно быстрее и дешевле, чем полировка кода. Тяжелым сайтам с большим трафиком гораздо уютнее на мощном железе с запасом места под столь нужный им кэш.

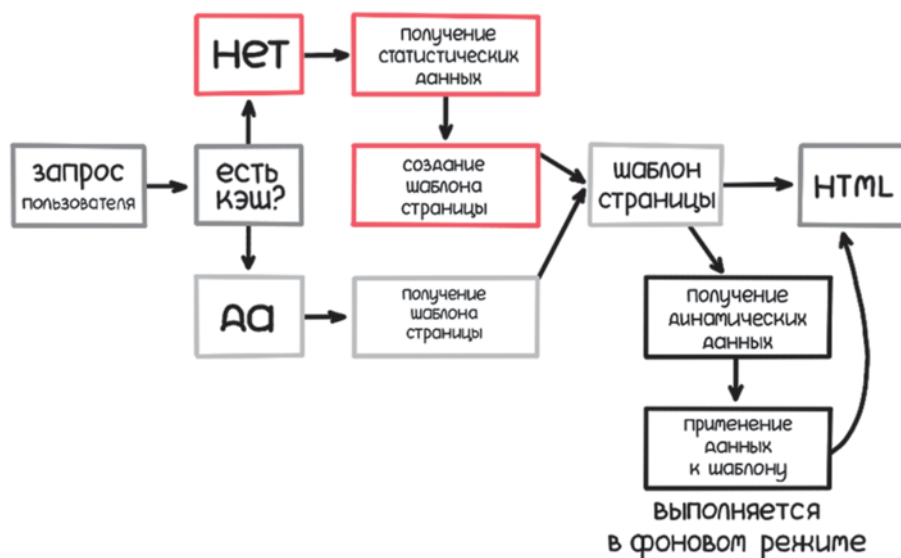
Если вы используете Shared-хостинг и модуль для оптимизации поставить там невозможно, то GPS может дать 30 баллов из 100 только потому, что картинок на сайте 5 Мб. Но если использовать VPS-сервер и настроить на нем оптимизацию изображений, можно легко оказаться в зеленой зоне (от 80 баллов).

12.5.1.8 Использовать отложенную (ленивую) загрузку

Картинки или часть данных грузим уже после отрисовки основного контента страницы.

12.5.1.9 Использовать композитный подход

Расширенная идея отложенной загрузки. Это технология, когда страница разделяется на статическую и динамическую части, где статическая кешируется и отображается мгновенно, а динамическая подгружается в фоновом режиме.



Подход лучше работает, если динамического контента мало: на корпоративных сайтах, например. В интернет-магазинах с частым применением фильтров, персональными скидками и ценами на товары с композитом будут проблемы. Например, на каждый вариант фильтра будет создаваться свой кэш — файлов кэша станет слишком много, и сервер может «лечь». А актуальность этого создаваемого кэша может быть сомнительной, например, если сортировка зависит от остатков товара, файлы кэша будут создаваться заново каждый раз. Другая проблема: обычно при композитном подходе в каталоге на товарах стоят «заглушки» — если пользователю понадобится полная информация о товаре, он будет ждать ее загрузки так же, как на обычном сайте.

В целом подход рабочий, но все зависит от множества факторов. Как обычно.

12.5.2 РЕАКТИВНОСТЬ

Мы уже обсуждали реактивный подход (как в Excel: меняем циферку в ячейке — тут же пересчитали все зависимости). Реактивные сайты — настоящий хайп и вершина технологий на текущий день. Если все сделать правильно, то они будут и молниеносно загружаться и нравиться поисковым системам. Для создания реактивного сайта используется связка технологий: Vue или React.js

React.js — библиотека JavaScript и каркас для веб-приложений, созданный Фейсбуком. Постоянно развивается, имеет развитое сообщество.

Ближайший сильный конкурент — Vue.js. Чертовски хорош и имеет большое комьюнити. А также низкий порог входа в разработку.

SPA, Single-page application — это веб-приложение или сайт, которые могут использовать единый HTML-документ как оболочку для всех страниц. Взаимодействие с пользователем происходит через динамически подгружаемые HTML, CSS, JavaScript.

SSR, Server Side Rendering — **серверный рендеринг** (о нем чуть ниже).

Реактивные сайты на Vue.js (или React.js) работают несколько по-другому: вместо отрисовки HTML-страниц, кода бизнес-логики и шаблонов пишется JavaScript. Зачастую при загрузке страницы сначала грузится пустой «контейнер» (HTML), а затем Vue/React отрисовывает в нем содержимое страницы. Если какая-то часть страницы меняется — заново подгружаются только данные для этой части страницы (а не вся страница), а обновление и отрисовка нужных частей страницы осуществляется на стороне клиента, в его браузере. Это ускоряет загрузку и рендеринг, делает сайты очень отзывчивыми.

Использование подходов **Redux** и **MobX** (эти страшные слова вы можете услышать от программистов) позволяет динамически менять содержимое страниц в ответ на действия пользователей (поменяли

что-то в одном месте — мгновенно изменения отразились во всех связанных местах, без перезагрузки страниц). Это позволяет строить мощные интерфейсы и хорошо работает в личных кабинетах, системах управления, CRM-системах и так далее.

Использование технологии PWA (прогрессивных веб-приложений) позволяет также кэшировать данные на стороне клиента и не обращаться за ними на сервер при повторной необходимости. Но появляется проблема: для поисковиков страницы сайта кажутся пустыми, они видят только каркас страницы — так в топ выдачи не попасть.

Чтобы решить эту проблему, есть серверный рендеринг: весь нужный HTML-код для загрузки страницы по запросу поисковой системы генерируется на сервере. Сайт отображается мгновенно, и поисковики довольны: видят все содержимое сразу. К счастью, поисковики становятся умнее и проблема потихоньку уходит в прошлое.

Плюсы реактивных сайтов (SPA+SSR):

- ▶ контент загружается быстрее (особенно при плохом интернете и на медленных устройствах);
- ▶ улучшает ранжирование в поисковиках (поисковые боты будут видеть полностью отрендеренную страницу);
- ▶ бонусом: готовый REST API, на базе которого легче впоследствии сделать мобильное приложение;
- ▶ все это выглядит перспективно.

Ограничение: вам нужны будут толковые и опытные программисты.

На текущий момент рекомендация такая: делайте реактивные сайты и следите за тенденциями.

12.6 СИСТЕМЫ КОНТРОЛЯ ВЕРСИЙ

В любом сложном и живом проекте вы столкнетесь с необходимостью контроля версий. Особенно это актуально для командной разработки программных продуктов.

Система управления версиями — программное обеспечение для облегчения работы с изменяющейся информацией, в частности с кодом проекта. Она позволяет хранить несколько версий одного и того же документа, при необходимости возвращаться к более ранним версиям, определять, кто и когда сделал то или иное изменение, делать «ветки»

кода для параллельной работы над несколькими функциями и решать конфликты, когда два разработчика вносили изменения в один и тот же файл.

В системе контроля не надо самостоятельно отслеживать ветви кода и изучать примечания к ним. Вместо этого используется центральный **репозиторий**, где все структурировано.

Главное отличие между системами контроля версий — в их типе: они бывают клиент-серверными и децентрализованными (p2p). Также они отличаются наличием или отсутствием центрального репозитория (сервера): бывает, что вместо него используется локальное хранилище, где хранятся копии кода. Среди других отличий: быстроедействие, функциональность и порог входа для разработчика.

До сих пор встречаются компании, в которых разработка идет без использования систем контроля версий. Это довольно прискорбно, особенно если проект большой, и работа ведется командно. Грустно смотреть, как здоровые дяди обмениваются файлами по FTP.

Если такое наблюдается на вашем проекте — подтолкните разработчиков к использованию системы контроля версий. И на начальных этапах следите, чтобы они не забывали делать ветки для разработки, например, отдельных спринтов.

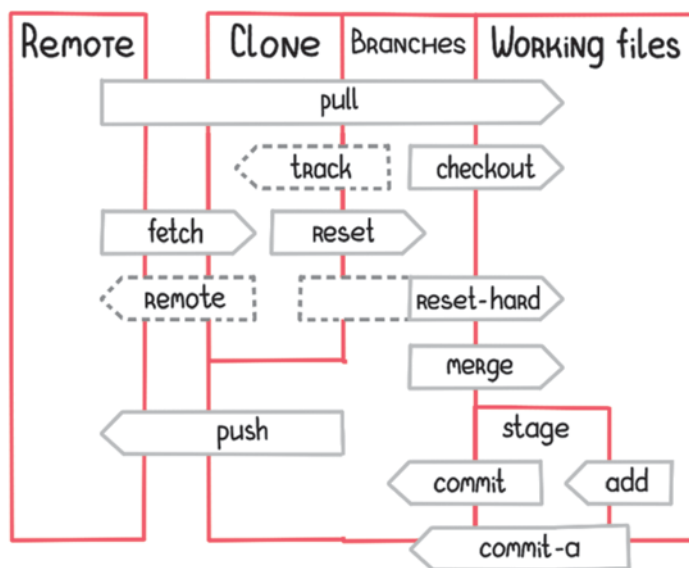
В 99% случаев на сегодняшний день вы столкнетесь с системой контроля версий Git. По сути, она стала стандартом. Остальное — экзотика.

Git создавалась для управления разработкой ядра Linux. Сегодня Git может работать и на Unix-подобных системах (как MacOS), и на Windows с использованием пакета mSysGit, но поскольку система создавалась под Linux, на этой ОС она работает быстрее всего.

Главное отличие: Git — распределенная система. В ней на каждом отдельном компьютере разработчика хранится полная копия с историей изменения всех файлов проекта.

С одной стороны, это очень хорошо: большая часть операций, например, просмотр истории изменения файлов, проходит значительно быстрее. И если сгорел сервер с главным хранилищем — можно будет восстановить все данные, включая историю изменения файлов из любой локальной рабочей копии.

С другой стороны, это изредка порождает у программистов паттерн поведения, когда они всю разработку ведут локально, не отдавая



на основной сервер. Или отдавая код только в самом конце, когда изменений сделано очень много и слияния делать довольно сложно. Решение: договориться, что перед стендапами программисты должны сделать выгрузку, push на главный сервер, и подготовить рабочую ветку к ежедневному тестированию.

И хотя эта система распределенная, как правило, все равно существует какой-то центральный хостинг или репозиторий. Вы можете использовать собственные сервера для хранения главного репозитория. Можете использовать GitLab — шикарную систему управления репозиториями кода для Git с собственной Вики, системой отслеживания ошибок, CI/CD пайплайном и другими функциями (есть платная и бесплатные версии). Или использовать один из многочисленных хостингов исходных кодов проектов: например, GitHub или Bitbucket. Оба проекта очень достойные, позволяют создавать хранилища исходных кодов, обработку обращений пользователей и совместную работу над проектами сторонних программистов.

Плюсы Git:

- ▶ распределенное хранение данных;
- ▶ лучше подходит для команд, работающих над одним проектом из разных мест;

- ▶ легко восстановить в случае потери локальных данных;
- ▶ быстрое действие;
- ▶ работа с ветвями кода — более простое слияние.

Существует несколько стратегий использования GIT в зависимости от зрелости продукта и жизненного цикла разработки. В любом случае стоит посмотреть официальную документацию и разобраться с основными командами, чтобы не впасть в ступор от фраз «Ветка», «Мердж-реквест» или «А мы ее черри-пикнем!». Оставляю как задание на дом.

12.7 АДАПТИВНЫЕ И МОБИЛЬНЫЕ САЙТЫ

Вспомните, когда вы заходили на какой-то сайт с мобильного, а страницы были такими же, как и на десктопе: приходилось масштабировать, пытаться попасть в мелкие кнопки или ссылки, или закрывать всплывающие окна, которые перекрывают весь контент. Бесит, правда? Если учесть, что сейчас пользователи все чаще серфят в интернете с мобильных, такие неповоротливые сайты просто непозволительны.

Мало того, что люди чаще заходят в интернет с мобильных устройств. Важная особенность мобильного трафика — время посещения сайта с мобильного телефона: оно примерно на 57% меньше времени, проводимого на сайте с компьютера.

Значит, сайт непременно надо делать таким, чтобы его было удобно использовать с мобильного. Но для начала разберемся с терминологией. Сайты с точки зрения адаптации под мобильные устройства бывают резиновые, адаптивные, и отзывчивые — их также называют «респонсив». Отдельно выделим мобильную версию и поговорим о ней чуть позже.

12.7.1 РЕЗИНА

При резиновой верстке колонки на странице меняют свою ширину пропорционально, в зависимости от ширины окна браузера, но при этом размер шрифтов не меняется. Картинки сами не перестраиваются —

масштабирование включается, только если экрана будет не хватать. Метод известен еще с 2000-х годов, когда верстали таблицами. Сейчас это изживший себя стандарт.

12.7.2 АДАПТИВ

У адаптивных сайтов есть точки излома — это размеры окна браузера, при которых отображение страницы меняется на другой вид. Точек излома может быть несколько. Обычно их две: для планшетов и смартфонов. Реже делают четыре точки — дополнительно для перевернутых горизонтально устройств. За точку излома берется самая частая ширина экрана группы устройств, например: 768px для планшетов и 320px для мобильных. И на этих двух точках страница будет резко менять свой вид. Пользователи мобильных устройств, конечно же, этого не заметят, а вот если уменьшать окно браузера на десктопе — вы увидите переход.

12.7.3 РЕСПОНСИВ ИЛИ ОТЗЫВЧИВАЯ ВЕРСТКА

Это уже продвинутый адаптив, который плавно сжимается-тянется между точками излома и выглядит хорошо на любом отрезке от десктопа до мобильного. С респонсив-версией можно не переживать, что владельцы телефонов с большим экраном на 480 пикселей по ширине будут видеть обрезанную версию с планшета — все будет в лучшем виде.

Технология реализации адаптива и респонсива схожа, но по затратам ресурсов респонсив выйдет подороже, чем адаптив. Основные расходы будут при этом на отладку под плавность и тестирование.

На сегодняшний день это основной вариант реализации.

12.7.4 МОБИЛЬНАЯ ВЕРСИЯ

Это отдельные шаблоны, которые показываются только на мобильных устройствах и обычно живут на отдельном поддомене. Функционал мобильной версии может использовать ресурсы основного сайта,

но визуально страницы могут очень сильно отличаться. Обычно на мобильной версии оставляют кнопку для перехода к обычному виду сайта (пример: если хоть раз вы заходили из браузера на сайт ВКонтакте — в самом низу на странице профиля есть переключатель на версию для ПК).

Это делается по двум причинам:

- 1) для удобства пользователей, которые привыкли к десктопу;
- 2) потому что между этими версиями могут быть функциональные различия (например, на мобильной версии может не быть функционала для сравнения товаров).

Разработка такой версии заметно дороже адаптива или респонсива — грубо говоря, это где-то 40–80% от объема работы над основной версией, так как нужны отдельные макеты, которые прототипируются, дизайнятся, верстаются и собираются в мобильный сайт. Такой вариант используется редко, так как очень затратен в поддержке. Любое изменение приходится переносить во все версии продукта и тестировать.

12.7.5 МОДУЛЬНЫЕ СЕТКИ. НУЖНО ЛИ ОТРИСОВЫВАТЬ ВСЕ СТРАНИЦЫ?

Если вы планируете разрабатывать адаптив с самого начала проекта и хотите, чтобы он был легким и удобным — используйте на дизайне модульную сетку. Это когда весь макет разбивается на колонки, и по ним выстраиваются модули. Модули — это прямоугольники заданной высоты и ширины, на основе которых строится композиция макета. Модульная сетка — это вертикальные и горизонтальные линии, которые делят страницу на прямоугольники. Сетка задается дизайнером на этапе отрисовки проекта.

Именно по границам модульных сеток ваши блоки будут перестраиваться в момент перехода от одной точки излома к другой. Чаще всего используют 12-ти колоночную сетку. Это не догма, но к ней все привыкли, и именно такая сетка по умолчанию используется в популярном фреймворке для верстки макетов — bootstrap.

Что такое бутстрап

Бутстраповская 12-ти колоночная сетка, которая используется для верстки сайтов и которая могла быть у вас на слуху, — это часть фреймворка Twitter Bootstrap. Также внутри фреймворка есть стандартные виды элементов, кнопочек и всего-всего.

Часто бывает, что фреймворки не монолитные и состоят из нескольких частей, которые можно использовать отдельно. Собственно, как бутстраповскую сетку, которая стала стандартом :)

Верстка может быть и не по сетке: как правило, общая канва проекта использует модульные сетки, но какие-то отдельные элементы могут намеренно ее нарушать. Верстальщики будут недовольны, но это добавляет проекту фишек.

Нет нужды согласовывать отдельно с заказчиком или вашим боссом каждый экран на каждом разрешении. Модульные сетки дают разумный, качественный респонсив, но без выноса мозга. Прорисуйте пару экранов, меню и всплывающие окна. Для остального в подавляющем большинстве проектов хватит подхода модульных сеток. Рекомендую делать именно так, и только если на каких-то страницах будет что-то смущать — начинать их отдельно отрисовывать.

12.7.6 ЧТО ЕЩЕ СТОИТ УЧЕСТЬ НА АДАПТИВЕ

Иконки. Только векторные — это обязательно. Растровые иконки на мобильных устройствах выглядят мыльно. Иногда приходится приложить много усилий, чтобы иконки смотрелись хорошо на всех устройствах.

Меню. На десктопе места много, а вот на адаптиве придется оставлять самое важное, например: кнопку для обратной связи или поиск, а в аккуратное бургер-меню прятать все основные ссылки. Помните о логике: важное наверх, управляющие элементы и настройки — отдельным блоком. Подвал (он же футер) на адаптиве вам уже не сильно нужен, оставьте дубль контактов и копирайты, а важные сервисные ссылки унесите в меню, которое можно вызвать из шапки.

Всплывающие окна. Подход «в лоб» к всплывающим окнам на мобильных устройствах дает довольно топорный результат. Убедитесь, что этим будет удобно пользоваться.

Элементы. Используйте меньше закрепленных элементов на экране: только самые важные, целевые действия. Не у всех пользователей огромный экран, и широкая плашка на половину мобильного явно будет раздражать.

Таблицы. Важно, чтобы пользователю было удобно их читать. Можно сделать аккуратные блоки из каждой строки, которые идут друг за другом. Где-то будет достаточно скролла. Можно даже сделать, чтобы пользователь мог сам скрывать и добавлять данные для просмотра. Такие фишки не являются излишеством, если таблица на вашем сайте несет важную информацию пользователю, и от нее зависит продажа или конверсия.

Производительность. Адаптив должен быстро загружаться и быстро реагировать на действия пользователей. Технологически адаптивная верстка использует единый шаблон и для мобильных, и для десктопа. Именно тут вам аукнется модульная сетка: если она была проработана как надо, то количество дополнительного кода и скриптов для реализации удобного адаптива будет минимальным.

12.7.7 MOBILE FIRST? НЕ ФАКТ

С какой версии начать? С мобильной или десктопной? Вопрос интересный. Подход Mobile First имеет смысл и своих сторонников. Его идея: раз большинство пользователей работает с мобильных устройств — делаем все для них в первую очередь.

Однако, как отмечали некоторые наши заказчики, подход дает довольно однообразные и некрасивые результаты. Все как бы по делу. Но нет «вау».

Как я уже говорил в главе о дизайне, рекомендую сразу вместе с концепцией проработать и десктоп, и мобильную версию главной страницы. Красотишки будет больше именно на десктоп-системах. На мобильных продумываем юзабилити «под палец» и отсекаем все то, что может тормозить и мешать.

Итого: при подготовке адаптива не начинайте прорисовывать все экраны под все устройства, а используйте модульные сетки. Отрисуйте основные элементы навигации для мобильных систем и пару экранов. Дальше сетки сами сделают свое дело. Это позволит быстро и качественно поставить ваш продукт конечному потребителю за адекватные деньги. Если потребуется замороченная версия с согласованиями пятисот экранов — ну тогда уже готовьте лишние тысячи часов на этот квест.

12.8 МОБИЛЬНАЯ РАЗРАБОТКА

На данный момент существует два принципиальных подхода к разработке мобильных приложений: по способу разработки приложения бывают нативные и кроссплатформенные.

Нативные — приложения, написанные с использованием нативного языка программирования (Java и Kotlin для Андроид, Swift или Objective C для iOS). Вам понадобится отдельно писать код под каждую из платформ. И каждое изменение делать дважды. Такова цена максимально высокой производительности и отзывчивости.

Кроссплатформенная разработка позволяет использовать единый код для iOS и Android.

12.8.1 КРОССПЛАТФОРМЕННОСТЬ

Среди кроссплатформенных технологий хочу выделить Flutter — технологию от Google, позволяющую строить портативные приложения с такой же производительностью и отзывчивостью, как и нативные. Порог входа в эту технологию для толкового разработчика порядка двух недель. Язык программирования — Dart.

На втором месте — React.Native. Детище Facebook, которое создавалось с целью сделать кроссплатформенные приложения такими же производительными, как нативные. Языки программирования: JavaScript + библиотека React.

React Native сейчас популярен, его применяют технологические гиганты, вроде того же Facebook, Instagram, Pinterest и даже Tesla. Плюс — созданные с его помощью приложения по поведению и внешнему виду близки к нативным.

На третьем месте, пожалуй, PhoneGap. Детище Adobe, по факту является **дистрибутивом** фреймворка Apache Cordova, который преобразовывает веб-приложения в мобильные программы. Язык программирования — JavaScript + HTML, CSS. Любой web-программист может сделать на нем приложение, порог входа минимальный. Минус — низкая отзывчивость интерфейса.

С развитием мобильных проектов в мире растет спрос на мобильную разработку, растет скорость создания приложений, растет интерес к простым решениям с доступной стоимостью разработки. Все это открывает огромные возможности для улучшения гибридных приложений, которые зачастую могут оказаться не хуже нативных.

12.8.2 BACKEND И FRONTEND В МОБИЛЬНЫХ СИСТЕМАХ

Большинство современных полезных приложений хранят и обрабатывают часть данных на серверах. Например, вы не смогли бы делиться с друзьями фотографиями в соцсети, если бы там не было серверной части.

Итак. В мобильных приложениях можно — точно так же, как и в сайтах — выделить Backend и Frontend. Однако в качестве Frontend-движка выступает не браузер (как это происходит в сайтах), а ваше приложение. Сделанное нативно или на кроссплатформенных технологиях.

Теперь шикарные новости. Если для сайта вы использовали реактивный подход (Vue или React) и заранее думали о мобильном приложении, вполне возможно, у вас уже готов Backend. Вы можете реализовать его как REST API и использовать совместный Backend для сайта и приложения, сэкономив кучу ресурсов. Конечно, если большая часть функций приложения уже есть на сайте.

Из этих новостей следуют требования к команде: у вас должен быть один Backend-программист, делающий серверную часть. И один или два (в зависимости от того, ведете ли вы нативную разработку или кроссплатформенную) программиста под мобильные платформы. Дальше, в зависимости от размеров проекта, команды можно масштабировать. Конечно, можно всю разработку свалить на одного разработчика, но это такое...

12.8.3 НЮАНСЫ УПРАВЛЕНИЯ МОБИЛЬНОЙ РАЗРАБОТКОЙ

Нюансов почти нет, так как нет ничего сакрального. Но на паре моментов хочу заострить внимание.

Тестирование. Его нужно будет больше, чем при тестировании сайтов. Нужно будет проверять под несколько платформ. Количество странных ситуаций, при которых мобильное приложение может упасть, выше, чем для сайта. Сам видел, как стабильно работающее приложение падало, если во время работы с ним на телефон кто-то звонил, и одновременно с этим пользователь переворачивал экран в альбомную ориентацию. Но такой тест еще придумать надо!

Не брезгуйте автотестами. Особенно если ваше приложение становится популярным. Вы можете запускать автотесты в облаках на разных устройствах, например через Google Firebase.

Подключите аналитику. Тот же Google Firebase и Firebase Crashlytics дают прекрасную статистику о том, как на самом деле и чем люди пользуются. И пользуются ли вообще. И что у вас упало и сколько раз. Регулярно смотрите на статистику отказов, **Retention**, **MAU**, **DAU**, чтобы понять, на каком шаге вы теряете пользователей.

Публикация в сторсах. Это порой очень утомительно. Мало того, что у вас должны быть куплены сертификаты разработчиков. Получены **DUNS-номера**, если вы хотите принимать оплаты. Написаны лицензионные соглашения... Для самих сторсов много времени уходит на описание, подготовку картинок и модерацию. Вы не сможете так же быстро, как это делаете для сайта, публиковать обновления для сторсов. Банально, модерация может занять от трех дней до недели. Особенно в рождественские праздники.

Поэтому больше времени уделяйте тестированию. Используйте группы пользователей для закрытого тестирования и поставки им бета-версий. Быстро исправить ошибки вы не сможете. Максимум — отозвать **релиз**, но это плохая карма.

Кроме того, Apple и Google чувствительно относятся к пользовательской информации, платежам, соглашениям на доступ к уведомлениям,

геолокации и т. д. Возможно, вам придется долго переписываться или созваниваться с модератором. Но тут такие правила, ничего не поделаешь.

Итак. Мобильные приложения — это довольно просто. Если вы правильно проектировали сайт — у вас уже может быть готов Backend. Для большинства приложений достаточно двух-трех разработчиков. Большие команды нужны редко. Уделяйте больше времени тестированию и закладывайте время на публикацию в сторсах. Проводите закрытые тестирования, например через TestFlight. Не брезгуйте автотестами. Подключайте аналитику и смотрите, что на самом деле важно для пользователей.

12.9 ИНТЕГРАЦИЯ СО ВСЯКОЙ ВСЯЧИНОЙ

Почти каждый современный проект не обходится без интеграций со сторонними сервисами. Системами оплаты, доставки, лояльности, аналитики, кассовыми чеками, чатами, sms-шлюзами и прочее, прочее, прочее. Мы уже говорили про интеграцию с **ERP** в главе 10, а здесь по верхам затронем те системы, с которыми чаще всего приходится иметь дело.

Таких систем — тысячи. Накопить опыт по ним всем нереально. С какими-то системами интеграция настраивается за пять минут (поставить чат на сайт, например). С какими-то вы будете иметь головную боль на протяжении всей жизни проекта. Заранее предсказать сложно, но, если система, с которой вы хотите интегрироваться — новая, а ваш проект — это первый подключаемый к ней в мире, — будет больно. Вы будете бета-тестером сторонней системы.

Существуют несколько подходов к интеграции. Расположу по степени сложности:

1. Вставка кусочка JavaScript-кода или **iFrame** в ваши страницы. Самый простой вариант. Чатики, шарилки и тому подобное. Копируете код, вставляете в нужные страницы (или в общий шаблон). Если никто нигде не ошибся (например, при пересылке JS-кода по электронной почте заменяются некоторые символы) и сторонний скрипт ведет себя, как надо, — на этом все и заканчивается. Имейте в виду, что не все сторонние скрипты могут быть совместимы с реактивным подходом. Про это лучше выяснить на берегу. Иногда сторонние скрипты воруют данные ваших пользователей.

2. Вставка кусочка кода и настройка внешнего вида. Чуть сложнее. Часто не все элементы внешнего вида стороннего скрипта можно настроить так, как задумал дизайнер. Бывает так, что кроме вставки куска кода нужно дописать много своего. Например, в случае работы с картографией, кастомизацией карт, прокладкой маршрутов.
3. Вставка кусочка кода в серверные скрипты. Вот это я не очень люблю. Бог его знает, что за ерунда там написана. Небезопасно. И сложно обновлять.
4. Фиды. Например, для Яндекс.Маркет. Ваш сервер должен по расписанию подготовить файлы в нужном формате и положить их в нужное место. Все зависит от формата и способа организации ваших данных. Кроме того, закладываете время на модерацию. Тот же Яндекс.Маркет далеко не с первого раза принимает к себе магазины. И дело может быть даже не в вашем коде, а в контенте. Например, выгрузки товаров, которых реально нет в наличии.
5. Модули к системе управления сайтом (CMS) или фреймворку. Большинство систем оплат, доставок, рассылок имеют готовые модули для CMS. Остается настроить (или дописать) нужный модуль. Нюансов — куча. Иногда настройка — трудоемкая процедура. Иногда код модулей ужасен. Иногда просто не работает, как надо. Иногда модуль просто сдох. Иногда вам надо совсем не то, что делает модуль, но про это вы узнаете уже после начала работ. Вам, как минимум, нужны будут доступы и ключи к таким сервисам, а также точки входа (адреса), по которым будет идти взаимодействие.
6. Серверные модули. Это могут быть движки поисковых систем (вроде Sphinx и Elastic). Вообще, хороший поиск: с синонимами, исправлением ошибок и хорошей релевантностью — это отдельная, большая и сложная тема. А у заказчиков про поиск могут быть завышенные ожидания. Порой очень сложно понять, почему поиск нашел именно то, что нашел, и почему вывел именно в таком порядке. Другие, часто добавляемые серверные модули — это генерация PDF и какие-то хитрые системы статистики.



Рекомендую
прочитать этот
материал по
теме поиско-
вых движков

7. Интеграция со сторонним API. В теории, со зрелыми сервисами тут проблем не возникает. Вы просто делаете интеграцию в соответствии со спецификацией. Но на практике даже зрелые сервисы доставки, существующие десятки лет, могут работать криво, косо, или совсем не так, как вы этого ожидаете по спецификации.
8. Интеграция по API, которого нет. Например, API должны разработать вы, а кто-то к нему подключается. Или API разрабатывает кто-то для вас. Это самый сложный вариант. Требуется предварительного проектирования, приемочных тестов, времени и ресурсов на ввод в эксплуатацию. К сожалению, сложно дать какие-то общие рекомендации, как и заранее предсказать, сколько ресурсов на это потребуется.

Интеграция со сторонними системами позволяет обогатить ваш проект самыми разными возможностями, причем, порой без необходимости долго писать код. С другой стороны, интеграция — это всегда точка потенциального отказа. Как поведет себя ваш проект, если сдохнет, перестанет отвечать или начнет себя вести неадекватно сторонний сервис? Заранее не скажешь.

Кроме того, интегрировать нужно добросовестные сервисы. К сожалению, в сети были громкие скандальные случаи, когда сервисы, интегрированные на тысячи сайтов, меняли владельцев и начинали по-тихому шпионить за пользователями. Такой риск есть. Добро пожаловать в наш нестабильный, реальный мир.

12.10 ЗАКЛЮЧЕНИЕ

Итак. Мы посмотрели, из чего состоят digital-проекты, как работает интернет, что такое backend, frontend, какие там есть технологии. Поговорили про организацию мобильной разработки. Системы контроля версий. И интеграции. Но это только начало.

Чтобы стать профи, заведите себе три привычки:

1. Выяснять у разработчиков непонятные вам термины и понятия до того состояния, что вы сами сможете рассказать про них простым языком аналогий. Так, чтобы ваша мама поняла, о чем вы. Услышали непонятное — задаем вопросы и чертим схемы, пока не разберемся. Программисты могут на это фыркать, но скоро

перестанут. Вам нельзя будет пустить пыль в глаза заумными терминами. Только так вы будете расти быстро.

2. Если в проекте появляется новый фреймворк, библиотека или интеграция — прочитайте сами официальную документацию. Как правило, это занимает не очень много времени. Точно меньше, чем на исправление управленческих ошибок.
3. Читать дайджесты и просматривать технические новости и новинки. Держите нос по ветру.

И еще. Факультативно, но очень полезно. Научитесь программировать. Чуть-чуть. Для себя. По фану.

12.11 ДОМАШНИЕ ЗАДАНИЯ

1. Изучите словарь IT-терминов (ссылку давали выше).
2. Изучите официальную документацию GIT.



Документация GIT

ПОЯСНЕНИЯ

Коммутатор — устройство, которое соединяет несколько узлов компьютерной сети в пределах одного или нескольких ее сегментов.

PHP, Python, Node.js — языки программирования.

Скрипт (сценарий) — последовательность действий, описанных с помощью скриптового языка программирования (JavaScript, PHP, Perl, Python и др.).

Гипертекст — текст, сформированный с помощью языка разметки (например, HTML) с расчетом на использование гиперссылок (ссылающихся на часть текстового документа или на любой другой источник).

Админ-панель — панель управления CMS сайта, с которой может работать пользователь без навыков программирования. С ее помощью вносятся все материалы на страницы сайта: добавляются тексты, изображения, цены и целые блоки.

Механизм шардирования — принцип проектирования базы данных, при котором логически независимые строки таблицы базы данных группируются в секции и хранятся раздельно на разных, физически и логически независимых серверах базы данных.

Инфоблок — модуль в системе 1С-Битрикс, который призван ускорить и упростить создание материалов на страницах сайта. Представляет собой таблицу с ячейками, где хранятся нужные данные. Например, название новости, текст новости, картинка новости и так далее.

Хит — запрос к веб-серверу для получения файла (веб-страницы, изображения, JavaScript-кода, таблицы стилей и т. д.).

NS-сервер, или Name Server — это сервер доменных имен с необходимой информацией о доменном имени для его правильной и корректной работы: он указывает, где именно должны находиться сайт и почта, для которых это доменное имя покупалось. NS сервер хранит информацию о доменной зоне и при запросе DNS сервера выдает ее.

Wildcard-сертификат — защищает домен и все его прямые поддомены: существующие и еще не созданные, на любом количестве серверов. Удобен тем, что не нужно устанавливать несколько сертификатов и настраивать их с каждым обновлением.

Ядро сайта — как правило, это сама CMS или Framework, немодифицируемая разработчиками основа проекта.

Continuous Integration — при этом подходе к разработке каждый функционал поставляется заказчику постепенно, за счет чего можно сразу получить его отклик и внести изменения, если необходимо.

Continuous Delivery — при этом подходе к разработке обновления продукта происходят почти постоянно. Это дает быстрое развертывание на продакшене без изменений текущего функционала. Непрерывность достигается за счет различных оптимизаций на ранних этапах процесса разработки.

Видеохостинг — веб-сервис, позволяющий загружать и просматривать видео в браузере, например через специальный проигрыватель.

Кэш — хранящиеся на ПК пользователя временные файлы: изображения, звуки и прочие элементы веб-страниц, которые тот посещал. При

посещении сайта пользователем страницы не загружаются заново, а берутся из кэша: это уменьшает время загрузки.

Серверный рендеринг — выполнение JavaScript-кода на стороне сервера, чтобы отдать клиенту (браузеру) уже готовый HTML. **Рендерить** — преобразовывать JS-код в HTML.

Redux и MobX — библиотеки, которые используются для управления состоянием в JavaScript-приложениях.

Дистрибутив — комплект файлов для распространения программного обеспечения.

Репозиторий — место, где хранится код и вся информация о его изменениях. Репозитории могут находиться на компьютере разработчика, на компьютерах его коллег и на удаленном сервере.

CI/CD пайплайн — «конвейер», по которому доставляется код, обычно в его роли выступают хостинги репозитория, вроде Github CI/CD или Bitbucket CI/CD.

Retention — в продуктовой разработке коэффициент удержания пользователя в продукте.

MAU (Monthly Active Users) и DAU (Daily Active Users) — метрики посещаемости мобильных приложений, входят в топ ключевых метрик мобильных приложений.

DUNS-номер — идентификатор компании в информационной базе D&B, по которому ее можно найти в международном справочнике. Нужен, чтобы пройти проверку Apple для размещения приложения в его магазине приложений.

Релиз — публикация очередной версии: сайта или приложения.

ERP-система — система управления ресурсами предприятия (заказами, складами, трудовыми ресурсами и так далее).

iFrame — плавающий встроенный фрейм, компонент HTML, с которым можно встраивать документы, видео и интерактивные медиафайлы на веб-страницу (например, вставлять на страницу видео с видеохостинга или пост из соцсети).

ЗАКЛЮЧЕНИЕ

And the eighth and final rule:
if this is your first time at Fight Club, you have
to fight.

Чак Паланик. Бойцовский Клуб

Сможете ли вы стать боксером, ни разу не надев перчатки и не выйдя на ринг? Сможете ли вы научиться управлять самолетом, штудирова формулы аэродинамики? Сможете ли вы управлять проектами, прочитав теорию и посмотрев пару курсов?

Нет. Нет. И нет.

Не мешкая пробуйте и проверяйте все сами. На практике. Получайте свои удары и синяки.

Управление, а особенно управление в digital, где мы имеем дело с сытыми, умными, талантливыми людьми — это сложно. Но невероятно интересно. Пожалуй, сейчас мало найдется профессий, кроме дизайна, программирования и управления проектами, где можно бесконечно расти в профессиональном плане. Да и профессий становится все меньше и меньше. Все уходит в цифру. И руководить этим будете вы.

Не бойтесь и не ленитесь.

Все будет хорошо. Даже если не будет.

Успехов!

*Чарышское,
заброшенный аэродром в горах Алтая,
Июль 2021 г.*

БЛАГОДАРНОСТИ

Эта книга никогда бы не появилась на свет без **Екатерины Королевой**. Ее настойчивости в пинании меня. Редактуры. Кропотливой подготовке текстов черновиков, тестов, сносок и учету миллиона мелочей. Катя, спасибо!

Анне Кожевиной, за поддержку и вычитку. Она просматривала весь текст по мере написания. Именно благодаря этому текст стал лучше, а я продолжал писать, даже когда хотелось бросить эту затею к чертовой матери.

Екатерина Мамонтова — моя правая рука и рациональная половина мозга, за дельные советы, подбадривание и расчистку моего времени от груды дел, которые поглощали меня целиком. Спасибо, Катя!

Кристине Неверовой — Крис, ты — талантище! Я не знал, как это будет в бумажном виде, но твои анимированные иллюстрации, юмор, схватывание идей на лету делают эту книгу прикольной.

Ирине Киреевой, Дине Гарбуз — за подготовку контента и съемки курса управления проектами 2016 года, а также терпение к моим бесконечным придиркам и сложному характеру. С теплотой вспоминаю то время. Часть материала я смог почти без изменений вставить в текст.

Елене Абрамовой — впервые воплотившей идею анимированных иллюстраций и написавшей сказку про Мышу.

Сергею Александровичу Кузнецову, Денису Колбину, Витале Лукьянченко, Сергею Попову и Михаилу Переверзеву, Сироткину Александру Сергеевичу, Паше Баркову, Валере Трудакину, открывшим мне удивительный и строгий мир авиации, спортивного пилотажа, повлиявшим на мой характер и научившим, что мелочь ничего не прощает большому.

Всем ребятам и девчатам из команд Сибирикс и SingularityApp. За опыт. Знания. И то, что в принципе появилась возможность посвятить время этому тексту. За те проекты, которые вы делаете и которыми можно гордиться.

И, наконец, заказчикам, доверяющим нам свои проекты. За энергию, опыт и обратную связь.

ОБ АВТОРЕ

Меня зовут Владимир Завертайлов. Я руководитель и главный бармалей scrum-студии Сибирикс. А также создатель популярного планировщика SingularityApp. Автор более сотни статей, спикер и модератор десятков digital-конференций и создатель авторского курса управления digital-проектами.

По образованию — инженер-программист. В диджитале с 2003 года. Моя компания входит в десятку лучших web-разработчиков России. За эти годы успели поработать с такими компаниями как Северсталь, Орматек, Металлпрофиль, «Дисней», TP-Link, Logitech, Атол, Гринфилд, Сочи-Парк, Adobe и другими крупными, хорошими ребятами. Специализируемся на сложных, интеграционных проектах. Сибирикс вообще первая в России студия, внедрившая гибкую методологию Scrum, когда это еще не было мейнстримом, и многое сделавшая для ее популяризации.

В свободное от работы время живу спортивной авиацией.

Приложение

КАРТА КОМПЕТЕНЦИЙ РУКОВОДИТЕЛЯ DIGITAL-ПРОЕКТОВ

Я использую следующие уровни оценки руководителей проектов:

- 0 — стажер;
- 1 — junior;
- 2 — junior ++;
- 3 — middle;
- 4 — middle ++;
- 5 — senior manager (старший менеджер);
- 6 — curator (куратор);
- 7 — executive director (исполнительный директор).

При заполнении проставьте себе оценки по пятибалльной шкале:

- 5 — знаю и регулярно применяю, могу научить кого-то;
- 4 — знаю, но не могу сказать, что регулярно применяю;
- 3 — изучил теорию;
- 2 — слышал теорию, но не знаю, зачем мне это на практике;
- 1 — слышал какие-то обрывки разговоров с таким термином;
- 0 — впервые слышу об этом.

Термином «Дополнительные компетенции» обозначены дополнительные компетенции, необходимые для кураторов, старших менеджеров и исполнительного директора. Приводятся на красном фоне.

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Базовые навыки					
Требовательность. Внимание к мелочам. Навык добиваться 100% качества от себя и других	1				
Ответственность. Умение брать ответственность на себя, не перекладывая ее на других и обстоятельства	1				
Центрирующие парадигмы. Знание, понимание	0				
Кодекс сотрудника (если внедрен в организации). Знание, понимание	0				
Навык постановки задач. SMART	1				
Навык делегирования	4				
Грамотная устная речь	1				
Управление голосом в переговорах. Интонация, паузы, акценты	3				
Навыки проведения переговоров. Активное слушание, формулирование вопросов, перехват управления	2				
Грамотная письменная речь. Структура писем	1				
Работа с электронной почтой. Чистый ящик. Подпись и заголовок у писем. CC, BCC, ответить всем; CC: (копия, carbon copy) — вторичные получатели письма, которым направляется копия. Они видят и знают о наличии друг друга. BCC: (скрытая копия, blind carbon copy) — скрытые получатели письма, чьи адреса не показываются другим получателям.	1				
Навыки самостоятельного поиска ответов на свои вопросы	0				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Знание позиционирования компании дословно	1				
Знание структуры отделов в компании и взаимосвязи между ними	2				
Знание основных этапов разработки digital-продукта, их последовательности и взаимосвязей	1				
Навыки создания комфортной рабочей атмосферы. Выстраивание коммуникаций в команде на проекте	3				
Навык выстраивания отношений в команде с исполнителями, имеющими большой опыт и стаж работы в компании	4				
Навык выстраивания отношений в команде с новичками, имеющими маленький опыт работы. Контроль новичков	4				
Навык выстраивания отношений с творческими людьми, дизайнерами	4				
Навык контроля бюджета в проекте. Предупреждение перерасходов, управление расходом времени, своевременная эскалация	3				
Выдача обратной связи исполнителям: позитивной, негативной, эмоциональной, нейтральной	3				
Эскалация руководителю по возникшим проблемам. Своевременная. Умение видеть проблемы и не прятать их	1				
Root Cause анализ	4				
Дополнительные компетенции:					
Ретроспективы менеджеров. Проведение и контроль внедрений	7				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Проведение обучения менеджеров	6				
Разбор факапов и выдача обратной связи исполнителям с применением техник устного поощрения и наказания	5				
Управление процессами на внутренних проектах компании (все этапы, включая подготовку контента)	5				
Проверка писем у менеджеров	6				
Проверка бюджетов и выполнения планов менеджерами	7				
Контроль стажеров-менеджеров и приемка дипломных работ	6				
Внедрение согласованных изменений в процессы компании. Проверка результатов	7				
Навыки (опыт) ведения (разработки) проектов определенных типов					
Корпоративные сайты	2				
Промо-сайты	3				
Интернет-магазины (без сложных личных кабинетов и интеграций)	3				
Интернет-магазины с личными кабинетами и интеграциями	4				
Стартапы и сервисы	5				
Мобильные приложения	4				
Корпоративные порталы	5				
Внутренние проекты	5				
Мелкая техническая поддержка	2				
Крупная техническая поддержка	4				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
HR					
Оценки / Опрос 360 по своим сотрудникам / подопечным по итогам спринтов / проектов	5				
Дополнительные компетенции:					
Подготовка scrum-мастеров	6				
Кураторство (наставничество)	6				
OKR. Формирование персональных планов обучения и контроль	6				
Первичные собеседования	7				
Результирующие собеседования	7				
Аттестации	7				
Увольнения	7				
Базовые инструменты					
Работа с тикет-системой. Постановка и контроль задач	1				
Работа с тикет-системой. Контроль рабочих отчетов (проверка по своим проектам)	2				
Работа с канбан-досками. Работа с бэклогами. Формирование спринтов. Отслеживание прогресса (Burndown Chart, Velocity)	2				
Планирование времени и нагрузки					
Планирование персональных (личных) задач	2				
Подготовка запроса ресурсов, плана недели	2				
Недельное планирование по проектам	3				
Основы тайм-менеджмента	3				
Способы концентрации и фокусировки. Pomodoro. Интервалы.	3				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Матрица Эйзенхауэра	3				
Профилактика нехватки времени	4				
Техники восстановления личного ресурса (ресурсного состояния)	4				
Дополнительные компетенции:					
Расстановка приоритетов всем исполнителям на день	6				
Проверка рабочих отчетов исполнителей	6				
Балансировка нагрузки на исполнителей на день (мало/много задач)	7				
Расстановка приоритетов на день менеджерам	6				
Проведение общей утренней планерки	6				
Проведение ежедневных планерок с менеджерами	6				
Проведение еженедельных планерок по проектам и запросам	7				
Проверка планов недели у менеджеров	6				
Балансировка потока проектов (производственный график)	7				
Передача проектов					
Обработка transition-файла от аккаунт-менеджера. Подготовка к передаче с клиентом	1				
Структура разговора «Подключение менеджера проекта к проекту»	1				
Знакомство с клиентом. Ведение разговора, резюме звонка	1				
Обработка transition-файла от менеджера проектов на время отпуски	1				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Подготовка transition-файла на время отпуска	2				
Дополнительные компетенции:					
Проверка, приемка transition-файлов от аккаунт-менеджеров и менеджеров проектов	6				
Отчет по нагрузке на исполнителей на время своего отпуска	7				
Аналитика					
Знание шаблона Lean Canvas	0				
Разработка mind map агрегации требований проекта	1				
Актуализация сметы после агрегации требований	2				
Проведение брейнштормов перед прототипом	3				
Знание принципов юзабилити, отличия хорошего прототипа от плохого	2				
Разработка интерактивных прототипов	2				
Знание шаблона и примеров технического задания	1				
Разработка технического задания (приемка, демонстрация, получение актов)	2				
Проектирование структуры баз данных (или инфоблоков) и их приемка у разработчиков	3				
Актуализация сметы после технического задания	2				
Разработка технического задания на интеграцию с внешней системой (получение выгрузок, приемка XMind, демонстрация, акт)	4				
Опыт работы с системами документации API (swagger)	4				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Дополнительные компетенции:					
Проверка результатов по каждому этапу аналитики незнакомого проекта, приемка, выдача обратной связи	7				
Обучение и проверка по актуализации смет	6				
Дизайн					
Ресурсы для ежедневного поиска новинок digital-проектов. Регулярный просмотр новинок	0				
Подготовка (приемка) и демонстрация мудборда	2				
Креативное мышление. Проведение брейн-штормов	3				
Подготовка скетчей (приемка, демо)	3				
Знание принципов визуального дизайна, отличия хорошего дизайна от плохого	2				
Постановка задач на дизайн	2				
Ежедневный контроль хода работ по дизайну (с привлечением арт-директора и без него)	2				
Доведение готового макета до крутого вместе с дизайнером	4				
Подготовка презентации (постановка, приемка)	2				
Внутренняя приемка	2				
Сдача дизайна удаленно (демо, резюме)	2				
Обработка возражений по дизайну	2				
Обработка итераций правок, демо, акт по дизайну главной страницы	2				
Подготовка внутренних страниц (тайминги, приемка, демо, правки, акты)	2				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Дизайн поиск и покупка фотографий/видео со стоков	2				
Знание всех необходимых страниц и элементов (404, favicon, UI Kit / guidelines)	1				
Дополнительные компетенции:					
Участие в брейнштормах сторонних проектов, доведение брейншторма до фишки	7				
Финальная приемка макетов, презентаций	7				
Участие во всех внутренних приемках	7				
Подготовка менеджера к демо дизайна, репетиции	6				
Работа с негативной обратной связью клиента	5				
Разбор ошибок менеджера по итогам демо, обратная связь	6				
Верстка					
Подготовка бэклога, тайминга и материалов на верстку	1				
Поиск анимационных фишек и внедрение в постановки на верстку	2				
Приемка баглистов и контроль фиксов верстки. Кроссбраузерность	2				
Ревью верстки (постановка, контроль фиксов)	2				
Ретроспектива по верстке (с дизайнером)	2				
Дополнительные компетенции:					
Участие в ретроспективах и контроль внедрений	6				
Решение спорных вопросов по реализации верстки	6				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Подготовка менеджера к демо верстке, репетиции	6				
Разбор ошибок менеджера по итогам демо, обратная связь	6				
Разработка					
Подготовка бэклога (самостоятельная)	2				
Делегирование подготовки бэклога и приема результата	3				
Планирование. Planning Poker. Актуализация планов	2				
Ежедневный митинг (стендап)	1				
Приемка баглистов и контроль фиксов	2				
Ревью сборки (постановка, контроль фиксов)	2				
Личное тестирование (приемка проекта)	1				
Тестирование внешним менеджером	2				
Наполнение сайта тестовым контентом	1				
Подготовка инструкции по наполнению контентом	1				
Рескоупинг спринта	3				
Ретроспектива по разработке	2				
Внедрение улучшений (внедрение конкретных решений с ретроспектив)	3				
Дополнительные компетенции:					
Приемка бэклогов финально к планингу	6				
Участие в планингах в роли координатора	6				
Участие в ретроспективах и контроль внедрений	7				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Мониторинг общих расходов спринта, отправка на рескоупинг	7				
Решение спорных вопросов по реализации сборки	7				
Релиз-менеджмент. Сдача проекта, доведение до релиза, гарантия					
Демо проекта, резюме звонка	2				
Получение и обработка обратной связи от заказчика (разделение на фичи и баги)	2				
Контроль фиксов по комментариям, сдача заказчику	2				
Получение доступа к серверу, проверка доступа	3				
Акты финальных доработок	4				
Получение отзывов (письменных) по итогам проекта (большого спринта)	3				
Выдача отзывов руководителю проекта на стороне клиента	3				
Получение актов	2				
Постановка задач на деплой, контроль	3				
Подготовка релиза в блог (приемка, согласование с заказчиком)	2				
Получение даты релиза от заказчика. Получение контента. Доведение до запуска проекта	3				
Обработка комментариев заказчика после деплоя. Разделение на гарантию и не-гарантию	3				
Контроль фиксов по гарантии, соблюдение сроков	3				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Дополнительные компетенции:					
Подготовка менеджера к демо сборки, репетиции	6				
Разбор ошибок менеджера по итогам демо, обратная связь	6				
Разбор возражений и решение конфликтов с клиентом	5				
Подготовка релизов	4				
Опрос сложных клиентов по итогу проекта	7				
Контроль коммуникаций с клиентом после релиза (не посориться)	7				
SEO и продвижение					
Знание базовых SEO-требований, которые учитываются при разработке	1				
Обработка запросов SEO-компаний. Вывод заказчика на приложение по техподдержке	3				
Навык работы с Google Analytics, Яндекс. Метрикой (минимально: умение строить простые отчеты и проверять работоспособность настроенных целей)	3				
Дополнительные компетенции:					
Участие в переговорах с заказчиком по вопросам seo-задач при конфликтных ситуациях	7				
Продуктовые техники					
JTBD. Опыт проведения решенческих и проблемных интервью	5				
RAT	5				
CJM	5				
Методы сегментации рынка	5				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Продуктовая аналитика. Unit-экономика	5				
Техники приоритизации бэклогов (RICE/ICE/Kano...)	5				
User Story Mapping	5				
HA DI-циклы	5				
Техническая поддержка и развитие					
Знание условий платной технической поддержки, регламента обработки обращений, биллингов	3				
Работа с тикет-системой в режиме технической поддержки	3				
Подготовка тикет-системы для заказчика (доска задач, постановка через почту, напоминания, отчеты). Обучение заказчика	3				
Работа со срочными задачами: постановка, приемка	3				
Работа со срочными задачами: биллинг, документы, получение оплат и актов, актуализация доски задач	3				
Работа с обычными задачами: оценка, формирование спринтов	3				
Работа с обычными задачами: спринты по контуру разработки (планинг, стендапы, приемка)	3				
Работа с обычными задачами: демо спринта, резюме, обработка комментариев	3				
Работа с обычными задачами: деплой спринта	3				
Работа с обычными задачами: биллинг спринта, документы, получение оплат и актов, актуализация доски задач	3				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Ретроспектива по спринтам или этапам	3				
Внедрение улучшений (внедрение конкретных решений с ретроспектив в проектах на технической поддержке)	3				
Дополнительные компетенции:					
Контроль бюджета проекта. Получение от менеджера проекта способов перекрытия перерасходов	7				
Выборочный контроль досок, задач, спринтов	6				
Участие в ключевых звонках с заказчиками	7				
Решение спорных и конфликтных ситуаций	6				
Участие в ретроспективах по проектам технической поддержки и контроль внедрений	6				
Коммуникации. Ведение переговоров					
Оцените от одного до пяти собственный уровень, как конструктивного переговорщика, добивающегося результата не поссорившись	1				
Митинг-репорты. Навык написания митинг-репортов по ходу разговора	2				
Формирование ожиданий (переговоры на опережение)	4				
Управление конфликтами. Решение конфликтов. Работа с трудными клиентами	5				
Решение конфликтов с сотрудниками	4				
Гарвардский метод переговоров	4				
Энкоды	5				
Переговоры с сотрудниками, разбор ошибок и проступков	4				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Дополнительные компетенции:					
Подготовка к переговорам и обучение переговорам менеджеров	6				
Проверка переговоров (записи, участие), выдача обратной связи	6				
Проверка резюме переговоров, обратная связь, приемка	6				
Решение вопросов при коммуникациях со сложными клиентами, конфликтах	7				
Оценки проектов					
Ценообразование. Знание средне-вилочных цен по проекту	3				
Декомпозиция	3				
Оценки проектов	3				
Подготовка смет в стандартном формате	3				
Подготовка таймлайнов	3				
Дополнительные компетенции:					
Проверка смет	6				
Проверка таймлайнов	6				
Оценка сложных функций	5				
Документооборот					
Работа в ERP	2				
Клиентские отчеты	2				
Подготовка коммерческих предложений	4				
Подготовка договоров и приложений	2				
Подготовка дополнительного соглашения	3				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Акты (выставление, получение, печать, ЭДО)	1				
Акт финальных доработок	3				
Акт сверки взаиморасчетов	3				
Счета	1				
Дополнительные компетенции:					
Ревью папок проектов. Проверка структуры и порядка	7				
PR					
Публичные выступления: навык подготовки тезисов	5				
Публичные выступления: навык проведения публичных выступлений	5				
Публичные выступления: навык импровизации	5				
Публикации на профессиональные темы. Навык подготовки	5				
Технологии (понимание сути, умение рассказать, умение проконтролировать реализацию)					
Системы контроля версий. Знание принципов GIT. Работа с ветками	3				
Верстка. Понимание принципов верстки/ умение верстать	4				
SEO-возможности 1С Битрикс	3				
Кроны и агенты на сервере. Знание принципов и отличий	4				
Хостинг: заказ dedicated и shared-хостинга. Заказ доменов.	4				
Хостинг: планирование нагрузок	5				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Принципы работы DNS	4				
Google PageSpeed: проверка, знание ограничений и требований	3				
Проверка скорости загрузки сайта/страниц	3				
Проверка валидации сайта	3				
Адаптивные сайты	3				
Мобильные сайты	3				
Проекты с версткой писем	3				
Сервера очередей (RabbitMQ...)	5				
Интеграция со сторонними ERP (API/SOAP/REST, файловые, промежуточные базы)	5				
Быстрые сайты для Яндекс и Google. Яндекс. Турбо и Google.AMP	5				
Размещение кодов метрик, аналитики	3				
Подключение платежных систем	3				
Подключение систем доставки	3				
Подключение сервисов комментариев и чатов	4				
Интеграция с SMS-шлюзами	3				
Возможности картографии	4				
Базовые знания Linux	5				
Мобильная разработка	5				
Основные теории					
Scrum, Канбан	1				
ТОС (принципы, деревья, умение работать с инструментами ТОС)	4				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
PTS (принципы, муда, способы устранения, инструменты Production Toyota System)	5				
LEAN	5				
5S	4				
Six sigma	4				
Литература к изучению и конспектированию					
Общий менеджмент					
М. Дорофеев. Джедайские техники	3				
М. Дорофеев. Путь джедая	4				
Д. Аллен. Как привести дела в порядок.	4				
А. Фридман. Вы или вас	3				
А. Фридман. Управление повседневным хаосом	4				
В. Тарасов. Персональное управленческое искусство (курс, книга)	6				
В. Тарасов. Восемь ступеней управленческого искусства.	6				
Э. Голдратт. Цель. Цель 2.	4				
PMI PMBOK® 5th Edition	6				
Менеджмент в IT					
Правила Ашманова https://www.ashmanov.com/education/articles/pravila-ashmanova/	3				
Правила Ашманова 2 https://www.ashmanov.com/education/articles/pravila-ashmanova-2-upravlenie-proektami/	3				
Словарь ненормативной лексики руководителя https://www.ashmanov.com/education/articles/slovar-nenormativnoy-leksiki-rukovoditelya/	3				

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
Словарь ненормативной лексики программиста https://www.ashmanov.com/education/articles/slovar-nenormativnoy-leksiki-programmista/	3				
Ф. Брукс. Мифический Человеко-месяц.	4				
Т. ДеМарко. Роман об управлении проектами	4				
Дж. Ханк Рейнвотер. Как пасти котов	4				
Х. Книберг и М. Скарин. SCRUM и KANBAN: выжимаем максимум.	0				
SCRUM and XP from the trenches	1				
PMbok	1				
С. Панкратов. Черная книга менеджера	1				
Продажи и переговоры					
А. Фридман. Секреты успешного переговорщика	3				
А. Фридман. Искусство конструктивного диалога	3				
Г. Кеннеди. Договориться можно обо всем	3				
Дж. Кэмп. Сначала скажите НЕТ	3				
Рызов. Переговоры с монстрами	4				
М. Ильяхов. Пиши, сокращай	3				
М. Ильяхов. Новые правила деловой переписки	2				
Р. Гандапас. Камасутра для оратора	6				
Курс управления голосом «Голос на миллион»	6				
Интерфейсы					
А. Лебедев. Ководство	1				
Аналитика / маркетинг / QA					

Навык, знание, литература	Уровень	Собственная оценка	Оценка куратора	Примечания куратора	Качаю в следующий раз
С. Круг. Не заставляйте меня думать	2				
А. Купер. Психбольница в руках пациентов	2				
Д. Коткин. Жесткие переговоры	6				
Р. Пихлер. Управление продуктом в Scrum	5				
К. Вигерс. Разработка требований к программному обеспечению	5				
Д. Норман. Дизайн привычных вещей	5				

УДК 005:004
ББК 65.290-2
3-13

Завертайлов, Владимир.

3-13 Настольная книга project-менеджера: что нужно знать, чтобы управлять IT, digital и другими проектами с учетом российских реалий / Владимир Завертайлов. — Москва : Эксмо, 2026. — 752 с.
ISBN 978-5-04-242785-5

Книга, которая объясняет, как стать востребованным project-менеджером и мастерски управлять it, digital и другими проектами с учетом специфики российского рынка. Автор — основатель и руководитель scrum-студии «Сибирикс», которая входит в топ-10 лучших веб-студий страны, Владимир Завертайлов. Из книги вы узнаете, чем kanban отличается от scrum и при чем тут agile; как быстро обучать новые кадры; как развить soft skills; научиться понимать ай-тишников и дизайнеров; а также успеть в уходящий вагон it-мира.

УДК 005:004
ББК 65.290-2

ISBN 978-5-04-242785-5

© Завертайлов В., текст, 2022
© Оформление. ООО «Издательство «Эксмо», 2026

Все права защищены. Книга или любая ее часть не может быть скопирована, воспроизведена в электронной или механической форме, в виде фотокопии, записи в память ЭВМ, репродукции или каким-либо иным способом, а также использована в любой информационной системе без получения разрешения от издателя. Копирование, воспроизведение и иное использование книги или ее части без согласия издателя является незаконным и влечет уголовную, административную и гражданскую ответственность.

Издание для досуга

Завертайлов Владимир

НАСТОЛЬНАЯ КНИГА ПРОЕКТ-МЕНЕДЖЕРА
ЧТО НУЖНО ЗНАТЬ, ЧТОБЫ УПРАВЛЯТЬ IT, DIGITAL
И ДРУГИМИ ПРОЕКТАМИ С УЧЕТОМ РОССИЙСКИХ РЕАЛИЙ

Главный редактор *Р. Фасхутдинов*
Руководитель направления *Л. Романова*
Ответственный редактор *М. Бердник*
Литературные редакторы *Р. Муртазина, Е. Королёва*
Младший редактор *К. Молокеева*
Художественный редактор *П. Петров*
Компьютерная верстка *О. Шуваловой*

Страна происхождения: Российская Федерация
Шығарушы ел: Ресей Федерациясы

ООО «Издательство «Эксмо»
123308, Россия, г. Москва, ул. Зорге, д. 1, стр. 1, эт. 20, каб. 2013. Тел.: 8 (495) 411-68-86.
Home page: www.eksmo.ru E-mail: info@eksmo.ru
Өндіруші: «Издательство «Эксмо» ЖШҚ
123308, Ресей, Мәскеу қаласы, Зорге көшесі, 1-үй, 1-құрылыс, 20 қабат, 2013-каб.
Тел.: 8 (495) 411-68-86. Home page: www.eksmo.ru E-mail: info@eksmo.ru.
Тауар белгісі: «Эксмо»

Интернет-магазин : www.book24.ru
Интернет-магазин : www.book24.kz
Интернет-дүкен : www.book24.kz

Импортёр в Республику Казахстан ТОО «РДЦ-Алматы».
Қазақстан Республикасына импорттаушы «РДЦ-Алматы» ЖШС.
Дистрибьютор и представитель по приему претензий на продукцию
в Республике Казахстан: ТОО «РДЦ-Алматы»
ТОО РДЦ Алматы, Алматы, ул. Домбровского, 3-«а», литер Б, офис 1.
Дистрибьютор және Қазақстан Республикасында өнімге шағымдар
қабылдау жөніндегі өкіл: «РДЦ-Алматы» ЖШС.
Алматы қ., Домбровский көш., 3-«а», литер Б, офис 1.
Тел.: 8 (727) 251-59-90/91/92. E-mail: RDC-Almaty@eksmo.kz

Сведения о подтверждении соответствия издания согласно законодательству РФ
о техническом регулировании можно получить на сайте Издательства «Эксмо»:
www.eksmo.ru/certification

Техникалық реттеу туралы РФ заңнамасына сай басылымның сәйкестігін растау
туралы мәліметтерді мына адрес бойынша алуға болады: <http://eksmo.ru/certification/>

Произведено в Российской Федерации
Ресей Федерациясында өндірілген

Сертификаттауға жатпайды

Дата изготовления / Подписано в печать 04.03.2026. Формат 70х100^{1/16}.
Гарнитура «Proxima Nova». Печать офсетная. Усл. печ. л. 60,93.
Тираж экз. Заказ

16+

БОМБОРА
ИЗДАТЕЛЬСТВО

БОМБОРА – лидер на рынке полезных и вдохновляющих книг.
Мы любим книги и создаем их, чтобы вы могли творить, открывать
мир, пробовать новое, расти. Быть счастливыми. Быть на волне.

bombara.ru bombarabooks bombara

ISBN 978-5-04-242785-5



9 785042 427855 >



ЧИТАЙТЕ
И СЛУШАЙТЕ
в Литрес

**ЧИТАЙ
ГОРОД**



Хочешь стать
автором «Эксмо»?



eksmo.ru

Официальный
интернет-магазин
издательства «Эксмо»



**ТЕРИТОРИЯ
КНИЖНЫЙ МАГАЗИН**

Официальная франшиза
издательства «Эксмо»

Проект

Максимально полный путеводитель по профессии российского project-менеджера. В нем Владимир Завертайлов, основатель и руководитель scrum-студии «Сибирикс», которая входит в топ-10 лучших веб-студий страны, рассказывает, как управлять собственным digital-производством и грамотно руководить проектом, заказывая digital-услуги на стороне. А еще как расти в этой профессии, опираясь на знания о продуктовых техниках и понимая потребности бизнеса.

ИЗ КНИГИ ВЫ УЗНАЕТЕ:

- чем kanban отличается от scrum и при чем тут agile;
- как управлять неуправляемыми — дизайнерами, копирайтерами, программистами и другими «инопланетянами»;
- что лучше: собственная команда или аутсорс — и как быть с техподдержкой;
- откуда берутся оценки времени и как понять, что исполнитель их завышает;
- что такое декомпозиция, зачем она нужна на проекте;
- как планировать экономику проекта, рассчитывать операционные затраты и анализировать P&L продукта.

«Управление созданием цифрового продукта — один из самых востребованных навыков в последние годы и в будущем останется таковым. У Владимира Завертайлова огромный и успешный опыт управления проектами и командами, а также конструктивной рефлексии и передачи знаний о менеджменте. Написанное многократно проверено на практике, а категоричные взгляды разных школ управления сведены к прозрачному и эффективному набору практик, которые можно свободно комбинировать».

Алексей Раменский, главный редактор и управляющий партнер «Тэглайна»

«Наиболее полная книга по управлению проектами в digital. Автор — практик, который делится массой нюансов и приемов, необходимых для работы. Книга будет полезна не только руководителям проектов на стороне агентств, но и для проектов на стороне клиентов, которые либо управляют внутренними отделами разработки, либо вынуждены по долгу службы заказывать digital-услуги на стороне».

Анатолий Денисов, главред «Рейтинга Рунета»

ISBN 978-5-04-242785-5



9 785042 427855 >

БОМБОРА
ИЗДАТЕЛЬСТВО

БОМБОРА — лидер на рынке полезных и вдохновляющих книг. Мы любим книги и создаем их, чтобы вы могли творить, открывать мир, пробовать новое, расти. Быть счастливыми. Быть на волне.